

Uno

Generated by Doxygen 1.13.2

1 Topic Index	1
1.1 Topics	1
2 Data Structure Index	3
2.1 Data Structures	3
3 File Index	7
3.1 File List	7
4 Topic Documentation	11
4.1 Entry Point	11
4.2 Autenticazione e Database Utenti	11
4.2.1 Detailed Description	12
4.3 Validazione Autenticazione	12
4.4 Tipi di Dato e Strutture	12
4.5 BST (Albero Binario Ricerca)	12
4.5.1 Detailed Description	13
4.5.2 Function Documentation	13
4.5.2.1 BST_Altezza()	13
4.5.2.2 BST_ContaNodi()	13
4.5.2.3 BST_CreaNodo()	14
4.5.2.4 BST_Distruggi()	14
4.5.2.5 BST_Inordina()	15
4.5.2.6 BST_Inserisci()	15
4.5.2.7 BST_RicercaEmail()	16
4.5.2.8 BST_RimuoviEmail()	16
4.6 Chat	16
4.6.1 Detailed Description	17
4.6.2 Function Documentation	17
4.6.2.1 Chat_Aggiungi()	17
4.6.2.2 Chat_Crea()	18
4.6.2.3 Chat_FormattaRiga()	18
4.6.2.4 Chat_OraCorrente()	18
4.6.2.5 Chat_Ottieni()	19
4.6.2.6 Chat_Svuota()	19
4.7 Queue (Coda)	20
4.7.1 Detailed Description	20
4.7.2 Function Documentation	21
4.7.2.1 Coda_CercaPerIndice()	21
4.7.2.2 Coda_CopiaPerIndice()	21
4.7.2.3 Coda_Crea()	22
4.7.2.4 Coda_Dequeue()	22
4.7.2.5 Coda_Dimensione()	22

4.7.2.6 Coda_Distruggi()	23
4.7.2.7 Coda_Enqueue()	23
4.7.2.8 Coda_Front()	24
4.7.2.9 Coda_InvertiVerso()	24
4.7.2.10 Coda_ProxNodo()	24
4.7.2.11 Coda_Rear()	25
4.7.2.12 Coda_Vuota()	25
4.8 Stack (Pila)	25
4.8.1 Detailed Description	26
4.8.2 Function Documentation	26
4.8.2.1 Pila_Crea()	26
4.8.2.2 Pila_Dimensione()	26
4.8.2.3 Pila_Distruggi()	27
4.8.2.4 Pila_Pop()	27
4.8.2.5 Pila_Push()	28
4.8.2.6 Pila_Svuota()	28
4.8.2.7 Pila_Top()	28
4.8.2.8 Pila_Vuota()	29
4.9 Logica di Gioco	29
4.9.1 Detailed Description	30
4.9.2 Function Documentation	30
4.9.2.1 GetCartaGiocatore()	30
4.9.2.2 GetLunghezzaMano()	30
4.9.2.3 GetNodoCartaGiocatore()	31
4.9.2.4 InizializzaMazzo()	31
4.9.2.5 Pesca()	31
4.9.2.6 RimescolaScartiNelMazzo()	32
4.9.2.7 RimuoviCartaGiocatore()	32
4.10 Lista (Lista Concatenata)	33
4.10.1 Detailed Description	33
4.10.2 Function Documentation	34
4.10.2.1 CreaLista()	34
4.10.2.2 EliminaLista()	34
4.10.2.3 GetAt()	34
4.10.2.4 InsertInCoda()	35
4.10.2.5 InsertInTesta()	35
4.10.2.6 Lista_InserisciDopo()	36
4.10.2.7 Lista_Precedente()	36
4.10.2.8 Lista_RimuoviNodoBidirezionale()	36
4.10.2.9 Lista_Ultimo()	37
4.10.2.10 Lista_VerificaBidirezionale()	37
4.10.2.11 RemoveAt()	37

4.10.2.12 RemoveNode()	38
4.10.2.13 StampaLista()	38
4.11 Admin Draw	39
4.11.1 Detailed Description	39
4.11.2 Function Documentation	39
4.11.2.1 DisegnaAdminModificaUtente()	39
4.11.2.2 DisegnaAdminPanel()	39
4.12 Auth Draw	40
4.12.1 Detailed Description	40
4.12.2 Function Documentation	40
4.12.2.1 DisegnaAuth()	40
4.13 Schermate Auth	41
4.13.1 Detailed Description	41
4.14 Auth Shared	41
4.15 Auth Update	41
4.15.1 Detailed Description	41
4.16 Input Field	42
4.16.1 Detailed Description	42
4.16.2 Function Documentation	42
4.16.2.1 AggiornaInputCampo()	42
4.16.2.2 DisegnaInputField()	43
4.17 Bot AI	43
4.17.1 Detailed Description	44
4.17.2 Function Documentation	44
4.17.2.1 BotAI_CostruisciAlbero()	44
4.17.2.2 BotAI_Decidi()	44
4.17.2.3 BotAI_DistruggiAlbero()	45
4.17.2.4 BotAI_ScegliColore()	45
4.17.2.5 BotAI_TrovaCarta()	46
4.18 End Screen	46
4.18.1 Detailed Description	46
4.19 Gameplay Animation	46
4.20 Gameplay Chat	47
4.20.1 Detailed Description	47
4.21 Gameplay Draw	47
4.21.1 Detailed Description	47
4.22 Gameplay Screen	47
4.22.1 Detailed Description	47
4.23 Gameplay Shared	48
4.24 Gameplay Update	48
4.24.1 Detailed Description	48
4.25 Menu Screen	48

4.25.1 Detailed Description	48
4.25.2 Function Documentation	49
4.25.2.1 DisegnaStatistiche()	49
4.26 Stats Screen	49
4.27 Multiplayer Lobby	49
4.27.1 Detailed Description	49
4.27.2 Function Documentation	50
4.27.2.1 DisegnaWaitingRoom()	50
4.28 Multiplayer Screen	50
4.28.1 Detailed Description	50
4.28.2 Function Documentation	51
4.28.2.1 DisegnaMultiplayer()	51
4.28.2.2 ResetMultiplayerError()	51
4.29 String Utils	51
4.29.1 Detailed Description	52
4.30 Protocollo LAN	52
4.31 Dati Utente LAN	52
4.32 Stats Sync Packet	52
4.33 Sincronizzazione LAN	52
4.34 Rete Multiplayer	53
4.34.1 Detailed Description	53
4.35 Network Send	53
4.36 Packet Handler	53
4.37 Packet Handler Admin	53
4.38 Packet Handler Game	54
4.39 State Sync	54
4.40 Server Manager	54
4.41 Network Utils	54
4.42 Rendering e Interfaccia Utente	54
4.42.1 Detailed Description	54
4.43 Implementazioni Auth	55
4.43.1 Detailed Description	55
4.43.2 Variable Documentation	55
4.43.2.1 campo_focusato	55
4.43.2.2 cognomeCount	55
4.43.2.3 dbUtenti	56
4.43.2.4 emailCount	56
4.43.2.5 errore_password	56
4.43.2.6 errore_registrazione_email	56
4.43.2.7 errore_registrazione_pass	56
4.43.2.8 errore_registrazione_username	56
4.43.2.9 id_temp_login	56

4.43.2.10 id_utente_corrente	56
4.43.2.11 inputCognome	56
4.43.2.12 inputEmail	57
4.43.2.13 inputNome	57
4.43.2.14 inputPassword	57
4.43.2.15 inputPin	57
4.43.2.16 inputUsername	57
4.43.2.17 nomeCount	57
4.43.2.18 passCount	57
4.43.2.19 pinCount	57
4.43.2.20 usernameCount	57
4.43.3 Lazy Deletion Implementazione	57
4.43.3.1 Detailed Description	58
4.43.3.2 Function Documentation	58
4.44 Texture e Risorse Grafiche	63
4.44.1 Detailed Description	63
4.44.2 Variable Documentation	63
4.44.2.1 coloriRaylib	63
4.44.2.2 nomiColori	64
4.44.2.3 textureCarte	64
4.44.2.4 textureDorso	64
4.44.2.5 textureSfondo	64
4.45 Eliminazione Diffusa (Lazy Deletion)	64
4.45.1 Detailed Description	65
4.45.2 Macro Definition Documentation	65
4.45.2.1 MAX_PENDING_DELETIONS	65
4.45.3 Function Documentation	65
4.45.3.1 Auth_IsMarkedForDeletion()	65
4.45.3.2 Auth_MarkForDeletion()	65
4.45.3.3 Auth_ProcessPendingDeletions()	65
4.46 Database API	66
4.46.1 Detailed Description	66
4.46.2 Function Documentation	66
4.46.2.1 Auth_RegistraUtente()	66
4.46.2.2 Auth_RimuoviUtente()	67
4.46.2.3 AuthBST_Altezza()	68
4.46.2.4 AuthBST_CercaIndicePerEmail()	68
4.46.2.5 AuthBST_ContaNodi()	68
4.46.2.6 AuthBST_Ricostruisci()	69
4.46.2.7 CaricaDB()	69
4.46.2.8 DB_Alloca()	69
4.46.2.9 SalvaDB()	70

4.47 Nodi BST	70
4.47.1 Detailed Description	70
4.47.2 Typedef Documentation	70
4.47.2.1 NodoAlbero	70
4.48 Operazioni BST	71
4.48.1 Detailed Description	71
4.48.2 Function Documentation	71
4.48.2.1 BST_Altezza()	71
4.48.2.2 BST_ContaNodi()	72
4.48.2.3 BST_CreaNodo()	72
4.48.2.4 BST_Distruggi()	73
4.48.2.5 BST_Inordina()	73
4.48.2.6 BST_Inserisci()	74
4.48.2.7 BST_RicercaEmail()	75
4.48.2.8 BST_RimuoviEmail()	76
4.49 Storico Chat	76
4.49.1 Detailed Description	77
4.50 API Chat	77
4.50.1 Detailed Description	77
4.50.2 Function Documentation	77
4.50.2.1 Chat_Aggiungi()	77
4.50.2.2 Chat_Crea()	78
4.50.2.3 Chat_FormattaRiga()	79
4.50.2.4 Chat_OraCorrente()	79
4.50.2.5 Chat_Ottieni()	80
4.50.2.6 Chat_Svuota()	81
4.51 Nodi Coda	81
4.51.1 Detailed Description	82
4.51.2 Typedef Documentation	82
4.51.2.1 NodoCoda	82
4.52 Struttura Coda	82
4.52.1 Detailed Description	82
4.52.2 Typedef Documentation	82
4.52.2.1 CodaTurni	82
4.53 API Queue	83
4.53.1 Detailed Description	83
4.53.2 Function Documentation	83
4.53.2.1 Coda_CercaPerIndice()	83
4.53.2.2 Coda_CopiaPerIndice()	84
4.53.2.3 Coda_Crea()	85
4.53.2.4 Coda_Dequeue()	85
4.53.2.5 Coda_Dimensione()	86

4.53.2.6 Coda_Distruggi()	87
4.53.2.7 Coda_Enqueue()	87
4.53.2.8 Coda_Front()	88
4.53.2.9 Coda_InvertiVerso()	89
4.53.2.10 Coda_ProxNodo()	89
4.53.2.11 Coda_Rear()	90
4.53.2.12 Coda_Vuota()	91
4.54 Nodi Pila	91
4.54.1 Detailed Description	92
4.54.2 Typedef Documentation	92
4.54.2.1 NodoPila	92
4.55 Struttura Pila	92
4.55.1 Detailed Description	92
4.55.2 Typedef Documentation	92
4.55.2.1 Pila	92
4.56 API Stack	92
4.56.1 Detailed Description	93
4.56.2 Function Documentation	93
4.56.2.1 Pila_Crea()	93
4.56.2.2 Pila_Dimensione()	94
4.56.2.3 Pila_Distruggi()	94
4.56.2.4 Pila_Pop()	95
4.56.2.5 Pila_Push()	96
4.56.2.6 Pila_Svuota()	96
4.56.2.7 Pila_Top()	97
4.56.2.8 Pila_Vuota()	98
4.57 Azioni Bot	98
4.57.1 Detailed Description	99
4.57.2 Enumeration Type Documentation	99
4.57.2.1 DecisioneBot	99
4.58 Nodo Albero Decisionale	99
4.58.1 Detailed Description	99
4.58.2 Typedef Documentation	99
4.58.2.1 NodoDecisione	99
4.59 API Bot AI	100
4.59.1 Detailed Description	100
4.59.2 Function Documentation	100
4.59.2.1 BotAI_CostruisciAlbero()	100
4.59.2.2 BotAI_Decidi()	101
4.59.2.3 BotAI_DistruggiAlbero()	101
4.59.2.4 BotAI_ScegliColore()	102
4.59.2.5 BotAI_TrovaCarta()	103

4.60 Ruoli di Rete	103
4.60.1 Detailed Description	104
4.60.2 Enumeration Type Documentation	104
4.60.2.1 NetworkRole	104
4.61 Tipi di Pacchetto	104
4.61.1 Detailed Description	104
4.61.2 Enumeration Type Documentation	104
4.61.2.1 PacketType	104
4.62 Strutture Pacchetti	105
4.62.1 Detailed Description	109
4.62.2 Macro Definition Documentation	109
4.62.2.1 MAX_SERVER_CLIENTS	109
4.62.3 Function Documentation	110
4.62.3.1 BroadcastHostDisconnect()	110
4.62.3.2 BroadcastHostReturnLobby()	110
4.62.3.3 BroadcastStatsSync()	110
4.62.3.4 CloseNetwork()	110
4.62.3.5 CodeToIp()	110
4.62.3.6 ConnettiTramiteCodiceStanza()	111
4.62.3.7 GestisciDisconnessione()	111
4.62.3.8 GetLocalIpAddress()	112
4.62.3.9 HandleReceivedPacket()	112
4.62.3.10 HostGame()	112
4.62.3.11 InitNetwork()	112
4.62.3.12 IpToCode()	112
4.62.3.13 IsPlayerConnected()	113
4.62.3.14 JoinGame()	113
4.62.3.15 RiassegnaSocket()	113
4.62.3.16 SendPlayAgain()	113
4.62.3.17 SendPlayerReady()	113
4.62.3.18 SendStatsSync()	113
4.62.3.19 ShiftNetworkSockets()	114
4.62.3.20 UpdateNetwork()	114
4.62.4 Variable Documentation	114
4.62.4.1 action	114
4.62.4.2 [] [1/2]	114
4.62.4.3 admin_action [2/2]	114
4.62.4.4 [] [1/2]	114
4.62.4.5 admin_db_packet [2/2]	114
4.62.4.6 card_index	114
4.62.4.7 carte	115
4.62.4.8 [] [1/2]	115

4.62.4.9 chat [2/2]	115
4.62.4.10 clientSockets	115
4.62.4.11 cognome	115
4.62.4.12 [] [1/2]	115
4.62.4.13 color_choice [2/2]	115
4.62.4.14 colore	115
4.62.4.15 colore_scelto	115
4.62.4.16 connectedClientUsernames	115
4.62.4.17 count	116
4.62.4.18 [] [1/2]	116
4.62.4.19 create_session [2/2]	116
4.62.4.20 current_session_id	116
4.62.4.21 currentRole	116
4.62.4.22 [union]	116
4.62.4.23 [] [1/2]	116
4.62.4.24 draw [2/2]	116
4.62.4.25 durata_ultima_partita	116
4.62.4.26 email	116
4.62.4.27 esito_ultima	117
4.62.4.28 [] [1/2]	117
4.62.4.29 game_finished [2/2]	117
4.62.4.30 game_started	117
4.62.4.31 [] [1/2]	117
4.62.4.32 game_state [2/2]	117
4.62.4.33 [] [1/2]	117
4.62.4.34 heartbeat [2/2]	117
4.62.4.35 host_in_prelobby	117
4.62.4.36 host_name	117
4.62.4.37 host_ready	118
4.62.4.38 [] [1/2]	118
4.62.4.39 host_return_lobby [2/2]	118
4.62.4.40 is_ready	118
4.62.4.41 isConnected	118
4.62.4.42 [] [1/2]	118
4.62.4.43 join_game [2/2]	118
4.62.4.44 join_rejected [1/3]	118
4.62.4.45 [] [2/3]	118
4.62.4.46 join_rejected [3/3]	118
4.62.4.47 [] [1/2]	119
4.62.4.48 join_session [2/2]	119
4.62.4.49 local_player_id	119
4.62.4.50 message	119

4.62.4.51 modalita_ultima	119
4.62.4.52 motivo	119
4.62.4.53 [] [1/2]	119
4.62.4.54 move [2/2]	119
4.62.4.55 network_client_joined	119
4.62.4.56 network_game_initialized	119
4.62.4.57 network_pending	120
4.62.4.58 network_start_game_flag	120
4.62.4.59 nome [1/2]	120
4.62.4.60 nome [2/2]	120
4.62.4.61 num_carte	120
4.62.4.62 num_players [1/2]	120
4.62.4.63 num_players [2/2]	120
4.62.4.64 partite_giocate [1/2]	120
4.62.4.65 partite_giocate [2/2]	120
4.62.4.66 password	120
4.62.4.67 pin_recupero	121
4.62.4.68 [] [1/3]	121
4.62.4.69 play_again [2/3]	121
4.62.4.70 play_again [3/3]	121
4.62.4.71 [] [1/2]	121
4.62.4.72 player_hand [2/2]	121
4.62.4.73 player_id [1/9]	121
4.62.4.74 player_id [2/9]	121
4.62.4.75 player_id [3/9]	121
4.62.4.76 player_id [4/9]	121
4.62.4.77 player_id [5/9]	122
4.62.4.78 player_id [6/9]	122
4.62.4.79 player_id [7/9]	122
4.62.4.80 player_id [8/9]	122
4.62.4.81 player_id [9/9]	122
4.62.4.82 player_name [1/2]	122
4.62.4.83 player_name [2/2]	122
4.62.4.84 player_names	122
4.62.4.85 [] [1/3]	122
4.62.4.86 player_ready [2/3]	122
4.62.4.87 player_ready [3/3]	123
4.62.4.88 posizione_ultima	123
4.62.4.89 [] [1/2]	123
4.62.4.90 raw [2/2]	123
4.62.4.91 reject_message	123
4.62.4.92 sender_id	123

4.62.4.93 session_count	123
4.62.4.94 session_id [1/4]	123
4.62.4.95 session_id [2/4]	123
4.62.4.96 session_id [3/4]	123
4.62.4.97 session_id [4/4]	124
4.62.4.98 [] [1/2]	124
4.62.4.99 session_list [2/2]	124
4.62.4.100 [] [1/2]	124
4.62.4.101 session_state [2/2]	124
4.62.4.102 sessions	124
4.62.4.103 stato_gioco	124
4.62.4.104 [] [1/2]	124
4.62.4.105 stats_sync [2/2]	124
4.62.4.106 storico_count	124
4.62.4.107 storico_partite	125
4.62.4.108 target_id	125
4.62.4.109 timestamp [1/2]	125
4.62.4.110 timestamp [2/2]	125
4.62.4.111 total_users	125
4.62.4.112 type	125
4.62.4.113 user [1/2]	125
4.62.4.114 user [2/2]	125
4.62.4.115 user_index	125
4.62.4.116 username [1/2]	125
4.62.4.117 username [2/2]	126
4.62.4.118 vittorie_bot [1/2]	126
4.62.4.119 vittorie_bot [2/2]	126
4.62.4.120 vittorie_giocatore [1/2]	126
4.62.4.121 vittorie_giocatore [2/2]	126
4.62.4.122 want_play_again	126
4.62.4.123 winner_name	126
4.62.4.124 winning_player_id	126
4.63 Admin Sync	126
4.63.1 Detailed Description	127
4.63.2 Function Documentation	127
4.63.2.1 BroadcastAdminDB()	127
4.63.2.2 BroadcastAdminUserAction()	127
4.63.2.3 SendAdminUserSync()	127
4.64 Stati Sessione	128
4.64.1 Detailed Description	128
4.64.2 Enumeration Type Documentation	128
4.64.2.1 SessionState	128

4.65 Giocatore Sessione	128
4.65.1 Detailed Description	129
4.66 Sessione di Gioco	129
4.66.1 Detailed Description	129
4.67 Struttura Server Manager	129
4.67.1 Detailed Description	129
4.68 API Server Manager	129
4.68.1 Detailed Description	130
4.68.2 Function Documentation	130
4.68.2.1 AddPlayerToSession()	130
4.68.2.2 CloseSession()	131
4.68.2.3 CreateSession()	131
4.68.2.4 GenerateSessionId()	132
4.68.2.5 GetSession()	132
4.68.2.6 GetSessionByPlayerId()	133
4.68.2.7 InitServerManager()	133
4.68.2.8 IsValidMove()	133
4.68.2.9 ProcessColorChoice()	134
4.68.2.10 ProcessDraw()	134
4.68.2.11 ProcessMove()	135
4.68.2.12 RemovePlayerFromSession()	135
4.68.2.13 ShutdownServerManager()	136
4.68.2.14 StartSession()	136
4.68.2.15 UpdateServerManager()	136
4.68.2.16 UpdateSessionLogic()	137
5 Data Structure Documentation	139
5.1 AdminDBSyncPacket Struct Reference	139
5.2 AdminUserAction Struct Reference	139
5.3 AdminUserSyncData Struct Reference	140
5.4 AudioStream Struct Reference	140
5.4.1 Field Documentation	140
5.4.1.1 buffer	140
5.4.1.2 channels	140
5.4.1.3 processor	140
5.4.1.4 sampleRate	141
5.4.1.5 sampleSize	141
5.5 AutomationEvent Struct Reference	141
5.5.1 Field Documentation	141
5.5.1.1 frame	141
5.5.1.2 params	141
5.5.1.3 type	141

5.6 AutomationEventList Struct Reference	141
5.6.1 Field Documentation	142
5.6.1.1 capacity	142
5.6.1.2 count	142
5.6.1.3 events	142
5.7 BoneInfo Struct Reference	142
5.7.1 Field Documentation	142
5.7.1.1 name	142
5.7.1.2 parent	142
5.8 BoundingBox Struct Reference	143
5.8.1 Field Documentation	143
5.8.1.1 max	143
5.8.1.2 min	143
5.9 Camera2D Struct Reference	143
5.9.1 Field Documentation	143
5.9.1.1 offset	143
5.9.1.2 rotation	143
5.9.1.3 target	144
5.9.1.4 zoom	144
5.10 Camera3D Struct Reference	144
5.10.1 Field Documentation	144
5.10.1.1 fovy	144
5.10.1.2 position	144
5.10.1.3 projection	144
5.10.1.4 target	144
5.10.1.5 up	145
5.11 CampoInput Struct Reference	145
5.11.1 Detailed Description	145
5.11.2 Field Documentation	145
5.11.2.1 area	145
5.11.2.2 campo_id	145
5.12 Carta Struct Reference	145
5.12.1 Detailed Description	146
5.12.2 Field Documentation	146
5.12.2.1 area	146
5.12.2.2 colore	146
5.12.2.3 isTrascinata	146
5.12.2.4 posOriginale	146
5.12.2.5 tipo	146
5.13 ChatStorico Struct Reference	147
5.13.1 Detailed Description	147
5.13.2 Field Documentation	147

5.13.2.1 buffer	147
5.13.2.2 count	147
5.13.2.3 head	147
5.14 ClassificaRecord Struct Reference	148
5.14.1 Detailed Description	148
5.14.2 Field Documentation	148
5.14.2.1 is_bot	148
5.14.2.2 nome	148
5.14.2.3 num_carte	148
5.14.2.4 posizione	148
5.15 CodaTurni Struct Reference	149
5.15.1 Detailed Description	149
5.15.2 Field Documentation	149
5.15.2.1 front	149
5.15.2.2 rear	149
5.15.2.3 size	149
5.15.2.4 verso	149
5.16 Color Struct Reference	150
5.16.1 Field Documentation	150
5.16.1.1 a	150
5.16.1.2 b	150
5.16.1.3 g	150
5.16.1.4 r	150
5.17 DatabaseUtenti Struct Reference	150
5.17.1 Detailed Description	151
5.17.2 Field Documentation	151
5.17.2.1 capacita	151
5.17.2.2 lista	151
5.17.2.3 num_utenti	151
5.18 FilePathList Struct Reference	151
5.18.1 Field Documentation	151
5.18.1.1 count	151
5.18.1.2 paths	152
5.19 float16 Struct Reference	152
5.19.1 Field Documentation	152
5.19.1.1 v	152
5.20 float3 Struct Reference	152
5.20.1 Field Documentation	152
5.20.1.1 v	152
5.21 Font Struct Reference	153
5.21.1 Field Documentation	153
5.21.1.1 baseSize	153

5.21.1.2 glyphCount	153
5.21.1.3 glyphPadding	153
5.21.1.4 glyphs	153
5.21.1.5 recs	153
5.21.1.6 texture	153
5.22 GameSession Struct Reference	154
5.22.1 Detailed Description	154
5.22.2 Field Documentation	154
5.22.2.1 game_state	154
5.22.2.2 host_id	154
5.22.2.3 idleTime	154
5.22.2.4 num_players	154
5.22.2.5 players	154
5.22.2.6 session_id	155
5.22.2.7 startCountdown	155
5.22.2.8 state	155
5.23 Giocatore Struct Reference	155
5.23.1 Detailed Description	155
5.23.2 Field Documentation	156
5.23.2.1 era_in_prelobby	156
5.23.2.2 info	156
5.23.2.3 is_bot	156
5.23.2.4 mano	156
5.23.2.5 mano_backup	156
5.23.2.6 nome	156
5.23.2.7 num_carte_backup	156
5.23.2.8 num_carte_mano	156
5.23.2.9 socket_id	156
5.23.2.10 stato_pronto	157
5.24 GlyphInfo Struct Reference	157
5.24.1 Field Documentation	157
5.24.1.1 advanceX	157
5.24.1.2 image	157
5.24.1.3 offsetX	157
5.24.1.4 offsetY	157
5.24.1.5 value	157
5.25 Image Struct Reference	158
5.25.1 Field Documentation	158
5.25.1.1 data	158
5.25.1.2 format	158
5.25.1.3 height	158
5.25.1.4 mipmaps	158

5.25.1.5 width	158
5.26 LanCompactUser Struct Reference	158
5.26.1 Field Documentation	159
5.26.1.1 cognome	159
5.26.1.2 email	159
5.26.1.3 nome	159
5.26.1.4 partite_giocate	159
5.26.1.5 password	159
5.26.1.6 pin_recupero	159
5.26.1.7 storico_count	160
5.26.1.8 storico_partite	160
5.26.1.9 username	160
5.26.1.10 vittorie_bot	160
5.26.1.11 vittorie_giocatore	160
5.27 LanDeleteSavePacket Struct Reference	160
5.27.1 Detailed Description	161
5.27.2 Field Documentation	161
5.27.2.1 data_salvataggio	161
5.27.2.2 magic	161
5.27.2.3 owner_username	161
5.28 LanPacketHeader Struct Reference	161
5.28.1 Detailed Description	161
5.28.2 Field Documentation	162
5.28.2.1 length	162
5.28.2.2 magic	162
5.29 LanStatsSyncPacket Struct Reference	162
5.29.1 Field Documentation	162
5.29.1.1 magic	162
5.29.1.2 partite_giocate	163
5.29.1.3 storico_count	163
5.29.1.4 storico_partite	163
5.29.1.5 username	163
5.29.1.6 vittorie_bot	163
5.29.1.7 vittorie_giocatore	163
5.30 ListaCarte Struct Reference	163
5.30.1 Detailed Description	164
5.30.2 Field Documentation	164
5.30.2.1 coda	164
5.30.2.2 lunghezza	164
5.30.2.3 testa	164
5.31 Material Struct Reference	164
5.31.1 Field Documentation	165

5.31.1.1 maps	165
5.31.1.2 params	165
5.31.1.3 shader	165
5.32 MaterialMap Struct Reference	165
5.32.1 Field Documentation	165
5.32.1.1 color	165
5.32.1.2 texture	165
5.32.1.3 value	165
5.33 Matrix Struct Reference	166
5.33.1 Field Documentation	166
5.33.1.1 m0	166
5.33.1.2 m1	166
5.33.1.3 m10	166
5.33.1.4 m11	166
5.33.1.5 m12	166
5.33.1.6 m13	167
5.33.1.7 m14	167
5.33.1.8 m15	167
5.33.1.9 m2	167
5.33.1.10 m3	167
5.33.1.11 m4	167
5.33.1.12 m5	167
5.33.1.13 m6	167
5.33.1.14 m7	167
5.33.1.15 m8	167
5.33.1.16 m9	168
5.34 Mesh Struct Reference	168
5.34.1 Field Documentation	168
5.34.1.1 animNormals	168
5.34.1.2 animVertices	168
5.34.1.3 boneCount	168
5.34.1.4 boneIndices	169
5.34.1.5 boneWeights	169
5.34.1.6 colors	169
5.34.1.7 indices	169
5.34.1.8 normals	169
5.34.1.9 tangents	169
5.34.1.10 texcoords	169
5.34.1.11 texcoords2	169
5.34.1.12 triangleCount	169
5.34.1.13 vaold	169
5.34.1.14 vbold	170

5.34.1.15 vertexCount	170
5.34.1.16 vertices	170
5.35 MessaggioChat Struct Reference	170
5.35.1 Detailed Description	170
5.35.2 Field Documentation	170
5.35.2.1 mittente	170
5.35.2.2 orario	170
5.35.2.3 testo	171
5.36 Model Struct Reference	171
5.36.1 Field Documentation	171
5.36.1.1 boneMatrices	171
5.36.1.2 currentPose	171
5.36.1.3 materialCount	171
5.36.1.4 materials	171
5.36.1.5 meshCount	172
5.36.1.6 meshes	172
5.36.1.7 meshMaterial	172
5.36.1.8 skeleton	172
5.36.1.9 transform	172
5.37 ModelAnimation Struct Reference	172
5.37.1 Field Documentation	172
5.37.1.1 boneCount	172
5.37.1.2 keyframeCount	173
5.37.1.3 keyframePoses	173
5.37.1.4 name	173
5.38 ModelSkeleton Struct Reference	173
5.38.1 Field Documentation	173
5.38.1.1 bindPose	173
5.38.1.2 boneCount	173
5.38.1.3 bones	173
5.39 Music Struct Reference	174
5.39.1 Field Documentation	174
5.39.1.1 ctxData	174
5.39.1.2 ctxType	174
5.39.1.3 frameCount	174
5.39.1.4 looping	174
5.39.1.5 stream	174
5.40 NetPacket Struct Reference	174
5.41 NodoAlbero Struct Reference	175
5.41.1 Detailed Description	175
5.41.2 Field Documentation	176
5.41.2.1 left	176

5.41.2.2 right	176
5.41.2.3 utente	176
5.42 NodoCarta Struct Reference	176
5.42.1 Detailed Description	176
5.42.2 Field Documentation	176
5.42.2.1 carta	176
5.42.2.2 prev	177
5.42.2.3 prossimo	177
5.43 NodoCoda Struct Reference	177
5.43.1 Detailed Description	177
5.43.2 Field Documentation	177
5.43.2.1 giocatore	177
5.43.2.2 next	177
5.43.2.3 prev	177
5.44 NodoDecisione Struct Reference	178
5.44.1 Detailed Description	178
5.44.2 Field Documentation	178
5.44.2.1 azione	178
5.44.2.2 no	178
5.44.2.3 priorit�	178
5.44.2.4 s�	178
5.45 NodoPila Struct Reference	178
5.45.1 Detailed Description	179
5.45.2 Field Documentation	179
5.45.2.1 carta	179
5.45.2.2 next	179
5.46 NPatchInfo Struct Reference	179
5.46.1 Field Documentation	179
5.46.1.1 bottom	179
5.46.1.2 layout	180
5.46.1.3 left	180
5.46.1.4 right	180
5.46.1.5 source	180
5.46.1.6 top	180
5.47 PacketChat Struct Reference	180
5.47.1 Detailed Description	180
5.48 PacketColorChoice Struct Reference	181
5.48.1 Detailed Description	181
5.49 PacketCreateSession Struct Reference	181
5.49.1 Detailed Description	181
5.50 PacketDraw Struct Reference	181
5.50.1 Detailed Description	182

5.51 PacketGameFinished Struct Reference	182
5.51.1 Detailed Description	182
5.52 PacketGameState Struct Reference	182
5.52.1 Detailed Description	183
5.53 PacketHeartbeat Struct Reference	183
5.53.1 Detailed Description	183
5.54 PacketHostReturnLobby Struct Reference	183
5.54.1 Detailed Description	183
5.55 PacketJoinGame Struct Reference	184
5.55.1 Detailed Description	184
5.56 PacketJoinRejected Struct Reference	184
5.56.1 Detailed Description	184
5.57 PacketJoinSession Struct Reference	184
5.57.1 Detailed Description	185
5.58 PacketMove Struct Reference	185
5.58.1 Detailed Description	185
5.59 PacketPlayAgain Struct Reference	185
5.59.1 Detailed Description	186
5.60 PacketPlayerHand Struct Reference	186
5.60.1 Detailed Description	186
5.61 PacketPlayerReady Struct Reference	186
5.61.1 Detailed Description	186
5.62 PacketSessionList Struct Reference	187
5.62.1 Detailed Description	187
5.63 PacketSessionState Struct Reference	187
5.63.1 Detailed Description	187
5.64 PartitaRegistrata Struct Reference	187
5.64.1 Detailed Description	188
5.64.2 Field Documentation	188
5.64.2.1 durata_secondi	188
5.64.2.2 esito	188
5.64.2.3 modalita	188
5.64.2.4 posizione	188
5.65 Pila Struct Reference	188
5.65.1 Detailed Description	189
5.65.2 Field Documentation	189
5.65.2.1 size	189
5.65.2.2 top	189
5.66 Ray Struct Reference	189
5.66.1 Field Documentation	189
5.66.1.1 direction	189
5.66.1.2 position	189

5.67 RayCollision Struct Reference	190
5.67.1 Field Documentation	190
5.67.1.1 distance	190
5.67.1.2 hit	190
5.67.1.3 normal	190
5.67.1.4 point	190
5.68 Rectangle Struct Reference	190
5.68.1 Field Documentation	191
5.68.1.1 height	191
5.68.1.2 width	191
5.68.1.3 x	191
5.68.1.4 y	191
5.69 RenderTexture Struct Reference	191
5.69.1 Field Documentation	191
5.69.1.1 depth	191
5.69.1.2 id	191
5.69.1.3 texture	192
5.70 rIDrawCall Struct Reference	192
5.70.1 Field Documentation	192
5.70.1.1 mode	192
5.70.1.2 textureId	192
5.70.1.3 vertexAlignment	192
5.70.1.4 vertexCount	192
5.71 rIRenderBatch Struct Reference	193
5.71.1 Field Documentation	193
5.71.1.1 bufferCount	193
5.71.1.2 currentBuffer	193
5.71.1.3 currentDepth	193
5.71.1.4 drawCounter	193
5.71.1.5 draws	193
5.71.1.6 vertexBuffer	193
5.72 rIVertexBuffer Struct Reference	194
5.72.1 Field Documentation	194
5.72.1.1 colors	194
5.72.1.2 elementCount	194
5.72.1.3 indices	194
5.72.1.4 normals	194
5.72.1.5 texcoords	194
5.72.1.6 vaold	194
5.72.1.7 vbold	194
5.72.1.8 vertices	195
5.73 SaveReceiver Struct Reference	195

5.73.1 Detailed Description	195
5.73.2 Field Documentation	195
5.73.2.1 attivo	195
5.73.2.2 file_size	196
5.73.2.3 filedata	196
5.73.2.4 key	196
5.73.2.5 ricevuti	196
5.73.2.6 timer	196
5.73.2.7 total	196
5.74 ServerManager Struct Reference	196
5.74.1 Detailed Description	197
5.74.2 Field Documentation	197
5.74.2.1 frameTime	197
5.74.2.2 num_active_sessions	197
5.74.2.3 sessions	197
5.75 SessionPlayer Struct Reference	197
5.75.1 Detailed Description	198
5.75.2 Field Documentation	198
5.75.2.1 is_bot	198
5.75.2.2 is_connected	198
5.75.2.3 lastHeartbeat	198
5.75.2.4 mano	198
5.75.2.5 nome	198
5.75.2.6 player_id	198
5.76 Shader Struct Reference	198
5.76.1 Field Documentation	199
5.76.1.1 id	199
5.76.1.2 locs	199
5.77 Sound Struct Reference	199
5.77.1 Field Documentation	199
5.77.1.1 frameCount	199
5.77.1.2 stream	199
5.78 Statistiche Struct Reference	199
5.78.1 Detailed Description	200
5.78.2 Field Documentation	200
5.78.2.1 cognome	200
5.78.2.2 email	200
5.78.2.3 nome	200
5.78.2.4 partite_giocate	200
5.78.2.5 password	200
5.78.2.6 pin_recupero	200
5.78.2.7 storico_partite	201

5.78.2.8 username	201
5.78.2.9 vittorie_bot	201
5.78.2.10 vittorie_giocatore	201
5.79 StatoGioco Struct Reference	201
5.79.1 Detailed Description	202
5.79.2 Field Documentation	202
5.79.2.1 anim_attiva	202
5.79.2.2 anim_carta	202
5.79.2.3 anim_end	202
5.79.2.4 anim_gia_applicata	202
5.79.2.5 anim_pesca_count	202
5.79.2.6 anim_start	203
5.79.2.7 anim_t	203
5.79.2.8 autore_animazione	203
5.79.2.9 carta_pendente	203
5.79.2.10 chat_aperta	203
5.79.2.11 chat_buffer	203
5.79.2.12 chat_history	203
5.79.2.13 colore_attivo	203
5.79.2.14 data_salvataggio	203
5.79.2.15 deve_chiamare_uno	203
5.79.2.16 direzione	204
5.79.2.17 durata_partita	204
5.79.2.18 giocatori	204
5.79.2.19 gioco_finito	204
5.79.2.20 id_carta_in_trascinamento	204
5.79.2.21 in_scelta_colore	204
5.79.2.22 mazzo	204
5.79.2.23 messaggio	204
5.79.2.24 modalita	204
5.79.2.25 nodo_carta_trascinata	204
5.79.2.26 notifica	205
5.79.2.27 num_carte_mazzo	205
5.79.2.28 num_carte_scarti	205
5.79.2.29 num_chat_msg	205
5.79.2.30 num_giocatori	205
5.79.2.31 scarti	205
5.79.2.32 slot_salvataggio	205
5.79.2.33 statistiche_gia_salvate	205
5.79.2.34 timer_notifica	205
5.79.2.35 timer_uno	205
5.79.2.36 tipo_animazione	206

5.79.2.37 turno_corrente	206
5.80 StatsSyncPacket Struct Reference	206
5.80.1 Detailed Description	206
5.81 TempSave Struct Reference	207
5.81.1 Detailed Description	207
5.81.2 Field Documentation	207
5.81.2.1 data	207
5.81.2.2 percorso	207
5.82 Texture Struct Reference	207
5.82.1 Field Documentation	207
5.82.1.1 format	207
5.82.1.2 height	208
5.82.1.3 id	208
5.82.1.4 mipmaps	208
5.82.1.5 width	208
5.83 Transform Struct Reference	208
5.83.1 Field Documentation	208
5.83.1.1 rotation	208
5.83.1.2 scale	208
5.83.1.3 translation	209
5.84 Utente Struct Reference	209
5.84.1 Detailed Description	209
5.84.2 Field Documentation	209
5.84.2.1 cognome	209
5.84.2.2 email	209
5.84.2.3 nome	209
5.84.2.4 password	210
5.84.2.5 username	210
5.85 Vector2 Struct Reference	210
5.85.1 Field Documentation	210
5.85.1.1 x	210
5.85.1.2 y	210
5.86 Vector3 Struct Reference	210
5.86.1 Field Documentation	211
5.86.1.1 x	211
5.86.1.2 y	211
5.86.1.3 z	211
5.87 Vector4 Struct Reference	211
5.87.1 Field Documentation	211
5.87.1.1 w	211
5.87.1.2 x	212
5.87.1.3 y	212

5.87.1.4 z	212
5.88 VrDeviceInfo Struct Reference	212
5.88.1 Field Documentation	212
5.88.1.1 chromaAbCorrection	212
5.88.1.2 eyeToScreenDistance	212
5.88.1.3 hResolution	213
5.88.1.4 hScreenSize	213
5.88.1.5 interpupillaryDistance	213
5.88.1.6 lensDistortionValues	213
5.88.1.7 lensSeparationDistance	213
5.88.1.8 vResolution	213
5.88.1.9 vScreenSize	213
5.89 VrStereoConfig Struct Reference	213
5.89.1 Field Documentation	214
5.89.1.1 leftLensCenter	214
5.89.1.2 leftScreenCenter	214
5.89.1.3 projection	214
5.89.1.4 rightLensCenter	214
5.89.1.5 rightScreenCenter	214
5.89.1.6 scale	214
5.89.1.7 scaleIn	214
5.89.1.8 viewOffset	214
5.90 Wave Struct Reference	214
5.90.1 Field Documentation	215
5.90.1.1 channels	215
5.90.1.2 data	215
5.90.1.3 frameCount	215
5.90.1.4 sampleRate	215
5.90.1.5 sampleSize	215
6 File Documentation	217
6.1 lib/auth.h File Reference	217
6.1.1 Detailed Description	218
6.1.2 Variable Documentation	218
6.1.2.1 cognomeCount	218
6.1.2.2 errore_registrazione_pass	218
6.1.2.3 errore_registrazione_username	218
6.1.2.4 id_temp_login	219
6.1.2.5 inputCognome	219
6.1.2.6 inputNome	219
6.1.2.7 inputPassword	219
6.1.2.8 inputPin	219

6.1.2.9 nomeCount	219
6.1.2.10 nuovo_utente	219
6.1.2.11 passCount	219
6.1.2.12 pinCount	219
6.2 auth.h	220
6.3 lib/auth_validation.h File Reference	220
6.3.1 Detailed Description	221
6.3.2 Function Documentation	221
6.3.2.1 Auth_EmailFormalmenteValida()	221
6.3.2.2 Auth_EmailGiaUsata()	222
6.3.2.3 Auth_PasswordValida()	222
6.3.2.4 Auth_PINValido()	223
6.3.2.5 Auth_UsernameGiaUsato()	224
6.3.2.6 Auth_UsernameNonVuoto()	224
6.3.2.7 Auth_VerificaCredenziali()	225
6.3.2.8 Auth_VerificaEmailPIN()	226
6.4 auth_validation.h	227
6.5 lib/data_structures.h File Reference	227
6.5.1 Detailed Description	228
6.5.2 Macro Definition Documentation	229
6.5.2.1 DB_CAPACITA_INIZIALE	229
6.5.2.2 GIOCATORE_NOME_BUF	229
6.5.2.3 MAX_PARTITE_STORICO	229
6.5.3 Typedef Documentation	229
6.5.3.1 NodoCarta	229
6.5.3.2 NodoMano	229
6.5.4 Enumeration Type Documentation	229
6.5.4.1 Colore	229
6.5.4.2 FaseApplicazione	230
6.5.4.3 TipoCarta	230
6.6 data_structures.h	231
6.7 lib/data_structures/bst.h File Reference	234
6.7.1 Detailed Description	235
6.8 bst.h	235
6.9 lib/data_structures/chat.h File Reference	235
6.9.1 Detailed Description	236
6.9.2 Macro Definition Documentation	236
6.9.2.1 CHAT_STORICO_MAX	236
6.10 chat.h	236
6.11 lib/data_structures/list.h File Reference	237
6.11.1 Detailed Description	237
6.12 list.h	237

6.13 lib/list.h File Reference	238
6.13.1 Detailed Description	238
6.14 list.h	238
6.15 lib/data_structures/queue.h File Reference	239
6.15.1 Detailed Description	240
6.16 queue.h	240
6.17 lib/data_structures/stack.h File Reference	240
6.17.1 Detailed Description	241
6.18 stack.h	241
6.19 lib/fs/fs.h File Reference	242
6.19.1 Detailed Description	242
6.20 fs.h	242
6.21 lib/game_logic.h File Reference	242
6.21.1 Detailed Description	243
6.21.2 Function Documentation	244
6.21.2.1 ApplicaEffetto()	244
6.21.2.2 CaricaPartitaDaSlot()	245
6.21.2.3 CompattaSalvataggiUtente()	245
6.21.2.4 EliminaPartita()	246
6.21.2.5 EliminaSalvataggio()	246
6.21.2.6 EliminaTuttiSalvataggiUtente()	247
6.21.2.7 GetSaveFilePath()	247
6.21.2.8 MossaValida()	247
6.21.2.9 MostraNotifica()	248
6.21.2.10 NuovaPartita()	249
6.21.2.11 RicaricaMazzo()	250
6.21.2.12 RicostruisciStatoADT()	250
6.21.2.13 SalvaPartita()	252
6.21.3 Variable Documentation	252
6.21.3.1 coda_turni	252
6.21.3.2 mazzo_pila	253
6.21.3.3 scarti_pila	253
6.22 game_logic.h	253
6.23 lib/raylib/include/raylib.h File Reference	254
6.23.1 Macro Definition Documentation	271
6.23.1.1 BEIGE	271
6.23.1.2 BLACK	271
6.23.1.3 BLANK	271
6.23.1.4 BLUE	271
6.23.1.5 BROWN	271
6.23.1.6 CLITERAL	271
6.23.1.7 DARKBLUE	271

6.23.1.8 DARKBROWN	271
6.23.1.9 DARKGRAY	272
6.23.1.10 DARKGREEN	272
6.23.1.11 DARKPURPLE	272
6.23.1.12 DEG2RAD	272
6.23.1.13 GetMouseRay	272
6.23.1.14 GOLD	272
6.23.1.15 GRAY	272
6.23.1.16 GREEN	272
6.23.1.17 LIGHTGRAY	272
6.23.1.18 LIME	272
6.23.1.19 MAGENTA	273
6.23.1.20 MAROON	273
6.23.1.21 MATERIAL_MAP_DIFFUSE	273
6.23.1.22 MATERIAL_MAP_SPECULAR	273
6.23.1.23 MOUSE_LEFT_BUTTON	273
6.23.1.24 MOUSE_MIDDLE_BUTTON	273
6.23.1.25 MOUSE_RIGHT_BUTTON	273
6.23.1.26 ORANGE	273
6.23.1.27 PI	273
6.23.1.28 PINK	273
6.23.1.29 PURPLE	274
6.23.1.30 RAD2DEG	274
6.23.1.31 RAYLIB_VERSION	274
6.23.1.32 RAYLIB_VERSION_MAJOR	274
6.23.1.33 RAYLIB_VERSION_MINOR	274
6.23.1.34 RAYLIB_VERSION_PATCH	274
6.23.1.35 RAYWHITE	274
6.23.1.36 RED	274
6.23.1.37 RL_BOOL_TYPE	274
6.23.1.38 RL_CALLOC	274
6.23.1.39 RL_COLOR_TYPE	275
6.23.1.40 RL_FREE	275
6.23.1.41 RL_MALLOC	275
6.23.1.42 RL_MATRIX_TYPE	275
6.23.1.43 RL_QUATERNION_TYPE	275
6.23.1.44 RL_REALLOC	275
6.23.1.45 RL_RECTANGLE_TYPE	275
6.23.1.46 RL_VECTOR2_TYPE	275
6.23.1.47 RL_VECTOR3_TYPE	276
6.23.1.48 RL_VECTOR4_TYPE	276
6.23.1.49 RLAPI	276

6.23.1.50 SHADER_LOC_MAP_DIFFUSE	276
6.23.1.51 SHADER_LOC_MAP_SPECULAR	276
6.23.1.52 SKYBLUE	276
6.23.1.53 VIOLET	276
6.23.1.54 WHITE	276
6.23.1.55 YELLOW	276
6.23.2 Typedef Documentation	276
6.23.2.1 AudioCallback	276
6.23.2.2 AudioStream	277
6.23.2.3 AutomationEvent	277
6.23.2.4 AutomationEventList	277
6.23.2.5 BoneInfo	277
6.23.2.6 bool	277
6.23.2.7 BoundingBox	277
6.23.2.8 Camera	277
6.23.2.9 Camera2D	277
6.23.2.10 Camera3D	277
6.23.2.11 Color	277
6.23.2.12 FilePathList	278
6.23.2.13 Font	278
6.23.2.14 GlyphInfo	278
6.23.2.15 Image	278
6.23.2.16 LoadFileDataCallback	278
6.23.2.17 LoadFileTextCallback	278
6.23.2.18 Material	278
6.23.2.19 MaterialMap	278
6.23.2.20 Matrix	278
6.23.2.21 Mesh	278
6.23.2.22 Model	279
6.23.2.23 ModelAnimation	279
6.23.2.24 ModelAnimPose	279
6.23.2.25 ModelSkeleton	279
6.23.2.26 Music	279
6.23.2.27 NPatchInfo	279
6.23.2.28 Quaternion	279
6.23.2.29 rAudioBuffer	279
6.23.2.30 rAudioProcessor	279
6.23.2.31 Ray	279
6.23.2.32 RayCollision	280
6.23.2.33 Rectangle	280
6.23.2.34 RenderTexture	280
6.23.2.35 RenderTexture2D	280

6.23.2.36 SaveFileDataCallback	280
6.23.2.37 SaveFileTextCallback	280
6.23.2.38 Shader	280
6.23.2.39 Sound	280
6.23.2.40 Texture	280
6.23.2.41 Texture2D	280
6.23.2.42 TextureCubemap	281
6.23.2.43 TraceLogCallback	281
6.23.2.44 Transform	281
6.23.2.45 Vector2	281
6.23.2.46 Vector3	281
6.23.2.47 Vector4	281
6.23.2.48 VrDeviceInfo	281
6.23.2.49 VrStereoConfig	281
6.23.2.50 Wave	281
6.23.3 Enumeration Type Documentation	281
6.23.3.1 BlendMode	281
6.23.3.2 bool	282
6.23.3.3 CameraMode	282
6.23.3.4 CameraProjection	282
6.23.3.5 ConfigFlags	282
6.23.3.6 CubemapLayout	283
6.23.3.7 FontType	283
6.23.3.8 GamepadAxis	283
6.23.3.9 GamepadButton	284
6.23.3.10 Gesture	284
6.23.3.11 KeyboardKey	284
6.23.3.12 MaterialMapIndex	287
6.23.3.13 MouseButton	287
6.23.3.14 MouseCursor	287
6.23.3.15 NPatchLayout	287
6.23.3.16 PixelFormat	288
6.23.3.17 ShaderAttributeDataType	288
6.23.3.18 ShaderLocationIndex	288
6.23.3.19 ShaderUniformDataType	289
6.23.3.20 TextureFilter	290
6.23.3.21 TextureWrap	290
6.23.3.22 TraceLogLevel	290
6.23.4 Function Documentation	290
6.23.4.1 AttachAudioMixedProcessor()	290
6.23.4.2 AttachAudioStreamProcessor()	291
6.23.4.3 BeginBlendMode()	291

6.23.4.4 BeginDrawing()	291
6.23.4.5 BeginMode2D()	291
6.23.4.6 BeginMode3D()	291
6.23.4.7 BeginScissorMode()	291
6.23.4.8 BeginShaderMode()	291
6.23.4.9 BeginTextureMode()	291
6.23.4.10 BeginVrStereoMode()	292
6.23.4.11 ChangeDirectory()	292
6.23.4.12 CheckCollisionBoxes()	292
6.23.4.13 CheckCollisionBoxSphere()	292
6.23.4.14 CheckCollisionCircleLine()	292
6.23.4.15 CheckCollisionCircleRec()	292
6.23.4.16 CheckCollisionCircles()	292
6.23.4.17 CheckCollisionLines()	293
6.23.4.18 CheckCollisionPointCircle()	293
6.23.4.19 CheckCollisionPointLine()	293
6.23.4.20 CheckCollisionPointPoly()	293
6.23.4.21 CheckCollisionPointRec()	293
6.23.4.22 CheckCollisionPointTriangle()	293
6.23.4.23 CheckCollisionRecs()	294
6.23.4.24 CheckCollisionSpheres()	294
6.23.4.25 ClearBackground()	294
6.23.4.26 ClearWindowState()	294
6.23.4.27 CloseAudioDevice()	294
6.23.4.28 CloseWindow()	294
6.23.4.29 CodepointToUTF8()	294
6.23.4.30 ColorAlpha()	294
6.23.4.31 ColorAlphaBlend()	295
6.23.4.32 ColorBrightness()	295
6.23.4.33 ColorContrast()	295
6.23.4.34 ColorFromHSV()	295
6.23.4.35 ColorFromNormalized()	295
6.23.4.36 ColorIsEqual()	295
6.23.4.37 ColorLerp()	295
6.23.4.38 ColorNormalize()	296
6.23.4.39 ColorTint()	296
6.23.4.40 ColorToHSV()	296
6.23.4.41 ColorToInt()	296
6.23.4.42 CompressData()	296
6.23.4.43 ComputeCRC32()	296
6.23.4.44 ComputeMD5()	296
6.23.4.45 ComputeSHA1()	296

6.23.4.46 ComputeSHA256()	297
6.23.4.47 DecodeDataBase64()	297
6.23.4.48 DecompressData()	297
6.23.4.49 DetachAudioMixedProcessor()	297
6.23.4.50 DetachAudioStreamProcessor()	297
6.23.4.51 DirectoryExists()	297
6.23.4.52 DisableCursor()	297
6.23.4.53 DisableEventWaiting()	297
6.23.4.54 DrawBillboard()	298
6.23.4.55 DrawBillboardPro()	298
6.23.4.56 DrawBillboardRec()	298
6.23.4.57 DrawBoundingBox()	298
6.23.4.58 DrawCapsule()	298
6.23.4.59 DrawCapsuleWires()	299
6.23.4.60 DrawCircle()	299
6.23.4.61 DrawCircle3D()	299
6.23.4.62 DrawCircleGradient()	299
6.23.4.63 DrawCircleLines()	299
6.23.4.64 DrawCircleLinesV()	299
6.23.4.65 DrawCircleSector()	300
6.23.4.66 DrawCircleSectorLines()	300
6.23.4.67 DrawCircleV()	300
6.23.4.68 DrawCube()	300
6.23.4.69 DrawCubeV()	300
6.23.4.70 DrawCubeWires()	300
6.23.4.71 DrawCubeWiresV()	301
6.23.4.72 DrawCylinder()	301
6.23.4.73 DrawCylinderEx()	301
6.23.4.74 DrawCylinderWires()	301
6.23.4.75 DrawCylinderWiresEx()	301
6.23.4.76 DrawEllipse()	302
6.23.4.77 DrawEllipseLines()	302
6.23.4.78 DrawEllipseLinesV()	302
6.23.4.79 DrawEllipseV()	302
6.23.4.80 DrawFPS()	302
6.23.4.81 DrawGrid()	302
6.23.4.82 DrawLine()	303
6.23.4.83 DrawLine3D()	303
6.23.4.84 DrawLineBezier()	303
6.23.4.85 DrawLineDashed()	303
6.23.4.86 DrawLineEx()	303
6.23.4.87 DrawLineStrip()	303

6.23.4.88 DrawLineV()	304
6.23.4.89 DrawMesh()	304
6.23.4.90 DrawMeshInstanced()	304
6.23.4.91 DrawModel()	304
6.23.4.92 DrawModelEx()	304
6.23.4.93 DrawModelWires()	304
6.23.4.94 DrawModelWiresEx()	305
6.23.4.95 DrawPixel()	305
6.23.4.96 DrawPixelV()	305
6.23.4.97 DrawPlane()	305
6.23.4.98 DrawPoint3D()	305
6.23.4.99 DrawPoly()	305
6.23.4.100 DrawPolyLines()	306
6.23.4.101 DrawPolyLinesEx()	306
6.23.4.102 DrawRay()	306
6.23.4.103 DrawRectangle()	306
6.23.4.104 DrawRectangleGradientEx()	306
6.23.4.105 DrawRectangleGradientH()	306
6.23.4.106 DrawRectangleGradientV()	307
6.23.4.107 DrawRectangleLines()	307
6.23.4.108 DrawRectangleLinesEx()	307
6.23.4.109 DrawRectanglePro()	307
6.23.4.110 DrawRectangleRec()	307
6.23.4.111 DrawRectangleRounded()	307
6.23.4.112 DrawRectangleRoundedLines()	308
6.23.4.113 DrawRectangleRoundedLinesEx()	308
6.23.4.114 DrawRectangleV()	308
6.23.4.115 DrawRing()	308
6.23.4.116 DrawRingLines()	308
6.23.4.117 DrawSphere()	308
6.23.4.118 DrawSphereEx()	309
6.23.4.119 DrawSphereWires()	309
6.23.4.120 DrawSplineBasis()	309
6.23.4.121 DrawSplineBezierCubic()	309
6.23.4.122 DrawSplineBezierQuadratic()	309
6.23.4.123 DrawSplineCatmullRom()	309
6.23.4.124 DrawSplineLinear()	310
6.23.4.125 DrawSplineSegmentBasis()	310
6.23.4.126 DrawSplineSegmentBezierCubic()	310
6.23.4.127 DrawSplineSegmentBezierQuadratic()	310
6.23.4.128 DrawSplineSegmentCatmullRom()	310
6.23.4.129 DrawSplineSegmentLinear()	311

6.23.4.130 DrawText()	311
6.23.4.131 DrawTextCodepoint()	311
6.23.4.132 DrawTextCodepoints()	311
6.23.4.133 DrawTextEx()	311
6.23.4.134 DrawTextPro()	312
6.23.4.135 DrawTexture()	312
6.23.4.136 DrawTextureEx()	312
6.23.4.137 DrawTextureNPatch()	312
6.23.4.138 DrawTexturePro()	312
6.23.4.139 DrawTextureRec()	313
6.23.4.140 DrawTextureV()	313
6.23.4.141 DrawTriangle()	313
6.23.4.142 DrawTriangle3D()	313
6.23.4.143 DrawTriangleFan()	313
6.23.4.144 DrawTriangleLines()	313
6.23.4.145 DrawTriangleStrip()	314
6.23.4.146 DrawTriangleStrip3D()	314
6.23.4.147 EnableCursor()	314
6.23.4.148 EnableEventWaiting()	314
6.23.4.149 EncodeDataBase64()	314
6.23.4.150 EndBlendMode()	314
6.23.4.151 EndDrawing()	314
6.23.4.152 EndMode2D()	314
6.23.4.153 EndMode3D()	315
6.23.4.154 EndScissorMode()	315
6.23.4.155 EndShaderMode()	315
6.23.4.156 EndTextureMode()	315
6.23.4.157 EndVrStereoMode()	315
6.23.4.158 ExportAutomationEventList()	315
6.23.4.159 ExportDataAsCode()	315
6.23.4.160 ExportFontAsCode()	315
6.23.4.161 ExportImage()	316
6.23.4.162 ExportImageAsCode()	316
6.23.4.163 ExportImageToMemory()	316
6.23.4.164 ExportMesh()	316
6.23.4.165 ExportMeshAsCode()	316
6.23.4.166 ExportWave()	316
6.23.4.167 ExportWaveAsCode()	316
6.23.4.168 Fade()	317
6.23.4.169 FileCopy()	317
6.23.4.170 FileExists()	317
6.23.4.171 FileMove()	317

6.23.4.172 FileRemove()	317
6.23.4.173 FileRename()	317
6.23.4.174 FileTextFindIndex()	317
6.23.4.175 FileTextReplace()	317
6.23.4.176 GenImageCellular()	318
6.23.4.177 GenImageChecked()	318
6.23.4.178 GenImageColor()	318
6.23.4.179 GenImageFontAtlas()	318
6.23.4.180 GenImageGradientLinear()	318
6.23.4.181 GenImageGradientRadial()	318
6.23.4.182 GenImageGradientSquare()	319
6.23.4.183 GenImagePerlinNoise()	319
6.23.4.184 GenImageText()	319
6.23.4.185 GenImageWhiteNoise()	319
6.23.4.186 GenMeshCone()	319
6.23.4.187 GenMeshCube()	319
6.23.4.188 GenMeshCubicmap()	320
6.23.4.189 GenMeshCylinder()	320
6.23.4.190 GenMeshHeightmap()	320
6.23.4.191 GenMeshHemiSphere()	320
6.23.4.192 GenMeshKnot()	320
6.23.4.193 GenMeshPlane()	320
6.23.4.194 GenMeshPoly()	320
6.23.4.195 GenMeshSphere()	321
6.23.4.196 GenMeshTangents()	321
6.23.4.197 GenMeshTorus()	321
6.23.4.198 GenTextureMipmaps()	321
6.23.4.199 GetApplicationDirectory()	321
6.23.4.200 GetCameraMatrix()	321
6.23.4.201 GetCameraMatrix2D()	321
6.23.4.202 GetCharPressed()	321
6.23.4.203 GetClipboardImage()	322
6.23.4.204 GetClipboardText()	322
6.23.4.205 GetCodepoint()	322
6.23.4.206 GetCodepointCount()	322
6.23.4.207 GetCodepointNext()	322
6.23.4.208 GetCodepointPrevious()	322
6.23.4.209 GetCollisionRec()	322
6.23.4.210 GetColor()	322
6.23.4.211 GetCurrentMonitor()	323
6.23.4.212 GetDirectoryFileCount()	323
6.23.4.213 GetDirectoryFileCountEx()	323

6.23.4.214 GetDirectoryPath()	323
6.23.4.215 GetFileExtension()	323
6.23.4.216 GetFileLength()	323
6.23.4.217 GetFileModTime()	323
6.23.4.218 GetFileName()	323
6.23.4.219 GetFileNameWithoutExt()	324
6.23.4.220 GetFontDefault()	324
6.23.4.221 GetFPS()	324
6.23.4.222 GetFrameTime()	324
6.23.4.223 GetGamepadAxisCount()	324
6.23.4.224 GetGamepadAxisMovement()	324
6.23.4.225 GetGamepadButtonPressed()	324
6.23.4.226 GetGamepadName()	324
6.23.4.227 GetGestureDetected()	324
6.23.4.228 GetGestureDragAngle()	325
6.23.4.229 GetGestureDragVector()	325
6.23.4.230 GetGestureHoldDuration()	325
6.23.4.231 GetGesturePinchAngle()	325
6.23.4.232 GetGesturePinchVector()	325
6.23.4.233 GetGlyphAtlasRec()	325
6.23.4.234 GetGlyphIndex()	325
6.23.4.235 GetGlyphInfo()	325
6.23.4.236 GetImageAlphaBorder()	326
6.23.4.237 GetImageColor()	326
6.23.4.238 GetKeyName()	326
6.23.4.239 GetKeyPressed()	326
6.23.4.240 GetMasterVolume()	326
6.23.4.241 GetMeshBoundingBox()	326
6.23.4.242 GetModelBoundingBox()	326
6.23.4.243 GetMonitorCount()	326
6.23.4.244 GetMonitorHeight()	327
6.23.4.245 GetMonitorName()	327
6.23.4.246 GetMonitorPhysicalHeight()	327
6.23.4.247 GetMonitorPhysicalWidth()	327
6.23.4.248 GetMonitorPosition()	327
6.23.4.249 GetMonitorRefreshRate()	327
6.23.4.250 GetMonitorWidth()	327
6.23.4.251 GetMouseDelta()	327
6.23.4.252 GetMousePosition()	327
6.23.4.253 GetMouseWheelMove()	328
6.23.4.254 GetMouseWheelMoveV()	328
6.23.4.255 GetMouseX()	328

6.23.4.256 GetMouseY()	328
6.23.4.257 GetMusicTimeLength()	328
6.23.4.258 GetMusicTimePlayed()	328
6.23.4.259 GetPixelColor()	328
6.23.4.260 GetPixelDataSize()	328
6.23.4.261 GetPrevDirectoryPath()	329
6.23.4.262 GetRandomValue()	329
6.23.4.263 GetRayCollisionBox()	329
6.23.4.264 GetRayCollisionMesh()	329
6.23.4.265 GetRayCollisionQuad()	329
6.23.4.266 GetRayCollisionSphere()	329
6.23.4.267 GetRayCollisionTriangle()	329
6.23.4.268 GetRenderHeight()	330
6.23.4.269 GetRenderWidth()	330
6.23.4.270 GetScreenHeight()	330
6.23.4.271 GetScreenToWorld2D()	330
6.23.4.272 GetScreenToWorldRay()	330
6.23.4.273 GetScreenToWorldRayEx()	330
6.23.4.274 GetScreenWidth()	330
6.23.4.275 GetShaderLocation()	330
6.23.4.276 GetShaderLocationAttrib()	331
6.23.4.277 GetShapesTexture()	331
6.23.4.278 GetShapesTextureRectangle()	331
6.23.4.279 GetSplinePointBasis()	331
6.23.4.280 GetSplinePointBezierCubic()	331
6.23.4.281 GetSplinePointBezierQuad()	331
6.23.4.282 GetSplinePointCatmullRom()	331
6.23.4.283 GetSplinePointLinear()	332
6.23.4.284 GetTextBetween()	332
6.23.4.285 GetTime()	332
6.23.4.286 GetTouchPointCount()	332
6.23.4.287 GetTouchPointId()	332
6.23.4.288 GetTouchPosition()	332
6.23.4.289 GetTouchX()	332
6.23.4.290 GetTouchY()	332
6.23.4.291 GetWindowHandle()	333
6.23.4.292 GetWindowPosition()	333
6.23.4.293 GetWindowScaledDPI()	333
6.23.4.294 GetWorkingDirectory()	333
6.23.4.295 GetWorldToScreen()	333
6.23.4.296 GetWorldToScreen2D()	333
6.23.4.297 GetWorldToScreenEx()	333

6.23.4.298 HideCursor()	333
6.23.4.299 ImageAlphaClear()	334
6.23.4.300 ImageAlphaCrop()	334
6.23.4.301 ImageAlphaMask()	334
6.23.4.302 ImageAlphaPremultiply()	334
6.23.4.303 ImageBlurGaussian()	334
6.23.4.304 ImageClearBackground()	334
6.23.4.305 ImageColorBrightness()	334
6.23.4.306 ImageColorContrast()	334
6.23.4.307 ImageColorGrayscale()	335
6.23.4.308 ImageColorInvert()	335
6.23.4.309 ImageColorReplace()	335
6.23.4.310 ImageColorTint()	335
6.23.4.311 ImageCopy()	335
6.23.4.312 ImageCrop()	335
6.23.4.313 ImageDither()	335
6.23.4.314 ImageDraw()	336
6.23.4.315 ImageDrawCircle()	336
6.23.4.316 ImageDrawCircleLines()	336
6.23.4.317 ImageDrawCircleLinesV()	336
6.23.4.318 ImageDrawCircleV()	336
6.23.4.319 ImageDrawLine()	336
6.23.4.320 ImageDrawLineEx()	337
6.23.4.321 ImageDrawLineV()	337
6.23.4.322 ImageDrawPixel()	337
6.23.4.323 ImageDrawPixelV()	337
6.23.4.324 ImageDrawRectangle()	337
6.23.4.325 ImageDrawRectangleLines()	337
6.23.4.326 ImageDrawRectangleRec()	338
6.23.4.327 ImageDrawRectangleV()	338
6.23.4.328 ImageDrawText()	338
6.23.4.329 ImageDrawTextEx()	338
6.23.4.330 ImageDrawTriangle()	338
6.23.4.331 ImageDrawTriangleEx()	339
6.23.4.332 ImageDrawTriangleFan()	339
6.23.4.333 ImageDrawTriangleLines()	339
6.23.4.334 ImageDrawTriangleStrip()	339
6.23.4.335 ImageFlipHorizontal()	339
6.23.4.336 ImageFlipVertical()	339
6.23.4.337 ImageFormat()	340
6.23.4.338 ImageFromChannel()	340
6.23.4.339 ImageFromImage()	340

6.23.4.340 ImageKernelConvolution()	340
6.23.4.341 ImageMipmaps()	340
6.23.4.342 ImageResize()	340
6.23.4.343 ImageResizeCanvas()	340
6.23.4.344 ImageResizeNN()	341
6.23.4.345 ImageRotate()	341
6.23.4.346 ImageRotateCCW()	341
6.23.4.347 ImageRotateCW()	341
6.23.4.348 ImageText()	341
6.23.4.349 ImageTextEx()	341
6.23.4.350 ImageToPOT()	341
6.23.4.351 InitAudioDevice()	342
6.23.4.352 InitWindow()	342
6.23.4.353 IsAudioDeviceReady()	342
6.23.4.354 IsAudioStreamPlaying()	342
6.23.4.355 IsAudioStreamProcessed()	342
6.23.4.356 IsAudioStreamValid()	342
6.23.4.357 IsCursorHidden()	342
6.23.4.358 IsCursorOnScreen()	342
6.23.4.359 IsFileDropped()	343
6.23.4.360 IsFileExtension()	343
6.23.4.361 IsFileNameValid()	343
6.23.4.362 IsFontValid()	343
6.23.4.363 IsGamepadAvailable()	343
6.23.4.364 IsGamepadButtonDown()	343
6.23.4.365 IsGamepadButtonPressed()	343
6.23.4.366 IsGamepadButtonReleased()	343
6.23.4.367 IsGamepadButtonUp()	344
6.23.4.368 IsGestureDetected()	344
6.23.4.369 IsImageValid()	344
6.23.4.370 IsKeyDown()	344
6.23.4.371 IsKeyPressed()	344
6.23.4.372 IsKeyPressedRepeat()	344
6.23.4.373 IsKeyReleased()	344
6.23.4.374 IsKeyUp()	344
6.23.4.375 IsMaterialValid()	344
6.23.4.376 IsModelAnimationValid()	345
6.23.4.377 IsModelValid()	345
6.23.4.378 IsMouseButtonDown()	345
6.23.4.379 IsMouseButtonPressed()	345
6.23.4.380 IsMouseButtonReleased()	345
6.23.4.381 IsMouseButtonUp()	345

6.23.4.382 IsMusicStreamPlaying()	345
6.23.4.383 IsMusicValid()	345
6.23.4.384 IsPathFile()	345
6.23.4.385 IsRenderTextureValid()	346
6.23.4.386 IsShaderValid()	346
6.23.4.387 IsSoundPlaying()	346
6.23.4.388 IsSoundValid()	346
6.23.4.389 IsTextureValid()	346
6.23.4.390 IsWaveValid()	346
6.23.4.391 IsWindowFocused()	346
6.23.4.392 IsWindowFullscreen()	346
6.23.4.393 IsWindowHidden()	346
6.23.4.394 IsWindowMaximized()	347
6.23.4.395 IsWindowMinimized()	347
6.23.4.396 IsWindowReady()	347
6.23.4.397 IsWindowResized()	347
6.23.4.398 IsWindowState()	347
6.23.4.399 LoadAudioStream()	347
6.23.4.400 LoadAutomationEventList()	347
6.23.4.401 LoadCodepoints()	347
6.23.4.402 LoadDirectoryFiles()	348
6.23.4.403 LoadDirectoryFilesEx()	348
6.23.4.404 LoadDroppedFiles()	348
6.23.4.405 LoadFileData()	348
6.23.4.406 LoadFileText()	348
6.23.4.407 LoadFont()	348
6.23.4.408 LoadFontData()	348
6.23.4.409 LoadFontEx()	349
6.23.4.410 LoadFontFromImage()	349
6.23.4.411 LoadFontFromMemory()	349
6.23.4.412 LoadImage()	349
6.23.4.413 LoadImageAnim()	349
6.23.4.414 LoadImageAnimFromMemory()	349
6.23.4.415 LoadImageColors()	349
6.23.4.416 LoadImageFromMemory()	350
6.23.4.417 LoadImageFromScreen()	350
6.23.4.418 LoadImageFromTexture()	350
6.23.4.419 LoadImagePalette()	350
6.23.4.420 LoadImageRaw()	350
6.23.4.421 LoadMaterialDefault()	350
6.23.4.422 LoadMaterials()	350
6.23.4.423 LoadModel()	351

6.23.4.424 LoadModelAnimations()	351
6.23.4.425 LoadModelFromMesh()	351
6.23.4.426 LoadMusicStream()	351
6.23.4.427 LoadMusicStreamFromMemory()	351
6.23.4.428 LoadRandomSequence()	351
6.23.4.429 LoadRenderTexture()	351
6.23.4.430 LoadShader()	351
6.23.4.431 LoadShaderFromMemory()	352
6.23.4.432 LoadSound()	352
6.23.4.433 LoadSoundAlias()	352
6.23.4.434 LoadSoundFromWave()	352
6.23.4.435 LoadTextLines()	352
6.23.4.436 LoadTexture()	352
6.23.4.437 LoadTextureCubemap()	352
6.23.4.438 LoadTextureFromImage()	352
6.23.4.439 LoadUTF8()	353
6.23.4.440 LoadVrStereoConfig()	353
6.23.4.441 LoadWave()	353
6.23.4.442 LoadWaveFromMemory()	353
6.23.4.443 LoadWaveSamples()	353
6.23.4.444 MakeDirectory()	353
6.23.4.445 MaximizeWindow()	353
6.23.4.446 MeasureText()	353
6.23.4.447 MeasureTextCodepoints()	354
6.23.4.448 MeasureTextEx()	354
6.23.4.449 MemAlloc()	354
6.23.4.450 MemFree()	354
6.23.4.451 MemRealloc()	354
6.23.4.452 MinimizeWindow()	354
6.23.4.453 OpenURL()	354
6.23.4.454 PauseAudioStream()	354
6.23.4.455 PauseMusicStream()	355
6.23.4.456 PauseSound()	355
6.23.4.457 PlayAudioStream()	355
6.23.4.458 PlayAutomationEvent()	355
6.23.4.459 PlayMusicStream()	355
6.23.4.460 PlaySound()	355
6.23.4.461 PollInputEvents()	355
6.23.4.462 RestoreWindow()	355
6.23.4.463 ResumeAudioStream()	355
6.23.4.464 ResumeMusicStream()	356
6.23.4.465 ResumeSound()	356

6.23.4.466 SaveFileData()	356
6.23.4.467 SaveFileText()	356
6.23.4.468 SeekMusicStream()	356
6.23.4.469 SetAudioStreamBufferSizeDefault()	356
6.23.4.470 SetAudioStreamCallback()	356
6.23.4.471 SetAudioStreamPan()	356
6.23.4.472 SetAudioStreamPitch()	357
6.23.4.473 SetAudioStreamVolume()	357
6.23.4.474 SetAutomationEventBaseFrame()	357
6.23.4.475 SetAutomationEventList()	357
6.23.4.476 SetClipboardText()	357
6.23.4.477 SetConfigFlags()	357
6.23.4.478 SetExitKey()	357
6.23.4.479 SetGamepadMappings()	357
6.23.4.480 SetGamepadVibration()	358
6.23.4.481 SetGesturesEnabled()	358
6.23.4.482 SetLoadFileDataCallback()	358
6.23.4.483 SetLoadFileTextCallback()	358
6.23.4.484 SetMasterVolume()	358
6.23.4.485 SetMaterialTexture()	358
6.23.4.486 SetModelMeshMaterial()	358
6.23.4.487 SetMouseCursor()	358
6.23.4.488 SetMouseOffset()	359
6.23.4.489 SetMousePosition()	359
6.23.4.490 SetMouseScale()	359
6.23.4.491 SetMusicPan()	359
6.23.4.492 SetMusicPitch()	359
6.23.4.493 SetMusicVolume()	359
6.23.4.494 SetPixelColor()	359
6.23.4.495 SetRandomSeed()	359
6.23.4.496 SetSaveFileDataCallback()	360
6.23.4.497 SetSaveFileTextCallback()	360
6.23.4.498 SetShaderValue()	360
6.23.4.499 SetShaderValueMatrix()	360
6.23.4.500 SetShaderValueTexture()	360
6.23.4.501 SetShaderValueV()	360
6.23.4.502 SetShapesTexture()	360
6.23.4.503 SetSoundPan()	361
6.23.4.504 SetSoundPitch()	361
6.23.4.505 SetSoundVolume()	361
6.23.4.506 SetTargetFPS()	361
6.23.4.507 SetTextLineSpacing()	361

6.23.4.508 SetTextureFilter()	361
6.23.4.509 SetTextureWrap()	361
6.23.4.510 SetTraceLogCallback()	361
6.23.4.511 SetTraceLogLevel()	362
6.23.4.512 SetWindowFocused()	362
6.23.4.513 SetWindowIcon()	362
6.23.4.514 SetWindowIcons()	362
6.23.4.515 SetWindowMaxSize()	362
6.23.4.516 SetWindowMinSize()	362
6.23.4.517 SetWindowMonitor()	362
6.23.4.518 SetWindowOpacity()	362
6.23.4.519 SetWindowPosition()	363
6.23.4.520 SetWindowSize()	363
6.23.4.521 SetWindowState()	363
6.23.4.522 SetWindowTitle()	363
6.23.4.523 ShowCursor()	363
6.23.4.524 StartAutomationEventRecording()	363
6.23.4.525 StopAudioStream()	363
6.23.4.526 StopAutomationEventRecording()	363
6.23.4.527 StopMusicStream()	364
6.23.4.528 StopSound()	364
6.23.4.529 SwapScreenBuffer()	364
6.23.4.530 TakeScreenshot()	364
6.23.4.531 TextAppend()	364
6.23.4.532 TextCopy()	364
6.23.4.533 TextFindIndex()	364
6.23.4.534 TextFormat()	364
6.23.4.535 TextInsert()	365
6.23.4.536 TextInsertAlloc()	365
6.23.4.537 TextIsEqual()	365
6.23.4.538 TextJoin()	365
6.23.4.539 TextLength()	365
6.23.4.540 TextRemoveSpaces()	365
6.23.4.541 TextReplace()	365
6.23.4.542 TextReplaceAlloc()	366
6.23.4.543 TextReplaceBetween()	366
6.23.4.544 TextReplaceBetweenAlloc()	366
6.23.4.545 TextSplit()	366
6.23.4.546 TextSubtext()	366
6.23.4.547 TextToCamel()	366
6.23.4.548 TextToFloat()	366
6.23.4.549 TextToInteger()	367

6.23.4.550 TextToLower()	367
6.23.4.551 TextToPascal()	367
6.23.4.552 TextToSnake()	367
6.23.4.553 TextToUpper()	367
6.23.4.554 ToggleBorderlessWindowed()	367
6.23.4.555 ToggleFullscreen()	367
6.23.4.556 TraceLog()	367
6.23.4.557 UnloadAudioStream()	368
6.23.4.558 UnloadAutomationEventList()	368
6.23.4.559 UnloadCodepoints()	368
6.23.4.560 UnloadDirectoryFiles()	368
6.23.4.561 UnloadDroppedFiles()	368
6.23.4.562 UnloadFileData()	368
6.23.4.563 UnloadFileText()	368
6.23.4.564 UnloadFont()	368
6.23.4.565 UnloadFontData()	368
6.23.4.566 UnloadImage()	369
6.23.4.567 UnloadImageColors()	369
6.23.4.568 UnloadImagePalette()	369
6.23.4.569 UnloadMaterial()	369
6.23.4.570 UnloadMesh()	369
6.23.4.571 UnloadModel()	369
6.23.4.572 UnloadModelAnimations()	369
6.23.4.573 UnloadMusicStream()	369
6.23.4.574 UnloadRandomSequence()	369
6.23.4.575 UnloadRenderTexture()	370
6.23.4.576 UnloadShader()	370
6.23.4.577 UnloadSound()	370
6.23.4.578 UnloadSoundAlias()	370
6.23.4.579 UnloadTextLines()	370
6.23.4.580 UnloadTexture()	370
6.23.4.581 UnloadUTF8()	370
6.23.4.582 UnloadVrStereoConfig()	370
6.23.4.583 UnloadWave()	370
6.23.4.584 UnloadWaveSamples()	371
6.23.4.585 UpdateAudioStream()	371
6.23.4.586 UpdateCamera()	371
6.23.4.587 UpdateCameraPro()	371
6.23.4.588 UpdateMeshBuffer()	371
6.23.4.589 UpdateModelAnimation()	371
6.23.4.590 UpdateModelAnimationEx()	371
6.23.4.591 UpdateMusicStream()	372

6.23.4.592 UpdateSound()	372
6.23.4.593 UpdateTexture()	372
6.23.4.594 UpdateTextureRec()	372
6.23.4.595 UploadMesh()	372
6.23.4.596 WaitTime()	372
6.23.4.597 WaveCopy()	372
6.23.4.598 WaveCrop()	372
6.23.4.599 WaveFormat()	373
6.23.4.600 WindowShouldClose()	373
6.24 raylib.h	373
6.25 lib/raylib/include/raymath.h File Reference	399
6.25.1 Macro Definition Documentation	403
6.25.1.1 DEG2RAD	403
6.25.1.2 EPSILON	403
6.25.1.3 MatrixToFloat	403
6.25.1.4 PI	403
6.25.1.5 RAD2DEG	403
6.25.1.6 RL_FLOAT16_TYPE	403
6.25.1.7 RL_FLOAT3_TYPE	404
6.25.1.8 RL_MATRIX_TYPE	404
6.25.1.9 RL_QUATERNION_TYPE	404
6.25.1.10 RL_VECTOR2_TYPE	404
6.25.1.11 RL_VECTOR3_TYPE	404
6.25.1.12 RL_VECTOR4_TYPE	404
6.25.1.13 RMAPI	404
6.25.1.14 Vector3ToFloat	404
6.25.2 Typedef Documentation	404
6.25.2.1 float16	404
6.25.2.2 float3	405
6.25.2.3 Matrix	405
6.25.2.4 Vector2	405
6.25.2.5 Vector3	405
6.25.2.6 Vector4	405
6.25.3 Function Documentation	405
6.25.3.1 Clamp()	405
6.25.3.2 FloatEquals()	405
6.25.3.3 Lerp()	405
6.25.3.4 MatrixAdd()	406
6.25.3.5 MatrixCompose()	406
6.25.3.6 MatrixDecompose()	406
6.25.3.7 MatrixDeterminant()	406
6.25.3.8 MatrixFrustum()	406

6.25.3.9 MatrixIdentity()	406
6.25.3.10 MatrixInvert()	406
6.25.3.11 MatrixLookAt()	407
6.25.3.12 MatrixMultiply()	407
6.25.3.13 MatrixMultiplyValue()	407
6.25.3.14 MatrixOrtho()	407
6.25.3.15 MatrixPerspective()	407
6.25.3.16 MatrixRotate()	407
6.25.3.17 MatrixRotateX()	407
6.25.3.18 MatrixRotateXYZ()	408
6.25.3.19 MatrixRotateY()	408
6.25.3.20 MatrixRotateZ()	408
6.25.3.21 MatrixRotateZYX()	408
6.25.3.22 MatrixScale()	408
6.25.3.23 MatrixSubtract()	408
6.25.3.24 MatrixToFloatV()	408
6.25.3.25 MatrixTrace()	408
6.25.3.26 MatrixTranslate()	409
6.25.3.27 MatrixTranspose()	409
6.25.3.28 Normalize()	409
6.25.3.29 QuaternionAdd()	409
6.25.3.30 QuaternionAddValue()	409
6.25.3.31 QuaternionCubicHermiteSpline()	409
6.25.3.32 QuaternionDivide()	409
6.25.3.33 QuaternionEquals()	410
6.25.3.34 QuaternionFromAxisAngle()	410
6.25.3.35 QuaternionFromEuler()	410
6.25.3.36 QuaternionFromMatrix()	410
6.25.3.37 QuaternionFromVector3ToVector3()	410
6.25.3.38 QuaternionIdentity()	410
6.25.3.39 QuaternionInvert()	410
6.25.3.40 QuaternionLength()	410
6.25.3.41 QuaternionLerp()	411
6.25.3.42 QuaternionMultiply()	411
6.25.3.43 QuaternionNlerp()	411
6.25.3.44 QuaternionNormalize()	411
6.25.3.45 QuaternionScale()	411
6.25.3.46 QuaternionSlerp()	411
6.25.3.47 QuaternionSubtract()	411
6.25.3.48 QuaternionSubtractValue()	412
6.25.3.49 QuaternionToAxisAngle()	412
6.25.3.50 QuaternionToEuler()	412

6.25.3.51 QuaternionToMatrix()	412
6.25.3.52 QuaternionTransform()	412
6.25.3.53 Remap()	412
6.25.3.54 Vector2Add()	412
6.25.3.55 Vector2AddValue()	413
6.25.3.56 Vector2Angle()	413
6.25.3.57 Vector2Clamp()	413
6.25.3.58 Vector2ClampValue()	413
6.25.3.59 Vector2CrossProduct()	413
6.25.3.60 Vector2Distance()	413
6.25.3.61 Vector2DistanceSqr()	413
6.25.3.62 Vector2Divide()	414
6.25.3.63 Vector2DotProduct()	414
6.25.3.64 Vector2Equals()	414
6.25.3.65 Vector2Invert()	414
6.25.3.66 Vector2Length()	414
6.25.3.67 Vector2LengthSqr()	414
6.25.3.68 Vector2Lerp()	414
6.25.3.69 Vector2LineAngle()	414
6.25.3.70 Vector2Max()	415
6.25.3.71 Vector2Min()	415
6.25.3.72 Vector2MoveTowards()	415
6.25.3.73 Vector2Multiply()	415
6.25.3.74 Vector2Negate()	415
6.25.3.75 Vector2Normalize()	415
6.25.3.76 Vector2One()	415
6.25.3.77 Vector2Reflect()	415
6.25.3.78 Vector2Refract()	416
6.25.3.79 Vector2Rotate()	416
6.25.3.80 Vector2Scale()	416
6.25.3.81 Vector2Subtract()	416
6.25.3.82 Vector2SubtractValue()	416
6.25.3.83 Vector2Transform()	416
6.25.3.84 Vector2Zero()	416
6.25.3.85 Vector3Add()	416
6.25.3.86 Vector3AddValue()	417
6.25.3.87 Vector3Angle()	417
6.25.3.88 Vector3Barycenter()	417
6.25.3.89 Vector3Clamp()	417
6.25.3.90 Vector3ClampValue()	417
6.25.3.91 Vector3CrossProduct()	417
6.25.3.92 Vector3CubicHermite()	417

6.25.3.93 Vector3Distance()	418
6.25.3.94 Vector3DistanceSqr()	418
6.25.3.95 Vector3Divide()	418
6.25.3.96 Vector3DotProduct()	418
6.25.3.97 Vector3Equals()	418
6.25.3.98 Vector3Invert()	418
6.25.3.99 Vector3Length()	418
6.25.3.100 Vector3LengthSqr()	418
6.25.3.101 Vector3Lerp()	419
6.25.3.102 Vector3Max()	419
6.25.3.103 Vector3Min()	419
6.25.3.104 Vector3MoveTowards()	419
6.25.3.105 Vector3Multiply()	419
6.25.3.106 Vector3Negate()	419
6.25.3.107 Vector3Normalize()	419
6.25.3.108 Vector3One()	419
6.25.3.109 Vector3OrthoNormalize()	420
6.25.3.110 Vector3Perpendicular()	420
6.25.3.111 Vector3Project()	420
6.25.3.112 Vector3Reflect()	420
6.25.3.113 Vector3Refract()	420
6.25.3.114 Vector3Reject()	420
6.25.3.115 Vector3RotateByAxisAngle()	420
6.25.3.116 Vector3RotateByQuaternion()	421
6.25.3.117 Vector3Scale()	421
6.25.3.118 Vector3Subtract()	421
6.25.3.119 Vector3SubtractValue()	421
6.25.3.120 Vector3ToFloatV()	421
6.25.3.121 Vector3Transform()	421
6.25.3.122 Vector3Unproject()	421
6.25.3.123 Vector3Zero()	421
6.25.3.124 Vector4Add()	422
6.25.3.125 Vector4AddValue()	422
6.25.3.126 Vector4Distance()	422
6.25.3.127 Vector4DistanceSqr()	422
6.25.3.128 Vector4Divide()	422
6.25.3.129 Vector4DotProduct()	422
6.25.3.130 Vector4Equals()	422
6.25.3.131 Vector4Invert()	422
6.25.3.132 Vector4Length()	423
6.25.3.133 Vector4LengthSqr()	423
6.25.3.134 Vector4Lerp()	423

6.25.3.135 Vector4Max()	423
6.25.3.136 Vector4Min()	423
6.25.3.137 Vector4MoveTowards()	423
6.25.3.138 Vector4Multiply()	423
6.25.3.139 Vector4Negate()	423
6.25.3.140 Vector4Normalize()	424
6.25.3.141 Vector4One()	424
6.25.3.142 Vector4Scale()	424
6.25.3.143 Vector4Subtract()	424
6.25.3.144 Vector4SubtractValue()	424
6.25.3.145 Vector4Zero()	424
6.25.3.146 Wrap()	424
6.26 raymath.h	425
6.27 lib/raylib/include/rlgl.h File Reference	461
6.27.1 Macro Definition Documentation	468
6.27.1.1 GRAPHICS_API_OPENGL_33	468
6.27.1.2 RL_BLEND_COLOR	468
6.27.1.3 RL_BLEND_DST_ALPHA	468
6.27.1.4 RL_BLEND_DST_RGB	468
6.27.1.5 RL_BLEND_EQUATION	468
6.27.1.6 RL_BLEND_EQUATION_ALPHA	468
6.27.1.7 RL_BLEND_EQUATION_RGB	468
6.27.1.8 RL_BLEND_SRC_ALPHA	469
6.27.1.9 RL_BLEND_SRC_RGB	469
6.27.1.10 RL_CALLOC	469
6.27.1.11 RL_COMPUTE_SHADER	469
6.27.1.12 RL_CONSTANT_ALPHA	469
6.27.1.13 RL_CONSTANT_COLOR	469
6.27.1.14 RL_CULL_DISTANCE_FAR	469
6.27.1.15 RL_CULL_DISTANCE_NEAR	469
6.27.1.16 RL_DEFAULT_BATCH_BUFFER_ELEMENTS	469
6.27.1.17 RL_DEFAULT_BATCH_BUFFERS	470
6.27.1.18 RL_DEFAULT_BATCH_DRAWCALLS	470
6.27.1.19 RL_DEFAULT_BATCH_MAX_TEXTURE_UNITS	470
6.27.1.20 RL_DEFAULT_SHADER_ATTRIB_LOCATION_BONEINDICES	470
6.27.1.21 RL_DEFAULT_SHADER_ATTRIB_LOCATION_BONEWEIGHTS	470
6.27.1.22 RL_DEFAULT_SHADER_ATTRIB_LOCATION_COLOR	470
6.27.1.23 RL_DEFAULT_SHADER_ATTRIB_LOCATION_INDICES	470
6.27.1.24 RL_DEFAULT_SHADER_ATTRIB_LOCATION_INSTANCETRANSFORM	470
6.27.1.25 RL_DEFAULT_SHADER_ATTRIB_LOCATION_NORMAL	470
6.27.1.26 RL_DEFAULT_SHADER_ATTRIB_LOCATION_POSITION	470
6.27.1.27 RL_DEFAULT_SHADER_ATTRIB_LOCATION_TANGENT	471

6.27.1.28 RL_DEFAULT_SHADER_ATTRIB_LOCATION_TEXCOORD	471
6.27.1.29 RL_DEFAULT_SHADER_ATTRIB_LOCATION_TEXCOORD2	471
6.27.1.30 RL_DRAW_FRAMEBUFFER	471
6.27.1.31 RL_DST_ALPHA	471
6.27.1.32 RL_DST_COLOR	471
6.27.1.33 RL_DYNAMIC_COPY	471
6.27.1.34 RL_DYNAMIC_DRAW	471
6.27.1.35 RL_DYNAMIC_READ	471
6.27.1.36 RL_FLOAT	471
6.27.1.37 RL_FRAGMENT_SHADER	472
6.27.1.38 RL_FREE	472
6.27.1.39 RL_FUNC_ADD	472
6.27.1.40 RL_FUNC_REVERSE_SUBTRACT	472
6.27.1.41 RL_FUNC_SUBTRACT	472
6.27.1.42 RL_LINES	472
6.27.1.43 RL_MALLOC	472
6.27.1.44 RL_MATRIX_TYPE	472
6.27.1.45 RL_MAX	472
6.27.1.46 RL_MAX_MATRIX_STACK_SIZE	473
6.27.1.47 RL_MAX_SHADER_LOCATIONS	473
6.27.1.48 RL_MIN	473
6.27.1.49 RL_MODELVIEW	473
6.27.1.50 RL_ONE	473
6.27.1.51 RL_ONE_MINUS_CONSTANT_ALPHA	473
6.27.1.52 RL_ONE_MINUS_CONSTANT_COLOR	473
6.27.1.53 RL_ONE_MINUS_DST_ALPHA	473
6.27.1.54 RL_ONE_MINUS_DST_COLOR	473
6.27.1.55 RL_ONE_MINUS_SRC_ALPHA	473
6.27.1.56 RL_ONE_MINUS_SRC_COLOR	474
6.27.1.57 RL_PROJECTION	474
6.27.1.58 RL_QUADS	474
6.27.1.59 RL_READ_FRAMEBUFFER	474
6.27.1.60 RL_REALLOC	474
6.27.1.61 RL_SHADER_LOC_MAP_DIFFUSE	474
6.27.1.62 RL_SHADER_LOC_MAP_SPECULAR	474
6.27.1.63 RL_SRC_ALPHA	474
6.27.1.64 RL_SRC_ALPHA_SATURATE	474
6.27.1.65 RL_SRC_COLOR	475
6.27.1.66 RL_STATIC_COPY	475
6.27.1.67 RL_STATIC_DRAW	475
6.27.1.68 RL_STATIC_READ	475
6.27.1.69 RL_STREAM_COPY	475

6.27.1.70 RL_STREAM_DRAW	475
6.27.1.71 RL_STREAM_READ	475
6.27.1.72 RL_TEXTURE	475
6.27.1.73 RL_TEXTURE_FILTER_ANISOTROPIC	475
6.27.1.74 RL_TEXTURE_FILTER_LINEAR	475
6.27.1.75 RL_TEXTURE_FILTER_LINEAR_MIP_NEAREST	476
6.27.1.76 RL_TEXTURE_FILTER_MIP_LINEAR	476
6.27.1.77 RL_TEXTURE_FILTER_MIP_NEAREST	476
6.27.1.78 RL_TEXTURE_FILTER_NEAREST	476
6.27.1.79 RL_TEXTURE_FILTER_NEAREST_MIP_LINEAR	476
6.27.1.80 RL_TEXTURE_MAG_FILTER	476
6.27.1.81 RL_TEXTURE_MIN_FILTER	476
6.27.1.82 RL_TEXTURE_MIPMAP_BIAS_RATIO	476
6.27.1.83 RL_TEXTURE_WRAP_CLAMP	476
6.27.1.84 RL_TEXTURE_WRAP_MIRROR_CLAMP	476
6.27.1.85 RL_TEXTURE_WRAP_MIRROR_REPEAT	477
6.27.1.86 RL_TEXTURE_WRAP_REPEAT	477
6.27.1.87 RL_TEXTURE_WRAP_S	477
6.27.1.88 RL_TEXTURE_WRAP_T	477
6.27.1.89 RL_TRIANGLES	477
6.27.1.90 RL_UNSIGNED_BYTE	477
6.27.1.91 RL_VERTEX_SHADER	477
6.27.1.92 RL_ZERO	477
6.27.1.93 RLAPI	477
6.27.1.94 RLGL_VERSION	477
6.27.1.95 TRACELOG	478
6.27.2 Typedef Documentation	478
6.27.2.1 bool	478
6.27.2.2 Matrix	478
6.27.2.3 rlDrawCall	478
6.27.2.4 rlRenderBatch	478
6.27.2.5 rlVertexBuffer	478
6.27.3 Enumeration Type Documentation	478
6.27.3.1 bool	478
6.27.3.2 rlBlendMode	478
6.27.3.3 rlCullMode	479
6.27.3.4 rlFramebufferAttachTextureType	479
6.27.3.5 rlFramebufferAttachType	479
6.27.3.6 rlGLVersion	480
6.27.3.7 rlPixelFormat	480
6.27.3.8 rlShaderAttributeDataType	480
6.27.3.9 rlShaderLocationIndex	481

6.27.3.10 rlShaderUniformDataType	481
6.27.3.11 rlTextureFilter	482
6.27.3.12 rlTraceLogLevel	482
6.27.4 Function Documentation	482
6.27.4.1 rlActiveDrawBuffers()	482
6.27.4.2 rlActiveTextureSlot()	482
6.27.4.3 rlBegin()	482
6.27.4.4 rlBindFramebuffer()	483
6.27.4.5 rlBindImageTexture()	483
6.27.4.6 rlBindShaderBuffer()	483
6.27.4.7 rlBlitFramebuffer()	483
6.27.4.8 rlCheckErrors()	483
6.27.4.9 rlCheckRenderBatchLimit()	483
6.27.4.10 rlClearColor()	483
6.27.4.11 rlClearScreenBuffers()	484
6.27.4.12 rlColor3f()	484
6.27.4.13 rlColor4f()	484
6.27.4.14 rlColor4ub()	484
6.27.4.15 rlColorMask()	484
6.27.4.16 rlComputeShaderDispatch()	484
6.27.4.17 rlCopyFramebuffer()	485
6.27.4.18 rlCopyShaderBuffer()	485
6.27.4.19 rlCubemapParameters()	485
6.27.4.20 rlDisableBackfaceCulling()	485
6.27.4.21 rlDisableColorBlend()	485
6.27.4.22 rlDisableDepthMask()	485
6.27.4.23 rlDisableDepthTest()	485
6.27.4.24 rlDisableFramebuffer()	486
6.27.4.25 rlDisablePointMode()	486
6.27.4.26 rlDisableScissorTest()	486
6.27.4.27 rlDisableShader()	486
6.27.4.28 rlDisableSmoothLines()	486
6.27.4.29 rlDisableStatePointer()	486
6.27.4.30 rlDisableStereoRender()	486
6.27.4.31 rlDisableTexture()	486
6.27.4.32 rlDisableTextureCubemap()	486
6.27.4.33 rlDisableVertexArray()	487
6.27.4.34 rlDisableVertexAttribute()	487
6.27.4.35 rlDisableVertexBuffer()	487
6.27.4.36 rlDisableVertexBufferElement()	487
6.27.4.37 rlDisableWireMode()	487
6.27.4.38 rlDrawRenderBatch()	487

6.27.4.39 rIDrawRenderBatchActive()	487
6.27.4.40 rIDrawVertexArray()	487
6.27.4.41 rIDrawVertexArrayElements()	488
6.27.4.42 rIDrawVertexArrayElementsInstanced()	488
6.27.4.43 rIDrawVertexArrayInstanced()	488
6.27.4.44 rIEnableBackfaceCulling()	488
6.27.4.45 rIEnableColorBlend()	488
6.27.4.46 rIEnableDepthMask()	488
6.27.4.47 rIEnableDepthTest()	488
6.27.4.48 rIEnableFramebuffer()	488
6.27.4.49 rIEnablePointMode()	489
6.27.4.50 rIEnableScissorTest()	489
6.27.4.51 rIEnableShader()	489
6.27.4.52 rIEnableSmoothLines()	489
6.27.4.53 rIEnableStatePointer()	489
6.27.4.54 rIEnableStereoRender()	489
6.27.4.55 rIEnableTexture()	489
6.27.4.56 rIEnableTextureCubemap()	489
6.27.4.57 rIEnableVertexArray()	489
6.27.4.58 rIEnableVertexAttribute()	490
6.27.4.59 rIEnableVertexBuffer()	490
6.27.4.60 rIEnableVertexBufferElement()	490
6.27.4.61 rIEnableWireMode()	490
6.27.4.62 rIEnd()	490
6.27.4.63 rIFramebufferAttach()	490
6.27.4.64 rIFramebufferComplete()	490
6.27.4.65 rIFrustum()	490
6.27.4.66 rIGenTextureMipmaps()	491
6.27.4.67 rIGetActiveFramebuffer()	491
6.27.4.68 rIGetCullDistanceFar()	491
6.27.4.69 rIGetCullDistanceNear()	491
6.27.4.70 rIGetFramebufferHeight()	491
6.27.4.71 rIGetFramebufferWidth()	491
6.27.4.72 rIGetGLTextureFormats()	491
6.27.4.73 rIGetLineWidth()	491
6.27.4.74 rIGetLocationAttrib()	492
6.27.4.75 rIGetLocationUniform()	492
6.27.4.76 rIGetMatrixModelview()	492
6.27.4.77 rIGetMatrixProjection()	492
6.27.4.78 rIGetMatrixProjectionStereo()	492
6.27.4.79 rIGetMatrixTransform()	492
6.27.4.80 rIGetMatrixViewOffsetStereo()	492

6.27.4.81	rlGetPixelFormatName()	492
6.27.4.82	rlGetPointSize()	493
6.27.4.83	rlGetProcAddress()	493
6.27.4.84	rlGetShaderBufferSize()	493
6.27.4.85	rlGetShaderIdDefault()	493
6.27.4.86	rlGetShaderLocsDefault()	493
6.27.4.87	rlGetTextureIdDefault()	493
6.27.4.88	rlGetVersion()	493
6.27.4.89	rglClose()	493
6.27.4.90	rglInit()	493
6.27.4.91	rIsStereoRenderEnabled()	494
6.27.4.92	rlLoadDrawCube()	494
6.27.4.93	rlLoadDrawQuad()	494
6.27.4.94	rlLoadExtensions()	494
6.27.4.95	rlLoadFramebuffer()	494
6.27.4.96	rlLoadIdentity()	494
6.27.4.97	rlLoadRenderBatch()	494
6.27.4.98	rlLoadShader()	494
6.27.4.99	rlLoadShaderBuffer()	495
6.27.4.100	rlLoadShaderProgram()	495
6.27.4.101	rlLoadShaderProgramCompute()	495
6.27.4.102	rlLoadShaderProgramEx()	495
6.27.4.103	rlLoadTexture()	495
6.27.4.104	rlLoadTextureCubemap()	495
6.27.4.105	rlLoadTextureDepth()	495
6.27.4.106	rlLoadVertexArray()	496
6.27.4.107	rlLoadVertexBuffer()	496
6.27.4.108	rlLoadVertexBufferElement()	496
6.27.4.109	rlMatrixMode()	496
6.27.4.110	rlMultMatrixf()	496
6.27.4.111	rlNormal3f()	496
6.27.4.112	rlOrtho()	496
6.27.4.113	rlPopMatrix()	497
6.27.4.114	rlPushMatrix()	497
6.27.4.115	rlReadScreenPixels()	497
6.27.4.116	rlReadShaderBuffer()	497
6.27.4.117	rlReadTexturePixels()	497
6.27.4.118	rlResizeFramebuffer()	497
6.27.4.119	rlRotatef()	497
6.27.4.120	rlScalef()	498
6.27.4.121	rlScissor()	498
6.27.4.122	rlSetBlendFactors()	498

6.27.4.123	rlSetBlendFactorsSeparate()	498
6.27.4.124	rlSetBlendMode()	498
6.27.4.125	rlSetClipPlanes()	498
6.27.4.126	rlSetCullFace()	498
6.27.4.127	rlSetFramebufferHeight()	499
6.27.4.128	rlSetFramebufferWidth()	499
6.27.4.129	rlSetLineWidth()	499
6.27.4.130	rlSetMatrixModelview()	499
6.27.4.131	rlSetMatrixProjection()	499
6.27.4.132	rlSetMatrixProjectionStereo()	499
6.27.4.133	rlSetMatrixViewOffsetStereo()	499
6.27.4.134	rlSetPointSize()	499
6.27.4.135	rlSetRenderBatchActive()	500
6.27.4.136	rlSetShader()	500
6.27.4.137	rlSetTexture()	500
6.27.4.138	rlSetUniform()	500
6.27.4.139	rlSetUniformMatrices()	500
6.27.4.140	rlSetUniformMatrix()	500
6.27.4.141	rlSetUniformSampler()	500
6.27.4.142	rlSetVertexAttribute()	501
6.27.4.143	rlSetVertexAttributeDefault()	501
6.27.4.144	rlSetVertexAttributeDivisor()	501
6.27.4.145	rlTexCoord2f()	501
6.27.4.146	rlTextureParameters()	501
6.27.4.147	rlTranslatef()	501
6.27.4.148	rlUnloadFramebuffer()	501
6.27.4.149	rlUnloadRenderBatch()	502
6.27.4.150	rlUnloadShader()	502
6.27.4.151	rlUnloadShaderBuffer()	502
6.27.4.152	rlUnloadShaderProgram()	502
6.27.4.153	rlUnloadTexture()	502
6.27.4.154	rlUnloadVertexArray()	502
6.27.4.155	rlUnloadVertexBuffer()	502
6.27.4.156	rlUpdateShaderBuffer()	502
6.27.4.157	rlUpdateTexture()	503
6.27.4.158	rlUpdateVertexBuffer()	503
6.27.4.159	rlUpdateVertexBufferElements()	503
6.27.4.160	rlVertex2f()	503
6.27.4.161	rlVertex2i()	503
6.27.4.162	rlVertex3f()	503
6.27.4.163	rlViewport()	504
6.28	rgl.h	504

6.29 lib/raylib/README.md File Reference	573
6.30 lib/screens/auth/admin_draw.h File Reference	573
6.30.1 Detailed Description	573
6.31 admin_draw.h	574
6.32 lib/screens/auth/auth_draw.h File Reference	574
6.32.1 Detailed Description	574
6.33 auth_draw.h	574
6.34 lib/screens/auth/auth_screen.h File Reference	574
6.34.1 Detailed Description	575
6.35 auth_screen.h	575
6.36 lib/screens/auth/auth_shared.h File Reference	575
6.36.1 Detailed Description	576
6.36.2 Variable Documentation	576
6.36.2.1 errore_recupero	576
6.36.2.2 mostra_popup_uscita_auth	576
6.36.2.3 nuovo_utente	576
6.36.2.4 recPassword	576
6.36.2.5 recupero_completato	576
6.36.2.6 recUsername	577
6.36.2.7 utente_modifica_admin	577
6.37 auth_shared.h	577
6.38 lib/screens/auth/auth_update.h File Reference	577
6.38.1 Detailed Description	578
6.38.2 Function Documentation	578
6.38.2.1 AggiornaAuth()	578
6.39 auth_update.h	578
6.40 lib/screens/auth/input_field.h File Reference	578
6.40.1 Detailed Description	579
6.40.2 Variable Documentation	579
6.40.2.1 submit_richiesto	579
6.41 input_field.h	579
6.42 lib/screens/game/bot_ai.h File Reference	579
6.42.1 Detailed Description	580
6.43 bot_ai.h	580
6.44 lib/screens/game/end_screen.h File Reference	581
6.44.1 Detailed Description	581
6.44.2 Function Documentation	581
6.44.2.1 CalcolaClassifica()	581
6.44.2.2 DisegnaFinePartita()	582
6.44.2.3 EsportaStatisticheTxt()	583
6.45 end_screen.h	584
6.46 lib/screens/game/gameplay_animation.h File Reference	584

6.46.1 Detailed Description	584
6.46.2 Function Documentation	584
6.46.2.1 AggiornaAnimazioni()	584
6.46.2.2 DisegnaAnimazioni()	585
6.47 gameplay_animation.h	585
6.48 lib/screens/game/gameplay_chat.h File Reference	585
6.48.1 Detailed Description	586
6.48.2 Function Documentation	586
6.48.2.1 AggiornaChat()	586
6.48.2.2 DisegnaChat()	586
6.48.3 Variable Documentation	587
6.48.3.1 inputChat	587
6.48.3.2 isTyping	587
6.49 gameplay_chat.h	587
6.50 lib/screens/game/gameplay_draw.h File Reference	587
6.50.1 Detailed Description	588
6.50.2 Function Documentation	588
6.50.2.1 DisegnaGameplay()	588
6.51 gameplay_draw.h	589
6.52 lib/screens/game/gameplay_screen.h File Reference	589
6.52.1 Detailed Description	589
6.53 gameplay_screen.h	589
6.54 lib/screens/game/gameplay_shared.h File Reference	590
6.54.1 Detailed Description	590
6.54.2 Function Documentation	590
6.54.2.1 DrawChatMsgSegmentato()	590
6.54.2.2 StartsWithIgnoreCase()	591
6.54.3 Variable Documentation	591
6.54.3.1 areaTavoloCentro	591
6.54.3.2 botTimer	591
6.54.3.3 inputChat	591
6.54.3.4 isTyping	591
6.54.3.5 offsetMouse	592
6.54.3.6 pending_move_index	592
6.54.3.7 popup_uscita_attivo	592
6.55 gameplay_shared.h	592
6.56 lib/screens/game/gameplay_update.h File Reference	592
6.56.1 Detailed Description	593
6.56.2 Function Documentation	593
6.56.2.1 AggiornaGameplay()	593
6.56.2.2 OttieniPosizioneGiocatore()	594
6.57 gameplay_update.h	595

6.58 lib/screens/menu/menu_screen.h File Reference	595
6.58.1 Detailed Description	595
6.58.2 Function Documentation	595
6.58.2.1 DisegnaMenu()	595
6.59 menu_screen.h	596
6.60 lib/screens/menu/stats_screen.h File Reference	596
6.60.1 Detailed Description	597
6.61 stats_screen.h	597
6.62 lib/screens/menu/string_utils.h File Reference	597
6.62.1 Function Documentation	597
6.62.1.1 CompareIgnoreCase()	597
6.63 string_utils.h	597
6.64 lib/screens/string_utils.h File Reference	597
6.64.1 Detailed Description	598
6.64.2 Function Documentation	598
6.64.2.1 CompareIgnoreCase()	598
6.65 string_utils.h	598
6.66 lib/screens/multiplayer/multiplayer_lobby.h File Reference	598
6.66.1 Detailed Description	599
6.66.2 Function Documentation	599
6.66.2.1 DisegnaMultiplayerLobby()	599
6.66.2.2 ResetTimerErroreAvvio()	599
6.67 multiplayer_lobby.h	600
6.68 lib/screens/multiplayer/multiplayer_screen.h File Reference	600
6.68.1 Detailed Description	600
6.69 multiplayer_screen.h	600
6.70 lib/socket/lan/lan_protocol.h File Reference	601
6.70.1 Detailed Description	602
6.70.2 Macro Definition Documentation	602
6.70.2.1 LAN_PACKET_DB	602
6.70.2.2 LAN_PACKET_DEL_SAVE	602
6.70.2.3 LAN_PACKET_DEL_USER	602
6.70.2.4 LAN_PACKET_SAVE	602
6.70.2.5 LAN_PACKET_STATS_SYNC	602
6.70.2.6 MAX_RECV	602
6.70.2.7 SAVE_CHUNK_SIZE	603
6.70.2.8 STORICO_SYNC_COUNT	603
6.70.3 Function Documentation	603
6.70.3.1 __attribute__()	603
6.70.4 Variable Documentation	603
6.70.4.1 LanSaveChunkPacket	603
6.71 lan_protocol.h	604

6.72 lib/socket/lan/lan_save_helpers.h File Reference	605
6.72.1 Detailed Description	605
6.72.2 Function Documentation	605
6.72.2.1 GetSlotPath()	605
6.72.2.2 GetSlotPathForUser()	606
6.72.2.3 GetSlotPathForUsername()	606
6.72.2.4 TrovaRecv()	607
6.73 lan_save_helpers.h	607
6.74 lib/socket/lan/lan_sync.h File Reference	607
6.74.1 Detailed Description	608
6.74.2 Macro Definition Documentation	609
6.74.2.1 LAN_SYNC_INTERVAL	609
6.74.2.2 LAN_SYNC_PORT	609
6.74.3 Function Documentation	609
6.74.3.1 CompattaSavesUtenteCorrente()	609
6.74.3.2 LAN_BroadcastDeleteSave()	609
6.74.3.3 LAN_BroadcastDeleteUser()	610
6.74.3.4 LAN_BroadcastNow()	610
6.74.3.5 LAN_BroadcastSavesNow()	610
6.74.3.6 LAN_BroadcastStatsSync()	611
6.74.3.7 LAN_Close()	611
6.74.3.8 LAN_EmailEsiste()	611
6.74.3.9 LAN_GetUtentiCount()	612
6.74.3.10 LAN_Init()	612
6.74.3.11 LAN_ProcessDBPacket()	612
6.74.3.12 LAN_ProcessSaveChunk()	613
6.74.3.13 LAN_Update()	613
6.74.3.14 LAN_UpdateSaves()	614
6.74.3.15 LAN_UsernameEsiste()	615
6.74.3.16 SerializzaDB()	615
6.74.4 Variable Documentation	616
6.74.4.1 deletedUsers	616
6.74.4.2 dentroSalvaDB	616
6.74.4.3 numDeletedUsers	616
6.75 lan_sync.h	617
6.76 lib/socket/network/network.h File Reference	617
6.76.1 Detailed Description	620
6.77 network.h	620
6.78 lib/socket/network/network_send.h File Reference	622
6.78.1 Detailed Description	623
6.78.2 Function Documentation	623
6.78.2.1 BroadcastChat()	623

6.78.2.2 BroadcastGameState()	624
6.78.2.3 BroadcastPlayerHands()	624
6.78.2.4 SendChat()	624
6.78.2.5 SendColorChoice()	624
6.78.2.6 SendCreateSession()	625
6.78.2.7 SendDraw()	625
6.78.2.8 SendGameState()	625
6.78.2.9 SendHeartbeat()	626
6.78.2.10 SendJoinGame()	626
6.78.2.11 SendJoinSession()	626
6.78.2.12 SendMove()	626
6.78.2.13 SendPlayerHand()	627
6.78.2.14 SendStartGame()	627
6.79 network_send.h	627
6.80 lib/socket/network/packet_handler.h File Reference	628
6.80.1 Detailed Description	628
6.81 packet_handler.h	628
6.82 lib/socket/network/packet_handler_admin.h File Reference	629
6.82.1 Detailed Description	629
6.82.2 Function Documentation	629
6.82.2.1 HandleAdminPacket()	629
6.83 packet_handler_admin.h	631
6.84 lib/socket/network/packet_handler_game.h File Reference	631
6.84.1 Detailed Description	632
6.84.2 Function Documentation	632
6.84.2.1 HandleGamePacket()	632
6.85 packet_handler_game.h	633
6.86 lib/socket/network/socket_compat.h File Reference	633
6.86.1 Detailed Description	635
6.86.2 Macro Definition Documentation	635
6.86.2.1 closesocket	635
6.86.2.2 INVALID_SOCKET	635
6.86.2.3 ioctlsocket	635
6.86.2.4 SOCKET_ERROR	635
6.86.2.5 WSACleanup	636
6.86.2.6 WSAEADDRINUSE	636
6.86.2.7 WSAEADDRNOTAVAIL	636
6.86.2.8 WSAEAFNOSUPPORT	636
6.86.2.9 WSAEALREADY	636
6.86.2.10 WSAECONNABORTED	636
6.86.2.11 WSAECONNRESET	636
6.86.2.12 WSAEDESTADDRREQ	636

6.86.2.13	WSAEINPROGRESS	636
6.86.2.14	WSAEISCONN	636
6.86.2.15	WSAELOOP	637
6.86.2.16	WSAEMSGSIZE	637
6.86.2.17	WSAENAMETOOLONG	637
6.86.2.18	WSAENETDOWN	637
6.86.2.19	WSAENETRESET	637
6.86.2.20	WSAENETUNREACH	637
6.86.2.21	WSAENOBUFS	637
6.86.2.22	WSAENOPROTOOPT	637
6.86.2.23	WSAENOTCONN	637
6.86.2.24	WSAENOTEMPTY	637
6.86.2.25	WSAENOTSOCK	638
6.86.2.26	WSAEOPNOTSUPP	638
6.86.2.27	WSAEPFNOSUPPORT	638
6.86.2.28	WSAEPROTONOSUPPORT	638
6.86.2.29	WSAEPROTOPTYPE	638
6.86.2.30	WSAESHUTDOWN	638
6.86.2.31	WSAESOCKTNOSUPPORT	638
6.86.2.32	WSAETIMEDOUT	638
6.86.2.33	WSAETOOMANYREFS	638
6.86.2.34	WSAEWOULDBLOCK	638
6.86.2.35	WSAGetLastError	639
6.86.2.36	WSAHOST_NOT_FOUND	639
6.86.2.37	WSANO_RECOVERY	639
6.86.2.38	WSANOTINITIALISED	639
6.86.2.39	WSAStartup	639
6.86.2.40	WSATRY_AGAIN	639
6.86.3	Typedef Documentation	639
6.86.3.1	SOCKET	639
6.87	socket_compat.h	640
6.88	lib/socket/network/state_sync.h File Reference	640
6.88.1	Detailed Description	641
6.88.2	Function Documentation	641
6.88.2.1	SyncGameStateFromNetwork()	641
6.89	state_sync.h	642
6.90	lib/socket/server/server_manager.h File Reference	642
6.90.1	Detailed Description	643
6.90.2	Macro Definition Documentation	643
6.90.2.1	MAX_CONNECTIONS	643
6.90.2.2	MAX_GAME_SESSIONS	644
6.90.2.3	MAX_PLAYERS_PER_SESSION	644

6.90.3 Variable Documentation	644
6.90.3.1 serverManager	644
6.91 server_manager.h	644
6.92 lib/socket/utils/network_utils.h File Reference	645
6.92.1 Detailed Description	645
6.93 network_utils.h	646
6.94 lib/ui.h File Reference	646
6.94.1 Detailed Description	647
6.94.2 Function Documentation	647
6.94.2.1 CaricaTextureReali()	647
6.94.2.2 DisegnaBottoneMenuAnimato()	647
6.94.2.3 DisegnaCartaRealistica()	648
6.94.2.4 ScaricaTextureReali()	649
6.95 ui.h	649
6.96 src/auth.c File Reference	650
6.96.1 Detailed Description	651
6.97 src/auth_validation.c File Reference	651
6.97.1 Detailed Description	652
6.97.2 Function Documentation	652
6.97.2.1 Auth_EmailFormalmenteValida()	652
6.97.2.2 Auth_EmailGiaUsata()	653
6.97.2.3 Auth_PasswordValida()	653
6.97.2.4 Auth_PINValido()	654
6.97.2.5 Auth_UsernameGiaUsato()	654
6.97.2.6 Auth_UsernameNonVuoto()	655
6.97.2.7 Auth_VerificaCredenziali()	655
6.97.2.8 Auth_VerificaEmailPIN()	656
6.98 src/data_structures/bst.c File Reference	656
6.98.1 Detailed Description	657
6.99 src/data_structures/chat.c File Reference	657
6.99.1 Detailed Description	658
6.100 src/data_structures/list.c File Reference	658
6.100.1 Detailed Description	658
6.101 src/list.c File Reference	658
6.101.1 Detailed Description	659
6.102 src/data_structures/queue.c File Reference	659
6.102.1 Detailed Description	660
6.103 src/data_structures/stack.c File Reference	660
6.103.1 Detailed Description	661
6.104 src/game/deck.c File Reference	661
6.104.1 Detailed Description	662
6.105 src/game/game_flow.c File Reference	662

6.105.1 Detailed Description	662
6.105.2 Function Documentation	662
6.105.2.1 EliminaPartita()	662
6.105.2.2 NuovaPartita()	663
6.105.2.3 RicostruisciStatoADT()	663
6.106 src/game/save_manager.c File Reference	664
6.106.1 Detailed Description	664
6.106.2 Function Documentation	665
6.106.2.1 CaricaPartitaDaSlot()	665
6.106.2.2 CompattaSalvataggiUtente()	665
6.106.2.3 EliminaSalvataggio()	665
6.106.2.4 EliminaTuttiSalvataggiUtente()	665
6.106.2.5 GetSaveFilePath()	666
6.106.2.6 SalvaPartita()	666
6.107 src/game_logic.c File Reference	667
6.107.1 Detailed Description	667
6.107.2 Function Documentation	668
6.107.2.1 ApplicaEffetto()	668
6.107.2.2 MossaValida()	668
6.107.2.3 MostraNotifica()	669
6.107.3 Variable Documentation	669
6.107.3.1 coda_turni	669
6.107.3.2 mazzo_pila	669
6.107.3.3 scarti_pila	670
6.108 src/main.c File Reference	670
6.108.1 Detailed Description	670
6.108.2 Function Documentation	671
6.108.2.1 main()	671
6.108.3 Variable Documentation	671
6.108.3.1 esc_consumato	671
6.109 src/screens/auth/admin_draw.c File Reference	671
6.109.1 Detailed Description	672
6.109.2 Macro Definition Documentation	673
6.109.2.1 ALTEZZA	673
6.109.2.2 LARGHEZZA	673
6.109.3 Variable Documentation	673
6.109.3.1 admin_scroll_reset	673
6.109.3.2 clientSockets	673
6.109.3.3 connectedClientUsernames	673
6.109.3.4 deletedUsers	673
6.109.3.5 errore_recupero	673
6.109.3.6 esc_consumato	674

6.109.3.7 mostra_popup_uscita_auth	674
6.109.3.8 numDeletedUsers	674
6.109.3.9 recPassword	674
6.109.3.10 recupero_completato	674
6.109.3.11 recUsername	674
6.109.3.12 utente_modifica_admin	674
6.110 src/screens/auth/auth_draw.c File Reference	675
6.110.1 Detailed Description	675
6.110.2 Macro Definition Documentation	676
6.110.2.1 ALTEZZA	676
6.110.2.2 LARGHEZZA	676
6.110.3 Variable Documentation	676
6.110.3.1 errore_recupero	676
6.110.3.2 esc_consumato	676
6.110.3.3 mostra_popup_uscita_auth	676
6.110.3.4 recPassword	676
6.110.3.5 recupero_completato	676
6.110.3.6 recUsername	677
6.110.3.7 utente_modifica_admin	677
6.111 src/screens/auth/auth_screen.c File Reference	677
6.111.1 Detailed Description	678
6.111.2 Macro Definition Documentation	678
6.111.2.1 ALTEZZA	678
6.111.2.2 LARGHEZZA	678
6.111.3 Variable Documentation	678
6.111.3.1 errore_recupero	678
6.111.3.2 esc_consumato	678
6.111.3.3 mostra_popup_uscita_auth	678
6.111.3.4 nuovo_utente	679
6.111.3.5 recPassword	679
6.111.3.6 recupero_completato	679
6.111.3.7 recUsername	679
6.111.3.8 utente_modifica_admin	679
6.112 src/screens/auth/auth_update.c File Reference	680
6.112.1 Detailed Description	680
6.112.2 Function Documentation	681
6.112.2.1 AggiornaAuth()	681
6.112.3 Variable Documentation	681
6.112.3.1 admin_scroll_reset	681
6.112.3.2 currentRole	681
6.112.3.3 errore_recupero	681
6.112.3.4 mostra_popup_uscita_auth	681

6.112.3.5 recPassword	681
6.112.3.6 recupero_completato	682
6.112.3.7 recUsername	682
6.112.3.8 utente_modifica_admin	682
6.113 src/screens/auth/input_field.c File Reference	682
6.113.1 Detailed Description	683
6.113.2 Variable Documentation	683
6.113.2.1 submit_richiesto	683
6.114 src/screens/game/bot_ai.c File Reference	683
6.114.1 Detailed Description	683
6.115 src/screens/game/chat_render.c File Reference	684
6.115.1 Detailed Description	685
6.115.2 Macro Definition Documentation	685
6.115.2.1 ALTEZZA	685
6.115.2.2 LARGHEZZA	685
6.115.3 Function Documentation	685
6.115.3.1 DisegnaPannelloChat()	685
6.115.3.2 DisegnaPopupUscita()	686
6.115.3.3 DrawChatMsgSegmentato()	686
6.115.3.4 StartsWithIgnoreCase()	687
6.115.4 Variable Documentation	687
6.115.4.1 inputChat	687
6.115.4.2 isTyping	687
6.115.4.3 popup_uscita_attivo	687
6.116 src/screens/game/end_screen.c File Reference	687
6.116.1 Detailed Description	688
6.116.2 Macro Definition Documentation	689
6.116.2.1 ALTEZZA	689
6.116.2.2 LARGHEZZA	689
6.116.3 Function Documentation	689
6.116.3.1 CalcolaClassifica()	689
6.116.3.2 DisegnaFinePartita()	689
6.116.3.3 EsportaStatisticheTxt()	690
6.116.4 Variable Documentation	690
6.116.4.1 client_play_again_attesa	690
6.117 src/screens/game/gameplay_draw.c File Reference	690
6.117.1 Detailed Description	691
6.117.2 Macro Definition Documentation	692
6.117.2.1 ALTEZZA	692
6.117.2.2 LARGHEZZA	692
6.117.3 Function Documentation	692
6.117.3.1 DisegnaGameplay()	692

6.117.4 Variable Documentation	693
6.117.4.1 areaTavoloCentro	693
6.117.4.2 botTimer	693
6.117.4.3 esc_consumato	693
6.117.4.4 inputChat	693
6.117.4.5 isTyping	693
6.117.4.6 offsetMouse	693
6.117.4.7 pending_move_index	693
6.117.4.8 popup_uscita_attivo	694
6.118 src/screens/game/gameplay_screen.c File Reference	694
6.118.1 Detailed Description	695
6.118.2 Macro Definition Documentation	695
6.118.2.1 ALTEZZA	695
6.118.2.2 LARGHEZZA	695
6.118.3 Variable Documentation	695
6.118.3.1 areaTavoloCentro	695
6.118.3.2 botTimer	695
6.118.3.3 esc_consumato	695
6.118.3.4 inputChat	696
6.118.3.5 isTyping	696
6.118.3.6 offsetMouse	696
6.118.3.7 pending_move_index	696
6.118.3.8 popup_uscita_attivo	696
6.119 src/screens/game/gameplay_update.c File Reference	696
6.119.1 Detailed Description	697
6.119.2 Macro Definition Documentation	697
6.119.2.1 ALTEZZA	697
6.119.2.2 LARGHEZZA	698
6.119.3 Function Documentation	698
6.119.3.1 AggiornaGameplay()	698
6.119.3.2 OttieniPosizioneGiocatore()	698
6.119.4 Variable Documentation	699
6.119.4.1 areaTavoloCentro	699
6.119.4.2 botTimer	699
6.119.4.3 esc_consumato	699
6.119.4.4 inputChat	699
6.119.4.5 isTyping	699
6.119.4.6 offsetMouse	699
6.119.4.7 pending_move_index	700
6.119.4.8 popup_uscita_attivo	700
6.120 src/screens/menu/menu_screen.c File Reference	700
6.120.1 Detailed Description	701

6.120.2 Macro Definition Documentation	701
6.120.2.1 ALTEZZA	701
6.120.2.2 LARGHEZZA	701
6.120.3 Function Documentation	701
6.120.3.1 DisegnaMenu()	701
6.120.4 Variable Documentation	702
6.120.4.1 esc_consumato	702
6.121 src/screens/menu/stats_screen.c File Reference	702
6.121.1 Detailed Description	702
6.121.2 Macro Definition Documentation	703
6.121.2.1 ALTEZZA	703
6.121.2.2 LARGHEZZA	703
6.122 src/screens/menu/string_utils.c File Reference	703
6.122.1 Detailed Description	703
6.122.2 Function Documentation	703
6.122.2.1 CompareIgnoreCase()	703
6.123 src/screens/multiplayer/multiplayer_lobby.c File Reference	704
6.123.1 Detailed Description	704
6.123.2 Macro Definition Documentation	705
6.123.2.1 ALTEZZA	705
6.123.2.2 LARGHEZZA	705
6.123.3 Function Documentation	705
6.123.3.1 DisegnaMultiplayerLobby()	705
6.123.3.2 ResetTimerErroreAvvio()	705
6.124 src/screens/multiplayer/multiplayer_screen.c File Reference	706
6.124.1 Detailed Description	706
6.124.2 Macro Definition Documentation	706
6.124.2.1 ALTEZZA	706
6.124.2.2 LARGHEZZA	707
6.125 src/socket/lan/lan_broadcast.c File Reference	707
6.125.1 Detailed Description	708
6.125.2 Function Documentation	708
6.125.2.1 LAN_BroadcastDeleteSave()	708
6.125.2.2 LAN_BroadcastDeleteUser()	709
6.125.2.3 LAN_BroadcastNow()	709
6.125.2.4 LAN_BroadcastStatsSync()	709
6.125.2.5 LAN_Close()	710
6.125.2.6 LAN_ConfirmDeleteUser()	710
6.125.2.7 LAN_GetUtentiCount()	710
6.125.2.8 LAN_Init()	710
6.125.2.9 LAN_Update()	711
6.125.3 Variable Documentation	712

6.125.3.1 broadcastAddr	712
6.125.3.2 deletedUsers	712
6.125.3.3 dentroSalvaDB	712
6.125.3.4 lanSocket	712
6.125.3.5 numDeletedUsers	712
6.126 src/socket/lan/lan_db_sync.c File Reference	712
6.126.1 Detailed Description	713
6.126.2 Function Documentation	713
6.126.2.1 LAN_EmailEsiste()	713
6.126.2.2 LAN_ProcessDBPacket()	714
6.126.2.3 LAN_UsernameEsiste()	714
6.126.2.4 SerializzaDB()	714
6.127 src/socket/lan/lan_save_helpers.c File Reference	715
6.127.1 Detailed Description	716
6.127.2 Macro Definition Documentation	716
6.127.2.1 MAX_RECV	716
6.127.3 Function Documentation	716
6.127.3.1 GetSlotPath()	716
6.127.3.2 GetSlotPathForUser()	716
6.127.3.3 GetSlotPathForUsername()	717
6.127.3.4 TrovaRecv()	717
6.127.4 Variable Documentation	717
6.127.4.1 recvs	717
6.128 src/socket/lan/lan_save_sync.c File Reference	718
6.128.1 Detailed Description	718
6.128.2 Macro Definition Documentation	719
6.128.2.1 LAN_SAVE_PATH_BUF	719
6.128.2.2 MAX_RECV	719
6.128.3 Function Documentation	719
6.128.3.1 CompattaSavesUtenteCorrente()	719
6.128.3.2 LAN_BroadcastSavesNow()	719
6.128.3.3 LAN_ProcessSaveChunk()	719
6.128.3.4 LAN_UpdateSaves()	720
6.129 src/socket/network/network.c File Reference	720
6.129.1 Detailed Description	721
6.129.2 Function Documentation	722
6.129.2.1 SetNonBlocking()	722
6.129.3 Variable Documentation	722
6.129.3.1 localSocket	722
6.129.3.2 numConnectedClients	722
6.129.3.3 remoteSocket	722
6.129.3.4 serverAddr	722

6.129.3.5 udpDiscoverySocket	722
6.130 src/socket/network/network_send.c File Reference	722
6.130.1 Detailed Description	724
6.130.2 Function Documentation	724
6.130.2.1 BroadcastChat()	724
6.130.2.2 BroadcastGameState()	724
6.130.2.3 BroadcastPlayerHands()	725
6.130.2.4 SendChat()	725
6.130.2.5 SendColorChoice()	725
6.130.2.6 SendCreateSession()	725
6.130.2.7 SendDraw()	726
6.130.2.8 SendGameState()	726
6.130.2.9 SendHeartbeat()	726
6.130.2.10 SendJoinGame()	726
6.130.2.11 SendJoinSession()	727
6.130.2.12 SendMove()	727
6.130.2.13 SendPlayerHand()	727
6.130.2.14 SendStartGame()	728
6.130.3 Variable Documentation	728
6.130.3.1 clientSockets	728
6.130.3.2 remoteSocket	728
6.131 src/socket/network/packet_handler.c File Reference	728
6.131.1 Detailed Description	729
6.132 src/socket/network/packet_handler_admin.c File Reference	729
6.132.1 Detailed Description	730
6.132.2 Function Documentation	730
6.132.2.1 HandleAdminPacket()	730
6.133 src/socket/network/packet_handler_game.c File Reference	732
6.133.1 Detailed Description	732
6.133.2 Function Documentation	733
6.133.2.1 HandleGamePacket()	733
6.134 src/socket/network/state_sync.c File Reference	734
6.134.1 Detailed Description	734
6.134.2 Function Documentation	734
6.134.2.1 SyncGameStateFromNetwork()	734
6.135 src/socket/server/server_manager.c File Reference	735
6.135.1 Detailed Description	736
6.135.2 Variable Documentation	736
6.135.2.1 serverManager	736
6.136 src/socket/utils/network_utils.c File Reference	737
6.136.1 Detailed Description	737
6.137 src/ui.c File Reference	738

6.137.1 Detailed Description	738
6.137.2 Function Documentation	739
6.137.2.1 CaricaTextureReali()	739
6.137.2.2 DisegnaBottoneMenuAnimato()	739
6.137.2.3 DisegnaCartaRealistica()	740
6.137.2.4 ScaricaTextureReali()	740

Chapter 1

Topic Index

1.1 Topics

Here is a list of all topics with brief descriptions:

Entry Point	11
Autenticazione e Database Utenti	11
Validazione Autenticazione	12
Tipi di Dato e Strutture	12
BST (Albero Binario Ricerca)	12
Chat	16
Queue (Coda)	20
Stack (Pila)	25
Logica di Gioco	29
Lista (Lista Concatenata)	33
Admin Draw	39
Auth Draw	40
Schermate Auth	41
Auth Shared	41
Auth Update	41
Input Field	42
Bot AI	43
End Screen	46
Gameplay Animation	46
Gameplay Chat	47
Gameplay Draw	47
Gameplay Screen	47
Gameplay Shared	48
Gameplay Update	48
Menu Screen	48
Stats Screen	49
Multiplayer Lobby	49
Multiplayer Screen	50
String Utils	51
Protocollo LAN	52
Dati Utente LAN	52
Stats Sync Packet	52
Sincronizzazione LAN	52
Rete Multiplayer	53
Network Send	53

Packet Handler	53
Packet Handler Admin	53
Packet Handler Game	54
State Sync	54
Server Manager	54
Network Utils	54
Rendering e Interfaccia Utente	54
Implementazioni Auth	55
Lazy Deletion Implementazione	57
Texture e Risorse Grafiche	63
Eliminazione Diffusa (Lazy Deletion)	64
Database API	66
Nodi BST	70
Operazioni BST	71
Storico Chat	76
API Chat	77
Nodi Coda	81
Struttura Coda	82
API Queue	83
Nodi Pila	91
Struttura Pila	92
API Stack	92
Azioni Bot	98
Nodo Albero Decisionale	99
API Bot AI	100
Ruoli di Rete	103
Tipi di Pacchetto	104
Strutture Pacchetti	105
Admin Sync	126
Stati Sessione	128
Giocatore Sessione	128
Sessione di Gioco	129
Struttura Server Manager	129
API Server Manager	129

Chapter 2

Data Structure Index

2.1 Data Structures

Here are the data structures with brief descriptions:

AdminDBSyncPacket	139
AdminUserAction	139
AdminUserSyncData	140
AudioStream	140
AutomationEvent	141
AutomationEventList	141
BoneInfo	142
BoundingBox	143
Camera2D	143
Camera3D	144
CampolInput	
Struttura per la gestione dei campi di input (area + ID)	145
Carta	
Rappresenta una singola carta UNO	145
ChatStorico	
Cronologia circolare (FIFO) di messaggi	147
ClassificaRecord	
Record per la classifica visualizzata a fine partita	148
CodaTurni	
Coda dei turni: coda doppiamente concatenata con direzione di scorrimento	149
Color	150
DatabaseUtenti	
Database degli utenti in memoria (allocazione DINAMICA)	150
FilePathList	151
float16	152
float3	152
Font	153
GameSession	
Sessione di gioco multiplayer	154
Giocatore	
Rappresenta un giocatore nella partita (umano o bot)	155
GlyphInfo	157
Image	158
LanCompactUser	158
LanDeleteSavePacket	
Pacchetto per segnalazione eliminazione salvataggio	160

LanPacketHeader	
Header comune a tutti i pacchetti database LAN	161
LanStatsSyncPacket	162
ListaCarte	
Struttura container per la lista di carte (mano del giocatore)	163
Material	164
MaterialMap	165
Matrix	166
Mesh	168
MessaggioChat	
Singolo messaggio nella chat multiplayer	170
Model	171
ModelAnimation	172
ModelSkeleton	173
Music	174
NetPacket	174
NodoAlbero	
Nodo dell'albero BST: contiene un Utente e puntatori ai figli	175
NodoCarta	
Nodo per la lista doppiamente concatenata della mano del giocatore	176
NodoCoda	
Nodo della coda: contiene un Giocatore e puntatori next/prev	177
NodoDecisione	
Nodo albero: azione, priorità, figli si/no	178
NodoPila	
Nodo della pila: contiene una carta e puntatore al next	178
NPatchInfo	179
PacketChat	
Pacchetto chat	180
PacketColorChoice	
Pacchetto scelta colore	181
PacketCreateSession	
Pacchetto creazione sessione	181
PacketDraw	
Pacchetto pesca: giocatore e quantità	181
PacketGameFinished	
Pacchetto fine partita	182
PacketGameState	
Pacchetto stato di gioco completo	182
PacketHeartbeat	
Pacchetto heartbeat per keep-alive	183
PacketHostReturnLobby	
Pacchetto host return lobby	183
PacketJoinGame	
Pacchetto join game	184
PacketJoinRejected	
Pacchetto join rifiutato	184
PacketJoinSession	
Pacchetto join sessione	184
PacketMove	
Pacchetto mossa: giocatore, indice carta, colore scelto	185
PacketPlayAgain	
Pacchetto play again (con nome giocatore per fix bug)	185
PacketPlayerHand	
Pacchetto mano giocatore	186
PacketPlayerReady	
Pacchetto ready/play again	186

PacketSessionList	
Pacchetto lista sessioni	187
PacketSessionState	
Pacchetto stato sessione multiplayer	187
PartitaRegistrata	
Record di una singola partita nello storico di un utente	187
Pila	
Pila LIFO: top punta all'ultimo inserito, size è la cardinalità	188
Ray	189
RayCollision	190
Rectangle	190
RenderTexture	191
rIDrawCall	192
rIRenderBatch	193
rIVertexBuffer	194
SaveReceiver	
Struttura per ricezione chunk di salvataggio	195
ServerManager	
Gestore di tutte le sessioni di gioco attive	196
SessionPlayer	
Informazioni giocatore nella sessione	197
Shader	198
Sound	199
Statistiche	
Statistiche complete di un utente registrato	199
StatoGioco	
Stato completo di una partita in corso	201
StatsSyncPacket	
StatsSyncPacket - Sincronizzazione live statistiche dopo una partita	206
TempSave	
Struttura temporanea per ordinamento salvataggi (usata in CompattaSalvataggiUtente)	207
Texture	207
Transform	208
Utente	
Rappresenta un utente registrato nel sistema	209
Vector2	210
Vector3	210
Vector4	211
VrDeviceInfo	212
VrStereoConfig	213
Wave	214

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

lib/ auth.h	Modulo di autenticazione e gestione database utenti	217
lib/ auth_validation.h	Modulo di validazione per l'autenticazione	220
lib/ data_structures.h	Header principale del progetto UNO - Definizione di tutti i tipi di dato	227
lib/ game_logic.h	Modulo di logica di gioco UNO	242
lib/ list.h	ADT Lista Concatenata Bidirezionale per le carte	238
lib/ ui.h	Modulo di rendering dell'interfaccia utente	646
lib/data_structures/ bst.h	Albero Binario di Ricerca (BST) per ricerca utenti per email $O(\log n)$	234
lib/data_structures/ chat.h	ADT per la gestione della cronologia chat multiplayer	235
lib/data_structures/ list.h	Interfaccia ADT ListaDoppia (lista doppiamente concatenata)	237
lib/data_structures/ queue.h	ADT Coda (Queue) bidirezionale per la gestione dei turni di gioco	239
lib/data_structures/ stack.h	ADT Pila (Stack LIFO) per gestione mazzo di pesca e scarti	240
lib/fs/ fs.h	Helper filesystem multiplatforma per creazione directory	242
lib/raylib/include/ raylib.h		254
lib/raylib/include/ raymath.h		399
lib/raylib/include/ rlgl.h		461
lib/screens/ string_utils.h	Utilità per la manipolazione di stringhe	597
lib/screens/auth/ admin_draw.h	Rendering del pannello admin	573
lib/screens/auth/ auth_draw.h	Rendering delle schermate di autenticazione	574
lib/screens/auth/ auth_screen.h	Header principale per le schermate di autenticazione	574

lib/screens/auth/ auth_shared.h	
Stato condiviso tra i moduli delle schermate di autenticazione	575
lib/screens/auth/ auth_update.h	
Logica di aggiornamento delle schermate di autenticazione	577
lib/screens/auth/ input_field.h	
Helper per la gestione dei campi di input nelle schermate auth	578
lib/screens/game/ bot_ai.h	
Albero decisionale per l'IA dei bot	579
lib/screens/game/ end_screen.h	
Schermata di fine partita con classifica e statistiche	581
lib/screens/game/ gameplay_animation.h	
Gestione delle animazioni di gioco	584
lib/screens/game/ gameplay_chat.h	
Gestione della chat in-game	585
lib/screens/game/ gameplay_draw.h	
Rendering della schermata di gioco	587
lib/screens/game/ gameplay_screen.h	
Header principale per la schermata di gioco	589
lib/screens/game/ gameplay_shared.h	
Stato condiviso tra i moduli della schermata di gioco	590
lib/screens/game/ gameplay_update.h	
Logica di aggiornamento del gameplay	592
lib/screens/menu/ menu_screen.h	
Schermata del menu principale e statistiche	595
lib/screens/menu/ stats_screen.h	
Schermata delle statistiche utente	596
lib/screens/menu/ string_utils.h	
.	597
lib/screens/multiplayer/ multiplayer_lobby.h	
Schermate della lobby multiplayer (host e waiting room)	598
lib/screens/multiplayer/ multiplayer_screen.h	
Schermate multiplayer (host, join, lobby)	600
lib/socket/lan/ lan_protocol.h	
Protocollo UDP per sincronizzazione LAN	601
lib/socket/lan/ lan_save_helpers.h	
Header per gli helper di sincronizzazione salvataggi LAN	605
lib/socket/lan/ lan_sync.h	
Sincronizzazione LAN del database utenti via UDP broadcast	607
lib/socket/network/ network.h	
Modulo di rete multiplayer TCP/UDP per partite e sincronizzazione	617
lib/socket/network/ network_send.h	
Funzioni di invio pacchetti multiplayer	622
lib/socket/network/ packet_handler.h	
Header principale per la gestione pacchetti di rete	628
lib/socket/network/ packet_handler_admin.h	
Gestione pacchetti admin e sessione multiplayer	629
lib/socket/network/ packet_handler_game.h	
Gestione pacchetti di gioco (mosse, pesche, chat, disconnessione)	631
lib/socket/network/ socket_compat.h	
Astrazione socket multiplatforma (Windows/macOS/Linux)	633
lib/socket/network/ state_sync.h	
Sincronizzazione stato di gioco da pacchetti di rete	640
lib/socket/server/ server_manager.h	
Gestione delle sessioni di gioco lato server/host	642
lib/socket/utils/ network_utils.h	
Funzioni di utilità per la gestione della rete (IP, codici stanza, disconnessioni)	645
src/ auth.c	
Implementazione modulo autenticazione e database utenti (allocazione dinamica)	650

src/auth_validation.c	Funzioni di validazione per autenticazione e gestione utenti	651
src/game_logic.c	Implementazione logica di gioco UNO: regole, effetti carte, validazione mosse e gestione ADT	667
src/list.c	Implementazione della lista concatenata per le carte UNO	658
src/main.c	Punto di ingresso dell'applicazione UNO Multiplayer	670
src/ui.c	Implementazione modulo di rendering dell'interfaccia utente	738
src/data_structures/bst.c	Implementazione Albero Binario di Ricerca (BST) per ricerca utenti	656
src/data_structures/chat.c	Implementazione ADT ChatStorico per la chat di gioco	657
src/data_structures/list.c	Implementazione lista doppiamente concatenata (funzioni specifiche)	658
src/data_structures/queue.c	Implementazione Coda (Queue) bidirezionale per i turni di gioco	659
src/data_structures/stack.c	Implementazione Pila (Stack LIFO) per mazzo di pesca e scarti	660
src/game/deck.c	Gestione mazzo di pesca e operazioni sulle carte	661
src/game/game_flow.c	Gestione flusso di gioco: inizializzazione, fine partita, ricostruzione ADT	662
src/game/save_manager.c	Gestione salvataggi partite su file binario	664
src/screens/auth/admin_draw.c	Rendering pannello admin e modifica utenti	671
src/screens/auth/auth_draw.c	Rendering delle schermate di autenticazione	675
src/screens/auth/auth_screen.c	Definizioni delle variabili di stato condivise per le schermate auth	677
src/screens/auth/auth_update.c	Logica di aggiornamento delle schermate di autenticazione	680
src/screens/auth/input_field.c	Implementazione del campo di input interattivo	682
src/screens/game/bot_ai.c	Implementazione intelligenza artificiale per i bot UNO	683
src/screens/game/chat_render.c	Rendering della chat e componenti UI correlati	684
src/screens/game/end_screen.c	Implementazione schermata fine partita	687
src/screens/game/gameplay_draw.c	Rendering del gameplay UNO	690
src/screens/game/gameplay_screen.c	Definizioni variabili di stato condivise del gameplay	694
src/screens/game/gameplay_update.c	Logica di aggiornamento del gameplay UNO	696
src/screens/menu/menu_screen.c	Implementazione schermata menu principale	700
src/screens/menu/stats_screen.c	Implementazione schermata statistiche utente	702
src/screens/menu/string_utils.c	Funzioni di utilita' per la gestione di stringhe	703
src/screens/multiplayer/multiplayer_lobby.c	Schermata della lobby multiplayer (host e client)	704
src/screens/multiplayer/multiplayer_screen.c	Schermata di selezione multiplayer (host/client)	706

src/socket/lan/ lan_broadcast.c	
Gestione broadcast LAN: inizializzazione, chiusura e smistamento pacchetti	707
src/socket/lan/ lan_db_sync.c	
Sincronizzazione database utenti via LAN (UDP broadcast)	712
src/socket/lan/ lan_save_helpers.c	
Helper per la sincronizzazione salvataggi partite via LAN	715
src/socket/lan/ lan_save_sync.c	
Sincronizzazione salvataggi partite via LAN (chunk UDP)	718
src/socket/network/ network.c	
Modulo di rete multiplayer TCP/UDP - host e client	720
src/socket/network/ network_send.c	
Funzioni di invio pacchetti per il multiplayer online	722
src/socket/network/ packet_handler.c	
Dispatcher principale dei pacchetti di rete	728
src/socket/network/ packet_handler_admin.c	
Gestione pacchetti admin e sessione di rete (allocazione DINAMICA)	729
src/socket/network/ packet_handler_game.c	
Gestore pacchetti di gioco per multiplayer online	732
src/socket/network/ state_sync.c	
Sincronizzazione dello stato di gioco da remoto a locale	734
src/socket/server/ server_manager.c	
Gestione sessioni di gioco multiplayer	735
src/socket/utils/ network_utils.c	
Funzioni di utilita' per il modulo di rete multiplayer	737

Chapter 4

Topic Documentation

4.1 Entry Point

Inizializza tutti i moduli, gestisce il loop principale e coordina la sincronizzazione LAN e il multiplayer.

Inizializza tutti i moduli, gestisce il loop principale e coordina la sincronizzazione LAN e il multiplayer.

Moduli coordinati:

- Raylib (window, textures, input)
- Database utenti (caricamento/salvataggio)
- Rete multiplayer (TCP per partite, UDP per discovery)
- Sincronizzazione LAN (UDP broadcast per DB utenti)

NOTA: La sincronizzazione LAN (LAN_Init/LAN_Update/LAN_Close) permette a tutti i dispositivi sulla stessa rete di condividere automaticamente il database utenti, in modo che:

- L'admin panel su OGNI dispositivo veda TUTTI gli utenti
- Non si possano creare username già esistenti su LAN
- Ogni dispositivo notifichi subito i nuovi utenti agli altri

4.2 Autenticazione e Database Utenti

Gestisce registrazione, login, recupero password, eliminazione utenti e sincronizzazione Lan dell'Admin Panel.

Files

- file [auth.c](#)
Implementazione modulo autenticazione e database utenti (allocazione dinamica).
- file [auth_validation.c](#)
Funzioni di validazione per autenticazione e gestione utenti.

4.2.1 Detailed Description

Gestisce registrazione, login, recupero password, eliminazione utenti e sincronizzazione Lan dell'Admin Panel.

Caratteristiche principali:

- Database utenti con allocazione dinamica (malloc/realloc)
- Eliminazione lazy: la rimozione fisica avviene solo al logout/disconnessione
- Admin Panel real-time via LAN + TCP broadcast
- BST parallelo per ricerca $O(\log n)$ dell'email

4.3 Validazione Autenticazione

Funzioni di validazione dei campi di input per registrazione, login e verifica delle credenziali nel database.

Funzioni di validazione dei campi di input per registrazione, login e verifica delle credenziali nel database.

Modulo fattorizzato da [auth.c](#) per separare la logica di validazione dalla logica di UI.

4.4 Tipi di Dato e Strutture

Definizione centralizzata di tutti i tipi di dato, enumerazioni, strutture e typedef del progetto UNO.

Definizione centralizzata di tutti i tipi di dato, enumerazioni, strutture e typedef del progetto UNO.

Questo header costituisce il dominio applicativo: contiene entità come [Carta](#), [Utente](#), [Giocatore](#), [StatoGioco](#) e le strutture dati per liste, pile, code e database dinamico. Viene incluso dalla maggior parte dei moduli.

Riferimento: Documentazione CdS - "Strutture dati e tipi (dati strutturati)"

NOTA: Il database utenti e' ora allocato DINAMICAMENTE (malloc/realloc) per garantire scalabilita'. La capacita' iniziale e' DB_CAPACITA_INIZIALE.

4.5 BST (Albero Binario Ricerca)

Implementa un BST con chiave email per ricerca, inserimento e cancellazione in $O(\log n)$ medio degli utenti nel database.

Files

- file [bst.c](#)

Implementazione Albero Binario di Ricerca (BST) per ricerca utenti.

Functions

- `NodoAlbero * BST_CreaNodo (Utente u)`
Crea un nuovo nodo BST.
- `NodoAlbero * BST_Inserisci (NodoAlbero *radice, Utente u, int *duplicato)`
Inserisce un utente nell'albero.
- `NodoAlbero * BST_RicercaEmail (NodoAlbero *radice, const char *email)`
Ricerca un utente per email.
- `NodoAlbero * BST_RimuoviEmail (NodoAlbero *radice, const char *email)`
Rimuove il nodo con l'email specificata.
- `void BST_Distruggi (NodoAlbero *radice)`
Distrugge ricorsivamente l'albero.
- `int BST_Inordina (NodoAlbero *radice, Utente *out, int max)`
Visita in-order: copia utenti in array ordinato.
- `int BST_Altezza (NodoAlbero *radice)`
Restituisce l'altezza dell'albero.
- `int BST_ContaNodi (NodoAlbero *radice)`
Restituisce il numero di nodi.

4.5.1 Detailed Description

Implementa un BST con chiave email per ricerca, inserimento e cancellazione in $O(\log n)$ medio degli utenti nel database.

4.5.2 Function Documentation

4.5.2.1 BST_Altezza()

```
int BST_Altezza (
    NodoAlbero * radice)
```

Restituisce l'altezza dell'albero.

Returns

Altezza (0 se vuoto).

Parameters

<code>radice</code>	Radice.
---------------------	---------

4.5.2.2 BST_ContaNodi()

```
int BST_ContaNodi (
    NodoAlbero * radice)
```

Restituisce il numero di nodi.

Restituisce il numero di nodi nell'albero.

Returns

Numero di nodi.

Parameters

	<i>radice</i>	Radice.
--	---------------	---------

4.5.2.3 BST_CreaNodo()

```
NodoAlbero * BST_CreaNodo (  
    Utente u)
```

Crea un nuovo nodo BST.

Crea un nuovo nodo BST a partire da un [Utente](#).

Precondition

u.email valida.

Postcondition

Nodo allocato e inizializzato.

Parameters

	<i>u</i>	Utente da inserire.
--	----------	-------------------------------------

Returns

Puntatore al nodo, o NULL.

4.5.2.4 BST_Distruggi()

```
void BST_Distruggi (  
    NodoAlbero * radice)
```

Distrugge ricorsivamente l'albero.

Libera ricorsivamente tutta la memoria dell'albero.

Precondition

radice valida o NULL.

Postcondition

Memoria liberata.

Parameters

<i>radice</i>	Radice.
---------------	---------

4.5.2.5 BST_Inordina()

```
int BST_Inordina (
    NodoAlbero * radice,
    Utente * out,
    int max)
```

Visita in-order: copia utenti in array ordinato.

Visita in-order e copia i puntatori agli Utenti in un array.

Precondition

radice valida; out != NULL; max > 0.

Returns

Numero di elementi scritti.

Parameters

<i>radice</i>	Radice.
<i>out</i>	Array output.
<i>max</i>	Dimensione massima.

4.5.2.6 BST_Inserisci()

```
NodoAlbero * BST_Inserisci (
    NodoAlbero * radice,
    Utente u,
    int * duplicato)
```

Inserisce un utente nell'albero.

Precondition

radice valida; u.email non duplicata.

Postcondition

Nuovo nodo inserito se non duplicato.

Parameters

<i>radice</i>	Radice corrente.
<i>u</i>	Utente da inserire.
<i>duplicato</i>	Flag duplicato (1 se email già presente).

Returns

Nuova radice.

4.5.2.7 BST_RicercaEmail()

```
NodoAlbero * BST_RicercaEmail (
    NodoAlbero * radice,
    const char * email)
```

Ricerca un utente per email.

Precondition

radice valida; email != NULL.

Returns

Nodo trovato o NULL.

Parameters

<i>radice</i>	Radice.
<i>email</i>	Email da cercare.

4.5.2.8 BST_RimuoviEmail()

```
NodoAlbero * BST_RimuoviEmail (
    NodoAlbero * radice,
    const char * email)
```

Rimuove il nodo con l'email specificata.

Precondition

radice valida; email esistente.

Postcondition

Nodo rimosso; albero aggiornato.

Parameters

<i>radice</i>	Radice.
<i>email</i>	Email da rimuovere.

Returns

Nuova radice.

4.6 Chat

Fornisce l'ADT [ChatStorico](#) (cronologia circolare FIFO) per memorizzare e formattare i messaggi della chat di gioco.

Files

- file [chat.c](#)
Implementazione ADT [ChatStorico](#) per la chat di gioco.

Functions

- void [Chat_Crea](#) ([ChatStorico](#) *c)
Crea uno storico chat vuoto.
- void [Chat_Aggiungi](#) ([ChatStorico](#) *c, const char *mittente, const char *testo)
Aggiunge un messaggio in coda (FIFO).
- void [Chat_OraCorrente](#) (char *out, size_t out_size)
Genera timestamp "HH:MM".
- const [MessaggioChat](#) * [Chat_Ottieni](#) (const [ChatStorico](#) *c, int i)
Ottiene messaggio per indice (0 = piu' vecchio).
- void [Chat_Svuota](#) ([ChatStorico](#) *c)
Svuota la cronologia chat.
- void [Chat_FormattaRiga](#) (const [MessaggioChat](#) *m, char *out, size_t out_size)
Formatta messaggio in riga "HH:MM @Mittente: testo".

4.6.1 Detailed Description

Fornisce l'ADT [ChatStorico](#) (cronologia circolare FIFO) per memorizzare e formattare i messaggi della chat di gioco.

La struttura [MessaggioChat](#) è definita in [data_structures.h](#).

4.6.2 Function Documentation

4.6.2.1 Chat_Aggiungi()

```
void Chat_Aggiungi (
    ChatStorico * c,
    const char * mittente,
    const char * testo)
```

Aggiunge un messaggio in coda (FIFO).

Precondition

c != NULL; mittente != NULL; testo != NULL.

Postcondition

Messaggio aggiunto; count aggiornato; head ruotato se pieno.

Parameters

	c	ChatStorico .
mittente		Mittente.
testo		Testo.

4.6.2.2 Chat_Crea()

```
void Chat_Crea (
    ChatStorico * c)
```

Crea uno storico chat vuoto.

Precondition

$c \neq \text{NULL}$.

Postcondition

count=0, head=0, buffer azzerato.

Parameters

c	Puntatore a ChatStorico .
-----	---

4.6.2.3 Chat_FormattaRiga()

```
void Chat_FormattaRiga (
    const MessaggioChat * m,
    char * out,
    size_t out_size)
```

Formatta messaggio in riga "HH:MM @Mittente: testo".

Formatta un [MessaggioChat](#) in una riga "HH:MM @Mittente: testo".

Precondition

$m \neq \text{NULL}$; $out \neq \text{NULL}$; $out_size \geq 320$.

Postcondition

Riga formattata in out.

Parameters

m	MessaggioChat .
out	Buffer output.
out_size	Dimensione buffer.

4.6.2.4 Chat_OraCorrente()

```
void Chat_OraCorrente (
    char * out,
    size_t out_size)
```

Genera timestamp "HH:MM".

Genera un timestamp "HH:MM" nel buffer fornito.

Precondition

$out \neq \text{NULL}$; $out_size \geq 6$.

Postcondition

Stringa "HH:MM" in out.

Parameters

	<i>out</i>	Buffer output.
	<i>out_size</i>	Dimensione buffer.

4.6.2.5 Chat_Ottieni()

```
const MessaggioChat * Chat_Ottieni (
    const ChatStorico * c,
    int i)
```

Ottiene messaggio per indice (0 = piu' vecchio).

Restituisce il messaggio in posizione i della cronologia.

Precondition

$c \neq \text{NULL}$; $0 \leq i < c \rightarrow \text{count}$.

Returns

Puntatore a [MessaggioChat](#), o NULL.

Parameters

	<i>c</i>	ChatStorico .
	<i>i</i>	Indice.

4.6.2.6 Chat_Svuota()

```
void Chat_Svuota (
    ChatStorico * c)
```

Svuota la cronologia chat.

Precondition

$c \neq \text{NULL}$.

Postcondition

count=0, head=0.

Parameters

	<i>c</i>	ChatStorico .
--	----------	-------------------------------

4.7 Queue (Coda)

Implementa una coda doppiamente concatenata (deque) per gestire i turni di gioco con supporto al Cambio Giro (inversione verso $O(1)$).

Files

- file `queue.c`
Implementazione Coda (Queue) bidirezionale per i turni di gioco.

Functions

- `CodaTurni * Coda_Crea` (void)
Crea una coda vuota.
- void `Coda_Distruggi` (`CodaTurni *c`)
Distrugge la coda e libera la memoria.
- int `Coda_Enqueue` (`CodaTurni *c`, `Giocatore g`)
Inserisce un giocatore in coda.
- int `Coda_Dequeue` (`CodaTurni *c`, `Giocatore *out`)
Rimuove il giocatore in testa.
- int `Coda_Front` (const `CodaTurni *c`, `Giocatore *out`)
Restituisce il giocatore in testa (senza rimuovere).
- int `Coda_Rear` (const `CodaTurni *c`, `Giocatore *out`)
Restituisce il giocatore in fondo.
- `NodoCoda * Coda_ProxNodo` (const `CodaTurni *c`, `NodoCoda *n`)
Restituisce il prossimo nodo nella direzione corrente.
- int `Coda_Vuota` (const `CodaTurni *c`)
Verifica se la coda e' vuota.
- int `Coda_Dimensione` (const `CodaTurni *c`)
Restituisce il numero di giocatori.
- void `Coda_InvertiVerso` (`CodaTurni *c`)
Inverte il verso di scorrimento (carta Cambio Giro).
- `NodoCoda * Coda_CercaPerIndice` (const `CodaTurni *c`, int indice)
Cerca un nodo per indice (0 = front).
- int `Coda_CopiaPerIndice` (const `CodaTurni *c`, int indice, `Giocatore *out`)
Copia il giocatore all'indice in out.

4.7.1 Detailed Description

Implementa una coda doppiamente concatenata (deque) per gestire i turni di gioco con supporto al Cambio Giro (inversione verso $O(1)$).

4.7.2 Function Documentation

4.7.2.1 Coda_CercaPerIndice()

```
NodoCoda * Coda_CercaPerIndice (
    const CodaTurni * c,
    int indice)
```

Cerca un nodo per indice (0 = front).

Cerca il nodo all'indice specificato (0 = front).

Precondition

$c \neq \text{NULL}$; indice valido.

Returns

[NodoCoda](#), o `NULL`.

Parameters

	<i>c</i>	CodaTurni .
<i>indice</i>	Indice (0-based).	

4.7.2.2 Coda_CopiaPerIndice()

```
int Coda_CopiaPerIndice (
    const CodaTurni * c,
    int indice,
    Giocatore * out)
```

Copia il giocatore all'indice in out.

Copia l'i-esimo giocatore in out (0 = front).

Precondition

$c \neq \text{NULL}$; $out \neq \text{NULL}$; indice valido.

Postcondition

[Giocatore](#) copiato.

Returns

1 se successo, 0 altrimenti.

Parameters

	<i>c</i>	CodaTurni .
	<i>indice</i>	Indice (0-based).
	<i>out</i>	Puntatore output.

4.7.2.3 Coda_Crea()

```
CodaTurni * Coda_Crea (
    void )
```

Crea una coda vuota.

Postcondition

Coda allocata; verso=1 (orario); size=0.

Returns

Puntatore a [CodaTurni](#), o NULL.

4.7.2.4 Coda_Dequeue()

```
int Coda_Dequeue (
    CodaTurni * c,
    Giocatore * out)
```

Rimuove il giocatore in testa.

Rimuove il giocatore in testa (front).

Precondition

c != NULL.

Postcondition

[Giocatore](#) rimosso; size decrementato.

Parameters

	<i>c</i>	CodaTurni .
	<i>out</i>	Puntatore output (puo' essere NULL).

Returns

1 se successo, 0 se vuota.

4.7.2.5 Coda_Dimensione()

```
int Coda_Dimensione (
    const CodaTurni * c)
```

Restituisce il numero di giocatori.

Restituisce il numero di giocatori in coda.

Returns

Dimensione, 0 se NULL.

Parameters

	<i>c</i>	CodaTurni .
--	----------	-----------------------------

4.7.2.6 Coda_Distruggi()

```
void Coda_Distruggi (
    CodaTurni * c)
```

Distrugge la coda e libera la memoria.

Distruttore: libera tutti i nodi e la struttura [CodaTurni](#).

Precondition

c != NULL.

Postcondition

Memoria liberata.

Parameters

	<i>c</i>	CodaTurni .
--	----------	-----------------------------

4.7.2.7 Coda_Enqueue()

```
int Coda_Enqueue (
    CodaTurni * c,
    Giocatore g)
```

Inserisce un giocatore in coda.

Inserisce un giocatore in coda (rear).

Precondition

c != NULL.

Postcondition

[Giocatore](#) aggiunto; size incrementato.

Parameters

	<i>c</i>	CodaTurni .
	<i>g</i>	Giocatore .

Returns

1 se successo, 0 altrimenti.

4.7.2.8 Coda_Front()

```
int Coda_Front (
    const CodaTurni * c,
    Giocatore * out)
```

Restituisce il giocatore in testa (senza rimuovere).

Restituisce il giocatore in testa SENZA rimuoverlo.

Precondition

$c \neq \text{NULL}$.

Returns

1 se successo, 0 se vuota.

Parameters

<i>c</i>	CodaTurni.
<i>out</i>	Puntatore output.

4.7.2.9 Coda_InvertiVerso()

```
void Coda_InvertiVerso (
    CodaTurni * c)
```

Inverte il verso di scorrimento (carta Cambio Giro).

Precondition

$c \neq \text{NULL}$.

Postcondition

verso invertito; front/rear scambiati.

Parameters

<i>c</i>	CodaTurni.
----------	------------

4.7.2.10 Coda_ProxNodo()

```
NodoCoda * Coda_ProxNodo (
    const CodaTurni * c,
    NodoCoda * n)
```

Restituisce il prossimo nodo nella direzione corrente.

Restituisce il nodo che segue *n* nella direzione corrente.

Precondition

$c \neq \text{NULL}$; $n \neq \text{NULL}$.

Returns

Prossimo nodo (con wrap-around), o NULL.

Parameters

<i>c</i>	CodaTurni.
<i>n</i>	Nodo corrente.

4.7.2.11 Coda_Rear()

```
int Coda_Rear (
    const CodaTurni * c,
    Giocatore * out)
```

Restituisce il giocatore in fondo.

Restituisce il giocatore in fondo (rear) SENZA rimuoverlo.

Precondition

c != NULL.

Returns

1 se successo, 0 se vuota.

Parameters

<i>c</i>	CodaTurni.
<i>out</i>	Puntatore output.

4.7.2.12 Coda_Vuota()

```
int Coda_Vuota (
    const CodaTurni * c)
```

Verifica se la coda e' vuota.

Verifica se la coda è vuota.

Returns

1 se vuota, 0 altrimenti.

Parameters

<i>c</i>	CodaTurni.
----------	------------

4.8 Stack (Pila)

Implementa una pila LIFO (Last In, First Out) basata su nodi collegati, utilizzata per il mazzo di pesca e gli scarti UNO.

Files

- file [stack.c](#)
Implementazione [Pila](#) (Stack LIFO) per mazzo di pesca e scarti.

Functions

- [Pila](#) * [Pila_Crea](#) (void)
Crea una pila vuota.
- void [Pila_Distruggi](#) ([Pila](#) *p)
Distrugge la pila e libera la memoria.
- int [Pila_Push](#) ([Pila](#) *p, [Carta](#) c)
Inserisce una carta in cima alla pila.
- int [Pila_Pop](#) ([Pila](#) *p, [Carta](#) *out)
Rimuove la carta in cima.
- int [Pila_Top](#) (const [Pila](#) *p, [Carta](#) *out)
Restituisce la carta in cima (senza rimuovere).
- int [Pila_Vuota](#) (const [Pila](#) *p)
Verifica se la pila e' vuota.
- int [Pila_Dimensione](#) (const [Pila](#) *p)
Restituisce la dimensione della pila.
- void [Pila_Svuota](#) ([Pila](#) *p)
Svuota la pila (libera nodi, mantiene struttura).

4.8.1 Detailed Description

Implementa una pila LIFO (Last In, First Out) basata su nodi collegati, utilizzata per il mazzo di pesca e gli scarti UNO.

4.8.2 Function Documentation

4.8.2.1 [Pila_Crea\(\)](#)

```
Pila * Pila\_Crea (  
    void )
```

Crea una pila vuota.

Postcondition

[Pila](#) allocata; top=NULL; size=0.

Returns

Puntatore a [Pila](#), o NULL.

4.8.2.2 [Pila_Dimensione\(\)](#)

```
int Pila\_Dimensione (  
    const Pila * p)
```

Restituisce la dimensione della pila.

Restituisce la dimensione corrente della pila.

Returns

Dimensione, 0 se NULL.

Parameters

	<i>p</i>	Pila.
--	----------	-------

4.8.2.3 Pila_Distruggi()

```
void Pila_Distruggi (
    Pila * p)
```

Distrugge la pila e libera la memoria.

Distruttore: libera tutti i nodi e la struttura Pila.

Precondition

$p \neq \text{NULL}$.

Postcondition

Memoria liberata.

Parameters

	<i>p</i>	Pila.
--	----------	-------

4.8.2.4 Pila_Pop()

```
int Pila_Pop (
    Pila * p,
    Carta * out)
```

Rimuove la carta in cima.

Rimuove la carta in cima alla pila.

Precondition

$p \neq \text{NULL}$.

Postcondition

Carta rimossa; size decrementato.

Parameters

	<i>p</i>	Pila.
	<i>out</i>	Puntatore output (puo' essere NULL).

Returns

1 se successo, 0 se vuota.

4.8.2.5 Pila_Push()

```
int Pila_Push (
    Pila * p,
    Carta c)
```

Inserisce una carta in cima alla pila.

Precondition

$p \neq \text{NULL}$.

Postcondition

[Carta](#) in cima; size incrementato.

Parameters

p	Pila .
c	Carta .

Returns

1 se successo, 0 altrimenti.

4.8.2.6 Pila_Svuota()

```
void Pila_Svuota (
    Pila * p)
```

Svuota la pila (libera nodi, mantiene struttura).

Svuota la pila lasciando la struttura allocata.

Precondition

$p \neq \text{NULL}$.

Postcondition

top=NULL; size=0.

Parameters

p	Pila .
-----	------------------------

4.8.2.7 Pila_Top()

```
int Pila_Top (
    const Pila * p,
    Carta * out)
```

Restituisce la carta in cima (senza rimuovere).

Restituisce la carta in cima SENZA rimuoverla.

Precondition

$p \neq \text{NULL}$.

Returns

1 se successo, 0 se vuota.

Parameters

<code>p</code>	Pila.
<code>out</code>	Puntatore output.

4.8.2.8 Pila_Vuota()

```
int Pila_Vuota (
    const Pila * p)
```

Verifica se la pila e' vuota.

Verifica se la pila è vuota.

Returns

1 se vuota, 0 altrimenti.

Parameters

<code>p</code>	Pila.
----------------	-----------------------

4.9 Logica di Gioco

Gestisce le regole del gioco UNO, validazione mosse, effetti carte, salvataggio/caricamento partite e manipolazione ADT (mazzo, scarti, mani).

Files

- file [deck.c](#)
Gestione mazzo di pesca e operazioni sulle carte.
- file [game_flow.c](#)
Gestione flusso di gioco: inizializzazione, fine partita, ricostruzione ADT.
- file [save_manager.c](#)
Gestione salvataggi partite su file binario.
- file [game_logic.c](#)
Implementazione logica di gioco UNO: regole, effetti carte, validazione mosse e gestione ADT.

Functions

- void [InizializzaMazzo](#) ([StatoGioco](#) *gioco)
Inizializza il mazzo di 108 carte UNO standard e le mescola (Fisher-Yates).
- void [RimescolaScartiNelMazzo](#) ([StatoGioco](#) *gioco)
Ricarica il mazzo dalla pila scarti (mazzo esaurito).
- void [Pesca](#) ([StatoGioco](#) *gioco, int id_giocatore, int quantita)
Fa pescare un numero di carte a un giocatore.
- int [GetLunghezzaMano](#) ([Giocatore](#) *g)
Restituisce il numero di carte nella mano di un giocatore.
- [NodoCarta](#) * [GetNodoCartaGiocatore](#) ([Giocatore](#) *g, int indice)
Restituisce il nodo carta per indice.
- [Carta](#) [GetCartaGiocatore](#) ([Giocatore](#) *g, int indice)
Restituisce una carta dalla mano per indice.
- void [RimuoviCartaGiocatore](#) ([Giocatore](#) *g, int indice)
Rimuove una carta dalla mano per indice.

4.9.1 Detailed Description

Gestisce le regole del gioco UNO, validazione mosse, effetti carte, salvataggio/caricamento partite e manipolazione ADT (mazzo, scarti, mani).

4.9.2 Function Documentation

4.9.2.1 GetCartaGiocatore()

```
Carta GetCartaGiocatore (
    Giocatore * g,
    int indice)
```

Restituisce una carta dalla mano per indice.

Restituisce una carta dalla mano di un giocatore per indice.

Precondition

$g \neq \text{NULL}$; indice valido.

Returns

[Carta](#), o carta vuota.

Parameters

g	Giocatore .
$indice$	Indice (0-based).

4.9.2.2 GetLunghezzaMano()

```
int GetLunghezzaMano (
    Giocatore * g)
```

Restituisce il numero di carte nella mano di un giocatore.

Precondition

$g \neq \text{NULL}$.

Returns

Numero di carte.

Parameters

g	Giocatore .
-----	-----------------------------

4.9.2.3 GetNodoCartaGiocatore()

```
NodoCarta * GetNodoCartaGiocatore (
    Giocatore * g,
    int indice)
```

Restituisce il nodo carta per indice.

Restituisce il puntatore al nodo di una carta per indice.

Precondition

$g \neq \text{NULL}$; indice valido.

Returns

[NodoCarta](#), o `NULL`.

Parameters

g	Giocatore .
$indice$	Indice (0-based).

4.9.2.4 InizializzaMazzo()

```
void InizializzaMazzo (
    StatoGioco * gioco)
```

Inizializza il mazzo di 108 carte UNO standard e le mescola (Fisher-Yates).

Inizializza il mazzo di pesca con 108 carte UNO standard.

Precondition

$gioco \neq \text{NULL}$.

Postcondition

Mazzo creato, mescolato, caricato in `mazzo_pila`.

Parameters

$gioco$	Stato di gioco.
---------	-----------------

4.9.2.5 Pesca()

```
void Pesca (
    StatoGioco * gioco,
    int id_giocatore,
    int quantita)
```

Fa pescare un numero di carte a un giocatore.

Fa pescare un certo numero di carte a un giocatore.

Precondition

$gioco \neq \text{NULL}$; $id_giocatore$ valido; $quantita > 0$.

Postcondition

Carte pescate nella mano del giocatore.

Parameters

	<i>gioco</i>	Stato di gioco.
	<i>id_giocatore</i>	Indice giocatore.
	<i>quantita</i>	Numero carte.

4.9.2.6 RimescolaScartiNelMazzo()

```
void RimescolaScartiNelMazzo (
    StatoGioco * gioco)
```

Ricarica il mazzo dalla pila scarti (mazzo esaurito).

Rimescola le carte dalla pila degli scarti nel mazzo di pesca.

Precondition

gioco != NULL; num_carte_scarti > 1.

Postcondition

Mazzo ricaricato e rimescolato; notifica mostrata.

Parameters

	<i>gioco</i>	Stato di gioco.
--	--------------	-----------------

4.9.2.7 RimuoviCartaGiocatore()

```
void RimuoviCartaGiocatore (
    Giocatore * g,
    int indice)
```

Rimuove una carta dalla mano per indice.

Rimuove una carta dalla mano di un giocatore per indice.

Precondition

g != NULL; indice valido.

Postcondition

Carta rimossa; num_carte_mano aggiornato.

Parameters

	<i>g</i>	<i>Giocatore</i> .
	<i>indice</i>	Indice (0-based).

4.10 Lista (Lista Concatenata)

Implementa la lista doppiamente concatenata per gestire la mano di ciascun giocatore.

Files

- file [list.c](#)
Implementazione lista doppiamente concatenata (funzioni specifiche).
- file [list.c](#)
Implementazione della lista concatenata per le carte UNO.
- file [list.h](#)
Interfaccia ADT ListaDoppia (lista doppiamente concatenata).

Functions

- [NodoCarta](#) * [Lista_InserisciDopo](#) ([ListaCarte](#) *lista, [NodoCarta](#) *nodo, [Carta](#) c)
Inserisce una carta dopo un nodo specifico (o in testa se nodo==NULL).
- [NodoCarta](#) * [Lista_Precedente](#) ([ListaCarte](#) *lista, [NodoCarta](#) *nodo)
Restituisce il nodo precedente a quello dato.
- [NodoCarta](#) * [Lista_Ultimo](#) ([ListaCarte](#) *lista)
Restituisce l'ultimo nodo della lista (coda).
- void [Lista_RimuoviNodoBidirezionale](#) ([ListaCarte](#) *lista, [NodoCarta](#) *nodo)
Rimuove un nodo dalla lista gestendo correttamente prev/next.
- int [Lista_VerificaBidirezionale](#) ([ListaCarte](#) *lista)
Verifica l'integrità dei puntatori prev/next (debug).
- [ListaCarte](#) * [CreaLista](#) (void)
Crea una nuova lista di carte vuota.
- void [InsertInCoda](#) ([ListaCarte](#) *lista, [Carta](#) carta)
Inserisce una carta in coda alla lista.
- void [InsertInTesta](#) ([ListaCarte](#) *lista, [Carta](#) carta)
Inserisce una carta in testa alla lista.
- [NodoCarta](#) * [GetAt](#) ([ListaCarte](#) *lista, int indice)
Ottiene il nodo all'indice specificato (0-based).
- void [RemoveAt](#) ([ListaCarte](#) *lista, int indice)
Rimuove la carta all'indice specificato.
- void [RemoveNode](#) ([ListaCarte](#) *lista, [NodoCarta](#) *nodo)
Rimuove un nodo specifico dalla lista.
- void [EliminaLista](#) ([ListaCarte](#) *lista)
Elimina completamente la lista e libera la memoria.
- void [StampaLista](#) ([ListaCarte](#) *lista)
Stampa il contenuto della lista su stdout (debug).

4.10.1 Detailed Description

Implementa la lista doppiamente concatenata per gestire la mano di ciascun giocatore.

Ogni nodo contiene una carta e puntatori al nodo successivo e precedente.

Riferimento: Documentazione CdS - "Lista (Lista Concatenata)" "Implementata per gestire la mano di ciascun giocatore. Poiché il numero di carte in mano varia continuamente (pescando o scartando), una lista doppiamente concatenata permette inserimenti ed eliminazioni efficienti senza sprecare memoria."

4.10.2 Function Documentation

4.10.2.1 CreaLista()

```
ListaCarte * CreaLista (  
    void )
```

Crea una nuova lista di carte vuota.

Postcondition

Lista allocata e inizializzata (testa=coda=NULL, lunghezza=0).

Returns

Puntatore alla lista, o NULL se out-of-memory.

4.10.2.2 EliminaLista()

```
void EliminaLista (  
    ListaCarte * lista)
```

Elimina completamente la lista e libera la memoria.

Elimina tutta la lista e libera la memoria.

Precondition

lista != NULL.

Postcondition

Tutti i nodi e la struttura deallocati.

Parameters

<i>lista</i>	Puntatore alla lista.
--------------	-----------------------

4.10.2.3 GetAt()

```
NodoCarta * GetAt (  
    ListaCarte * lista,  
    int indice)
```

Ottiene il nodo all'indice specificato (0-based).

Restituisce il puntatore al nodo all'indice specificato.

Precondition

lista != NULL; 0 <= indice < lista->lunghezza.

Returns

Puntatore al [NodoCarta](#), o NULL se fuori range.

Parameters

	<i>lista</i>	Puntatore alla lista.
	<i>indice</i>	Indice (0-based).

4.10.2.4 InsertInCoda()

```
void InsertInCoda (  
    ListaCarte * lista,  
    Carta carta)
```

Inserisce una carta in coda alla lista.

Precondition

`lista != NULL`.

Postcondition

`Carta` aggiunta in coda; lunghezza incrementata.

Parameters

	<i>lista</i>	Puntatore alla lista.
	<i>carta</i>	La carta da inserire.

4.10.2.5 InsertInTesta()

```
void InsertInTesta (  
    ListaCarte * lista,  
    Carta carta)
```

Inserisce una carta in testa alla lista.

Precondition

`lista != NULL`.

Postcondition

`Carta` aggiunta in testa; lunghezza incrementata.

Parameters

	<i>lista</i>	Puntatore alla lista.
	<i>carta</i>	La carta da inserire.

4.10.2.6 Lista_InserisciDopo()

```
NodoCarta * Lista_InserisciDopo (
    ListaCarte * lista,
    NodoCarta * nodo,
    Carta c)
```

Inserisce una carta dopo un nodo specifico (o in testa se nodo==NULL).

Precondition

lista != NULL.

Postcondition

Carta inserita; lunghezza incrementata.

Parameters

	lista	Lista.
	nodo	Nodo dopo cui inserire (NULL per inserimento in testa).
	c	Carta.

Returns

Puntatore al nuovo nodo, o NULL.

4.10.2.7 Lista_Precedente()

```
NodoCarta * Lista_Precedente (
    ListaCarte * lista,
    NodoCarta * nodo)
```

Restituisce il nodo precedente a quello dato.

Parameters

	lista	Lista (non usato).
	nodo	Nodo di riferimento.

Returns

Nodo precedente, o NULL.

4.10.2.8 Lista_RimuoviNodoBidirezionale()

```
void Lista_RimuoviNodoBidirezionale (
    ListaCarte * lista,
    NodoCarta * nodo)
```

Rimuove un nodo dalla lista gestendo correttamente prev/next.

Precondition

lista != NULL; nodo appartiene alla lista.

Postcondition

Nodo rimosso; testa/coda aggiornati; lunghezza decrementata.

Parameters

	<i>lista</i>	Lista.
	<i>nodo</i>	Nodo da rimuovere.

4.10.2.9 Lista_Ultimo()

```
NodoCarta * Lista_Ultimo (  
    ListaCarte * lista)
```

Restituisce l'ultimo nodo della lista (coda).

Precondition

lista != NULL.

Returns

Puntatore alla coda, o NULL.

Parameters

	<i>lista</i>	Lista.
--	--------------	--------

4.10.2.10 Lista_VerificaBidirezionale()

```
int Lista_VerificaBidirezionale (  
    ListaCarte * lista)
```

Verifica l'integrita' dei puntatori prev/next (debug).

Parameters

	<i>lista</i>	Lista.
--	--------------	--------

Returns

1 se valida, 0 se inconsistente.

4.10.2.11 RemoveAt()

```
void RemoveAt (  
    ListaCarte * lista,  
    int indice)
```

Rimuove la carta all'indice specificato.

Rimuove una carta dalla lista per indice.

Precondition

lista != NULL; 0 <= indice < *lista*->lunghezza.

Postcondition

Carta rimossa; nodo deallocato; lunghezza decrementata.

Parameters

	<i>lista</i>	Puntatore alla lista.
	<i>indice</i>	Indice (0-based).

4.10.2.12 RemoveNode()

```
void RemoveNode (  
    ListaCarte * lista,  
    NodoCarta * nodo)
```

Rimuove un nodo specifico dalla lista.

Precondition

`lista != NULL`; nodo appartiene alla lista.

Postcondition

Nodo rimosso; lunghezza decrementata.

Parameters

	<i>lista</i>	Puntatore alla lista.
	<i>nodo</i>	Puntatore al nodo da rimuovere.

4.10.2.13 StampaLista()

```
void StampaLista (  
    ListaCarte * lista)
```

Stampa il contenuto della lista su stdout (debug).

Precondition

`lista != NULL` o lista puo' essere NULL.

Postcondition

Contenuto stampato su console.

Parameters

	<i>lista</i>	Puntatore alla lista.
--	--------------	-----------------------

4.11 Admin Draw

Funzioni di rendering per il pannello admin: visualizzazione e modifica utenti.

Files

- file `admin_draw.c`
Rendering pannello admin e modifica utenti.

Functions

- void `DisegnaAdminPanel` (`FaseApplicazione` *fase)
Disegna il pannello admin con lista utenti scrollabile.
- void `DisegnaAdminModificaUtente` (`FaseApplicazione` *fase)
Disegna il form di modifica/creazione utente (admin).

4.11.1 Detailed Description

Funzioni di rendering per il pannello admin: visualizzazione e modifica utenti.

4.11.2 Function Documentation

4.11.2.1 DisegnaAdminModificaUtente()

```
void DisegnaAdminModificaUtente (
    FaseApplicazione * fase)
```

Disegna il form di modifica/creazione utente (admin).

Disegna la schermata di modifica utente admin.

Precondition

fase != NULL.

Postcondition

Form renderizzato; salvataggio gestito.

Parameters

<code>fase</code>	Fase corrente.
-------------------	----------------

4.11.2.2 DisegnaAdminPanel()

```
void DisegnaAdminPanel (
    FaseApplicazione * fase)
```

Disegna il pannello admin con lista utenti scrollabile.

Disegna il pannello admin con la lista utenti.

Precondition

fase != NULL.

Postcondition

Lista utenti renderizzata; azioni modifica/elimina gestite.

Parameters

<i>fase</i>	Fase corrente.
-------------	----------------

4.12 Auth Draw

Disegna tutte le schermate di autenticazione: login, registrazione, recupero password, profilo, modifica profilo, pannello admin e modifica utente.

Files

- file [auth_draw.c](#)

Rendering delle schermate di autenticazione.

Functions

- int [DisegnaAuth](#) ([FaseApplicazione](#) *fase)

Disegna la schermata di autenticazione corrente.

4.12.1 Detailed Description

Disegna tutte le schermate di autenticazione: login, registrazione, recupero password, profilo, modifica profilo, pannello admin e modifica utente.

4.12.2 Function Documentation

4.12.2.1 DisegnaAuth()

```
int DisegnaAuth (  
    FaseApplicazione * fase)
```

Disegna la schermata di autenticazione corrente.

Precondition

fase != NULL.

Postcondition

Scena auth renderizzata.

Returns

1 se confermata uscita, 0 altrimenti.

Parameters

<code>/fase</code>	Fase corrente.
--------------------	----------------

4.13 Schermate Auth

Include tutti i sotto-moduli delle schermate di login, registrazione, recupero password e admin panel.

Files

- file [auth_screen.c](#)

Definizioni delle variabili di stato condivise per le schermate auth.

4.13.1 Detailed Description

Include tutti i sotto-moduli delle schermate di login, registrazione, recupero password e admin panel.

4.14 Auth Shared

Dichiara la struttura [CampoInput](#) e le variabili di stato condivise tra [auth_update.c](#), [auth_draw.c](#) e [admin_draw.c](#).

Dichiara la struttura [CampoInput](#) e le variabili di stato condivise tra [auth_update.c](#), [auth_draw.c](#) e [admin_draw.c](#).

[auth_screen.c](#) definisce queste variabili, mentre i moduli separati le includono come extern.

4.15 Auth Update

Gestisce l'input utente e le transizioni tra le fasi di login, registrazione, recupero password, modifica profilo e admin panel.

Files

- file [auth_update.c](#)

Logica di aggiornamento delle schermate di autenticazione.

4.15.1 Detailed Description

Gestisce l'input utente e le transizioni tra le fasi di login, registrazione, recupero password, modifica profilo e admin panel.

4.16 Input Field

Funzioni per aggiornare e disegnare campi di input di testo (login, registrazione, recupero password).

Files

- file [input_field.c](#)

Implementazione del campo di input interattivo.

Functions

- void [AggiornaInputCampo](#) (char *buffer, int *count, int maxLength)
Aggiorna lo stato di un campo input.
- void [DisegnaInputField](#) ([Rectangle](#) area, const char *label, const char *text, int count, int isFocused, int isPassword, [Color](#) labelColor)
Disegna un campo input su schermo.

4.16.1 Detailed Description

Funzioni per aggiornare e disegnare campi di input di testo (login, registrazione, recupero password).

4.16.2 Function Documentation

4.16.2.1 AggiornaInputCampo()

```
void AggiornaInputCampo (
    char * buffer,
    int * count,
    int maxLength)
```

Aggiorna lo stato di un campo input.

Aggiorna lo stato di un campo input (carattere premuto, backspace).

Precondition

buffer != NULL; count != NULL; maxLength > 0.

Postcondition

buffer e count aggiornati.

Parameters

	<i>buffer</i>	Buffer campo.
	<i>count</i>	Contatore caratteri.
	<i>maxLength</i>	Lunghezza massima.

4.16.2.2 DisegnaInputField()

```
void DisegnaInputField (
    Rectangle area,
    const char * label,
    const char * text,
    int count,
    int isFocused,
    int isPassword,
    Color labelColor)
```

Disegna un campo input su schermo.

Precondition

text != NULL; label != NULL.

Postcondition

Campo renderizzato.

Parameters

	<i>area</i>	Rettangolo.
	<i>label</i>	Etichetta.
	<i>text</i>	Testo.
	<i>count</i>	Lunghezza testo.
	<i>isFocused</i>	Focus.
	<i>isPassword</i>	Password mode.
	<i>labelColor</i>	Colore label.

4.17 Bot AI

Implementazione di un albero decisionale per la strategia dell'IA dei bot UNO.

Files

- file [bot_ai.c](#)
Implementazione intelligenza artificiale per i bot UNO.

Functions

- [NodoDecisione](#) * [BotAI_CostruisciAlbero](#) (void)
Costruisce l'albero decisionale dell'IA bot (radice).
- void [BotAI_DistruggiAlbero](#) ([NodoDecisione](#) *radice)
Distrugge ricorsivamente l'albero decisionale.
- [DecisioneBot](#) [BotAI_Decidi](#) ([NodoDecisione](#) *radice, [StatoGioco](#) *gioco, int id_giocatore)
Decide la mossa del bot in base alla strategia.
- int [BotAI_TrovaCarta](#) ([StatoGioco](#) *gioco, int id_giocatore, [DecisioneBot](#) azione)
Trova l'indice della carta corrispondente all'azione.
- [Colore](#) [BotAI_ScegliColore](#) ([StatoGioco](#) *gioco, int id_giocatore)
Sceglie il colore piu' frequente nella mano del bot (dopo Jolly).

4.17.1 Detailed Description

Implementazione di un albero decisionale per la strategia dell'IA dei bot UNO.

Priorità IA (albero decisionale):

1. Gioca Jolly+4 (se poche carte in mano)
2. Gioca +2
3. Gioca Cambio Colore
4. Gioca Cambio Giro / Salta
5. Gioca carta numerata più alta
6. Se nessuna mossa: pesca

4.17.2 Function Documentation

4.17.2.1 BotAI_CostruisciAlbero()

```
NodoDecisione * BotAI_CostruisciAlbero (  
    void )
```

Costruisce l'albero decisionale dell'IA bot (radice).

Costruisce l'albero decisionale per l'IA del bot.

Postcondition

Radice allocata con priorit  definita.

Returns

Puntatore alla radice, o NULL.

4.17.2.2 BotAI_Decidi()

```
DecisioneBot BotAI_Decidi (  
    NodoDecisione * radice,  
    StatoGioco * gioco,  
    int id_giocatore)
```

Decide la mossa del bot in base alla strategia.

Valuta l'albero decisionale e restituisce l'azione.

Precondition

radice e gioco validi; id_giocatore valido.

Returns

DecisioneBot presa.

Parameters

	<i>radice</i>	Radice albero.
	<i>gioco</i>	Stato di gioco.
	<i>id_giocatore</i>	Indice del bot.

4.17.2.3 BotAI_DistruggiAlbero()

```
void BotAI_DistruggiAlbero (  
    NodoDecisione * radice)
```

Distrugge ricorsivamente l'albero decisionale.

Distrugge l'albero decisionale.

Precondition

radice valida o NULL.

Postcondition

Memoria liberata.

Parameters

	<i>radice</i>	Radice.
--	---------------	---------

4.17.2.4 BotAI_ScegliColore()

```
Colore BotAI_ScegliColore (  
    StatoGioco * gioco,  
    int id_giocatore)
```

Sceglie il colore piu' frequente nella mano del bot (dopo Jolly).

Sceglie il colore migliore per un Jolly in base alla mano.

Precondition

gioco valido; id_giocatore valido.

Returns

Colore con piu' carte.

Parameters

	<i>gioco</i>	Stato di gioco.
	<i>id_giocatore</i>	Indice del bot.

4.17.2.5 BotAI_TrovaCarta()

```
int BotAI_TrovaCarta (
    StatoGioco * gioco,
    int id_giocatore,
    DecisioneBot azione)
```

Trova l'indice della carta corrispondente all'azione.

Trova l'indice di una carta da giocare per la decisione presa.

Precondition

gioco valido; azione valida.

Returns

Indice carta, -1 se non trovata.

Parameters

	<i>gioco</i>	Stato di gioco.
	<i>id_giocatore</i>	Indice del bot.
	<i>azione</i>	DecisioneBot .

4.18 End Screen

Calcola la classifica finale, esporta statistiche su file .txt e disegna la schermata di fine partita.

Files

- file [end_screen.c](#)
Implementazione schermata fine partita.

4.18.1 Detailed Description

Calcola la classifica finale, esporta statistiche su file .txt e disegna la schermata di fine partita.

4.19 Gameplay Animation

Funzioni per l'aggiornamento e il rendering delle animazioni durante il gameplay (spostamento carte, effetti visivi, etc.).

Funzioni per l'aggiornamento e il rendering delle animazioni durante il gameplay (spostamento carte, effetti visivi, etc.).

4.20 Gameplay Chat

Funzioni di aggiornamento e rendering della chat durante la partita.

Files

- file [chat_render.c](#)

Rendering della chat e componenti UI correlati.

4.20.1 Detailed Description

Funzioni di aggiornamento e rendering della chat durante la partita.

4.21 Gameplay Draw

Contiene la funzione DisegnaGameplay che disegna l'intera schermata di gioco: tavolo, carte scartate, mano, info giocatori, chat, popup uscita e scelta colore.

Files

- file [gameplay_draw.c](#)

Rendering del gameplay UNO.

4.21.1 Detailed Description

Contiene la funzione DisegnaGameplay che disegna l'intera schermata di gioco: tavolo, carte scartate, mano, info giocatori, chat, popup uscita e scelta colore.

4.22 Gameplay Screen

Include tutti i sotto-moduli della schermata di gioco (aggiornamento, rendering, logica condivisa, AI, chat, animazioni).

Files

- file [gameplay_screen.c](#)

Definizioni variabili di stato condivise del gameplay.

4.22.1 Detailed Description

Include tutti i sotto-moduli della schermata di gioco (aggiornamento, rendering, logica condivisa, AI, chat, animazioni).

4.23 Gameplay Shared

Variabili globali condivise e funzioni helper tra [gameplay_update.c](#), [gameplay_draw.c](#), [chat_render.c](#) e [bot_ai.c](#).

Variabili globali condivise e funzioni helper tra [gameplay_update.c](#), [gameplay_draw.c](#), [chat_render.c](#) e [bot_ai.c](#).

4.24 Gameplay Update

Funzioni di aggiornamento dello stato di gioco: gestione input, animazioni, turni bot, drag-and-drop delle carte e scelta colore.

Files

- file [gameplay_update.c](#)
Logica di aggiornamento del gameplay UNO.

4.24.1 Detailed Description

Funzioni di aggiornamento dello stato di gioco: gestione input, animazioni, turni bot, drag-and-drop delle carte e scelta colore.

4.25 Menu Screen

Disegna il menu principale e le schermate annesse (statistiche).

Files

- file [menu_screen.c](#)
Implementazione schermata menu principale.
- file [stats_screen.c](#)
Implementazione schermata statistiche utente.

Functions

- void [DisegnaStatistiche](#) ([FaseApplicazione](#) *fase)
Disegna la schermata statistiche dell'utente loggato.

4.25.1 Detailed Description

Disegna il menu principale e le schermate annesse (statistiche).

4.25.2 Function Documentation

4.25.2.1 DisegnaStatistiche()

```
void DisegnaStatistiche (
    FaseApplicazione * fase)
```

Disegna la schermata statistiche dell'utente loggato.

Disegna la schermata delle statistiche.

Precondition

`fase != NULL`.

Postcondition

`Statistiche` renderizzate; navigazione gestita.

Parameters

<code>fase</code>	Fase corrente.
-------------------	----------------

4.26 Stats Screen

Schermata avanzata per la visualizzazione delle statistiche del giocatore (partite giocate, vittorie, storico partite).

Schermata avanzata per la visualizzazione delle statistiche del giocatore (partite giocate, vittorie, storico partite).

4.27 Multiplayer Lobby

Gestisce la lobby host e la waiting room per il multiplayer con fino a 4 giocatori.

Files

- file `multiplayer_lobby.c`
Schermata della lobby multiplayer (host e client).

Functions

- void `DisegnaWaitingRoom` (`GameSession` *session, `FaseApplicazione` *fase, `StatoGioco` *gioco)
Disegna la waiting room multiplayer.

4.27.1 Detailed Description

Gestisce la lobby host e la waiting room per il multiplayer con fino a 4 giocatori.

4.27.2 Function Documentation

4.27.2.1 DisegnaWaitingRoom()

```
void DisegnaWaitingRoom (
    GameSession * session,
    FaseApplicazione * fase,
    StatoGioco * gioco)
```

Disegna la waiting room multiplayer.

Disegna la waiting room (schermata client in attesa).

Precondition

gioco != NULL.

Postcondition

Lobby renderizzata; logica avvio/attesa gestita.

Parameters

<i>session</i>	Sessione di gioco.
<i>fase</i>	Fase corrente.
<i>gioco</i>	Stato di gioco.

4.28 Multiplayer Screen

Gestisce la logica e il disegno delle schermate multiplayer: creazione/join sessione, lobby host, lobby client.

Files

- file [multiplayer_screen.c](#)
Schermata di selezione multiplayer (host/client).

Functions

- void [ResetMultiplayerError](#) (void)
Resetta l'errore di caricamento multiplayer.
- void [DisegnaMultiplayer](#) ([StatoGioco](#) *gioco, [FaseApplicazione](#) *fase, float *network_countdown, int *network_countdown_active)
Disegna la schermata di selezione multiplayer.

4.28.1 Detailed Description

Gestisce la logica e il disegno delle schermate multiplayer: creazione/join sessione, lobby host, lobby client.

4.28.2 Function Documentation

4.28.2.1 DisegnaMultiplayer()

```
void DisegnaMultiplayer (
    StatoGioco * gioco,
    FaseApplicazione * fase,
    float * network_countdown,
    int * network_countdown_active)
```

Disegna la schermata di selezione multiplayer.

Disegna e gestisce le schermate multiplayer.

Gestisce tre fasi:

- FASE_MULTIPLAYER_SCELTA: bottoni "Crea Stanza" e "Unisciti"
- FASE_CLIENT_JOIN: inserimento codice stanza
- FASE_HOST_LOBBY / FASE_CLIENT_LOBBY: waiting room (delegata)

Parameters

	<i>gioco</i>	Puntatore allo stato di gioco.
	<i>fase</i>	Puntatore alla fase applicazione.
	<i>network_countdown</i>	Puntatore al timer countdown inizio partita.
	<i>network_countdown_active</i>	Puntatore a flag countdown attivo.

4.28.2.2 ResetMultiplayerError()

```
void ResetMultiplayerError (
    void )
```

Resetta l'errore di caricamento multiplayer.

Resetta gli errori della schermata multiplayer.

4.29 String Utils

Funzioni helper per confronto e manipolazione di stringhe.

Files

- file [string_utils.c](#)

Funzioni di utilita' per la gestione di stringhe.

4.29.1 Detailed Description

Funzioni helper per confronto e manipolazione di stringhe.

4.30 Protocollo LAN

Definizione dei magic number, strutture pacchetti e costanti per il protocollo UDP di sincronizzazione LAN.

Definizione dei magic number, strutture pacchetti e costanti per il protocollo UDP di sincronizzazione LAN.

4.31 Dati Utente LAN

Versione compatta di [Statistiche](#) per trasmissione via UDP.

Versione compatta di [Statistiche](#) per trasmissione via UDP.

Struttura compatta per sincronizzazione utenti via UDP.

BUG 2 FIX: Aggiunto campo storico_partite (ultime STORICO_SYNC_COUNT partite) per garantire che la sincronizzazione LAN preservi lo storico partite completo, non solo i contatori numerici.

4.32 Stats Sync Packet

Pacchetto per sincronizzazione COMPLETA delle statistiche (incluso storico partite).

Pacchetto per sincronizzazione COMPLETA delle statistiche (incluso storico partite).

Inviato dopo ogni partita o quando un utente si connette come client.

4.33 Sincronizzazione LAN

Permette a tutti i dispositivi sulla stessa LAN di sincronizzare i database utenti in tempo reale via UDP broadcast (porta 27017).

Permette a tutti i dispositivi sulla stessa LAN di sincronizzare i database utenti in tempo reale via UDP broadcast (porta 27017).

Funzionalita':

- Admin panel su ogni dispositivo vede tutti gli utenti
- Controllo unicità username/email in tutta la LAN
- Notifica immediata di nuovi utenti ai dispositivi connessi

ADT: BST per merge $O(n \log n)$.

4.34 Rete Multiplayer

Gestisce connessioni TCP, sessioni di gioco, pacchetti di rete e stato multiplayer.

Files

- file [network.c](#)
Modulo di rete multiplayer TCP/UDP - host e client.
- file [network_send.c](#)
Funzioni di invio pacchetti per il multiplayer online.
- file [packet_handler.c](#)
Dispatcher principale dei pacchetti di rete.
- file [packet_handler_admin.c](#)
Gestione pacchetti admin e sessione di rete (allocazione DINAMICA).
- file [packet_handler_game.c](#)
Gestore pacchetti di gioco per multiplayer online.
- file [state_sync.c](#)
Sincronizzazione dello stato di gioco da remoto a locale.
- file [network_utils.c](#)
Funzioni di utilità per il modulo di rete multiplayer.

4.34.1 Detailed Description

Gestisce connessioni TCP, sessioni di gioco, pacchetti di rete e stato multiplayer.

Include anche sincronizzazione live statistiche.

4.35 Network Send

Wrapper per l'invio di pacchetti di rete: game state, chat, mosse, admin sync, statistiche e gestione sessioni.

Wrapper per l'invio di pacchetti di rete: game state, chat, mosse, admin sync, statistiche e gestione sessioni.

4.36 Packet Handler

Include tutti i sotto-moduli di gestione pacchetti per retrocompatibilità.

Include tutti i sotto-moduli di gestione pacchetti per retrocompatibilità.

4.37 Packet Handler Admin

Funzioni che gestiscono i pacchetti di amministrazione e sessioni: CREATE_SESSION, JOIN_SESSION, PLAYER_READY, GAME_FINISHED, HEARTBEAT, ADMIN_USER_SYNC, ADMIN_DB_SYNC.

Funzioni che gestiscono i pacchetti di amministrazione e sessioni: CREATE_SESSION, JOIN_SESSION, PLAYER_READY, GAME_FINISHED, HEARTBEAT, ADMIN_USER_SYNC, ADMIN_DB_SYNC.

4.38 Packet Handler Game

Funzioni che gestiscono i pacchetti relativi al gioco: JOIN_GAME, GAME_STATE, PLAYER_HAND, CHAT, DISCONNECT, COLOR_CHOICE, MOVE, DRAW, START_GAME.

Funzioni che gestiscono i pacchetti relativi al gioco: JOIN_GAME, GAME_STATE, PLAYER_HAND, CHAT, DISCONNECT, COLOR_CHOICE, MOVE, DRAW, START_GAME.

4.39 State Sync

Funzione per sincronizzare lo stato locale con lo stato ricevuto via rete.

Funzione per sincronizzare lo stato locale con lo stato ricevuto via rete.

4.40 Server Manager

Implementa il server manager per gestire multiple sessioni di gioco, con validazione lato server delle mosse.

Implementa il server manager per gestire multiple sessioni di gioco, con validazione lato server delle mosse.

4.41 Network Utils

Helper per il modulo di rete: conversione IP, gestione disconnessioni, riallocazione socket e connessione tramite codice stanza.

Helper per il modulo di rete: conversione IP, gestione disconnessioni, riallocazione socket e connessione tramite codice stanza.

4.42 Rendering e Interfaccia Utente

Gestisce il caricamento delle texture, il disegno delle carte e dei componenti UI comuni (bottoni animati, menu).

Files

- file [ui.c](#)

Implementazione modulo di rendering dell'interfaccia utente.

4.42.1 Detailed Description

Gestisce il caricamento delle texture, il disegno delle carte e dei componenti UI comuni (bottoni animati, menu).

Utilizza Raylib per tutte le operazioni di rendering.

4.43 Implementazioni Auth

Implementazioni delle funzionalità di autenticazione e database.

Topics

- [Lazy Deletion Implementazione](#)

Strutture e funzioni per eliminazione differita utenti.

Variables

- [DatabaseUtenti dbUtenti](#) = {0}
Database utenti globale in memoria (allocazione dinamica).
- [int id_utente_corrente](#) = -1
Indice dell'utente correntemente loggato (-1 se nessuno).
- [char inputEmail](#) [100] = "\0"
- [int emailCount](#) = 0
- [char inputNome](#) [30] = "\0"
- [int nomeCount](#) = 0
- [char inputCognome](#) [30] = "\0"
- [int cognomeCount](#) = 0
- [char inputUsername](#) [30] = "\0"
- [int usernameCount](#) = 0
- [char inputPassword](#) [30] = "\0"
- [int passCount](#) = 0
- [char inputPin](#) [7] = "\0"
- [int pinCount](#) = 0
- [int id_temp_login](#) = -1
- [int errore_password](#) = 0
- [int errore_registrazione_pass](#) = 0
- [int errore_registrazione_email](#) = 0
- [int errore_registrazione_username](#) = 0
- [int campo_focusato](#) = 0

4.43.1 Detailed Description

Implementazioni delle funzionalità di autenticazione e database.

4.43.2 Variable Documentation

4.43.2.1 `campo_focusato`

```
int campo_focusato = 0
```

4.43.2.2 `cognomeCount`

```
int cognomeCount = 0
```

4.43.2.3 dbUtenti

```
DatabaseUtenti dbUtenti = {}
```

Database utenti globale in memoria (allocazione dinamica).

4.43.2.4 emailCount

```
int emailCount = 0
```

4.43.2.5 errore_password

```
int errore_password = 0
```

4.43.2.6 errore_registrazione_email

```
int errore_registrazione_email = 0
```

4.43.2.7 errore_registrazione_pass

```
int errore_registrazione_pass = 0
```

4.43.2.8 errore_registrazione_username

```
int errore_registrazione_username = 0
```

4.43.2.9 id_temp_login

```
int id_temp_login = -1
```

4.43.2.10 id_utente_corrente

```
int id_utente_corrente = -1
```

Indice dell'utente correntemente loggato (-1 se nessuno).

4.43.2.11 inputCognome

```
char inputCognome[30] = "\\0"
```


4.43.2.12 inputEmail

```
char inputEmail[100] = "\0"
```

4.43.2.13 inputNome

```
char inputNome[30] = "\0"
```

4.43.2.14 inputPassword

```
char inputPassword[30] = "\0"
```

4.43.2.15 inputPin

```
char inputPin[7] = "\0"
```

4.43.2.16 inputUsername

```
char inputUsername[30] = "\0"
```

4.43.2.17 nomeCount

```
int nomeCount = 0
```

4.43.2.18 passCount

```
int passCount = 0
```

4.43.2.19 pinCount

```
int pinCount = 0
```

4.43.2.20 usernameCount

```
int usernameCount = 0
```

4.43.3 Lazy Deletion Implementazione

Strutture e funzioni per eliminazione differita utenti.

Functions

- int [Auth_MarkForDeletion](#) (const char *email)
Marca un utente per eliminazione differita.
- int [Auth_IsMarkedForDeletion](#) (const char *email)
Verifica se un utente e' marcato per eliminazione.
- void [Auth_ProcessPendingDeletions](#) (const char *email)
Processa le eliminazioni pendenti per un utente.
- void [AuthBST_Ricostruisci](#) (void)
Ricostruisce il BST parallelo a partire dall'array dbUtenti.
- int [DB_Alloca](#) (int min_slot)
Alloca o rialloca dinamicamente il database utenti.
- void [CifraDecifraDB](#) (void)
Cifra/decifra password e PIN in memoria (XOR 0xAA).
- void [SalvaDB](#) (void)
Salva il database su file binario (data/users/users.dat).
- void [CaricaDB](#) (void)
Carica il database da file binario (data/users/users.dat).
- int [Auth_RegistraUtente](#) (const char *email, const char *nome, const char *cognome, const char *username, const char *password, const char *pin)
Registra un nuovo utente nel sistema.
- void [Auth_RimuoviUtente](#) (const char *email)
Rimuove un utente da database e BST.
- int [AuthBST_CercaIndicePerEmail](#) (const char *email)
Ricerca $O(\log n)$ di un utente per email tramite BST.
- int [AuthBST_Altezza](#) (void)
Restituisce l'altezza del BST (utile per debug/test).
- int [AuthBST_ContaNodi](#) (void)
Restituisce il numero di nodi nel BST.

4.43.3.1 Detailed Description

Strutture e funzioni per eliminazione differita utenti.

4.43.3.2 Function Documentation

4.43.3.2.1 Auth_IsMarkedForDeletion()

```
int Auth_IsMarkedForDeletion (
    const char * email)
```

Verifica se un utente e' marcato per eliminazione.

Parameters

<i>email</i>	Email dell'utente.
--------------	--------------------

Returns

1 se marcato, 0 altrimenti.

4.43.3.2.2 Auth_MarkForDeletion()

```
int Auth_MarkForDeletion (  
    const char * email)
```

Marca un utente per eliminazione differita.

Parameters

<i>email</i>	Email dell'utente.
--------------	--------------------

Returns

1 se marcato, 0 altrimenti.

4.43.3.2.3 Auth_ProcessPendingDeletions()

```
void Auth_ProcessPendingDeletions (  
    const char * email)
```

Processa le eliminazioni pendenti per un utente.

Parameters

<i>email</i>	Email dell'utente.
--------------	--------------------

4.43.3.2.4 Auth_RegistraUtente()

```
int Auth_RegistraUtente (  
    const char * email,  
    const char * nome,  
    const char * cognome,  
    const char * username,  
    const char * password,  
    const char * pin)
```

Registra un nuovo utente nel sistema.

Registra un nuovo utente dopo aver generato l'ID.

Parameters

<i>email</i>	Email dell'utente.
<i>nome</i>	Nome.
<i>cognome</i>	Cognome.
<i>username</i>	Username univoco.
<i>password</i>	Password.
<i>pin</i>	PIN di recupero.

Returns

1 se registrazione OK, 0 se errore allocazione o PIN duplicato.

4.43.3.2.5 Auth_RimuoviUtente()

```
void Auth_RimuoviUtente (
    const char * email)
```

Rimuove un utente da database e BST.

Precondition

email esiste nel database.

Postcondition

Utente rimosso da dbUtenti e BST.

Parameters

<i>email</i>	Email dell'utente da rimuovere.
--------------	---------------------------------

4.43.3.2.6 AuthBST_Altezza()

```
int AuthBST_Altezza (
    void )
```

Restituisce l'altezza del BST (utile per debug/test).

Restituisce l'altezza del BST (per debug).

Returns

Altezza dell'albero.

4.43.3.2.7 AuthBST_CercaIndicePerEmail()

```
int AuthBST_CercaIndicePerEmail (
    const char * email)
```

Ricerca $O(\log n)$ di un utente per email tramite BST.

Ricerca l'indice di un utente per email tramite BST.

Returns

Indice nel dbUtenti, -1 se non trovato.

4.43.3.2.8 AuthBST_ContaNodi()

```
int AuthBST_ContaNodi (
    void )
```

Restituisce il numero di nodi nel BST.

Returns

Numero di nodi.

4.43.3.2.9 AuthBST_Ricostruisci()

```
void AuthBST_Ricostruisci (
    void )
```

Ricostruisce il BST parallelo a partire dall'array dbUtenti.

4.43.3.2.10 CaricaDB()

```
void CaricaDB (
    void )
```

Carica il database da file binario (data/users/users.dat).

Carica il database da file binario.

4.43.3.2.11 CifraDecifraDB()

```
void CifraDecifraDB (
    void )
```

Cifra/decifra password e PIN in memoria (XOR 0xAA).

4.43.3.2.12 DB_Alloca()

```
int DB_Alloca (
    int min_slot)
```

Alloca o rialloca dinamicamente il database utenti.

Alloca o rialloca il database per garantire almeno min_slot.

Se la capacita' corrente e' insufficiente, raddoppia l'array con realloc finche' non raggiunge min_slot.

Parameters

<i>min_slot</i>	Numero minimo di slot richiesti.
-----------------	----------------------------------

Returns

1 se allocazione OK, 0 se out-of-memory.

4.43.3.2.13 SalvaDB()

```
void SalvaDB (
    void )
```

Salva il database su file binario (data/users/users.dat).

Salva il database su file binario con cifratura XOR.

4.44 Texture e Risorse Grafiche

Texture globali per il rendering delle carte e sfondo.

Variables

- Texture2D textureCarte [5][15]
Texture delle carte UNO: textureCarte[colore][tipo].
- Texture2D textureDorso
Texture del dorso delle carte (per carte coperte).
- Texture2D textureSfondo
Texture dello sfondo del gioco.
- const Color coloriRaylib []
Colori Raylib associati ai colori UNO (ROSSO, GIALLO, VERDE, BLU, NERO).
- const char * nomiColori [] = { "Rosso", "Giallo", "Verde", "Blu", "Jolly" }
Nomi descrittivi dei colori UNO (per debug/UI).

4.44.1 Detailed Description

Texture globali per il rendering delle carte e sfondo.

4.44.2 Variable Documentation

4.44.2.1 coloriRaylib

```
const Color coloriRaylib[]
```

Initial value:

```
= {
    (Color){ 218, 41, 28, 255 },
    (Color){ 255, 199, 44, 255 },
    (Color){ 0, 154, 68, 255 },
    (Color){ 0, 130, 202, 255 },
    (Color){ 30, 30, 30, 255 }
}
```

Colori Raylib associati ai colori UNO (ROSSO, GIALLO, VERDE, BLU, NERO).

Colori Raylib associati ai colori UNO.

4.44.2.2 nomiColori

```
const char* nomiColori[] = { "Rosso", "Giallo", "Verde", "Blu", "Jolly" }
```

Nomi descrittivi dei colori UNO (per debug/UI).

Nomi descrittivi dei colori UNO.

4.44.2.3 textureCarte

```
Texture2D textureCarte[5][15]
```

Texture delle carte UNO: textureCarte[colore][tipo].

Cache globale delle texture delle carte UNO: textureCarte[colore][tipo].

4.44.2.4 textureDorso

```
Texture2D textureDorso
```

Texture del dorso delle carte (per carte coperte).

4.44.2.5 textureSfondo

```
Texture2D textureSfondo
```

Texture dello sfondo del gioco.

4.45 Eliminazione Diffusa (Lazy Deletion)

API per la rimozione differita degli utenti.

Macros

- `#define MAX_PENDING_DELETIONS 64`
Numero massimo di eliminazioni pendenti.

Functions

- int `Auth_MarkForDeletion` (const char *email)
Marca un utente per eliminazione differita.
- int `Auth_IsMarkedForDeletion` (const char *email)
Verifica se un utente e' marcato per eliminazione.
- void `Auth_ProcessPendingDeletions` (const char *email)
Processa le eliminazioni pendenti per un utente.

4.45.1 Detailed Description

API per la rimozione differita degli utenti.

4.45.2 Macro Definition Documentation

4.45.2.1 MAX_PENDING_DELETIONS

```
#define MAX_PENDING_DELETIONS 64
```

Numero massimo di eliminazioni pendenti.

4.45.3 Function Documentation

4.45.3.1 Auth_IsMarkedForDeletion()

```
int Auth_IsMarkedForDeletion (  
    const char * email)
```

Verifica se un utente e' marcato per eliminazione.

Parameters

<i>email</i>	Email dell'utente.
--------------	--------------------

Returns

1 se marcato, 0 altrimenti.

4.45.3.2 Auth_MarkForDeletion()

```
int Auth_MarkForDeletion (  
    const char * email)
```

Marca un utente per eliminazione differita.

Parameters

<i>email</i>	Email dell'utente.
--------------	--------------------

Returns

1 se marcato, 0 altrimenti.

4.45.3.3 Auth_ProcessPendingDeletions()

```
void Auth_ProcessPendingDeletions (  
    const char * email)
```

Processa le eliminazioni pendenti per un utente.

Parameters

<code>email</code>	Email dell'utente.
--------------------	--------------------

4.46 Database API

Funzioni per la gestione del database utenti in memoria e su file.

Functions

- int [DB_Alloca](#) (int min_slot)
Alloca o rialloca il database per garantire almeno min_slot.
- void [SalvaDB](#) (void)
Salva il database su file binario con cifratura XOR.
- void [AuthBST_Ricostruisci](#) (void)
Ricostruisce il BST parallelo a partire dall'array dbUtenti.
- void [CaricaDB](#) (void)
Carica il database da file binario.
- int [Auth_RegistraUtente](#) (const char *, const char *, const char *, const char *, const char *, const char *)
Registra un nuovo utente dopo aver generato l'ID.
- void [Auth_RimuoviUtente](#) (const char *email)
Rimuove un utente da database e BST.
- int [AuthBST_CercaIndicePerEmail](#) (const char *)
Ricerca l'indice di un utente per email tramite BST.
- int [AuthBST_Altezza](#) (void)
Restituisce l'altezza del BST (per debug).
- int [AuthBST_ContaNodi](#) (void)
Restituisce il numero di nodi nel BST.

4.46.1 Detailed Description

Funzioni per la gestione del database utenti in memoria e su file.

4.46.2 Function Documentation

4.46.2.1 Auth_RegistraUtente()

```
int Auth_RegistraUtente (
    const char * email,
    const char * nome,
    const char * cognome,
    const char * username,
    const char * password,
    const char * pin)
```

Registra un nuovo utente dopo aver generato l'ID.

Precondition

email, username univoci; password conforme ai requisiti; PIN valido.

Postcondition

[Utente](#) aggiunto a dbUtenti e BST; ID incrementato.

Parameters

	<i>email</i>	Email univoca.
	<i>nome</i>	Nome utente.
	<i>cognome</i>	Cognome utente.
	<i>username</i>	Username univoco.
	<i>password</i>	Password in chiaro.
	<i>pin</i>	PIN di recupero.

Returns

1 se registrato, 0 altrimenti.

Registra un nuovo utente dopo aver generato l'ID.

Parameters

	<i>email</i>	Email dell'utente.
	<i>nome</i>	Nome.
	<i>cognome</i>	Cognome.
	<i>username</i>	Username univoco.
	<i>password</i>	Password.
	<i>pin</i>	PIN di recupero.

Returns

1 se registrazione OK, 0 se errore allocazione o PIN duplicato.

4.46.2.2 Auth_RimuoviUtente()

```
void Auth_RimuoviUtente (  
    const char * email)
```

Rimuove un utente da database e BST.

Precondition

email esiste nel database.

Postcondition

Utente rimosso da dbUtenti e BST.

Parameters

	<i>email</i>	Email dell'utente da rimuovere.
--	--------------	---------------------------------

4.46.2.3 AuthBST_Altezza()

```
int AuthBST_Altezza (
    void )
```

Restituisce l'altezza del BST (per debug).

Returns

Altezza dell'albero.

Restituisce l'altezza del BST (per debug).

Returns

Altezza dell'albero.

4.46.2.4 AuthBST_CercaIndicePerEmail()

```
int AuthBST_CercaIndicePerEmail (
    const char * email)
```

Ricerca l'indice di un utente per email tramite BST.

Parameters

<i>email</i>	Email da cercare.
--------------	-------------------

Returns

Indice in dbUtenti, -1 se non trovato.

Ricerca l'indice di un utente per email tramite BST.

Returns

Indice nel dbUtenti, -1 se non trovato.

4.46.2.5 AuthBST_ContaNodi()

```
int AuthBST_ContaNodi (
    void )
```

Restituisce il numero di nodi nel BST.

Returns

Numero di nodi.

4.46.2.6 AuthBST_Ricostruisci()

```
void AuthBST_Ricostruisci (  
    void )
```

Ricostruisce il BST parallelo a partire dall'array dbUtenti.

4.46.2.7 CaricaDB()

```
void CaricaDB (  
    void )
```

Carica il database da file binario.

Precondition

File data/users/users.dat esiste.

Postcondition

dbUtenti popolato; BST ricostruito.

Carica il database da file binario.

4.46.2.8 DB_Alloca()

```
int DB_Alloca (  
    int min_slot)
```

Alloca o rialloca il database per garantire almeno min_slot.

Parameters

<i>min_slot</i>	Numero minimo di slot.
-----------------	------------------------

Returns

1 se OK, 0 se out-of-memory.

Alloca o rialloca il database per garantire almeno min_slot.

Se la capacita' corrente e' insufficiente, raddoppia l'array con realloc finche' non raggiunge min_slot.

Parameters

<i>min_slot</i>	Numero minimo di slot richiesti.
-----------------	----------------------------------

Returns

1 se allocazione OK, 0 se out-of-memory.

4.46.2.9 SalvaDB()

```
void SalvaDB (
    void )
```

Salva il database su file binario con cifratura XOR.

Precondition

DB inizializzato e utenti presenti.

Postcondition

File data/users/users.dat aggiornato; eventuale broadcast LAN eseguito.

Salva il database su file binario con cifratura XOR.

4.47 Nodi BST

Struttura nodo dell'albero binario di ricerca.

Data Structures

- struct [NodoAlbero](#)
Nodo dell'albero BST: contiene un [Utente](#) e puntatori ai figli.

Typedefs

- typedef struct NodoAlbero [NodoAlbero](#)
Nodo dell'albero BST: contiene un [Utente](#) e puntatori ai figli.

4.47.1 Detailed Description

Struttura nodo dell'albero binario di ricerca.

4.47.2 Typedef Documentation

4.47.2.1 NodoAlbero

```
typedef struct NodoAlbero NodoAlbero
```

Nodo dell'albero BST: contiene un [Utente](#) e puntatori ai figli.

4.48 Operazioni BST

Funzioni di gestione dell'albero binario di ricerca.

Functions

- `NodoAlbero * BST_CreaNodo (Utente u)`
Crea un nuovo nodo BST a partire da un `Utente`.
- `NodoAlbero * BST_Inserisci (NodoAlbero *radice, Utente u, int *duplicato)`
Inserisce un utente nell'albero.
- `NodoAlbero * BST_RicercaEmail (NodoAlbero *radice, const char *email)`
Ricerca un utente per email.
- `NodoAlbero * BST_RimuoviEmail (NodoAlbero *radice, const char *email)`
Rimuove il nodo con l'email specificata.
- `void BST_Distruggi (NodoAlbero *radice)`
Libera ricorsivamente tutta la memoria dell'albero.
- `int BST_Inordina (NodoAlbero *radice, Utente *out, int max)`
Visita in-order e copia i puntatori agli Utenti in un array.
- `int BST_Altezza (NodoAlbero *radice)`
Restituisce l'altezza dell'albero.
- `int BST_ContaNodi (NodoAlbero *radice)`
Restituisce il numero di nodi nell'albero.

4.48.1 Detailed Description

Funzioni di gestione dell'albero binario di ricerca.

4.48.2 Function Documentation

4.48.2.1 BST_Altezza()

```
int BST_Altezza (  
    NodoAlbero * radice)
```

Restituisce l'altezza dell'albero.

Precondition

radice valida o NULL.

Returns

Altezza (0 se vuoto).

Altezza (0 se vuoto).

Parameters

	<i>radice</i>	Radice.
--	---------------	---------

4.48.2.2 BST_ContaNodi()

```
int BST_ContaNodi (
    NodoAlbero * radice)
```

Restituisce il numero di nodi nell'albero.

Precondition

radice valida o NULL.

Returns

Numero di nodi.

Restituisce il numero di nodi nell'albero.

Returns

Numero di nodi.

Parameters

	<i>radice</i>	Radice.
--	---------------	---------

4.48.2.3 BST_CreaNodo()

```
NodoAlbero * BST_CreaNodo (
    Utente u)
```

Crea un nuovo nodo BST a partire da un *Utente*.

Precondition

u.email valida.

Postcondition

Nodo allocato e inizializzato, o NULL se out-of-memory.

Parameters

	<i>u</i>	<i>Utente</i> da inserire nel nodo.
--	----------	-------------------------------------

Returns

Puntatore al nodo creato, o NULL.

Crea un nuovo nodo BST a partire da un *Utente*.

Precondition

u.email valida.

Postcondition

Nodo allocato e inizializzato.

Parameters

<i>u</i>	Utente da inserire.
----------	---------------------

Returns

Puntatore al nodo, o NULL.

4.48.2.4 BST_Distruggi()

```
void BST_Distruggi (
    NodoAlbero * radice)
```

Libera ricorsivamente tutta la memoria dell'albero.

Precondition

radice valida o NULL.

Postcondition

Tutti i nodi deallocati.

Parameters

<i>radice</i>	Radice dell'albero da distruggere.
---------------	------------------------------------

Libera ricorsivamente tutta la memoria dell'albero.

Precondition

radice valida o NULL.

Postcondition

Memoria liberata.

Parameters

<i>radice</i>	Radice.
---------------	---------

4.48.2.5 BST_Inordina()

```
int BST_Inordina (
    NodoAlbero * radice,
    Utente * out,
    int max)
```

Visita in-order e copia i puntatori agli Utenti in un array.

Precondition

radice valida; out != NULL; max > 0.

Postcondition

Array out popolato con i nodi in ordine; restituisce il conteggio.

Parameters

	<i>radice</i>	Radice dell'albero.
	<i>out</i>	Array di Utente da riempire.
	<i>max</i>	Dimensione massima di out.

Returns

Numero di elementi scritti.

Visita in-order e copia i puntatori agli Utenti in un array.

Precondition

radice valida; *out* != NULL; *max* > 0.

Returns

Numero di elementi scritti.

Parameters

	<i>radice</i>	Radice.
	<i>out</i>	Array output.
	<i>max</i>	Dimensione massima.

4.48.2.6 BST_Inserisci()

```
NodoAlbero * BST_Inserisci (
    NodoAlbero * radice,
    Utente u,
    int * duplicato)
```

Inserisce un utente nell'albero.

Precondition

radice valida (o NULL); u.email non duplicata.

Postcondition

Albero aggiornato; nuovo nodo inserito se non duplicato.

Parameters

	<i>radice</i>	Radice corrente dell'albero.
	<i>u</i>	Utente da inserire.
	<i>duplicato</i>	Flag impostato a 1 se email duplicata.

Returns

Nuova radice dell'albero.

Precondition

radice valida; u.email non duplicata.

Postcondition

Nuovo nodo inserito se non duplicato.

Parameters

	<i>radice</i>	Radice corrente.
	<i>u</i>	Utente da inserire.
	<i>duplicato</i>	Flag duplicato (1 se email già presente).

Returns

Nuova radice.

4.48.2.7 BST_RicercaEmail()

```
NodoAlbero * BST_RicercaEmail (  
    NodoAlbero * radice,  
    const char * email)
```

Ricerca un utente per email.

Precondition

radice valida; *email* != NULL.

Postcondition

Restituisce nodo trovato o NULL.

Parameters

	<i>radice</i>	Radice dell'albero.
	<i>email</i>	Email da cercare.

Returns

Puntatore al [NodoAlbero](#) trovato, o NULL.

Precondition

radice valida; *email* != NULL.

Returns

Nodo trovato o NULL.

Parameters

	<i>radice</i>	Radice.
	<i>email</i>	Email da cercare.

4.48.2.8 BST_RimuoviEmail()

```
NodoAlbero * BST_RimuoviEmail (
    NodoAlbero * radice,
    const char * email)
```

Rimuove il nodo con l'email specificata.

Precondition

radice valida; email esistente nell'albero.

Postcondition

Nodo rimosso; albero bilanciato.

Parameters

<i>radice</i>	Radice corrente.
<i>email</i>	Email del nodo da rimuovere.

Returns

Nuova radice dell'albero.

Precondition

radice valida; email esistente.

Postcondition

Nodo rimosso; albero aggiornato.

Parameters

<i>radice</i>	Radice.
<i>email</i>	Email da rimuovere.

Returns

Nuova radice.

4.49 Storico Chat

Struttura per la cronologia circolare dei messaggi chat.

Data Structures

- struct [ChatStorico](#)
Cronologia circolare (FIFO) di messaggi.

4.49.1 Detailed Description

Struttura per la cronologia circolare dei messaggi chat.

4.50 API Chat

Funzioni per la gestione della chat (creazione, inserimento, accesso, formattazione).

Functions

- void [Chat_Crea](#) ([ChatStorico](#) *c)
Crea uno storico chat vuoto.
- void [Chat_Aggiungi](#) ([ChatStorico](#) *c, const char *mittente, const char *testo)
Aggiunge un messaggio in coda (FIFO).
- void [Chat_OraCorrente](#) (char *out, size_t out_size)
Genera un timestamp "HH:MM" nel buffer fornito.
- const [MessaggioChat](#) * [Chat_Ottieni](#) (const [ChatStorico](#) *c, int i)
Restituisce il messaggio in posizione i della cronologia.
- void [Chat_Svuota](#) ([ChatStorico](#) *c)
Svuota la cronologia chat.
- void [Chat_FormattaRiga](#) (const [MessaggioChat](#) *m, char *out, size_t out_size)
Formatta un [MessaggioChat](#) in una riga "HH:MM @Mittente: testo".

4.50.1 Detailed Description

Funzioni per la gestione della chat (creazione, inserimento, accesso, formattazione).

4.50.2 Function Documentation

4.50.2.1 Chat_Aggiungi()

```
void Chat_Aggiungi (  
    ChatStorico * c,  
    const char * mittente,  
    const char * testo)
```

Aggiunge un messaggio in coda (FIFO).

Se la cronologia è piena, sovrascrive il messaggio più vecchio.

Precondition

c != NULL; mittente != NULL; testo != NULL.

Postcondition

count aggiornato; head avanzato se pieno.

Parameters

	<i>c</i>	Puntatore a ChatStorico .
	<i>mittente</i>	Username del mittente.
	<i>testo</i>	Testo del messaggio.

Precondition

`c != NULL; mittente != NULL; testo != NULL.`

Postcondition

Messaggio aggiunto; count aggiornato; head ruotato se pieno.

Parameters

	<i>c</i>	ChatStorico .
	<i>mittente</i>	Mittente.
	<i>testo</i>	Testo.

4.50.2.2 Chat_Crea()

```
void Chat_Crea (
    ChatStorico * c)
```

Crea uno storico chat vuoto.

Precondition

`c != NULL.`

Postcondition

Storico inizializzato (count=0, head=0).

Parameters

	<i>c</i>	Puntatore a ChatStorico da inizializzare.
--	----------	---

Precondition

`c != NULL.`

Postcondition

count=0, head=0, buffer azzerato.

Parameters

<code>c</code>	Puntatore a ChatStorico .
----------------	---

4.50.2.3 Chat_FormattaRiga()

```
void Chat_FormattaRiga (
    const MessaggioChat * m,
    char * out,
    size_t out_size)
```

Formatta un [MessaggioChat](#) in una riga "HH:MM @Mittente: testo".

Helper per il rendering delle schermate.

Precondition

`m != NULL; out != NULL; out_size >= 320.`

Postcondition

Riga formattata scritta in out.

Parameters

<code>m</code>	Puntatore a MessaggioChat .
<code>out</code>	Buffer di output (almeno 320 byte).
<code>out_size</code>	Dimensione del buffer.

Formatta un [MessaggioChat](#) in una riga "HH:MM @Mittente: testo".

Precondition

`m != NULL; out != NULL; out_size >= 320.`

Postcondition

Riga formattata in out.

Parameters

<code>m</code>	MessaggioChat .
<code>out</code>	Buffer output.
<code>out_size</code>	Dimensione buffer.

4.50.2.4 Chat_OraCorrente()

```
void Chat_OraCorrente (
    char * out,
    size_t out_size)
```

Genera un timestamp "HH:MM" nel buffer fornito.

Precondition

`out != NULL; out_size >= 6.`

Postcondition

Stringa "HH:MM" scritta in out.

Parameters

	<i>out</i>	Buffer di output (almeno 6 byte).
	<i>out_size</i>	Dimensione del buffer.

Genera un timestamp "HH:MM" nel buffer fornito.

Precondition

`out != NULL; out_size >= 6.`

Postcondition

Stringa "HH:MM" in out.

Parameters

	<i>out</i>	Buffer output.
	<i>out_size</i>	Dimensione buffer.

4.50.2.5 Chat_Ottieni()

```
const MessaggioChat * Chat_Ottieni (
    const ChatStorico * c,
    int i)
```

Restituisce il messaggio in posizione i della cronologia.

Ordine: 0 = più vecchio, count-1 = più recente.

Precondition

`c != NULL; 0 <= i < c->count.`

Returns

Puntatore a [MessaggioChat](#), o NULL se indice non valido.

Parameters

	<i>c</i>	Puntatore a ChatStorico .
	<i>i</i>	Indice del messaggio (0-based).

Restituisce il messaggio in posizione i della cronologia.

Precondition

`c != NULL; 0 <= i < c->count.`

Returns

Puntatore a [MessaggioChat](#), o NULL.

Parameters

<i>c</i>	ChatStorico .
<i>i</i>	Indice.

4.50.2.6 Chat_Svuota()

```
void Chat_Svuota (
    ChatStorico * c)
```

Svuota la cronologia chat.

Precondition

c != NULL.

Postcondition

count=0, head=0.

Parameters

<i>c</i>	Puntatore a ChatStorico .
----------	---

Precondition

c != NULL.

Postcondition

count=0, head=0.

Parameters

<i>c</i>	ChatStorico .
----------	-------------------------------

4.51 Nodi Coda

Struttura nodo per la coda dei turni.

Data Structures

- struct [NodoCoda](#)
Nodo della coda: contiene un [Giocatore](#) e puntatori next/prev.

Typedefs

- typedef struct NodoCoda [NodoCoda](#)
Nodo della coda: contiene un [Giocatore](#) e puntatori next/prev.

4.51.1 Detailed Description

Struttura nodo per la coda dei turni.

4.51.2 Typedef Documentation

4.51.2.1 NodoCoda

```
typedef struct NodoCoda NodoCoda
```

Nodo della coda: contiene un [Giocatore](#) e puntatori next/prev.

4.52 Struttura Coda

Container della coda dei turni di gioco.

Data Structures

- struct [CodaTurni](#)
Coda dei turni: coda doppiamente concatenata con direzione di scorrimento.

Typedefs

- typedef struct CodaTurni [CodaTurni](#)
Coda dei turni: coda doppiamente concatenata con direzione di scorrimento.

4.52.1 Detailed Description

Container della coda dei turni di gioco.

4.52.2 Typedef Documentation

4.52.2.1 CodaTurni

```
typedef struct CodaTurni CodaTurni
```

Coda dei turni: coda doppiamente concatenata con direzione di scorrimento.

`front` è l'inizio della coda (prossimo turno), `rear` è la fine. `verso` memorizza la direzione (+1 orario, -1 antiorario) per Cambio Giro.

4.53 API Queue

Funzioni per la gestione della coda dei turni.

Functions

- `CodaTurni * Coda_Crea` (void)
Crea una coda vuota.
- void `Coda_Distruggi` (`CodaTurni *c`)
Distruttore: libera tutti i nodi e la struttura CodaTurni.
- int `Coda_Enqueue` (`CodaTurni *c`, `Giocatore g`)
Inserisce un giocatore in coda (rear).
- int `Coda_Dequeue` (`CodaTurni *c`, `Giocatore *out`)
Rimuove il giocatore in testa (front).
- int `Coda_Front` (const `CodaTurni *c`, `Giocatore *out`)
Restituisce il giocatore in testa SENZA rimuoverlo.
- int `Coda_Rear` (const `CodaTurni *c`, `Giocatore *out`)
Restituisce il giocatore in fondo (rear) SENZA rimuoverlo.
- `NodoCoda * Coda_ProxNodo` (const `CodaTurni *c`, `NodoCoda *n`)
Restituisce il nodo che segue n nella direzione corrente.
- int `Coda_Vuota` (const `CodaTurni *c`)
Verifica se la coda è vuota.
- int `Coda_Dimensione` (const `CodaTurni *c`)
Restituisce il numero di giocatori in coda.
- void `Coda_InvertiVerso` (`CodaTurni *c`)
Inverte il verso di scorrimento (carta Cambio Giro).
- `NodoCoda * Coda_CercaPerIndice` (const `CodaTurni *c`, int `indice`)
Cerca il nodo all'indice specificato (0 = front).
- int `Coda_CopiaPerIndice` (const `CodaTurni *c`, int `indice`, `Giocatore *out`)
Copia l'i-esimo giocatore in out (0 = front).

4.53.1 Detailed Description

Funzioni per la gestione della coda dei turni.

4.53.2 Function Documentation

4.53.2.1 Coda_CercaPerIndice()

```
NodoCoda * Coda_CercaPerIndice (
    const CodaTurni * c,
    int indice)
```

Cerca il nodo all'indice specificato (0 = front).

Precondition

`c != NULL`; `indice` nel range `[0, size-1]`.

Returns

Puntatore a `NodoCoda`, o `NULL` se fuori range.

Parameters

	<i>c</i>	Puntatore a CodaTurni .
	<i>indice</i>	Indice del nodo (0-based).

Cerca il nodo all'indice specificato (0 = front).

Precondition

c != NULL; indice valido.

Returns

[NodoCoda](#), o NULL.

Parameters

	<i>c</i>	CodaTurni .
	<i>indice</i>	Indice (0-based).

4.53.2.2 Coda_CopiaPerIndice()

```
int Coda_CopiaPerIndice (
    const CodaTurni * c,
    int indice,
    Giocatore * out)
```

Copia l'i-esimo giocatore in out (0 = front).

Precondition

c != NULL; indice nel range [0, size-1]; out != NULL.

Postcondition

[Giocatore](#) copiato in out.

Parameters

	<i>c</i>	Puntatore a CodaTurni .
	<i>indice</i>	Indice del giocatore (0-based).
	<i>out</i>	Puntatore a Giocatore per il risultato.

Returns

1 se successo, 0 altrimenti.

Copia l'i-esimo giocatore in out (0 = front).

Precondition

c != NULL; out != NULL; indice valido.

Postcondition

[Giocatore](#) copiato.

Returns

1 se successo, 0 altrimenti.

Parameters

	<i>c</i>	CodaTurni .
	<i>indice</i>	Indice (0-based).
	<i>out</i>	Puntatore output.

4.53.2.3 Coda_Crea()

```
CodaTurni * Coda_Crea (
    void )
```

Crea una coda vuota.

Postcondition

Coda allocata e inizializzata (size=0, verso=1).

Returns

Puntatore a [CodaTurni](#), o NULL se out-of-memory.

Postcondition

Coda allocata; verso=1 (orario); size=0.

Returns

Puntatore a [CodaTurni](#), o NULL.

4.53.2.4 Coda_Dequeue()

```
int Coda_Dequeue (
    CodaTurni * c,
    Giocatore * out)
```

Rimuove il giocatore in testa (front).

Precondition

c != NULL.

Postcondition

[Giocatore](#) rimosso; size decrementato; dati scritti in out (se non NULL).

Parameters

	<i>c</i>	Puntatore a CodaTurni .
	<i>out</i>	Puntatore a Giocatore per ricevere l'elemento rimosso (può essere NULL).

Returns

1 se successo, 0 se coda vuota.

Rimuove il giocatore in testa (front).

Precondition

`c != NULL`.

Postcondition

[Giocatore](#) rimosso; size decrementato.

Parameters

	<code>c</code>	CodaTurni .
	<code>out</code>	Puntatore output (puo' essere NULL).

Returns

1 se successo, 0 se vuota.

4.53.2.5 Coda_Dimensione()

```
int Coda_Dimensione (
    const CodaTurni * c)
```

Restituisce il numero di giocatori in coda.

Precondition

`c != NULL`.

Returns

Numero di elementi.

Parameters

	<code>c</code>	Puntatore a CodaTurni .
--	----------------	---

Restituisce il numero di giocatori in coda.

Returns

Dimensione, 0 se NULL.

Parameters

	<i>c</i>	CodaTurni .
--	----------	-----------------------------

4.53.2.6 Coda_Distruggi()

```
void Coda_Distruggi (  
    CodaTurni * c)
```

Distruttore: libera tutti i nodi e la struttura [CodaTurni](#).

Precondition

c != NULL.

Postcondition

Memoria della coda completamente liberata.

Parameters

	<i>c</i>	Puntatore a CodaTurni .
--	----------	---

Distruttore: libera tutti i nodi e la struttura [CodaTurni](#).

Precondition

c != NULL.

Postcondition

Memoria liberata.

Parameters

	<i>c</i>	CodaTurni .
--	----------	-----------------------------

4.53.2.7 Coda_Enqueue()

```
int Coda_Enqueue (  
    CodaTurni * c,  
    Giocatore g)
```

Inserisce un giocatore in coda (rear).

Precondition

c != NULL.

Postcondition

[Giocatore](#) aggiunto in coda; size incrementato.

Parameters

<i>c</i>	Puntatore a CodaTurni .
<i>g</i>	Giocatore da inserire.

Returns

1 se successo, 0 altrimenti.

Inserisce un giocatore in coda (rear).

Precondition

c != NULL.

Postcondition

[Giocatore](#) aggiunto; size incrementato.

Parameters

<i>c</i>	CodaTurni .
<i>g</i>	Giocatore .

Returns

1 se successo, 0 altrimenti.

4.53.2.8 Coda_Front()

```
int Coda_Front (
    const CodaTurni * c,
    Giocatore * out)
```

Restituisce il giocatore in testa SENZA rimuoverlo.

Precondition

c != NULL e coda non vuota.

Postcondition

[Giocatore](#) in front scritto in *out*.

Parameters

<i>c</i>	Puntatore a CodaTurni .
<i>out</i>	Puntatore a Giocatore per ricevere l'elemento in testa.

Returns

1 se successo, 0 se coda vuota.

Restituisce il giocatore in testa SENZA rimuoverlo.

Precondition

c != NULL.

Returns

1 se successo, 0 se vuota.

Parameters

	<i>c</i>	CodaTurni .
	<i>out</i>	Puntatore output.

4.53.2.9 Coda_InvertiVerso()

```
void Coda_InvertiVerso (
    CodaTurni * c)
```

Inverte il verso di scorrimento (carta Cambio Giro).

Operazione $O(1)$: inverte il segno di `verso`.

Precondition

`c != NULL`.

Postcondition

`verso *= -1`.

Parameters

	<i>c</i>	Puntatore a CodaTurni .
--	----------	---

Precondition

`c != NULL`.

Postcondition

`verso` invertito; `front/rear` scambiati.

Parameters

	<i>c</i>	CodaTurni .
--	----------	-----------------------------

4.53.2.10 Coda_ProxNodo()

```
NodoCoda * Coda_ProxNodo (
    const CodaTurni * c,
    NodoCoda * n)
```

Restituisce il nodo che segue `n` nella direzione corrente.

Utilizza `verso` e i puntatori `prev/next` per determinare il prossimo turno in $O(1)$.

Precondition

`c != NULL`; `n != NULL`.

Returns

Puntatore al [NodoCoda](#) successivo, o `NULL` se non trovato.

Parameters

<i>c</i>	Puntatore a CodaTurni .
<i>n</i>	Nodo corrente.

Restituisce il nodo che segue *n* nella direzione corrente.

Precondition

c != NULL; *n* != NULL.

Returns

Prossimo nodo (con wrap-around), o NULL.

Parameters

<i>c</i>	CodaTurni .
<i>n</i>	Nodo corrente.

4.53.2.11 Coda_Rear()

```
int Coda_Rear (
    const CodaTurni * c,
    Giocatore * out)
```

Restituisce il giocatore in fondo (rear) SENZA rimuoverlo.

Precondition

c != NULL e coda non vuota.

Postcondition

[Giocatore](#) in rear scritto in *out*.

Parameters

<i>c</i>	Puntatore a CodaTurni .
<i>out</i>	Puntatore a Giocatore .

Returns

1 se successo, 0 se coda vuota.

Restituisce il giocatore in fondo (rear) SENZA rimuoverlo.

Precondition

c != NULL.

Returns

1 se successo, 0 se vuota.

Parameters

	<i>c</i>	CodaTurni .
	<i>out</i>	Puntatore output.

4.53.2.12 Coda_Vuota()

```
int Coda_Vuota (
    const CodaTurni * c)
```

Verifica se la coda è vuota.

Precondition

c != NULL.

Returns

1 se vuota, 0 altrimenti.

Parameters

	<i>c</i>	Puntatore a CodaTurni .
--	----------	---

Verifica se la coda è vuota.

Returns

1 se vuota, 0 altrimenti.

Parameters

	<i>c</i>	CodaTurni .
--	----------	-----------------------------

4.54 Nodi Pila

Struttura nodo per la pila LIFO.

Data Structures

- struct [NodoPila](#)
Nodo della pila: contiene una carta e puntatore al next.

Typedefs

- typedef struct [NodoPila](#) [NodoPila](#)
Nodo della pila: contiene una carta e puntatore al next.

4.54.1 Detailed Description

Struttura nodo per la pila LIFO.

4.54.2 Typedef Documentation

4.54.2.1 NodoPila

```
typedef struct NodoPila NodoPila
```

Nodo della pila: contiene una carta e puntatore al next.

4.55 Struttura Pila

Container della pila LIFO.

Data Structures

- struct [Pila](#)
[Pila](#) LIFO: top punta all'ultimo inserito, size è la cardinalità.

Typedefs

- typedef struct Pila [Pila](#)
[Pila](#) LIFO: top punta all'ultimo inserito, size è la cardinalità.

4.55.1 Detailed Description

Container della pila LIFO.

4.55.2 Typedef Documentation

4.55.2.1 Pila

```
typedef struct Pila Pila
```

[Pila](#) LIFO: top punta all'ultimo inserito, size è la cardinalità.

4.56 API Stack

Funzioni per la gestione della pila (push, pop, top, distruzione).

Functions

- `Pila * Pila_Crea` (void)
Crea una pila vuota.
- void `Pila_Distruggi` (`Pila *p`)
Distruttore: libera tutti i nodi e la struttura `Pila`.
- int `Pila_Push` (`Pila *p`, `Carta c`)
Inserisce una carta in cima alla pila.
- int `Pila_Pop` (`Pila *p`, `Carta *out`)
Rimuove la carta in cima alla pila.
- int `Pila_Top` (const `Pila *p`, `Carta *out`)
Restituisce la carta in cima SENZA rimuoverla.
- int `Pila_Vuota` (const `Pila *p`)
Verifica se la pila è vuota.
- int `Pila_Dimensione` (const `Pila *p`)
Restituisce la dimensione corrente della pila.
- void `Pila_Svuota` (`Pila *p`)
Svuota la pila lasciando la struttura allocata.

4.56.1 Detailed Description

Funzioni per la gestione della pila (push, pop, top, distruzione).

4.56.2 Function Documentation

4.56.2.1 `Pila_Crea()`

```
Pila * Pila_Crea (  
    void )
```

Crea una pila vuota.

Postcondition

`Pila` allocata e inizializzata (size=0).

Returns

Puntatore a `Pila`, o NULL se out-of-memory.

Postcondition

`Pila` allocata; top=NULL; size=0.

Returns

Puntatore a `Pila`, o NULL.

4.56.2.2 Pila_Dimensione()

```
int Pila_Dimensione (  
    const Pila * p)
```

Restituisce la dimensione corrente della pila.

Precondition

$p \neq \text{NULL}$.

Returns

Numero di elementi nella pila.

Parameters

p	Puntatore a Pila .
-----	------------------------------------

Restituisce la dimensione corrente della pila.

Returns

Dimensione, 0 se NULL.

Parameters

p	Pila .
-----	------------------------

4.56.2.3 Pila_Distruggi()

```
void Pila_Distruggi (  
    Pila * p)
```

Distruttore: libera tutti i nodi e la struttura [Pila](#).

Precondition

$p \neq \text{NULL}$.

Postcondition

Tutta la memoria della pila liberata.

Parameters

p	Puntatore a Pila .
-----	------------------------------------

Distruttore: libera tutti i nodi e la struttura [Pila](#).

Precondition

$p \neq \text{NULL}$.

Postcondition

Memoria liberata.

Parameters

	<i>p</i>	Pila.
--	----------	-------

4.56.2.4 Pila_Pop()

```
int Pila_Pop (
    Pila * p,
    Carta * out)
```

Rimuove la carta in cima alla pila.

Precondition

p != NULL.

Postcondition

top aggiornato; size decrementato; carta rimossa scritta in out (se non NULL).

Parameters

	<i>p</i>	Puntatore a Pila.
	<i>out</i>	Puntatore a Carta per ricevere l'elemento rimosso (può essere NULL).

Returns

1 se successo, 0 se pila vuota.

Rimuove la carta in cima alla pila.

Precondition

p != NULL.

Postcondition

Carta rimossa; size decrementato.

Parameters

	<i>p</i>	Pila.
	<i>out</i>	Puntatore output (puo' essere NULL).

Returns

1 se successo, 0 se vuota.

4.56.2.5 Pila_Push()

```
int Pila_Push (
    Pila * p,
    Carta c)
```

Inserisce una carta in cima alla pila.

Precondition

$p \neq \text{NULL}$.

Postcondition

top aggiornato; size incrementato.

Parameters

p	Puntatore a Pila .
c	Carta da inserire.

Returns

1 se successo, 0 altrimenti.

Precondition

$p \neq \text{NULL}$.

Postcondition

[Carta](#) in cima; size incrementato.

Parameters

p	Pila .
c	Carta .

Returns

1 se successo, 0 altrimenti.

4.56.2.6 Pila_Svuota()

```
void Pila_Svuota (
    Pila * p)
```

Svuota la pila lasciando la struttura allocata.

Precondition

$p \neq \text{NULL}$.

Postcondition

top=NULL, size=0, tutti i nodi liberati.

Parameters

<code>p</code>	Puntatore a Pila .
----------------	------------------------------------

Svuota la pila lasciando la struttura allocata.

Precondition

`p != NULL`.

Postcondition

`top=NULL; size=0`.

Parameters

<code>p</code>	Pila .
----------------	------------------------

4.56.2.7 Pila_Top()

```
int Pila_Top (
    const Pila * p,
    Carta * out)
```

Restituisce la carta in cima SENZA rimuoverla.

Precondition

`p != NULL`.

Postcondition

[Carta](#) in `top` scritta in `out`.

Parameters

<code>p</code>	Puntatore a Pila .
<code>out</code>	Puntatore a Carta per ricevere l'elemento in cima.

Returns

1 se successo, 0 se pila vuota.

Restituisce la carta in cima SENZA rimuoverla.

Precondition

`p != NULL`.

Returns

1 se successo, 0 se vuota.

Parameters

p	Pila .
<i>out</i>	Puntatore output.

4.56.2.8 Pila_Vuota()

```
int Pila_Vuota (
    const Pila * p)
```

Verifica se la pila è vuota.

Precondition

$p \neq \text{NULL}$.

Returns

1 se vuota, 0 altrimenti.

Parameters

p	Puntatore a Pila .
-----	------------------------------------

Verifica se la pila è vuota.

Returns

1 se vuota, 0 altrimenti.

Parameters

p	Pila .
-----	------------------------

4.57 Azioni Bot

Enumerazione delle possibili decisioni del bot.

Enumerations

- enum [DecisioneBot](#) {
[AZIONE_GIOCA_JOLLY_PIU4](#) , [AZIONE_GIOCA_PIU2](#) , [AZIONE_GIOCA_CAMBIO_COLORE](#) ,
[AZIONE_GIOCA_CAMBIO_GIRO](#) ,
[AZIONE_GIOCA_SALTA](#) , [AZIONE_GIOCA_NUMERO_ALTO](#) , [AZIONE_GIOCA_NUMERO_BASSO](#) ,
[AZIONE_PESCA](#) }
Gioca Jolly+4 (priorità max).

4.57.1 Detailed Description

Enumerazione delle possibili decisioni del bot.

4.57.2 Enumeration Type Documentation

4.57.2.1 DecisioneBot

```
enum DecisioneBot
```

Gioca Jolly+4 (priorità max).

Enumerator

	AZIONE_GIOCA_JOLLY_PIU4
	AZIONE_GIOCA_PIU2
	AZIONE_GIOCA_CAMBIO_COLORE
	AZIONE_GIOCA_CAMBIO_GIRO
	AZIONE_GIOCA_SALTA
	AZIONE_GIOCA_NUMERO_ALTO
	AZIONE_GIOCA_NUMERO_BASSO
	AZIONE_PESCA

4.58 Nodo Albero Decisionale

Nodo dell'albero decisionale binario.

Data Structures

- struct [NodoDecisione](#)
Nodo albero: azione, priorità, figli si/no.

Typedefs

- typedef struct NodoDecisione [NodoDecisione](#)
Nodo albero: azione, priorità, figli si/no.

4.58.1 Detailed Description

Nodo dell'albero decisionale binario.

4.58.2 Typedef Documentation

4.58.2.1 NodoDecisione

```
typedef struct NodoDecisione NodoDecisione
```

Nodo albero: azione, priorità, figli si/no.

4.59 API Bot AI

Funzioni per costruire, valutare e distruggere l'albero decisionale.

Functions

- `NodoDecisione * BotAI_CostruisciAlbero` (void)
Costruisce l'albero decisionale per l'IA del bot.
- void `BotAI_DistruggiAlbero` (`NodoDecisione *radice`)
Distrugge l'albero decisionale.
- `DecisioneBot BotAI_Decidi` (`NodoDecisione *radice`, `StatoGioco *gioco`, int id_giocatore)
Valuta l'albero decisionale e restituisce l'azione.
- int `BotAI_TrovaCarta` (`StatoGioco *gioco`, int id_giocatore, `DecisioneBot` azione)
Trova l'indice di una carta da giocare per la decisione presa.
- `Colore BotAI_ScegliColore` (`StatoGioco *gioco`, int id_giocatore)
Sceglie il colore migliore per un Jolly in base alla mano.

4.59.1 Detailed Description

Funzioni per costruire, valutare e distruggere l'albero decisionale.

4.59.2 Function Documentation

4.59.2.1 BotAI_CostruisciAlbero()

```
NodoDecisione * BotAI_CostruisciAlbero (  
    void )
```

Costruisce l'albero decisionale per l'IA del bot.

Postcondition

Albero allocato con priorità definite.

Returns

Radice dell'albero decisionale.

Costruisce l'albero decisionale per l'IA del bot.

Postcondition

Radice allocata con priorit  definita.

Returns

Puntatore alla radice, o NULL.

4.59.2.2 BotAI_Decidi()

```
DecisioneBot BotAI_Decidi (  
    NodoDecisione * radice,  
    StatoGioco * gioco,  
    int id_giocatore)
```

Valuta l'albero decisionale e restituisce l'azione.

Precondition

radice e gioco validi; id_giocatore valido.

Returns

DecisioneBot presa.

Parameters

	<i>radice</i>	Radice dell'albero.
	<i>gioco</i>	Stato del gioco.
	<i>id_giocatore</i>	ID del giocatore bot.

Valuta l'albero decisionale e restituisce l'azione.

Precondition

radice e gioco validi; id_giocatore valido.

Returns

DecisioneBot presa.

Parameters

	<i>radice</i>	Radice albero.
	<i>gioco</i>	Stato di gioco.
	<i>id_giocatore</i>	Indice del bot.

4.59.2.3 BotAI_DistruggiAlbero()

```
void BotAI_DistruggiAlbero (  
    NodoDecisione * radice)
```

Distrugge l'albero decisionale.

Precondition

radice != NULL.

Postcondition

Memoria dell'albero completamente liberata.

Parameters

	<i>radice</i>	Radice dell'albero.
--	---------------	---------------------

Distrugge l'albero decisionale.

Precondition

radice valida o NULL.

Postcondition

Memoria liberata.

Parameters

	<i>radice</i>	Radice.
--	---------------	---------

4.59.2.4 BotAI_ScegliColore()

```
Colore BotAI_ScegliColore (
    StatoGioco * gioco,
    int id_giocatore)
```

Sceglie il colore migliore per un Jolly in base alla mano.

Precondition

gioco valido.

Returns

Colore con più carte nella mano del bot.

Parameters

	<i>gioco</i>	Stato del gioco.
	<i>id_giocatore</i>	ID del giocatore bot.

Sceglie il colore migliore per un Jolly in base alla mano.

Precondition

gioco valido; id_giocatore valido.

Returns

Colore con piu' carte.

Parameters

	<i>gioco</i>	Stato di gioco.
	<i>id_giocatore</i>	Indice del bot.

4.59.2.5 BotAI_TrovaCarta()

```
int BotAI_TrovaCarta (  
    StatoGioco * gioco,  
    int id_giocatore,  
    DecisioneBot azione)
```

Trova l'indice di una carta da giocare per la decisione presa.

Precondition

gioco valido; [DecisioneBot](#) valida.

Returns

Indice carta, o -1 se non disponibile.

Parameters

	<i>gioco</i>	Stato del gioco.
	<i>id_giocatore</i>	ID del giocatore.
	<i>azione</i>	Decisione da eseguire.

Trova l'indice di una carta da giocare per la decisione presa.

Precondition

gioco valido; azione valida.

Returns

Indice carta, -1 se non trovata.

Parameters

	<i>gioco</i>	Stato di gioco.
	<i>id_giocatore</i>	Indice del bot.
	<i>azione</i>	DecisioneBot .

4.60 Ruoli di Rete

Enumerazione dei ruoli multiplayer.

Enumerations

- enum `NetworkRole` { `NET_OFFLINE` , `NET_HOST` , `NET_CLIENT` }
Giocatore offline (multigiocatore disattivato).

4.60.1 Detailed Description

Enumerazione dei ruoli multiplayer.

4.60.2 Enumeration Type Documentation

4.60.2.1 NetworkRole

enum `NetworkRole`

Giocatore offline (multigiocatore disattivato).

Enumerator

	NET_OFFLINE
	NET_HOST
	NET_CLIENT

4.61 Tipi di Pacchetto

Enumerazione di tutti i tipi di pacchetti scambiati in rete.

Enumerations

- enum `PacketType` {
`PACKET_JOIN` , `PACKET_START_GAME` , `PACKET_GAME_STATE` , `PACKET_MOVE` ,
`PACKET_DRAW` , `PACKET_CHAT` , `PACKET_DISCONNECT` , `PACKET_JOIN_GAME` ,
`PACKET_PLAYER_HAND` , `PACKET_COLOR_CHOICE` , `PACKET_CREATE_SESSION` , `PACKET_JOIN_SESSION`
, `PACKET_SESSION_LIST` , `PACKET_SESSION_STATE` , `PACKET_HEARTBEAT` , `PACKET_PLAYER_READY`
, `PACKET_GAME_FINISHED` , `PACKET_ADMIN_USER_SYNC` , `PACKET_ADMIN_DB_SYNC` , `PACKET_ADMIN_DB_SYNC_D`
, `PACKET_HOST_DISCONNECT` , `PACKET_STATS_SYNC` , `PACKET_JOIN_REJECTED` , `PACKET_PLAY_AGAIN`
, `PACKET_HOST_RETURN_LOBBY` }

4.61.1 Detailed Description

Enumerazione di tutti i tipi di pacchetti scambiati in rete.

4.61.2 Enumeration Type Documentation

4.61.2.1 PacketType

enum `PacketType`

Enumerator

	PACKET_JOIN
	PACKET_START_GAME
	PACKET_GAME_STATE
	PACKET_MOVE
	PACKET_DRAW
	PACKET_CHAT
	PACKET_DISCONNECT
	PACKET_JOIN_GAME
	PACKET_PLAYER_HAND
	PACKET_COLOR_CHOICE
	PACKET_CREATE_SESSION
	PACKET_JOIN_SESSION
	PACKET_SESSION_LIST
	PACKET_SESSION_STATE
	PACKET_HEARTBEAT
	PACKET_PLAYER_READY
	PACKET_GAME_FINISHED
	PACKET_ADMIN_USER_SYNC
	PACKET_ADMIN_DB_SYNC
	PACKET_ADMIN_DB_SYNC_DONE
	PACKET_HOST_DISCONNECT
	PACKET_STATS_SYNC
	PACKET_JOIN_REJECTED
	PACKET_PLAY_AGAIN
	PACKET_HOST_RETURN_LOBBY

4.62 Strutture Pacchetti

Strutture dati per ogni tipo di pacchetto multiplayer.

Data Structures

- struct [PacketMove](#)
Pacchetto mossa: giocatore, indice carta, colore scelto.
- struct [PacketDraw](#)
Pacchetto pesca: giocatore e quantità.
- struct [PacketGameState](#)
Pacchetto stato di gioco completo.
- struct [PacketColorChoice](#)
Pacchetto scelta colore.
- struct [PacketChat](#)
Pacchetto chat.
- struct [PacketJoinGame](#)
Pacchetto join game.
- struct [PacketJoinRejected](#)
Pacchetto join rifiutato.
- struct [PacketPlayerHand](#)
Pacchetto mano giocatore.

- struct [PacketCreateSession](#)
Pacchetto creazione sessione.
- struct [PacketJoinSession](#)
Pacchetto join sessione.
- struct [PacketSessionList](#)
Pacchetto lista sessioni.
- struct [PacketSessionState](#)
Pacchetto stato sessione multiplayer.
- struct [PacketPlayerReady](#)
Pacchetto ready/play again.
- struct [PacketGameFinished](#)
Pacchetto fine partita.
- struct [PacketPlayAgain](#)
Pacchetto play again (con nome giocatore per fix bug).
- struct [PacketHeartbeat](#)
Pacchetto heartbeat per keep-alive.
- struct [PacketHostReturnLobby](#)
Pacchetto host return lobby.
- struct [AdminUserSyncData](#)
- struct [AdminUserAction](#)
- struct [AdminDBSyncPacket](#)
- struct [StatsSyncPacket](#)
StatsSyncPacket - Sincronizzazione live statistiche dopo una partita.
- struct [NetPacket](#)

Macros

- #define [MAX_SERVER_CLIENTS](#) 4

Functions

- int [GetLocalIPAddress](#) (char *outbuf, int buflen)
Ottiene l'indirizzo IP locale del dispositivo.
- int [IsPlayerConnected](#) (int player_id)
- void [RiassegnaSocket](#) (int old_id, int new_id, int is_reconnect, int partita_iniziata)
- void [BroadcastHostDisconnect](#) (void)
Invia un pacchetto di notifica disconnessione host a tutti i client.
- void [SendPlayerReady](#) (int ready)
- void [SendPlayAgain](#) (void)
Invia un pacchetto PACKET_PLAY_AGAIN all'host.
- void [BroadcastHostReturnLobby](#) (void)
Broadcasta PACKET_HOST_RETURN_LOBBY a tutti i client.
- int [ConnettiTramiteCodiceStanza](#) (const char *codice, int portaBase)
Connette a una stanza tramite codice stanza (UDP discovery).
- int [InitNetwork](#) (void)
- void [ShiftNetworkSockets](#) (int removed_index, int num_players)
Ricomatta l'array di socket client dopo una disconnessione.
- void [CloseNetwork](#) (void)
- int [HostGame](#) (int port)
- int [JoinGame](#) (const char *ip_address, int port)

- void `UpdateNetwork` (`StatoGioco *gioco`)
- void `HandleReceivedPacket` (`NetPacket *packet`, `StatoGioco *gioco`)
HandleReceivedPacket - Dispatcher principale dei pacchetti.
- void `GestisciDisconnessione` (`StatoGioco *gioco`)
Gestisce la disconnessione di un giocatore (host o client).
- void `IpToCode` (`const char *ip`, `char *code`)
Converte un indirizzo IP in codice esadecimale.
- int `CodeToIp` (`const char *code`, `char *ip`)
Converte un codice esadecimale in indirizzo IP.
- void `BroadcastStatsSync` (`const StatsSyncPacket *stats`)
Invia un pacchetto di sincronizzazione statistiche COMPLETA a tutti i client.
- void `SendStatsSync` (`const StatsSyncPacket *stats`)
Invia un pacchetto di sincronizzazione statistiche COMPLETA all'host (client).

Variables

- int `PacketMove::player_id`
- int `PacketMove::card_index`
- `Colore` `PacketMove::colore_scelto`
- int `PacketDraw::player_id`
- int `PacketDraw::count`
- `StatoGioco` `PacketGameState::stato_gioco`
- int `PacketColorChoice::player_id`
- `Colore` `PacketColorChoice::colore`
- int `PacketChat::player_id`
- char `PacketChat::message` [128]
- char `PacketJoinGame::nome` [50]
- char `PacketJoinRejected::motivo` [100]
- int `PacketPlayerHand::player_id`
- `Carta` `PacketPlayerHand::carte` [108]
- int `PacketPlayerHand::num_carte`
- char `PacketCreateSession::session_id` [16]
- char `PacketCreateSession::host_name` [30]
- int `PacketCreateSession::num_players`
- char `PacketJoinSession::session_id` [16]
- char `PacketJoinSession::player_name` [30]
- int `PacketSessionList::session_count`
- char `PacketSessionList::sessions` [10][16]
- char `PacketSessionState::session_id` [16]
- int `PacketSessionState::num_players`
- char `PacketSessionState::player_names` [4][30]
- int `PacketSessionState::player_ready` [4]
- int `PacketSessionState::game_started`
- int `PacketPlayerReady::player_id`
- int `PacketPlayerReady::is_ready`
- int `PacketPlayerReady::want_play_again`
- int `PacketGameFinished::winning_player_id`
- char `PacketGameFinished::winner_name` [30]
- int `PacketPlayAgain::player_id`
- float `PacketPlayAgain::timestamp`
- int `PacketPlayAgain::play_again`
- char `PacketPlayAgain::player_name` [30]
- int `PacketHeartbeat::player_id`

- float PacketHeartbeat::timestamp
- int PacketHostReturnLobby::host_ready
- char AdminUserSyncData::email [100]
- char AdminUserSyncData::nome [30]
- char AdminUserSyncData::cognome [30]
- char AdminUserSyncData::username [30]
- char AdminUserSyncData::password [30]
- char AdminUserSyncData::pin_recupero [7]
- int AdminUserSyncData::partite_giocate
- int AdminUserSyncData::vittorie_giocatore
- int AdminUserSyncData::vittorie_bot
- AdminUserSyncData AdminUserAction::user
- int AdminUserAction::action
- int AdminUserAction::target_id
- int AdminDBSyncPacket::user_index
- int AdminDBSyncPacket::total_users
- AdminUserSyncData AdminDBSyncPacket::user
- char StatsSyncPacket::username [30]
- int StatsSyncPacket::partite_giocate
- int StatsSyncPacket::vittorie_giocatore
- int StatsSyncPacket::vittorie_bot
- float StatsSyncPacket::durata_ultima_partita
- char StatsSyncPacket::modalita_ultima [20]
- char StatsSyncPacket::esito_ultima [15]
- int StatsSyncPacket::posizione_ultima
- PartitaRegistrata StatsSyncPacket::storico_partite [STORICO_SYNC_COUNT]
- int StatsSyncPacket::storico_count
- PacketType NetPacket::type
- int NetPacket::sender_id
- PacketMove NetPacket::move
- PacketDraw NetPacket::draw
- PacketGameState NetPacket::game_state
- PacketColorChoice NetPacket::color_choice
- PacketChat NetPacket::chat
- PacketJoinGame NetPacket::join_game
- PacketJoinRejected NetPacket::join_rejected
- PacketPlayerHand NetPacket::player_hand
- PacketCreateSession NetPacket::create_session
- PacketJoinSession NetPacket::join_session
- PacketSessionList NetPacket::session_list
- PacketSessionState NetPacket::session_state
- PacketPlayerReady NetPacket::player_ready
- PacketGameFinished NetPacket::game_finished
- PacketPlayAgain NetPacket::play_again
- PacketHostReturnLobby NetPacket::host_return_lobby
- PacketHeartbeat NetPacket::heartbeat
- AdminUserAction NetPacket::admin_action
- AdminDBSyncPacket NetPacket::admin_db_packet
- StatsSyncPacket NetPacket::stats_sync
- char NetPacket::raw [512]

- union {
 - PacketMove NetPacket::move
 - PacketDraw NetPacket::draw
 - PacketGameState NetPacket::game_state
 - PacketColorChoice NetPacket::color_choice
 - PacketChat NetPacket::chat
 - PacketJoinGame NetPacket::join_game
 - PacketJoinRejected NetPacket::join_rejected
 - PacketPlayerHand NetPacket::player_hand
 - PacketCreateSession NetPacket::create_session
 - PacketJoinSession NetPacket::join_session
 - PacketSessionList NetPacket::session_list
 - PacketSessionState NetPacket::session_state
 - PacketPlayerReady NetPacket::player_ready
 - PacketGameFinished NetPacket::game_finished
 - PacketPlayAgain NetPacket::play_again
 - PacketHostReturnLobby NetPacket::host_return_lobby
 - PacketHeartbeat NetPacket::heartbeat
 - AdminUserAction NetPacket::admin_action
 - AdminDBSyncPacket NetPacket::admin_db_packet
 - StatsSyncPacket NetPacket::stats_sync
 - char NetPacket::raw [512]
- } NetPacket::data
- int network_start_game_flag
- NetworkRole currentRole
- int join_rejected
- char reject_message [100]
- int isConnected
- int network_pending
- int network_game_initialized
- int network_client_joined
- int player_id
- char current_session_id [16]
- char session_id [16]
- int local_player_id
- int host_in_prelobby
- SOCKET clientSockets [MAX_SERVER_CLIENTS]
- char connectedClientUsernames [4][30]

Username dei client TCP connessi all'host (per admin panel).

4.62.1 Detailed Description

Strutture dati per ogni tipo di pacchetto multiplayer.

4.62.2 Macro Definition Documentation

4.62.2.1 MAX_SERVER_CLIENTS

```
#define MAX_SERVER_CLIENTS 4
```

4.62.3 Function Documentation

4.62.3.1 BroadcastHostDisconnect()

```
void BroadcastHostDisconnect (
    void )
```

Invia un pacchetto di notifica disconnessione host a tutti i client.

Usato quando l'host chiude la stanza (lobby o fine partita). I client ricevono l'avviso e vengono reindirizzati al menu principale.

4.62.3.2 BroadcastHostReturnLobby()

```
void BroadcastHostReturnLobby (
    void )
```

Broadcasta PACKET_HOST_RETURN_LOBBY a tutti i client.

L'host segnala che e' tornato in pre-lobby dopo aver premuto "Gioca Ancora". I client possono cosi' decidere se unirsi o andarsene.

4.62.3.3 BroadcastStatsSync()

```
void BroadcastStatsSync (
    const StatsSyncPacket * stats)
```

Invia un pacchetto di sincronizzazione statistiche COMPLETA a tutti i client.

Usato dopo ogni partita per aggiornare le statistiche di tutti i giocatori.

Garantisce che i client connessi via TCP vedano TUTTI i dettagli delle partite storiche, non solo i contatori numerici. Popola l'intero array storico_partite[] nel pacchetto StatsSync.

Parameters

<i>stats</i>	Puntatore al pacchetto StatsSync (gia' compilato con username e contatori).
--------------	---

4.62.3.4 CloseNetwork()

```
void CloseNetwork (
    void )
```

4.62.3.5 CodeToIp()

```
int CodeToIp (
    const char * code,
    char * ip)
```

Converte un codice esadecimale in indirizzo IP.

Formato: "C0A80164" -> "192.168.1.100"

Parameters

<i>code</i>	Codice esadecimale (8 caratteri).
<i>ip</i>	Buffer di output per l'IP.

Returns

1 se conversione riuscita, 0 se formato non valido.

4.62.3.6 ConnettiTramiteCodiceStanza()

```
int ConnettiTramiteCodiceStanza (  
    const char * codice,  
    int portaBase)
```

Connette a una stanza tramite codice stanza (UDP discovery).

Procedura:

1. Crea socket UDP broadcast
2. Invia "REQ_CODE:<codice>" in broadcast sulla porta base+1
3. Attende risposta "CODE_FOUND" con timeout 1 secondo
4. Se risposta ricevuta: chiama [JoinGame\(\)](#) con IP del server

Parameters

<i>codice</i>	Codice stanza da cercare (8 caratteri hex).
<i>portaBase</i>	Porta TCP del gioco (27015).

Returns

1 se connessione riuscita, 0 altrimenti.

4.62.3.7 GestisciDisconnessione()

```
void GestisciDisconnessione (  
    StatoGioco * gioco)
```

Gestisce la disconnessione di un giocatore (host o client).

Comportamento:

- HOST: Sostituisce il giocatore disconnesso con un BOT, notifica tutti, broadcasta nuovo stato.
- CLIENT: Imposta stato "__HOST_DISCONNECT__", salva statistiche solo se partita non era finita (evita duplicati).

Bug fissati:

- Bug 1: NON salva statistiche se partita già finita (duplicati)
- Bug 4: Partita interrotta non registra in classifica

Parameters

<i>gioco</i>	Puntatore allo stato di gioco.
--------------	--------------------------------

4.62.3.8 GetLocalIPAddress()

```
int GetLocalIPAddress (
    char * outbuf,
    int buflen)
```

Ottiene l'indirizzo IP locale del dispositivo.

Utilizza metodo multiplo per robustezza:

1. gethostname + gethostbyname (scorre TUTTI gli indirizzi) Filtra: loopback (127.0.0.1), APIPA (169.254.x.x), VirtualBox (10.0.2.x), VPN (tunnel interfaces). Accetta: 192.168.x.x, 10.x.x.x, 172.16-31.x.x
2. Fallback: UDP connect a 8.8.8.8:53 (Google DNS) Non richiede header speciali, usa getsockname() per IP.

Bug fissato: prendeva solo h_addr_list[0] che su Windows spesso e' 127.0.0.1. Ora scorre tutta la lista.

Parameters

<i>outbuf</i>	Buffer di output per l'IP trovato.
<i>buflen</i>	Dimensione del buffer output.

Returns

1 se trovato IP valido, 0 altrimenti.

4.62.3.9 HandleReceivedPacket()

```
void HandleReceivedPacket (
    NetPacket * packet,
    StatoGioco * gioco)
```

HandleReceivedPacket - Dispatcher principale dei pacchetti.

Classifica il tipo di pacchetto e delega la gestione al modulo specializzato appropriato.

4.62.3.10 HostGame()

```
int HostGame (
    int port)
```

4.62.3.11 InitNetwork()

```
int InitNetwork (
    void )
```

4.62.3.12 IpToCode()

```
void IpToCode (
    const char * ip,
    char * code)
```

Converte un indirizzo IP in codice esadecimale.

Formato: "192.168.1.100" -> "C0A80164" Usato per generare codici stanza leggibili dai client.

Parameters

	<i>ip</i>	Indirizzo IP in formato stringa.
	<i>code</i>	Buffer di output per il codice (min 9 byte).

4.62.3.13 IsPlayerConnected()

```
int IsPlayerConnected (
    int player_id)
```

4.62.3.14 JoinGame()

```
int JoinGame (
    const char * ip_address,
    int port)
```

4.62.3.15 RiassegnaSocket()

```
void RiassegnaSocket (
    int old_id,
    int new_id,
    int is_reconnect,
    int partita_iniziata)
```

4.62.3.16 SendPlayAgain()

```
void SendPlayAgain (
    void )
```

Invia un pacchetto PACKET_PLAY_AGAIN all'host.

Il client segnala all'host che vuole giocare ancora (tornare in pre-lobby). Invia anche il nome del giocatore nel campo `player_name[]` per permettere all'host di ricreare lo slot correttamente dopo aver pulito la lobby.

4.62.3.17 SendPlayerReady()

```
void SendPlayerReady (
    int ready)
```

4.62.3.18 SendStatsSync()

```
void SendStatsSync (
    const StatsSyncPacket * stats)
```

Invia un pacchetto di sincronizzazione statistiche COMPLETA all'host (client).

Usato quando il client salva le proprie statistiche dopo una partita.

Garantisce che l'host (e tutti gli altri client tramite broadcast) ricevano TUTTI i dettagli delle partite, non solo i contatori numerici.

Popola l'intero array `storico_partite[]` nel pacchetto StatsSync.

4.62.3.19 ShiftNetworkSockets()

```
void ShiftNetworkSockets (
    int removed_index,
    int num_players)
```

Ricompatta l'array di socket client dopo una disconnessione.

Sposta tutti gli socket dopo l'indice rimosso di una posizione indietro per mantenere gli slot contigui.

Parameters

<code>removed_index</code>	Indice dello slot rimosso.
<code>num_players</code>	Numero totale giocatori (dopo rimozione).

4.62.3.20 UpdateNetwork()

```
void UpdateNetwork (
    StatoGioco * gioco)
```

4.62.4 Variable Documentation

4.62.4.1 action

```
int AdminUserAction::action
```

4.62.4.2 [] [1/2]

```
AdminUserAction { ... } ::admin_action
```

4.62.4.3 admin_action [2/2]

```
AdminUserAction NetPacket::admin_action
```

4.62.4.4 [] [1/2]

```
AdminDBSyncPacket { ... } ::admin_db_packet
```

4.62.4.5 admin_db_packet [2/2]

```
AdminDBSyncPacket NetPacket::admin_db_packet
```

4.62.4.6 card_index

```
int PacketMove::card_index
```

4.62.4.7 carte

```
Carta PacketPlayerHand::carte[108]
```

4.62.4.8 [] [1/2]

```
PacketChat { ... } ::chat
```

4.62.4.9 chat [2/2]

```
PacketChat NetPacket::chat
```

4.62.4.10 clientSockets

```
SOCKET clientSockets[MAX_SERVER_CLIENTS] [extern]
```

4.62.4.11 cognome

```
char AdminUserSyncData::cognome[30]
```

4.62.4.12 [] [1/2]

```
PacketColorChoice { ... } ::color_choice
```

4.62.4.13 color_choice [2/2]

```
PacketColorChoice NetPacket::color_choice
```

4.62.4.14 colore

```
Colore PacketColorChoice::colore
```

4.62.4.15 colore_scelto

```
Colore PacketMove::colore_scelto
```

4.62.4.16 connectedClientUsernames

```
char connectedClientUsernames[4][30] [extern]
```

Username dei client TCP connessi all'host (per admin panel).

NOTA: Indice 0 corrisponde allo slot 1 del client.

4.62.4.17 count

```
int PacketDraw::count
```

4.62.4.18 [] [1/2]

```
PacketCreateSession { ... } ::create_session
```

4.62.4.19 create_session [2/2]

```
PacketCreateSession NetPacket::create_session
```

4.62.4.20 current_session_id

```
char current_session_id[16] [extern]
```

4.62.4.21 currentRole

```
NetworkRole currentRole [extern]
```

4.62.4.22 [union]

```
union { ... } NetPacket::data
```

4.62.4.23 [] [1/2]

```
PacketDraw { ... } ::draw
```

4.62.4.24 draw [2/2]

```
PacketDraw NetPacket::draw
```

4.62.4.25 durata_ultima_partita

```
float StatsSyncPacket::durata_ultima_partita
```

4.62.4.26 email

```
char AdminUserSyncData::email[100]
```

4.62.4.27 esito_ultima

```
char StatsSyncPacket::esito_ultima[15]
```

4.62.4.28 [] [1/2]

```
PacketGameFinished { ... } ::game_finished
```

4.62.4.29 game_finished [2/2]

```
PacketGameFinished NetPacket::game_finished
```

4.62.4.30 game_started

```
int PacketSessionState::game_started
```

4.62.4.31 [] [1/2]

```
PacketGameState { ... } ::game_state
```

4.62.4.32 game_state [2/2]

```
PacketGameState NetPacket::game_state
```

4.62.4.33 [] [1/2]

```
PacketHeartbeat { ... } ::heartbeat
```

4.62.4.34 heartbeat [2/2]

```
PacketHeartbeat NetPacket::heartbeat
```

4.62.4.35 host_in_prelobby

```
int host_in_prelobby [extern]
```

4.62.4.36 host_name

```
char PacketCreateSession::host_name[30]
```

4.62.4.37 host_ready

```
int PacketHostReturnLobby::host_ready
```

4.62.4.38 [] [1/2]

```
PacketHostReturnLobby { ... } ::host_return_lobby
```

4.62.4.39 host_return_lobby [2/2]

```
PacketHostReturnLobby NetPacket::host_return_lobby
```

4.62.4.40 is_ready

```
int PacketPlayerReady::is_ready
```

4.62.4.41 isConnected

```
int isConnected [extern]
```

4.62.4.42 [] [1/2]

```
PacketJoinGame { ... } ::join_game
```

4.62.4.43 join_game [2/2]

```
PacketJoinGame NetPacket::join_game
```

4.62.4.44 join_rejected [1/3]

```
int join_rejected [extern]
```

4.62.4.45 [] [2/3]

```
PacketJoinRejected { ... } ::join_rejected
```

4.62.4.46 join_rejected [3/3]

```
PacketJoinRejected NetPacket::join_rejected
```

4.62.4.47 [] [1/2]

```
PacketJoinSession { ... } ::join_session
```

4.62.4.48 join_session [2/2]

```
PacketJoinSession NetPacket::join_session
```

4.62.4.49 local_player_id

```
int local_player_id [extern]
```

4.62.4.50 message

```
char PacketChat::message[128]
```

4.62.4.51 modalita_ultima

```
char StatsSyncPacket::modalita_ultima[20]
```

4.62.4.52 motivo

```
char PacketJoinRejected::motivo[100]
```

4.62.4.53 [] [1/2]

```
PacketMove { ... } ::move
```

4.62.4.54 move [2/2]

```
PacketMove NetPacket::move
```

4.62.4.55 network_client_joined

```
int network_client_joined [extern]
```

4.62.4.56 network_game_initialized

```
int network_game_initialized [extern]
```

4.62.4.57 network_pending

```
int network_pending [extern]
```

4.62.4.58 network_start_game_flag

```
int network_start_game_flag [extern]
```

4.62.4.59 nome [1/2]

```
char AdminUserSyncData::nome[30]
```

4.62.4.60 nome [2/2]

```
char PacketJoinGame::nome[50]
```

4.62.4.61 num_carte

```
int PacketPlayerHand::num_carte
```

4.62.4.62 num_players [1/2]

```
int PacketCreateSession::num_players
```

4.62.4.63 num_players [2/2]

```
int PacketSessionState::num_players
```

4.62.4.64 partite_giocate [1/2]

```
int AdminUserSyncData::partite_giocate
```

4.62.4.65 partite_giocate [2/2]

```
int StatsSyncPacket::partite_giocate
```

4.62.4.66 password

```
char AdminUserSyncData::password[30]
```


4.62.4.67 pin_recupero

```
char AdminUserSyncData::pin_recupero[7]
```

4.62.4.68 [] [1/3]

```
PacketPlayAgain { ... } ::play_again
```

4.62.4.69 play_again [2/3]

```
PacketPlayAgain NetPacket::play_again
```

4.62.4.70 play_again [3/3]

```
int PacketPlayAgain::play_again
```

4.62.4.71 [] [1/2]

```
PacketPlayerHand { ... } ::player_hand
```

4.62.4.72 player_hand [2/2]

```
PacketPlayerHand NetPacket::player_hand
```

4.62.4.73 player_id [1/9]

```
int PacketChat::player_id
```

4.62.4.74 player_id [2/9]

```
int PacketColorChoice::player_id
```

4.62.4.75 player_id [3/9]

```
int PacketDraw::player_id
```

4.62.4.76 player_id [4/9]

```
int PacketHeartbeat::player_id
```

4.62.4.77 player_id [5/9]

```
int PacketMove::player_id
```

4.62.4.78 player_id [6/9]

```
int PacketPlayAgain::player_id
```

4.62.4.79 player_id [7/9]

```
int PacketPlayerHand::player_id
```

4.62.4.80 player_id [8/9]

```
int PacketPlayerReady::player_id
```

4.62.4.81 player_id [9/9]

```
int player_id [extern]
```

4.62.4.82 player_name [1/2]

```
char PacketJoinSession::player_name[30]
```

4.62.4.83 player_name [2/2]

```
char PacketPlayAgain::player_name[30]
```

4.62.4.84 player_names

```
char PacketSessionState::player_names[4][30]
```

4.62.4.85 [] [1/3]

```
PacketPlayerReady { ... } ::player_ready
```

4.62.4.86 player_ready [2/3]

```
PacketPlayerReady NetPacket::player_ready
```

4.62.4.87 player_ready [3/3]

```
int PacketSessionState::player_ready[4]
```

4.62.4.88 posizione_ultima

```
int StatsSyncPacket::posizione_ultima
```

4.62.4.89 [] [1/2]

```
char { ... } ::raw[512]
```

4.62.4.90 raw [2/2]

```
char NetPacket::raw[512]
```

4.62.4.91 reject_message

```
char reject_message[100] [extern]
```

4.62.4.92 sender_id

```
int NetPacket::sender_id
```

4.62.4.93 session_count

```
int PacketSessionList::session_count
```

4.62.4.94 session_id [1/4]

```
char PacketCreateSession::session_id[16]
```

4.62.4.95 session_id [2/4]

```
char PacketJoinSession::session_id[16]
```

4.62.4.96 session_id [3/4]

```
char PacketSessionState::session_id[16]
```

4.62.4.97 session_id [4/4]

```
char session_id[16] [extern]
```

4.62.4.98 [] [1/2]

```
PacketSessionList { ... } ::session_list
```

4.62.4.99 session_list [2/2]

```
PacketSessionList NetPacket::session_list
```

4.62.4.100 [] [1/2]

```
PacketSessionState { ... } ::session_state
```

4.62.4.101 session_state [2/2]

```
PacketSessionState NetPacket::session_state
```

4.62.4.102 sessions

```
char PacketSessionList::sessions[10][16]
```

4.62.4.103 stato_gioco

```
StatoGioco PacketGameState::stato_gioco
```

4.62.4.104 [] [1/2]

```
StatsSyncPacket { ... } ::stats_sync
```

4.62.4.105 stats_sync [2/2]

```
StatsSyncPacket NetPacket::stats_sync
```

4.62.4.106 storico_count

```
int StatsSyncPacket::storico_count
```

4.62.4.107 storico_partite

`PartitaRegistrata StatsSyncPacket::storico_partite[STORICO_SYNC_COUNT]`

4.62.4.108 target_id

`int AdminUserAction::target_id`

4.62.4.109 timestamp [1/2]

`float PacketHeartbeat::timestamp`

4.62.4.110 timestamp [2/2]

`float PacketPlayAgain::timestamp`

4.62.4.111 total_users

`int AdminDBSyncPacket::total_users`

4.62.4.112 type

`PacketType NetPacket::type`

4.62.4.113 user [1/2]

`AdminUserSyncData AdminDBSyncPacket::user`

4.62.4.114 user [2/2]

`AdminUserSyncData AdminUserAction::user`

4.62.4.115 user_index

`int AdminDBSyncPacket::user_index`

4.62.4.116 username [1/2]

`char AdminUserSyncData::username[30]`

4.62.4.117 username [2/2]

```
char StatsSyncPacket::username[30]
```

4.62.4.118 vittorie_bot [1/2]

```
int AdminUserSyncData::vittorie_bot
```

4.62.4.119 vittorie_bot [2/2]

```
int StatsSyncPacket::vittorie_bot
```

4.62.4.120 vittorie_giocatore [1/2]

```
int AdminUserSyncData::vittorie_giocatore
```

4.62.4.121 vittorie_giocatore [2/2]

```
int StatsSyncPacket::vittorie_giocatore
```

4.62.4.122 want_play_again

```
int PacketPlayerReady::want_play_again
```

4.62.4.123 winner_name

```
char PacketGameFinished::winner_name[30]
```

4.62.4.124 winning_player_id

```
int PacketGameFinished::winning_player_id
```

4.63 Admin Sync

Funzioni per la sincronizzazione dell'admin panel.

Functions

- void [SendAdminUserSync](#) (const [AdminUserSyncData](#) *userData)
Invia i dati utente del client all'host (al collegamento).
- void [BroadcastAdminUserAction](#) (const [AdminUserAction](#) *action)
Invia l'azione admin (modifica/elimina/crea) a tutti i client.
- void [BroadcastAdminDB](#) (void)
Invia il [DatabaseUtenti](#) aggiornato a tutti i client.

4.63.1 Detailed Description

Funzioni per la sincronizzazione dell'admin panel.

4.63.2 Function Documentation

4.63.2.1 BroadcastAdminDB()

```
void BroadcastAdminDB (  
    void )
```

Invia il [DatabaseUtenti](#) aggiornato a tutti i client.

Precondition

DB valido.

4.63.2.2 BroadcastAdminUserAction()

```
void BroadcastAdminUserAction (  
    const AdminUserAction * action)
```

Invia l'azione admin (modifica/elimina/crea) a tutti i client.

Precondition

action != NULL.

Parameters

	<i>action</i>	Azione da eseguire.
--	---------------	---------------------

Invia l'azione admin (modifica/elimina/crea) a tutti i client.

Usato quando l'admin modifica un utente nel pannello di controllo.

4.63.2.3 SendAdminUserSync()

```
void SendAdminUserSync (  
    const AdminUserSyncData * userData)
```

Invia i dati utente del client all'host (al collegamento).

Precondition

userData != NULL.

Parameters

<code>userData</code>	Dati dell'utente.
-----------------------	-------------------

Invia i dati utente del client all'host (al collegamento).

Viene chiamato quando un client si collega alla stanza, per far conoscere i propri dati anagrafici all'host (admin).

4.64 Stati Sessione

Enumerazione degli stati possibili di una sessione.

Enumerations

- enum `SessionState` {
 `SESSION_WAITING` , `SESSION_STARTING` , `SESSION_IN_PROGRESS` , `SESSION_FINISHED` ,
 `SESSION_CLOSED` }

Sessione in attesa di giocatori.

4.64.1 Detailed Description

Enumerazione degli stati possibili di una sessione.

4.64.2 Enumeration Type Documentation

4.64.2.1 SessionState

enum `SessionState`

Sessione in attesa di giocatori.

Enumerator

	<code>SESSION_WAITING</code>
	<code>SESSION_STARTING</code>
	<code>SESSION_IN_PROGRESS</code>
	<code>SESSION_FINISHED</code>
	<code>SESSION_CLOSED</code>

4.65 Giocatore Sessione

Informazioni di un giocatore in una sessione.

Data Structures

- struct [SessionPlayer](#)
Informazioni giocatore nella sessione.

4.65.1 Detailed Description

Informazioni di un giocatore in una sessione.

4.66 Sessione di Gioco

Struttura di una sessione di gioco.

Data Structures

- struct [GameSession](#)
Sessione di gioco multiplayer.

4.66.1 Detailed Description

Struttura di una sessione di gioco.

4.67 Struttura Server Manager

Container per tutte le sessioni attive.

Data Structures

- struct [ServerManager](#)
Gestore di tutte le sessioni di gioco attive.

4.67.1 Detailed Description

Container per tutte le sessioni attive.

4.68 API Server Manager

Funzioni per la gestione del server manager e delle sessioni.

Functions

- void [InitServerManager](#) (void)
Inizializza il server manager.
- void [UpdateServerManager](#) (float dt)
Aggiorna lo stato di tutte le sessioni.
- void [ShutdownServerManager](#) (void)
Chiude il server manager e libera risorse.
- [GameSession](#) * [CreateSession](#) (const char *host_name, int host_id)
Crea una nuova sessione di gioco.
- [GameSession](#) * [GetSession](#) (const char *sess_id)
Cerca una sessione per ID.
- [GameSession](#) * [GetSessionByPlayerId](#) (int player_id)
Cerca una sessione contenente un giocatore specifico.
- int [AddPlayerToSession](#) ([GameSession](#) *session, int player_id, const char *player_name, int is_bot)
Aggiunge un giocatore a una sessione.
- void [RemovePlayerFromSession](#) ([GameSession](#) *session, int player_id)
Rimuove un giocatore da una sessione.
- void [StartSession](#) ([GameSession](#) *session)
Avvia il conteggio per l'inizio della partita.
- void [UpdateSessionLogic](#) ([GameSession](#) *session, float dt)
Aggiorna la logica di una sessione (bot AI, timer).
- void [CloseSession](#) ([GameSession](#) *session)
Chiude una sessione.
- int [IsValidMove](#) ([GameSession](#) *session, int player_id, int card_index)
Verifica se una mossa e' valida (lato server).
- int [ProcessMove](#) ([GameSession](#) *session, int player_id, int card_index, [Colore](#) color_choice)
Processa una mossa (lato server).
- int [ProcessDraw](#) ([GameSession](#) *session, int player_id, int draw_count)
Processa una pescata (lato server).
- int [ProcessColorChoice](#) ([GameSession](#) *session, int player_id, [Colore](#) color)
Processa una scelta colore (lato server).
- char * [GenerateSessionId](#) (void)
Genera un ID di sessione univoco (stringa casuale).

4.68.1 Detailed Description

Funzioni per la gestione del server manager e delle sessioni.

4.68.2 Function Documentation

4.68.2.1 AddPlayerToSession()

```
int AddPlayerToSession (
    GameSession * session,
    int player_id,
    const char * player_name,
    int is_bot)
```

Aggiunge un giocatore a una sessione.

Precondition

session valida; numero giocatori < MAX_PLAYERS_PER_SESSION.

Postcondition

[Giocatore](#) aggiunto; num_players incrementato.

Parameters

	<i>session</i>	Sessione target.
	<i>player_id</i>	ID del giocatore.
	<i>player_name</i>	Nome del giocatore.
	<i>is_bot</i>	1 se bot, 0 se umano.

Returns

1 se aggiunto, 0 altrimenti.

4.68.2.2 CloseSession()

```
void CloseSession (  
    GameSession * session)
```

Chiude una sessione.

Precondition

session valida.

Postcondition

state = SESSION_CLOSED; risorse liberate.

Parameters

	<i>session</i>	Sessione da chiudere.
--	----------------	-----------------------

4.68.2.3 CreateSession()

```
GameSession * CreateSession (  
    const char * host_name,  
    int host_id)
```

Crea una nuova sessione di gioco.

Precondition

host_name != NULL.

Postcondition

Sessione aggiunta a serverManager.

Parameters

<i>host_name</i>	Nome del host.
<i>host_id</i>	ID del giocatore host.

Returns

Puntatore alla sessione creata, o NULL se errore.

Crea una nuova sessione di gioco.

Alloca una sessione nell'array statico, genera un ID univoco, inizializza lo stato di gioco con il mazzo e distribuisce 7 carte all'host.

Parameters

<i>host_name</i>	Nome dell'host (giocatore 0).
<i>host_id</i>	ID del giocatore host.

Returns

Puntatore alla sessione creata, NULL se errore (max sessioni raggiunto).

4.68.2.4 GenerateSessionId()

```
char * GenerateSessionId (
    void )
```

Genera un ID di sessione univoco (stringa casuale).

Returns

Puntatore a stringa allocata (da liberare con free).

BUG 3: Il vecchio codice generava solo 4 caratteri casuali, che potevano collidere quando piu' dispositivi sulla stessa rete creavano stanze contemporaneamente.

CORREZIONE: Codice di 8 caratteri che combina:

- Parte bassa del timestamp (millisecondi / 10) per unicità temporale
- Un carattere casuale per ridurre ulteriormente la probabilità di collisione. Questo garantisce unicità anche su dispositivi sulla stessa rete.

4.68.2.5 GetSession()

```
GameSession * GetSession (
    const char * sess_id)
```

Cerca una sessione per ID.

Precondition

`sess_id != NULL`.

Returns

Puntatore a [GameSession](#), o NULL se non trovata.

Parameters

<code>sess_id</code>	ID della sessione.
----------------------	--------------------

4.68.2.6 GetSessionByPlayerId()

```
GameSession * GetSessionByPlayerId (  
    int player_id)
```

Cerca una sessione contenente un giocatore specifico.

Returns

Puntatore a [GameSession](#), o NULL se non trovata.

Parameters

<code>player_id</code>	ID del giocatore.
------------------------	-------------------

4.68.2.7 InitServerManager()

```
void InitServerManager (  
    void )
```

Inizializza il server manager.

Postcondition

Tutte le sessioni resettate; num_active_sessions=0.

Inizializza il server manager.

Resetta a zero tutte le sessioni e inizializza il seed per la generazione di ID sessione casuali.

4.68.2.8 IsValidMove()

```
int IsValidMove (  
    GameSession * session,  
    int player_id,  
    int card_index)
```

Verifica se una mossa e' valida (lato server).

Precondition

session valida; player_id nella sessione.

Returns

1 se mossa valida, 0 altrimenti.

Parameters

	<i>session</i>	Sessione di gioco.
	<i>player_id</i>	ID del giocatore.
	<i>card_index</i>	Indice della carta da giocare.

4.68.2.9 ProcessColorChoice()

```
int ProcessColorChoice (  
    GameSession * session,  
    int player_id,  
    Colore color)
```

Processa una scelta colore (lato server).

Precondition

session valida; player_id nella sessione.

Postcondition

colore_attivo aggiornato.

Parameters

	<i>session</i>	Sessione.
	<i>player_id</i>	ID del giocatore.
	<i>color</i>	Colore scelto.

Returns

1 se successo, 0 altrimenti.

4.68.2.10 ProcessDraw()

```
int ProcessDraw (  
    GameSession * session,  
    int player_id,  
    int draw_count)
```

Processa una pescata (lato server).

Precondition

session valida; player_id nella sessione.

Postcondition

Carte pescate aggiunte alla mano.

Parameters

	<i>session</i>	Sessione.
	<i>player_id</i>	ID del giocatore.
	<i>draw_count</i>	Numero di carte da pescare.

Returns

1 se successo, 0 altrimenti.

4.68.2.11 ProcessMove()

```
int ProcessMove (  
    GameSession * session,  
    int player_id,  
    int card_index,  
    Colore color_choice)
```

Processa una mossa (lato server).

Precondition

session valida; mossa valida.

Postcondition

Stato gioco aggiornato.

Parameters

	<i>session</i>	Sessione.
	<i>player_id</i>	ID del giocatore.
	<i>card_index</i>	Indice carta.
	<i>color_choice</i>	Colore scelto (per jolly).

Returns

1 se successo, 0 altrimenti.

4.68.2.12 RemovePlayerFromSession()

```
void RemovePlayerFromSession (  
    GameSession * session,  
    int player_id)
```

Rimuove un giocatore da una sessione.

Precondition

session valida; player_id presente.

Postcondition

Giocatore rimosso; num_players decrementato.

Parameters

	<i>session</i>	Sessione.
	<i>player_id</i>	ID del giocatore.

4.68.2.13 ShutdownServerManager()

```
void ShutdownServerManager (
    void )
```

Chiude il server manager e libera risorse.

Precondition

serverManager inizializzato.

Postcondition

Tutte le sessioni chiuse; memoria liberata.

Chiude il server manager e libera risorse.

Chiamato alla chiusura dell'applicazione o quando si esce dalla modalita' multiplayer.

4.68.2.14 StartSession()

```
void StartSession (
    GameSession * session)
```

Avvia il conteggio per l'inizio della partita.

Precondition

session valida e in SESSION_WAITING con abbastanza giocatori.

Postcondition

state = SESSION_STARTING; startCountdown impostato.

Parameters

	<i>session</i>	Sessione da avviare.
--	----------------	----------------------

4.68.2.15 UpdateServerManager()

```
void UpdateServerManager (
    float dt)
```

Aggiorna lo stato di tutte le sessioni.

Precondition

serverManager inizializzato.

Parameters

	<i>dt</i>	Delta time.
--	-----------	-------------

4.68.2.16 UpdateSessionLogic()

```
void UpdateSessionLogic (  
    GameSession * session,  
    float dt)
```

Aggiorna la logica di una sessione (bot AI, timer).

Precondition

session valida.

Parameters

	<i>session</i>	Sessione.
	<i>dt</i>	Delta time.

Chapter 5

Data Structure Documentation

5.1 AdminDBSyncPacket Struct Reference

```
#include <network.h>
```

Data Fields

- int [user_index](#)
- int [total_users](#)
- [AdminUserSyncData](#) user

The documentation for this struct was generated from the following file:

- lib/socket/network/[network.h](#)

5.2 AdminUserAction Struct Reference

```
#include <network.h>
```

Data Fields

- [AdminUserSyncData](#) user
- int [action](#)
- int [target_id](#)

The documentation for this struct was generated from the following file:

- lib/socket/network/[network.h](#)

5.3 AdminUserSyncData Struct Reference

```
#include <network.h>
```

Data Fields

- char [email](#) [100]
- char [nome](#) [30]
- char [cognome](#) [30]
- char [username](#) [30]
- char [password](#) [30]
- char [pin_recupero](#) [7]
- int [partite_giocate](#)
- int [vittorie_giocatore](#)
- int [vittorie_bot](#)

The documentation for this struct was generated from the following file:

- lib/socket/network/[network.h](#)

5.4 AudioStream Struct Reference

```
#include <raylib.h>
```

Data Fields

- [rAudioBuffer](#) * [buffer](#)
- [rAudioProcessor](#) * [processor](#)
- unsigned int [sampleRate](#)
- unsigned int [sampleSize](#)
- unsigned int [channels](#)

5.4.1 Field Documentation

5.4.1.1 buffer

```
rAudioBuffer* AudioStream::buffer
```

5.4.1.2 channels

```
unsigned int AudioStream::channels
```

5.4.1.3 processor

```
rAudioProcessor* AudioStream::processor
```

5.4.1.4 sampleRate

```
unsigned int AudioStream::sampleRate
```

5.4.1.5 sampleSize

```
unsigned int AudioStream::sampleSize
```

The documentation for this struct was generated from the following file:

- lib/raylib/include/[raylib.h](#)

5.5 AutomationEvent Struct Reference

```
#include <raylib.h>
```

Data Fields

- unsigned int [frame](#)
- unsigned int [type](#)
- int [params](#) [4]

5.5.1 Field Documentation

5.5.1.1 frame

```
unsigned int AutomationEvent::frame
```

5.5.1.2 params

```
int AutomationEvent::params[4]
```

5.5.1.3 type

```
unsigned int AutomationEvent::type
```

The documentation for this struct was generated from the following file:

- lib/raylib/include/[raylib.h](#)

5.6 AutomationEventList Struct Reference

```
#include <raylib.h>
```

Data Fields

- unsigned int [capacity](#)
- unsigned int [count](#)
- [AutomationEvent](#) * [events](#)

5.6.1 Field Documentation

5.6.1.1 capacity

```
unsigned int AutomationEventList::capacity
```

5.6.1.2 count

```
unsigned int AutomationEventList::count
```

5.6.1.3 events

```
AutomationEvent* AutomationEventList::events
```

The documentation for this struct was generated from the following file:

- [lib/raylib/include/raylib.h](#)

5.7 BoneInfo Struct Reference

```
#include <raylib.h>
```

Data Fields

- char [name](#) [32]
- int [parent](#)

5.7.1 Field Documentation

5.7.1.1 name

```
char BoneInfo::name[32]
```

5.7.1.2 parent

```
int BoneInfo::parent
```

The documentation for this struct was generated from the following file:

- [lib/raylib/include/raylib.h](#)

5.8 BoundingBox Struct Reference

```
#include <raylib.h>
```

Data Fields

- [Vector3 min](#)
- [Vector3 max](#)

5.8.1 Field Documentation

5.8.1.1 max

[Vector3](#) BoundingBox::max

5.8.1.2 min

[Vector3](#) BoundingBox::min

The documentation for this struct was generated from the following file:

- lib/raylib/include/[raylib.h](#)

5.9 Camera2D Struct Reference

```
#include <raylib.h>
```

Data Fields

- [Vector2 offset](#)
- [Vector2 target](#)
- float [rotation](#)
- float [zoom](#)

5.9.1 Field Documentation

5.9.1.1 offset

[Vector2](#) Camera2D::offset

5.9.1.2 rotation

float Camera2D::rotation

5.9.1.3 target

`Vector2` Camera2D::target

5.9.1.4 zoom

`float` Camera2D::zoom

The documentation for this struct was generated from the following file:

- lib/raylib/include/raylib.h

5.10 Camera3D Struct Reference

```
#include <raylib.h>
```

Data Fields

- `Vector3` position
- `Vector3` target
- `Vector3` up
- `float` fovy
- `int` projection

5.10.1 Field Documentation

5.10.1.1 fovy

`float` Camera3D::fovy

5.10.1.2 position

`Vector3` Camera3D::position

5.10.1.3 projection

`int` Camera3D::projection

5.10.1.4 target

`Vector3` Camera3D::target

5.10.1.5 up

[Vector3](#) Camera3D::up

The documentation for this struct was generated from the following file:

- lib/raylib/include/[raylib.h](#)

5.11 CampoInput Struct Reference

Struttura per la gestione dei campi di input (area + ID).

```
#include <auth_shared.h>
```

Data Fields

- [Rectangle](#) area
- int [campo_id](#)

5.11.1 Detailed Description

Struttura per la gestione dei campi di input (area + ID).

5.11.2 Field Documentation

5.11.2.1 area

[Rectangle](#) CampoInput::area

5.11.2.2 campo_id

```
int CampoInput::campo_id
```

The documentation for this struct was generated from the following file:

- lib/screens/auth/[auth_shared.h](#)

5.12 Carta Struct Reference

Rappresenta una singola carta UNO.

```
#include <data_structures.h>
```

Data Fields

- [Colore](#) colore
- [TipoCarta](#) tipo
- [Rectangle](#) area
- [Vector2](#) posOriginale
- int [isTrascinata](#)

5.12.1 Detailed Description

Rappresenta una singola carta UNO.

Contiene il colore, il tipo, l'area di rendering ([Rectangle](#)), la posizione originale (per animazioni di trascinamento) e il flag di trascinamento.

5.12.2 Field Documentation

5.12.2.1 area

[Rectangle](#) Carta::area

5.12.2.2 colore

[Colore](#) Carta::colore

5.12.2.3 isTrascinata

int Carta::isTrascinata

5.12.2.4 posOriginale

[Vector2](#) Carta::posOriginale

5.12.2.5 tipo

[TipoCarta](#) Carta::tipo

The documentation for this struct was generated from the following file:

- [lib/data_structures.h](#)

5.13 ChatStorico Struct Reference

Cronologia circolare (FIFO) di messaggi.

```
#include <chat.h>
```

Data Fields

- [MessaggioChat buffer](#) [[CHAT_STORICO_MAX](#)]
Buffer circolare di messaggi.
- [int count](#)
Numero di messaggi attualmente memorizzati ($\leq$ [CHAT_STORICO_MAX](#)).
- [int head](#)
Indice del messaggio più vecchio nella FIFO.

5.13.1 Detailed Description

Cronologia circolare (FIFO) di messaggi.

Quando si supera la capacità, il messaggio più vecchio viene sovrascritto.

5.13.2 Field Documentation

5.13.2.1 buffer

```
MessaggioChat ChatStorico::buffer[CHAT_STORICO_MAX]
```

Buffer circolare di messaggi.

5.13.2.2 count

```
int ChatStorico::count
```

Numero di messaggi attualmente memorizzati ($\leq$ [CHAT_STORICO_MAX](#)).

5.13.2.3 head

```
int ChatStorico::head
```

Indice del messaggio più vecchio nella FIFO.

The documentation for this struct was generated from the following file:

- [lib/data_structures/chat.h](#)

5.14 ClassificaRecord Struct Reference

Record per la classifica visualizzata a fine partita.

```
#include <data_structures.h>
```

Data Fields

- char [nome](#) [30]
- int [num_carte](#)
- int [posizione](#)
- int [is_bot](#)

5.14.1 Detailed Description

Record per la classifica visualizzata a fine partita.

Contiene il nome del giocatore, il numero di carte rimaste e la posizione finale (1 = vincitore).

5.14.2 Field Documentation

5.14.2.1 [is_bot](#)

```
int ClassificaRecord::is_bot
```

5.14.2.2 [nome](#)

```
char ClassificaRecord::nome[30]
```

5.14.2.3 [num_carte](#)

```
int ClassificaRecord::num_carte
```

5.14.2.4 [posizione](#)

```
int ClassificaRecord::posizione
```

The documentation for this struct was generated from the following file:

- lib/[data_structures.h](#)

5.15 CodaTurni Struct Reference

Coda dei turni: coda doppiamente concatenata con direzione di scorrimento.

```
#include <queue.h>
```

Data Fields

- [NodoCoda](#) * [front](#)
- [NodoCoda](#) * [rear](#)
- int [size](#)
- int [verso](#)
+1 orario, -1 antiorario (modificato da carta Cambio Giro)

5.15.1 Detailed Description

Coda dei turni: coda doppiamente concatenata con direzione di scorrimento.

[front](#) è l'inizio della coda (prossimo turno), [rear](#) è la fine. [verso](#) memorizza la direzione (+1 orario, -1 antiorario) per Cambio Giro.

5.15.2 Field Documentation

5.15.2.1 front

```
NodoCoda* CodaTurni::front
```

5.15.2.2 rear

```
NodoCoda* CodaTurni::rear
```

5.15.2.3 size

```
int CodaTurni::size
```

5.15.2.4 verso

```
int CodaTurni::verso
```

+1 orario, -1 antiorario (modificato da carta Cambio Giro)

The documentation for this struct was generated from the following file:

- [lib/data_structures/queue.h](#)

5.16 Color Struct Reference

```
#include <raylib.h>
```

Data Fields

- unsigned char [r](#)
- unsigned char [g](#)
- unsigned char [b](#)
- unsigned char [a](#)

5.16.1 Field Documentation

5.16.1.1 [a](#)

```
unsigned char Color::a
```

5.16.1.2 [b](#)

```
unsigned char Color::b
```

5.16.1.3 [g](#)

```
unsigned char Color::g
```

5.16.1.4 [r](#)

```
unsigned char Color::r
```

The documentation for this struct was generated from the following file:

- lib/raylib/include/[raylib.h](#)

5.17 DatabaseUtenti Struct Reference

Database degli utenti in memoria (allocazione DINAMICA).

```
#include <data_structures.h>
```

Data Fields

- [Statistiche](#) * [lista](#)
- int [num_utenti](#)
- int [capacita](#)

5.17.1 Detailed Description

Database degli utenti in memoria (allocazione DINAMICA).

Il campo `lista` e' un puntatore a un array allocato con `malloc/realloc`. La capacita' corrente e' `capacita`, il numero effettivo di utenti e' `num_utenti`. Quando `num_utenti` raggiunge `capacita`, l'array viene riallocato con `DB_CAPACITA_INIZIALE` nuovi slot.

5.17.2 Field Documentation

5.17.2.1 `capacita`

```
int DatabaseUtenti::capacita
```

5.17.2.2 `lista`

```
Statistiche* DatabaseUtenti::lista
```

5.17.2.3 `num_utenti`

```
int DatabaseUtenti::num_utenti
```

The documentation for this struct was generated from the following file:

- [lib/data_structures.h](#)

5.18 FilePathList Struct Reference

```
#include <raylib.h>
```

Data Fields

- unsigned int `count`
- char ** `paths`

5.18.1 Field Documentation

5.18.1.1 `count`

```
unsigned int FilePathList::count
```

5.18.1.2 paths

```
char** FilePathList::paths
```

The documentation for this struct was generated from the following file:

- lib/raylib/include/[raylib.h](#)

5.19 float16 Struct Reference

```
#include <raymath.h>
```

Data Fields

- float [v](#) [16]

5.19.1 Field Documentation

5.19.1.1 v

```
float float16::v[16]
```

The documentation for this struct was generated from the following file:

- lib/raylib/include/[raymath.h](#)

5.20 float3 Struct Reference

```
#include <raymath.h>
```

Data Fields

- float [v](#) [3]

5.20.1 Field Documentation

5.20.1.1 v

```
float float3::v[3]
```

The documentation for this struct was generated from the following file:

- lib/raylib/include/[raymath.h](#)

5.21 Font Struct Reference

```
#include <raylib.h>
```

Data Fields

- int [baseSize](#)
- int [glyphCount](#)
- int [glyphPadding](#)
- [Texture2D](#) texture
- [Rectangle](#) * [recs](#)
- [GlyphInfo](#) * [glyphs](#)

5.21.1 Field Documentation

5.21.1.1 baseSize

```
int Font::baseSize
```

5.21.1.2 glyphCount

```
int Font::glyphCount
```

5.21.1.3 glyphPadding

```
int Font::glyphPadding
```

5.21.1.4 glyphs

```
GlyphInfo* Font::glyphs
```

5.21.1.5 recs

```
Rectangle* Font::recs
```

5.21.1.6 texture

```
Texture2D Font::texture
```

The documentation for this struct was generated from the following file:

- lib/raylib/include/[raylib.h](#)

5.22 GameSession Struct Reference

Sessione di gioco multiplayer.

```
#include <server_manager.h>
```

Data Fields

- char [session_id](#) [16]
- int [num_players](#)
- [SessionPlayer](#) [players](#) [MAX_PLAYERS_PER_SESSION]
- [StatoGioco](#) [game_state](#)
- [SessionState](#) [state](#)
- float [startCountdown](#)
- float [idleTime](#)
- int [host_id](#)

5.22.1 Detailed Description

Sessione di gioco multiplayer.

5.22.2 Field Documentation

5.22.2.1 [game_state](#)

[StatoGioco](#) [GameSession::game_state](#)

5.22.2.2 [host_id](#)

int [GameSession::host_id](#)

5.22.2.3 [idleTime](#)

float [GameSession::idleTime](#)

5.22.2.4 [num_players](#)

int [GameSession::num_players](#)

5.22.2.5 [players](#)

[SessionPlayer](#) [GameSession::players](#) [MAX_PLAYERS_PER_SESSION]

5.22.2.6 session_id

```
char GameSession::session_id[16]
```

5.22.2.7 startCountdown

```
float GameSession::startCountdown
```

5.22.2.8 state

```
SessionState GameSession::state
```

The documentation for this struct was generated from the following file:

- lib/socket/server/[server_manager.h](#)

5.23 Giocatore Struct Reference

Rappresenta un giocatore nella partita (umano o bot).

```
#include <data_structures.h>
```

Data Fields

- [Utente info](#)
- char [nome](#) [30]
- [ListaCarte](#) * [mano](#)
- int [is_bot](#)
- int [stato_pronto](#)
- int [num_carte_mano](#)
- int [socket_id](#)
- int [era_in_prelobby](#)
- [Carta](#) [mano_backup](#) [108]
- int [num_carte_backup](#)

5.23.1 Detailed Description

Rappresenta un giocatore nella partita (umano o bot).

Contiene i dati anagrafici dell'utente (o nome del bot), la mano di carte (lista concatenata bidirezionale), e lo stato nella lobby (pronto/in attesa).

[mano_backup\[\]](#) e [num_carte_backup](#) servono per serializzare su file: la lista linkata ([mano](#)) non e' serializzabile via fwrite, quindi prima di salvare si copiano le carte in [mano_backup](#) e si setta [mano](#) = NULL. Al caricamento si ricostruisce la lista da [mano_backup](#).

5.23.2 Field Documentation

5.23.2.1 era_in_prelobby

```
int Giocatore::era_in_prelobby
```

5.23.2.2 info

```
Utente Giocatore::info
```

5.23.2.3 is_bot

```
int Giocatore::is_bot
```

5.23.2.4 mano

```
ListaCarte* Giocatore::mano
```

5.23.2.5 mano_backup

```
Carta Giocatore::mano_backup[108]
```

5.23.2.6 nome

```
char Giocatore::nome[30]
```

5.23.2.7 num_carte_backup

```
int Giocatore::num_carte_backup
```

5.23.2.8 num_carte_mano

```
int Giocatore::num_carte_mano
```

5.23.2.9 socket_id

```
int Giocatore::socket_id
```

5.23.2.10 stato_pronto

```
int Giocatore::stato_pronto
```

The documentation for this struct was generated from the following file:

- lib/[data_structures.h](#)

5.24 GlyphInfo Struct Reference

```
#include <raylib.h>
```

Data Fields

- int [value](#)
- int [offsetX](#)
- int [offsetY](#)
- int [advanceX](#)
- [Image](#) [image](#)

5.24.1 Field Documentation

5.24.1.1 advanceX

```
int GlyphInfo::advanceX
```

5.24.1.2 image

```
Image GlyphInfo::image
```

5.24.1.3 offsetX

```
int GlyphInfo::offsetX
```

5.24.1.4 offsetY

```
int GlyphInfo::offsetY
```

5.24.1.5 value

```
int GlyphInfo::value
```

The documentation for this struct was generated from the following file:

- lib/raylib/include/[raylib.h](#)

5.25 Image Struct Reference

```
#include <raylib.h>
```

Data Fields

- void * [data](#)
- int [width](#)
- int [height](#)
- int [mipmaps](#)
- int [format](#)

5.25.1 Field Documentation

5.25.1.1 data

```
void* Image::data
```

5.25.1.2 format

```
int Image::format
```

5.25.1.3 height

```
int Image::height
```

5.25.1.4 mipmaps

```
int Image::mipmaps
```

5.25.1.5 width

```
int Image::width
```

The documentation for this struct was generated from the following file:

- lib/raylib/include/[raylib.h](#)

5.26 LanCompactUser Struct Reference

```
#include <lan_protocol.h>
```

Data Fields

- char [email](#) [100]
- char [nome](#) [30]
- char [cognome](#) [30]
- char [username](#) [30]
- char [password](#) [30]
- char [pin_recupero](#) [7]
- int [partite_giocate](#)
- int [vittorie_giocatore](#)
- int [vittorie_bot](#)
- [PartitaRegistrata storico_partite](#) [STORICO_SYNC_COUNT]
Storico partite per sincronizzazione completa.
- int [storico_count](#)
Numero effettivo di partite storiche valide.

5.26.1 Field Documentation

5.26.1.1 cognome

```
char LanCompactUser::cognome[30]
```

5.26.1.2 email

```
char LanCompactUser::email[100]
```

5.26.1.3 nome

```
char LanCompactUser::nome[30]
```

5.26.1.4 partite_giocate

```
int LanCompactUser::partite_giocate
```

5.26.1.5 password

```
char LanCompactUser::password[30]
```

5.26.1.6 pin_recupero

```
char LanCompactUser::pin_recupero[7]
```

5.26.1.7 storico_count

```
int LanCompactUser::storico_count
```

Numero effettivo di partite storiche valide.

5.26.1.8 storico_partite

```
PartitaRegistrata LanCompactUser::storico_partite[STORICO_SYNC_COUNT]
```

Storico partite per sincronizzazione completa.

5.26.1.9 username

```
char LanCompactUser::username[30]
```

5.26.1.10 vittorie_bot

```
int LanCompactUser::vittorie_bot
```

5.26.1.11 vittorie_giocatore

```
int LanCompactUser::vittorie_giocatore
```

The documentation for this struct was generated from the following file:

- lib/socket/lan/[lan_protocol.h](#)

5.27 LanDeleteSavePacket Struct Reference

Pacchetto per segnalazione eliminazione salvataggio.

```
#include <lan_protocol.h>
```

Data Fields

- int [magic](#)
Magic number.
- char [owner_username](#) [30]
Username del proprietario.
- char [data_salvataggio](#) [30]
Data del salvataggio.

5.27.1 Detailed Description

Pacchetto per segnalazione eliminazione salvataggio.

5.27.2 Field Documentation

5.27.2.1 data_salvataggio

```
char LanDeleteSavePacket::data_salvataggio[30]
```

Data del salvataggio.

5.27.2.2 magic

```
int LanDeleteSavePacket::magic
```

Magic number.

5.27.2.3 owner_username

```
char LanDeleteSavePacket::owner_username[30]
```

Username del proprietario.

The documentation for this struct was generated from the following file:

- [lib/socket/lan/lan_protocol.h](#)

5.28 LanPacketHeader Struct Reference

Header comune a tutti i pacchetti database LAN.

```
#include <lan_protocol.h>
```

Data Fields

- int [magic](#)
Magic number per identificare il tipo pacchetto.
- int [length](#)
Lunghezza totale del pacchetto.

5.28.1 Detailed Description

Header comune a tutti i pacchetti database LAN.

5.28.2 Field Documentation

5.28.2.1 length

```
int LanPacketHeader::length
```

Lunghezza totale del pacchetto.

5.28.2.2 magic

```
int LanPacketHeader::magic
```

Magic number per identificare il tipo pacchetto.

The documentation for this struct was generated from the following file:

- [lib/socket/lan/lan_protocol.h](#)

5.29 LanStatsSyncPacket Struct Reference

```
#include <lan_protocol.h>
```

Data Fields

- int [magic](#)
magic = LAN_PACKET_STATS_SYNC
- char [username](#) [30]
Username del giocatore.
- int [partite_giocate](#)
Totale partite giocate.
- int [vittorie_giocatore](#)
Totale vittorie.
- int [vittorie_bot](#)
Totale sconfitte.
- int [storico_count](#)
Numero di partite storiche valide (0-STORICO_SYNC_COUNT)
- [PartitaRegistrata storico_partite](#) [STORICO_SYNC_COUNT]
Ultime N partite.

5.29.1 Field Documentation

5.29.1.1 magic

```
int LanStatsSyncPacket::magic
```

```
magic = LAN_PACKET_STATS_SYNC
```

5.29.1.2 partite_giocate

```
int LanStatsSyncPacket::partite_giocate
```

Totale partite giocate.

5.29.1.3 storico_count

```
int LanStatsSyncPacket::storico_count
```

Numero di partite storiche valide (0-STORICO_SYNC_COUNT)

5.29.1.4 storico_partite

```
PartitaRegistrata LanStatsSyncPacket::storico_partite[STORICO_SYNC_COUNT]
```

Ultime N partite.

5.29.1.5 username

```
char LanStatsSyncPacket::username[30]
```

Username del giocatore.

5.29.1.6 vittorie_bot

```
int LanStatsSyncPacket::vittorie_bot
```

Totale sconfitte.

5.29.1.7 vittorie_giocatore

```
int LanStatsSyncPacket::vittorie_giocatore
```

Totale vittorie.

The documentation for this struct was generated from the following file:

- [lib/socket/lan/lan_protocol.h](#)

5.30 ListaCarte Struct Reference

Struttura container per la lista di carte (mano del giocatore).

```
#include <data_structures.h>
```

Data Fields

- [NodoCarta](#) * [testa](#)
- [NodoCarta](#) * [coda](#)
- int [lunghezza](#)

5.30.1 Detailed Description

Struttura container per la lista di carte (mano del giocatore).

Mantiene i puntatori a testa e coda per operazioni $O(1)$ e la lunghezza corrente per contare rapidamente le carte in mano.

5.30.2 Field Documentation

5.30.2.1 coda

```
NodoCarta* ListaCarte::coda
```

5.30.2.2 lunghezza

```
int ListaCarte::lunghezza
```

5.30.2.3 testa

```
NodoCarta* ListaCarte::testa
```

The documentation for this struct was generated from the following file:

- lib/[data_structures.h](#)

5.31 Material Struct Reference

```
#include <raylib.h>
```

Data Fields

- [Shader](#) [shader](#)
- [MaterialMap](#) * [maps](#)
- float [params](#) [4]

5.31.1 Field Documentation

5.31.1.1 maps

[MaterialMap*](#) `Material::maps`

5.31.1.2 params

`float` `Material::params[4]`

5.31.1.3 shader

[Shader](#) `Material::shader`

The documentation for this struct was generated from the following file:

- `lib/raylib/include/`[raylib.h](#)

5.32 MaterialMap Struct Reference

```
#include <raylib.h>
```

Data Fields

- [Texture2D](#) `texture`
- [Color](#) `color`
- `float` `value`

5.32.1 Field Documentation

5.32.1.1 color

[Color](#) `MaterialMap::color`

5.32.1.2 texture

[Texture2D](#) `MaterialMap::texture`

5.32.1.3 value

`float` `MaterialMap::value`

The documentation for this struct was generated from the following file:

- `lib/raylib/include/`[raylib.h](#)

5.33 Matrix Struct Reference

```
#include <raylib.h>
```

Data Fields

- float [m0](#)
- float [m4](#)
- float [m8](#)
- float [m12](#)
- float [m1](#)
- float [m5](#)
- float [m9](#)
- float [m13](#)
- float [m2](#)
- float [m6](#)
- float [m10](#)
- float [m14](#)
- float [m3](#)
- float [m7](#)
- float [m11](#)
- float [m15](#)

5.33.1 Field Documentation

5.33.1.1 m0

```
float Matrix::m0
```

5.33.1.2 m1

```
float Matrix::m1
```

5.33.1.3 m10

```
float Matrix::m10
```

5.33.1.4 m11

```
float Matrix::m11
```

5.33.1.5 m12

```
float Matrix::m12
```

5.33.1.6 m13

```
float Matrix::m13
```

5.33.1.7 m14

```
float Matrix::m14
```

5.33.1.8 m15

```
float Matrix::m15
```

5.33.1.9 m2

```
float Matrix::m2
```

5.33.1.10 m3

```
float Matrix::m3
```

5.33.1.11 m4

```
float Matrix::m4
```

5.33.1.12 m5

```
float Matrix::m5
```

5.33.1.13 m6

```
float Matrix::m6
```

5.33.1.14 m7

```
float Matrix::m7
```

5.33.1.15 m8

```
float Matrix::m8
```

5.33.1.16 m9

```
float Matrix::m9
```

The documentation for this struct was generated from the following files:

- lib/raylib/include/[raylib.h](#)
- lib/raylib/include/[raymath.h](#)
- lib/raylib/include/[rgl.h](#)

5.34 Mesh Struct Reference

```
#include <raylib.h>
```

Data Fields

- int [vertexCount](#)
- int [triangleCount](#)
- float * [vertices](#)
- float * [texcoords](#)
- float * [texcoords2](#)
- float * [normals](#)
- float * [tangents](#)
- unsigned char * [colors](#)
- unsigned short * [indices](#)
- int [boneCount](#)
- unsigned char * [boneIndices](#)
- float * [boneWeights](#)
- float * [animVertices](#)
- float * [animNormals](#)
- unsigned int [vaold](#)
- unsigned int * [vbold](#)

5.34.1 Field Documentation

5.34.1.1 animNormals

```
float* Mesh::animNormals
```

5.34.1.2 animVertices

```
float* Mesh::animVertices
```

5.34.1.3 boneCount

```
int Mesh::boneCount
```


5.34.1.4 boneIndices

```
unsigned char* Mesh::boneIndices
```

5.34.1.5 boneWeights

```
float* Mesh::boneWeights
```

5.34.1.6 colors

```
unsigned char* Mesh::colors
```

5.34.1.7 indices

```
unsigned short* Mesh::indices
```

5.34.1.8 normals

```
float* Mesh::normals
```

5.34.1.9 tangents

```
float* Mesh::tangents
```

5.34.1.10 texcoords

```
float* Mesh::texcoords
```

5.34.1.11 texcoords2

```
float* Mesh::texcoords2
```

5.34.1.12 triangleCount

```
int Mesh::triangleCount
```

5.34.1.13 vaoid

```
unsigned int Mesh::vaoid
```

5.34.1.14 vbold

```
unsigned int* Mesh::vboId
```

5.34.1.15 vertexCount

```
int Mesh::vertexCount
```

5.34.1.16 vertices

```
float* Mesh::vertices
```

The documentation for this struct was generated from the following file:

- lib/raylib/include/[raylib.h](#)

5.35 MessaggioChat Struct Reference

Singolo messaggio nella chat multiplayer.

```
#include <data_structures.h>
```

Data Fields

- char [mittente](#) [30]
- char [testo](#) [256]
- char [orario](#) [10]

5.35.1 Detailed Description

Singolo messaggio nella chat multiplayer.

Contiene mittente, testo e orario di invio.

5.35.2 Field Documentation

5.35.2.1 mittente

```
char MessaggioChat::mittente[30]
```

5.35.2.2 orario

```
char MessaggioChat::orario[10]
```

5.35.2.3 testo

```
char MessaggioChat::testo[256]
```

The documentation for this struct was generated from the following file:

- [lib/data_structures.h](#)

5.36 Model Struct Reference

```
#include <raylib.h>
```

Data Fields

- [Matrix](#) transform
- int [meshCount](#)
- int [materialCount](#)
- [Mesh](#) * [meshes](#)
- [Material](#) * [materials](#)
- int * [meshMaterial](#)
- [ModelSkeleton](#) [skeleton](#)
- [ModelAnimPose](#) [currentPose](#)
- [Matrix](#) * [boneMatrices](#)

5.36.1 Field Documentation

5.36.1.1 boneMatrices

```
Matrix* Model::boneMatrices
```

5.36.1.2 currentPose

```
ModelAnimPose Model::currentPose
```

5.36.1.3 materialCount

```
int Model::materialCount
```

5.36.1.4 materials

```
Material* Model::materials
```

5.36.1.5 meshCount

```
int Model::meshCount
```

5.36.1.6 meshes

```
Mesh* Model::meshes
```

5.36.1.7 meshMaterial

```
int* Model::meshMaterial
```

5.36.1.8 skeleton

```
ModelSkeleton Model::skeleton
```

5.36.1.9 transform

```
Matrix Model::transform
```

The documentation for this struct was generated from the following file:

- lib/raylib/include/[raylib.h](#)

5.37 ModelAnimation Struct Reference

```
#include <raylib.h>
```

Data Fields

- char [name](#) [32]
- int [boneCount](#)
- int [keyframeCount](#)
- [ModelAnimPose](#) * [keyframePoses](#)

5.37.1 Field Documentation

5.37.1.1 boneCount

```
int ModelAnimation::boneCount
```

5.37.1.2 keyframeCount

```
int ModelAnimation::keyframeCount
```

5.37.1.3 keyframePoses

```
ModelAnimPose* ModelAnimation::keyframePoses
```

5.37.1.4 name

```
char ModelAnimation::name[32]
```

The documentation for this struct was generated from the following file:

- lib/raylib/include/[raylib.h](#)

5.38 ModelSkeleton Struct Reference

```
#include <raylib.h>
```

Data Fields

- int [boneCount](#)
- [BoneInfo](#) * [bones](#)
- [ModelAnimPose](#) [bindPose](#)

5.38.1 Field Documentation

5.38.1.1 bindPose

```
ModelAnimPose ModelSkeleton::bindPose
```

5.38.1.2 boneCount

```
int ModelSkeleton::boneCount
```

5.38.1.3 bones

```
BoneInfo* ModelSkeleton::bones
```

The documentation for this struct was generated from the following file:

- lib/raylib/include/[raylib.h](#)

5.39 Music Struct Reference

```
#include <raylib.h>
```

Data Fields

- [AudioStream](#) `stream`
- unsigned int `frameCount`
- bool `looping`
- int `ctxType`
- void * `ctxData`

5.39.1 Field Documentation

5.39.1.1 `ctxData`

```
void* Music::ctxData
```

5.39.1.2 `ctxType`

```
int Music::ctxType
```

5.39.1.3 `frameCount`

```
unsigned int Music::frameCount
```

5.39.1.4 `looping`

```
bool Music::looping
```

5.39.1.5 `stream`

```
AudioStream Music::stream
```

The documentation for this struct was generated from the following file:

- `lib/raylib/include/raylib.h`

5.40 NetPacket Struct Reference

```
#include <network.h>
```

Data Fields

- [PacketType](#) type
- int [sender_id](#)
- union {
 - [PacketMove](#) [NetPacket::move](#)
 - [PacketDraw](#) [NetPacket::draw](#)
 - [PacketGameState](#) [NetPacket::game_state](#)
 - [PacketColorChoice](#) [NetPacket::color_choice](#)
 - [PacketChat](#) [NetPacket::chat](#)
 - [PacketJoinGame](#) [NetPacket::join_game](#)
 - [PacketJoinRejected](#) [NetPacket::join_rejected](#)
 - [PacketPlayerHand](#) [NetPacket::player_hand](#)
 - [PacketCreateSession](#) [NetPacket::create_session](#)
 - [PacketJoinSession](#) [NetPacket::join_session](#)
 - [PacketSessionList](#) [NetPacket::session_list](#)
 - [PacketSessionState](#) [NetPacket::session_state](#)
 - [PacketPlayerReady](#) [NetPacket::player_ready](#)
 - [PacketGameFinished](#) [NetPacket::game_finished](#)
 - [PacketPlayAgain](#) [NetPacket::play_again](#)
 - [PacketHostReturnLobby](#) [NetPacket::host_return_lobby](#)
 - [PacketHeartbeat](#) [NetPacket::heartbeat](#)
 - [AdminUserAction](#) [NetPacket::admin_action](#)
 - [AdminDBSyncPacket](#) [NetPacket::admin_db_packet](#)
 - [StatsSyncPacket](#) [NetPacket::stats_sync](#)
 - char [NetPacket::raw](#) [512]
- } [data](#)

The documentation for this struct was generated from the following file:

- [lib/socket/network/network.h](#)

5.41 NodoAlbero Struct Reference

Nodo dell'albero BST: contiene un [Utente](#) e puntatori ai figli.

```
#include <bst.h>
```

Data Fields

- [Utente](#) [utente](#)
- struct [NodoAlbero](#) * [left](#)
- struct [NodoAlbero](#) * [right](#)

5.41.1 Detailed Description

Nodo dell'albero BST: contiene un [Utente](#) e puntatori ai figli.

5.41.2 Field Documentation

5.41.2.1 left

```
struct NodoAlbero* NodoAlbero::left
```

5.41.2.2 right

```
struct NodoAlbero* NodoAlbero::right
```

5.41.2.3 utente

```
Utente NodoAlbero::utente
```

The documentation for this struct was generated from the following file:

- [lib/data_structures/bst.h](#)

5.42 NodoCarta Struct Reference

Nodo per la lista doppiamente concatenata della mano del giocatore.

```
#include <data_structures.h>
```

Data Fields

- [Carta carta](#)
- struct [NodoCarta](#) * [prossimo](#)
- struct [NodoCarta](#) * [prev](#)

5.42.1 Detailed Description

Nodo per la lista doppiamente concatenata della mano del giocatore.

Ogni nodo contiene una carta e due puntatori: uno al nodo successivo (forward link) e uno al nodo precedente (backward link). Questo permette inserimenti ed eliminazioni efficienti in qualsiasi posizione.

5.42.2 Field Documentation

5.42.2.1 carta

```
Carta NodoCarta::carta
```


5.42.2.2 prev

```
struct NodoCarta* NodoCarta::prev
```

5.42.2.3 prossimo

```
struct NodoCarta* NodoCarta::prossimo
```

The documentation for this struct was generated from the following file:

- [lib/data_structures.h](#)

5.43 NodoCoda Struct Reference

Nodo della coda: contiene un [Giocatore](#) e puntatori next/prev.

```
#include <queue.h>
```

Data Fields

- [Giocatore giocatore](#)
- struct [NodoCoda](#) * [next](#)
- struct [NodoCoda](#) * [prev](#)

5.43.1 Detailed Description

Nodo della coda: contiene un [Giocatore](#) e puntatori next/prev.

5.43.2 Field Documentation

5.43.2.1 giocatore

```
Giocatore NodoCoda::giocatore
```

5.43.2.2 next

```
struct NodoCoda* NodoCoda::next
```

5.43.2.3 prev

```
struct NodoCoda* NodoCoda::prev
```

The documentation for this struct was generated from the following file:

- [lib/data_structures/queue.h](#)

5.44 NodoDecisione Struct Reference

Nodo albero: azione, priorità, figli si/no.

```
#include <bot_ai.h>
```

Data Fields

- [DecisioneBot](#) azione
- int [priorita](#)
- struct [NodoDecisione](#) * [si](#)
- struct [NodoDecisione](#) * [no](#)

5.44.1 Detailed Description

Nodo albero: azione, priorità, figli si/no.

5.44.2 Field Documentation

5.44.2.1 azione

```
DecisioneBot NodoDecisione::azione
```

5.44.2.2 no

```
struct NodoDecisione* NodoDecisione::no
```

5.44.2.3 priorit a

```
int NodoDecisione::priorita
```

5.44.2.4 si

```
struct NodoDecisione* NodoDecisione::si
```

The documentation for this struct was generated from the following file:

- lib/screens/game/[bot_ai.h](#)

5.45 NodoPila Struct Reference

Nodo della pila: contiene una carta e puntatore al next.

```
#include <stack.h>
```

Data Fields

- [Carta](#) [carta](#)
- struct [NodoPila](#) * [next](#)

5.45.1 Detailed Description

Nodo della pila: contiene una carta e puntatore al next.

5.45.2 Field Documentation

5.45.2.1 carta

```
Carta NodoPila::carta
```

5.45.2.2 next

```
struct NodoPila* NodoPila::next
```

The documentation for this struct was generated from the following file:

- lib/data_structures/[stack.h](#)

5.46 NPatchInfo Struct Reference

```
#include <raylib.h>
```

Data Fields

- [Rectangle](#) [source](#)
- int [left](#)
- int [top](#)
- int [right](#)
- int [bottom](#)
- int [layout](#)

5.46.1 Field Documentation

5.46.1.1 bottom

```
int NPatchInfo::bottom
```

5.46.1.2 layout

```
int NPatchInfo::layout
```

5.46.1.3 left

```
int NPatchInfo::left
```

5.46.1.4 right

```
int NPatchInfo::right
```

5.46.1.5 source

```
Rectangle NPatchInfo::source
```

5.46.1.6 top

```
int NPatchInfo::top
```

The documentation for this struct was generated from the following file:

- lib/raylib/include/[raylib.h](#)

5.47 PacketChat Struct Reference

Pacchetto chat.

```
#include <network.h>
```

Data Fields

- int [player_id](#)
- char [message](#) [128]

5.47.1 Detailed Description

Pacchetto chat.

The documentation for this struct was generated from the following file:

- lib/socket/network/[network.h](#)

5.48 PacketColorChoice Struct Reference

Pacchetto scelta colore.

```
#include <network.h>
```

Data Fields

- int [player_id](#)
- [Colore](#) colore

5.48.1 Detailed Description

Pacchetto scelta colore.

The documentation for this struct was generated from the following file:

- lib/socket/network/[network.h](#)

5.49 PacketCreateSession Struct Reference

Pacchetto creazione sessione.

```
#include <network.h>
```

Data Fields

- char [session_id](#) [16]
- char [host_name](#) [30]
- int [num_players](#)

5.49.1 Detailed Description

Pacchetto creazione sessione.

The documentation for this struct was generated from the following file:

- lib/socket/network/[network.h](#)

5.50 PacketDraw Struct Reference

Pacchetto pesca: giocatore e quantità.

```
#include <network.h>
```

Data Fields

- int [player_id](#)
- int [count](#)

5.50.1 Detailed Description

Pacchetto pesca: giocatore e quantità.

The documentation for this struct was generated from the following file:

- lib/socket/network/[network.h](#)

5.51 PacketGameFinished Struct Reference

Pacchetto fine partita.

```
#include <network.h>
```

Data Fields

- int [winning_player_id](#)
- char [winner_name](#) [30]

5.51.1 Detailed Description

Pacchetto fine partita.

The documentation for this struct was generated from the following file:

- lib/socket/network/[network.h](#)

5.52 PacketGameState Struct Reference

Pacchetto stato di gioco completo.

```
#include <network.h>
```

Data Fields

- [StatoGioco stato_gioco](#)

5.52.1 Detailed Description

Pacchetto stato di gioco completo.

The documentation for this struct was generated from the following file:

- lib/socket/network/[network.h](#)

5.53 PacketHeartbeat Struct Reference

Pacchetto heartbeat per keep-alive.

```
#include <network.h>
```

Data Fields

- int [player_id](#)
- float [timestamp](#)

5.53.1 Detailed Description

Pacchetto heartbeat per keep-alive.

The documentation for this struct was generated from the following file:

- lib/socket/network/[network.h](#)

5.54 PacketHostReturnLobby Struct Reference

Pacchetto host return lobby.

```
#include <network.h>
```

Data Fields

- int [host_ready](#)

5.54.1 Detailed Description

Pacchetto host return lobby.

The documentation for this struct was generated from the following file:

- lib/socket/network/[network.h](#)

5.55 PacketJoinGame Struct Reference

Pacchetto join game.

```
#include <network.h>
```

Data Fields

- char [nome](#) [50]

5.55.1 Detailed Description

Pacchetto join game.

The documentation for this struct was generated from the following file:

- lib/socket/network/[network.h](#)

5.56 PacketJoinRejected Struct Reference

Pacchetto join rifiutato.

```
#include <network.h>
```

Data Fields

- char [motivo](#) [100]

5.56.1 Detailed Description

Pacchetto join rifiutato.

The documentation for this struct was generated from the following file:

- lib/socket/network/[network.h](#)

5.57 PacketJoinSession Struct Reference

Pacchetto join sessione.

```
#include <network.h>
```


Data Fields

- char [session_id](#) [16]
- char [player_name](#) [30]

5.57.1 Detailed Description

Pacchetto join sessione.

The documentation for this struct was generated from the following file:

- lib/socket/network/[network.h](#)

5.58 PacketMove Struct Reference

Pacchetto mossa: giocatore, indice carta, colore scelto.

```
#include <network.h>
```

Data Fields

- int [player_id](#)
- int [card_index](#)
- [Colore](#) [colore_scelto](#)

5.58.1 Detailed Description

Pacchetto mossa: giocatore, indice carta, colore scelto.

The documentation for this struct was generated from the following file:

- lib/socket/network/[network.h](#)

5.59 PacketPlayAgain Struct Reference

Pacchetto play again (con nome giocatore per fix bug).

```
#include <network.h>
```

Data Fields

- int [player_id](#)
- float [timestamp](#)
- int [play_again](#)
- char [player_name](#) [30]

5.59.1 Detailed Description

Pacchetto play again (con nome giocatore per fix bug).

The documentation for this struct was generated from the following file:

- lib/socket/network/[network.h](#)

5.60 PacketPlayerHand Struct Reference

Pacchetto mano giocatore.

```
#include <network.h>
```

Data Fields

- int [player_id](#)
- [Carta carte](#) [108]
- int [num_carte](#)

5.60.1 Detailed Description

Pacchetto mano giocatore.

The documentation for this struct was generated from the following file:

- lib/socket/network/[network.h](#)

5.61 PacketPlayerReady Struct Reference

Pacchetto ready/play again.

```
#include <network.h>
```

Data Fields

- int [player_id](#)
- int [is_ready](#)
- int [want_play_again](#)

5.61.1 Detailed Description

Pacchetto ready/play again.

The documentation for this struct was generated from the following file:

- lib/socket/network/[network.h](#)

5.62 PacketSessionList Struct Reference

Pacchetto lista sessioni.

```
#include <network.h>
```

Data Fields

- int [session_count](#)
- char [sessions](#) [10][16]

5.62.1 Detailed Description

Pacchetto lista sessioni.

The documentation for this struct was generated from the following file:

- lib/socket/network/[network.h](#)

5.63 PacketSessionState Struct Reference

Pacchetto stato sessione multiplayer.

```
#include <network.h>
```

Data Fields

- char [session_id](#) [16]
- int [num_players](#)
- char [player_names](#) [4][30]
- int [player_ready](#) [4]
- int [game_started](#)

5.63.1 Detailed Description

Pacchetto stato sessione multiplayer.

The documentation for this struct was generated from the following file:

- lib/socket/network/[network.h](#)

5.64 PartitaRegistrata Struct Reference

Record di una singola partita nello storico di un utente.

```
#include <data_structures.h>
```

Data Fields

- char [modalita](#) [20]
- char [esito](#) [15]
- int [durata_secondi](#)
- int [posizione](#)

5.64.1 Detailed Description

Record di una singola partita nello storico di un utente.

Viene salvato nell'array storico_partite della struttura [Statistiche](#).

5.64.2 Field Documentation

5.64.2.1 durata_secondi

```
int PartitaRegistrata::durata_secondi
```

5.64.2.2 esito

```
char PartitaRegistrata::esito[15]
```

5.64.2.3 modalita

```
char PartitaRegistrata::modalita[20]
```

5.64.2.4 posizione

```
int PartitaRegistrata::posizione
```

The documentation for this struct was generated from the following file:

- lib/[data_structures.h](#)

5.65 Pila Struct Reference

[Pila](#) LIFO: top punta all'ultimo inserito, size è la cardinalità.

```
#include <stack.h>
```

Data Fields

- [NodoPila](#) * [top](#)
- int [size](#)

5.65.1 Detailed Description

[Pila](#) LIFO: top punta all'ultimo inserito, size è la cardinalità.

5.65.2 Field Documentation

5.65.2.1 size

```
int Pila::size
```

5.65.2.2 top

```
NodoPila* Pila::top
```

The documentation for this struct was generated from the following file:

- [lib/data_structures/stack.h](#)

5.66 Ray Struct Reference

```
#include <raylib.h>
```

Data Fields

- [Vector3 position](#)
- [Vector3 direction](#)

5.66.1 Field Documentation

5.66.1.1 direction

```
Vector3 Ray::direction
```

5.66.1.2 position

```
Vector3 Ray::position
```

The documentation for this struct was generated from the following file:

- [lib/raylib/include/raylib.h](#)

5.67 RayCollision Struct Reference

```
#include <raylib.h>
```

Data Fields

- [bool](#) `hit`
- [float](#) `distance`
- [Vector3](#) `point`
- [Vector3](#) `normal`

5.67.1 Field Documentation

5.67.1.1 `distance`

```
float RayCollision::distance
```

5.67.1.2 `hit`

```
bool RayCollision::hit
```

5.67.1.3 `normal`

```
Vector3 RayCollision::normal
```

5.67.1.4 `point`

```
Vector3 RayCollision::point
```

The documentation for this struct was generated from the following file:

- `lib/raylib/include/raylib.h`

5.68 Rectangle Struct Reference

```
#include <raylib.h>
```

Data Fields

- [float](#) `x`
- [float](#) `y`
- [float](#) `width`
- [float](#) `height`

5.68.1 Field Documentation

5.68.1.1 height

```
float Rectangle::height
```

5.68.1.2 width

```
float Rectangle::width
```

5.68.1.3 x

```
float Rectangle::x
```

5.68.1.4 y

```
float Rectangle::y
```

The documentation for this struct was generated from the following file:

- lib/raylib/include/[raylib.h](#)

5.69 RenderTexture Struct Reference

```
#include <raylib.h>
```

Data Fields

- unsigned int [id](#)
- [Texture](#) [texture](#)
- [Texture](#) [depth](#)

5.69.1 Field Documentation

5.69.1.1 depth

```
Texture RenderTexture::depth
```

5.69.1.2 id

```
unsigned int RenderTexture::id
```

5.69.1.3 texture

`Texture` `RenderTexture::texture`

The documentation for this struct was generated from the following file:

- `lib/raylib/include/raylib.h`

5.70 rlDrawCall Struct Reference

```
#include <rlgl.h>
```

Data Fields

- `int` `mode`
- `int` `vertexCount`
- `int` `vertexAlignment`
- `unsigned int` `textureId`

5.70.1 Field Documentation

5.70.1.1 mode

```
int rlDrawCall::mode
```

5.70.1.2 textureId

```
unsigned int rlDrawCall::textureId
```

5.70.1.3 vertexAlignment

```
int rlDrawCall::vertexAlignment
```

5.70.1.4 vertexCount

```
int rlDrawCall::vertexCount
```

The documentation for this struct was generated from the following file:

- `lib/raylib/include/rlgl.h`

5.71 rlRenderBatch Struct Reference

```
#include <rlgl.h>
```

Data Fields

- int [bufferCount](#)
- int [currentBuffer](#)
- [rlVertexBuffer](#) * [vertexBuffer](#)
- [rlDrawCall](#) * [draws](#)
- int [drawCounter](#)
- float [currentDepth](#)

5.71.1 Field Documentation

5.71.1.1 bufferCount

```
int rlRenderBatch::bufferCount
```

5.71.1.2 currentBuffer

```
int rlRenderBatch::currentBuffer
```

5.71.1.3 currentDepth

```
float rlRenderBatch::currentDepth
```

5.71.1.4 drawCounter

```
int rlRenderBatch::drawCounter
```

5.71.1.5 draws

```
rlDrawCall* rlRenderBatch::draws
```

5.71.1.6 vertexBuffer

```
rlVertexBuffer* rlRenderBatch::vertexBuffer
```

The documentation for this struct was generated from the following file:

- [lib/raylib/include/rlgl.h](#)

5.72 rlVertexBuffer Struct Reference

```
#include <rlgl.h>
```

Data Fields

- int [elementCount](#)
- float * [vertices](#)
- float * [texcoords](#)
- float * [normals](#)
- unsigned char * [colors](#)
- unsigned int * [indices](#)
- unsigned int [vaold](#)
- unsigned int [vbold](#) [5]

5.72.1 Field Documentation

5.72.1.1 colors

```
unsigned char* rlVertexBuffer::colors
```

5.72.1.2 elementCount

```
int rlVertexBuffer::elementCount
```

5.72.1.3 indices

```
unsigned int* rlVertexBuffer::indices
```

5.72.1.4 normals

```
float* rlVertexBuffer::normals
```

5.72.1.5 texcoords

```
float* rlVertexBuffer::texcoords
```

5.72.1.6 vaold

```
unsigned int rlVertexBuffer::vaold
```

5.72.1.7 vbold

```
unsigned int rlVertexBuffer::vbold[5]
```

5.72.1.8 vertices

```
float* rlVertexBuffer::vertices
```

The documentation for this struct was generated from the following file:

- lib/raylib/include/[rlgl.h](#)

5.73 SaveReceiver Struct Reference

Struttura per ricezione chunk di salvataggio.

```
#include <lan_protocol.h>
```

Data Fields

- int [attivo](#)
Flag di attività (1 se ricezione in corso).
- char [key](#) [70]
Chiave identificativa (username + data).
- int [total](#)
Numero totale di chunk attesi.
- int [ricevuti](#)
Numero di chunk ricevuti finora.
- int [file_size](#)
Dimensione totale del file.
- unsigned char * [filedata](#)
Buffer per i dati ricevuti (allocato dinamicamente).
- float [timer](#)
Timer per timeout ricezione.

5.73.1 Detailed Description

Struttura per ricezione chunk di salvataggio.

5.73.2 Field Documentation

5.73.2.1 attivo

```
int SaveReceiver::attivo
```

Flag di attività (1 se ricezione in corso).

5.73.2.2 file_size

```
int SaveReceiver::file_size
```

Dimensione totale del file.

5.73.2.3 filedata

```
unsigned char* SaveReceiver::filedata
```

Buffer per i dati ricevuti (allocato dinamicamente).

5.73.2.4 key

```
char SaveReceiver::key[70]
```

Chiave identificativa (username + data).

5.73.2.5 ricevuti

```
int SaveReceiver::ricevuti
```

Numero di chunk ricevuti finora.

5.73.2.6 timer

```
float SaveReceiver::timer
```

Timer per timeout ricezione.

5.73.2.7 total

```
int SaveReceiver::total
```

Numero totale di chunk attesi.

The documentation for this struct was generated from the following file:

- [lib/socket/lan/lan_protocol.h](#)

5.74 ServerManager Struct Reference

Gestore di tutte le sessioni di gioco attive.

```
#include <server_manager.h>
```

Data Fields

- [GameSession sessions](#) [[MAX_GAME_SESSIONS](#)]
- int [num_active_sessions](#)
- float [frameTime](#)

5.74.1 Detailed Description

Gestore di tutte le sessioni di gioco attive.

5.74.2 Field Documentation

5.74.2.1 frameTime

```
float ServerManager::frameTime
```

5.74.2.2 num_active_sessions

```
int ServerManager::num_active_sessions
```

5.74.2.3 sessions

```
GameSession ServerManager::sessions [MAX\_GAME\_SESSIONS]
```

The documentation for this struct was generated from the following file:

- lib/socket/server/[server_manager.h](#)

5.75 SessionPlayer Struct Reference

Informazioni giocatore nella sessione.

```
#include <server_manager.h>
```

Data Fields

- int [player_id](#)
- char [nome](#) [30]
- int [is_connected](#)
- int [is_bot](#)
- [ListaCarte](#) * [mano](#)
- float [lastHeartbeat](#)

5.75.1 Detailed Description

Informazioni giocatore nella sessione.

5.75.2 Field Documentation

5.75.2.1 is_bot

```
int SessionPlayer::is_bot
```

5.75.2.2 is_connected

```
int SessionPlayer::is_connected
```

5.75.2.3 lastHeartbeat

```
float SessionPlayer::lastHeartbeat
```

5.75.2.4 mano

```
ListaCarte* SessionPlayer::mano
```

5.75.2.5 nome

```
char SessionPlayer::nome[30]
```

5.75.2.6 player_id

```
int SessionPlayer::player_id
```

The documentation for this struct was generated from the following file:

- lib/socket/server/[server_manager.h](#)

5.76 Shader Struct Reference

```
#include <raylib.h>
```

Data Fields

- unsigned int [id](#)
- int * [locs](#)

5.76.1 Field Documentation

5.76.1.1 id

```
unsigned int Shader::id
```

5.76.1.2 locs

```
int* Shader::locs
```

The documentation for this struct was generated from the following file:

- lib/raylib/include/[raylib.h](#)

5.77 Sound Struct Reference

```
#include <raylib.h>
```

Data Fields

- [AudioStream](#) stream
- unsigned int [frameCount](#)

5.77.1 Field Documentation

5.77.1.1 frameCount

```
unsigned int Sound::frameCount
```

5.77.1.2 stream

```
AudioStream Sound::stream
```

The documentation for this struct was generated from the following file:

- lib/raylib/include/[raylib.h](#)

5.78 Statistiche Struct Reference

[Statistiche](#) complete di un utente registrato.

```
#include <data_structures.h>
```

Data Fields

- char [email](#) [100]
- char [nome](#) [30]
- char [cognome](#) [30]
- char [username](#) [30]
- char [password](#) [30]
- char [pin_recupero](#) [7]
- int [partite_giocate](#)
- int [vittorie_giocatore](#)
- int [vittorie_bot](#)
- [PartitaRegistrata storico_partite](#) [[MAX_PARTITE_STORICO](#)]

5.78.1 Detailed Description

[Statistiche](#) complete di un utente registrato.

Contiene tutti i dati dell'utente (denormalizzati per velocita' di accesso) e lo storico delle partite giocate. L'array storico_partite ha dimensione fissa MAX_PARTITE_STORICO.

5.78.2 Field Documentation

5.78.2.1 cognome

```
char Statistiche::cognome[30]
```

5.78.2.2 email

```
char Statistiche::email[100]
```

5.78.2.3 nome

```
char Statistiche::nome[30]
```

5.78.2.4 partite_giocate

```
int Statistiche::partite_giocate
```

5.78.2.5 password

```
char Statistiche::password[30]
```

5.78.2.6 pin_recupero

```
char Statistiche::pin_recupero[7]
```


5.78.2.7 storico_partite

```
PartitaRegistrata Statistiche::storico_partite[MAX_PARTITE_STORICO]
```

5.78.2.8 username

```
char Statistiche::username[30]
```

5.78.2.9 vittorie_bot

```
int Statistiche::vittorie_bot
```

5.78.2.10 vittorie_giocatore

```
int Statistiche::vittorie_giocatore
```

The documentation for this struct was generated from the following file:

- [lib/data_structures.h](#)

5.79 StatoGioco Struct Reference

Stato completo di una partita in corso.

```
#include <data_structures.h>
```

Data Fields

- [Giocatore giocatori](#) [4]
- [int num_giocatori](#)
- [Carta mazzo](#) [108]
- [int num_carte_mazzo](#)
- [Carta scarti](#) [108]
- [int num_carte_scarti](#)
- [int turno_corrente](#)
- [int direzione](#)
- [Colore colore_attivo](#)
- [int id_carta_in_trascinamento](#)
- [NodoCarta * nodo_carta_trascinata](#)
- [int gioco_finito](#)
- [char messaggio](#) [100]
- [int anim_attiva](#)
- [Carta anim_carta](#)
- [Vector2 anim_start](#)
- [Vector2 anim_end](#)
- [float anim_t](#)
- [int tipo_animazione](#)

- int [in_scelta_colore](#)
- [Carta](#) [carta_pendente](#)
- int [deve_chiamare_uno](#)
- float [timer_uno](#)
- char [notifica](#) [128]
- float [timer_notifica](#)
- int [autore_animazione](#)
- int [anim_pesca_count](#)
- int [anim_gia_applicata](#)
- char [chat_buffer](#) [256]
- float [durata_partita](#)
- char [chat_history](#) [15][128]
- int [num_chat_msg](#)
- int [chat_aperta](#)
- int [slot_salvataggio](#)
- char [data_salvataggio](#) [30]
- int [statistiche_gia_salvate](#)
- char [modalita](#) [20]

5.79.1 Detailed Description

Stato completo di una partita in corso.

Contiene tutti i dati necessari per gestire il gameplay: giocatori, mazzo, scarti, turno, animazioni, chat, UNO, ecc. Questa struttura viene serializzata per il salvataggio su file e per la sincronizzazione multiplayer via socket.

5.79.2 Field Documentation

5.79.2.1 [anim_attiva](#)

```
int StatoGioco::anim_attiva
```

5.79.2.2 [anim_carta](#)

```
Carta StatoGioco::anim_carta
```

5.79.2.3 [anim_end](#)

```
Vector2 StatoGioco::anim_end
```

5.79.2.4 [anim_gia_applicata](#)

```
int StatoGioco::anim_gia_applicata
```

5.79.2.5 [anim_pesca_count](#)

```
int StatoGioco::anim_pesca_count
```

5.79.2.6 anim_start

```
Vector2 StatoGioco::anim_start
```

5.79.2.7 anim_t

```
float StatoGioco::anim_t
```

5.79.2.8 autore_animazione

```
int StatoGioco::autore_animazione
```

5.79.2.9 carta_pendente

```
Carta StatoGioco::carta_pendente
```

5.79.2.10 chat_aperta

```
int StatoGioco::chat_aperta
```

5.79.2.11 chat_buffer

```
char StatoGioco::chat_buffer[256]
```

5.79.2.12 chat_history

```
char StatoGioco::chat_history[15][128]
```

5.79.2.13 colore_attivo

```
Colore StatoGioco::colore_attivo
```

5.79.2.14 data_salvataggio

```
char StatoGioco::data_salvataggio[30]
```

5.79.2.15 deve_chiamare_uno

```
int StatoGioco::deve_chiamare_uno
```

5.79.2.16 direzione

```
int StatoGioco::direzione
```

5.79.2.17 durata_partita

```
float StatoGioco::durata_partita
```

5.79.2.18 giocatori

```
Giocatore StatoGioco::giocatori[4]
```

5.79.2.19 gioco_finito

```
int StatoGioco::gioco_finito
```

5.79.2.20 id_carta_in_trascinamento

```
int StatoGioco::id_carta_in_trascinamento
```

5.79.2.21 in_scelta_colore

```
int StatoGioco::in_scelta_colore
```

5.79.2.22 mazzo

```
Carta StatoGioco::mazzo[108]
```

5.79.2.23 messaggio

```
char StatoGioco::messaggio[100]
```

5.79.2.24 modalita

```
char StatoGioco::modalita[20]
```

5.79.2.25 nodo_carta_trascinata

```
NodoCarta* StatoGioco::nodo_carta_trascinata
```

5.79.2.26 notifica

```
char StatoGioco::notifica[128]
```

5.79.2.27 num_carte_mazzo

```
int StatoGioco::num_carte_mazzo
```

5.79.2.28 num_carte_scarti

```
int StatoGioco::num_carte_scarti
```

5.79.2.29 num_chat_msg

```
int StatoGioco::num_chat_msg
```

5.79.2.30 num_giocatori

```
int StatoGioco::num_giocatori
```

5.79.2.31 scarti

```
Carta StatoGioco::scarti[108]
```

5.79.2.32 slot_salvataggio

```
int StatoGioco::slot_salvataggio
```

5.79.2.33 statistiche_gia_salvate

```
int StatoGioco::statistiche_gia_salvate
```

5.79.2.34 timer_notifica

```
float StatoGioco::timer_notifica
```

5.79.2.35 timer_uno

```
float StatoGioco::timer_uno
```

5.79.2.36 tipo_animazione

```
int StatoGioco::tipo_animazione
```

5.79.2.37 turno_corrente

```
int StatoGioco::turno_corrente
```

The documentation for this struct was generated from the following file:

- [lib/data_structures.h](#)

5.80 StatsSyncPacket Struct Reference

[StatsSyncPacket](#) - Sincronizzazione live statistiche dopo una partita.

```
#include <network.h>
```

Data Fields

- char [username](#) [30]
- int [partite_giocate](#)
- int [vittorie_giocatore](#)
- int [vittorie_bot](#)
- float [durata_ultima_partita](#)
- char [modalita_ultima](#) [20]
- char [esito_ultima](#) [15]
- int [posizione_ultima](#)
- [PartitaRegistrata storico_partite](#) [STORICO_SYNC_COUNT]
- int [storico_count](#)

5.80.1 Detailed Description

[StatsSyncPacket](#) - Sincronizzazione live statistiche dopo una partita.

BUG 3 FIX: Il campo `storico_partite[]` (fino a `STORICO_SYNC_COUNT` partite) e `storico_count` sono stati aggiunti per trasmettere anche i dettagli delle partite recenti via TCP, non solo i contatori numerici. Il ricevente merge questi dati nel proprio `storico_partite[]` locale.

Il merge avviene nel packet handler `PACKET_STATS_SYNC` usando (durata, esito, modalita, posizione) come chiave composta di deduplica, identica alla logica usata in [lan_db_sync.c](#) e [lan_broadcast.c](#) per la sincronizzazione LAN.

IMPORTANTE: Il pacchetto [StatsSyncPacket](#) e' contenuto nell'unione [NetPacket](#) che include gia' [StatoGioco](#) (grande ~20KB), quindi l'aggiunta di un piccolo storico non impatta la dimensione totale del pacchetto.

The documentation for this struct was generated from the following file:

- `lib/socket/network/network.h`

5.81 TempSave Struct Reference

Struttura temporanea per ordinamento salvataggi (usata in `CompattaSalvataggiUtente`).

Data Fields

- char [percorso](#) [70]
- char [data](#) [30]

5.81.1 Detailed Description

Struttura temporanea per ordinamento salvataggi (usata in `CompattaSalvataggiUtente`).

5.81.2 Field Documentation

5.81.2.1 data

```
char TempSave::data[30]
```

5.81.2.2 percorso

```
char TempSave::percorso[70]
```

The documentation for this struct was generated from the following file:

- `src/game/save_manager.c`

5.82 Texture Struct Reference

```
#include <raylib.h>
```

Data Fields

- unsigned int [id](#)
- int [width](#)
- int [height](#)
- int [mipmaps](#)
- int [format](#)

5.82.1 Field Documentation

5.82.1.1 format

```
int Texture::format
```

5.82.1.2 height

```
int Texture::height
```

5.82.1.3 id

```
unsigned int Texture::id
```

5.82.1.4 mipmaps

```
int Texture::mipmaps
```

5.82.1.5 width

```
int Texture::width
```

The documentation for this struct was generated from the following file:

- lib/raylib/include/[raylib.h](#)

5.83 Transform Struct Reference

```
#include <raylib.h>
```

Data Fields

- [Vector3](#) translation
- [Quaternion](#) rotation
- [Vector3](#) scale

5.83.1 Field Documentation

5.83.1.1 rotation

```
Quaternion Transform::rotation
```

5.83.1.2 scale

```
Vector3 Transform::scale
```


5.83.1.3 translation

`Vector3 Transform::translation`

The documentation for this struct was generated from the following file:

- `lib/raylib/include/raylib.h`

5.84 Utente Struct Reference

Rappresenta un utente registrato nel sistema.

```
#include <data_structures.h>
```

Data Fields

- char `nome` [50]
- char `cognome` [50]
- char `username` [30]
- char `email` [100]
- char `password` [50]

5.84.1 Detailed Description

Rappresenta un utente registrato nel sistema.

Contiene i dati anagrafici e le credenziali di accesso. L'email funge da chiave primaria per il database.

5.84.2 Field Documentation

5.84.2.1 cognome

```
char Utente::cognome[50]
```

5.84.2.2 email

```
char Utente::email[100]
```

5.84.2.3 nome

```
char Utente::nome[50]
```

5.84.2.4 password

```
char Utente::password[50]
```

5.84.2.5 username

```
char Utente::username[30]
```

The documentation for this struct was generated from the following file:

- [lib/data_structures.h](#)

5.85 Vector2 Struct Reference

```
#include <raylib.h>
```

Data Fields

- float [x](#)
- float [y](#)

5.85.1 Field Documentation

5.85.1.1 x

```
float Vector2::x
```

5.85.1.2 y

```
float Vector2::y
```

The documentation for this struct was generated from the following files:

- [lib/raylib/include/raylib.h](#)
- [lib/raylib/include/raymath.h](#)

5.86 Vector3 Struct Reference

```
#include <raylib.h>
```

Data Fields

- float [x](#)
- float [y](#)
- float [z](#)

5.86.1 Field Documentation

5.86.1.1 x

```
float Vector3::x
```

5.86.1.2 y

```
float Vector3::y
```

5.86.1.3 z

```
float Vector3::z
```

The documentation for this struct was generated from the following files:

- lib/raylib/include/[raylib.h](#)
- lib/raylib/include/[raymath.h](#)

5.87 Vector4 Struct Reference

```
#include <raylib.h>
```

Data Fields

- float [x](#)
- float [y](#)
- float [z](#)
- float [w](#)

5.87.1 Field Documentation

5.87.1.1 w

```
float Vector4::w
```

5.87.1.2 x

```
float Vector4::x
```

5.87.1.3 y

```
float Vector4::y
```

5.87.1.4 z

```
float Vector4::z
```

The documentation for this struct was generated from the following files:

- lib/raylib/include/[raylib.h](#)
- lib/raylib/include/[raymath.h](#)

5.88 VrDeviceInfo Struct Reference

```
#include <raylib.h>
```

Data Fields

- int [hResolution](#)
- int [vResolution](#)
- float [hScreenSize](#)
- float [vScreenSize](#)
- float [eyeToScreenDistance](#)
- float [lensSeparationDistance](#)
- float [interpupillaryDistance](#)
- float [lensDistortionValues](#) [4]
- float [chromaAbCorrection](#) [4]

5.88.1 Field Documentation

5.88.1.1 chromaAbCorrection

```
float VrDeviceInfo::chromaAbCorrection[4]
```

5.88.1.2 eyeToScreenDistance

```
float VrDeviceInfo::eyeToScreenDistance
```

5.88.1.3 hResolution

```
int VrDeviceInfo::hResolution
```

5.88.1.4 hScreenSize

```
float VrDeviceInfo::hScreenSize
```

5.88.1.5 interpupillaryDistance

```
float VrDeviceInfo::interpupillaryDistance
```

5.88.1.6 lensDistortionValues

```
float VrDeviceInfo::lensDistortionValues[4]
```

5.88.1.7 lensSeparationDistance

```
float VrDeviceInfo::lensSeparationDistance
```

5.88.1.8 vResolution

```
int VrDeviceInfo::vResolution
```

5.88.1.9 vScreenSize

```
float VrDeviceInfo::vScreenSize
```

The documentation for this struct was generated from the following file:

- `lib/raylib/include/raylib.h`

5.89 VrStereoConfig Struct Reference

```
#include <raylib.h>
```

Data Fields

- `Matrix projection` [2]
- `Matrix viewOffset` [2]
- `float leftLensCenter` [2]
- `float rightLensCenter` [2]
- `float leftScreenCenter` [2]
- `float rightScreenCenter` [2]
- `float scale` [2]
- `float scaleIn` [2]

5.89.1 Field Documentation

5.89.1.1 leftLensCenter

```
float VrStereoConfig::leftLensCenter[2]
```

5.89.1.2 leftScreenCenter

```
float VrStereoConfig::leftScreenCenter[2]
```

5.89.1.3 projection

```
Matrix VrStereoConfig::projection[2]
```

5.89.1.4 rightLensCenter

```
float VrStereoConfig::rightLensCenter[2]
```

5.89.1.5 rightScreenCenter

```
float VrStereoConfig::rightScreenCenter[2]
```

5.89.1.6 scale

```
float VrStereoConfig::scale[2]
```

5.89.1.7 scaleIn

```
float VrStereoConfig::scaleIn[2]
```

5.89.1.8 viewOffset

```
Matrix VrStereoConfig::viewOffset[2]
```

The documentation for this struct was generated from the following file:

- lib/raylib/include/[raylib.h](#)

5.90 Wave Struct Reference

```
#include <raylib.h>
```

Data Fields

- unsigned int [frameCount](#)
- unsigned int [sampleRate](#)
- unsigned int [sampleSize](#)
- unsigned int [channels](#)
- void * [data](#)

5.90.1 Field Documentation

5.90.1.1 channels

```
unsigned int Wave::channels
```

5.90.1.2 data

```
void* Wave::data
```

5.90.1.3 frameCount

```
unsigned int Wave::frameCount
```

5.90.1.4 sampleRate

```
unsigned int Wave::sampleRate
```

5.90.1.5 sampleSize

```
unsigned int Wave::sampleSize
```

The documentation for this struct was generated from the following file:

- [lib/raylib/include/raylib.h](#)

Chapter 6

File Documentation

6.1 lib/auth.h File Reference

Modulo di autenticazione e gestione database utenti.

```
#include "data_structures.h"
```

Macros

- `#define MAX_PENDING_DELETIONS 64`
Numero massimo di eliminazioni pendenti.

Functions

- `int Auth_MarkForDeletion (const char *email)`
Marca un utente per eliminazione differita.
- `int Auth_IsMarkedForDeletion (const char *email)`
Verifica se un utente e' marcato per eliminazione.
- `void Auth_ProcessPendingDeletions (const char *email)`
Processa le eliminazioni pendenti per un utente.
- `int DB_Alloca (int min_slot)`
Alloca o rialloca il database per garantire almeno min_slot.
- `void SalvaDB (void)`
Salva il database su file binario con cifratura XOR.
- `void AuthBST_Ricostruisci (void)`
Ricostruisce il BST parallelo a partire dall'array dbUtenti.
- `void CaricaDB (void)`
Carica il database da file binario.
- `int Auth_RegistraUtente (const char *, const char *, const char *, const char *, const char *, const char *)`
Registra un nuovo utente dopo aver generato l'ID.
- `void Auth_RimuoviUtente (const char *email)`
Rimuove un utente da database e BST.
- `int AuthBST_CercaIndicePerEmail (const char *)`
Ricerca l'indice di un utente per email tramite BST.
- `int AuthBST_Altezza (void)`
Restituisce l'altezza del BST (per debug).
- `int AuthBST_ContaNodi (void)`
Restituisce il numero di nodi nel BST.

Variables

- [DatabaseUtenti dbUtenti](#)
Database utenti globale in memoria (allocazione dinamica).
- [int id_utente_corrente](#)
Indice dell'utente correntemente loggato (-1 se nessuno).
- [int nuovo_utente](#)
Flag nuovo utente (submit).
- [char inputEmail](#) [100]
- [char inputNome](#) [30]
- [char inputCognome](#) [30]
- [char inputUsername](#) [30]
- [char inputPassword](#) [30]
- [char inputPin](#) [7]
- [int emailCount](#)
- [int nomeCount](#)
- [int cognomeCount](#)
- [int usernameCount](#)
- [int passCount](#)
- [int pinCount](#)
- [int id_temp_login](#)
- [int errore_password](#)
- [int errore_registrazione_pass](#)
- [int errore_registrazione_email](#)
- [int errore_registrazione_username](#)
- [int campo_focusato](#)

6.1.1 Detailed Description

Modulo di autenticazione e gestione database utenti.

6.1.2 Variable Documentation

6.1.2.1 cognomeCount

```
int cognomeCount
```

6.1.2.2 errore_registrazione_pass

```
int errore_registrazione_pass
```

6.1.2.3 errore_registrazione_username

```
int errore_registrazione_username
```

6.1.2.4 id_temp_login

```
int id_temp_login
```

6.1.2.5 inputCognome

```
char inputCognome[30]
```

6.1.2.6 inputNome

```
char inputNome[30]
```

6.1.2.7 inputPassword

```
char inputPassword[30]
```

6.1.2.8 inputPin

```
char inputPin[7]
```

6.1.2.9 nomeCount

```
int nomeCount
```

6.1.2.10 nuovo_utente

```
int nuovo_utente
```

Flag nuovo utente (submit).

Flag nuovo utente (submit).

6.1.2.11 passCount

```
int passCount
```

6.1.2.12 pinCount

```
int pinCount
```

6.2 auth.h

[Go to the documentation of this file.](#)

```

00001
00014
00015 #ifndef AUTH_H
00016 #define AUTH_H
00017 #include "data_structures.h"
00018
00019 extern DatabaseUtenti dbUtenti;
00020 extern int id_utente_corrente, nuovo_utente;
00021 extern char inputEmail[100], inputNome[30], inputCognome[30];
00022 extern char inputUsername[30], inputPassword[30], inputPin[7];
00023 extern int emailCount, nomeCount, cognomeCount;
00024 extern int usernameCount, passCount, pinCount, id_temp_login;
00025 extern int errore_password, errore_registrazione_pass;
00026 extern int errore_registrazione_email, errore_registrazione_username;
00027 extern int campo_focusato;
00028
00035 #define MAX_PENDING_DELETIONS 64
00037 int Auth_MarkForDeletion(const char* email);
00039 int Auth_IsMarkedForDeletion(const char* email);
00041 void Auth_ProcessPendingDeletions(const char* email);
00043
00050 int DB_Alloca(int min_slot);
00056 void SalvaDB(void);
00058 void AuthBST_Ricostruisci(void);
00064 void CaricaDB(void);
00077 int Auth_RegistraUtente(const char*, const char*, const char*,
00078                        const char*, const char*, const char*);
00085 void Auth_RimuoviUtente(const char* email);
00087 int AuthBST_CercaIndicePerEmail(const char*);
00089 int AuthBST_Altezza(void);
00091 int AuthBST_ContaNodi(void);
00093 #endif
00094

```

6.3 lib/auth_validation.h File Reference

Modulo di validazione per l'autenticazione.

```
#include "data_structures.h"
```

Functions

- int [Auth_EmailFormalmenteValida](#) (const char *email)
Verifica se un'email e' formalmente valida.
- int [Auth_PasswordValida](#) (const char *password, int len)
Verifica se una password soddisfa i requisiti minimi.
- int [Auth_PINValido](#) (const char *pin, int len)
Verifica se un PIN di recupero e' valido.
- int [Auth_UsernameNonVuoto](#) (const char *username, int len)
Verifica se uno username non e' vuoto.
- int [Auth_UsernameGiaUsato](#) (const char *username)
Verifica se uno username e' gia' in uso nel database.
- int [Auth_EmailGiaUsata](#) (const char *email, int ignora_indice)
Verifica se un'email e' gia' in uso nel database.
- int [Auth_VerificaCredenziali](#) (const char *username, const char *password)
Cerca un utente nel database per username e password.
- int [Auth_VerificaEmailPIN](#) (const char *email, const char *pin)
Cerca un utente nel database per email e PIN.

6.3.1 Detailed Description

Modulo di validazione per l'autenticazione.

6.3.2 Function Documentation

6.3.2.1 Auth_EmailFormalmenteValida()

```
int Auth_EmailFormalmenteValida (  
    const char * email)
```

Verifica se un'email e' formalmente valida.

Contiene almeno un carattere prima della '@' e almeno uno dopo.

Precondition

email != NULL.

Postcondition

Restituisce 1 se valida, 0 altrimenti.

Parameters

	<i>email</i>	Stringa email da validare.
--	--------------	----------------------------

Returns

1 se valida, 0 altrimenti.

Controlla che la stringa contenga almeno un carattere '@'.

Precondition

email != NULL.

Postcondition

Restituisce 1 se contiene '@', 0 altrimenti.

Parameters

	<i>email</i>	Stringa email da validare.
--	--------------	----------------------------

Returns

1 se valida, 0 altrimenti.

6.3.2.2 Auth_EmailGiaUsata()

```
int Auth_EmailGiaUsata (  
    const char * email,  
    int ignora_indice)
```

Verifica se un'email e' gia' in uso nel database.

Scansiona dbUtenti.lista[] alla ricerca di un'email duplicata (case-insensitive). Il parametro ignora_indice permette di escludere un utente dal controllo (utile nella modifica profilo per non bloccare il proprio indirizzo email).

Precondition

dbUtenti inizializzato.

Postcondition

Restituisce 1 se duplicata, 0 se disponibile.

Parameters

<i>email</i>	Email da verificare.
<i>ignora_indice</i>	Indice dell'utente da ignorare (-1 per controllare tutti).

Returns

1 se gia' in uso, 0 se disponibile.

6.3.2.3 Auth_PasswordValida()

```
int Auth_PasswordValida (  
    const char * password,  
    int len)
```

Verifica se una password soddisfa i requisiti minimi.

Requisiti: almeno 8 caratteri, almeno una lettera maiuscola.

Precondition

password != NULL.

Postcondition

Restituisce 1 se valida, 0 altrimenti.

Parameters

<i>password</i>	Stringa password da validare.
<i>len</i>	Lunghezza della password.

Returns

1 se valida, 0 altrimenti.

Controlla lunghezza ≥ 8 caratteri e presenza di almeno una lettera maiuscola.

Precondition

password != NULL.

Postcondition

Restituisce 1 se lunghezza ≥ 8 e contiene almeno una maiuscola.

Parameters

<i>password</i>	Stringa password da validare.
<i>len</i>	Lunghezza della password.

Returns

1 se valida, 0 altrimenti.

6.3.2.4 Auth_PINValido()

```
int Auth_PINValido (  
    const char * pin,  
    int len)
```

Verifica se un PIN di recupero e' valido.

Deve essere esattamente 6 cifre.

Precondition

pin != NULL.

Postcondition

Restituisce 1 se *len* >= 6, 0 altrimenti.

Parameters

<i>pin</i>	Stringa PIN da validare.
<i>len</i>	Lunghezza del PIN.

Returns

1 se valido, 0 altrimenti.

Precondition

pin != NULL.

Postcondition

Restituisce 1 se *len* >= 6, 0 altrimenti.

Parameters

<i>pin</i>	Stringa PIN da validare.
<i>len</i>	Lunghezza del PIN.

Returns

1 se valido, 0 altrimenti.

6.3.2.5 Auth_UsernameGiaUsato()

```
int Auth_UsernameGiaUsato (  
    const char * username)
```

Verifica se uno username e' gia' in uso nel database.

Scansiona dbUtenti.lista[] alla ricerca di un username duplicato.

Precondition

dbUtenti inizializzato.

Postcondition

Restituisce 1 se trovato, 0 altrimenti.

Parameters

	<i>username</i>	Stringa username da verificare.
--	-----------------	---------------------------------

Returns

1 se gia' in uso, 0 se disponibile.

6.3.2.6 Auth_UsernameNonVuoto()

```
int Auth_UsernameNonVuoto (  
    const char * username,  
    int len)
```

Verifica se uno username non e' vuoto.

Precondition

username != NULL.

Postcondition

Restituisce 1 se len > 0, 0 altrimenti.

Parameters

	<i>username</i>	Stringa username da verificare.
	<i>len</i>	Lunghezza dello username.

Returns

1 se non vuoto, 0 altrimenti.

6.3.2.7 Auth_VerificaCredenziali()

```
int Auth_VerificaCredenziali (
    const char * username,
    const char * password)
```

Cerca un utente nel database per username e password.

Scansiona dbUtenti.lista[] alla ricerca di una corrispondenza. La password viene confrontata dopo eventuale decifrazione XOR.

Precondition

dbUtenti inizializzato.

Postcondition

Restituisce indice, -1 o -2 in base al risultato.

Parameters

<i>username</i>	Username inserito dall'utente.
<i>password</i>	Password inserita dall'utente.

Returns

Indice dell'utente trovato, o -1 se non trovato, -2 se password errata.

Scansiona dbUtenti.lista[] alla ricerca di una corrispondenza. La password viene confrontata in chiaro (eventuale decifrazione XOR gestita esternamente).

Precondition

dbUtenti inizializzato.

Postcondition

Restituisce indice, -1 o -2 in base al risultato.

Parameters

<i>username</i>	Username inserito dall'utente.
<i>password</i>	Password inserita dall'utente.

Returns

Indice dell'utente trovato, -1 se non trovato, -2 se password errata.

6.3.2.8 Auth_VerificaEmailPIN()

```
int Auth_VerificaEmailPIN (  
    const char * email,  
    const char * pin)
```

Cerca un utente nel database per email e PIN.

Usato per il recupero password. Confronta email e PIN di recupero.

Precondition

dbUtenti inizializzato.

Postcondition

Restituisce indice se trovato, -1 altrimenti.

Parameters

<i>email</i>	Email inserita dall'utente.
<i>pin</i>	PIN di recupero inserito.

Returns

Indice dell'utente trovato, o -1 se non trovato.

Usato per il recupero password. Confronta email (case-insensitive) e PIN.

Precondition

dbUtenti inizializzato.

Postcondition

Restituisce indice se trovato, -1 altrimenti.

Parameters

<i>email</i>	Email inserita dall'utente.
<i>pin</i>	PIN di recupero inserito.

Returns

Indice dell'utente trovato, o -1 se non trovato.

6.4 auth_validation.h

[Go to the documentation of this file.](#)

```

00001
00011
00012 #ifndef AUTH_VALIDATION_H
00013 #define AUTH_VALIDATION_H
00014
00015 #include "data_structures.h"
00016
00017 /* =====
00018  *  VALIDAZIONE CAMPI INPUT
00019  *  ===== */
00020
00031 int Auth_EmailFormalmenteValida(const char* email);
00032
00044 int Auth_PasswordValida(const char* password, int len);
00045
00057 int Auth_PINValido(const char* pin, int len);
00058
00068 int Auth_UsernameNonVuoto(const char* username, int len);
00069
00080 int Auth_UsernameGiaUsato(const char* username);
00081
00095 int Auth_EmailGiaUsata(const char* email, int ignora_indice);
00096
00097 /* =====
00098  *  VERIFICA CREDENZIALI
00099  *  ===== */
00100
00113 int Auth_VerificaCredenziali(const char* username, const char* password);
00114
00126 int Auth_VerificaEmailPIN(const char* email, const char* pin);
00127
00128
00129 #endif /* AUTH_VALIDATION_H */

```

6.5 lib/data_structures.h File Reference

Header principale del progetto UNO - Definizione di tutti i tipi di dato.

```

#include "raylib.h"
#include <stddef.h>
#include <string.h>
#include <stdio.h>

```

Data Structures

- struct [Carta](#)
Rappresenta una singola carta UNO.
- struct [NodoCarta](#)
Nodo per la lista doppiamente concatenata della mano del giocatore.
- struct [ListaCarte](#)
Struttura container per la lista di carte (mano del giocatore).
- struct [Utente](#)
Rappresenta un utente registrato nel sistema.
- struct [Giocatore](#)
Rappresenta un giocatore nella partita (umano o bot).
- struct [StatoGioco](#)
Stato completo di una partita in corso.
- struct [PartitaRegistrata](#)

- Record di una singola partita nello storico di un utente.
- struct [Statistiche](#)
Statistiche complete di un utente registrato.
- struct [DatabaseUtenti](#)
Database degli utenti in memoria (allocazione DINAMICA).
- struct [MessaggioChat](#)
Singolo messaggio nella chat multiplayer.
- struct [ClassificaRecord](#)
Record per la classifica visualizzata a fine partita.

Macros

- #define [DB_CAPACITA_INIZIALE](#) 64
Capacita' iniziale del database utenti (allocazione dinamica).
- #define [MAX_PARTITE_STORICO](#) 100
Numero massimo di partite nello storico di ogni utente.
- #define [GIOCATORE_NOME_BUF](#) 60
Dimensione del buffer per Giocatore_NomeBot.

Typedefs

- typedef struct NodoCarta [NodoCarta](#)
Nodo per la lista doppiamente concatenata della mano del giocatore.
- typedef [NodoCarta](#) [NodoMano](#)
Alias per [NodoCarta](#) (usato nella documentazione CdS come [NodoMano](#)).

Enumerations

- enum [Colore](#) {
 [ROSSO](#) , [GIALLO](#) , [VERDE](#) , [BLU](#) ,
 [NERO](#) }
Colori disponibili per le carte UNO.
- enum [TipoCarta](#) {
 [NUM_0](#) , [NUM_1](#) , [NUM_2](#) , [NUM_3](#) ,
 [NUM_4](#) , [NUM_5](#) , [NUM_6](#) , [NUM_7](#) ,
 [NUM_8](#) , [NUM_9](#) , [SALTA](#) , [CAMBIA_GIRO](#) ,
 [PESCA_DUE](#) , [CAMBIO_COLORE](#) , [PESCA_QUATTRO](#) }
Tipi di carta UNO.
- enum [FaseApplicazione](#) {
 [FASE_SCELTA_ACCESSO](#) , [FASE_REGISTRAZIONE](#) , [FASE_LOGIN](#) , [FASE_RECUPERO_PASSWORD](#) ,
 [FASE_PROFILO_VISUALIZZA](#) , [FASE_PROFILO_MODIFICA](#) , [FASE_ADMIN_PANEL](#) , [FASE_ADMIN_MODIFICA_UTENTE](#)
 ,
 [FASE_MENU](#) , [FASE_GIOCO](#) , [FASE_FINE_PARTITA](#) , [FASE_STATISTICHE](#) ,
 [FASE_MULTIPLAYER_SCELTA](#) , [FASE_HOST_LOBBY](#) , [FASE_CLIENT_JOIN](#) , [FASE_CLIENT_LOBBY](#) }
Enum che rappresenta tutte le fasi dello stato macchina dell'applicazione.

6.5.1 Detailed Description

Header principale del progetto UNO - Definizione di tutti i tipi di dato.

6.5.2 Macro Definition Documentation

6.5.2.1 DB_CAPACITA_INIZIALE

```
#define DB_CAPACITA_INIZIALE 64
```

Capacita' iniziale del database utenti (allocazione dinamica).

6.5.2.2 GIOCATORE_NOME_BUF

```
#define GIOCATORE_NOME_BUF 60
```

Dimensione del buffer per Giocatore_NomeBot.

6.5.2.3 MAX_PARTITE_STORICO

```
#define MAX_PARTITE_STORICO 100
```

Numero massimo di partite nello storico di ogni utente.

6.5.3 Typedef Documentation

6.5.3.1 NodoCarta

```
typedef struct NodoCarta NodoCarta
```

Nodo per la lista doppiamente concatenata della mano del giocatore.

Ogni nodo contiene una carta e due puntatori: uno al nodo successivo (forward link) e uno al nodo precedente (backward link). Questo permette inserimenti ed eliminazioni efficienti in qualsiasi posizione.

6.5.3.2 NodoMano

```
typedef NodoCarta NodoMano
```

Alias per [NodoCarta](#) (usato nella documentazione CdS come [NodoMano](#)).

6.5.4 Enumeration Type Documentation

6.5.4.1 Colore

```
enum Colore
```

Colori disponibili per le carte UNO.

NERO = jolly/cambio colore.

Enumerator

	ROSSO
	GIALLO
	VERDE
	BLU
	NERO

6.5.4.2 FaseApplicazione

enum `FaseApplicazione`

Enum che rappresenta tutte le fasi dello stato macchina dell'applicazione.

Ogni fase corrisponde a una schermata o modalita' dell'applicazione. Le transizioni tra le fasi sono gestite dalle funzioni `AggiornaAuth` e `AggiornaGameplay`.

Enumerator

	FASE_SCELTA_ACCESSE
	FASE_REGISTRAZIONE
	FASE_LOGIN
	FASE_RECUPERO_PASSWORD
	FASE_PROFILO_VISUALIZZA
	FASE_PROFILO_MODIFICA
	FASE_ADMIN_PANEL
	FASE_ADMIN_MODIFICA_UTENTE
	FASE_MENU
	FASE_GIOCO
	FASE_FINE_PARTITA
	FASE_STATISTICHE
	FASE_MULTIPLAYER_SCELTA
	FASE_HOST_LOBBY
	FASE_CLIENT_JOIN
	FASE_CLIENT_LOBBY

6.5.4.3 TipoCarta

enum `TipoCarta`

Tipi di carta UNO.

Da NUM_0 a NUM_9 = carte normali, poi speciali.

Enumerator

	NUM_0
	NUM_1
	NUM_2
	NUM_3
	NUM_4
	NUM_5

Enumerator

	NUM_6
	NUM_7
	NUM_8
	NUM_9
	SALTA
CAMBIA_GIRO	
PESCA_DUE	
CAMBIO_COLORE	
PESCA_QUATTRO	

6.6 data_structures.h

[Go to the documentation of this file.](#)

```

00001
00016
00017 #ifndef DATA_STRUCTURES_H
00018 #define DATA_STRUCTURES_H
00019
00020 #include "raylib.h"
00021 #include <stddef.h> /* per size_t */
00022 #include <string.h> /* per strncpy usato nelle inline functions */
00023 #include <stdio.h> /* per snprintf in Giocatore_NomeBot */
00024
00026 #define DB_CAPACITA_INIZIALE 64
00027
00029 #define MAX_PARTITE_STORICO 100
00030
00031 /* =====
00032 * COLORI DELLE CARTE (Enum)
00033 * Riferimento: Documentazione CdS - "Gestione carta"
00034 * ===== */
00035
00037 typedef enum { ROSSO, GIALLO, VERDE, BLU, NERO } Colore;
00038
00039 /* =====
00040 * TIPI DI CARTA (Enum)
00041 * Riferimento: Documentazione CdS - "Gestione carta"
00042 * ===== */
00043
00045 typedef enum {
00046     NUM_0, NUM_1, NUM_2, NUM_3, NUM_4, NUM_5, NUM_6, NUM_7, NUM_8, NUM_9,
00047     SALTA, /* Salta il prossimo giocatore */
00048     CAMBIA_GIRO, /* Inverte la direzione del turno */
00049     PESCA_DUE, /* Il prossimo giocatore pesca 2 carte */
00050     CAMBIO_COLORE, /* Jolly: scegli un colore */
00051     PESCA_QUATTRO /* Jolly +4: il prossimo pesca 4 carte e scegli un colore */
00052 } TipoCarta;
00053
00054 /* =====
00055 * CARTA - Entita' di dominio fondamentale
00056 * Riferimento: Documentazione CdS - "Gestione carta"
00057 * ===== */
00058
00066 typedef struct {
00067     Colore colore; /* Colore della carta (ROSSO, GIALLO, VERDE, BLU, NERO) */
00068     TipoCarta tipo; /* Tipo della carta (NUM_0-NUM_9, SALTA, CAMBIA_GIRO, ecc.) */
00069     Rectangle area; /* Rettangolo di rendering per Raylib */
00070     Vector2 posOriginale; /* Posizione originale prima del trascinamento */
00071     int isTrascinata; /* 1 se la carta e' in fase di trascinamento, 0 altrimenti */
00072 } Carta;
00073
00074 /* =====
00075 * NODO CARTA - Lista DOPPIAMENTE concatenata - ADT 1/4
00076 * Riferimento: Documentazione CdS - "NodoMano"
00077 * "Gestione nodo per la mano del giocatore (lista doppiamente concatenata)"
00078 * ===== */
00079
00087 typedef struct NodoCarta {
00088     Carta carta; /* La carta contenuta nel nodo */
00089     struct NodoCarta* prossimo; /* Puntatore al nodo successivo (next) */
00090     struct NodoCarta* prev; /* Puntatore al nodo precedente (prev) */

```

```

00091 } NodoCarta;
00092
00094 typedef NodoCarta NodoMano;
00095
00102 typedef struct {
00103     NodoCarta* testa;        /* Primo nodo della lista */
00104     NodoCarta* coda;        /* Ultimo nodo della lista */
00105     int lunghezza;          /* Numero totale di carte nella lista */
00106 } ListaCarte;
00107
00108 /* =====
00109  * UTENTE - Entita' di dominio
00110  * Riferimento: Documentazione CdS - "Gestione utente"
00111  * ===== */
00112
00119 typedef struct {
00120     char nome[50];           /* Nome dell'utente */
00121     char cognome[50];        /* Cognome dell'utente */
00122     char username[30];       /* Username univoco per il login */
00123     char email[100];         /* Email (chiave primaria per il database) */
00124     char password[50];       /* Password (eventualmente cifrata con XOR) */
00125 } Utente;
00126
00127 /* =====
00128  * GIOCATORE - Entita' di flusso/stato
00129  * Riferimento: Documentazione CdS - "Gestione giocatore"
00130  * ===== */
00131
00144 typedef struct {
00145     Utente info;             /* Dati anagrafici completi dell'utente */
00146     char nome[30];           /* Nome breve (alias per retrocompatibilita') */
00147     ListaCarte* mano;        /* Puntatore alla lista delle carte in mano (ADT Lista) */
00148     int is_bot;              /* 1 se controllato dall'IA, 0 se umano */
00149     int stato_pronto;        /* 1 = Pronto nella lobby, 0 = In attesa */
00150     int num_carte_mano;       /* Numero di carte (usato a fine partita per classifica) */
00151     int socket_id;           /* ID socket del giocatore (-1 se bot o locale) */
00152     int era_in_prelobby;     /* 1 se il giocatore era presente in prelobby prima di iniziare (per
riconconnessione) */
00153     /* Campi per serializzazione: backup flat delle carte mano */
00154     Carta mano_backup[108];  /* Backup flat delle carte mano per salvataggio su file */
00155     int num_carte_backup;     /* Numero di carte valide in mano_backup */
00156 } Giocatore;
00157
00158 /* =====
00159  * HELPER INLINE PER GIOCATORE
00160  * ===== */
00161
00167 static inline const char* Giocatore_Nome(const Giocatore* g) {
00168     return g->nome[0] ? g->nome : "Sconosciuto";
00169 }
00170
00172 #define GIOCATORE_NOME_BUF 60
00173
00179 static inline const char* Giocatore_NomeBot(const Giocatore* g) {
00180     static char buf[GIOCATORE_NOME_BUF];
00181     if (g->is_bot) {
00182         snprintf(buf, GIOCATORE_NOME_BUF, "%s (Bot)", Giocatore_Nome(g));
00183         return buf;
00184     }
00185     return Giocatore_Nome(g);
00186 }
00187
00196 static inline void Giocatore_SetNome(Giocatore* g, const char* nome) {
00197     if (g && nome) {
00198         strncpy(g->nome, nome, sizeof(g->nome) - 1);
00199         strncpy(g->info.nome, nome, sizeof(g->info.nome) - 1);
00200         strncpy(g->info.username, nome, sizeof(g->info.username) - 1);
00201     }
00202 }
00203
00204 /* =====
00205  * STATO GIOCO - Stato completo della partita in corso
00206  * ===== */
00207
00216 typedef struct {
00217     Giocatore giocatori[4];  /* Array dei giocatori (max 4) */
00218     int num_giocatori;        /* Numero effettivo di giocatori nella partita */
00219     Carta mazzo[108];         /* Mazzo di pesca (max 108 carte UNO standard) */
00220     int num_carte_mazzo;      /* Carte rimanenti nel mazzo di pesca */
00221     Carta scarti[108];        /* Pila degli scarti (max 108) */
00222     int num_carte_scarti;     /* Carte nella pila degli scarti */
00223     int turno_corrente;       /* Indice del giocatore di turno */
00224     int direzione;            /* 1 = orario, -1 = antiorario (CambioGiro) */
00225     Colore colore_attivo;      /* Colore attualmente valido per giocare */
00226     int id_carta_in_trascinamento; /* Indice della carta trascinata (-1 = nessuna) */
00227     NodoCarta* nodo_carta_trascinata; /* Puntatore al nodo della carta trascinata */
00228     int gioco_finito;         /* 1 se la partita e' terminata */

```



```

00229     char messaggio[100];                /* Messaggio da mostrare (es. "HA VINTO X!") */
00230
00231     /* --- Stato animazioni --- */
00232     int anim_attiva;                     /* 1 se un'animazione e' in corso */
00233     Carta anim_carta;                   /* Carta coinvolta nell'animazione */
00234     Vector2 anim_start;                 /* Punto di partenza dell'animazione */
00235     Vector2 anim_end;                   /* Punto di arrivo dell'animazione */
00236     float anim_t;                       /* Progresso animazione (0.0 - 1.0) */
00237     int tipo_animazione;                 /* 0 = normale, 1 = animazione speciale */
00238
00239     /* --- Stato scelta colore e UNO --- */
00240     int in_scelta_colore;                /* 1 se il giocatore deve scegliere un colore */
00241     Carta carta_pendente;                /* Carta giocata in attesa della scelta del colore */
00242     int deve_chiamare_uno;               /* ID+1 del giocatore che deve premere UNO */
00243     float timer_uno;                    /* Timer countdown per il pulsante UNO */
00244     char notifica[128];                  /* Testo della notifica temporanea (aumentato da 50 a 128 per
evitare overflow con nomi lunghi) */
00245     float timer_notifica;                /* Timer di visualizzazione della notifica */
00246     int autore_animazione;               /* ID del giocatore che ha scatenato l'animazione */
00247     int anim_pesca_count;                /* Quante carte pescare quando animazione DRAW termina */
00248     int anim_gia_applicata;              /* 1 = stato gia' applicato dal packet_handler */
00249
00250     /* --- Stato chat --- */
00251     char chat_buffer[256];                /* Buffer per il messaggio chat in scrittura */
00252
00253     /* --- Informazioni partita --- */
00254     float durata_partita;                /* Durata in secondi della partita in corso */
00255     char chat_history[15][128];           /* Cronologia ultimi 15 messaggi chat */
00256     int num_chat_msg;                    /* Numero di messaggi nella cronologia */
00257     int chat_aperta;                     /* 1 se il pannello chat e' visibile */
00258     int slot_salvataggio;                 /* Slot di salvataggio corrente */
00259     char data_salvataggio[30];            /* Data/ora del salvataggio */
00260     int statistiche_gia_salvate;          /* 1 se le statistiche fine-partita sono gia' state salvate */
00261     char modalita[20];                    /* BUG 3 FIX: Modalita' di gioco salvata (es. "1 vs 3", "Online")
per ripristino corretto al caricamento della partita. */
00262 } StatoGioco;
00263
00264 /* =====
00265 * PARTITA REGISTRATA - Storico di una singola partita
00266 * ===== */
00267
00268 typedef struct {
00269     char modalita[20];                    /* Modalita' di gioco (es. "1 vs 2", "Online") */
00270     char esito[15];                       /* Esito della partita (es. "Vittoria", "Sconfitta") */
00271     int durata_secondi;                   /* Durata della partita in secondi */
00272     int posizione;                        /* Piazzamento finale (1 = vincitore, 2+ = perdenti) */
00273 } PartitaRegistrata;
00274
00275 /* =====
00276 * STATISTICHE - Dati anagrafici + cronologia partite
00277 * Riferimento: Documentazione CdS - "Gestione statistiche"
00278 * ===== */
00279
00280 typedef struct {
00281     char email[100];                      /* Email dell'utente (chiave primaria) */
00282     char nome[30];                        /* Nome dell'utente */
00283     char cognome[30];                     /* Cognome dell'utente */
00284     char username[30];                     /* Username univoco */
00285     char password[30];                     /* Password (cifrata) */
00286     char pin_recupero[7];                  /* PIN di 6 cifre per recupero password */
00287     int partite_giocate;                    /* Contatore totale partite giocate */
00288     int vittorie_giocatore;                 /* Contatore vittorie */
00289     int vittorie_bot;                       /* Contatore sconfitte (perdite contro bot) */
00290     PartitaRegistrata storico_partite[MAX_PARTITE_STORICO]; /* Cronologia partite */
00291 } Statistiche;
00292
00293 typedef struct {
00294     Statistiche* lista;                    /* Array DINAMICO degli utenti registrati (malloc/realloc) */
00295     int num_utenti;                        /* Numero effettivo di utenti nel database */
00296     int capacita;                          /* Capacita' corrente dell'array allocato */
00297 } DatabaseUtenti;
00298
00299 /* =====
00300 * MESSAGGIO CHAT - Entita' di flusso multiplayer
00301 * Riferimento: Documentazione CdS - "Gestione interazioni multiplayer"
00302 * ===== */
00303
00304 typedef struct {
00305     char mittente[30];                     /* Username di chi ha inviato il messaggio */
00306     char testo[256];                       /* Contenuto del messaggio */
00307     char orario[10];                       /* Timestamp (es. "14:35") */
00308 } MessaggioChat;
00309
00310 /* =====
00311 * CLASSIFICA RECORD - Record per la classifica fine partita
00312 * ===== */
00313

```

```

00346 typedef struct {
00347     char nome[30];          /* Nome del giocatore */
00348     int num_carte;          /* Numero di carte rimaste in mano */
00349     int posizione;          /* Posizione finale (1 = vincitore) */
00350     int is_bot;             /* 1 se e' un bot, 0 se umano */
00351 } ClassificaRecord;
00352
00353 /* =====
00354  * FASE APPLICAZIONE - Enum per lo stato macchina
00355  * ===== */
00356
00363 typedef enum {
00364     FASE_SCELTA_ACCESSO,      /* Schermata iniziale: Login / Registrazione / Esci */
00365     FASE_REGISTRAZIONE,      /* Form di registrazione nuovo utente */
00366     FASE_LOGIN,              /* Form di login */
00367     FASE_RECUPERO_PASSWORD,   /* Recupero password tramite PIN */
00368     FASE_PROFILO_VISUALIZZA,  /* Visualizzazione profilo utente */
00369     FASE_PROFILO_MODIFICA,    /* Modifica profilo utente */
00370     FASE_ADMIN_PANEL,        /* Pannello di controllo admin */
00371     FASE_ADMIN_MODIFICA_UTENTE, /* Modifica/crea utente da admin */
00372     FASE_MENU,               /* Menu principale dopo il login */
00373     FASE_GIOCO,              /* Gameplay loop (tavolo verde) */
00374     FASE_FINE_PARTITA,        /* Schermata di fine partita con classifica */
00375     FASE_STATISTICHE,         /* Schermata statistiche e storico */
00376     FASE_MULTIPLAYER_SCELTA,  /* Scelta modalita' multiplayer (host/client) */
00377     FASE_HOST_LOBBY,          /* Lobby come host (in attesa di giocatori) */
00378     FASE_CLIENT_JOIN,         /* Join a una stanza multiplayer */
00379     FASE_CLIENT_LOBBY         /* Lobby come client (in attesa di avvio) */
00380 } FaseApplicazione;
00381
00382 #endif

```

6.7 lib/data_structures/bst.h File Reference

Albero Binario di Ricerca (BST) per ricerca utenti per email $O(\log n)$.

```
#include "../data_structures.h"
```

Data Structures

- struct [NodoAlbero](#)
Nodo dell'albero BST: contiene un [Utente](#) e puntatori ai figli.

Typedefs

- typedef struct [NodoAlbero](#) [NodoAlbero](#)
Nodo dell'albero BST: contiene un [Utente](#) e puntatori ai figli.

Functions

- [NodoAlbero](#) * [BST_CreaNodo](#) ([Utente](#) u)
Crea un nuovo nodo BST a partire da un [Utente](#).
- [NodoAlbero](#) * [BST_Inserisci](#) ([NodoAlbero](#) *radice, [Utente](#) u, int *duplicato)
Inserisce un utente nell'albero.
- [NodoAlbero](#) * [BST_RicercaEmail](#) ([NodoAlbero](#) *radice, const char *email)
Ricerca un utente per email.
- [NodoAlbero](#) * [BST_RimuoviEmail](#) ([NodoAlbero](#) *radice, const char *email)
Rimuove il nodo con l'email specificata.
- void [BST_Distruggi](#) ([NodoAlbero](#) *radice)

- *Libera ricorsivamente tutta la memoria dell'albero.*
- `int BST_Inordina (NodoAlbero *radice, Utente *out, int max)`
Visita in-order e copia i puntatori agli Utenti in un array.
- `int BST_Altezza (NodoAlbero *radice)`
Restituisce l'altezza dell'albero.
- `int BST_ContaNodi (NodoAlbero *radice)`
Restituisce il numero di nodi nell'albero.

6.7.1 Detailed Description

Albero Binario di Ricerca (BST) per ricerca utenti per email $O(\log n)$.

6.8 bst.h

[Go to the documentation of this file.](#)

```

00001
00008 #ifndef DATA_STRUCTURES_BST_H
00009 #define DATA_STRUCTURES_BST_H
00010
00011 #include "../data_structures.h"
00012
00019 typedef struct NodoAlbero {
00020     Utente utente;
00021     struct NodoAlbero* left;
00022     struct NodoAlbero* right;
00023 } NodoAlbero;
00025
00038 NodoAlbero* BST_CreaNodo(Utente u);
00039
00049 NodoAlbero* BST_Inserisci(NodoAlbero* radice, Utente u, int* duplicato);
00050
00059 NodoAlbero* BST_RicercaEmail(NodoAlbero* radice, const char* email);
00060
00069 NodoAlbero* BST_RimuoviEmail(NodoAlbero* radice, const char* email);
00070
00077 void BST_Distruggi(NodoAlbero* radice);
00078
00088 int BST_Inordina(NodoAlbero* radice, Utente* out, int max);
00089
00095 int BST_Altezza(NodoAlbero* radice);
00096
00102 int BST_ContaNodi(NodoAlbero* radice);
00104
00105 #endif // DATA_STRUCTURES_BST_H

```

6.9 lib/data_structures/chat.h File Reference

ADT per la gestione della cronologia chat multiplayer.

```
#include "../data_structures.h"
```

Data Structures

- struct `ChatStorico`
Cronologia circolare (FIFO) di messaggi.

Macros

- `#define CHAT_STORICO_MAX 15`
Numero massimo di messaggi nella cronologia chat.

Functions

- void `Chat_Crea` (`ChatStorico *c`)
Crea uno storico chat vuoto.
- void `Chat_Aggiungi` (`ChatStorico *c`, `const char *mittente`, `const char *testo`)
Aggiunge un messaggio in coda (FIFO).
- void `Chat_OraCorrente` (`char *out`, `size_t out_size`)
Genera un timestamp "HH:MM" nel buffer fornito.
- const `MessaggioChat * Chat_Ottieni` (`const ChatStorico *c`, `int i`)
Restituisce il messaggio in posizione i della cronologia.
- void `Chat_Svuota` (`ChatStorico *c`)
Svuota la cronologia chat.
- void `Chat_FormattaRiga` (`const MessaggioChat *m`, `char *out`, `size_t out_size`)
Formatta un `MessaggioChat` in una riga "HH:MM @Mittente: testo".

6.9.1 Detailed Description

ADT per la gestione della cronologia chat multiplayer.

6.9.2 Macro Definition Documentation

6.9.2.1 CHAT_STORICO_MAX

```
#define CHAT_STORICO_MAX 15
```

Numero massimo di messaggi nella cronologia chat.

6.10 chat.h

[Go to the documentation of this file.](#)

```
00001
00010 #ifndef DATA_STRUCTURES_CHAT_H
00011 #define DATA_STRUCTURES_CHAT_H
00012
00013 #include "../data_structures.h"
00014
00016 #define CHAT_STORICO_MAX 15
00017
00028 typedef struct {
00030     MessaggioChat buffer[CHAT_STORICO_MAX];
00032     int count;
00034     int head;
00035 } ChatStorico;
00037
00049 void Chat_Crea(ChatStorico* c);
00050
00062 void Chat_Aggiungi(ChatStorico* c, const char* mittente, const char* testo);
00063
00071 void Chat_OraCorrente(char* out, size_t out_size);
00072
00083 const MessaggioChat* Chat_Ottieni(const ChatStorico* c, int i);
00084
00091 void Chat_Svuota(ChatStorico* c);
00092
00104 void Chat_FormattaRiga(const MessaggioChat* m, char* out, size_t out_size);
00106
00107 #endif // DATA_STRUCTURES_CHAT_H
```

6.11 lib/data_structures/list.h File Reference

Interfaccia ADT ListaDoppia (lista doppiamente concatenata).

```
#include "../data_structures.h"
```

Functions

- `NodoCarta * Lista_InserisciDopo (ListaCarte *lista, NodoCarta *nodo, Carta c)`
Inserisce una carta dopo un nodo specifico (o in testa se nodo==NULL).
- `NodoCarta * Lista_Precedente (ListaCarte *lista, NodoCarta *nodo)`
Restituisce il nodo precedente a quello dato.
- `NodoCarta * Lista_Ultimo (ListaCarte *lista)`
Restituisce l'ultimo nodo della lista (coda).
- `void Lista_RimuoviNodoBidirezionale (ListaCarte *lista, NodoCarta *nodo)`
Rimuove un nodo dalla lista gestendo correttamente prev/next.
- `int Lista_VerificaBidirezionale (ListaCarte *lista)`
Verifica l'integrita' dei puntatori prev/next (debug).

6.11.1 Detailed Description

Interfaccia ADT ListaDoppia (lista doppiamente concatenata).

Modulo specializzato per le liste bidirezionali utilizzato per la gestione della mano dei giocatori.

`NodoCarta` e' dotato dei campi prossimo/next e prev per supportare la navigazione bidirezionale.

Funzioni specifiche rispetto a list.h generale:

- `Lista_InserisciDopo`: inserimento dopo nodo specifico
- `Lista_Precedente`: accesso $O(1)$ al nodo precedente
- `Lista_Ultimo`: accesso $O(1)$ all'ultimo nodo
- `Lista_RimuoviNodoBidirezionale`: rimozione sicura con aggiornamento prev/next
- `Lista_VerificaBidirezionale`: verifica integrita' puntatori (debug)

6.12 list.h

[Go to the documentation of this file.](#)

```
00001
00019 #ifndef DATA_STRUCTURES_LIST_H
00020 #define DATA_STRUCTURES_LIST_H
00021
00022 #include "../data_structures.h"
00023
00034 NodoCarta* Lista_InserisciDopo(ListaCarte* lista, NodoCarta* nodo, Carta c);
00035
00043 NodoCarta* Lista_Precedente(ListaCarte* lista, NodoCarta* nodo);
00044
00052 NodoCarta* Lista_Ultimo(ListaCarte* lista);
00053
00062 void Lista_RimuoviNodoBidirezionale(ListaCarte* lista, NodoCarta* nodo);
00063
00070 int Lista_VerificaBidirezionale(ListaCarte* lista);
00071
00072 #endif // DATA_STRUCTURES_LIST_H
```

6.13 lib/list.h File Reference

ADT Lista Concatenata Bidirezionale per le carte.

```
#include "data_structures.h"
```

Functions

- [ListaCarte](#) * [CreaLista](#) (void)
Crea una nuova lista di carte vuota.
- void [InsertInCoda](#) ([ListaCarte](#) *lista, [Carta](#) carta)
Inserisce una carta in coda alla lista.
- void [InsertInTesta](#) ([ListaCarte](#) *lista, [Carta](#) carta)
Inserisce una carta in testa alla lista.
- void [RemoveAt](#) ([ListaCarte](#) *lista, int indice)
Rimuove una carta dalla lista per indice.
- void [RemoveNode](#) ([ListaCarte](#) *lista, [NodoCarta](#) *nodo)
Rimuove un nodo specifico dalla lista.
- [NodoCarta](#) * [GetAt](#) ([ListaCarte](#) *lista, int indice)
Restituisce il puntatore al nodo all'indice specificato.
- void [EliminaLista](#) ([ListaCarte](#) *lista)
Elimina tutta la lista e libera la memoria.
- void [StampaLista](#) ([ListaCarte](#) *lista)
Stampa il contenuto della lista su stdout (debug).

6.13.1 Detailed Description

ADT Lista Concatenata Bidirezionale per le carte.

6.14 list.h

[Go to the documentation of this file.](#)

```
00001
00015
00016 #ifndef LIST_H
00017 #define LIST_H
00018
00019 #include "data_structures.h"
00020
00021 /* Le definizioni di NodoCarta e ListaCarte sono in data_structures.h */
00022
00023 /* =====
00024  * FUNZIONI DI BASE PER LA LISTA
00025  * ===== */
00026
00032 ListaCarte* CreaLista(void);
00033
00041 void InsertInCoda(ListaCarte* lista, Carta carta);
00042
00050 void InsertInTesta(ListaCarte* lista, Carta carta);
00051
00059 void RemoveAt(ListaCarte* lista, int indice);
00060
00068 void RemoveNode(ListaCarte* lista, NodoCarta* nodo);
00069
00077 NodoCarta* GetAt(ListaCarte* lista, int indice);
00078
00085 void EliminaLista(ListaCarte* lista);
00086
00093 void StampaLista(ListaCarte* lista);
00094
00095 #endif /* LIST_H */
```

6.15 lib/data_structures/queue.h File Reference

ADT Coda (Queue) bidirezionale per la gestione dei turni di gioco.

```
#include "../data_structures.h"
```

Data Structures

- struct [NodoCoda](#)
Nodo della coda: contiene un [Giocatore](#) e puntatori next/prev.
- struct [CodaTurni](#)
Coda dei turni: coda doppiamente concatenata con direzione di scorrimento.

Typedefs

- typedef struct [NodoCoda](#) [NodoCoda](#)
Nodo della coda: contiene un [Giocatore](#) e puntatori next/prev.
- typedef struct [CodaTurni](#) [CodaTurni](#)
Coda dei turni: coda doppiamente concatenata con direzione di scorrimento.

Functions

- [CodaTurni](#) * [Coda_Crea](#) (void)
Crea una coda vuota.
- void [Coda_Distruggi](#) ([CodaTurni](#) *c)
Distruttore: libera tutti i nodi e la struttura [CodaTurni](#).
- int [Coda_Enqueue](#) ([CodaTurni](#) *c, [Giocatore](#) g)
Inserisce un giocatore in coda (rear).
- int [Coda_Dequeue](#) ([CodaTurni](#) *c, [Giocatore](#) *out)
Rimuove il giocatore in testa (front).
- int [Coda_Front](#) (const [CodaTurni](#) *c, [Giocatore](#) *out)
Restituisce il giocatore in testa SENZA rimuoverlo.
- int [Coda_Rear](#) (const [CodaTurni](#) *c, [Giocatore](#) *out)
Restituisce il giocatore in fondo (rear) SENZA rimuoverlo.
- [NodoCoda](#) * [Coda_ProxNodo](#) (const [CodaTurni](#) *c, [NodoCoda](#) *n)
Restituisce il nodo che segue n nella direzione corrente.
- int [Coda_Vuota](#) (const [CodaTurni](#) *c)
Verifica se la coda è vuota.
- int [Coda_Dimensione](#) (const [CodaTurni](#) *c)
Restituisce il numero di giocatori in coda.
- void [Coda_InvertiVerso](#) ([CodaTurni](#) *c)
Inverte il verso di scorrimento (carta Cambio Giro).
- [NodoCoda](#) * [Coda_CercaPerIndice](#) (const [CodaTurni](#) *c, int indice)
Cerca il nodo all'indice specificato (0 = front).
- int [Coda_CopiaPerIndice](#) (const [CodaTurni](#) *c, int indice, [Giocatore](#) *out)
Copia l'i-esimo giocatore in out (0 = front).

6.15.1 Detailed Description

ADT Coda (Queue) bidirezionale per la gestione dei turni di gioco.

6.16 queue.h

[Go to the documentation of this file.](#)

```

00001
00008 #ifndef DATA_STRUCTURES_QUEUE_H
00009 #define DATA_STRUCTURES_QUEUE_H
00010
00011 #include "../data_structures.h"
00012
00019 typedef struct NodoCoda {
00020     Giocatore giocatore;
00021     struct NodoCoda* next;
00022     struct NodoCoda* prev;
00023 } NodoCoda;
00025
00037 typedef struct CodaTurni {
00038     NodoCoda* front;
00039     NodoCoda* rear;
00040     int size;
00041     int verso;
00042 } CodaTurni;
00044
00055 CodaTurni* Coda_Crea(void);
00056
00063 void Coda_Distruggi(CodaTurni* c);
00064
00073 int Coda_Enqueue(CodaTurni* c, Giocatore g);
00074
00083 int Coda_Dequeue(CodaTurni* c, Giocatore* out);
00084
00093 int Coda_Front(const CodaTurni* c, Giocatore* out);
00094
00103 int Coda_Rear(const CodaTurni* c, Giocatore* out);
00104
00115 NodoCoda* Coda_ProxNodo(const CodaTurni* c, NodoCoda* n);
00116
00123 int Coda_Vuota(const CodaTurni* c);
00124
00131 int Coda_Dimensione(const CodaTurni* c);
00132
00142 void Coda_InvertiVerso(CodaTurni* c);
00143
00151 NodoCoda* Coda_CercaPerIndice(const CodaTurni* c, int indice);
00152
00162 int Coda_CopiaPerIndice(const CodaTurni* c, int indice, Giocatore* out);
00164
00165 #endif // DATA_STRUCTURES_QUEUE_H

```

6.17 lib/data_structures/stack.h File Reference

ADT [Pila](#) (Stack LIFO) per gestione mazzo di pesca e scarti.

```
#include "../data_structures.h"
```

Data Structures

- struct [NodoPila](#)
Nodo della pila: contiene una carta e puntatore al next.
- struct [Pila](#)
Pila LIFO: top punta all'ultimo inserito, size è la cardinalità.

Typedefs

- typedef struct NodoPila [NodoPila](#)
Nodo della pila: contiene una carta e puntatore al next.
- typedef struct Pila [Pila](#)
Pila LIFO: top punta all'ultimo inserito, size è la cardinalità.

Functions

- [Pila * Pila_Crea](#) (void)
Crea una pila vuota.
- void [Pila_Distruggi](#) ([Pila](#) *p)
Distruttore: libera tutti i nodi e la struttura Pila.
- int [Pila_Push](#) ([Pila](#) *p, [Carta](#) c)
Inserisce una carta in cima alla pila.
- int [Pila_Pop](#) ([Pila](#) *p, [Carta](#) *out)
Rimuove la carta in cima alla pila.
- int [Pila_Top](#) (const [Pila](#) *p, [Carta](#) *out)
Restituisce la carta in cima SENZA rimuoverla.
- int [Pila_Vuota](#) (const [Pila](#) *p)
Verifica se la pila è vuota.
- int [Pila_Dimensione](#) (const [Pila](#) *p)
Restituisce la dimensione corrente della pila.
- void [Pila_Svuota](#) ([Pila](#) *p)
Svuota la pila lasciando la struttura allocata.

6.17.1 Detailed Description

ADT [Pila](#) (Stack LIFO) per gestione mazzo di pesca e scarti.

6.18 stack.h

[Go to the documentation of this file.](#)

```

00001
00008 #ifndef DATA_STRUCTURES_STACK_H
00009 #define DATA_STRUCTURES_STACK_H
00010
00011 #include "../data_structures.h"
00012
00019 typedef struct NodoPila {
00020     Carta carta;
00021     struct NodoPila* next;
00022 } NodoPila;
00024
00031 typedef struct Pila {
00032     NodoPila* top;
00033     int size;
00034 } Pila;
00036
00047 Pila* Pila_Crea(void);
00048
00055 void Pila_Distruggi(Pila* p);
00056
00065 int Pila_Push(Pila* p, Carta c);
00066
00075 int Pila_Pop(Pila* p, Carta* out);
00076
00085 int Pila_Top(const Pila* p, Carta* out);
00086
00093 int Pila_Vuota(const Pila* p);
00094
00101 int Pila_Dimensione(const Pila* p);
00102
00109 void Pila_Svuota(Pila* p);
00111
00112 #endif // DATA_STRUCTURES_STACK_H

```

6.19 lib/fs/fs.h File Reference

Helper filesystem multiplatforma per creazione directory.

```
#include <stdlib.h>
```

6.19.1 Detailed Description

Helper filesystem multiplatforma per creazione directory.

6.20 fs.h

[Go to the documentation of this file.](#)

```
00001
00005 #ifndef FS_H
00006 #define FS_H
00007 #include <stdlib.h>
00008 static inline void fs_mkdirs(void) {
00009 #ifdef _WIN32
00010     system("if not exist data mkdir data");
00011     system("if not exist data\\users mkdir data\\users");
00012     system("if not exist data\\games mkdir data\\games");
00013 #else
00014     system("mkdir -p data/users data/games");
00015 #endif
00016 }
00017 #endif
```

6.21 lib/game_logic.h File Reference

Modulo di logica di gioco UNO.

```
#include "data_structures.h"
#include "data_structures/stack.h"
#include "data_structures/queue.h"
```

Functions

- int [CaricaPartitaDaSlot](#) ([StatoGioco](#) *gioco, int slot)
Carica una partita da un file di salvataggio.
- void [RicostruisciStatoADT](#) ([StatoGioco](#) *gioco)
Ricostruisce le strutture ADT statiche (mazzo_pila, scarti_pila, coda_turni) e le mani dei giocatori dopo un caricamento da file.
- void [EliminaSalvataggio](#) (int slot)
Elimina un file di salvataggio esistente.
- void [SalvaPartita](#) ([StatoGioco](#) *gioco)
Salva lo stato corrente della partita su file.
- void [CompattaSalvataggiUtente](#) (void)
Compatta i salvataggi dell'utente loggato in ordine cronologico.
- void [EliminaTuttiSalvataggiUtente](#) (const char *username)

- Elimina TUTTI i salvataggi di un utente specifico.*
- void [GetSaveFilePath](#) (int user_index, int slot, char *buffer, size_t bufsize)
Restituisce il path del file di salvataggio per un dato utente e slot.
- void [InizializzaMazzo](#) (StatoGioco *gioco)
Inizializza il mazzo di pesca con 108 carte UNO standard.
- void [RicaricaMazzo](#) (StatoGioco *gioco)
Ricarica il mazzo di pesca dagli scarti.
- int [MossaValida](#) (StatoGioco *gioco, Carta c)
Verifica se una carta puo' essere giocata legalmente.
- void [ApplicaEffetto](#) (StatoGioco *gioco, Carta c, Colore colore_scelto)
Applica l'effetto speciale di una carta giocata.
- void [NuovaPartita](#) (StatoGioco *gioco, int num_avversari)
Inizializza una nuova partita con il numero di avversari specificato.
- void [EliminaPartita](#) (StatoGioco *gioco)
Libera tutte le risorse allocate per una partita.
- void [MostraNotifica](#) (StatoGioco *gioco, const char *msg)
Mostra una notifica temporanea a schermo.
- int [GetLunghezzaMano](#) (Giocatore *giocatore)
Restituisce il numero di carte nella mano di un giocatore.
- Carta [GetCartaGiocatore](#) (Giocatore *giocatore, int indice)
Restituisce una carta dalla mano di un giocatore per indice.
- NodoCarta * [GetNodoCartaGiocatore](#) (Giocatore *giocatore, int indice)
Restituisce il puntatore al nodo di una carta per indice.
- void [RimuoviCartaGiocatore](#) (Giocatore *giocatore, int indice)
Rimuove una carta dalla mano di un giocatore per indice.
- void [RimescolaScartiNelMazzo](#) (StatoGioco *gioco)
Rimescola le carte dalla pila degli scarti nel mazzo di pesca.
- void [Pesca](#) (StatoGioco *gioco, int id_giocatore, int quantita)
Fa pescare un certo numero di carte a un giocatore.

Variables

- Pila * [mazzo_pila](#)
ADT globali del gioco: pila mazzo pesca, pila scarti, coda turni.
- Pila * [scarti_pila](#)
Pila globale degli scarti.
- CodaTurni * [coda_turni](#)
Coda globale dei turni.

6.21.1 Detailed Description

Modulo di logica di gioco UNO.

6.21.2 Function Documentation

6.21.2.1 ApplicaEffetto()

```
void ApplicaEffetto (
    StatoGioco * gioco,
    Carta c,
    Colore colore_scelto)
```

Applica l'effetto speciale di una carta giocata.

Gestisce gli effetti di tutte le carte speciali:

- CAMBIA_GIRO: Inverte la direzione del turno.
- SALTA: Salta il giocatore successivo.
- PIU_2: Il giocatore successivo pesca 2 carte.
- JOLLY: Il giocatore sceglie un colore.
- JOLLY_PIU_4: Il giocatore successivo pesca 4 carte e si sceglie un colore.

Precondition

gioco != NULL; carta valida e giocabile.

Postcondition

Stato gioco aggiornato con effetto applicato.

Parameters

<i>gioco</i>	Puntatore allo stato di gioco.
<i>c</i>	La carta giocata con effetto speciale.
<i>colore_scelto</i>	Il colore scelto dal giocatore (per Jolly/JollyPiù4).

Applica l'effetto speciale di una carta giocata.

Gestisce: cambio colore, cambio giro, salta turno, pesca carte, aggiornamento colore attivo e rotazione coda turni.

Precondition

gioco != NULL; carta valida.

Postcondition

Stato aggiornato con effetto applicato; notifica mostrata.

Parameters

<i>gioco</i>	Stato di gioco.
<i>c</i>	Carta giocata.
<i>colore_scelto</i>	Colore scelto dal giocatore (per jolly).

6.21.2.2 CaricaPartitaDaSlot()

```
int CaricaPartitaDaSlot (
    StatoGioco * gioco,
    int slot)
```

Carica una partita da un file di salvataggio.

Precondition

gioco != NULL; slot nel range valido.

Postcondition

Stato partita ricaricato; ADT ricostruiti se necessario.

Parameters

<i>gioco</i>	Puntatore allo stato di gioco da popolare.
<i>slot</i>	Indice dello slot di salvataggio (1-5).

Returns

1 se il caricamento e' riuscito, 0 altrimenti.

Carica una partita da un file di salvataggio.

Cerca prima nel formato save_user{ID}_slot_{N}.dat. Fallback: se non trova, controlla nel vecchio formato save_slot_{N}.dat per retrocompatibilità con salvataggi preesistenti.

Parameters

<i>gioco</i>	Puntatore allo stato di gioco da popolare.
<i>slot</i>	Indice dello slot (1-100).

Returns

1 se caricamento riuscito, 0 se slot vuoto o errore.

6.21.2.3 CompattaSalvataggiUtente()

```
void CompattaSalvataggiUtente (
    void )
```

Compatta i salvataggi dell'utente loggato in ordine cronologico.

Raccoglie tutti i salvataggi dell'utente (cerca sia formato nuovo save_user{ID}_slot_{N}.dat che vecchio save_slot_{N}.dat), li ordina per data crescente (bubble sort) e li riscrive nei primi slot liberi con il nuovo formato.

6.21.2.4 EliminaPartita()

```
void EliminaPartita (
    StatoGioco * gioco)
```

Libera tutte le risorse allocate per una partita.

Distrugge la mano di ogni giocatore (liste collegate), svuota le pile di pesca e scarti, libera l'albero BST.

Precondition

gioco != NULL.

Postcondition

Tutte le risorse liberate; ADT resettate.

Parameters

<i>gioco</i>	Puntatore allo stato di gioco da deallocare.
--------------	--

Libera tutte le risorse allocate per una partita.

Distrugge: mani giocatori (liste), mazzo_pila, scarti_pila, coda_turni. Resetta num_giocatori, num_carte_mazzo, num_carte_scarti a 0.

Parameters

<i>gioco</i>	Puntatore allo stato di gioco da deallocare.
--------------	--

6.21.2.5 EliminaSalvataggio()

```
void EliminaSalvataggio (
    int slot)
```

Elimina un file di salvataggio esistente.

Precondition

slot nel range valido.

Postcondition

File di salvataggio rimosso.

Parameters

<i>slot</i>	Indice dello slot di salvataggio da eliminare.
-------------	--

Elimina un file di salvataggio esistente.

Legge i metadati (owner, data) prima di eliminare, poi: rimuove il file, compatta, notifica altri dispositivi LAN. Cerca nel formato save_user{ID}_slot_{N}.dat.

Parameters

<i>slot</i>	Indice dello slot da eliminare.
-------------	---------------------------------

6.21.2.6 EliminaTuttiSalvataggiUtente()

```
void EliminaTuttiSalvataggiUtente (
    const char * username)
```

Elimina TUTTI i salvataggi di un utente specifico.

Scandisce tutti gli slot 1-100 e rimuove i file .dat il cui proprietario corrisponde allo username specificato. Controlla sia il formato save_user{ID}_slot_{N}.dat che il vecchio save_slot_N.dat.

Parameters

<i>username</i>	Username dell'utente i cui salvataggi vanno eliminati.
-----------------	--

6.21.2.7 GetSaveFilePath()

```
void GetSaveFilePath (
    int user_index,
    int slot,
    char * buffer,
    size_t bufsize)
```

Restituisce il path del file di salvataggio per un dato utente e slot.

Implementazione della funzione dichiarata in [game_logic.h](#) per compatibilità. Formato: data/games/save_↵ user{ID}_slot_{N}.dat

Parameters

<i>user_index</i>	Indice dell'utente nel db (non usato, si usa id_utente_corrente).
<i>slot</i>	Indice dello slot (1-100).
<i>buffer</i>	Buffer di destinazione.
<i>bufsize</i>	Dimensione del buffer.

6.21.2.8 MossaValida()

```
int MossaValida (
    StatoGioco * gioco,
    Carta c)
```

Verifica se una carta puo' essere giocata legalmente.

Controlla se il colore o il valore della carta corrisponde alla carta in cima alla pila degli scarti, oppure se e' un Jolly.

Precondition

gioco != NULL; gioco->num_carte_scarti > 0.

Postcondition

Restituisce 1 se mossa valida, 0 altrimenti.

Parameters

<i>gioco</i>	Puntatore allo stato di gioco corrente.
<i>c</i>	La carta che si vuole giocare.

Returns

1 se la mossa e' valida, 0 altrimenti.

Verifica se una carta puo' essere giocata legalmente.

Regole UNO:

- Jolly (cambiocolore/PescaQuattro): sempre validi
- Altrimenti: match colore attivo OPPURE match tipo/valore

Precondition

`gioco != NULL; gioco->num_carte_scarti > 0.`

Postcondition

Restituisce 1 se corrispondenza colore/tipo o carta nera.

Parameters

<i>gioco</i>	Stato di gioco corrente.
<i>c</i>	Carta da giocare.

Returns

1 se mossa valida, 0 altrimenti.

6.21.2.9 MostraNotifica()

```
void MostraNotifica (
    StatoGioco * gioco,
    const char * msg)
```

Mostra una notifica temporanea a schermo.

Imposta il testo della notifica e un timer di visualizzazione. La notifica viene disegnata in DisegnaGameplay.

Precondition

`gioco != NULL; msg != NULL.`

Postcondition

Notifica attiva per 3 secondi.

Parameters

<i>gioco</i>	Puntatore allo stato di gioco.
<i>msg</i>	Testo della notifica da mostrare.

Sostituisce eventuale notifica precedente. Si auto-cancella dopo un certo tempo gestito dal loop di gioco.

Precondition

`gioco != NULL; msg != NULL.`

Postcondition

Notifica visibile per 3 secondi.

Parameters

<i>gioco</i>	Stato di gioco.
<i>msg</i>	Testo della notifica.

6.21.2.10 NuovaPartita()

```
void NuovaPartita (
    StatoGioco * gioco,
    int num_avversari)
```

Inizializza una nuova partita con il numero di avversari specificato.

Crea il mazzo, distribuisce 7 carte a ogni giocatore, pesca la prima carta dagli scarti e gestisce le eventuali carte speciali iniziali.

Precondition

`gioco != NULL; num_avversari in range 1-3.`

Postcondition

Stato partita inizializzato e pronto per il gioco.

Parameters

<i>gioco</i>	Puntatore allo stato di gioco da inizializzare.
<i>num_avversari</i>	Numero di avversari (bot).

Inizializza una nuova partita con il numero di avversari specificato.

Procedura:

1. Azzerare lo [StatoGioco](#)
2. Imposta il giocatore umano (slot 0) e i bot (1..N)
3. Crea il mazzo standard UNO (108 carte)
4. Distribuisce 7 carte per giocatore
5. Pesca la prima carta scarto (evitando jolly)
6. Inizializza la coda turni
7. Trova il primo slot di salvataggio libero

Parameters

	<i>gioco</i>	Puntatore allo stato di gioco da inizializzare.
	<i>num_avversari</i>	Numero di avversari bot (1..3).

6.21.2.11 RicaricaMazzo()

```
void RicaricaMazzo (
    StatoGioco * gioco)
```

Ricarica il mazzo di pesca dagli scarti.

Quando il mazzo di pesca e' vuoto, prende tutte le carte dalla pila degli scarti (tranne l'ultima in cima) e le rimette nel mazzo di pesca dopo averle rimescolate con Fisher-Yates.

Precondition

gioco != NULL; *scarti_pila* non vuota.

Postcondition

mazzo_pila ricaricata e rimescolata.

Parameters

	<i>gioco</i>	Puntatore allo stato di gioco.
--	--------------	--------------------------------

6.21.2.12 RicostruisciStatoADT()

```
void RicostruisciStatoADT (
    StatoGioco * gioco)
```

Ricostruisce le strutture ADT statiche (*mazzo_pila*, *scarti_pila*, *coda_turni*) e le mani dei giocatori dopo un caricamento da file.

Deve essere chiamata SUBITO DOPO [CaricaPartitaDaSlot\(\)](#) per rendere consistente lo stato interno con quello caricato dal file.

Precondition

gioco caricato con dati consistenti.

Postcondition

ADT ricostruiti e mani ricreate.

Parameters

<i>gioco</i>	Puntatore allo stato di gioco appena caricato.
--------------	--

Ricostruisce le strutture ADT statiche (mazzo_pila, scarti_pila, coda_turni) e le mani dei giocatori dopo un caricamento da file.

Dopo [CaricaPartitaDaSlot\(\)](#), le strutture ADT dinamiche (pile, code, liste) sono NULL. Questa funzione ricrea tutto partendo dai dati serializzati:

- Ricrea mazzo_pila da `gioco->mazzo[]`
- Ricrea scarti_pila da `gioco->scarti[]`
- Ricrea coda_turni da `gioco->giocatori[]`
- Ricrea la mano di ogni giocatore da `gioco->mano_backup[]`

Parameters

<i>gioco</i>	Puntatore allo stato di gioco appena caricato.
--------------	--

6.21.2.13 SalvaPartita()

```
void SalvaPartita (  
    StatoGioco * gioco)
```

Salva lo stato corrente della partita su file.

Precondition

gioco != NULL; partita inizializzata.

Postcondition

File di salvataggio aggiornato.

Parameters

<i>gioco</i>	Puntatore allo stato di gioco da salvare.
--------------	---

Salva lo stato corrente della partita su file.

Procedura:

1. Compatta i salvataggi esistenti dell'utente
2. Genera il timestamp corrente
3. Trova il primo slot libero
4. Serializza le mani su array di backup
5. Scrive lo *StatoGioco* su file .dat
6. Ricostruisce gli ADT
7. Broadcast LAN dei nuovi salvataggi

Parameters

<i>gioco</i>	Puntatore allo stato di gioco da salvare.
--------------	---

6.21.3 Variable Documentation**6.21.3.1 coda_turni**

```
CodaTurni* coda_turni [extern]
```

Coda globale dei turni.

6.21.3.2 mazzo_pila

```
Pila* mazzo_pila [extern]
```

ADT globali del gioco: pila mazzo pesca, pila scarti, coda turni.

ADT globali del gioco: pila mazzo pesca, pila scarti, coda turni.

6.21.3.3 scarti_pila

```
Pila* scarti_pila [extern]
```

Pila globale degli scarti.

6.22 game_logic.h

[Go to the documentation of this file.](#)

```
00001
00008
00009 #ifndef GAME_LOGIC_H
00010 #define GAME_LOGIC_H
00011
00012 #include "data_structures.h"
00013 #include "data_structures/stack.h"
00014 #include "data_structures/queue.h"
00015
00017 extern Pila* mazzo_pila;
00018 extern Pila* scarti_pila;
00019 extern CodaTurni* coda_turni;
00020
00021 /* =====
00022  * GESTIONE PARTITE (Salvataggio / Caricamento)
00023  * ===== */
00024
00033 int CaricaPartitaDaSlot(StatoGioco *gioco, int slot);
00034
00045 void RicostruisciStatoADT(StatoGioco *gioco);
00046
00053 void EliminaSalvataggio(int slot);
00054
00061 void SalvaPartita(StatoGioco *gioco);
00062 void CompattaSalvataggiUtente(void);
00063 void EliminaTuttiSalvataggiUtente(const char* username);
00064 void GetSaveFilePath(int user_index, int slot, char* buffer, size_t bufsize);
00065
00066 /* =====
00067  * GESTIONE MAZZO E PESCA
00068  * ===== */
00069
00082 void InizializzaMazzo(StatoGioco *gioco);
00083
00095 void RicaricaMazzo(StatoGioco *gioco);
00096
00097 /* =====
00098  * VALIDAZIONE E APPLICAZIONE MOSSE
00099  * ===== */
00100
00113 int MossaValida(StatoGioco *gioco, Carta c);
00114
00131 void ApplicaEffetto(StatoGioco *gioco, Carta c, Colore colore_scelto);
00132
00133 /* =====
00134  * INIZIALIZZAZIONE E FINE PARTITA
00135  * ===== */
00136
00148 void NuovaPartita(StatoGioco *gioco, int num_avversari);
00149
00160 void EliminaPartita(StatoGioco *gioco);
00161
00173 void MostraNotifica(StatoGioco *gioco, const char* msg);
00174
```

```

00175  /* =====
00176  *  FUNZIONI HELPER PER ACCESSO ALLE MANI (Liste collegate)
00177  *  ===== */
00178
00185  int GetLunghezzaMano(Giocatore *giocatore);
00186
00194  Carta GetCartaGiocatore(Giocatore *giocatore, int indice);
00195
00203  NodoCarta* GetNodoCartaGiocatore(Giocatore *giocatore, int indice);
00204
00212  void RimuoviCartaGiocatore(Giocatore *giocatore, int indice);
00213
00225  void RimescolaScartiNelMazzo(StatoGioco *gioco);
00226
00239  void Pesca(StatoGioco *gioco, int id_giocatore, int quantita);
00240
00241  #endif

```

6.23 lib/raylib/include/raylib.h File Reference

```
#include <stdarg.h>
```

Data Structures

- struct [Vector2](#)
- struct [Vector3](#)
- struct [Vector4](#)
- struct [Matrix](#)
- struct [Color](#)
- struct [Rectangle](#)
- struct [Image](#)
- struct [Texture](#)
- struct [RenderTexture](#)
- struct [NPatchInfo](#)
- struct [GlyphInfo](#)
- struct [Font](#)
- struct [Camera3D](#)
- struct [Camera2D](#)
- struct [Mesh](#)
- struct [Shader](#)
- struct [MaterialMap](#)
- struct [Material](#)
- struct [Transform](#)
- struct [BoneInfo](#)
- struct [ModelSkeleton](#)
- struct [Model](#)
- struct [ModelAnimation](#)
- struct [Ray](#)
- struct [RayCollision](#)
- struct [BoundingBox](#)
- struct [Wave](#)
- struct [AudioStream](#)
- struct [Sound](#)
- struct [Music](#)
- struct [VrDeviceInfo](#)
- struct [VrStereoConfig](#)
- struct [FilePathList](#)
- struct [AutomationEvent](#)
- struct [AutomationEventList](#)

Macros

- #define [RAYLIB_VERSION_MAJOR](#) 6
- #define [RAYLIB_VERSION_MINOR](#) 0
- #define [RAYLIB_VERSION_PATCH](#) 0
- #define [RAYLIB_VERSION](#) "6.0"
- #define [RLAPI](#)
- #define [PI](#) 3.14159265358979323846f
- #define [DEG2RAD](#) ([PI](#)/180.0f)
- #define [RAD2DEG](#) (180.0f/[PI](#))
- #define [RL_MALLOC](#)(sz)
- #define [RL_CALLOC](#)(n, sz)
- #define [RL_REALLOC](#)(ptr, sz)
- #define [RL_FREE](#)(ptr)
- #define [CLITERAL](#)(type)
- #define [RL_COLOR_TYPE](#)
- #define [RL_RECTANGLE_TYPE](#)
- #define [RL_VECTOR2_TYPE](#)
- #define [RL_VECTOR3_TYPE](#)
- #define [RL_VECTOR4_TYPE](#)
- #define [RL_QUATERNION_TYPE](#)
- #define [RL_MATRIX_TYPE](#)
- #define [LIGHTGRAY CLITERAL](#)(Color){ 200, 200, 200, 255 }
- #define [GRAY CLITERAL](#)(Color){ 130, 130, 130, 255 }
- #define [DARKGRAY CLITERAL](#)(Color){ 80, 80, 80, 255 }
- #define [YELLOW CLITERAL](#)(Color){ 253, 249, 0, 255 }
- #define [GOLD CLITERAL](#)(Color){ 255, 203, 0, 255 }
- #define [ORANGE CLITERAL](#)(Color){ 255, 161, 0, 255 }
- #define [PINK CLITERAL](#)(Color){ 255, 109, 194, 255 }
- #define [RED CLITERAL](#)(Color){ 230, 41, 55, 255 }
- #define [MAROON CLITERAL](#)(Color){ 190, 33, 55, 255 }
- #define [GREEN CLITERAL](#)(Color){ 0, 228, 48, 255 }
- #define [LIME CLITERAL](#)(Color){ 0, 158, 47, 255 }
- #define [DARKGREEN CLITERAL](#)(Color){ 0, 117, 44, 255 }
- #define [SKYBLUE CLITERAL](#)(Color){ 102, 191, 255, 255 }
- #define [BLUE CLITERAL](#)(Color){ 0, 121, 241, 255 }
- #define [DARKBLUE CLITERAL](#)(Color){ 0, 82, 172, 255 }
- #define [PURPLE CLITERAL](#)(Color){ 200, 122, 255, 255 }
- #define [VIOLET CLITERAL](#)(Color){ 135, 60, 190, 255 }
- #define [DARKPURPLE CLITERAL](#)(Color){ 112, 31, 126, 255 }
- #define [BEIGE CLITERAL](#)(Color){ 211, 176, 131, 255 }
- #define [BROWN CLITERAL](#)(Color){ 127, 106, 79, 255 }
- #define [DARKBROWN CLITERAL](#)(Color){ 76, 63, 47, 255 }
- #define [WHITE CLITERAL](#)(Color){ 255, 255, 255, 255 }
- #define [BLACK CLITERAL](#)(Color){ 0, 0, 0, 255 }
- #define [BLANK CLITERAL](#)(Color){ 0, 0, 0, 0 }
- #define [MAGENTA CLITERAL](#)(Color){ 255, 0, 255, 255 }
- #define [RAYWHITE CLITERAL](#)(Color){ 245, 245, 245, 255 }
- #define [RL_BOOL_TYPE](#)
- #define [MOUSE_LEFT_BUTTON](#) [MOUSE_BUTTON_LEFT](#)
- #define [MOUSE_RIGHT_BUTTON](#) [MOUSE_BUTTON_RIGHT](#)
- #define [MOUSE_MIDDLE_BUTTON](#) [MOUSE_BUTTON_MIDDLE](#)
- #define [MATERIAL_MAP_DIFFUSE](#) [MATERIAL_MAP_ALBEDO](#)
- #define [MATERIAL_MAP_SPECULAR](#) [MATERIAL_MAP_METALNESS](#)
- #define [SHADER_LOC_MAP_DIFFUSE](#) [SHADER_LOC_MAP_ALBEDO](#)
- #define [SHADER_LOC_MAP_SPECULAR](#) [SHADER_LOC_MAP_METALNESS](#)
- #define [GetMouseRay](#) [GetScreenToWorldRay](#)

Typedefs

- typedef enum [bool](#) [bool](#)
- typedef struct Vector2 [Vector2](#)
- typedef struct Vector3 [Vector3](#)
- typedef struct Vector4 [Vector4](#)
- typedef [Vector4](#) [Quaternion](#)
- typedef struct Matrix [Matrix](#)
- typedef struct Color [Color](#)
- typedef struct Rectangle [Rectangle](#)
- typedef struct Image [Image](#)
- typedef struct Texture [Texture](#)
- typedef [Texture](#) [Texture2D](#)
- typedef [Texture](#) [TextureCubemap](#)
- typedef struct RenderTexture [RenderTexture](#)
- typedef [RenderTexture](#) [RenderTexture2D](#)
- typedef struct NPatchInfo [NPatchInfo](#)
- typedef struct GlyphInfo [GlyphInfo](#)
- typedef struct Font [Font](#)
- typedef struct Camera3D [Camera3D](#)
- typedef [Camera3D](#) [Camera](#)
- typedef struct Camera2D [Camera2D](#)
- typedef struct Mesh [Mesh](#)
- typedef struct Shader [Shader](#)
- typedef struct MaterialMap [MaterialMap](#)
- typedef struct Material [Material](#)
- typedef struct Transform [Transform](#)
- typedef [Transform](#) * [ModelAnimPose](#)
- typedef struct BoneInfo [BoneInfo](#)
- typedef struct ModelSkeleton [ModelSkeleton](#)
- typedef struct Model [Model](#)
- typedef struct ModelAnimation [ModelAnimation](#)
- typedef struct Ray [Ray](#)
- typedef struct RayCollision [RayCollision](#)
- typedef struct BoundingBox [BoundingBox](#)
- typedef struct Wave [Wave](#)
- typedef struct [rAudioBuffer](#) [rAudioBuffer](#)
- typedef struct [rAudioProcessor](#) [rAudioProcessor](#)
- typedef struct AudioStream [AudioStream](#)
- typedef struct Sound [Sound](#)
- typedef struct Music [Music](#)
- typedef struct VrDeviceInfo [VrDeviceInfo](#)
- typedef struct VrStereoConfig [VrStereoConfig](#)
- typedef struct FilePathList [FilePathList](#)
- typedef struct AutomationEvent [AutomationEvent](#)
- typedef struct AutomationEventList [AutomationEventList](#)
- typedef void(* [TraceLogCallback](#)) (int logLevel, const char *text, va_list args)
- typedef unsigned char *(* [LoadFileDataCallback](#)) (const char *fileName, int *dataSize)
- typedef bool(* [SaveFileDataCallback](#)) (const char *fileName, void *data, int dataSize)
- typedef char *(* [LoadFileTextCallback](#)) (const char *fileName)
- typedef bool(* [SaveFileTextCallback](#)) (const char *fileName, const char *text)
- typedef void(* [AudioCallback](#)) (void *bufferData, unsigned int frames)

Enumerations

- enum `bool` { `false` = 0 , `true` = !false }
- enum `ConfigFlags` {
`FLAG_VSYNC_HINT` = 0x00000040 , `FLAG_FULLSCREEN_MODE` = 0x00000002 , `FLAG_WINDOW_RESIZABLE` = 0x00000004 , `FLAG_WINDOW_UNDECORATED` = 0x00000008 ,
`FLAG_WINDOW_HIDDEN` = 0x00000080 , `FLAG_WINDOW_MINIMIZED` = 0x00000200 , `FLAG_WINDOW_MAXIMIZED` = 0x00000400 , `FLAG_WINDOW_UNFOCUSED` = 0x00000800 ,
`FLAG_WINDOW_TOPMOST` = 0x00001000 , `FLAG_WINDOW_ALWAYS_RUN` = 0x00000100 ,
`FLAG_WINDOW_TRANSPARENT` = 0x00000010 , `FLAG_WINDOW_HIGHDPI` = 0x00002000 ,
`FLAG_WINDOW_MOUSE_PASSTHROUGH` = 0x00004000 , `FLAG_BORDERLESS_WINDOWED_MODE` = 0x00008000 , `FLAG_MSAA_4X_HINT` = 0x00000020 , `FLAG_INTERLACED_HINT` = 0x00010000 }
- enum `TraceLogLevel` {
`LOG_ALL` = 0 , `LOG_TRACE` , `LOG_DEBUG` , `LOG_INFO` ,
`LOG_WARNING` , `LOG_ERROR` , `LOG_FATAL` , `LOG_NONE` }
- enum `KeyboardKey` {
`KEY_NULL` = 0 , `KEY_APOSTROPHE` = 39 , `KEY_COMMA` = 44 , `KEY_MINUS` = 45 ,
`KEY_PERIOD` = 46 , `KEY_SLASH` = 47 , `KEY_ZERO` = 48 , `KEY_ONE` = 49 ,
`KEY_TWO` = 50 , `KEY_THREE` = 51 , `KEY_FOUR` = 52 , `KEY_FIVE` = 53 ,
`KEY_SIX` = 54 , `KEY_SEVEN` = 55 , `KEY_EIGHT` = 56 , `KEY_NINE` = 57 ,
`KEY_SEMICOLON` = 59 , `KEY_EQUAL` = 61 , `KEY_A` = 65 , `KEY_B` = 66 ,
`KEY_C` = 67 , `KEY_D` = 68 , `KEY_E` = 69 , `KEY_F` = 70 ,
`KEY_G` = 71 , `KEY_H` = 72 , `KEY_I` = 73 , `KEY_J` = 74 ,
`KEY_K` = 75 , `KEY_L` = 76 , `KEY_M` = 77 , `KEY_N` = 78 ,
`KEY_O` = 79 , `KEY_P` = 80 , `KEY_Q` = 81 , `KEY_R` = 82 ,
`KEY_S` = 83 , `KEY_T` = 84 , `KEY_U` = 85 , `KEY_V` = 86 ,
`KEY_W` = 87 , `KEY_X` = 88 , `KEY_Y` = 89 , `KEY_Z` = 90 ,
`KEY_LEFT_BRACKET` = 91 , `KEY_BACKSLASH` = 92 , `KEY_RIGHT_BRACKET` = 93 , `KEY_GRAVE` = 96 ,
`KEY_SPACE` = 32 , `KEY_ESCAPE` = 256 , `KEY_ENTER` = 257 , `KEY_TAB` = 258 ,
`KEY_BACKSPACE` = 259 , `KEY_INSERT` = 260 , `KEY_DELETE` = 261 , `KEY_RIGHT` = 262 ,
`KEY_LEFT` = 263 , `KEY_DOWN` = 264 , `KEY_UP` = 265 , `KEY_PAGE_UP` = 266 ,
`KEY_PAGE_DOWN` = 267 , `KEY_HOME` = 268 , `KEY_END` = 269 , `KEY_CAPS_LOCK` = 280 ,
`KEY_SCROLL_LOCK` = 281 , `KEY_NUM_LOCK` = 282 , `KEY_PRINT_SCREEN` = 283 , `KEY_PAUSE` = 284 ,

`KEY_F1` = 290 , `KEY_F2` = 291 , `KEY_F3` = 292 , `KEY_F4` = 293 ,
`KEY_F5` = 294 , `KEY_F6` = 295 , `KEY_F7` = 296 , `KEY_F8` = 297 ,
`KEY_F9` = 298 , `KEY_F10` = 299 , `KEY_F11` = 300 , `KEY_F12` = 301 ,
`KEY_LEFT_SHIFT` = 340 , `KEY_LEFT_CONTROL` = 341 , `KEY_LEFT_ALT` = 342 , `KEY_LEFT_SUPER` = 343 ,
`KEY_RIGHT_SHIFT` = 344 , `KEY_RIGHT_CONTROL` = 345 , `KEY_RIGHT_ALT` = 346 , `KEY_RIGHT_SUPER` = 347 ,
`KEY_KB_MENU` = 348 , `KEY_KP_0` = 320 , `KEY_KP_1` = 321 , `KEY_KP_2` = 322 ,
`KEY_KP_3` = 323 , `KEY_KP_4` = 324 , `KEY_KP_5` = 325 , `KEY_KP_6` = 326 ,
`KEY_KP_7` = 327 , `KEY_KP_8` = 328 , `KEY_KP_9` = 329 , `KEY_KP_DECIMAL` = 330 ,
`KEY_KP_DIVIDE` = 331 , `KEY_KP_MULTIPLY` = 332 , `KEY_KP_SUBTRACT` = 333 , `KEY_KP_ADD` = 334 ,
`KEY_KP_ENTER` = 335 , `KEY_KP_EQUAL` = 336 , `KEY_BACK` = 4 , `KEY_MENU` = 5 ,
`KEY_VOLUME_UP` = 24 , `KEY_VOLUME_DOWN` = 25 }
- enum `MouseButton` {
`MOUSE_BUTTON_LEFT` = 0 , `MOUSE_BUTTON_RIGHT` = 1 , `MOUSE_BUTTON_MIDDLE` = 2 ,
`MOUSE_BUTTON_SIDE` = 3 ,
`MOUSE_BUTTON_EXTRA` = 4 , `MOUSE_BUTTON_FORWARD` = 5 , `MOUSE_BUTTON_BACK` = 6 }
- enum `MouseCursor` {
`MOUSE_CURSOR_DEFAULT` = 0 , `MOUSE_CURSOR_ARROW` = 1 , `MOUSE_CURSOR_IBEAM` = 2 ,
`MOUSE_CURSOR_CROSSHAIR` = 3 ,
`MOUSE_CURSOR_POINTING_HAND` = 4 , `MOUSE_CURSOR_RESIZE_EW` = 5 , `MOUSE_CURSOR_RESIZE_NS` = 6 ,
`MOUSE_CURSOR_RESIZE_NWSE` = 7 ,
`MOUSE_CURSOR_RESIZE_NESW` = 8 , `MOUSE_CURSOR_RESIZE_ALL` = 9 , `MOUSE_CURSOR_NOT_ALLOWED` = 10 }

- enum GamepadButton {
GAMEPAD_BUTTON_UNKNOWN = 0, GAMEPAD_BUTTON_LEFT_FACE_UP, GAMEPAD_BUTTON_LEFT_FACE_RIGHT,
GAMEPAD_BUTTON_LEFT_FACE_DOWN,
GAMEPAD_BUTTON_LEFT_FACE_LEFT, GAMEPAD_BUTTON_RIGHT_FACE_UP, GAMEPAD_BUTTON_RIGHT_FACE_RIGHT,
GAMEPAD_BUTTON_RIGHT_FACE_DOWN,
GAMEPAD_BUTTON_RIGHT_FACE_LEFT, GAMEPAD_BUTTON_LEFT_TRIGGER_1, GAMEPAD_BUTTON_LEFT_TRIGGER_2,
GAMEPAD_BUTTON_RIGHT_TRIGGER_1,
GAMEPAD_BUTTON_RIGHT_TRIGGER_2, GAMEPAD_BUTTON_MIDDLE_LEFT, GAMEPAD_BUTTON_MIDDLE_RIGHT,
GAMEPAD_BUTTON_MIDDLE_LEFT, GAMEPAD_BUTTON_MIDDLE_RIGHT,
GAMEPAD_BUTTON_LEFT_THUMB, GAMEPAD_BUTTON_RIGHT_THUMB }
- enum GamepadAxis {
GAMEPAD_AXIS_LEFT_X = 0, GAMEPAD_AXIS_LEFT_Y = 1, GAMEPAD_AXIS_RIGHT_X = 2,
GAMEPAD_AXIS_RIGHT_Y = 3,
GAMEPAD_AXIS_LEFT_TRIGGER = 4, GAMEPAD_AXIS_RIGHT_TRIGGER = 5 }
- enum MaterialMapIndex {
MATERIAL_MAP_ALBEDO = 0, MATERIAL_MAP_METALNESS, MATERIAL_MAP_NORMAL,
MATERIAL_MAP_ROUGHNESS,
MATERIAL_MAP_OCCLUSION, MATERIAL_MAP_EMISSION, MATERIAL_MAP_HEIGHT, MATERIAL_MAP_CUBEMAP,
MATERIAL_MAP_IRRADIANCE, MATERIAL_MAP_PREFILTER, MATERIAL_MAP_BRDF }
- enum ShaderLocationIndex {
SHADER_LOC_VERTEX_POSITION = 0, SHADER_LOC_VERTEX_TEXCOORD01, SHADER_LOC_VERTEX_TEXCOORD02,
SHADER_LOC_VERTEX_NORMAL,
SHADER_LOC_VERTEX_TANGENT, SHADER_LOC_VERTEX_COLOR, SHADER_LOC_MATRIX_MVP,
SHADER_LOC_MATRIX_VIEW,
SHADER_LOC_MATRIX_PROJECTION, SHADER_LOC_MATRIX_MODEL, SHADER_LOC_MATRIX_NORMAL,
SHADER_LOC_VECTOR_VIEW,
SHADER_LOC_COLOR_DIFFUSE, SHADER_LOC_COLOR_SPECULAR, SHADER_LOC_COLOR_AMBIENT,
SHADER_LOC_MAP_ALBEDO,
SHADER_LOC_MAP_METALNESS, SHADER_LOC_MAP_NORMAL, SHADER_LOC_MAP_ROUGHNESS,
SHADER_LOC_MAP_OCCLUSION,
SHADER_LOC_MAP_EMISSION, SHADER_LOC_MAP_HEIGHT, SHADER_LOC_MAP_CUBEMAP,
SHADER_LOC_MAP_IRRADIANCE,
SHADER_LOC_MAP_PREFILTER, SHADER_LOC_MAP_BRDF, SHADER_LOC_VERTEX_BONEIDS,
SHADER_LOC_VERTEX_BONEWEIGHTS,
SHADER_LOC_MATRIX_BONETRANSFORMS, SHADER_LOC_VERTEX_INSTANCE_TRANSFORM }
- enum ShaderUniformDataType {
SHADER_UNIFORM_FLOAT = 0, SHADER_UNIFORM_VEC2, SHADER_UNIFORM_VEC3, SHADER_UNIFORM_VEC4,
SHADER_UNIFORM_INT, SHADER_UNIFORM_IVEC2, SHADER_UNIFORM_IVEC3, SHADER_UNIFORM_IVEC4,
SHADER_UNIFORM_UINT, SHADER_UNIFORM_UVEC2, SHADER_UNIFORM_UVEC3, SHADER_UNIFORM_UVEC4,
SHADER_UNIFORM_SAMPLER2D }
- enum ShaderAttributeDataType { SHADER_ATTRIB_FLOAT = 0, SHADER_ATTRIB_VEC2, SHADER_ATTRIB_VEC3,
SHADER_ATTRIB_VEC4 }
- enum PixelFormat {
PIXELFORMAT_UNCOMPRESSED_GRAYSCALE = 1, PIXELFORMAT_UNCOMPRESSED_GRAY_ALPHA,
PIXELFORMAT_UNCOMPRESSED_R5G6B5, PIXELFORMAT_UNCOMPRESSED_R8G8B8,
PIXELFORMAT_UNCOMPRESSED_R5G5B5A1, PIXELFORMAT_UNCOMPRESSED_R4G4B4A4,
PIXELFORMAT_UNCOMPRESSED_R8G8B8A8, PIXELFORMAT_UNCOMPRESSED_R32,
PIXELFORMAT_UNCOMPRESSED_R32G32B32, PIXELFORMAT_UNCOMPRESSED_R32G32B32A32,
PIXELFORMAT_UNCOMPRESSED_R16, PIXELFORMAT_UNCOMPRESSED_R16G16B16,
PIXELFORMAT_UNCOMPRESSED_R16G16B16A16, PIXELFORMAT_COMPRESSED_DXT1_RGB,
PIXELFORMAT_COMPRESSED_DXT1_RGBA, PIXELFORMAT_COMPRESSED_DXT3_RGBA,
PIXELFORMAT_COMPRESSED_DXT5_RGBA, PIXELFORMAT_COMPRESSED_ETC1_RGB, PIXELFORMAT_COMPRESSED_ETC2_RGB,
PIXELFORMAT_COMPRESSED_ETC2_RGBA, PIXELFORMAT_COMPRESSED_PVRT_RGB, PIXELFORMAT_COMPRESSED_PVRT_RGBA, PIXELFORMAT_COMPRESSED_ASTC_4x4_RGBA }

- , `PIXELFORMAT_COMPRESSED_ASTC_8x8_RGBA` }
- enum `TextureFilter` {
`TEXTURE_FILTER_POINT` = 0 , `TEXTURE_FILTER_BILINEAR` , `TEXTURE_FILTER_TRILINEAR` ,
`TEXTURE_FILTER_ANISOTROPIC_4X` ,
`TEXTURE_FILTER_ANISOTROPIC_8X` , `TEXTURE_FILTER_ANISOTROPIC_16X` }
- enum `TextureWrap` { `TEXTURE_WRAP_REPEAT` = 0 , `TEXTURE_WRAP_CLAMP` , `TEXTURE_WRAP_MIRROR_REPEAT` ,
`TEXTURE_WRAP_MIRROR_CLAMP` }
- enum `CubemapLayout` {
`CUBEMAP_LAYOUT_AUTO_DETECT` = 0 , `CUBEMAP_LAYOUT_LINE_VERTICAL` , `CUBEMAP_LAYOUT_LINE_HORIZONTAL` ,
`CUBEMAP_LAYOUT_CROSS_THREE_BY_FOUR` ,
`CUBEMAP_LAYOUT_CROSS_FOUR_BY_THREE` }
- enum `FontType` { `FONT_DEFAULT` = 0 , `FONT_BITMAP` , `FONT_SDF` }
- enum `BlendMode` {
`BLEND_ALPHA` = 0 , `BLEND_ADDITIVE` , `BLEND_MULTIPLIED` , `BLEND_ADD_COLORS` ,
`BLEND_SUBTRACT_COLORS` , `BLEND_ALPHA_PREMULTIPLY` , `BLEND_CUSTOM` , `BLEND_CUSTOM_SEPARATE`
}
- enum `Gesture` {
`GESTURE_NONE` = 0 , `GESTURE_TAP` = 1 , `GESTURE_DOUBLETAP` = 2 , `GESTURE_HOLD` = 4 ,
`GESTURE_DRAG` = 8 , `GESTURE_SWIPE_RIGHT` = 16 , `GESTURE_SWIPE_LEFT` = 32 , `GESTURE_SWIPE_UP`
= 64 ,
`GESTURE_SWIPE_DOWN` = 128 , `GESTURE_PINCH_IN` = 256 , `GESTURE_PINCH_OUT` = 512 }
- enum `CameraMode` {
`CAMERA_CUSTOM` = 0 , `CAMERA_FREE` , `CAMERA_ORBITAL` , `CAMERA_FIRST_PERSON` ,
`CAMERA_THIRD_PERSON` }
- enum `CameraProjection` { `CAMERA_PERSPECTIVE` = 0 , `CAMERA_ORTHOGRAPHIC` }
- enum `NPatchLayout` { `NPATCH_NINE_PATCH` = 0 , `NPATCH_THREE_PATCH_VERTICAL` , `NPATCH_THREE_PATCH_HORIZONTAL` }

Functions

- **RLAPI** void `InitWindow` (int width, int height, const char *title)
- **RLAPI** void `CloseWindow` (void)
- **RLAPI** bool `WindowShouldClose` (void)
- **RLAPI** bool `IsWindowReady` (void)
- **RLAPI** bool `IsWindowFullscreen` (void)
- **RLAPI** bool `IsWindowHidden` (void)
- **RLAPI** bool `IsWindowMinimized` (void)
- **RLAPI** bool `IsWindowMaximized` (void)
- **RLAPI** bool `IsWindowFocused` (void)
- **RLAPI** bool `IsWindowResized` (void)
- **RLAPI** bool `IsWindowState` (unsigned int flag)
- **RLAPI** void `SetWindowState` (unsigned int flags)
- **RLAPI** void `ClearWindowState` (unsigned int flags)
- **RLAPI** void `ToggleFullscreen` (void)
- **RLAPI** void `ToggleBorderlessWindowed` (void)
- **RLAPI** void `MaximizeWindow` (void)
- **RLAPI** void `MinimizeWindow` (void)
- **RLAPI** void `RestoreWindow` (void)
- **RLAPI** void `SetWindowIcon` (Image image)
- **RLAPI** void `SetWindowIcons` (Image *images, int count)
- **RLAPI** void `SetWindowTitle` (const char *title)
- **RLAPI** void `SetWindowPosition` (int x, int y)
- **RLAPI** void `SetWindowMonitor` (int monitor)
- **RLAPI** void `SetWindowMinSize` (int width, int height)
- **RLAPI** void `SetWindowMaxSize` (int width, int height)

- [RLAPI](#) void [SetWindowSize](#) (int width, int height)
- [RLAPI](#) void [SetWindowOpacity](#) (float opacity)
- [RLAPI](#) void [SetWindowFocused](#) (void)
- [RLAPI](#) void * [GetWindowHandle](#) (void)
- [RLAPI](#) int [GetScreenWidth](#) (void)
- [RLAPI](#) int [GetScreenHeight](#) (void)
- [RLAPI](#) int [GetRenderWidth](#) (void)
- [RLAPI](#) int [GetRenderHeight](#) (void)
- [RLAPI](#) int [GetMonitorCount](#) (void)
- [RLAPI](#) int [GetCurrentMonitor](#) (void)
- [RLAPI](#) [Vector2](#) [GetMonitorPosition](#) (int monitor)
- [RLAPI](#) int [GetMonitorWidth](#) (int monitor)
- [RLAPI](#) int [GetMonitorHeight](#) (int monitor)
- [RLAPI](#) int [GetMonitorPhysicalWidth](#) (int monitor)
- [RLAPI](#) int [GetMonitorPhysicalHeight](#) (int monitor)
- [RLAPI](#) int [GetMonitorRefreshRate](#) (int monitor)
- [RLAPI](#) [Vector2](#) [GetWindowPosition](#) (void)
- [RLAPI](#) [Vector2](#) [GetWindowScaleDPI](#) (void)
- [RLAPI](#) const char * [GetMonitorName](#) (int monitor)
- [RLAPI](#) void [SetClipboardText](#) (const char *text)
- [RLAPI](#) const char * [GetClipboardText](#) (void)
- [RLAPI](#) [Image](#) [GetClipboardImage](#) (void)
- [RLAPI](#) void [EnableEventWaiting](#) (void)
- [RLAPI](#) void [DisableEventWaiting](#) (void)
- [RLAPI](#) void [ShowCursor](#) (void)
- [RLAPI](#) void [HideCursor](#) (void)
- [RLAPI](#) bool [IsCursorHidden](#) (void)
- [RLAPI](#) void [EnableCursor](#) (void)
- [RLAPI](#) void [DisableCursor](#) (void)
- [RLAPI](#) bool [IsCursorOnScreen](#) (void)
- [RLAPI](#) void [ClearBackground](#) ([Color](#) color)
- [RLAPI](#) void [BeginDrawing](#) (void)
- [RLAPI](#) void [EndDrawing](#) (void)
- [RLAPI](#) void [BeginMode2D](#) ([Camera2D](#) camera)
- [RLAPI](#) void [EndMode2D](#) (void)
- [RLAPI](#) void [BeginMode3D](#) ([Camera3D](#) camera)
- [RLAPI](#) void [EndMode3D](#) (void)
- [RLAPI](#) void [BeginTextureMode](#) ([RenderTexture2D](#) target)
- [RLAPI](#) void [EndTextureMode](#) (void)
- [RLAPI](#) void [BeginShaderMode](#) ([Shader](#) shader)
- [RLAPI](#) void [EndShaderMode](#) (void)
- [RLAPI](#) void [BeginBlendMode](#) (int mode)
- [RLAPI](#) void [EndBlendMode](#) (void)
- [RLAPI](#) void [BeginScissorMode](#) (int x, int y, int width, int height)
- [RLAPI](#) void [EndScissorMode](#) (void)
- [RLAPI](#) void [BeginVrStereoMode](#) ([VrStereoConfig](#) config)
- [RLAPI](#) void [EndVrStereoMode](#) (void)
- [RLAPI](#) [VrStereoConfig](#) [LoadVrStereoConfig](#) ([VrDeviceInfo](#) device)
- [RLAPI](#) void [UnloadVrStereoConfig](#) ([VrStereoConfig](#) config)
- [RLAPI](#) [Shader](#) [LoadShader](#) (const char *vsFileName, const char *fsFileName)
- [RLAPI](#) [Shader](#) [LoadShaderFromMemory](#) (const char *vsCode, const char *fsCode)
- [RLAPI](#) bool [IsShaderValid](#) ([Shader](#) shader)
- [RLAPI](#) int [GetShaderLocation](#) ([Shader](#) shader, const char *uniformName)
- [RLAPI](#) int [GetShaderLocationAttrib](#) ([Shader](#) shader, const char *attribName)
- [RLAPI](#) void [SetShaderValue](#) ([Shader](#) shader, int locIndex, const void *value, int uniformType)

- [RLAPI](#) void [SetShaderValueV](#) ([Shader](#) shader, int locIndex, const void *value, int uniformType, int count)
- [RLAPI](#) void [SetShaderValueMatrix](#) ([Shader](#) shader, int locIndex, [Matrix](#) mat)
- [RLAPI](#) void [SetShaderValueTexture](#) ([Shader](#) shader, int locIndex, [Texture2D](#) texture)
- [RLAPI](#) void [UnloadShader](#) ([Shader](#) shader)
- [RLAPI](#) Ray [GetScreenToWorldRay](#) ([Vector2](#) position, [Camera](#) camera)
- [RLAPI](#) Ray [GetScreenToWorldRayEx](#) ([Vector2](#) position, [Camera](#) camera, int width, int height)
- [RLAPI](#) [Vector2](#) [GetWorldToScreen](#) ([Vector3](#) position, [Camera](#) camera)
- [RLAPI](#) [Vector2](#) [GetWorldToScreenEx](#) ([Vector3](#) position, [Camera](#) camera, int width, int height)
- [RLAPI](#) [Vector2](#) [GetWorldToScreen2D](#) ([Vector2](#) position, [Camera2D](#) camera)
- [RLAPI](#) [Vector2](#) [GetScreenToWorld2D](#) ([Vector2](#) position, [Camera2D](#) camera)
- [RLAPI](#) [Matrix](#) [GetCameraMatrix](#) ([Camera](#) camera)
- [RLAPI](#) [Matrix](#) [GetCameraMatrix2D](#) ([Camera2D](#) camera)
- [RLAPI](#) void [SetTargetFPS](#) (int fps)
- [RLAPI](#) float [GetFrameTime](#) (void)
- [RLAPI](#) double [GetTime](#) (void)
- [RLAPI](#) int [GetFPS](#) (void)
- [RLAPI](#) void [SwapScreenBuffer](#) (void)
- [RLAPI](#) void [PollInputEvents](#) (void)
- [RLAPI](#) void [WaitTime](#) (double seconds)
- [RLAPI](#) void [SetRandomSeed](#) (unsigned int seed)
- [RLAPI](#) int [GetRandomValue](#) (int min, int max)
- [RLAPI](#) int * [LoadRandomSequence](#) (unsigned int count, int min, int max)
- [RLAPI](#) void [UnloadRandomSequence](#) (int *sequence)
- [RLAPI](#) void [TakeScreenshot](#) (const char *fileName)
- [RLAPI](#) void [SetConfigFlags](#) (unsigned int flags)
- [RLAPI](#) void [OpenURL](#) (const char *url)
- [RLAPI](#) void [SetTraceLogLevel](#) (int logLevel)
- [RLAPI](#) void [TraceLog](#) (int logLevel, const char *text,...)
- [RLAPI](#) void [SetTraceLogCallback](#) ([TraceLogCallback](#) callback)
- [RLAPI](#) void * [MemAlloc](#) (unsigned int size)
- [RLAPI](#) void * [MemRealloc](#) (void *ptr, unsigned int size)
- [RLAPI](#) void [MemFree](#) (void *ptr)
- [RLAPI](#) unsigned char * [LoadFileData](#) (const char *fileName, int *dataSize)
- [RLAPI](#) void [UnloadFileData](#) (unsigned char *data)
- [RLAPI](#) bool [SaveFileData](#) (const char *fileName, void *data, int dataSize)
- [RLAPI](#) bool [ExportDataAsCode](#) (const unsigned char *data, int dataSize, const char *fileName)
- [RLAPI](#) char * [LoadFileText](#) (const char *fileName)
- [RLAPI](#) void [UnloadFileText](#) (char *text)
- [RLAPI](#) bool [SaveFileText](#) (const char *fileName, const char *text)
- [RLAPI](#) void [SetLoadFileDataCallback](#) ([LoadFileDataCallback](#) callback)
- [RLAPI](#) void [SetSaveFileDataCallback](#) ([SaveFileDataCallback](#) callback)
- [RLAPI](#) void [SetLoadFileTextCallback](#) ([LoadFileTextCallback](#) callback)
- [RLAPI](#) void [SetSaveFileTextCallback](#) ([SaveFileTextCallback](#) callback)
- [RLAPI](#) int [FileRename](#) (const char *fileName, const char *fileRename)
- [RLAPI](#) int [FileRemove](#) (const char *fileName)
- [RLAPI](#) int [FileCopy](#) (const char *srcPath, const char *dstPath)
- [RLAPI](#) int [FileMove](#) (const char *srcPath, const char *dstPath)
- [RLAPI](#) int [FileTextReplace](#) (const char *fileName, const char *search, const char *replacement)
- [RLAPI](#) int [FileTextFindIndex](#) (const char *fileName, const char *search)
- [RLAPI](#) bool [FileExists](#) (const char *fileName)
- [RLAPI](#) bool [DirectoryExists](#) (const char *dirPath)
- [RLAPI](#) bool [IsFileExtension](#) (const char *fileName, const char *ext)
- [RLAPI](#) int [GetFileLength](#) (const char *fileName)
- [RLAPI](#) long [GetFileModTime](#) (const char *fileName)
- [RLAPI](#) const char * [GetFileExtension](#) (const char *fileName)

- [RLAPI](#) `const char * GetFileName` (`const char *filePath`)
- [RLAPI](#) `const char * GetFileNameWithoutExt` (`const char *filePath`)
- [RLAPI](#) `const char * GetDirectoryPath` (`const char *filePath`)
- [RLAPI](#) `const char * GetPrevDirectoryPath` (`const char *dirPath`)
- [RLAPI](#) `const char * GetWorkingDirectory` (`void`)
- [RLAPI](#) `const char * GetApplicationDirectory` (`void`)
- [RLAPI](#) `int MakeDirectory` (`const char *dirPath`)
- [RLAPI](#) `bool ChangeDirectory` (`const char *dirPath`)
- [RLAPI](#) `bool IsPathFile` (`const char *path`)
- [RLAPI](#) `bool IsFileNameValid` (`const char *fileName`)
- [RLAPI](#) `FilePathList LoadDirectoryFiles` (`const char *dirPath`)
- [RLAPI](#) `FilePathList LoadDirectoryFilesEx` (`const char *basePath`, `const char *filter`, `bool scanSubdirs`)
- [RLAPI](#) `void UnloadDirectoryFiles` (`FilePathList files`)
- [RLAPI](#) `bool IsFileDropped` (`void`)
- [RLAPI](#) `FilePathList LoadDroppedFiles` (`void`)
- [RLAPI](#) `void UnloadDroppedFiles` (`FilePathList files`)
- [RLAPI](#) `unsigned int GetDirectoryFileCount` (`const char *dirPath`)
- [RLAPI](#) `unsigned int GetDirectoryFileCountEx` (`const char *basePath`, `const char *filter`, `bool scanSubdirs`)
- [RLAPI](#) `unsigned char * CompressData` (`const unsigned char *data`, `int dataSize`, `int *compDataSize`)
- [RLAPI](#) `unsigned char * DecompressData` (`const unsigned char *compData`, `int compDataSize`, `int *dataSize`)
- [RLAPI](#) `char * EncodeDataBase64` (`const unsigned char *data`, `int dataSize`, `int *outputSize`)
- [RLAPI](#) `unsigned char * DecodeDataBase64` (`const char *text`, `int *outputSize`)
- [RLAPI](#) `unsigned int ComputeCRC32` (`unsigned char *data`, `int dataSize`)
- [RLAPI](#) `unsigned int * ComputeMD5` (`unsigned char *data`, `int dataSize`)
- [RLAPI](#) `unsigned int * ComputeSHA1` (`unsigned char *data`, `int dataSize`)
- [RLAPI](#) `unsigned int * ComputeSHA256` (`unsigned char *data`, `int dataSize`)
- [RLAPI](#) `AutomationEventList LoadAutomationEventList` (`const char *fileName`)
- [RLAPI](#) `void UnloadAutomationEventList` (`AutomationEventList list`)
- [RLAPI](#) `bool ExportAutomationEventList` (`AutomationEventList list`, `const char *fileName`)
- [RLAPI](#) `void SetAutomationEventList` (`AutomationEventList *list`)
- [RLAPI](#) `void SetAutomationEventBaseFrame` (`int frame`)
- [RLAPI](#) `void StartAutomationEventRecording` (`void`)
- [RLAPI](#) `void StopAutomationEventRecording` (`void`)
- [RLAPI](#) `void PlayAutomationEvent` (`AutomationEvent event`)
- [RLAPI](#) `bool IsKeyPressed` (`int key`)
- [RLAPI](#) `bool IsKeyPressedRepeat` (`int key`)
- [RLAPI](#) `bool IsKeyDown` (`int key`)
- [RLAPI](#) `bool IsKeyReleased` (`int key`)
- [RLAPI](#) `bool IsKeyUp` (`int key`)
- [RLAPI](#) `int GetKeyPressed` (`void`)
- [RLAPI](#) `int GetCharPressed` (`void`)
- [RLAPI](#) `const char * GetKeyName` (`int key`)
- [RLAPI](#) `void SetExitKey` (`int key`)
- [RLAPI](#) `bool IsGamepadAvailable` (`int gamepad`)
- [RLAPI](#) `const char * GetGamepadName` (`int gamepad`)
- [RLAPI](#) `bool IsGamepadButtonPressed` (`int gamepad`, `int button`)
- [RLAPI](#) `bool IsGamepadButtonDown` (`int gamepad`, `int button`)
- [RLAPI](#) `bool IsGamepadButtonReleased` (`int gamepad`, `int button`)
- [RLAPI](#) `bool IsGamepadButtonUp` (`int gamepad`, `int button`)
- [RLAPI](#) `int GetGamepadButtonPressed` (`void`)
- [RLAPI](#) `int GetGamepadAxisCount` (`int gamepad`)
- [RLAPI](#) `float GetGamepadAxisMovement` (`int gamepad`, `int axis`)
- [RLAPI](#) `int SetGamepadMappings` (`const char *mappings`)
- [RLAPI](#) `void SetGamepadVibration` (`int gamepad`, `float leftMotor`, `float rightMotor`, `float duration`)
- [RLAPI](#) `bool IsMouseButtonPressed` (`int button`)

- [RLAPI bool IsMouseButtonDown](#) (int button)
- [RLAPI bool IsMouseButtonReleased](#) (int button)
- [RLAPI bool IsMouseButtonUp](#) (int button)
- [RLAPI int GetMouseX](#) (void)
- [RLAPI int GetMouseY](#) (void)
- [RLAPI Vector2 GetMousePosition](#) (void)
- [RLAPI Vector2 GetMouseDelta](#) (void)
- [RLAPI void SetMousePosition](#) (int x, int y)
- [RLAPI void SetMouseOffset](#) (int offsetX, int offsetY)
- [RLAPI void SetMouseScale](#) (float scaleX, float scaleY)
- [RLAPI float GetMouseWheelMove](#) (void)
- [RLAPI Vector2 GetMouseWheelMoveV](#) (void)
- [RLAPI void SetMouseCursor](#) (int cursor)
- [RLAPI int GetTouchX](#) (void)
- [RLAPI int GetTouchY](#) (void)
- [RLAPI Vector2 GetTouchPosition](#) (int index)
- [RLAPI int GetTouchPointId](#) (int index)
- [RLAPI int GetTouchPointCount](#) (void)
- [RLAPI void SetGesturesEnabled](#) (unsigned int flags)
- [RLAPI bool IsGestureDetected](#) (unsigned int gesture)
- [RLAPI int GetGestureDetected](#) (void)
- [RLAPI float GetGestureHoldDuration](#) (void)
- [RLAPI Vector2 GetGestureDragVector](#) (void)
- [RLAPI float GetGestureDragAngle](#) (void)
- [RLAPI Vector2 GetGesturePinchVector](#) (void)
- [RLAPI float GetGesturePinchAngle](#) (void)
- [RLAPI void UpdateCamera](#) ([Camera](#) *camera, int mode)
- [RLAPI void UpdateCameraPro](#) ([Camera](#) *camera, [Vector3](#) movement, [Vector3](#) rotation, float zoom)
- [RLAPI void SetShapesTexture](#) ([Texture2D](#) texture, [Rectangle](#) source)
- [RLAPI Texture2D GetShapesTexture](#) (void)
- [RLAPI Rectangle GetShapesTextureRectangle](#) (void)
- [RLAPI void DrawPixel](#) (int posX, int posY, [Color](#) color)
- [RLAPI void DrawPixelV](#) ([Vector2](#) position, [Color](#) color)
- [RLAPI void DrawLine](#) (int startPosX, int startPosY, int endPosX, int endPosY, [Color](#) color)
- [RLAPI void DrawLineV](#) ([Vector2](#) startPos, [Vector2](#) endPos, [Color](#) color)
- [RLAPI void DrawLineEx](#) ([Vector2](#) startPos, [Vector2](#) endPos, float thick, [Color](#) color)
- [RLAPI void DrawLineStrip](#) (const [Vector2](#) *points, int pointCount, [Color](#) color)
- [RLAPI void DrawLineBezier](#) ([Vector2](#) startPos, [Vector2](#) endPos, float thick, [Color](#) color)
- [RLAPI void DrawLineDashed](#) ([Vector2](#) startPos, [Vector2](#) endPos, int dashSize, int spaceSize, [Color](#) color)
- [RLAPI void DrawCircle](#) (int centerX, int centerY, float radius, [Color](#) color)
- [RLAPI void DrawCircleV](#) ([Vector2](#) center, float radius, [Color](#) color)
- [RLAPI void DrawCircleGradient](#) ([Vector2](#) center, float radius, [Color](#) inner, [Color](#) outer)
- [RLAPI void DrawCircleSector](#) ([Vector2](#) center, float radius, float startAngle, float endAngle, int segments, [Color](#) color)
- [RLAPI void DrawCircleSectorLines](#) ([Vector2](#) center, float radius, float startAngle, float endAngle, int segments, [Color](#) color)
- [RLAPI void DrawCircleLines](#) (int centerX, int centerY, float radius, [Color](#) color)
- [RLAPI void DrawCircleLinesV](#) ([Vector2](#) center, float radius, [Color](#) color)
- [RLAPI void DrawEllipse](#) (int centerX, int centerY, float radiusH, float radiusV, [Color](#) color)
- [RLAPI void DrawEllipseV](#) ([Vector2](#) center, float radiusH, float radiusV, [Color](#) color)
- [RLAPI void DrawEllipseLines](#) (int centerX, int centerY, float radiusH, float radiusV, [Color](#) color)
- [RLAPI void DrawEllipseLinesV](#) ([Vector2](#) center, float radiusH, float radiusV, [Color](#) color)
- [RLAPI void DrawRing](#) ([Vector2](#) center, float innerRadius, float outerRadius, float startAngle, float endAngle, int segments, [Color](#) color)

- **RLAPI** void [DrawRingLines](#) ([Vector2](#) center, float innerRadius, float outerRadius, float startAngle, float endAngle, int segments, [Color](#) color)
- **RLAPI** void [DrawRectangle](#) (int posX, int posY, int width, int height, [Color](#) color)
- **RLAPI** void [DrawRectangleV](#) ([Vector2](#) position, [Vector2](#) size, [Color](#) color)
- **RLAPI** void [DrawRectangleRec](#) ([Rectangle](#) rec, [Color](#) color)
- **RLAPI** void [DrawRectanglePro](#) ([Rectangle](#) rec, [Vector2](#) origin, float rotation, [Color](#) color)
- **RLAPI** void [DrawRectangleGradientV](#) (int posX, int posY, int width, int height, [Color](#) top, [Color](#) bottom)
- **RLAPI** void [DrawRectangleGradientH](#) (int posX, int posY, int width, int height, [Color](#) left, [Color](#) right)
- **RLAPI** void [DrawRectangleGradientEx](#) ([Rectangle](#) rec, [Color](#) topLeft, [Color](#) bottomLeft, [Color](#) bottomRight, [Color](#) topRight)
- **RLAPI** void [DrawRectangleLines](#) (int posX, int posY, int width, int height, [Color](#) color)
- **RLAPI** void [DrawRectangleLinesEx](#) ([Rectangle](#) rec, float lineThick, [Color](#) color)
- **RLAPI** void [DrawRectangleRounded](#) ([Rectangle](#) rec, float roundness, int segments, [Color](#) color)
- **RLAPI** void [DrawRectangleRoundedLines](#) ([Rectangle](#) rec, float roundness, int segments, [Color](#) color)
- **RLAPI** void [DrawRectangleRoundedLinesEx](#) ([Rectangle](#) rec, float roundness, int segments, float lineThick, [Color](#) color)
- **RLAPI** void [DrawTriangle](#) ([Vector2](#) v1, [Vector2](#) v2, [Vector2](#) v3, [Color](#) color)
- **RLAPI** void [DrawTriangleLines](#) ([Vector2](#) v1, [Vector2](#) v2, [Vector2](#) v3, [Color](#) color)
- **RLAPI** void [DrawTriangleFan](#) (const [Vector2](#) *points, int pointCount, [Color](#) color)
- **RLAPI** void [DrawTriangleStrip](#) (const [Vector2](#) *points, int pointCount, [Color](#) color)
- **RLAPI** void [DrawPoly](#) ([Vector2](#) center, int sides, float radius, float rotation, [Color](#) color)
- **RLAPI** void [DrawPolyLines](#) ([Vector2](#) center, int sides, float radius, float rotation, [Color](#) color)
- **RLAPI** void [DrawPolyLinesEx](#) ([Vector2](#) center, int sides, float radius, float rotation, float lineThick, [Color](#) color)
- **RLAPI** void [DrawSplineLinear](#) (const [Vector2](#) *points, int pointCount, float thick, [Color](#) color)
- **RLAPI** void [DrawSplineBasis](#) (const [Vector2](#) *points, int pointCount, float thick, [Color](#) color)
- **RLAPI** void [DrawSplineCatmullRom](#) (const [Vector2](#) *points, int pointCount, float thick, [Color](#) color)
- **RLAPI** void [DrawSplineBezierQuadratic](#) (const [Vector2](#) *points, int pointCount, float thick, [Color](#) color)
- **RLAPI** void [DrawSplineBezierCubic](#) (const [Vector2](#) *points, int pointCount, float thick, [Color](#) color)
- **RLAPI** void [DrawSplineSegmentLinear](#) ([Vector2](#) p1, [Vector2](#) p2, float thick, [Color](#) color)
- **RLAPI** void [DrawSplineSegmentBasis](#) ([Vector2](#) p1, [Vector2](#) p2, [Vector2](#) p3, [Vector2](#) p4, float thick, [Color](#) color)
- **RLAPI** void [DrawSplineSegmentCatmullRom](#) ([Vector2](#) p1, [Vector2](#) p2, [Vector2](#) p3, [Vector2](#) p4, float thick, [Color](#) color)
- **RLAPI** void [DrawSplineSegmentBezierQuadratic](#) ([Vector2](#) p1, [Vector2](#) c2, [Vector2](#) p3, float thick, [Color](#) color)
- **RLAPI** void [DrawSplineSegmentBezierCubic](#) ([Vector2](#) p1, [Vector2](#) c2, [Vector2](#) c3, [Vector2](#) p4, float thick, [Color](#) color)
- **RLAPI** [Vector2](#) [GetSplinePointLinear](#) ([Vector2](#) startPos, [Vector2](#) endPos, float t)
- **RLAPI** [Vector2](#) [GetSplinePointBasis](#) ([Vector2](#) p1, [Vector2](#) p2, [Vector2](#) p3, [Vector2](#) p4, float t)
- **RLAPI** [Vector2](#) [GetSplinePointCatmullRom](#) ([Vector2](#) p1, [Vector2](#) p2, [Vector2](#) p3, [Vector2](#) p4, float t)
- **RLAPI** [Vector2](#) [GetSplinePointBezierQuad](#) ([Vector2](#) p1, [Vector2](#) c2, [Vector2](#) p3, float t)
- **RLAPI** [Vector2](#) [GetSplinePointBezierCubic](#) ([Vector2](#) p1, [Vector2](#) c2, [Vector2](#) c3, [Vector2](#) p4, float t)
- **RLAPI** bool [CheckCollisionRecs](#) ([Rectangle](#) rec1, [Rectangle](#) rec2)
- **RLAPI** bool [CheckCollisionCircles](#) ([Vector2](#) center1, float radius1, [Vector2](#) center2, float radius2)
- **RLAPI** bool [CheckCollisionCircleRec](#) ([Vector2](#) center, float radius, [Rectangle](#) rec)
- **RLAPI** bool [CheckCollisionCircleLine](#) ([Vector2](#) center, float radius, [Vector2](#) p1, [Vector2](#) p2)
- **RLAPI** bool [CheckCollisionPointRec](#) ([Vector2](#) point, [Rectangle](#) rec)
- **RLAPI** bool [CheckCollisionPointCircle](#) ([Vector2](#) point, [Vector2](#) center, float radius)
- **RLAPI** bool [CheckCollisionPointTriangle](#) ([Vector2](#) point, [Vector2](#) p1, [Vector2](#) p2, [Vector2](#) p3)
- **RLAPI** bool [CheckCollisionPointLine](#) ([Vector2](#) point, [Vector2](#) p1, [Vector2](#) p2, int threshold)
- **RLAPI** bool [CheckCollisionPointPoly](#) ([Vector2](#) point, const [Vector2](#) *points, int pointCount)
- **RLAPI** bool [CheckCollisionLines](#) ([Vector2](#) startPos1, [Vector2](#) endPos1, [Vector2](#) startPos2, [Vector2](#) endPos2, [Vector2](#) *collisionPoint)
- **RLAPI** [Rectangle](#) [GetCollisionRec](#) ([Rectangle](#) rec1, [Rectangle](#) rec2)
- **RLAPI** [Image](#) [LoadImage](#) (const char *fileName)
- **RLAPI** [Image](#) [LoadImageRaw](#) (const char *fileName, int width, int height, int format, int headerSize)

- [RLAPI Image LoadImageAnim](#) (const char *fileName, int *frames)
- [RLAPI Image LoadImageAnimFromMemory](#) (const char *fileType, const unsigned char *fileData, int dataSize, int *frames)
- [RLAPI Image LoadImageFromMemory](#) (const char *fileType, const unsigned char *fileData, int dataSize)
- [RLAPI Image LoadImageFromTexture](#) ([Texture2D](#) texture)
- [RLAPI Image LoadImageFromScreen](#) (void)
- [RLAPI bool IsImageValid](#) ([Image](#) image)
- [RLAPI void UnloadImage](#) ([Image](#) image)
- [RLAPI bool ExportImage](#) ([Image](#) image, const char *fileName)
- [RLAPI unsigned char * ExportImageToMemory](#) ([Image](#) image, const char *fileType, int *fileSize)
- [RLAPI bool ExportImageAsCode](#) ([Image](#) image, const char *fileName)
- [RLAPI Image GenImageColor](#) (int width, int height, [Color](#) color)
- [RLAPI Image GenImageGradientLinear](#) (int width, int height, int direction, [Color](#) start, [Color](#) end)
- [RLAPI Image GenImageGradientRadial](#) (int width, int height, float density, [Color](#) inner, [Color](#) outer)
- [RLAPI Image GenImageGradientSquare](#) (int width, int height, float density, [Color](#) inner, [Color](#) outer)
- [RLAPI Image GenImageChecked](#) (int width, int height, int checksX, int checksY, [Color](#) col1, [Color](#) col2)
- [RLAPI Image GenImageWhiteNoise](#) (int width, int height, float factor)
- [RLAPI Image GenImagePerlinNoise](#) (int width, int height, int offsetX, int offsetY, float scale)
- [RLAPI Image GenImageCellular](#) (int width, int height, int tileSize)
- [RLAPI Image GenImageText](#) (int width, int height, const char *text)
- [RLAPI Image ImageCopy](#) ([Image](#) image)
- [RLAPI Image ImageFromImage](#) ([Image](#) image, [Rectangle](#) rec)
- [RLAPI Image ImageFromChannel](#) ([Image](#) image, int selectedChannel)
- [RLAPI Image ImageText](#) (const char *text, int fontSize, [Color](#) color)
- [RLAPI Image ImageTextEx](#) ([Font](#) font, const char *text, float fontSize, float spacing, [Color](#) tint)
- [RLAPI void ImageFormat](#) ([Image](#) *image, int newFormat)
- [RLAPI void ImageToPOT](#) ([Image](#) *image, [Color](#) fill)
- [RLAPI void ImageCrop](#) ([Image](#) *image, [Rectangle](#) crop)
- [RLAPI void ImageAlphaCrop](#) ([Image](#) *image, float threshold)
- [RLAPI void ImageAlphaClear](#) ([Image](#) *image, [Color](#) color, float threshold)
- [RLAPI void ImageAlphaMask](#) ([Image](#) *image, [Image](#) alphaMask)
- [RLAPI void ImageAlphaPremultiply](#) ([Image](#) *image)
- [RLAPI void ImageBlurGaussian](#) ([Image](#) *image, int blurSize)
- [RLAPI void ImageKernelConvolution](#) ([Image](#) *image, const float *kernel, int kernelSize)
- [RLAPI void ImageResize](#) ([Image](#) *image, int newWidth, int newHeight)
- [RLAPI void ImageResizeNN](#) ([Image](#) *image, int newWidth, int newHeight)
- [RLAPI void ImageResizeCanvas](#) ([Image](#) *image, int newWidth, int newHeight, int offsetX, int offsetY, [Color](#) fill)
- [RLAPI void ImageMipmaps](#) ([Image](#) *image)
- [RLAPI void ImageDither](#) ([Image](#) *image, int rBpp, int gBpp, int bBpp, int aBpp)
- [RLAPI void ImageFlipVertical](#) ([Image](#) *image)
- [RLAPI void ImageFlipHorizontal](#) ([Image](#) *image)
- [RLAPI void ImageRotate](#) ([Image](#) *image, int degrees)
- [RLAPI void ImageRotateCW](#) ([Image](#) *image)
- [RLAPI void ImageRotateCCW](#) ([Image](#) *image)
- [RLAPI void ImageColorTint](#) ([Image](#) *image, [Color](#) color)
- [RLAPI void ImageColorInvert](#) ([Image](#) *image)
- [RLAPI void ImageColorGrayscale](#) ([Image](#) *image)
- [RLAPI void ImageColorContrast](#) ([Image](#) *image, float contrast)
- [RLAPI void ImageColorBrightness](#) ([Image](#) *image, int brightness)
- [RLAPI void ImageColorReplace](#) ([Image](#) *image, [Color](#) color, [Color](#) replace)
- [RLAPI Color * LoadImageColors](#) ([Image](#) image)
- [RLAPI Color * LoadImagePalette](#) ([Image](#) image, int maxPaletteSize, int *colorCount)
- [RLAPI void UnloadImageColors](#) ([Color](#) *colors)
- [RLAPI void UnloadImagePalette](#) ([Color](#) *colors)

- [RLAPI Rectangle GetImageAlphaBorder](#) ([Image](#) image, float threshold)
- [RLAPI Color GetImageColor](#) ([Image](#) image, int x, int y)
- [RLAPI void ImageClearBackground](#) ([Image](#) *dst, [Color](#) color)
- [RLAPI void ImageDrawPixel](#) ([Image](#) *dst, int posX, int posY, [Color](#) color)
- [RLAPI void ImageDrawPixelV](#) ([Image](#) *dst, [Vector2](#) position, [Color](#) color)
- [RLAPI void ImageDrawLine](#) ([Image](#) *dst, int startPosX, int startPosY, int endPosX, int endPosY, [Color](#) color)
- [RLAPI void ImageDrawLineV](#) ([Image](#) *dst, [Vector2](#) start, [Vector2](#) end, [Color](#) color)
- [RLAPI void ImageDrawLineEx](#) ([Image](#) *dst, [Vector2](#) start, [Vector2](#) end, int thick, [Color](#) color)
- [RLAPI void ImageDrawCircle](#) ([Image](#) *dst, int centerX, int centerY, int radius, [Color](#) color)
- [RLAPI void ImageDrawCircleV](#) ([Image](#) *dst, [Vector2](#) center, int radius, [Color](#) color)
- [RLAPI void ImageDrawCircleLines](#) ([Image](#) *dst, int centerX, int centerY, int radius, [Color](#) color)
- [RLAPI void ImageDrawCircleLinesV](#) ([Image](#) *dst, [Vector2](#) center, int radius, [Color](#) color)
- [RLAPI void ImageDrawRectangle](#) ([Image](#) *dst, int posX, int posY, int width, int height, [Color](#) color)
- [RLAPI void ImageDrawRectangleV](#) ([Image](#) *dst, [Vector2](#) position, [Vector2](#) size, [Color](#) color)
- [RLAPI void ImageDrawRectangleRec](#) ([Image](#) *dst, [Rectangle](#) rec, [Color](#) color)
- [RLAPI void ImageDrawRectangleLines](#) ([Image](#) *dst, [Rectangle](#) rec, int thick, [Color](#) color)
- [RLAPI void ImageDrawTriangle](#) ([Image](#) *dst, [Vector2](#) v1, [Vector2](#) v2, [Vector2](#) v3, [Color](#) color)
- [RLAPI void ImageDrawTriangleEx](#) ([Image](#) *dst, [Vector2](#) v1, [Vector2](#) v2, [Vector2](#) v3, [Color](#) c1, [Color](#) c2, [Color](#) c3)
- [RLAPI void ImageDrawTriangleLines](#) ([Image](#) *dst, [Vector2](#) v1, [Vector2](#) v2, [Vector2](#) v3, [Color](#) color)
- [RLAPI void ImageDrawTriangleFan](#) ([Image](#) *dst, const [Vector2](#) *points, int pointCount, [Color](#) color)
- [RLAPI void ImageDrawTriangleStrip](#) ([Image](#) *dst, const [Vector2](#) *points, int pointCount, [Color](#) color)
- [RLAPI void ImageDraw](#) ([Image](#) *dst, [Image](#) src, [Rectangle](#) srcRec, [Rectangle](#) dstRec, [Color](#) tint)
- [RLAPI void ImageDrawText](#) ([Image](#) *dst, const char *text, int posX, int posY, int fontSize, [Color](#) color)
- [RLAPI void ImageDrawTextEx](#) ([Image](#) *dst, [Font](#) font, const char *text, [Vector2](#) position, float fontSize, float spacing, [Color](#) tint)
- [RLAPI Texture2D LoadTexture](#) (const char *fileName)
- [RLAPI Texture2D LoadTextureFromImage](#) ([Image](#) image)
- [RLAPI TextureCubemap LoadTextureCubemap](#) ([Image](#) image, int layout)
- [RLAPI RenderTexture2D LoadRenderTexture](#) (int width, int height)
- [RLAPI bool IsTextureValid](#) ([Texture2D](#) texture)
- [RLAPI void UnloadTexture](#) ([Texture2D](#) texture)
- [RLAPI bool IsRenderTextureValid](#) ([RenderTexture2D](#) target)
- [RLAPI void UnloadRenderTexture](#) ([RenderTexture2D](#) target)
- [RLAPI void UpdateTexture](#) ([Texture2D](#) texture, const void *pixels)
- [RLAPI void UpdateTextureRec](#) ([Texture2D](#) texture, [Rectangle](#) rec, const void *pixels)
- [RLAPI void GenTextureMipmaps](#) ([Texture2D](#) *texture)
- [RLAPI void SetTextureFilter](#) ([Texture2D](#) texture, int filter)
- [RLAPI void SetTextureWrap](#) ([Texture2D](#) texture, int wrap)
- [RLAPI void DrawTexture](#) ([Texture2D](#) texture, int posX, int posY, [Color](#) tint)
- [RLAPI void DrawTextureV](#) ([Texture2D](#) texture, [Vector2](#) position, [Color](#) tint)
- [RLAPI void DrawTextureEx](#) ([Texture2D](#) texture, [Vector2](#) position, float rotation, float scale, [Color](#) tint)
- [RLAPI void DrawTextureRec](#) ([Texture2D](#) texture, [Rectangle](#) source, [Vector2](#) position, [Color](#) tint)
- [RLAPI void DrawTexturePro](#) ([Texture2D](#) texture, [Rectangle](#) source, [Rectangle](#) dest, [Vector2](#) origin, float rotation, [Color](#) tint)
- [RLAPI void DrawTextureNPatch](#) ([Texture2D](#) texture, [NPatchInfo](#) nPatchInfo, [Rectangle](#) dest, [Vector2](#) origin, float rotation, [Color](#) tint)
- [RLAPI bool ColorIsEqual](#) ([Color](#) col1, [Color](#) col2)
- [RLAPI Color Fade](#) ([Color](#) color, float alpha)
- [RLAPI int ColorToInt](#) ([Color](#) color)
- [RLAPI Vector4 ColorNormalize](#) ([Color](#) color)
- [RLAPI Color ColorFromNormalized](#) ([Vector4](#) normalized)
- [RLAPI Vector3 ColorToHSV](#) ([Color](#) color)
- [RLAPI Color ColorFromHSV](#) (float hue, float saturation, float value)
- [RLAPI Color ColorTint](#) ([Color](#) color, [Color](#) tint)

- [RLAPI Color ColorBrightness](#) ([Color](#) color, float factor)
- [RLAPI Color ColorContrast](#) ([Color](#) color, float contrast)
- [RLAPI Color ColorAlpha](#) ([Color](#) color, float alpha)
- [RLAPI Color ColorAlphaBlend](#) ([Color](#) dst, [Color](#) src, [Color](#) tint)
- [RLAPI Color ColorLerp](#) ([Color](#) color1, [Color](#) color2, float factor)
- [RLAPI Color GetColor](#) (unsigned int hexValue)
- [RLAPI Color GetPixelColor](#) (void *srcPtr, int format)
- [RLAPI void SetPixelColor](#) (void *dstPtr, [Color](#) color, int format)
- [RLAPI int GetPixelDataSize](#) (int width, int height, int format)
- [RLAPI Font GetFontDefault](#) (void)
- [RLAPI Font LoadFont](#) (const char *fileName)
- [RLAPI Font LoadFontEx](#) (const char *fileName, int fontSize, const int *codepoints, int codepointCount)
- [RLAPI Font LoadFontFromImage](#) ([Image](#) image, [Color](#) key, int firstChar)
- [RLAPI Font LoadFontFromMemory](#) (const char *fileType, const unsigned char *fileData, int dataSize, int fontSize, const int *codepoints, int codepointCount)
- [RLAPI bool IsFontValid](#) ([Font](#) font)
- [RLAPI GlyphInfo * LoadFontData](#) (const unsigned char *fileData, int dataSize, int fontSize, const int *codepoints, int codepointCount, int type, int *glyphCount)
- [RLAPI Image GenImageFontAtlas](#) (const [GlyphInfo](#) *glyphs, [Rectangle](#) **glyphRecs, int glyphCount, int fontSize, int padding, int packMethod)
- [RLAPI void UnloadFontData](#) ([GlyphInfo](#) *glyphs, int glyphCount)
- [RLAPI void UnloadFont](#) ([Font](#) font)
- [RLAPI bool ExportFontAsCode](#) ([Font](#) font, const char *fileName)
- [RLAPI void DrawFPS](#) (int posX, int posY)
- [RLAPI void DrawText](#) (const char *text, int posX, int posY, int fontSize, [Color](#) color)
- [RLAPI void DrawTextEx](#) ([Font](#) font, const char *text, [Vector2](#) position, float fontSize, float spacing, [Color](#) tint)
- [RLAPI void DrawTextPro](#) ([Font](#) font, const char *text, [Vector2](#) position, [Vector2](#) origin, float rotation, float fontSize, float spacing, [Color](#) tint)
- [RLAPI void DrawTextCodepoint](#) ([Font](#) font, int codepoint, [Vector2](#) position, float fontSize, [Color](#) tint)
- [RLAPI void DrawTextCodepoints](#) ([Font](#) font, const int *codepoints, int codepointCount, [Vector2](#) position, float fontSize, float spacing, [Color](#) tint)
- [RLAPI void SetTextLineSpacing](#) (int spacing)
- [RLAPI int MeasureText](#) (const char *text, int fontSize)
- [RLAPI Vector2 MeasureTextEx](#) ([Font](#) font, const char *text, float fontSize, float spacing)
- [RLAPI Vector2 MeasureTextCodepoints](#) ([Font](#) font, const int *codepoints, int length, float fontSize, float spacing)
- [RLAPI int GetGlyphIndex](#) ([Font](#) font, int codepoint)
- [RLAPI GlyphInfo GetGlyphInfo](#) ([Font](#) font, int codepoint)
- [RLAPI Rectangle GetGlyphAtlasRec](#) ([Font](#) font, int codepoint)
- [RLAPI char * LoadUTF8](#) (const int *codepoints, int length)
- [RLAPI void UnloadUTF8](#) (char *text)
- [RLAPI int * LoadCodepoints](#) (const char *text, int *count)
- [RLAPI void UnloadCodepoints](#) (int *codepoints)
- [RLAPI int GetCodepointCount](#) (const char *text)
- [RLAPI int GetCodepoint](#) (const char *text, int *codepointSize)
- [RLAPI int GetCodepointNext](#) (const char *text, int *codepointSize)
- [RLAPI int GetCodepointPrevious](#) (const char *text, int *codepointSize)
- [RLAPI const char * CodepointToUTF8](#) (int codepoint, int *utf8Size)
- [RLAPI char ** LoadTextLines](#) (const char *text, int *count)
- [RLAPI void UnloadTextLines](#) (char **text, int lineCount)
- [RLAPI int TextCopy](#) (char *dst, const char *src)
- [RLAPI bool TextIsEqual](#) (const char *text1, const char *text2)
- [RLAPI unsigned int TextLength](#) (const char *text)
- [RLAPI const char * TextFormat](#) (const char *text,...)
- [RLAPI const char * TextSubtext](#) (const char *text, int position, int length)

- **RLAPI** const char * [TextRemoveSpaces](#) (const char *text)
- **RLAPI** char * [GetTextBetween](#) (const char *text, const char *begin, const char *end)
- **RLAPI** char * [TextReplace](#) (const char *text, const char *search, const char *replacement)
- **RLAPI** char * [TextReplaceAlloc](#) (const char *text, const char *search, const char *replacement)
- **RLAPI** char * [TextReplaceBetween](#) (const char *text, const char *begin, const char *end, const char *replacement)
- **RLAPI** char * [TextReplaceBetweenAlloc](#) (const char *text, const char *begin, const char *end, const char *replacement)
- **RLAPI** char * [TextInsert](#) (const char *text, const char *insert, int position)
- **RLAPI** char * [TextInsertAlloc](#) (const char *text, const char *insert, int position)
- **RLAPI** char * [TextJoin](#) (char **textList, int count, const char *delimiter)
- **RLAPI** char ** [TextSplit](#) (const char *text, char delimiter, int *count)
- **RLAPI** void [TextAppend](#) (char *text, const char *append, int *position)
- **RLAPI** int [TextFindIndex](#) (const char *text, const char *search)
- **RLAPI** char * [TextToUpper](#) (const char *text)
- **RLAPI** char * [TextToLower](#) (const char *text)
- **RLAPI** char * [TextToPascal](#) (const char *text)
- **RLAPI** char * [TextToSnake](#) (const char *text)
- **RLAPI** char * [TextToCamel](#) (const char *text)
- **RLAPI** int [TextToInteger](#) (const char *text)
- **RLAPI** float [TextToFloat](#) (const char *text)
- **RLAPI** void [DrawLine3D](#) ([Vector3](#) startPos, [Vector3](#) endPos, [Color](#) color)
- **RLAPI** void [DrawPoint3D](#) ([Vector3](#) position, [Color](#) color)
- **RLAPI** void [DrawCircle3D](#) ([Vector3](#) center, float radius, [Vector3](#) rotationAxis, float rotationAngle, [Color](#) color)
- **RLAPI** void [DrawTriangle3D](#) ([Vector3](#) v1, [Vector3](#) v2, [Vector3](#) v3, [Color](#) color)
- **RLAPI** void [DrawTriangleStrip3D](#) (const [Vector3](#) *points, int pointCount, [Color](#) color)
- **RLAPI** void [DrawCube](#) ([Vector3](#) position, float width, float height, float length, [Color](#) color)
- **RLAPI** void [DrawCubeV](#) ([Vector3](#) position, [Vector3](#) size, [Color](#) color)
- **RLAPI** void [DrawCubeWires](#) ([Vector3](#) position, float width, float height, float length, [Color](#) color)
- **RLAPI** void [DrawCubeWiresV](#) ([Vector3](#) position, [Vector3](#) size, [Color](#) color)
- **RLAPI** void [DrawSphere](#) ([Vector3](#) centerPos, float radius, [Color](#) color)
- **RLAPI** void [DrawSphereEx](#) ([Vector3](#) centerPos, float radius, int rings, int slices, [Color](#) color)
- **RLAPI** void [DrawSphereWires](#) ([Vector3](#) centerPos, float radius, int rings, int slices, [Color](#) color)
- **RLAPI** void [DrawCylinder](#) ([Vector3](#) position, float radiusTop, float radiusBottom, float height, int slices, [Color](#) color)
- **RLAPI** void [DrawCylinderEx](#) ([Vector3](#) startPos, [Vector3](#) endPos, float startRadius, float endRadius, int sides, [Color](#) color)
- **RLAPI** void [DrawCylinderWires](#) ([Vector3](#) position, float radiusTop, float radiusBottom, float height, int slices, [Color](#) color)
- **RLAPI** void [DrawCylinderWiresEx](#) ([Vector3](#) startPos, [Vector3](#) endPos, float startRadius, float endRadius, int sides, [Color](#) color)
- **RLAPI** void [DrawCapsule](#) ([Vector3](#) startPos, [Vector3](#) endPos, float radius, int slices, int rings, [Color](#) color)
- **RLAPI** void [DrawCapsuleWires](#) ([Vector3](#) startPos, [Vector3](#) endPos, float radius, int slices, int rings, [Color](#) color)
- **RLAPI** void [DrawPlane](#) ([Vector3](#) centerPos, [Vector2](#) size, [Color](#) color)
- **RLAPI** void [DrawRay](#) ([Ray](#) ray, [Color](#) color)
- **RLAPI** void [DrawGrid](#) (int slices, float spacing)
- **RLAPI** [Model](#) [LoadModel](#) (const char *fileName)
- **RLAPI** [Model](#) [LoadModelFromMesh](#) ([Mesh](#) mesh)
- **RLAPI** bool [IsValidModel](#) ([Model](#) model)
- **RLAPI** void [UnloadModel](#) ([Model](#) model)
- **RLAPI** [BoundingBox](#) [GetModelBoundingBox](#) ([Model](#) model)
- **RLAPI** void [DrawModel](#) ([Model](#) model, [Vector3](#) position, float scale, [Color](#) tint)
- **RLAPI** void [DrawModelEx](#) ([Model](#) model, [Vector3](#) position, [Vector3](#) rotationAxis, float rotationAngle, [Vector3](#) scale, [Color](#) tint)

- [RLAPI](#) void [DrawModelWires](#) ([Model](#) model, [Vector3](#) position, float scale, [Color](#) tint)
- [RLAPI](#) void [DrawModelWiresEx](#) ([Model](#) model, [Vector3](#) position, [Vector3](#) rotationAxis, float rotationAngle, [Vector3](#) scale, [Color](#) tint)
- [RLAPI](#) void [DrawBoundingBox](#) ([BoundingBox](#) box, [Color](#) color)
- [RLAPI](#) void [DrawBillboard](#) ([Camera](#) camera, [Texture2D](#) texture, [Vector3](#) position, float scale, [Color](#) tint)
- [RLAPI](#) void [DrawBillboardRec](#) ([Camera](#) camera, [Texture2D](#) texture, [Rectangle](#) source, [Vector3](#) position, [Vector2](#) size, [Color](#) tint)
- [RLAPI](#) void [DrawBillboardPro](#) ([Camera](#) camera, [Texture2D](#) texture, [Rectangle](#) source, [Vector3](#) position, [Vector3](#) up, [Vector2](#) size, [Vector2](#) origin, float rotation, [Color](#) tint)
- [RLAPI](#) void [UploadMesh](#) ([Mesh](#) *mesh, bool dynamic)
- [RLAPI](#) void [UpdateMeshBuffer](#) ([Mesh](#) mesh, int index, const void *data, int dataSize, int offset)
- [RLAPI](#) void [UnloadMesh](#) ([Mesh](#) mesh)
- [RLAPI](#) void [DrawMesh](#) ([Mesh](#) mesh, [Material](#) material, [Matrix](#) transform)
- [RLAPI](#) void [DrawMeshInstanced](#) ([Mesh](#) mesh, [Material](#) material, const [Matrix](#) *transforms, int instances)
- [RLAPI](#) [BoundingBox](#) [GetMeshBoundingBox](#) ([Mesh](#) mesh)
- [RLAPI](#) void [GenMeshTangents](#) ([Mesh](#) *mesh)
- [RLAPI](#) bool [ExportMesh](#) ([Mesh](#) mesh, const char *fileName)
- [RLAPI](#) bool [ExportMeshAsCode](#) ([Mesh](#) mesh, const char *fileName)
- [RLAPI](#) [Mesh](#) [GenMeshPoly](#) (int sides, float radius)
- [RLAPI](#) [Mesh](#) [GenMeshPlane](#) (float width, float length, int resX, int resZ)
- [RLAPI](#) [Mesh](#) [GenMeshCube](#) (float width, float height, float length)
- [RLAPI](#) [Mesh](#) [GenMeshSphere](#) (float radius, int rings, int slices)
- [RLAPI](#) [Mesh](#) [GenMeshHemiSphere](#) (float radius, int rings, int slices)
- [RLAPI](#) [Mesh](#) [GenMeshCylinder](#) (float radius, float height, int slices)
- [RLAPI](#) [Mesh](#) [GenMeshCone](#) (float radius, float height, int slices)
- [RLAPI](#) [Mesh](#) [GenMeshTorus](#) (float radius, float size, int radSeg, int sides)
- [RLAPI](#) [Mesh](#) [GenMeshKnot](#) (float radius, float size, int radSeg, int sides)
- [RLAPI](#) [Mesh](#) [GenMeshHeightmap](#) ([Image](#) heightmap, [Vector3](#) size)
- [RLAPI](#) [Mesh](#) [GenMeshCubicmap](#) ([Image](#) cubicmap, [Vector3](#) cubeSize)
- [RLAPI](#) [Material](#) * [LoadMaterials](#) (const char *fileName, int *materialCount)
- [RLAPI](#) [Material](#) [LoadMaterialDefault](#) (void)
- [RLAPI](#) bool [IsMaterialValid](#) ([Material](#) material)
- [RLAPI](#) void [UnloadMaterial](#) ([Material](#) material)
- [RLAPI](#) void [SetMaterialTexture](#) ([Material](#) *material, int mapType, [Texture2D](#) texture)
- [RLAPI](#) void [SetModelMeshMaterial](#) ([Model](#) *model, int meshId, int materialId)
- [RLAPI](#) [ModelAnimation](#) * [LoadModelAnimations](#) (const char *fileName, int *animCount)
- [RLAPI](#) void [UpdateModelAnimation](#) ([Model](#) model, [ModelAnimation](#) anim, float frame)
- [RLAPI](#) void [UpdateModelAnimationEx](#) ([Model](#) model, [ModelAnimation](#) animA, float frameA, [ModelAnimation](#) animB, float frameB, float blend)
- [RLAPI](#) void [UnloadModelAnimations](#) ([ModelAnimation](#) *animations, int animCount)
- [RLAPI](#) bool [IsModelAnimationValid](#) ([Model](#) model, [ModelAnimation](#) anim)
- [RLAPI](#) bool [CheckCollisionSpheres](#) ([Vector3](#) center1, float radius1, [Vector3](#) center2, float radius2)
- [RLAPI](#) bool [CheckCollisionBoxes](#) ([BoundingBox](#) box1, [BoundingBox](#) box2)
- [RLAPI](#) bool [CheckCollisionBoxSphere](#) ([BoundingBox](#) box, [Vector3](#) center, float radius)
- [RLAPI](#) [RayCollision](#) [GetRayCollisionSphere](#) ([Ray](#) ray, [Vector3](#) center, float radius)
- [RLAPI](#) [RayCollision](#) [GetRayCollisionBox](#) ([Ray](#) ray, [BoundingBox](#) box)
- [RLAPI](#) [RayCollision](#) [GetRayCollisionMesh](#) ([Ray](#) ray, [Mesh](#) mesh, [Matrix](#) transform)
- [RLAPI](#) [RayCollision](#) [GetRayCollisionTriangle](#) ([Ray](#) ray, [Vector3](#) p1, [Vector3](#) p2, [Vector3](#) p3)
- [RLAPI](#) [RayCollision](#) [GetRayCollisionQuad](#) ([Ray](#) ray, [Vector3](#) p1, [Vector3](#) p2, [Vector3](#) p3, [Vector3](#) p4)
- [RLAPI](#) void [InitAudioDevice](#) (void)
- [RLAPI](#) void [CloseAudioDevice](#) (void)
- [RLAPI](#) bool [IsAudioDeviceReady](#) (void)
- [RLAPI](#) void [SetMasterVolume](#) (float volume)
- [RLAPI](#) float [GetMasterVolume](#) (void)
- [RLAPI](#) [Wave](#) [LoadWave](#) (const char *fileName)

- [RLAPI Wave LoadWaveFromMemory](#) (const char *fileType, const unsigned char *fileData, int dataSize)
- [RLAPI bool IsWaveValid](#) ([Wave](#) wave)
- [RLAPI Sound LoadSound](#) (const char *fileName)
- [RLAPI Sound LoadSoundFromWave](#) ([Wave](#) wave)
- [RLAPI Sound LoadSoundAlias](#) ([Sound](#) source)
- [RLAPI bool IsSoundValid](#) ([Sound](#) sound)
- [RLAPI void UpdateSound](#) ([Sound](#) sound, const void *data, int sampleCount)
- [RLAPI void UnloadWave](#) ([Wave](#) wave)
- [RLAPI void UnloadSound](#) ([Sound](#) sound)
- [RLAPI void UnloadSoundAlias](#) ([Sound](#) alias)
- [RLAPI bool ExportWave](#) ([Wave](#) wave, const char *fileName)
- [RLAPI bool ExportWaveAsCode](#) ([Wave](#) wave, const char *fileName)
- [RLAPI void PlaySound](#) ([Sound](#) sound)
- [RLAPI void StopSound](#) ([Sound](#) sound)
- [RLAPI void PauseSound](#) ([Sound](#) sound)
- [RLAPI void ResumeSound](#) ([Sound](#) sound)
- [RLAPI bool IsSoundPlaying](#) ([Sound](#) sound)
- [RLAPI void SetSoundVolume](#) ([Sound](#) sound, float volume)
- [RLAPI void SetSoundPitch](#) ([Sound](#) sound, float pitch)
- [RLAPI void SetSoundPan](#) ([Sound](#) sound, float pan)
- [RLAPI Wave WaveCopy](#) ([Wave](#) wave)
- [RLAPI void WaveCrop](#) ([Wave](#) *wave, int initFrame, int finalFrame)
- [RLAPI void WaveFormat](#) ([Wave](#) *wave, int sampleRate, int sampleSize, int channels)
- [RLAPI float * LoadWaveSamples](#) ([Wave](#) wave)
- [RLAPI void UnloadWaveSamples](#) (float *samples)
- [RLAPI Music LoadMusicStream](#) (const char *fileName)
- [RLAPI Music LoadMusicStreamFromMemory](#) (const char *fileType, const unsigned char *data, int dataSize)
- [RLAPI bool IsMusicValid](#) ([Music](#) music)
- [RLAPI void UnloadMusicStream](#) ([Music](#) music)
- [RLAPI void PlayMusicStream](#) ([Music](#) music)
- [RLAPI bool IsMusicStreamPlaying](#) ([Music](#) music)
- [RLAPI void UpdateMusicStream](#) ([Music](#) music)
- [RLAPI void StopMusicStream](#) ([Music](#) music)
- [RLAPI void PauseMusicStream](#) ([Music](#) music)
- [RLAPI void ResumeMusicStream](#) ([Music](#) music)
- [RLAPI void SeekMusicStream](#) ([Music](#) music, float position)
- [RLAPI void SetMusicVolume](#) ([Music](#) music, float volume)
- [RLAPI void SetMusicPitch](#) ([Music](#) music, float pitch)
- [RLAPI void SetMusicPan](#) ([Music](#) music, float pan)
- [RLAPI float GetMusicTimeLength](#) ([Music](#) music)
- [RLAPI float GetMusicTimePlayed](#) ([Music](#) music)
- [RLAPI AudioStream LoadAudioStream](#) (unsigned int sampleRate, unsigned int sampleSize, unsigned int channels)
- [RLAPI bool IsAudioStreamValid](#) ([AudioStream](#) stream)
- [RLAPI void UnloadAudioStream](#) ([AudioStream](#) stream)
- [RLAPI void UpdateAudioStream](#) ([AudioStream](#) stream, const void *data, int frameCount)
- [RLAPI bool IsAudioStreamProcessed](#) ([AudioStream](#) stream)
- [RLAPI void PlayAudioStream](#) ([AudioStream](#) stream)
- [RLAPI void PauseAudioStream](#) ([AudioStream](#) stream)
- [RLAPI void ResumeAudioStream](#) ([AudioStream](#) stream)
- [RLAPI bool IsAudioStreamPlaying](#) ([AudioStream](#) stream)
- [RLAPI void StopAudioStream](#) ([AudioStream](#) stream)
- [RLAPI void SetAudioStreamVolume](#) ([AudioStream](#) stream, float volume)
- [RLAPI void SetAudioStreamPitch](#) ([AudioStream](#) stream, float pitch)
- [RLAPI void SetAudioStreamPan](#) ([AudioStream](#) stream, float pan)

- [RLAPI](#) void [SetAudioStreamBufferSizeDefault](#) (int size)
- [RLAPI](#) void [SetAudioStreamCallback](#) ([AudioStream](#) stream, [AudioCallback](#) callback)
- [RLAPI](#) void [AttachAudioStreamProcessor](#) ([AudioStream](#) stream, [AudioCallback](#) processor)
- [RLAPI](#) void [DetachAudioStreamProcessor](#) ([AudioStream](#) stream, [AudioCallback](#) processor)
- [RLAPI](#) void [AttachAudioMixedProcessor](#) ([AudioCallback](#) processor)
- [RLAPI](#) void [DetachAudioMixedProcessor](#) ([AudioCallback](#) processor)

6.23.1 Macro Definition Documentation

6.23.1.1 BEIGE

```
#define BEIGE CLITERAL(Color){ 211, 176, 131, 255 }
```

6.23.1.2 BLACK

```
#define BLACK CLITERAL(Color){ 0, 0, 0, 255 }
```

6.23.1.3 BLANK

```
#define BLANK CLITERAL(Color){ 0, 0, 0, 0 }
```

6.23.1.4 BLUE

```
#define BLUE CLITERAL(Color){ 0, 121, 241, 255 }
```

6.23.1.5 BROWN

```
#define BROWN CLITERAL(Color){ 127, 106, 79, 255 }
```

6.23.1.6 CLITERAL

```
#define CLITERAL(  
    type)
```

Value:

(type)

6.23.1.7 DARKBLUE

```
#define DARKBLUE CLITERAL(Color){ 0, 82, 172, 255 }
```

6.23.1.8 DARKBROWN

```
#define DARKBROWN CLITERAL(Color){ 76, 63, 47, 255 }
```

6.23.1.9 DARKGRAY

```
#define DARKGRAY CLITERAL(Color){ 80, 80, 80, 255 }
```

6.23.1.10 DARKGREEN

```
#define DARKGREEN CLITERAL(Color){ 0, 117, 44, 255 }
```

6.23.1.11 DARKPURPLE

```
#define DARKPURPLE CLITERAL(Color){ 112, 31, 126, 255 }
```

6.23.1.12 DEG2RAD

```
#define DEG2RAD (PI/180.0f)
```

6.23.1.13 GetMouseRay

```
#define GetMouseRay GetScreenToWorldRay
```

6.23.1.14 GOLD

```
#define GOLD CLITERAL(Color){ 255, 203, 0, 255 }
```

6.23.1.15 GRAY

```
#define GRAY CLITERAL(Color){ 130, 130, 130, 255 }
```

6.23.1.16 GREEN

```
#define GREEN CLITERAL(Color){ 0, 228, 48, 255 }
```

6.23.1.17 LIGHTGRAY

```
#define LIGHTGRAY CLITERAL(Color){ 200, 200, 200, 255 }
```

6.23.1.18 LIME

```
#define LIME CLITERAL(Color){ 0, 158, 47, 255 }
```


6.23.1.19 MAGENTA

```
#define MAGENTA CLITERAL(Color){ 255, 0, 255, 255 }
```

6.23.1.20 MAROON

```
#define MAROON CLITERAL(Color){ 190, 33, 55, 255 }
```

6.23.1.21 MATERIAL_MAP_DIFFUSE

```
#define MATERIAL_MAP_DIFFUSE MATERIAL_MAP_ALBEDO
```

6.23.1.22 MATERIAL_MAP_SPECULAR

```
#define MATERIAL_MAP_SPECULAR MATERIAL_MAP_METALNESS
```

6.23.1.23 MOUSE_LEFT_BUTTON

```
#define MOUSE_LEFT_BUTTON MOUSE_BUTTON_LEFT
```

6.23.1.24 MOUSE_MIDDLE_BUTTON

```
#define MOUSE_MIDDLE_BUTTON MOUSE_BUTTON_MIDDLE
```

6.23.1.25 MOUSE_RIGHT_BUTTON

```
#define MOUSE_RIGHT_BUTTON MOUSE_BUTTON_RIGHT
```

6.23.1.26 ORANGE

```
#define ORANGE CLITERAL(Color){ 255, 161, 0, 255 }
```

6.23.1.27 PI

```
#define PI 3.14159265358979323846f
```

6.23.1.28 PINK

```
#define PINK CLITERAL(Color){ 255, 109, 194, 255 }
```

6.23.1.29 PURPLE

```
#define PURPLE CLITERAL(Color){ 200, 122, 255, 255 }
```

6.23.1.30 RAD2DEG

```
#define RAD2DEG (180.0f/PI)
```

6.23.1.31 RAYLIB_VERSION

```
#define RAYLIB_VERSION "6.0"
```

6.23.1.32 RAYLIB_VERSION_MAJOR

```
#define RAYLIB_VERSION_MAJOR 6
```

6.23.1.33 RAYLIB_VERSION_MINOR

```
#define RAYLIB_VERSION_MINOR 0
```

6.23.1.34 RAYLIB_VERSION_PATCH

```
#define RAYLIB_VERSION_PATCH 0
```

6.23.1.35 RAYWHITE

```
#define RAYWHITE CLITERAL(Color){ 245, 245, 245, 255 }
```

6.23.1.36 RED

```
#define RED CLITERAL(Color){ 230, 41, 55, 255 }
```

6.23.1.37 RL_BOOL_TYPE

```
#define RL_BOOL_TYPE
```

6.23.1.38 RL_CALLOC

```
#define RL_CALLOC(  
    n,  
    sz)
```

Value:

```
calloc(n,sz)
```

6.23.1.39 RL_COLOR_TYPE

```
#define RL_COLOR_TYPE
```

6.23.1.40 RL_FREE

```
#define RL_FREE(  
    ptr)
```

Value:

```
free(ptr)
```

6.23.1.41 RL_MALLOC

```
#define RL_MALLOC(  
    sz)
```

Value:

```
malloc(sz)
```

6.23.1.42 RL_MATRIX_TYPE

```
#define RL_MATRIX_TYPE
```

6.23.1.43 RL_QUATERNION_TYPE

```
#define RL_QUATERNION_TYPE
```

6.23.1.44 RL_REALLOC

```
#define RL_REALLOC(  
    ptr,  
    sz)
```

Value:

```
realloc(ptr, sz)
```

6.23.1.45 RL_RECTANGLE_TYPE

```
#define RL_RECTANGLE_TYPE
```

6.23.1.46 RL_VECTOR2_TYPE

```
#define RL_VECTOR2_TYPE
```

6.23.1.47 RL_VECTOR3_TYPE

```
#define RL_VECTOR3_TYPE
```

6.23.1.48 RL_VECTOR4_TYPE

```
#define RL_VECTOR4_TYPE
```

6.23.1.49 RLAPI

```
#define RLAPI
```

6.23.1.50 SHADER_LOC_MAP_DIFFUSE

```
#define SHADER_LOC_MAP_DIFFUSE SHADER_LOC_MAP_ALBEDO
```

6.23.1.51 SHADER_LOC_MAP_SPECULAR

```
#define SHADER_LOC_MAP_SPECULAR SHADER_LOC_MAP_METALNESS
```

6.23.1.52 SKYBLUE

```
#define SKYBLUE CLITERAL(Color){ 102, 191, 255, 255 }
```

6.23.1.53 VIOLET

```
#define VIOLET CLITERAL(Color){ 135, 60, 190, 255 }
```

6.23.1.54 WHITE

```
#define WHITE CLITERAL(Color){ 255, 255, 255, 255 }
```

6.23.1.55 YELLOW

```
#define YELLOW CLITERAL(Color){ 253, 249, 0, 255 }
```

6.23.2 Typedef Documentation

6.23.2.1 AudioCallback

```
typedef void(* AudioCallback) (void *bufferData, unsigned int frames)
```

6.23.2.2 AudioStream

```
typedef struct AudioStream AudioStream
```

6.23.2.3 AutomationEvent

```
typedef struct AutomationEvent AutomationEvent
```

6.23.2.4 AutomationEventList

```
typedef struct AutomationEventList AutomationEventList
```

6.23.2.5 BoneInfo

```
typedef struct BoneInfo BoneInfo
```

6.23.2.6 bool

```
typedef enum bool bool
```

6.23.2.7 BoundingBox

```
typedef struct BoundingBox BoundingBox
```

6.23.2.8 Camera

```
typedef Camera3D Camera
```

6.23.2.9 Camera2D

```
typedef struct Camera2D Camera2D
```

6.23.2.10 Camera3D

```
typedef struct Camera3D Camera3D
```

6.23.2.11 Color

```
typedef struct Color Color
```

6.23.2.12 FilePathList

```
typedef struct FilePathList FilePathList
```

6.23.2.13 Font

```
typedef struct Font Font
```

6.23.2.14 GlyphInfo

```
typedef struct GlyphInfo GlyphInfo
```

6.23.2.15 Image

```
typedef struct Image Image
```

6.23.2.16 LoadFileDataCallback

```
typedef unsigned char *(* LoadFileDataCallback) (const char *fileName, int *dataSize)
```

6.23.2.17 LoadFileTextCallback

```
typedef char *(* LoadFileTextCallback) (const char *fileName)
```

6.23.2.18 Material

```
typedef struct Material Material
```

6.23.2.19 MaterialMap

```
typedef struct MaterialMap MaterialMap
```

6.23.2.20 Matrix

```
typedef struct Matrix Matrix
```

6.23.2.21 Mesh

```
typedef struct Mesh Mesh
```

6.23.2.22 Model

```
typedef struct Model Model
```

6.23.2.23 ModelAnimation

```
typedef struct ModelAnimation ModelAnimation
```

6.23.2.24 ModelAnimPose

```
typedef Transform* ModelAnimPose
```

6.23.2.25 ModelSkeleton

```
typedef struct ModelSkeleton ModelSkeleton
```

6.23.2.26 Music

```
typedef struct Music Music
```

6.23.2.27 NPatchInfo

```
typedef struct NPatchInfo NPatchInfo
```

6.23.2.28 Quaternion

```
typedef Vector4 Quaternion
```

6.23.2.29 rAudioBuffer

```
typedef struct rAudioBuffer rAudioBuffer
```

6.23.2.30 rAudioProcessor

```
typedef struct rAudioProcessor rAudioProcessor
```

6.23.2.31 Ray

```
typedef struct Ray Ray
```

6.23.2.32 RayCollision

```
typedef struct RayCollision RayCollision
```

6.23.2.33 Rectangle

```
typedef struct Rectangle Rectangle
```

6.23.2.34 RenderTexture

```
typedef struct RenderTexture RenderTexture
```

6.23.2.35 RenderTexture2D

```
typedef RenderTexture RenderTexture2D
```

6.23.2.36 SaveFileDataCallback

```
typedef bool(* SaveFileDataCallback) (const char *fileName, void *data, int dataSize)
```

6.23.2.37 SaveFileTextCallback

```
typedef bool(* SaveFileTextCallback) (const char *fileName, const char *text)
```

6.23.2.38 Shader

```
typedef struct Shader Shader
```

6.23.2.39 Sound

```
typedef struct Sound Sound
```

6.23.2.40 Texture

```
typedef struct Texture Texture
```

6.23.2.41 Texture2D

```
typedef Texture Texture2D
```


6.23.2.42 TextureCubemap

```
typedef Texture TextureCubemap
```

6.23.2.43 TraceLogCallback

```
typedef void(* TraceLogCallback) (int logLevel, const char *text, va_list args)
```

6.23.2.44 Transform

```
typedef struct Transform Transform
```

6.23.2.45 Vector2

```
typedef struct Vector2 Vector2
```

6.23.2.46 Vector3

```
typedef struct Vector3 Vector3
```

6.23.2.47 Vector4

```
typedef struct Vector4 Vector4
```

6.23.2.48 VrDeviceInfo

```
typedef struct VrDeviceInfo VrDeviceInfo
```

6.23.2.49 VrStereoConfig

```
typedef struct VrStereoConfig VrStereoConfig
```

6.23.2.50 Wave

```
typedef struct Wave Wave
```

6.23.3 Enumeration Type Documentation

6.23.3.1 BlendMode

```
enum BlendMode
```

Enumerator

	BLEND_ALPHA
	BLEND_ADDITIVE
	BLEND_MULTIPLIED
	BLEND_ADD_COLORS
	BLEND_SUBTRACT_COLORS
	BLEND_ALPHA_PREMULTIPLY
	BLEND_CUSTOM
	BLEND_CUSTOM_SEPARATE

6.23.3.2 bool

enum [bool](#)

Enumerator

	false
	true

6.23.3.3 CameraMode

enum [CameraMode](#)

Enumerator

	CAMERA_CUSTOM
	CAMERA_FREE
	CAMERA_ORBITAL
	CAMERA_FIRST_PERSON
	CAMERA_THIRD_PERSON

6.23.3.4 CameraProjection

enum [CameraProjection](#)

Enumerator

	CAMERA_PERSPECTIVE
	CAMERA_ORTHOGRAPHIC

6.23.3.5 ConfigFlags

enum [ConfigFlags](#)

Enumerator

	FLAG_VSYNC_HINT
	FLAG_FULLSCREEN_MODE
	FLAG_WINDOW_RESIZABLE
	FLAG_WINDOW_UNDECORATED
	FLAG_WINDOW_HIDDEN
	FLAG_WINDOW_MINIMIZED
	FLAG_WINDOW_MAXIMIZED
	FLAG_WINDOW_UNFOCUSED
	FLAG_WINDOW_TOPMOST
	FLAG_WINDOW_ALWAYS_RUN
	FLAG_WINDOW_TRANSPARENT
	FLAG_WINDOW_HIGHDPI
	FLAG_WINDOW_MOUSE_PASSTHROUGH
	FLAG_BORDERLESS_WINDOWED_MODE
	FLAG_MSAA_4X_HINT
	FLAG_INTERLACED_HINT

6.23.3.6 CubemapLayout

enum [CubemapLayout](#)

Enumerator

	CUBEMAP_LAYOUT_AUTO_DETECT
	CUBEMAP_LAYOUT_LINE_VERTICAL
	CUBEMAP_LAYOUT_LINE_HORIZONTAL
	CUBEMAP_LAYOUT_CROSS_THREE_BY_FOUR
	CUBEMAP_LAYOUT_CROSS_FOUR_BY_THREE

6.23.3.7 FontType

enum [FontType](#)

Enumerator

	FONT_DEFAULT
	FONT_BITMAP
	FONT_SDF

6.23.3.8 GamepadAxis

enum [GamepadAxis](#)

Enumerator

	GAMEPAD_AXIS_LEFT_X
	GAMEPAD_AXIS_LEFT_Y
	GAMEPAD_AXIS_RIGHT_X
	GAMEPAD_AXIS_RIGHT_Y
	GAMEPAD_AXIS_LEFT_TRIGGER
	GAMEPAD_AXIS_RIGHT_TRIGGER

6.23.3.9 GamepadButton

enum [GamepadButton](#)

Enumerator

	GAMEPAD_BUTTON_UNKNOWN
	GAMEPAD_BUTTON_LEFT_FACE_UP
	GAMEPAD_BUTTON_LEFT_FACE_RIGHT
	GAMEPAD_BUTTON_LEFT_FACE_DOWN
	GAMEPAD_BUTTON_LEFT_FACE_LEFT
	GAMEPAD_BUTTON_RIGHT_FACE_UP
	GAMEPAD_BUTTON_RIGHT_FACE_RIGHT
	GAMEPAD_BUTTON_RIGHT_FACE_DOWN
	GAMEPAD_BUTTON_RIGHT_FACE_LEFT
	GAMEPAD_BUTTON_LEFT_TRIGGER_1
	GAMEPAD_BUTTON_LEFT_TRIGGER_2
	GAMEPAD_BUTTON_RIGHT_TRIGGER_1
	GAMEPAD_BUTTON_RIGHT_TRIGGER_2
	GAMEPAD_BUTTON_MIDDLE_LEFT
	GAMEPAD_BUTTON_MIDDLE
	GAMEPAD_BUTTON_MIDDLE_RIGHT
	GAMEPAD_BUTTON_LEFT_THUMB
	GAMEPAD_BUTTON_RIGHT_THUMB

6.23.3.10 Gesture

enum [Gesture](#)

Enumerator

	GESTURE_NONE
	GESTURE_TAP
	GESTURE_DOUBLETAP
	GESTURE_HOLD
	GESTURE_DRAG
	GESTURE_SWIPE_RIGHT
	GESTURE_SWIPE_LEFT
	GESTURE_SWIPE_UP
	GESTURE_SWIPE_DOWN
	GESTURE_PINCH_IN
	GESTURE_PINCH_OUT

6.23.3.11 KeyboardKey

enum [KeyboardKey](#)

Enumerator

	KEY_NULL
	KEY_APOSTROPHE
	KEY_COMMA

Enumerator

	KEY_MINUS
	KEY_PERIOD
	KEY_SLASH
	KEY_ZERO
	KEY_ONE
	KEY_TWO
	KEY_THREE
	KEY_FOUR
	KEY_FIVE
	KEY_SIX
	KEY_SEVEN
	KEY_EIGHT
	KEY_NINE
	KEY_SEMICOLON
	KEY_EQUAL
	KEY_A
	KEY_B
	KEY_C
	KEY_D
	KEY_E
	KEY_F
	KEY_G
	KEY_H
	KEY_I
	KEY_J
	KEY_K
	KEY_L
	KEY_M
	KEY_N
	KEY_O
	KEY_P
	KEY_Q
	KEY_R
	KEY_S
	KEY_T
	KEY_U
	KEY_V
	KEY_W
	KEY_X
	KEY_Y
	KEY_Z
	KEY_LEFT_BRACKET
	KEY_BACKSLASH
	KEY_RIGHT_BRACKET
	KEY_GRAVE
	KEY_SPACE
	KEY_ESCAPE
	KEY_ENTER
	KEY_TAB
	KEY_BACKSPACE
	KEY_INSERT
	KEY_DELETE
	KEY_RIGHT
	KEY_LEFT
	KEY_DOWN

Enumerator

	KEY_UP
	KEY_PAGE_UP
	KEY_PAGE_DOWN
	KEY_HOME
	KEY_END
	KEY_CAPS_LOCK
	KEY_SCROLL_LOCK
	KEY_NUM_LOCK
	KEY_PRINT_SCREEN
	KEY_PAUSE
	KEY_F1
	KEY_F2
	KEY_F3
	KEY_F4
	KEY_F5
	KEY_F6
	KEY_F7
	KEY_F8
	KEY_F9
	KEY_F10
	KEY_F11
	KEY_F12
	KEY_LEFT_SHIFT
	KEY_LEFT_CONTROL
	KEY_LEFT_ALT
	KEY_LEFT_SUPER
	KEY_RIGHT_SHIFT
	KEY_RIGHT_CONTROL
	KEY_RIGHT_ALT
	KEY_RIGHT_SUPER
	KEY_KB_MENU
	KEY_KP_0
	KEY_KP_1
	KEY_KP_2
	KEY_KP_3
	KEY_KP_4
	KEY_KP_5
	KEY_KP_6
	KEY_KP_7
	KEY_KP_8
	KEY_KP_9
	KEY_KP_DECIMAL
	KEY_KP_DIVIDE
	KEY_KP_MULTIPLY
	KEY_KP_SUBTRACT
	KEY_KP_ADD
	KEY_KP_ENTER
	KEY_KP_EQUAL
	KEY_BACK
	KEY_MENU
	KEY_VOLUME_UP
	KEY_VOLUME_DOWN

6.23.3.12 MaterialMapIndex

enum [MaterialMapIndex](#)

Enumerator

	MATERIAL_MAP_ALBEDO
	MATERIAL_MAP_METALNESS
	MATERIAL_MAP_NORMAL
	MATERIAL_MAP_ROUGHNESS
	MATERIAL_MAP_OCCLUSION
	MATERIAL_MAP_EMISSION
	MATERIAL_MAP_HEIGHT
	MATERIAL_MAP_CUBEMAP
	MATERIAL_MAP_IRRADIANCE
	MATERIAL_MAP_PREFILTER
	MATERIAL_MAP_BRDF

6.23.3.13 MouseButton

enum [MouseButton](#)

Enumerator

	MOUSE_BUTTON_LEFT
	MOUSE_BUTTON_RIGHT
	MOUSE_BUTTON_MIDDLE
	MOUSE_BUTTON_SIDE
	MOUSE_BUTTON_EXTRA
	MOUSE_BUTTON_FORWARD
	MOUSE_BUTTON_BACK

6.23.3.14 MouseCursor

enum [MouseCursor](#)

Enumerator

	MOUSE_CURSOR_DEFAULT
	MOUSE_CURSOR_ARROW
	MOUSE_CURSOR_IBEAM
	MOUSE_CURSOR_CROSSHAIR
	MOUSE_CURSOR_POINTING_HAND
	MOUSE_CURSOR_RESIZE_EW
	MOUSE_CURSOR_RESIZE_NS
	MOUSE_CURSOR_RESIZE_NWSE
	MOUSE_CURSOR_RESIZE_NESW
	MOUSE_CURSOR_RESIZE_ALL
	MOUSE_CURSOR_NOT_ALLOWED

6.23.3.15 NPatchLayout

enum [NPatchLayout](#)

Enumerator

	NPATCH_NINE_PATCH
	NPATCH_THREE_PATCH_VERTICAL
	NPATCH_THREE_PATCH_HORIZONTAL

6.23.3.16 PixelFormatenum `PixelFormat`

Enumerator

	PIXELFORMAT_UNCOMPRESSED_GRAYSCALE
	PIXELFORMAT_UNCOMPRESSED_GRAY_ALPHA
	PIXELFORMAT_UNCOMPRESSED_R5G6B5
	PIXELFORMAT_UNCOMPRESSED_R8G8B8
	PIXELFORMAT_UNCOMPRESSED_R5G5B5A1
	PIXELFORMAT_UNCOMPRESSED_R4G4B4A4
	PIXELFORMAT_UNCOMPRESSED_R8G8B8A8
	PIXELFORMAT_UNCOMPRESSED_R32
	PIXELFORMAT_UNCOMPRESSED_R32G32B32
	PIXELFORMAT_UNCOMPRESSED_R32G32B32A32
	PIXELFORMAT_UNCOMPRESSED_R16
	PIXELFORMAT_UNCOMPRESSED_R16G16B16
	PIXELFORMAT_UNCOMPRESSED_R16G16B16A16
	PIXELFORMAT_COMPRESSED_DXT1_RGB
	PIXELFORMAT_COMPRESSED_DXT1_RGBA
	PIXELFORMAT_COMPRESSED_DXT3_RGBA
	PIXELFORMAT_COMPRESSED_DXT5_RGBA
	PIXELFORMAT_COMPRESSED_ETC1_RGB
	PIXELFORMAT_COMPRESSED_ETC2_RGB
	PIXELFORMAT_COMPRESSED_ETC2_EAC_RGBA
	PIXELFORMAT_COMPRESSED_PVRT_RGB
	PIXELFORMAT_COMPRESSED_PVRT_RGBA
	PIXELFORMAT_COMPRESSED_ASTC_4x4_RGBA
	PIXELFORMAT_COMPRESSED_ASTC_8x8_RGBA

6.23.3.17 ShaderAttributeDataTypeenum `ShaderAttributeDataType`

Enumerator

	SHADER_ATTRIB_FLOAT
	SHADER_ATTRIB_VEC2
	SHADER_ATTRIB_VEC3
	SHADER_ATTRIB_VEC4

6.23.3.18 ShaderLocationIndexenum `ShaderLocationIndex`

Enumerator

	SHADER_LOC_VERTEX_POSITION
	SHADER_LOC_VERTEX_TEXCOORD01
	SHADER_LOC_VERTEX_TEXCOORD02
	SHADER_LOC_VERTEX_NORMAL
	SHADER_LOC_VERTEX_TANGENT
	SHADER_LOC_VERTEX_COLOR
	SHADER_LOC_MATRIX_MVP
	SHADER_LOC_MATRIX_VIEW
	SHADER_LOC_MATRIX_PROJECTION
	SHADER_LOC_MATRIX_MODEL
	SHADER_LOC_MATRIX_NORMAL
	SHADER_LOC_VECTOR_VIEW
	SHADER_LOC_COLOR_DIFFUSE
	SHADER_LOC_COLOR_SPECULAR
	SHADER_LOC_COLOR_AMBIENT
	SHADER_LOC_MAP_ALBEDO
	SHADER_LOC_MAP_METALNESS
	SHADER_LOC_MAP_NORMAL
	SHADER_LOC_MAP_ROUGHNESS
	SHADER_LOC_MAP_OCCLUSION
	SHADER_LOC_MAP_EMISSION
	SHADER_LOC_MAP_HEIGHT
	SHADER_LOC_MAP_CUBEMAP
	SHADER_LOC_MAP_IRRADIANCE
	SHADER_LOC_MAP_PREFILTER
	SHADER_LOC_MAP_BRDF
	SHADER_LOC_VERTEX_BONEIDS
	SHADER_LOC_VERTEX_BONEWEIGHTS
	SHADER_LOC_MATRIX_BONETRANSFORMS
	SHADER_LOC_VERTEX_INSTANCETRANSFORM

6.23.3.19 ShaderUniformDataType

enum [ShaderUniformDataType](#)

Enumerator

	SHADER_UNIFORM_FLOAT
	SHADER_UNIFORM_VEC2
	SHADER_UNIFORM_VEC3
	SHADER_UNIFORM_VEC4
	SHADER_UNIFORM_INT
	SHADER_UNIFORM_IVEC2
	SHADER_UNIFORM_IVEC3
	SHADER_UNIFORM_IVEC4
	SHADER_UNIFORM_UINT
	SHADER_UNIFORM_UIVEC2
	SHADER_UNIFORM_UIVEC3
	SHADER_UNIFORM_UIVEC4
	SHADER_UNIFORM_SAMPLER2D

6.23.3.20 TextureFilter

enum [TextureFilter](#)

Enumerator

	TEXTURE_FILTER_POINT
	TEXTURE_FILTER_BILINEAR
	TEXTURE_FILTER_TRILINEAR
	TEXTURE_FILTER_ANISOTROPIC_4X
	TEXTURE_FILTER_ANISOTROPIC_8X
	TEXTURE_FILTER_ANISOTROPIC_16X

6.23.3.21 TextureWrap

enum [TextureWrap](#)

Enumerator

	TEXTURE_WRAP_REPEAT
	TEXTURE_WRAP_CLAMP
	TEXTURE_WRAP_MIRROR_REPEAT
	TEXTURE_WRAP_MIRROR_CLAMP

6.23.3.22 TraceLogLevel

enum [TraceLogLevel](#)

Enumerator

	LOG_ALL
	LOG_TRACE
	LOG_DEBUG
	LOG_INFO
	LOG_WARNING
	LOG_ERROR
	LOG_FATAL
	LOG_NONE

6.23.4 Function Documentation

6.23.4.1 AttachAudioMixedProcessor()

```
RLAPI void AttachAudioMixedProcessor (  
    AudioCallback processor)
```

6.23.4.2 AttachAudioStreamProcessor()

```
RLAPI void AttachAudioStreamProcessor (
    AudioStream stream,
    AudioCallback processor)
```

6.23.4.3 BeginBlendMode()

```
RLAPI void BeginBlendMode (
    int mode)
```

6.23.4.4 BeginDrawing()

```
RLAPI void BeginDrawing (
    void )
```

6.23.4.5 BeginMode2D()

```
RLAPI void BeginMode2D (
    Camera2D camera)
```

6.23.4.6 BeginMode3D()

```
RLAPI void BeginMode3D (
    Camera3D camera)
```

6.23.4.7 BeginScissorMode()

```
RLAPI void BeginScissorMode (
    int x,
    int y,
    int width,
    int height)
```

6.23.4.8 BeginShaderMode()

```
RLAPI void BeginShaderMode (
    Shader shader)
```

6.23.4.9 BeginTextureMode()

```
RLAPI void BeginTextureMode (
    RenderTexture2D target)
```

6.23.4.10 BeginVrStereoMode()

```
RLAPI void BeginVrStereoMode (  
    VrStereoConfig config)
```

6.23.4.11 ChangeDirectory()

```
RLAPI bool ChangeDirectory (  
    const char * dirPath)
```

6.23.4.12 CheckCollisionBoxes()

```
RLAPI bool CheckCollisionBoxes (  
    BoundingBox box1,  
    BoundingBox box2)
```

6.23.4.13 CheckCollisionBoxSphere()

```
RLAPI bool CheckCollisionBoxSphere (  
    BoundingBox box,  
    Vector3 center,  
    float radius)
```

6.23.4.14 CheckCollisionCircleLine()

```
RLAPI bool CheckCollisionCircleLine (  
    Vector2 center,  
    float radius,  
    Vector2 p1,  
    Vector2 p2)
```

6.23.4.15 CheckCollisionCircleRec()

```
RLAPI bool CheckCollisionCircleRec (  
    Vector2 center,  
    float radius,  
    Rectangle rec)
```

6.23.4.16 CheckCollisionCircles()

```
RLAPI bool CheckCollisionCircles (  
    Vector2 center1,  
    float radius1,  
    Vector2 center2,  
    float radius2)
```

6.23.4.17 CheckCollisionLines()

```
RLAPI bool CheckCollisionLines (
    Vector2 startPos1,
    Vector2 endPos1,
    Vector2 startPos2,
    Vector2 endPos2,
    Vector2 * collisionPoint)
```

6.23.4.18 CheckCollisionPointCircle()

```
RLAPI bool CheckCollisionPointCircle (
    Vector2 point,
    Vector2 center,
    float radius)
```

6.23.4.19 CheckCollisionPointLine()

```
RLAPI bool CheckCollisionPointLine (
    Vector2 point,
    Vector2 p1,
    Vector2 p2,
    int threshold)
```

6.23.4.20 CheckCollisionPointPoly()

```
RLAPI bool CheckCollisionPointPoly (
    Vector2 point,
    const Vector2 * points,
    int pointCount)
```

6.23.4.21 CheckCollisionPointRec()

```
RLAPI bool CheckCollisionPointRec (
    Vector2 point,
    Rectangle rec)
```

6.23.4.22 CheckCollisionPointTriangle()

```
RLAPI bool CheckCollisionPointTriangle (
    Vector2 point,
    Vector2 p1,
    Vector2 p2,
    Vector2 p3)
```

6.23.4.23 CheckCollisionRecs()

```
RLAPI bool CheckCollisionRecs (
    Rectangle rec1,
    Rectangle rec2)
```

6.23.4.24 CheckCollisionSpheres()

```
RLAPI bool CheckCollisionSpheres (
    Vector3 center1,
    float radius1,
    Vector3 center2,
    float radius2)
```

6.23.4.25 ClearBackground()

```
RLAPI void ClearBackground (
    Color color)
```

6.23.4.26 ClearWindowState()

```
RLAPI void ClearWindowState (
    unsigned int flags)
```

6.23.4.27 CloseAudioDevice()

```
RLAPI void CloseAudioDevice (
    void )
```

6.23.4.28 CloseWindow()

```
RLAPI void CloseWindow (
    void )
```

6.23.4.29 CodepointToUTF8()

```
RLAPI const char * CodepointToUTF8 (
    int codepoint,
    int * utf8Size)
```

6.23.4.30 ColorAlpha()

```
RLAPI Color ColorAlpha (
    Color color,
    float alpha)
```

6.23.4.31 ColorAlphaBlend()

```
RLAPI Color ColorAlphaBlend (
    Color dst,
    Color src,
    Color tint)
```

6.23.4.32 ColorBrightness()

```
RLAPI Color ColorBrightness (
    Color color,
    float factor)
```

6.23.4.33 ColorContrast()

```
RLAPI Color ColorContrast (
    Color color,
    float contrast)
```

6.23.4.34 ColorFromHSV()

```
RLAPI Color ColorFromHSV (
    float hue,
    float saturation,
    float value)
```

6.23.4.35 ColorFromNormalized()

```
RLAPI Color ColorFromNormalized (
    Vector4 normalized)
```

6.23.4.36 ColorIsEqual()

```
RLAPI bool ColorIsEqual (
    Color col1,
    Color col2)
```

6.23.4.37 ColorLerp()

```
RLAPI Color ColorLerp (
    Color color1,
    Color color2,
    float factor)
```

6.23.4.38 ColorNormalize()

```
RLAPI Vector4 ColorNormalize (  
    Color color)
```

6.23.4.39 ColorTint()

```
RLAPI Color ColorTint (  
    Color color,  
    Color tint)
```

6.23.4.40 ColorToHSV()

```
RLAPI Vector3 ColorToHSV (  
    Color color)
```

6.23.4.41 ColorToInt()

```
RLAPI int ColorToInt (  
    Color color)
```

6.23.4.42 CompressData()

```
RLAPI unsigned char * CompressData (  
    const unsigned char * data,  
    int dataSize,  
    int * compDataSize)
```

6.23.4.43 ComputeCRC32()

```
RLAPI unsigned int ComputeCRC32 (  
    unsigned char * data,  
    int dataSize)
```

6.23.4.44 ComputeMD5()

```
RLAPI unsigned int * ComputeMD5 (  
    unsigned char * data,  
    int dataSize)
```

6.23.4.45 ComputeSHA1()

```
RLAPI unsigned int * ComputeSHA1 (  
    unsigned char * data,  
    int dataSize)
```


6.23.4.46 ComputeSHA256()

```
RLAPI unsigned int * ComputeSHA256 (
    unsigned char * data,
    int dataSize)
```

6.23.4.47 DecodeDataBase64()

```
RLAPI unsigned char * DecodeDataBase64 (
    const char * text,
    int * outputSize)
```

6.23.4.48 DecompressData()

```
RLAPI unsigned char * DecompressData (
    const unsigned char * compData,
    int compDataSize,
    int * dataSize)
```

6.23.4.49 DetachAudioMixedProcessor()

```
RLAPI void DetachAudioMixedProcessor (
    AudioCallback processor)
```

6.23.4.50 DetachAudioStreamProcessor()

```
RLAPI void DetachAudioStreamProcessor (
    AudioStream stream,
    AudioCallback processor)
```

6.23.4.51 DirectoryExists()

```
RLAPI bool DirectoryExists (
    const char * dirPath)
```

6.23.4.52 DisableCursor()

```
RLAPI void DisableCursor (
    void )
```

6.23.4.53 DisableEventWaiting()

```
RLAPI void DisableEventWaiting (
    void )
```

6.23.4.54 DrawBillboard()

```
RLAPI void DrawBillboard (
    Camera camera,
    Texture2D texture,
    Vector3 position,
    float scale,
    Color tint)
```

6.23.4.55 DrawBillboardPro()

```
RLAPI void DrawBillboardPro (
    Camera camera,
    Texture2D texture,
    Rectangle source,
    Vector3 position,
    Vector3 up,
    Vector2 size,
    Vector2 origin,
    float rotation,
    Color tint)
```

6.23.4.56 DrawBillboardRec()

```
RLAPI void DrawBillboardRec (
    Camera camera,
    Texture2D texture,
    Rectangle source,
    Vector3 position,
    Vector2 size,
    Color tint)
```

6.23.4.57 DrawBoundingBox()

```
RLAPI void DrawBoundingBox (
    BoundingBox box,
    Color color)
```

6.23.4.58 DrawCapsule()

```
RLAPI void DrawCapsule (
    Vector3 startPos,
    Vector3 endPos,
    float radius,
    int slices,
    int rings,
    Color color)
```

6.23.4.59 DrawCapsuleWires()

```
RLAPI void DrawCapsuleWires (
    Vector3 startPos,
    Vector3 endPos,
    float radius,
    int slices,
    int rings,
    Color color)
```

6.23.4.60 DrawCircle()

```
RLAPI void DrawCircle (
    int centerX,
    int centerY,
    float radius,
    Color color)
```

6.23.4.61 DrawCircle3D()

```
RLAPI void DrawCircle3D (
    Vector3 center,
    float radius,
    Vector3 rotationAxis,
    float rotationAngle,
    Color color)
```

6.23.4.62 DrawCircleGradient()

```
RLAPI void DrawCircleGradient (
    Vector2 center,
    float radius,
    Color inner,
    Color outer)
```

6.23.4.63 DrawCircleLines()

```
RLAPI void DrawCircleLines (
    int centerX,
    int centerY,
    float radius,
    Color color)
```

6.23.4.64 DrawCircleLinesV()

```
RLAPI void DrawCircleLinesV (
    Vector2 center,
    float radius,
    Color color)
```

6.23.4.65 DrawCircleSector()

```
RLAPI void DrawCircleSector (
    Vector2 center,
    float radius,
    float startAngle,
    float endAngle,
    int segments,
    Color color)
```

6.23.4.66 DrawCircleSectorLines()

```
RLAPI void DrawCircleSectorLines (
    Vector2 center,
    float radius,
    float startAngle,
    float endAngle,
    int segments,
    Color color)
```

6.23.4.67 DrawCircleV()

```
RLAPI void DrawCircleV (
    Vector2 center,
    float radius,
    Color color)
```

6.23.4.68 DrawCube()

```
RLAPI void DrawCube (
    Vector3 position,
    float width,
    float height,
    float length,
    Color color)
```

6.23.4.69 DrawCubeV()

```
RLAPI void DrawCubeV (
    Vector3 position,
    Vector3 size,
    Color color)
```

6.23.4.70 DrawCubeWires()

```
RLAPI void DrawCubeWires (
    Vector3 position,
    float width,
    float height,
    float length,
    Color color)
```

6.23.4.71 DrawCubeWiresV()

```
RLAPI void DrawCubeWiresV (
    Vector3 position,
    Vector3 size,
    Color color)
```

6.23.4.72 DrawCylinder()

```
RLAPI void DrawCylinder (
    Vector3 position,
    float radiusTop,
    float radiusBottom,
    float height,
    int slices,
    Color color)
```

6.23.4.73 DrawCylinderEx()

```
RLAPI void DrawCylinderEx (
    Vector3 startPos,
    Vector3 endPos,
    float startRadius,
    float endRadius,
    int sides,
    Color color)
```

6.23.4.74 DrawCylinderWires()

```
RLAPI void DrawCylinderWires (
    Vector3 position,
    float radiusTop,
    float radiusBottom,
    float height,
    int slices,
    Color color)
```

6.23.4.75 DrawCylinderWiresEx()

```
RLAPI void DrawCylinderWiresEx (
    Vector3 startPos,
    Vector3 endPos,
    float startRadius,
    float endRadius,
    int sides,
    Color color)
```

6.23.4.76 DrawEllipse()

```
RLAPI void DrawEllipse (
    int centerX,
    int centerY,
    float radiusH,
    float radiusV,
    Color color)
```

6.23.4.77 DrawEllipseLines()

```
RLAPI void DrawEllipseLines (
    int centerX,
    int centerY,
    float radiusH,
    float radiusV,
    Color color)
```

6.23.4.78 DrawEllipseLinesV()

```
RLAPI void DrawEllipseLinesV (
    Vector2 center,
    float radiusH,
    float radiusV,
    Color color)
```

6.23.4.79 DrawEllipseV()

```
RLAPI void DrawEllipseV (
    Vector2 center,
    float radiusH,
    float radiusV,
    Color color)
```

6.23.4.80 DrawFPS()

```
RLAPI void DrawFPS (
    int posX,
    int posY)
```

6.23.4.81 DrawGrid()

```
RLAPI void DrawGrid (
    int slices,
    float spacing)
```

6.23.4.82 DrawLine()

```
RLAPI void DrawLine (
    int startPosX,
    int startPosY,
    int endPosX,
    int endPosY,
    Color color)
```

6.23.4.83 DrawLine3D()

```
RLAPI void DrawLine3D (
    Vector3 startPos,
    Vector3 endPos,
    Color color)
```

6.23.4.84 DrawLineBezier()

```
RLAPI void DrawLineBezier (
    Vector2 startPos,
    Vector2 endPos,
    float thick,
    Color color)
```

6.23.4.85 DrawLineDashed()

```
RLAPI void DrawLineDashed (
    Vector2 startPos,
    Vector2 endPos,
    int dashSize,
    int spaceSize,
    Color color)
```

6.23.4.86 DrawLineEx()

```
RLAPI void DrawLineEx (
    Vector2 startPos,
    Vector2 endPos,
    float thick,
    Color color)
```

6.23.4.87 DrawLineStrip()

```
RLAPI void DrawLineStrip (
    const Vector2 * points,
    int pointCount,
    Color color)
```

6.23.4.88 DrawLineV()

```
RLAPI void DrawLineV (  
    Vector2 startPos,  
    Vector2 endPos,  
    Color color)
```

6.23.4.89 DrawMesh()

```
RLAPI void DrawMesh (  
    Mesh mesh,  
    Material material,  
    Matrix transform)
```

6.23.4.90 DrawMeshInstanced()

```
RLAPI void DrawMeshInstanced (  
    Mesh mesh,  
    Material material,  
    const Matrix * transforms,  
    int instances)
```

6.23.4.91 DrawModel()

```
RLAPI void DrawModel (  
    Model model,  
    Vector3 position,  
    float scale,  
    Color tint)
```

6.23.4.92 DrawModelEx()

```
RLAPI void DrawModelEx (  
    Model model,  
    Vector3 position,  
    Vector3 rotationAxis,  
    float rotationAngle,  
    Vector3 scale,  
    Color tint)
```

6.23.4.93 DrawModelWires()

```
RLAPI void DrawModelWires (  
    Model model,  
    Vector3 position,  
    float scale,  
    Color tint)
```


6.23.4.94 DrawModelWiresEx()

```
RLAPI void DrawModelWiresEx (
    Model model,
    Vector3 position,
    Vector3 rotationAxis,
    float rotationAngle,
    Vector3 scale,
    Color tint)
```

6.23.4.95 DrawPixel()

```
RLAPI void DrawPixel (
    int posX,
    int posY,
    Color color)
```

6.23.4.96 DrawPixelV()

```
RLAPI void DrawPixelV (
    Vector2 position,
    Color color)
```

6.23.4.97 DrawPlane()

```
RLAPI void DrawPlane (
    Vector3 centerPos,
    Vector2 size,
    Color color)
```

6.23.4.98 DrawPoint3D()

```
RLAPI void DrawPoint3D (
    Vector3 position,
    Color color)
```

6.23.4.99 DrawPoly()

```
RLAPI void DrawPoly (
    Vector2 center,
    int sides,
    float radius,
    float rotation,
    Color color)
```

6.23.4.100 DrawPolyLines()

```
RLAPI void DrawPolyLines (
    Vector2 center,
    int sides,
    float radius,
    float rotation,
    Color color)
```

6.23.4.101 DrawPolyLinesEx()

```
RLAPI void DrawPolyLinesEx (
    Vector2 center,
    int sides,
    float radius,
    float rotation,
    float lineThick,
    Color color)
```

6.23.4.102 DrawRay()

```
RLAPI void DrawRay (
    Ray ray,
    Color color)
```

6.23.4.103 DrawRectangle()

```
RLAPI void DrawRectangle (
    int posX,
    int posY,
    int width,
    int height,
    Color color)
```

6.23.4.104 DrawRectangleGradientEx()

```
RLAPI void DrawRectangleGradientEx (
    Rectangle rec,
    Color topLeft,
    Color bottomLeft,
    Color bottomRight,
    Color topRight)
```

6.23.4.105 DrawRectangleGradientH()

```
RLAPI void DrawRectangleGradientH (
    int posX,
    int posY,
    int width,
    int height,
    Color left,
    Color right)
```

6.23.4.106 DrawRectangleGradientV()

```
RLAPI void DrawRectangleGradientV (
    int posX,
    int posY,
    int width,
    int height,
    Color top,
    Color bottom)
```

6.23.4.107 DrawRectangleLines()

```
RLAPI void DrawRectangleLines (
    int posX,
    int posY,
    int width,
    int height,
    Color color)
```

6.23.4.108 DrawRectangleLinesEx()

```
RLAPI void DrawRectangleLinesEx (
    Rectangle rec,
    float lineThick,
    Color color)
```

6.23.4.109 DrawRectanglePro()

```
RLAPI void DrawRectanglePro (
    Rectangle rec,
    Vector2 origin,
    float rotation,
    Color color)
```

6.23.4.110 DrawRectangleRec()

```
RLAPI void DrawRectangleRec (
    Rectangle rec,
    Color color)
```

6.23.4.111 DrawRectangleRounded()

```
RLAPI void DrawRectangleRounded (
    Rectangle rec,
    float roundness,
    int segments,
    Color color)
```

6.23.4.112 DrawRectangleRoundedLines()

```
RLAPI void DrawRectangleRoundedLines (
    Rectangle rec,
    float roundness,
    int segments,
    Color color)
```

6.23.4.113 DrawRectangleRoundedLinesEx()

```
RLAPI void DrawRectangleRoundedLinesEx (
    Rectangle rec,
    float roundness,
    int segments,
    float lineThick,
    Color color)
```

6.23.4.114 DrawRectangleV()

```
RLAPI void DrawRectangleV (
    Vector2 position,
    Vector2 size,
    Color color)
```

6.23.4.115 DrawRing()

```
RLAPI void DrawRing (
    Vector2 center,
    float innerRadius,
    float outerRadius,
    float startAngle,
    float endAngle,
    int segments,
    Color color)
```

6.23.4.116 DrawRingLines()

```
RLAPI void DrawRingLines (
    Vector2 center,
    float innerRadius,
    float outerRadius,
    float startAngle,
    float endAngle,
    int segments,
    Color color)
```

6.23.4.117 DrawSphere()

```
RLAPI void DrawSphere (
    Vector3 centerPos,
    float radius,
    Color color)
```

6.23.4.118 DrawSphereEx()

```
RLAPI void DrawSphereEx (
    Vector3 centerPos,
    float radius,
    int rings,
    int slices,
    Color color)
```

6.23.4.119 DrawSphereWires()

```
RLAPI void DrawSphereWires (
    Vector3 centerPos,
    float radius,
    int rings,
    int slices,
    Color color)
```

6.23.4.120 DrawSplineBasis()

```
RLAPI void DrawSplineBasis (
    const Vector2 * points,
    int pointCount,
    float thick,
    Color color)
```

6.23.4.121 DrawSplineBezierCubic()

```
RLAPI void DrawSplineBezierCubic (
    const Vector2 * points,
    int pointCount,
    float thick,
    Color color)
```

6.23.4.122 DrawSplineBezierQuadratic()

```
RLAPI void DrawSplineBezierQuadratic (
    const Vector2 * points,
    int pointCount,
    float thick,
    Color color)
```

6.23.4.123 DrawSplineCatmullRom()

```
RLAPI void DrawSplineCatmullRom (
    const Vector2 * points,
    int pointCount,
    float thick,
    Color color)
```

6.23.4.124 DrawSplineLinear()

```
RLAPI void DrawSplineLinear (
    const Vector2 * points,
    int pointCount,
    float thick,
    Color color)
```

6.23.4.125 DrawSplineSegmentBasis()

```
RLAPI void DrawSplineSegmentBasis (
    Vector2 p1,
    Vector2 p2,
    Vector2 p3,
    Vector2 p4,
    float thick,
    Color color)
```

6.23.4.126 DrawSplineSegmentBezierCubic()

```
RLAPI void DrawSplineSegmentBezierCubic (
    Vector2 p1,
    Vector2 c2,
    Vector2 c3,
    Vector2 p4,
    float thick,
    Color color)
```

6.23.4.127 DrawSplineSegmentBezierQuadratic()

```
RLAPI void DrawSplineSegmentBezierQuadratic (
    Vector2 p1,
    Vector2 c2,
    Vector2 p3,
    float thick,
    Color color)
```

6.23.4.128 DrawSplineSegmentCatmullRom()

```
RLAPI void DrawSplineSegmentCatmullRom (
    Vector2 p1,
    Vector2 p2,
    Vector2 p3,
    Vector2 p4,
    float thick,
    Color color)
```

6.23.4.129 DrawSplineSegmentLinear()

```
RLAPI void DrawSplineSegmentLinear (
    Vector2 p1,
    Vector2 p2,
    float thick,
    Color color)
```

6.23.4.130 DrawText()

```
RLAPI void DrawText (
    const char * text,
    int posX,
    int posY,
    int fontSize,
    Color color)
```

6.23.4.131 DrawTextCodepoint()

```
RLAPI void DrawTextCodepoint (
    Font font,
    int codepoint,
    Vector2 position,
    float fontSize,
    Color tint)
```

6.23.4.132 DrawTextCodepoints()

```
RLAPI void DrawTextCodepoints (
    Font font,
    const int * codepoints,
    int codepointCount,
    Vector2 position,
    float fontSize,
    float spacing,
    Color tint)
```

6.23.4.133 DrawTextEx()

```
RLAPI void DrawTextEx (
    Font font,
    const char * text,
    Vector2 position,
    float fontSize,
    float spacing,
    Color tint)
```

6.23.4.134 DrawTextPro()

```
RLAPI void DrawTextPro (  
    Font font,  
    const char * text,  
    Vector2 position,  
    Vector2 origin,  
    float rotation,  
    float fontSize,  
    float spacing,  
    Color tint)
```

6.23.4.135 DrawTexture()

```
RLAPI void DrawTexture (  
    Texture2D texture,  
    int posX,  
    int posY,  
    Color tint)
```

6.23.4.136 DrawTextureEx()

```
RLAPI void DrawTextureEx (  
    Texture2D texture,  
    Vector2 position,  
    float rotation,  
    float scale,  
    Color tint)
```

6.23.4.137 DrawTextureNPatch()

```
RLAPI void DrawTextureNPatch (  
    Texture2D texture,  
    NPatchInfo nPatchInfo,  
    Rectangle dest,  
    Vector2 origin,  
    float rotation,  
    Color tint)
```

6.23.4.138 DrawTexturePro()

```
RLAPI void DrawTexturePro (  
    Texture2D texture,  
    Rectangle source,  
    Rectangle dest,  
    Vector2 origin,  
    float rotation,  
    Color tint)
```


6.23.4.139 DrawTextureRec()

```
RLAPI void DrawTextureRec (
    Texture2D texture,
    Rectangle source,
    Vector2 position,
    Color tint)
```

6.23.4.140 DrawTextureV()

```
RLAPI void DrawTextureV (
    Texture2D texture,
    Vector2 position,
    Color tint)
```

6.23.4.141 DrawTriangle()

```
RLAPI void DrawTriangle (
    Vector2 v1,
    Vector2 v2,
    Vector2 v3,
    Color color)
```

6.23.4.142 DrawTriangle3D()

```
RLAPI void DrawTriangle3D (
    Vector3 v1,
    Vector3 v2,
    Vector3 v3,
    Color color)
```

6.23.4.143 DrawTriangleFan()

```
RLAPI void DrawTriangleFan (
    const Vector2 * points,
    int pointCount,
    Color color)
```

6.23.4.144 DrawTriangleLines()

```
RLAPI void DrawTriangleLines (
    Vector2 v1,
    Vector2 v2,
    Vector2 v3,
    Color color)
```

6.23.4.145 DrawTriangleStrip()

```
RLAPI void DrawTriangleStrip (  
    const Vector2 * points,  
    int pointCount,  
    Color color)
```

6.23.4.146 DrawTriangleStrip3D()

```
RLAPI void DrawTriangleStrip3D (  
    const Vector3 * points,  
    int pointCount,  
    Color color)
```

6.23.4.147 EnableCursor()

```
RLAPI void EnableCursor (  
    void )
```

6.23.4.148 EnableEventWaiting()

```
RLAPI void EnableEventWaiting (  
    void )
```

6.23.4.149 EncodeDataBase64()

```
RLAPI char * EncodeDataBase64 (  
    const unsigned char * data,  
    int dataSize,  
    int * outputSize)
```

6.23.4.150 EndBlendMode()

```
RLAPI void EndBlendMode (  
    void )
```

6.23.4.151 EndDrawing()

```
RLAPI void EndDrawing (  
    void )
```

6.23.4.152 EndMode2D()

```
RLAPI void EndMode2D (  
    void )
```

6.23.4.153 EndMode3D()

```
RLAPI void EndMode3D (
    void )
```

6.23.4.154 EndScissorMode()

```
RLAPI void EndScissorMode (
    void )
```

6.23.4.155 EndShaderMode()

```
RLAPI void EndShaderMode (
    void )
```

6.23.4.156 EndTextureMode()

```
RLAPI void EndTextureMode (
    void )
```

6.23.4.157 EndVrStereoMode()

```
RLAPI void EndVrStereoMode (
    void )
```

6.23.4.158 ExportAutomationEventList()

```
RLAPI bool ExportAutomationEventList (
    AutomationEventList list,
    const char * fileName)
```

6.23.4.159 ExportDataAsCode()

```
RLAPI bool ExportDataAsCode (
    const unsigned char * data,
    int dataSize,
    const char * fileName)
```

6.23.4.160 ExportFontAsCode()

```
RLAPI bool ExportFontAsCode (
    Font font,
    const char * fileName)
```

6.23.4.161 ExportImage()

```
RLAPI bool ExportImage (  
    Image image,  
    const char * fileName)
```

6.23.4.162 ExportImageAsCode()

```
RLAPI bool ExportImageAsCode (  
    Image image,  
    const char * fileName)
```

6.23.4.163 ExportImageToMemory()

```
RLAPI unsigned char * ExportImageToMemory (  
    Image image,  
    const char * fileType,  
    int * fileSize)
```

6.23.4.164 ExportMesh()

```
RLAPI bool ExportMesh (  
    Mesh mesh,  
    const char * fileName)
```

6.23.4.165 ExportMeshAsCode()

```
RLAPI bool ExportMeshAsCode (  
    Mesh mesh,  
    const char * fileName)
```

6.23.4.166 ExportWave()

```
RLAPI bool ExportWave (  
    Wave wave,  
    const char * fileName)
```

6.23.4.167 ExportWaveAsCode()

```
RLAPI bool ExportWaveAsCode (  
    Wave wave,  
    const char * fileName)
```

6.23.4.168 Fade()

```
RLAPI Color Fade (
    Color color,
    float alpha)
```

6.23.4.169 FileCopy()

```
RLAPI int FileCopy (
    const char * srcPath,
    const char * dstPath)
```

6.23.4.170 FileExists()

```
RLAPI bool FileExists (
    const char * fileName)
```

6.23.4.171 FileMove()

```
RLAPI int FileMove (
    const char * srcPath,
    const char * dstPath)
```

6.23.4.172 FileRemove()

```
RLAPI int FileRemove (
    const char * fileName)
```

6.23.4.173 FileRename()

```
RLAPI int FileRename (
    const char * fileName,
    const char * fileRename)
```

6.23.4.174 FileTextFindIndex()

```
RLAPI int FileTextFindIndex (
    const char * fileName,
    const char * search)
```

6.23.4.175 FileTextReplace()

```
RLAPI int FileTextReplace (
    const char * fileName,
    const char * search,
    const char * replacement)
```

6.23.4.176 GenImageCellular()

```
RLAPI Image GenImageCellular (
    int width,
    int height,
    int tileSize)
```

6.23.4.177 GenImageChecked()

```
RLAPI Image GenImageChecked (
    int width,
    int height,
    int checksX,
    int checksY,
    Color col1,
    Color col2)
```

6.23.4.178 GenImageColor()

```
RLAPI Image GenImageColor (
    int width,
    int height,
    Color color)
```

6.23.4.179 GenImageFontAtlas()

```
RLAPI Image GenImageFontAtlas (
    const GlyphInfo * glyphs,
    Rectangle ** glyphRecs,
    int glyphCount,
    int fontSize,
    int padding,
    int packMethod)
```

6.23.4.180 GenImageGradientLinear()

```
RLAPI Image GenImageGradientLinear (
    int width,
    int height,
    int direction,
    Color start,
    Color end)
```

6.23.4.181 GenImageGradientRadial()

```
RLAPI Image GenImageGradientRadial (
    int width,
    int height,
    float density,
    Color inner,
    Color outer)
```

6.23.4.182 GenImageGradientSquare()

```
RLAPI Image GenImageGradientSquare (
    int width,
    int height,
    float density,
    Color inner,
    Color outer)
```

6.23.4.183 GenImagePerlinNoise()

```
RLAPI Image GenImagePerlinNoise (
    int width,
    int height,
    int offsetX,
    int offsetY,
    float scale)
```

6.23.4.184 GenImageText()

```
RLAPI Image GenImageText (
    int width,
    int height,
    const char * text)
```

6.23.4.185 GenImageWhiteNoise()

```
RLAPI Image GenImageWhiteNoise (
    int width,
    int height,
    float factor)
```

6.23.4.186 GenMeshCone()

```
RLAPI Mesh GenMeshCone (
    float radius,
    float height,
    int slices)
```

6.23.4.187 GenMeshCube()

```
RLAPI Mesh GenMeshCube (
    float width,
    float height,
    float length)
```

6.23.4.188 GenMeshCubicmap()

```
RLAPI Mesh GenMeshCubicmap (
    Image cubicmap,
    Vector3 cubeSize)
```

6.23.4.189 GenMeshCylinder()

```
RLAPI Mesh GenMeshCylinder (
    float radius,
    float height,
    int slices)
```

6.23.4.190 GenMeshHeightmap()

```
RLAPI Mesh GenMeshHeightmap (
    Image heightmap,
    Vector3 size)
```

6.23.4.191 GenMeshHemiSphere()

```
RLAPI Mesh GenMeshHemiSphere (
    float radius,
    int rings,
    int slices)
```

6.23.4.192 GenMeshKnot()

```
RLAPI Mesh GenMeshKnot (
    float radius,
    float size,
    int radSeg,
    int sides)
```

6.23.4.193 GenMeshPlane()

```
RLAPI Mesh GenMeshPlane (
    float width,
    float length,
    int resX,
    int resZ)
```

6.23.4.194 GenMeshPoly()

```
RLAPI Mesh GenMeshPoly (
    int sides,
    float radius)
```


6.23.4.195 GenMeshSphere()

```
RLAPI Mesh GenMeshSphere (
    float radius,
    int rings,
    int slices)
```

6.23.4.196 GenMeshTangents()

```
RLAPI void GenMeshTangents (
    Mesh * mesh)
```

6.23.4.197 GenMeshTorus()

```
RLAPI Mesh GenMeshTorus (
    float radius,
    float size,
    int radSeg,
    int sides)
```

6.23.4.198 GenTextureMipmaps()

```
RLAPI void GenTextureMipmaps (
    Texture2D * texture)
```

6.23.4.199 GetApplicationDirectory()

```
RLAPI const char * GetApplicationDirectory (
    void )
```

6.23.4.200 GetCameraMatrix()

```
RLAPI Matrix GetCameraMatrix (
    Camera camera)
```

6.23.4.201 GetCameraMatrix2D()

```
RLAPI Matrix GetCameraMatrix2D (
    Camera2D camera)
```

6.23.4.202 GetCharPressed()

```
RLAPI int GetCharPressed (
    void )
```

6.23.4.203 GetClipboardImage()

```
RLAPI Image GetClipboardImage (  
    void )
```

6.23.4.204 GetClipboardText()

```
RLAPI const char * GetClipboardText (  
    void )
```

6.23.4.205 GetCodepoint()

```
RLAPI int GetCodepoint (  
    const char * text,  
    int * codepointSize)
```

6.23.4.206 GetCodepointCount()

```
RLAPI int GetCodepointCount (  
    const char * text)
```

6.23.4.207 GetCodepointNext()

```
RLAPI int GetCodepointNext (  
    const char * text,  
    int * codepointSize)
```

6.23.4.208 GetCodepointPrevious()

```
RLAPI int GetCodepointPrevious (  
    const char * text,  
    int * codepointSize)
```

6.23.4.209 GetCollisionRec()

```
RLAPI Rectangle GetCollisionRec (  
    Rectangle rec1,  
    Rectangle rec2)
```

6.23.4.210 GetColor()

```
RLAPI Color GetColor (  
    unsigned int hexValue)
```

6.23.4.211 GetCurrentMonitor()

```
RLAPI int GetCurrentMonitor (
    void )
```

6.23.4.212 GetDirectoryFileCount()

```
RLAPI unsigned int GetDirectoryFileCount (
    const char * dirPath)
```

6.23.4.213 GetDirectoryFileCountEx()

```
RLAPI unsigned int GetDirectoryFileCountEx (
    const char * basePath,
    const char * filter,
    bool scanSubdirs)
```

6.23.4.214 GetDirectoryPath()

```
RLAPI const char * GetDirectoryPath (
    const char * filePath)
```

6.23.4.215 GetFileExtension()

```
RLAPI const char * GetFileExtension (
    const char * fileName)
```

6.23.4.216 GetFileLength()

```
RLAPI int GetFileLength (
    const char * fileName)
```

6.23.4.217 GetFileModTime()

```
RLAPI long GetFileModTime (
    const char * fileName)
```

6.23.4.218 GetFileName()

```
RLAPI const char * GetFileName (
    const char * filePath)
```

6.23.4.219 GetFileNameWithoutExt()

```
RLAPI const char * GetFileNameWithoutExt (
    const char * filePath)
```

6.23.4.220 GetFontDefault()

```
RLAPI Font GetFontDefault (
    void )
```

6.23.4.221 GetFPS()

```
RLAPI int GetFPS (
    void )
```

6.23.4.222 GetFrameTime()

```
RLAPI float GetFrameTime (
    void )
```

6.23.4.223 GetGamepadAxisCount()

```
RLAPI int GetGamepadAxisCount (
    int gamepad)
```

6.23.4.224 GetGamepadAxisMovement()

```
RLAPI float GetGamepadAxisMovement (
    int gamepad,
    int axis)
```

6.23.4.225 GetGamepadButtonPressed()

```
RLAPI int GetGamepadButtonPressed (
    void )
```

6.23.4.226 GetGamepadName()

```
RLAPI const char * GetGamepadName (
    int gamepad)
```

6.23.4.227 GetGestureDetected()

```
RLAPI int GetGestureDetected (
    void )
```

6.23.4.228 GetGestureDragAngle()

```
RLAPI float GetGestureDragAngle (
    void )
```

6.23.4.229 GetGestureDragVector()

```
RLAPI Vector2 GetGestureDragVector (
    void )
```

6.23.4.230 GetGestureHoldDuration()

```
RLAPI float GetGestureHoldDuration (
    void )
```

6.23.4.231 GetGesturePinchAngle()

```
RLAPI float GetGesturePinchAngle (
    void )
```

6.23.4.232 GetGesturePinchVector()

```
RLAPI Vector2 GetGesturePinchVector (
    void )
```

6.23.4.233 GetGlyphAtlasRec()

```
RLAPI Rectangle GetGlyphAtlasRec (
    Font font,
    int codepoint)
```

6.23.4.234 GetGlyphIndex()

```
RLAPI int GetGlyphIndex (
    Font font,
    int codepoint)
```

6.23.4.235 GetGlyphInfo()

```
RLAPI GlyphInfo GetGlyphInfo (
    Font font,
    int codepoint)
```

6.23.4.236 GetImageAlphaBorder()

```
RLAPI Rectangle GetImageAlphaBorder (
    Image image,
    float threshold)
```

6.23.4.237 GetImageColor()

```
RLAPI Color GetImageColor (
    Image image,
    int x,
    int y)
```

6.23.4.238 GetKeyName()

```
RLAPI const char * GetKeyName (
    int key)
```

6.23.4.239 GetKeyPressed()

```
RLAPI int GetKeyPressed (
    void )
```

6.23.4.240 GetMasterVolume()

```
RLAPI float GetMasterVolume (
    void )
```

6.23.4.241 GetMeshBoundingBox()

```
RLAPI BoundingBox GetMeshBoundingBox (
    Mesh mesh)
```

6.23.4.242 GetModelBoundingBox()

```
RLAPI BoundingBox GetModelBoundingBox (
    Model model)
```

6.23.4.243 GetMonitorCount()

```
RLAPI int GetMonitorCount (
    void )
```

6.23.4.244 GetMonitorHeight()

```
RLAPI int GetMonitorHeight (
    int monitor)
```

6.23.4.245 GetMonitorName()

```
RLAPI const char * GetMonitorName (
    int monitor)
```

6.23.4.246 GetMonitorPhysicalHeight()

```
RLAPI int GetMonitorPhysicalHeight (
    int monitor)
```

6.23.4.247 GetMonitorPhysicalWidth()

```
RLAPI int GetMonitorPhysicalWidth (
    int monitor)
```

6.23.4.248 GetMonitorPosition()

```
RLAPI Vector2 GetMonitorPosition (
    int monitor)
```

6.23.4.249 GetMonitorRefreshRate()

```
RLAPI int GetMonitorRefreshRate (
    int monitor)
```

6.23.4.250 GetMonitorWidth()

```
RLAPI int GetMonitorWidth (
    int monitor)
```

6.23.4.251 GetMouseDelta()

```
RLAPI Vector2 GetMouseDelta (
    void )
```

6.23.4.252 GetMousePosition()

```
RLAPI Vector2 GetMousePosition (
    void )
```

6.23.4.253 GetMouseWheelMove()

```
RLAPI float GetMouseWheelMove (
    void )
```

6.23.4.254 GetMouseWheelMoveV()

```
RLAPI Vector2 GetMouseWheelMoveV (
    void )
```

6.23.4.255 GetMouseX()

```
RLAPI int GetMouseX (
    void )
```

6.23.4.256 GetMouseY()

```
RLAPI int GetMouseY (
    void )
```

6.23.4.257 GetMusicTimeLength()

```
RLAPI float GetMusicTimeLength (
    Music music)
```

6.23.4.258 GetMusicTimePlayed()

```
RLAPI float GetMusicTimePlayed (
    Music music)
```

6.23.4.259 GetPixelColor()

```
RLAPI Color GetPixelColor (
    void * srcPtr,
    int format)
```

6.23.4.260 GetPixelDataSize()

```
RLAPI int GetPixelDataSize (
    int width,
    int height,
    int format)
```


6.23.4.261 GetPrevDirectoryPath()

```
RLAPI const char * GetPrevDirectoryPath (
    const char * dirPath)
```

6.23.4.262 GetRandomValue()

```
RLAPI int GetRandomValue (
    int min,
    int max)
```

6.23.4.263 GetRayCollisionBox()

```
RLAPI RayCollision GetRayCollisionBox (
    Ray ray,
    BoundingBox box)
```

6.23.4.264 GetRayCollisionMesh()

```
RLAPI RayCollision GetRayCollisionMesh (
    Ray ray,
    Mesh mesh,
    Matrix transform)
```

6.23.4.265 GetRayCollisionQuad()

```
RLAPI RayCollision GetRayCollisionQuad (
    Ray ray,
    Vector3 p1,
    Vector3 p2,
    Vector3 p3,
    Vector3 p4)
```

6.23.4.266 GetRayCollisionSphere()

```
RLAPI RayCollision GetRayCollisionSphere (
    Ray ray,
    Vector3 center,
    float radius)
```

6.23.4.267 GetRayCollisionTriangle()

```
RLAPI RayCollision GetRayCollisionTriangle (
    Ray ray,
    Vector3 p1,
    Vector3 p2,
    Vector3 p3)
```

6.23.4.268 GetRenderHeight()

```
RLAPI int GetRenderHeight (
    void )
```

6.23.4.269 GetRenderWidth()

```
RLAPI int GetRenderWidth (
    void )
```

6.23.4.270 GetScreenHeight()

```
RLAPI int GetScreenHeight (
    void )
```

6.23.4.271 GetScreenToWorld2D()

```
RLAPI Vector2 GetScreenToWorld2D (
    Vector2 position,
    Camera2D camera)
```

6.23.4.272 GetScreenToWorldRay()

```
RLAPI Ray GetScreenToWorldRay (
    Vector2 position,
    Camera camera)
```

6.23.4.273 GetScreenToWorldRayEx()

```
RLAPI Ray GetScreenToWorldRayEx (
    Vector2 position,
    Camera camera,
    int width,
    int height)
```

6.23.4.274 GetScreenWidth()

```
RLAPI int GetScreenWidth (
    void )
```

6.23.4.275 GetShaderLocation()

```
RLAPI int GetShaderLocation (
    Shader shader,
    const char * uniformName)
```

6.23.4.276 GetShaderLocationAttrib()

```
RLAPI int GetShaderLocationAttrib (
    Shader shader,
    const char * attribName)
```

6.23.4.277 GetShapesTexture()

```
RLAPI Texture2D GetShapesTexture (
    void )
```

6.23.4.278 GetShapesTextureRectangle()

```
RLAPI Rectangle GetShapesTextureRectangle (
    void )
```

6.23.4.279 GetSplinePointBasis()

```
RLAPI Vector2 GetSplinePointBasis (
    Vector2 p1,
    Vector2 p2,
    Vector2 p3,
    Vector2 p4,
    float t)
```

6.23.4.280 GetSplinePointBezierCubic()

```
RLAPI Vector2 GetSplinePointBezierCubic (
    Vector2 p1,
    Vector2 c2,
    Vector2 c3,
    Vector2 p4,
    float t)
```

6.23.4.281 GetSplinePointBezierQuad()

```
RLAPI Vector2 GetSplinePointBezierQuad (
    Vector2 p1,
    Vector2 c2,
    Vector2 p3,
    float t)
```

6.23.4.282 GetSplinePointCatmullRom()

```
RLAPI Vector2 GetSplinePointCatmullRom (
    Vector2 p1,
    Vector2 p2,
    Vector2 p3,
    Vector2 p4,
    float t)
```

6.23.4.283 GetSplinePointLinear()

```
RLAPI Vector2 GetSplinePointLinear (
    Vector2 startPos,
    Vector2 endPos,
    float t)
```

6.23.4.284 GetTextBetween()

```
RLAPI char * GetTextBetween (
    const char * text,
    const char * begin,
    const char * end)
```

6.23.4.285 GetTime()

```
RLAPI double GetTime (
    void )
```

6.23.4.286 GetTouchPointCount()

```
RLAPI int GetTouchPointCount (
    void )
```

6.23.4.287 GetTouchPointId()

```
RLAPI int GetTouchPointId (
    int index)
```

6.23.4.288 GetTouchPosition()

```
RLAPI Vector2 GetTouchPosition (
    int index)
```

6.23.4.289 GetTouchX()

```
RLAPI int GetTouchX (
    void )
```

6.23.4.290 GetTouchY()

```
RLAPI int GetTouchY (
    void )
```

6.23.4.291 GetWindowHandle()

```
RLAPI void * GetWindowHandle (
    void )
```

6.23.4.292 GetWindowPosition()

```
RLAPI Vector2 GetWindowPosition (
    void )
```

6.23.4.293 GetWindowScaleDPI()

```
RLAPI Vector2 GetWindowScaleDPI (
    void )
```

6.23.4.294 GetWorkingDirectory()

```
RLAPI const char * GetWorkingDirectory (
    void )
```

6.23.4.295 GetWorldToScreen()

```
RLAPI Vector2 GetWorldToScreen (
    Vector3 position,
    Camera camera)
```

6.23.4.296 GetWorldToScreen2D()

```
RLAPI Vector2 GetWorldToScreen2D (
    Vector2 position,
    Camera2D camera)
```

6.23.4.297 GetWorldToScreenEx()

```
RLAPI Vector2 GetWorldToScreenEx (
    Vector3 position,
    Camera camera,
    int width,
    int height)
```

6.23.4.298 HideCursor()

```
RLAPI void HideCursor (
    void )
```

6.23.4.299 ImageAlphaClear()

```
RLAPI void ImageAlphaClear (  
    Image * image,  
    Color color,  
    float threshold)
```

6.23.4.300 ImageAlphaCrop()

```
RLAPI void ImageAlphaCrop (  
    Image * image,  
    float threshold)
```

6.23.4.301 ImageAlphaMask()

```
RLAPI void ImageAlphaMask (  
    Image * image,  
    Image alphaMask)
```

6.23.4.302 ImageAlphaPremultiply()

```
RLAPI void ImageAlphaPremultiply (  
    Image * image)
```

6.23.4.303 ImageBlurGaussian()

```
RLAPI void ImageBlurGaussian (  
    Image * image,  
    int blurSize)
```

6.23.4.304 ImageClearBackground()

```
RLAPI void ImageClearBackground (  
    Image * dst,  
    Color color)
```

6.23.4.305 ImageColorBrightness()

```
RLAPI void ImageColorBrightness (  
    Image * image,  
    int brightness)
```

6.23.4.306 ImageColorContrast()

```
RLAPI void ImageColorContrast (  
    Image * image,  
    float contrast)
```

6.23.4.307 ImageColorGrayscale()

```
RLAPI void ImageColorGrayscale (  
    Image * image)
```

6.23.4.308 ImageColorInvert()

```
RLAPI void ImageColorInvert (  
    Image * image)
```

6.23.4.309 ImageColorReplace()

```
RLAPI void ImageColorReplace (  
    Image * image,  
    Color color,  
    Color replace)
```

6.23.4.310 ImageColorTint()

```
RLAPI void ImageColorTint (  
    Image * image,  
    Color color)
```

6.23.4.311 ImageCopy()

```
RLAPI Image ImageCopy (  
    Image image)
```

6.23.4.312 ImageCrop()

```
RLAPI void ImageCrop (  
    Image * image,  
    Rectangle crop)
```

6.23.4.313 ImageDither()

```
RLAPI void ImageDither (  
    Image * image,  
    int rBpp,  
    int gBpp,  
    int bBpp,  
    int aBpp)
```

6.23.4.314 ImageDraw()

```
RLAPI void ImageDraw (
    Image * dst,
    Image src,
    Rectangle srcRec,
    Rectangle dstRec,
    Color tint)
```

6.23.4.315 ImageDrawCircle()

```
RLAPI void ImageDrawCircle (
    Image * dst,
    int centerX,
    int centerY,
    int radius,
    Color color)
```

6.23.4.316 ImageDrawCircleLines()

```
RLAPI void ImageDrawCircleLines (
    Image * dst,
    int centerX,
    int centerY,
    int radius,
    Color color)
```

6.23.4.317 ImageDrawCircleLinesV()

```
RLAPI void ImageDrawCircleLinesV (
    Image * dst,
    Vector2 center,
    int radius,
    Color color)
```

6.23.4.318 ImageDrawCircleV()

```
RLAPI void ImageDrawCircleV (
    Image * dst,
    Vector2 center,
    int radius,
    Color color)
```

6.23.4.319 ImageDrawLine()

```
RLAPI void ImageDrawLine (
    Image * dst,
    int startPosX,
    int startPosY,
    int endPosX,
    int endPosY,
    Color color)
```


6.23.4.320 ImageDrawLineEx()

```
RLAPI void ImageDrawLineEx (
    Image * dst,
    Vector2 start,
    Vector2 end,
    int thick,
    Color color)
```

6.23.4.321 ImageDrawLineV()

```
RLAPI void ImageDrawLineV (
    Image * dst,
    Vector2 start,
    Vector2 end,
    Color color)
```

6.23.4.322 ImageDrawPixel()

```
RLAPI void ImageDrawPixel (
    Image * dst,
    int posX,
    int posY,
    Color color)
```

6.23.4.323 ImageDrawPixelV()

```
RLAPI void ImageDrawPixelV (
    Image * dst,
    Vector2 position,
    Color color)
```

6.23.4.324 ImageDrawRectangle()

```
RLAPI void ImageDrawRectangle (
    Image * dst,
    int posX,
    int posY,
    int width,
    int height,
    Color color)
```

6.23.4.325 ImageDrawRectangleLines()

```
RLAPI void ImageDrawRectangleLines (
    Image * dst,
    Rectangle rec,
    int thick,
    Color color)
```

6.23.4.326 ImageDrawRectangleRec()

```
RLAPI void ImageDrawRectangleRec (  
    Image * dst,  
    Rectangle rec,  
    Color color)
```

6.23.4.327 ImageDrawRectangleV()

```
RLAPI void ImageDrawRectangleV (  
    Image * dst,  
    Vector2 position,  
    Vector2 size,  
    Color color)
```

6.23.4.328 ImageDrawText()

```
RLAPI void ImageDrawText (  
    Image * dst,  
    const char * text,  
    int posX,  
    int posY,  
    int fontSize,  
    Color color)
```

6.23.4.329 ImageDrawTextEx()

```
RLAPI void ImageDrawTextEx (  
    Image * dst,  
    Font font,  
    const char * text,  
    Vector2 position,  
    float fontSize,  
    float spacing,  
    Color tint)
```

6.23.4.330 ImageDrawTriangle()

```
RLAPI void ImageDrawTriangle (  
    Image * dst,  
    Vector2 v1,  
    Vector2 v2,  
    Vector2 v3,  
    Color color)
```

6.23.4.331 ImageDrawTriangleEx()

```
RLAPI void ImageDrawTriangleEx (
    Image * dst,
    Vector2 v1,
    Vector2 v2,
    Vector2 v3,
    Color c1,
    Color c2,
    Color c3)
```

6.23.4.332 ImageDrawTriangleFan()

```
RLAPI void ImageDrawTriangleFan (
    Image * dst,
    const Vector2 * points,
    int pointCount,
    Color color)
```

6.23.4.333 ImageDrawTriangleLines()

```
RLAPI void ImageDrawTriangleLines (
    Image * dst,
    Vector2 v1,
    Vector2 v2,
    Vector2 v3,
    Color color)
```

6.23.4.334 ImageDrawTriangleStrip()

```
RLAPI void ImageDrawTriangleStrip (
    Image * dst,
    const Vector2 * points,
    int pointCount,
    Color color)
```

6.23.4.335 ImageFlipHorizontal()

```
RLAPI void ImageFlipHorizontal (
    Image * image)
```

6.23.4.336 ImageFlipVertical()

```
RLAPI void ImageFlipVertical (
    Image * image)
```

6.23.4.337 ImageFormat()

```
RLAPI void ImageFormat (  
    Image * image,  
    int newFormat)
```

6.23.4.338 ImageFromChannel()

```
RLAPI Image ImageFromChannel (  
    Image image,  
    int selectedChannel)
```

6.23.4.339 ImageFromImage()

```
RLAPI Image ImageFromImage (  
    Image image,  
    Rectangle rec)
```

6.23.4.340 ImageKernelConvolution()

```
RLAPI void ImageKernelConvolution (  
    Image * image,  
    const float * kernel,  
    int kernelSize)
```

6.23.4.341 ImageMipmaps()

```
RLAPI void ImageMipmaps (  
    Image * image)
```

6.23.4.342 ImageResize()

```
RLAPI void ImageResize (  
    Image * image,  
    int newWidth,  
    int newHeight)
```

6.23.4.343 ImageResizeCanvas()

```
RLAPI void ImageResizeCanvas (  
    Image * image,  
    int newWidth,  
    int newHeight,  
    int offsetX,  
    int offsetY,  
    Color fill)
```

6.23.4.344 ImageResizeNN()

```
RLAPI void ImageResizeNN (
    Image * image,
    int newWidth,
    int newHeight)
```

6.23.4.345 ImageRotate()

```
RLAPI void ImageRotate (
    Image * image,
    int degrees)
```

6.23.4.346 ImageRotateCCW()

```
RLAPI void ImageRotateCCW (
    Image * image)
```

6.23.4.347 ImageRotateCW()

```
RLAPI void ImageRotateCW (
    Image * image)
```

6.23.4.348 ImageText()

```
RLAPI Image ImageText (
    const char * text,
    int fontSize,
    Color color)
```

6.23.4.349 ImageTextEx()

```
RLAPI Image ImageTextEx (
    Font font,
    const char * text,
    float fontSize,
    float spacing,
    Color tint)
```

6.23.4.350 ImageToPOT()

```
RLAPI void ImageToPOT (
    Image * image,
    Color fill)
```

6.23.4.351 InitAudioDevice()

```
RLAPI void InitAudioDevice (  
    void )
```

6.23.4.352 InitWindow()

```
RLAPI void InitWindow (  
    int width,  
    int height,  
    const char * title)
```

6.23.4.353 IsAudioDeviceReady()

```
RLAPI bool IsAudioDeviceReady (  
    void )
```

6.23.4.354 IsAudioStreamPlaying()

```
RLAPI bool IsAudioStreamPlaying (  
    AudioStream stream)
```

6.23.4.355 IsAudioStreamProcessed()

```
RLAPI bool IsAudioStreamProcessed (  
    AudioStream stream)
```

6.23.4.356 IsAudioStreamValid()

```
RLAPI bool IsAudioStreamValid (  
    AudioStream stream)
```

6.23.4.357 IsCursorHidden()

```
RLAPI bool IsCursorHidden (  
    void )
```

6.23.4.358 IsCursorOnScreen()

```
RLAPI bool IsCursorOnScreen (  
    void )
```

6.23.4.359 IsFileDropped()

```
RLAPI bool IsFileDropped (  
    void )
```

6.23.4.360 IsFileExtension()

```
RLAPI bool IsFileExtension (  
    const char * fileName,  
    const char * ext)
```

6.23.4.361 IsFileNameValid()

```
RLAPI bool IsFileNameValid (  
    const char * fileName)
```

6.23.4.362 IsFontValid()

```
RLAPI bool IsFontValid (  
    Font font)
```

6.23.4.363 IsGamepadAvailable()

```
RLAPI bool IsGamepadAvailable (  
    int gamepad)
```

6.23.4.364 IsGamepadButtonDown()

```
RLAPI bool IsGamepadButtonDown (  
    int gamepad,  
    int button)
```

6.23.4.365 IsGamepadButtonPressed()

```
RLAPI bool IsGamepadButtonPressed (  
    int gamepad,  
    int button)
```

6.23.4.366 IsGamepadButtonReleased()

```
RLAPI bool IsGamepadButtonReleased (  
    int gamepad,  
    int button)
```

6.23.4.367 IsGamepadButtonUp()

```
RLAPI bool IsGamepadButtonUp (
    int gamepad,
    int button)
```

6.23.4.368 IsGestureDetected()

```
RLAPI bool IsGestureDetected (
    unsigned int gesture)
```

6.23.4.369 IsImageValid()

```
RLAPI bool IsImageValid (
    Image image)
```

6.23.4.370 IsKeyDown()

```
RLAPI bool IsKeyDown (
    int key)
```

6.23.4.371 IsKeyPressed()

```
RLAPI bool IsKeyPressed (
    int key)
```

6.23.4.372 IsKeyPressedRepeat()

```
RLAPI bool IsKeyPressedRepeat (
    int key)
```

6.23.4.373 IsKeyReleased()

```
RLAPI bool IsKeyReleased (
    int key)
```

6.23.4.374 IsKeyUp()

```
RLAPI bool IsKeyUp (
    int key)
```

6.23.4.375 IsMaterialValid()

```
RLAPI bool IsMaterialValid (
    Material material)
```


6.23.4.376 IsModelAnimationValid()

```
RLAPI bool IsModelAnimationValid (
    Model model,
    ModelAnimation anim)
```

6.23.4.377 IsModelValid()

```
RLAPI bool IsModelValid (
    Model model)
```

6.23.4.378 IsMouseButtonDown()

```
RLAPI bool IsMouseButtonDown (
    int button)
```

6.23.4.379 IsMouseButtonPressed()

```
RLAPI bool IsMouseButtonPressed (
    int button)
```

6.23.4.380 IsMouseButtonReleased()

```
RLAPI bool IsMouseButtonReleased (
    int button)
```

6.23.4.381 IsMouseButtonUp()

```
RLAPI bool IsMouseButtonUp (
    int button)
```

6.23.4.382 IsMusicStreamPlaying()

```
RLAPI bool IsMusicStreamPlaying (
    Music music)
```

6.23.4.383 IsMusicValid()

```
RLAPI bool IsMusicValid (
    Music music)
```

6.23.4.384 IsPathFile()

```
RLAPI bool IsPathFile (
    const char * path)
```

6.23.4.385 IsRenderTextureValid()

```
RLAPI bool IsRenderTextureValid (
    RenderTexture2D target)
```

6.23.4.386 IsShaderValid()

```
RLAPI bool IsShaderValid (
    Shader shader)
```

6.23.4.387 IsSoundPlaying()

```
RLAPI bool IsSoundPlaying (
    Sound sound)
```

6.23.4.388 IsSoundValid()

```
RLAPI bool IsSoundValid (
    Sound sound)
```

6.23.4.389 IsTextureValid()

```
RLAPI bool IsTextureValid (
    Texture2D texture)
```

6.23.4.390 IsWaveValid()

```
RLAPI bool IsWaveValid (
    Wave wave)
```

6.23.4.391 IsWindowFocused()

```
RLAPI bool IsWindowFocused (
    void )
```

6.23.4.392 IsWindowFullscreen()

```
RLAPI bool IsWindowFullscreen (
    void )
```

6.23.4.393 IsWindowHidden()

```
RLAPI bool IsWindowHidden (
    void )
```

6.23.4.394 IsWindowMaximized()

```
RLAPI bool IsWindowMaximized (  
    void )
```

6.23.4.395 IsWindowMinimized()

```
RLAPI bool IsWindowMinimized (  
    void )
```

6.23.4.396 IsWindowReady()

```
RLAPI bool IsWindowReady (  
    void )
```

6.23.4.397 IsWindowResized()

```
RLAPI bool IsWindowResized (  
    void )
```

6.23.4.398 IsWindowState()

```
RLAPI bool IsWindowState (  
    unsigned int flag)
```

6.23.4.399 LoadAudioStream()

```
RLAPI AudioStream LoadAudioStream (  
    unsigned int sampleRate,  
    unsigned int sampleSize,  
    unsigned int channels)
```

6.23.4.400 LoadAutomationEventList()

```
RLAPI AutomationEventList LoadAutomationEventList (  
    const char * fileName)
```

6.23.4.401 LoadCodepoints()

```
RLAPI int * LoadCodepoints (  
    const char * text,  
    int * count)
```

6.23.4.402 LoadDirectoryFiles()

```
RLAPI FilePathList LoadDirectoryFiles (
    const char * dirPath)
```

6.23.4.403 LoadDirectoryFilesEx()

```
RLAPI FilePathList LoadDirectoryFilesEx (
    const char * basePath,
    const char * filter,
    bool scanSubdirs)
```

6.23.4.404 LoadDroppedFiles()

```
RLAPI FilePathList LoadDroppedFiles (
    void )
```

6.23.4.405 LoadFileData()

```
RLAPI unsigned char * LoadFileData (
    const char * fileName,
    int * dataSize)
```

6.23.4.406 LoadFileText()

```
RLAPI char * LoadFileText (
    const char * fileName)
```

6.23.4.407 LoadFont()

```
RLAPI Font LoadFont (
    const char * fileName)
```

6.23.4.408 LoadFontData()

```
RLAPI GlyphInfo * LoadFontData (
    const unsigned char * fileData,
    int dataSize,
    int fontSize,
    const int * codepoints,
    int codepointCount,
    int type,
    int * glyphCount)
```

6.23.4.409 LoadFontEx()

```
RLAPI Font LoadFontEx (
    const char * fileName,
    int fontSize,
    const int * codepoints,
    int codepointCount)
```

6.23.4.410 LoadFontFromImage()

```
RLAPI Font LoadFontFromImage (
    Image image,
    Color key,
    int firstChar)
```

6.23.4.411 LoadFontFromMemory()

```
RLAPI Font LoadFontFromMemory (
    const char * fileType,
    const unsigned char * fileData,
    int dataSize,
    int fontSize,
    const int * codepoints,
    int codepointCount)
```

6.23.4.412 LoadImage()

```
RLAPI Image LoadImage (
    const char * fileName)
```

6.23.4.413 LoadImageAnim()

```
RLAPI Image LoadImageAnim (
    const char * fileName,
    int * frames)
```

6.23.4.414 LoadImageAnimFromMemory()

```
RLAPI Image LoadImageAnimFromMemory (
    const char * fileType,
    const unsigned char * fileData,
    int dataSize,
    int * frames)
```

6.23.4.415 LoadImageColors()

```
RLAPI Color * LoadImageColors (
    Image image)
```

6.23.4.416 LoadImageFromMemory()

```
RLAPI Image LoadImageFromMemory (
    const char * fileType,
    const unsigned char * fileData,
    int dataSize)
```

6.23.4.417 LoadImageFromScreen()

```
RLAPI Image LoadImageFromScreen (
    void )
```

6.23.4.418 LoadImageFromTexture()

```
RLAPI Image LoadImageFromTexture (
    Texture2D texture)
```

6.23.4.419 LoadImagePalette()

```
RLAPI Color * LoadImagePalette (
    Image image,
    int maxPaletteSize,
    int * colorCount)
```

6.23.4.420 LoadImageRaw()

```
RLAPI Image LoadImageRaw (
    const char * fileName,
    int width,
    int height,
    int format,
    int headerSize)
```

6.23.4.421 LoadMaterialDefault()

```
RLAPI Material LoadMaterialDefault (
    void )
```

6.23.4.422 LoadMaterials()

```
RLAPI Material * LoadMaterials (
    const char * fileName,
    int * materialCount)
```

6.23.4.423 LoadModel()

```
RLAPI Model LoadModel (
    const char * fileName)
```

6.23.4.424 LoadModelAnimations()

```
RLAPI ModelAnimation * LoadModelAnimations (
    const char * fileName,
    int * animCount)
```

6.23.4.425 LoadModelFromMesh()

```
RLAPI Model LoadModelFromMesh (
    Mesh mesh)
```

6.23.4.426 LoadMusicStream()

```
RLAPI Music LoadMusicStream (
    const char * fileName)
```

6.23.4.427 LoadMusicStreamFromMemory()

```
RLAPI Music LoadMusicStreamFromMemory (
    const char * fileType,
    const unsigned char * data,
    int dataSize)
```

6.23.4.428 LoadRandomSequence()

```
RLAPI int * LoadRandomSequence (
    unsigned int count,
    int min,
    int max)
```

6.23.4.429 LoadRenderTexture()

```
RLAPI RenderTexture2D LoadRenderTexture (
    int width,
    int height)
```

6.23.4.430 LoadShader()

```
RLAPI Shader LoadShader (
    const char * vsFileName,
    const char * fsFileName)
```

6.23.4.431 LoadShaderFromMemory()

```
RLAPI Shader LoadShaderFromMemory (
    const char * vsCode,
    const char * fsCode)
```

6.23.4.432 LoadSound()

```
RLAPI Sound LoadSound (
    const char * fileName)
```

6.23.4.433 LoadSoundAlias()

```
RLAPI Sound LoadSoundAlias (
    Sound source)
```

6.23.4.434 LoadSoundFromWave()

```
RLAPI Sound LoadSoundFromWave (
    Wave wave)
```

6.23.4.435 LoadTextLines()

```
RLAPI char ** LoadTextLines (
    const char * text,
    int * count)
```

6.23.4.436 LoadTexture()

```
RLAPI Texture2D LoadTexture (
    const char * fileName)
```

6.23.4.437 LoadTextureCubemap()

```
RLAPI TextureCubemap LoadTextureCubemap (
    Image image,
    int layout)
```

6.23.4.438 LoadTextureFromImage()

```
RLAPI Texture2D LoadTextureFromImage (
    Image image)
```


6.23.4.439 LoadUTF8()

```
RLAPI char * LoadUTF8 (
    const int * codepoints,
    int length)
```

6.23.4.440 LoadVrStereoConfig()

```
RLAPI VrStereoConfig LoadVrStereoConfig (
    VrDeviceInfo device)
```

6.23.4.441 LoadWave()

```
RLAPI Wave LoadWave (
    const char * fileName)
```

6.23.4.442 LoadWaveFromMemory()

```
RLAPI Wave LoadWaveFromMemory (
    const char * fileType,
    const unsigned char * fileData,
    int dataSize)
```

6.23.4.443 LoadWaveSamples()

```
RLAPI float * LoadWaveSamples (
    Wave wave)
```

6.23.4.444 MakeDirectory()

```
RLAPI int MakeDirectory (
    const char * dirPath)
```

6.23.4.445 MaximizeWindow()

```
RLAPI void MaximizeWindow (
    void )
```

6.23.4.446 MeasureText()

```
RLAPI int MeasureText (
    const char * text,
    int fontSize)
```

6.23.4.447 MeasureTextCodepoints()

```
RLAPI Vector2 MeasureTextCodepoints (  
    Font font,  
    const int * codepoints,  
    int length,  
    float fontSize,  
    float spacing)
```

6.23.4.448 MeasureTextEx()

```
RLAPI Vector2 MeasureTextEx (  
    Font font,  
    const char * text,  
    float fontSize,  
    float spacing)
```

6.23.4.449 MemAlloc()

```
RLAPI void * MemAlloc (  
    unsigned int size)
```

6.23.4.450 MemFree()

```
RLAPI void MemFree (  
    void * ptr)
```

6.23.4.451 MemRealloc()

```
RLAPI void * MemRealloc (  
    void * ptr,  
    unsigned int size)
```

6.23.4.452 MinimizeWindow()

```
RLAPI void MinimizeWindow (  
    void )
```

6.23.4.453 OpenURL()

```
RLAPI void OpenURL (  
    const char * url)
```

6.23.4.454 PauseAudioStream()

```
RLAPI void PauseAudioStream (  
    AudioStream stream)
```

6.23.4.455 PauseMusicStream()

```
RLAPI void PauseMusicStream (
    Music music)
```

6.23.4.456 PauseSound()

```
RLAPI void PauseSound (
    Sound sound)
```

6.23.4.457 PlayAudioStream()

```
RLAPI void PlayAudioStream (
    AudioStream stream)
```

6.23.4.458 PlayAutomationEvent()

```
RLAPI void PlayAutomationEvent (
    AutomationEvent event)
```

6.23.4.459 PlayMusicStream()

```
RLAPI void PlayMusicStream (
    Music music)
```

6.23.4.460 PlaySound()

```
RLAPI void PlaySound (
    Sound sound)
```

6.23.4.461 PollInputEvents()

```
RLAPI void PollInputEvents (
    void )
```

6.23.4.462 RestoreWindow()

```
RLAPI void RestoreWindow (
    void )
```

6.23.4.463 ResumeAudioStream()

```
RLAPI void ResumeAudioStream (
    AudioStream stream)
```

6.23.4.464 ResumeMusicStream()

```
RLAPI void ResumeMusicStream (  
    Music music)
```

6.23.4.465 ResumeSound()

```
RLAPI void ResumeSound (  
    Sound sound)
```

6.23.4.466 SaveFileData()

```
RLAPI bool SaveFileData (  
    const char * fileName,  
    void * data,  
    int dataSize)
```

6.23.4.467 SaveFileText()

```
RLAPI bool SaveFileText (  
    const char * fileName,  
    const char * text)
```

6.23.4.468 SeekMusicStream()

```
RLAPI void SeekMusicStream (  
    Music music,  
    float position)
```

6.23.4.469 SetAudioStreamBufferSizeDefault()

```
RLAPI void SetAudioStreamBufferSizeDefault (  
    int size)
```

6.23.4.470 SetAudioStreamCallback()

```
RLAPI void SetAudioStreamCallback (  
    AudioStream stream,  
    AudioCallback callback)
```

6.23.4.471 SetAudioStreamPan()

```
RLAPI void SetAudioStreamPan (  
    AudioStream stream,  
    float pan)
```

6.23.4.472 SetAudioStreamPitch()

```
RLAPI void SetAudioStreamPitch (
    AudioStream stream,
    float pitch)
```

6.23.4.473 SetAudioStreamVolume()

```
RLAPI void SetAudioStreamVolume (
    AudioStream stream,
    float volume)
```

6.23.4.474 SetAutomationEventBaseFrame()

```
RLAPI void SetAutomationEventBaseFrame (
    int frame)
```

6.23.4.475 SetAutomationEventList()

```
RLAPI void SetAutomationEventList (
    AutomationEventList * list)
```

6.23.4.476 SetClipboardText()

```
RLAPI void SetClipboardText (
    const char * text)
```

6.23.4.477 SetConfigFlags()

```
RLAPI void SetConfigFlags (
    unsigned int flags)
```

6.23.4.478 SetExitKey()

```
RLAPI void SetExitKey (
    int key)
```

6.23.4.479 SetGamepadMappings()

```
RLAPI int SetGamepadMappings (
    const char * mappings)
```

6.23.4.480 SetGamepadVibration()

```
RLAPI void SetGamepadVibration (
    int gamepad,
    float leftMotor,
    float rightMotor,
    float duration)
```

6.23.4.481 SetGesturesEnabled()

```
RLAPI void SetGesturesEnabled (
    unsigned int flags)
```

6.23.4.482 SetLoadFileDataCallback()

```
RLAPI void SetLoadFileDataCallback (
    LoadFileDataCallback callback)
```

6.23.4.483 SetLoadFileTextCallback()

```
RLAPI void SetLoadFileTextCallback (
    LoadFileTextCallback callback)
```

6.23.4.484 SetMasterVolume()

```
RLAPI void SetMasterVolume (
    float volume)
```

6.23.4.485 SetMaterialTexture()

```
RLAPI void SetMaterialTexture (
    Material * material,
    int mapType,
    Texture2D texture)
```

6.23.4.486 SetModelMeshMaterial()

```
RLAPI void SetModelMeshMaterial (
    Model * model,
    int meshId,
    int materialId)
```

6.23.4.487 SetMouseCursor()

```
RLAPI void SetMouseCursor (
    int cursor)
```

6.23.4.488 SetMouseOffset()

```
RLAPI void SetMouseOffset (
    int offsetX,
    int offsetY)
```

6.23.4.489 SetMousePosition()

```
RLAPI void SetMousePosition (
    int x,
    int y)
```

6.23.4.490 SetMouseScale()

```
RLAPI void SetMouseScale (
    float scaleX,
    float scaleY)
```

6.23.4.491 SetMusicPan()

```
RLAPI void SetMusicPan (
    Music music,
    float pan)
```

6.23.4.492 SetMusicPitch()

```
RLAPI void SetMusicPitch (
    Music music,
    float pitch)
```

6.23.4.493 SetMusicVolume()

```
RLAPI void SetMusicVolume (
    Music music,
    float volume)
```

6.23.4.494 SetPixelColor()

```
RLAPI void SetPixelColor (
    void * dstPtr,
    Color color,
    int format)
```

6.23.4.495 SetRandomSeed()

```
RLAPI void SetRandomSeed (
    unsigned int seed)
```

6.23.4.496 SetSaveFileDataCallback()

```
RLAPI void SetSaveFileDataCallback (  
    SaveFileDataCallback callback)
```

6.23.4.497 SetSaveFileTextCallback()

```
RLAPI void SetSaveFileTextCallback (  
    SaveFileTextCallback callback)
```

6.23.4.498 SetShaderValue()

```
RLAPI void SetShaderValue (  
    Shader shader,  
    int locIndex,  
    const void * value,  
    int uniformType)
```

6.23.4.499 SetShaderValueMatrix()

```
RLAPI void SetShaderValueMatrix (  
    Shader shader,  
    int locIndex,  
    Matrix mat)
```

6.23.4.500 SetShaderValueTexture()

```
RLAPI void SetShaderValueTexture (  
    Shader shader,  
    int locIndex,  
    Texture2D texture)
```

6.23.4.501 SetShaderValueV()

```
RLAPI void SetShaderValueV (  
    Shader shader,  
    int locIndex,  
    const void * value,  
    int uniformType,  
    int count)
```

6.23.4.502 SetShapesTexture()

```
RLAPI void SetShapesTexture (  
    Texture2D texture,  
    Rectangle source)
```


6.23.4.503 SetSoundPan()

```
RLAPI void SetSoundPan (
    Sound sound,
    float pan)
```

6.23.4.504 SetSoundPitch()

```
RLAPI void SetSoundPitch (
    Sound sound,
    float pitch)
```

6.23.4.505 SetSoundVolume()

```
RLAPI void SetSoundVolume (
    Sound sound,
    float volume)
```

6.23.4.506 SetTargetFPS()

```
RLAPI void SetTargetFPS (
    int fps)
```

6.23.4.507 SetTextLineSpacing()

```
RLAPI void SetTextLineSpacing (
    int spacing)
```

6.23.4.508 SetTextureFilter()

```
RLAPI void SetTextureFilter (
    Texture2D texture,
    int filter)
```

6.23.4.509 SetTextureWrap()

```
RLAPI void SetTextureWrap (
    Texture2D texture,
    int wrap)
```

6.23.4.510 SetTraceLogCallback()

```
RLAPI void SetTraceLogCallback (
    TraceLogCallback callback)
```

6.23.4.511 SetTraceLogLevel()

```
RLAPI void SetTraceLogLevel (
    int logLevel)
```

6.23.4.512 SetWindowFocused()

```
RLAPI void SetWindowFocused (
    void )
```

6.23.4.513 SetWindowIcon()

```
RLAPI void SetWindowIcon (
    Image image)
```

6.23.4.514 SetWindowIcons()

```
RLAPI void SetWindowIcons (
    Image * images,
    int count)
```

6.23.4.515 SetWindowMaxSize()

```
RLAPI void SetWindowMaxSize (
    int width,
    int height)
```

6.23.4.516 SetWindowMinSize()

```
RLAPI void SetWindowMinSize (
    int width,
    int height)
```

6.23.4.517 SetWindowMonitor()

```
RLAPI void SetWindowMonitor (
    int monitor)
```

6.23.4.518 SetWindowOpacity()

```
RLAPI void SetWindowOpacity (
    float opacity)
```

6.23.4.519 SetWindowPosition()

```
RLAPI void SetWindowPosition (
    int x,
    int y)
```

6.23.4.520 SetWindowSize()

```
RLAPI void SetWindowSize (
    int width,
    int height)
```

6.23.4.521 SetWindowState()

```
RLAPI void SetWindowState (
    unsigned int flags)
```

6.23.4.522 SetWindowTitle()

```
RLAPI void SetWindowTitle (
    const char * title)
```

6.23.4.523 ShowCursor()

```
RLAPI void ShowCursor (
    void )
```

6.23.4.524 StartAutomationEventRecording()

```
RLAPI void StartAutomationEventRecording (
    void )
```

6.23.4.525 StopAudioStream()

```
RLAPI void StopAudioStream (
    AudioStream stream)
```

6.23.4.526 StopAutomationEventRecording()

```
RLAPI void StopAutomationEventRecording (
    void )
```

6.23.4.527 StopMusicStream()

```
RLAPI void StopMusicStream (  
    Music music)
```

6.23.4.528 StopSound()

```
RLAPI void StopSound (  
    Sound sound)
```

6.23.4.529 SwapScreenBuffer()

```
RLAPI void SwapScreenBuffer (  
    void )
```

6.23.4.530 TakeScreenshot()

```
RLAPI void TakeScreenshot (  
    const char * fileName)
```

6.23.4.531 TextAppend()

```
RLAPI void TextAppend (  
    char * text,  
    const char * append,  
    int * position)
```

6.23.4.532 TextCopy()

```
RLAPI int TextCopy (  
    char * dst,  
    const char * src)
```

6.23.4.533 TextFindIndex()

```
RLAPI int TextFindIndex (  
    const char * text,  
    const char * search)
```

6.23.4.534 TextFormat()

```
RLAPI const char * TextFormat (  
    const char * text,  
    ...)
```

6.23.4.535 TextInsert()

```
RLAPI char * TextInsert (
    const char * text,
    const char * insert,
    int position)
```

6.23.4.536 TextInsertAlloc()

```
RLAPI char * TextInsertAlloc (
    const char * text,
    const char * insert,
    int position)
```

6.23.4.537 TextIsEqual()

```
RLAPI bool TextIsEqual (
    const char * text1,
    const char * text2)
```

6.23.4.538 TextJoin()

```
RLAPI char * TextJoin (
    char ** textList,
    int count,
    const char * delimiter)
```

6.23.4.539 TextLength()

```
RLAPI unsigned int TextLength (
    const char * text)
```

6.23.4.540 TextRemoveSpaces()

```
RLAPI const char * TextRemoveSpaces (
    const char * text)
```

6.23.4.541 TextReplace()

```
RLAPI char * TextReplace (
    const char * text,
    const char * search,
    const char * replacement)
```

6.23.4.542 TextReplaceAlloc()

```
RLAPI char * TextReplaceAlloc (
    const char * text,
    const char * search,
    const char * replacement)
```

6.23.4.543 TextReplaceBetween()

```
RLAPI char * TextReplaceBetween (
    const char * text,
    const char * begin,
    const char * end,
    const char * replacement)
```

6.23.4.544 TextReplaceBetweenAlloc()

```
RLAPI char * TextReplaceBetweenAlloc (
    const char * text,
    const char * begin,
    const char * end,
    const char * replacement)
```

6.23.4.545 TextSplit()

```
RLAPI char ** TextSplit (
    const char * text,
    char delimiter,
    int * count)
```

6.23.4.546 TextSubtext()

```
RLAPI const char * TextSubtext (
    const char * text,
    int position,
    int length)
```

6.23.4.547 TextToCamel()

```
RLAPI char * TextToCamel (
    const char * text)
```

6.23.4.548 TextToFloat()

```
RLAPI float TextToFloat (
    const char * text)
```

6.23.4.549 TextToInteger()

```
RLAPI int TextToInteger (
    const char * text)
```

6.23.4.550 TextToLower()

```
RLAPI char * TextToLower (
    const char * text)
```

6.23.4.551 TextToPascal()

```
RLAPI char * TextToPascal (
    const char * text)
```

6.23.4.552 TextToSnake()

```
RLAPI char * TextToSnake (
    const char * text)
```

6.23.4.553 TextToUpper()

```
RLAPI char * TextToUpper (
    const char * text)
```

6.23.4.554 ToggleBorderlessWindowed()

```
RLAPI void ToggleBorderlessWindowed (
    void )
```

6.23.4.555 ToggleFullscreen()

```
RLAPI void ToggleFullscreen (
    void )
```

6.23.4.556 TraceLog()

```
RLAPI void TraceLog (
    int logLevel,
    const char * text,
    ...)
```

6.23.4.557 UnloadAudioStream()

```
RLAPI void UnloadAudioStream (  
    AudioStream stream)
```

6.23.4.558 UnloadAutomationEventList()

```
RLAPI void UnloadAutomationEventList (  
    AutomationEventList list)
```

6.23.4.559 UnloadCodepoints()

```
RLAPI void UnloadCodepoints (  
    int * codepoints)
```

6.23.4.560 UnloadDirectoryFiles()

```
RLAPI void UnloadDirectoryFiles (  
    FilePathList files)
```

6.23.4.561 UnloadDroppedFiles()

```
RLAPI void UnloadDroppedFiles (  
    FilePathList files)
```

6.23.4.562 UnloadFileData()

```
RLAPI void UnloadFileData (  
    unsigned char * data)
```

6.23.4.563 UnloadFileText()

```
RLAPI void UnloadFileText (  
    char * text)
```

6.23.4.564 UnloadFont()

```
RLAPI void UnloadFont (  
    Font font)
```

6.23.4.565 UnloadFontData()

```
RLAPI void UnloadFontData (  
    GlyphInfo * glyphs,  
    int glyphCount)
```


6.23.4.566 UnloadImage()

```
RLAPI void UnloadImage (  
    Image image)
```

6.23.4.567 UnloadImageColors()

```
RLAPI void UnloadImageColors (  
    Color * colors)
```

6.23.4.568 UnloadImagePalette()

```
RLAPI void UnloadImagePalette (  
    Color * colors)
```

6.23.4.569 UnloadMaterial()

```
RLAPI void UnloadMaterial (  
    Material material)
```

6.23.4.570 UnloadMesh()

```
RLAPI void UnloadMesh (  
    Mesh mesh)
```

6.23.4.571 UnloadModel()

```
RLAPI void UnloadModel (  
    Model model)
```

6.23.4.572 UnloadModelAnimations()

```
RLAPI void UnloadModelAnimations (  
    ModelAnimation * animations,  
    int animCount)
```

6.23.4.573 UnloadMusicStream()

```
RLAPI void UnloadMusicStream (  
    Music music)
```

6.23.4.574 UnloadRandomSequence()

```
RLAPI void UnloadRandomSequence (  
    int * sequence)
```

6.23.4.575 UnloadRenderTexture()

```
RLAPI void UnloadRenderTexture (  
    RenderTexture2D target)
```

6.23.4.576 UnloadShader()

```
RLAPI void UnloadShader (  
    Shader shader)
```

6.23.4.577 UnloadSound()

```
RLAPI void UnloadSound (  
    Sound sound)
```

6.23.4.578 UnloadSoundAlias()

```
RLAPI void UnloadSoundAlias (  
    Sound alias)
```

6.23.4.579 UnloadTextLines()

```
RLAPI void UnloadTextLines (  
    char ** text,  
    int lineCount)
```

6.23.4.580 UnloadTexture()

```
RLAPI void UnloadTexture (  
    Texture2D texture)
```

6.23.4.581 UnloadUTF8()

```
RLAPI void UnloadUTF8 (  
    char * text)
```

6.23.4.582 UnloadVrStereoConfig()

```
RLAPI void UnloadVrStereoConfig (  
    VrStereoConfig config)
```

6.23.4.583 UnloadWave()

```
RLAPI void UnloadWave (  
    Wave wave)
```

6.23.4.584 UnloadWaveSamples()

```
RLAPI void UnloadWaveSamples (
    float * samples)
```

6.23.4.585 UpdateAudioStream()

```
RLAPI void UpdateAudioStream (
    AudioStream stream,
    const void * data,
    int frameCount)
```

6.23.4.586 UpdateCamera()

```
RLAPI void UpdateCamera (
    Camera * camera,
    int mode)
```

6.23.4.587 UpdateCameraPro()

```
RLAPI void UpdateCameraPro (
    Camera * camera,
    Vector3 movement,
    Vector3 rotation,
    float zoom)
```

6.23.4.588 UpdateMeshBuffer()

```
RLAPI void UpdateMeshBuffer (
    Mesh mesh,
    int index,
    const void * data,
    int dataSize,
    int offset)
```

6.23.4.589 UpdateModelAnimation()

```
RLAPI void UpdateModelAnimation (
    Model model,
    ModelAnimation anim,
    float frame)
```

6.23.4.590 UpdateModelAnimationEx()

```
RLAPI void UpdateModelAnimationEx (
    Model model,
    ModelAnimation animA,
    float frameA,
    ModelAnimation animB,
    float frameB,
    float blend)
```

6.23.4.591 UpdateMusicStream()

```
RLAPI void UpdateMusicStream (  
    Music music)
```

6.23.4.592 UpdateSound()

```
RLAPI void UpdateSound (  
    Sound sound,  
    const void * data,  
    int sampleCount)
```

6.23.4.593 UpdateTexture()

```
RLAPI void UpdateTexture (  
    Texture2D texture,  
    const void * pixels)
```

6.23.4.594 UpdateTextureRec()

```
RLAPI void UpdateTextureRec (  
    Texture2D texture,  
    Rectangle rec,  
    const void * pixels)
```

6.23.4.595 UploadMesh()

```
RLAPI void UploadMesh (  
    Mesh * mesh,  
    bool dynamic)
```

6.23.4.596 WaitTime()

```
RLAPI void WaitTime (  
    double seconds)
```

6.23.4.597 WaveCopy()

```
RLAPI Wave WaveCopy (  
    Wave wave)
```

6.23.4.598 WaveCrop()

```
RLAPI void WaveCrop (  
    Wave * wave,  
    int initFrame,  
    int finalFrame)
```

6.23.4.599 WaveFormat()

```
RLAPI void WaveFormat (
    Wave * wave,
    int sampleRate,
    int sampleSize,
    int channels)
```

6.23.4.600 WindowShouldClose()

```
RLAPI bool WindowShouldClose (
    void )
```

6.24 raylib.h

[Go to the documentation of this file.](#)

```
00001 /*****
00002 *
00003 *   raylib v6.0 - A simple and easy-to-use library to enjoy videogames programming (www.raylib.com)
00004 *
00005 *   FEATURES:
00006 *       - NO external dependencies, all required libraries included with raylib
00007 *       - Multiplatform: Windows, Linux, macOS, FreeBSD, Web, Android, Raspberry Pi, DRM native...
00008 *       - Written in plain C code (C99) in PascalCase/camelCase notation
00009 *       - Hardware accelerated with OpenGL (1.1, 2.1, 3.3, 4.3, ES2, ES3 - choose at compile)
00010 *       - Software renderer optional, for systems with no GPU: [rlsw]
00011 *       - Custom OpenGL abstraction layer (usable as standalone module): [rlgl]
00012 *       - Multiple Fonts formats supported (TTF, OTF, FNT, BDF, Sprite fonts)
00013 *       - Many texture formats supported, including compressed formats (DXT, ETC, ASTC)
00014 *       - Full 3d support for 3d Shapes, Models, Billboards, Heightmaps and more!
00015 *       - Flexible Materials system, supporting classic maps and PBR maps
00016 *       - Animated 3D models supported (skeletal bones animation) (IQM, M3D, GLTF)
00017 *       - Shaders support, including Model shaders and Postprocessing shaders
00018 *       - Powerful math module for Vector, Matrix and Quaternion operations: [raymath]
00019 *       - Audio loading and playing with streaming support (WAV, OGG, MP3, FLAC, QOA, XM, MOD)
00020 *       - VR stereo rendering with configurable HMD device parameters
00021 *       - Bindings to multiple programming languages available!
00022 *
00023 *   NOTES:
00024 *       - One default Font is loaded on InitWindow()->LoadFontDefault() [core, text]
00025 *       - One default Texture2D is loaded on rlglInit(), 1x1 white pixel R8G8B8A8 [rlgl] (OpenGL 3.3
00026 *         or ES2)
00027 *       - One default Shader is loaded on rlglInit()->rlLoadShaderDefault() [rlgl] (OpenGL 3.3 or ES2)
00028 *       - One default RenderBatch is loaded on rlglInit()->rlLoadRenderBatch() [rlgl] (OpenGL 3.3 or
00029 *         ES2)
00030 *
00031 *   DEPENDENCIES:
00032 *       [rcore] Depends on the selected platform backend, check rc core.c header for details
00033 *       [rlgl] glad/glad_gles2 (David Herberth - github.com/Dav1dde/glad) for OpenGL extensions
00034 *       loading
00035 *       [raudio] miniaudio (David Reid - github.com/mackron/miniaudio) for audio device/context
00036 *       management
00037 *
00038 *   OPTIONAL DEPENDENCIES (included):
00039 *       [rcore] sinfl (Micha Mettke) for DEFLATE decompression algorithm
00040 *       [rcore] sdefl (Micha Mettke) for DEFLATE compression algorithm
00041 *       [rcore] rprand (Ramon Santamaria) for pseudo-random numbers generation
00042 *       [rttextures] qoi (Dominic Szablewski - https://phoboslab.org) for QOI image management
00043 *       [rttextures] stb_image (Sean Barrett) for images loading (BMP, TGA, PNG, JPEG, HDR...)
00044 *       [rttextures] stb_image_write (Sean Barrett) for image writing (BMP, TGA, PNG, JPG)
00045 *       [rttextures] stb_image_resize2 (Sean Barrett) for image resizing algorithms
00046 *       [rttextures] stb_perlin (Sean Barrett) for Perlin Noise image generation
00047 *       [rttextures] rltexgpu (Ramon Santamaria) for GPU-compressed texture formats
00048 *       [rttext] stb_truetype (Sean Barrett) for ttf fonts loading
00049 *       [rttext] stb_rect_pack (Sean Barrett) for rectangles packing
00050 *       [rmodels] par_shapes (Philip Rideout) for parametric 3d shapes generation
00051 *       [rmodels] tinyobj_loader_c (Syoyo Fujita) for models loading (OBJ, MTL)
00052 *       [rmodels] cgltf (Johannes Kuhlmann) for models loading (glTF)
00053 *       [rmodels] m3d (bzt) for models loading (M3D, https://bztsrc.gitlab.io/model3d)
00054 *       [rmodels] vox_loader (Johann Nadalutti) for models loading (VOX)
00055 *       [raudio] dr_wav (David Reid) for WAV audio file loading
00056 *       [raudio] dr_flac (David Reid) for FLAC audio file loading
00057 */***/
```

```

00053 *      [audio] dr_mp3 (David Reid) for MP3 audio file loading
00054 *      [audio] stb_vorbis (Sean Barrett) for OGG audio loading
00055 *      [audio] jar_xm (Joshua Reisenauer) for XM audio module loading
00056 *      [audio] jar_mod (Joshua Reisenauer) for MOD audio module loading
00057 *      [audio] goa (Dominic Szablewski - https://phoboslab.org) for QOA audio management
00058 *
00059 *
00060 *      LICENSE: zlib/libpng
00061 *
00062 *      raylib is licensed under an unmodified zlib/libpng license, which is an OSI-certified,
00063 *      BSD-like license that allows static linking with closed source software:
00064 *
00065 *      Copyright (c) 2013-2026 Ramon Santamaria (@raysan5)
00066 *
00067 *      This software is provided "as-is", without any express or implied warranty. In no event
00068 *      will the authors be held liable for any damages arising from the use of this software.
00069 *
00070 *      Permission is granted to anyone to use this software for any purpose, including commercial
00071 *      applications, and to alter it and redistribute it freely, subject to the following restrictions:
00072 *
00073 *      1. The origin of this software must not be misrepresented; you must not claim that you
00074 *      wrote the original software. If you use this software in a product, an acknowledgment
00075 *      in the product documentation would be appreciated but is not required.
00076 *
00077 *      2. Altered source versions must be plainly marked as such, and must not be misrepresented
00078 *      as being the original software.
00079 *
00080 *      3. This notice may not be removed or altered from any source distribution.
00081 *
00082 *      *****/
00083
00084 #ifndef RAYLIB_H
00085 #define RAYLIB_H
00086
00087 #include <stdarg.h>          // Required for: va_list - Only used by TraceLogCallback
00088
00089 #define RAYLIB_VERSION_MAJOR 6
00090 #define RAYLIB_VERSION_MINOR 0
00091 #define RAYLIB_VERSION_PATCH 0
00092 #define RAYLIB_VERSION "6.0"
00093
00094 // Function specifiers in case library is build/used as a shared library
00095 // NOTE: Microsoft specifiers to tell compiler that symbols are imported/exported from a .dll
00096 // NOTE: visibility("default") attribute makes symbols "visible" when compiled with
// -fvisibility=hidden
00097 #if defined(_WIN32)
00098     #if defined(__TINYC__)
00099         #define __declspec(x) __attribute__((x))
00100     #endif
00101     #if defined(BUILD_LIBTYPE_SHARED)
00102         #define RLAPI __declspec(dllexport)    // Building the library as a Win32 shared library
00103     #elif defined(USE_LIBTYPE_SHARED)
00104         #define RLAPI __declspec(dllimport)    // Using the library as a Win32 shared library (.dll)
00105     #endif
00106 #else
00107     #if defined(BUILD_LIBTYPE_SHARED)
00108         #define RLAPI __attribute__((visibility("default"))) // Building as a Unix shared library
00109     #endif
00110 #endif
00111
00112 #ifndef RLAPI
00113     #define RLAPI          // Functions defined as 'extern' by default (implicit specifiers)
00114 #endif
00115
00116 //-----
00117 // Defines and Macros
00118 //-----
00119 #ifndef PI
00120     #define PI 3.14159265358979323846f
00121 #endif
00122 #ifndef DEG2RAD
00123     #define DEG2RAD (PI/180.0f)
00124 #endif
00125 #ifndef RAD2DEG
00126     #define RAD2DEG (180.0f/PI)
00127 #endif
00128
00129 // Allow custom memory allocators
00130 // NOTE: Require recompiling raylib sources
00131 #ifndef RL_MALLOC
00132     #define RL_MALLOC(sz)      malloc(sz)
00133 #endif
00134 #ifndef RL_CALLOC
00135     #define RL_CALLOC(n,sz)    calloc(n,sz)
00136 #endif

```

```

00137 #ifndef RL_REALLOC
00138     #define RL_REALLOC(ptr,sz)   realloc(ptr,sz)
00139 #endif
00140 #ifndef RL_FREE
00141     #define RL_FREE(ptr)         free(ptr)
00142 #endif
00143
00144 // NOTE: MSVC C++ compiler does not support compound literals (C99 feature)
00145 // Plain structures in C++ (without constructors) can be initialized with { }
00146 // This is called aggregate initialization (C++11 feature)
00147 #if defined(__cplusplus)
00148     #define CLITERAL(type)      type
00149 #else
00150     #define CLITERAL(type)      (type)
00151 #endif
00152
00153 // Some compilers (mostly macos clang) default to C++98,
00154 // where aggregate initialization can't be used
00155 // So, give a more clear error stating how to fix this
00156 #if !defined(_MSC_VER) && (defined(__cplusplus) && __cplusplus < 201103L)
00157     #error "C++11 or later is required. Add -std=c++11"
00158 #endif
00159
00160 // NOTE: Set some defines with some data types declared by raylib
00161 // Other modules (raymath, rlgl) also require some of those types, so,
00162 // to be able to use those other modules as standalone (not depending on raylib)
00163 // this defines are useful for internal check and avoid type (re)definitions
00164 #define RL_COLOR_TYPE
00165 #define RL_RECTANGLE_TYPE
00166 #define RL_VECTOR2_TYPE
00167 #define RL_VECTOR3_TYPE
00168 #define RL_VECTOR4_TYPE
00169 #define RL_QUATERNION_TYPE
00170 #define RL_MATRIX_TYPE
00171
00172 // Some Basic Colors
00173 // NOTE: Custom raylib color palette for amazing visuals on WHITE background
00174 #define LIGHTGRAY CLITERAL(Color){ 200, 200, 200, 255 } // Light Gray
00175 #define GRAY CLITERAL(Color){ 130, 130, 130, 255 } // Gray
00176 #define DARKGRAY CLITERAL(Color){ 80, 80, 80, 255 } // Dark Gray
00177 #define YELLOW CLITERAL(Color){ 253, 249, 0, 255 } // Yellow
00178 #define GOLD CLITERAL(Color){ 255, 203, 0, 255 } // Gold
00179 #define ORANGE CLITERAL(Color){ 255, 161, 0, 255 } // Orange
00180 #define PINK CLITERAL(Color){ 255, 109, 194, 255 } // Pink
00181 #define RED CLITERAL(Color){ 230, 41, 55, 255 } // Red
00182 #define MAROON CLITERAL(Color){ 190, 33, 55, 255 } // Maroon
00183 #define GREEN CLITERAL(Color){ 0, 228, 48, 255 } // Green
00184 #define LIME CLITERAL(Color){ 0, 158, 47, 255 } // Lime
00185 #define DARKGREEN CLITERAL(Color){ 0, 117, 44, 255 } // Dark Green
00186 #define SKYBLUE CLITERAL(Color){ 102, 191, 255, 255 } // Sky Blue
00187 #define BLUE CLITERAL(Color){ 0, 121, 241, 255 } // Blue
00188 #define DARKBLUE CLITERAL(Color){ 0, 82, 172, 255 } // Dark Blue
00189 #define PURPLE CLITERAL(Color){ 200, 122, 255, 255 } // Purple
00190 #define VIOLET CLITERAL(Color){ 135, 60, 190, 255 } // Violet
00191 #define DARKPURPLE CLITERAL(Color){ 112, 31, 126, 255 } // Dark Purple
00192 #define BEIGE CLITERAL(Color){ 211, 176, 131, 255 } // Beige
00193 #define BROWN CLITERAL(Color){ 127, 106, 79, 255 } // Brown
00194 #define DARKBROWN CLITERAL(Color){ 76, 63, 47, 255 } // Dark Brown
00195
00196 #define WHITE CLITERAL(Color){ 255, 255, 255, 255 } // White
00197 #define BLACK CLITERAL(Color){ 0, 0, 0, 255 } // Black
00198 #define BLANK CLITERAL(Color){ 0, 0, 0, 0 } // Blank (Transparent)
00199 #define MAGENTA CLITERAL(Color){ 255, 0, 255, 255 } // Magenta
00200 #define RAYWHITE CLITERAL(Color){ 245, 245, 245, 255 } // My own White (raylib logo)
00201
00202 //-----
00203 // Types and Structures Definition
00204 //-----
00205 // Boolean type
00206 #if (defined(__STDC__) && __STDC_VERSION__ >= 199901L) || (defined(_MSC_VER) && _MSC_VER >= 1800)
00207     #include <stdbool.h>
00208 #elif !defined(__cplusplus) && !defined(bool)
00209     typedef enum bool { false = 0, true = !false } bool;
00210     #define RL_BOOL_TYPE
00211 #endif
00212
00213 // Vector2, 2 components
00214 typedef struct Vector2 {
00215     float x; // Vector x component
00216     float y; // Vector y component
00217 } Vector2;
00218
00219 // Vector3, 3 components
00220 typedef struct Vector3 {
00221     float x; // Vector x component
00222     float y; // Vector y component
00223     float z; // Vector z component

```

```

00224 } Vector3;
00225
00226 // Vector4, 4 components
00227 typedef struct Vector4 {
00228     float x;           // Vector x component
00229     float y;           // Vector y component
00230     float z;           // Vector z component
00231     float w;           // Vector w component
00232 } Vector4;
00233
00234 // Quaternion, 4 components (Vector4 alias)
00235 typedef Vector4 Quaternion;
00236
00237 // Matrix, 4x4 components, column major, OpenGL style, right-handed
00238 typedef struct Matrix {
00239     float m0, m4, m8, m12; // Matrix first row (4 components)
00240     float m1, m5, m9, m13; // Matrix second row (4 components)
00241     float m2, m6, m10, m14; // Matrix third row (4 components)
00242     float m3, m7, m11, m15; // Matrix fourth row (4 components)
00243 } Matrix;
00244
00245 // Color, 4 components, R8G8B8A8 (32bit)
00246 typedef struct Color {
00247     unsigned char r;      // Color red value
00248     unsigned char g;      // Color green value
00249     unsigned char b;      // Color blue value
00250     unsigned char a;      // Color alpha value
00251 } Color;
00252
00253 // Rectangle, 4 components
00254 typedef struct Rectangle {
00255     float x;              // Rectangle top-left corner position x
00256     float y;              // Rectangle top-left corner position y
00257     float width;          // Rectangle width
00258     float height;         // Rectangle height
00259 } Rectangle;
00260
00261 // Image, pixel data stored in CPU memory (RAM)
00262 typedef struct Image {
00263     void *data;           // Image raw data
00264     int width;            // Image base width
00265     int height;           // Image base height
00266     int mipmaps;          // Mipmap levels, 1 by default
00267     int format;           // Data format (PixelFormat type)
00268 } Image;
00269
00270 // Texture, tex data stored in GPU memory (VRAM)
00271 typedef struct Texture {
00272     unsigned int id;       // OpenGL texture id
00273     int width;            // Texture base width
00274     int height;           // Texture base height
00275     int mipmaps;          // Mipmap levels, 1 by default
00276     int format;           // Data format (PixelFormat type)
00277 } Texture;
00278
00279 // Texture2D, same as Texture
00280 typedef Texture Texture2D;
00281
00282 // TextureCubemap, same as Texture
00283 typedef Texture TextureCubemap;
00284
00285 // RenderTexture, fbo for texture rendering
00286 typedef struct RenderTexture {
00287     unsigned int id;       // OpenGL framebuffer object id
00288     Texture texture;       // Color buffer attachment texture
00289     Texture depth;        // Depth buffer attachment texture
00290 } RenderTexture;
00291
00292 // RenderTexture2D, same as RenderTexture
00293 typedef RenderTexture RenderTexture2D;
00294
00295 // NPatchInfo, n-patch layout info
00296 typedef struct NPatchInfo {
00297     Rectangle source;      // Texture source rectangle
00298     int left;              // Left border offset
00299     int top;               // Top border offset
00300     int right;             // Right border offset
00301     int bottom;            // Bottom border offset
00302     int layout;            // Layout of the n-patch: 3x3, 1x3 or 3x1
00303 } NPatchInfo;
00304
00305 // GlyphInfo, font characters glyphs info
00306 typedef struct GlyphInfo {
00307     int value;             // Character value (Unicode)
00308     int offsetX;           // Character offset X when drawing
00309     int offsetY;           // Character offset Y when drawing
00310     int advanceX;          // Character advance position X

```



```

00311     Image image;           // Character image data
00312 } GlyphInfo;
00313
00314 // Font, font texture and GlyphInfo array data
00315 typedef struct Font {
00316     int baseSize;           // Base size (default chars height)
00317     int glyphCount;         // Number of glyph characters
00318     int glyphPadding;       // Padding around the glyph characters
00319     Texture2D texture;      // Texture atlas containing the glyphs
00320     Rectangle *recs;        // Rectangles in texture for the glyphs
00321     GlyphInfo *glyphs;      // Glyphs info data
00322 } Font;
00323
00324 // Camera, defines position/orientation in 3d space
00325 typedef struct Camera3D {
00326     Vector3 position;       // Camera position
00327     Vector3 target;         // Camera target it looks-at
00328     Vector3 up;             // Camera up vector (rotation over its axis)
00329     float fovy;             // Camera field-of-view aperture in Y (degrees) in perspective, used as
    near plane height in world units in orthographic
00330     int projection;         // Camera projection: CAMERA_PERSPECTIVE or CAMERA_ORTHOGRAPHIC
00331 } Camera3D;
00332
00333 typedef Camera3D Camera;    // Camera type fallback, defaults to Camera3D
00334
00335 // Camera2D, defines position/orientation in 2d space
00336 typedef struct Camera2D {
00337     Vector2 offset;         // Camera offset (screen space offset from window origin)
00338     Vector2 target;         // Camera target (world space target point that is mapped to screen space
    offset)
00339     float rotation;         // Camera rotation in degrees (pivots around target)
00340     float zoom;             // Camera zoom (scaling around target), must not be set to 0, set to 1.0f
    for no scale
00341 } Camera2D;
00342
00343 // Mesh, vertex data and vao/vbo
00344 typedef struct Mesh {
00345     int vertexCount;        // Number of vertices stored in arrays
00346     int triangleCount;      // Number of triangles stored (indexed or not)
00347
00348     // Vertex attributes data
00349     float *vertices;         // Vertex position (XYZ - 3 components per vertex) (shader-location = 0)
00350     float *texcoords;        // Vertex texture coordinates (UV - 2 components per vertex)
    (shader-location = 1)
00351     float *texcoords2;       // Vertex texture second coordinates (UV - 2 components per vertex)
    (shader-location = 5)
00352     float *normals;          // Vertex normals (XYZ - 3 components per vertex) (shader-location = 2)
00353     float *tangents;         // Vertex tangents (XYZW - 4 components per vertex) (shader-location = 4)
00354     unsigned char *colors;   // Vertex colors (RGBA - 4 components per vertex) (shader-location = 3)
00355     unsigned short *indices; // Vertex indices (in case vertex data comes indexed)
00356
00357     // Skin data for animation
00358     int boneCount;           // Number of bones (MAX: 256 bones)
00359     unsigned char *boneIndices; // Vertex bone indices, up to 4 bones influence by vertex (skinning)
    (shader-location = 6)
00360     float *boneWeights;      // Vertex bone weight, up to 4 bones influence by vertex (skinning)
    (shader-location = 7)
00361
00362     // Runtime animation vertex data (CPU skinning)
00363     // NOTE: In case of GPU skinning, not used, pointers are NULL
00364     float *animVertices;     // Animated vertex positions (after bones transformations)
00365     float *animNormals;      // Animated normals (after bones transformations)
00366
00367     // OpenGL identifiers
00368     unsigned int vaoId;      // OpenGL Vertex Array Object id
00369     unsigned int *vboids;    // OpenGL Vertex Buffer Objects id (default vertex data)
00370 } Mesh;
00371
00372 // Shader
00373 typedef struct Shader {
00374     unsigned int id;         // Shader program id
00375     int *locs;              // Shader locations array (RL_MAX_SHADER_LOCATIONS)
00376 } Shader;
00377
00378 // MaterialMap
00379 typedef struct MaterialMap {
00380     Texture2D texture;       // Material map texture
00381     Color color;             // Material map color
00382     float value;             // Material map value
00383 } MaterialMap;
00384
00385 // Material, includes shader and maps
00386 typedef struct Material {
00387     Shader shader;           // Material shader
00388     MaterialMap *maps;       // Material maps array (MAX_MATERIAL_MAPS)
00389     float params[4];         // Material generic parameters (if required)
00390 } Material;

```

```

00391
00392 // Transform, vertex transformation data
00393 typedef struct Transform {
00394     Vector3 translation; // Translation
00395     Quaternion rotation; // Rotation
00396     Vector3 scale; // Scale
00397 } Transform;
00398
00399 // Anim pose, an array of Transform[]
00400 typedef Transform *ModelAnimPose;
00401
00402 // Bone, skeletal animation bone
00403 typedef struct BoneInfo {
00404     char name[32]; // Bone name
00405     int parent; // Bone parent
00406 } BoneInfo;
00407
00408 // Skeleton, animation bones hierarchy
00409 typedef struct ModelSkeleton {
00410     int boneCount; // Number of bones
00411     BoneInfo *bones; // Bones information (skeleton)
00412     ModelAnimPose bindPose; // Bones base transformation (Transform[])
00413 } ModelSkeleton;
00414
00415 // Model, meshes, materials and animation data
00416 typedef struct Model {
00417     Matrix transform; // Local transform matrix
00418
00419     int meshCount; // Number of meshes
00420     int materialCount; // Number of materials
00421     Mesh *meshes; // Meshes array
00422     Material *materials; // Materials array
00423     int *meshMaterial; // Mesh material number
00424
00425     // Animation data
00426     ModelSkeleton skeleton; // Skeleton for animation
00427
00428     // Runtime animation data (CPU/GPU skinning)
00429     ModelAnimPose currentPose; // Current animation pose (Transform[])
00430     Matrix *boneMatrices; // Bones animated transformation matrices
00431 } Model;
00432
00433 // ModelAnimation, contains a full animation sequence
00434 typedef struct ModelAnimation {
00435     char name[32]; // Animation name
00436
00437     int boneCount; // Number of bones (per pose)
00438     int keyframeCount; // Number of animation key frames
00439     ModelAnimPose *keyframePoses; // Animation sequence keyframe poses [keyframe][pose]
00440 } ModelAnimation;
00441
00442 // Ray, ray for raycasting
00443 typedef struct Ray {
00444     Vector3 position; // Ray position (origin)
00445     Vector3 direction; // Ray direction (normalized)
00446 } Ray;
00447
00448 // RayCollision, ray hit information
00449 typedef struct RayCollision {
00450     bool hit; // Did the ray hit something?
00451     float distance; // Distance to the nearest hit
00452     Vector3 point; // Point of the nearest hit
00453     Vector3 normal; // Surface normal of hit
00454 } RayCollision;
00455
00456 // BoundingBox
00457 typedef struct BoundingBox {
00458     Vector3 min; // Minimum vertex box-corner
00459     Vector3 max; // Maximum vertex box-corner
00460 } BoundingBox;
00461
00462 // Wave, audio wave data
00463 typedef struct Wave {
00464     unsigned int frameCount; // Total number of frames (considering channels)
00465     unsigned int sampleRate; // Frequency (samples per second)
00466     unsigned int sampleSize; // Bit depth (bits per sample): 8, 16, 32 (24 not supported)
00467     unsigned int channels; // Number of channels (1-mono, 2-stereo, ...)
00468     void *data; // Buffer data pointer
00469 } Wave;
00470
00471 // Opaque structs declaration
00472 // NOTE: Actual structs are defined internally in raudio module
00473 typedef struct rAudioBuffer rAudioBuffer;
00474 typedef struct rAudioProcessor rAudioProcessor;
00475
00476 // AudioStream, custom audio stream
00477 typedef struct AudioStream {

```

```

00478     rAudioBuffer *buffer;           // Pointer to internal data used by the audio system
00479     rAudioProcessor *processor;     // Pointer to internal data processor, useful for audio effects
00480
00481     unsigned int sampleRate;        // Frequency (samples per second)
00482     unsigned int sampleSize;        // Bit depth (bits per sample): 8, 16, 32 (24 not supported)
00483     unsigned int channels;          // Number of channels (1-mono, 2-stereo, ...)
00484 } AudioStream;
00485
00486 // Sound
00487 typedef struct Sound {
00488     AudioStream stream;             // Audio stream
00489     unsigned int frameCount;        // Total number of frames (considering channels)
00490 } Sound;
00491
00492 // Music, audio stream, anything longer than ~10 seconds should be streamed
00493 typedef struct Music {
00494     AudioStream stream;             // Audio stream
00495     unsigned int frameCount;        // Total number of frames (considering channels)
00496     bool looping;                  // Music looping enable
00497
00498     int ctxType;                   // Type of music context (audio filetype)
00499     void *ctxData;                 // Audio context data, depends on type
00500 } Music;
00501
00502 // VrDeviceInfo, Head-Mounted-Display device parameters
00503 typedef struct VrDeviceInfo {
00504     int hResolution;               // Horizontal resolution in pixels
00505     int vResolution;               // Vertical resolution in pixels
00506     float hScreenSize;             // Horizontal size in meters
00507     float vScreenSize;             // Vertical size in meters
00508     float eyeToScreenDistance;     // Distance between eye and display in meters
00509     float lensSeparationDistance;  // Lens separation distance in meters
00510     float interpupillaryDistance;  // IPD (distance between pupils) in meters
00511     float lensDistortionValues[4]; // Lens distortion constant parameters
00512     float chromaAbCorrection[4];   // Chromatic aberration correction parameters
00513 } VrDeviceInfo;
00514
00515 // VrStereoConfig, VR stereo rendering configuration for simulator
00516 typedef struct VrStereoConfig {
00517     Matrix projection[2];          // VR projection matrices (per eye)
00518     Matrix viewOffset[2];          // VR view offset matrices (per eye)
00519     float leftLensCenter[2];       // VR left lens center
00520     float rightLensCenter[2];      // VR right lens center
00521     float leftScreenCenter[2];     // VR left screen center
00522     float rightScreenCenter[2];    // VR right screen center
00523     float scale[2];               // VR distortion scale
00524     float scaleIn[2];             // VR distortion scale in
00525 } VrStereoConfig;
00526
00527 // File path list
00528 typedef struct FilePathList {
00529     unsigned int count;            // Filepaths entries count
00530     char **paths;                 // Filepaths entries
00531 } FilePathList;
00532
00533 // Automation event
00534 typedef struct AutomationEvent {
00535     unsigned int frame;            // Event frame
00536     unsigned int type;             // Event type (AutomationEventType)
00537     int params[4];                // Event parameters (if required)
00538 } AutomationEvent;
00539
00540 // Automation event list
00541 typedef struct AutomationEventList {
00542     unsigned int capacity;         // Events max entries (MAX_AUTOMATION_EVENTS)
00543     unsigned int count;           // Events entries count
00544     AutomationEvent *events;      // Events entries
00545 } AutomationEventList;
00546
00547 //-----
00548 // Enumerators Definition
00549 //-----
00550 // System/Window config flags
00551 // NOTE: Every bit registers one state (use it with bit masks)
00552 // By default all flags are set to 0
00553 typedef enum {
00554     FLAG_VSYNC_HINT = 0x00000040, // Set to try enabling V-Sync on GPU
00555     FLAG_FULLSCREEN_MODE = 0x00000002, // Set to run program in fullscreen
00556     FLAG_WINDOW_RESIZABLE = 0x00000004, // Set to allow resizable window
00557     FLAG_WINDOW_UNDECORATED = 0x00000008, // Set to disable window decoration (frame and buttons)
00558     FLAG_WINDOW_HIDDEN = 0x00000080, // Set to hide window
00559     FLAG_WINDOW_MINIMIZED = 0x00000200, // Set to minimize window (iconify)
00560     FLAG_WINDOW_MAXIMIZED = 0x00000400, // Set to maximize window (expanded to monitor)
00561     FLAG_WINDOW_UNFOCUSED = 0x00000800, // Set to window non focused
00562     FLAG_WINDOW_TOPMOST = 0x00001000, // Set to window always on top
00563     FLAG_WINDOW_ALWAYS_RUN = 0x00000100, // Set to allow windows running while minimized
00564     FLAG_WINDOW_TRANSPARENT = 0x00000010, // Set to allow transparent framebuffer

```

```

00565     FLAG_WINDOW_HIGHDPI      = 0x00002000,    // Set to support HighDPI
00566     FLAG_WINDOW_MOUSE_PASSTHROUGH = 0x00004000, // Set to support mouse passthrough, only supported
when FLAG_WINDOW_UNDECORATED
00567     FLAG_BORDERLESS_WINDOWED_MODE = 0x00008000, // Set to run program in borderless windowed mode
00568     FLAG_MSAA_4X_HINT          = 0x00000020,    // Set to try enabling MSAA 4X
00569     FLAG_INTERLACED_HINT      = 0x00010000      // Set to try enabling interlaced video format (for V3D)
00570 } ConfigFlags;
00571
00572 // Trace log level
00573 // NOTE: Organized by priority level
00574 typedef enum {
00575     LOG_ALL = 0,           // Display all logs
00576     LOG_TRACE,           // Trace logging, intended for internal use only
00577     LOG_DEBUG,           // Debug logging, used for internal debugging, it should be disabled on
release builds
00578     LOG_INFO,            // Info logging, used for program execution info
00579     LOG_WARNING,         // Warning logging, used on recoverable failures
00580     LOG_ERROR,           // Error logging, used on unrecoverable failures
00581     LOG_FATAL,           // Fatal logging, used to abort program: exit(EXIT_FAILURE)
00582     LOG_NONE             // Disable logging
00583 } TraceLogLevel;
00584
00585 // Keyboard keys (US keyboard layout)
00586 // NOTE: Use GetKeyPressed() to allow redefining required keys for alternative layouts
00587 typedef enum {
00588     KEY_NULL = 0,         // Key: NULL, used for no key pressed
00589     // Alphanumeric keys
00590     KEY_APOSTROPHE = 39,  // Key: '
00591     KEY_COMMA = 44,       // Key: ,
00592     KEY_MINUS = 45,       // Key: -
00593     KEY_PERIOD = 46,      // Key: .
00594     KEY_SLASH = 47,       // Key: /
00595     KEY_ZERO = 48,        // Key: 0
00596     KEY_ONE = 49,         // Key: 1
00597     KEY_TWO = 50,         // Key: 2
00598     KEY_THREE = 51,       // Key: 3
00599     KEY_FOUR = 52,        // Key: 4
00600     KEY_FIVE = 53,        // Key: 5
00601     KEY_SIX = 54,         // Key: 6
00602     KEY_SEVEN = 55,       // Key: 7
00603     KEY_EIGHT = 56,       // Key: 8
00604     KEY_NINE = 57,        // Key: 9
00605     KEY_SEMICOLON = 59,   // Key: ;
00606     KEY_EQUAL = 61,       // Key: =
00607     KEY_A = 65,           // Key: A | a
00608     KEY_B = 66,           // Key: B | b
00609     KEY_C = 67,           // Key: C | c
00610     KEY_D = 68,           // Key: D | d
00611     KEY_E = 69,           // Key: E | e
00612     KEY_F = 70,           // Key: F | f
00613     KEY_G = 71,           // Key: G | g
00614     KEY_H = 72,           // Key: H | h
00615     KEY_I = 73,           // Key: I | i
00616     KEY_J = 74,           // Key: J | j
00617     KEY_K = 75,           // Key: K | k
00618     KEY_L = 76,           // Key: L | l
00619     KEY_M = 77,           // Key: M | m
00620     KEY_N = 78,           // Key: N | n
00621     KEY_O = 79,           // Key: O | o
00622     KEY_P = 80,           // Key: P | p
00623     KEY_Q = 81,           // Key: Q | q
00624     KEY_R = 82,           // Key: R | r
00625     KEY_S = 83,           // Key: S | s
00626     KEY_T = 84,           // Key: T | t
00627     KEY_U = 85,           // Key: U | u
00628     KEY_V = 86,           // Key: V | v
00629     KEY_W = 87,           // Key: W | w
00630     KEY_X = 88,           // Key: X | x
00631     KEY_Y = 89,           // Key: Y | y
00632     KEY_Z = 90,           // Key: Z | z
00633     KEY_LEFT_BRACKET = 91, // Key: [
00634     KEY_BACKSLASH = 92,    // Key: '\
00635     KEY_RIGHT_BRACKET = 93, // Key: ]
00636     KEY_GRAVE = 96,        // Key: `
00637     // Function keys
00638     KEY_SPACE = 32,        // Key: Space
00639     KEY_ESCAPE = 256,      // Key: Esc
00640     KEY_ENTER = 257,       // Key: Enter
00641     KEY_TAB = 258,         // Key: Tab
00642     KEY_BACKSPACE = 259,   // Key: Backspace
00643     KEY_INSERT = 260,      // Key: Ins
00644     KEY_DELETE = 261,      // Key: Del
00645     KEY_RIGHT = 262,       // Key: Cursor right
00646     KEY_LEFT = 263,        // Key: Cursor left
00647     KEY_DOWN = 264,        // Key: Cursor down
00648     KEY_UP = 265,          // Key: Cursor up
00649     KEY_PAGE_UP = 266,     // Key: Page up

```

```

00650     KEY_PAGE_DOWN      = 267,      // Key: Page down
00651     KEY_HOME           = 268,      // Key: Home
00652     KEY_END            = 269,      // Key: End
00653     KEY_CAPS_LOCK      = 280,      // Key: Caps lock
00654     KEY_SCROLL_LOCK    = 281,      // Key: Scroll down
00655     KEY_NUM_LOCK       = 282,      // Key: Num lock
00656     KEY_PRINT_SCREEN   = 283,      // Key: Print screen
00657     KEY_PAUSE          = 284,      // Key: Pause
00658     KEY_F1             = 290,      // Key: F1
00659     KEY_F2             = 291,      // Key: F2
00660     KEY_F3             = 292,      // Key: F3
00661     KEY_F4             = 293,      // Key: F4
00662     KEY_F5             = 294,      // Key: F5
00663     KEY_F6             = 295,      // Key: F6
00664     KEY_F7             = 296,      // Key: F7
00665     KEY_F8             = 297,      // Key: F8
00666     KEY_F9             = 298,      // Key: F9
00667     KEY_F10            = 299,      // Key: F10
00668     KEY_F11            = 300,      // Key: F11
00669     KEY_F12            = 301,      // Key: F12
00670     KEY_LEFT_SHIFT     = 340,      // Key: Shift left
00671     KEY_LEFT_CONTROL   = 341,      // Key: Control left
00672     KEY_LEFT_ALT       = 342,      // Key: Alt left
00673     KEY_LEFT_SUPER     = 343,      // Key: Super left
00674     KEY_RIGHT_SHIFT    = 344,      // Key: Shift right
00675     KEY_RIGHT_CONTROL  = 345,      // Key: Control right
00676     KEY_RIGHT_ALT      = 346,      // Key: Alt right
00677     KEY_RIGHT_SUPER    = 347,      // Key: Super right
00678     KEY_KB_MENU        = 348,      // Key: KB menu
00679     // Keypad keys
00680     KEY_KP_0           = 320,      // Key: Keypad 0
00681     KEY_KP_1           = 321,      // Key: Keypad 1
00682     KEY_KP_2           = 322,      // Key: Keypad 2
00683     KEY_KP_3           = 323,      // Key: Keypad 3
00684     KEY_KP_4           = 324,      // Key: Keypad 4
00685     KEY_KP_5           = 325,      // Key: Keypad 5
00686     KEY_KP_6           = 326,      // Key: Keypad 6
00687     KEY_KP_7           = 327,      // Key: Keypad 7
00688     KEY_KP_8           = 328,      // Key: Keypad 8
00689     KEY_KP_9           = 329,      // Key: Keypad 9
00690     KEY_KP_DECIMAL     = 330,      // Key: Keypad .
00691     KEY_KP_DIVIDE      = 331,      // Key: Keypad /
00692     KEY_KP_MULTIPLY    = 332,      // Key: Keypad *
00693     KEY_KP_SUBTRACT    = 333,      // Key: Keypad -
00694     KEY_KP_ADD         = 334,      // Key: Keypad +
00695     KEY_KP_ENTER       = 335,      // Key: Keypad Enter
00696     KEY_KP_EQUAL       = 336,      // Key: Keypad =
00697     // Android key buttons
00698     KEY_BACK           = 4,        // Key: Android back button
00699     KEY_MENU           = 5,        // Key: Android menu button
00700     KEY_VOLUME_UP      = 24,      // Key: Android volume up button
00701     KEY_VOLUME_DOWN    = 25,      // Key: Android volume down button
00702 } KeyboardKey;
00703
00704 // Add backwards compatibility support for deprecated names
00705 #define MOUSE_LEFT_BUTTON  MOUSE_BUTTON_LEFT
00706 #define MOUSE_RIGHT_BUTTON MOUSE_BUTTON_RIGHT
00707 #define MOUSE_MIDDLE_BUTTON MOUSE_BUTTON_MIDDLE
00708
00709 // Mouse buttons
00710 typedef enum {
00711     MOUSE_BUTTON_LEFT   = 0,      // Mouse button left
00712     MOUSE_BUTTON_RIGHT  = 1,      // Mouse button right
00713     MOUSE_BUTTON_MIDDLE = 2,      // Mouse button middle (pressed wheel)
00714     MOUSE_BUTTON_SIDE   = 3,      // Mouse button side (advanced mouse device)
00715     MOUSE_BUTTON_EXTRA  = 4,      // Mouse button extra (advanced mouse device)
00716     MOUSE_BUTTON_FORWARD = 5,     // Mouse button forward (advanced mouse device)
00717     MOUSE_BUTTON_BACK   = 6,      // Mouse button back (advanced mouse device)
00718 } MouseButton;
00719
00720 // Mouse cursor
00721 typedef enum {
00722     MOUSE_CURSOR_DEFAULT = 0,     // Default pointer shape
00723     MOUSE_CURSOR_ARROW   = 1,     // Arrow shape
00724     MOUSE_CURSOR_IBEAM   = 2,     // Text writing cursor shape
00725     MOUSE_CURSOR_CROSSHAIR = 3,   // Cross shape
00726     MOUSE_CURSOR_POINTING_HAND = 4, // Pointing hand cursor
00727     MOUSE_CURSOR_RESIZE_EW = 5,    // Horizontal resize/move arrow shape
00728     MOUSE_CURSOR_RESIZE_NS = 6,    // Vertical resize/move arrow shape
00729     MOUSE_CURSOR_RESIZE_NWSE = 7,  // Top-left to bottom-right diagonal resize/move arrow shape
00730     MOUSE_CURSOR_RESIZE_NESW = 8,  // The top-right to bottom-left diagonal resize/move arrow
00731     shape MOUSE_CURSOR_RESIZE_ALL = 9, // The omnidirectional resize/move cursor shape
00732     MOUSE_CURSOR_NOT_ALLOWED = 10 // The operation-not-allowed shape
00733 } MouseCursor;
00734
00735 // Gamepad buttons

```

```

00736 typedef enum {
00737     GAMEPAD_BUTTON_UNKNOWN = 0,           // Unknown button, for error checking
00738     GAMEPAD_BUTTON_LEFT_FACE_UP,         // Gamepad left DPAD up button
00739     GAMEPAD_BUTTON_LEFT_FACE_RIGHT,      // Gamepad left DPAD right button
00740     GAMEPAD_BUTTON_LEFT_FACE_DOWN,      // Gamepad left DPAD down button
00741     GAMEPAD_BUTTON_LEFT_FACE_LEFT,      // Gamepad left DPAD left button
00742     GAMEPAD_BUTTON_RIGHT_FACE_UP,        // Gamepad right button up (i.e. PS3: Triangle, Xbox: Y)
00743     GAMEPAD_BUTTON_RIGHT_FACE_RIGHT,     // Gamepad right button right (i.e. PS3: Circle, Xbox: B)
00744     GAMEPAD_BUTTON_RIGHT_FACE_DOWN,     // Gamepad right button down (i.e. PS3: Cross, Xbox: A)
00745     GAMEPAD_BUTTON_RIGHT_FACE_LEFT,     // Gamepad right button left (i.e. PS3: Square, Xbox: X)
00746     GAMEPAD_BUTTON_LEFT_TRIGGER_1,      // Gamepad top/back trigger left (first), it could be a
trailing button
00747     GAMEPAD_BUTTON_LEFT_TRIGGER_2,      // Gamepad top/back trigger left (second), it could be a
trailing button
00748     GAMEPAD_BUTTON_RIGHT_TRIGGER_1,     // Gamepad top/back trigger right (first), it could be a
trailing button
00749     GAMEPAD_BUTTON_RIGHT_TRIGGER_2,     // Gamepad top/back trigger right (second), it could be a
trailing button
00750     GAMEPAD_BUTTON_MIDDLE_LEFT,         // Gamepad center buttons, left one (i.e. PS3: Select)
00751     GAMEPAD_BUTTON_MIDDLE,             // Gamepad center buttons, middle one (i.e. PS3: PS, Xbox:
XBOX)
00752     GAMEPAD_BUTTON_MIDDLE_RIGHT,        // Gamepad center buttons, right one (i.e. PS3: Start)
00753     GAMEPAD_BUTTON_LEFT_THUMB,          // Gamepad joystick pressed button left
00754     GAMEPAD_BUTTON_RIGHT_THUMB,         // Gamepad joystick pressed button right
00755 } GamepadButton;
00756
00757 // Gamepad axes
00758 typedef enum {
00759     GAMEPAD_AXIS_LEFT_X = 0,           // Gamepad left stick X axis
00760     GAMEPAD_AXIS_LEFT_Y = 1,           // Gamepad left stick Y axis
00761     GAMEPAD_AXIS_RIGHT_X = 2,          // Gamepad right stick X axis
00762     GAMEPAD_AXIS_RIGHT_Y = 3,          // Gamepad right stick Y axis
00763     GAMEPAD_AXIS_LEFT_TRIGGER = 4,     // Gamepad back trigger left, pressure level: [1..-1]
00764     GAMEPAD_AXIS_RIGHT_TRIGGER = 5,    // Gamepad back trigger right, pressure level: [1..-1]
00765 } GamepadAxis;
00766
00767 // Material map index
00768 typedef enum {
00769     MATERIAL_MAP_ALBEDO = 0,           // Albedo material (same as: MATERIAL_MAP_DIFFUSE)
00770     MATERIAL_MAP_METALNESS,           // Metalness material (same as: MATERIAL_MAP_SPECULAR)
00771     MATERIAL_MAP_NORMAL,              // Normal material
00772     MATERIAL_MAP_ROUGHNESS,           // Roughness material
00773     MATERIAL_MAP_OCCLUSION,           // Ambient occlusion material
00774     MATERIAL_MAP_EMISSION,            // Emission material
00775     MATERIAL_MAP_HEIGHT,              // Heightmap material
00776     MATERIAL_MAP_CUBEMAP,             // Cubemap material (NOTE: Uses GL_TEXTURE_CUBE_MAP)
00777     MATERIAL_MAP_IRRADIANCE,           // Irradiance material (NOTE: Uses GL_TEXTURE_CUBE_MAP)
00778     MATERIAL_MAP_PREFILTER,           // Prefilter material (NOTE: Uses GL_TEXTURE_CUBE_MAP)
00779     MATERIAL_MAP_BRDF,                // Brdf material
00780 } MaterialMapIndex;
00781
00782 #define MATERIAL_MAP_DIFFUSE MATERIAL_MAP_ALBEDO
00783 #define MATERIAL_MAP_SPECULAR MATERIAL_MAP_METALNESS
00784
00785 // Shader location index
00786 // NOTE: Some locations are tried to be set automatically on shader loading,
00787 // but only if default attributes/uniforms names are found, check config.h for names
00788 typedef enum {
00789     SHADER_LOC_VERTEX_POSITION = 0,    // Shader location: vertex attribute: position
00790     SHADER_LOC_VERTEX_TEXCOORD01,     // Shader location: vertex attribute: texcoord01
00791     SHADER_LOC_VERTEX_TEXCOORD02,     // Shader location: vertex attribute: texcoord02
00792     SHADER_LOC_VERTEX_NORMAL,          // Shader location: vertex attribute: normal
00793     SHADER_LOC_VERTEX_TANGENT,          // Shader location: vertex attribute: tangent
00794     SHADER_LOC_VERTEX_COLOR,           // Shader location: vertex attribute: color
00795     SHADER_LOC_MATRIX_MVP,             // Shader location: matrix uniform: model-view-projection
00796     SHADER_LOC_MATRIX_VIEW,            // Shader location: matrix uniform: view (camera transform)
00797     SHADER_LOC_MATRIX_PROJECTION,      // Shader location: matrix uniform: projection
00798     SHADER_LOC_MATRIX_MODEL,           // Shader location: matrix uniform: model (transform)
00799     SHADER_LOC_MATRIX_NORMAL,          // Shader location: matrix uniform: normal
00800     SHADER_LOC_VECTOR_VIEW,            // Shader location: vector uniform: view
00801     SHADER_LOC_COLOR_DIFFUSE,          // Shader location: vector uniform: diffuse color
00802     SHADER_LOC_COLOR_SPECULAR,         // Shader location: vector uniform: specular color
00803     SHADER_LOC_COLOR_AMBIENT,          // Shader location: vector uniform: ambient color
00804     SHADER_LOC_MAP_ALBEDO,             // Shader location: sampler2d texture: albedo (same as:
SHADER_LOC_MAP_DIFFUSE)
00805     SHADER_LOC_MAP_METALNESS,          // Shader location: sampler2d texture: metalness (same as:
SHADER_LOC_MAP_SPECULAR)
00806     SHADER_LOC_MAP_NORMAL,             // Shader location: sampler2d texture: normal
00807     SHADER_LOC_MAP_ROUGHNESS,          // Shader location: sampler2d texture: roughness
00808     SHADER_LOC_MAP_OCCLUSION,          // Shader location: sampler2d texture: occlusion
00809     SHADER_LOC_MAP_EMISSION,           // Shader location: sampler2d texture: emission
00810     SHADER_LOC_MAP_HEIGHT,            // Shader location: sampler2d texture: heightmap
00811     SHADER_LOC_MAP_CUBEMAP,            // Shader location: samplerCube texture: cubemap
00812     SHADER_LOC_MAP_IRRADIANCE,         // Shader location: samplerCube texture: irradiance
00813     SHADER_LOC_MAP_PREFILTER,          // Shader location: samplerCube texture: prefilter
00814     SHADER_LOC_MAP_BRDF,              // Shader location: sampler2d texture: brdf
00815     SHADER_LOC_VERTEX_BONEIDS,         // Shader location: vertex attribute: bone indices

```

```

00816     SHADER_LOC_VERTEX_BONEWEIGHTS, // Shader location: vertex attribute: bone weights
00817     SHADER_LOC_MATRIX_BONETRANSFORMS, // Shader location: matrix attribute: bone transforms
    (animation)
00818     SHADER_LOC_VERTEX_INSTANCETRANSFORM // Shader location: vertex attribute: instance transforms
00819 } ShaderLocationIndex;
00820
00821 #define SHADER_LOC_MAP_DIFFUSE        SHADER_LOC_MAP_ALBEDO
00822 #define SHADER_LOC_MAP_SPECULAR      SHADER_LOC_MAP_METALNESS
00823
00824 // Shader uniform data type
00825 typedef enum {
00826     SHADER_UNIFORM_FLOAT = 0,          // Shader uniform type: float
00827     SHADER_UNIFORM_VEC2,              // Shader uniform type: vec2 (2 float)
00828     SHADER_UNIFORM_VEC3,              // Shader uniform type: vec3 (3 float)
00829     SHADER_UNIFORM_VEC4,              // Shader uniform type: vec4 (4 float)
00830     SHADER_UNIFORM_INT,               // Shader uniform type: int
00831     SHADER_UNIFORM_IVEC2,             // Shader uniform type: ivec2 (2 int)
00832     SHADER_UNIFORM_IVEC3,             // Shader uniform type: ivec3 (3 int)
00833     SHADER_UNIFORM_IVEC4,             // Shader uniform type: ivec4 (4 int)
00834     SHADER_UNIFORM_UINT,              // Shader uniform type: unsigned int
00835     SHADER_UNIFORM_UIVEC2,            // Shader uniform type: uivec2 (2 unsigned int)
00836     SHADER_UNIFORM_UIVEC3,            // Shader uniform type: uivec3 (3 unsigned int)
00837     SHADER_UNIFORM_UIVEC4,            // Shader uniform type: uivec4 (4 unsigned int)
00838     SHADER_UNIFORM_SAMPLER2D          // Shader uniform type: sampler2d
00839 } ShaderUniformDataType;
00840
00841 // Shader attribute data types
00842 typedef enum {
00843     SHADER_ATTRIB_FLOAT = 0,          // Shader attribute type: float
00844     SHADER_ATTRIB_VEC2,               // Shader attribute type: vec2 (2 float)
00845     SHADER_ATTRIB_VEC3,               // Shader attribute type: vec3 (3 float)
00846     SHADER_ATTRIB_VEC4                // Shader attribute type: vec4 (4 float)
00847 } ShaderAttributeDataType;
00848
00849 // Pixel formats
00850 // NOTE: Support depends on OpenGL version and platform
00851 typedef enum {
00852     PIXELFORMAT_UNCOMPRESSED_GRAYSCALE = 1, // 8 bit per pixel (no alpha)
00853     PIXELFORMAT_UNCOMPRESSED_GRAY_ALPHA,    // 8*2 bpp (2 channels)
00854     PIXELFORMAT_UNCOMPRESSED_R5G6B5,       // 16 bpp
00855     PIXELFORMAT_UNCOMPRESSED_R8G8B8,       // 24 bpp
00856     PIXELFORMAT_UNCOMPRESSED_R5G5B5A1,     // 16 bpp (1 bit alpha)
00857     PIXELFORMAT_UNCOMPRESSED_R4G4B4A4,     // 16 bpp (4 bit alpha)
00858     PIXELFORMAT_UNCOMPRESSED_R8G8B8A8,     // 32 bpp
00859     PIXELFORMAT_UNCOMPRESSED_R32,           // 32 bpp (1 channel - float)
00860     PIXELFORMAT_UNCOMPRESSED_R32G32B32,    // 32*3 bpp (3 channels - float)
00861     PIXELFORMAT_UNCOMPRESSED_R32G32B32A32, // 32*4 bpp (4 channels - float)
00862     PIXELFORMAT_UNCOMPRESSED_R16,           // 16 bpp (1 channel - half float)
00863     PIXELFORMAT_UNCOMPRESSED_R16G16B16,    // 16*3 bpp (3 channels - half float)
00864     PIXELFORMAT_UNCOMPRESSED_R16G16B16A16, // 16*4 bpp (4 channels - half float)
00865     PIXELFORMAT_COMPRESSED_DXT1_RGB,        // 4 bpp (no alpha)
00866     PIXELFORMAT_COMPRESSED_DXT1_RGBA,      // 4 bpp (1 bit alpha)
00867     PIXELFORMAT_COMPRESSED_DXT3_RGBA,      // 8 bpp
00868     PIXELFORMAT_COMPRESSED_DXT5_RGBA,      // 8 bpp
00869     PIXELFORMAT_COMPRESSED_ETC1_RGB,       // 4 bpp
00870     PIXELFORMAT_COMPRESSED_ETC2_RGB,       // 4 bpp
00871     PIXELFORMAT_COMPRESSED_ETC2_EAC_RGBA,  // 8 bpp
00872     PIXELFORMAT_COMPRESSED_PVRT_RGB,       // 4 bpp
00873     PIXELFORMAT_COMPRESSED_PVRT_RGBA,      // 4 bpp
00874     PIXELFORMAT_COMPRESSED_ASTC_4x4_RGBA,  // 8 bpp
00875     PIXELFORMAT_COMPRESSED_ASTC_8x8_RGBA   // 2 bpp
00876 } PixelFormat;
00877
00878 // Texture parameters: filter mode
00879 // NOTE 1: Filtering considers mipmaps if available in the texture
00880 // NOTE 2: Filter is accordingly set for minification and magnification
00881 typedef enum {
00882     TEXTURE_FILTER_POINT = 0,              // No filter, pixel approximation
00883     TEXTURE_FILTER_BILINEAR,               // Linear filtering
00884     TEXTURE_FILTER_TRILINEAR,              // Trilinear filtering (linear with mipmaps)
00885     TEXTURE_FILTER_ANISOTROPIC_4X,        // Anisotropic filtering 4x
00886     TEXTURE_FILTER_ANISOTROPIC_8X,        // Anisotropic filtering 8x
00887     TEXTURE_FILTER_ANISOTROPIC_16X,       // Anisotropic filtering 16x
00888 } TextureFilter;
00889
00890 // Texture parameters: wrap mode
00891 typedef enum {
00892     TEXTURE_WRAP_REPEAT = 0,               // Repeats texture in tiled mode
00893     TEXTURE_WRAP_CLAMP,                    // Clamps texture to edge pixel in tiled mode
00894     TEXTURE_WRAP_MIRROR_REPEAT,            // Mirrors and repeats the texture in tiled mode
00895     TEXTURE_WRAP_MIRROR_CLAMP              // Mirrors and clamps to border the texture in tiled mode
00896 } TextureWrap;
00897
00898 // Cubemap layouts
00899 typedef enum {
00900     CUBEMAP_LAYOUT_AUTO_DETECT = 0,        // Automatically detect layout type
00901     CUBEMAP_LAYOUT_LINE_VERTICAL,          // Layout is defined by a vertical line with faces

```



```

00902     CUBEMAP_LAYOUT_LINE_HORIZONTAL,          // Layout is defined by a horizontal line with faces
00903     CUBEMAP_LAYOUT_CROSS_THREE_BY_FOUR,      // Layout is defined by a 3x4 cross with cubemap faces
00904     CUBEMAP_LAYOUT_CROSS_FOUR_BY_THREE      // Layout is defined by a 4x3 cross with cubemap faces
00905 } CubemapLayout;
00906
00907 // Font type, defines generation method
00908 typedef enum {
00909     FONT_DEFAULT = 0,          // Default font generation, anti-aliased
00910     FONT_BITMAP,              // Bitmap font generation, no anti-aliasing
00911     FONT_SDF                   // SDF font generation, requires external shader
00912 } FontType;
00913
00914 // Color blending modes (pre-defined)
00915 typedef enum {
00916     BLEND_ALPHA = 0,          // Blend textures considering alpha (default)
00917     BLEND_ADDITIVE,           // Blend textures adding colors
00918     BLEND_MULTIPLIED,         // Blend textures multiplying colors
00919     BLEND_ADD_COLORS,          // Blend textures adding colors (alternative)
00920     BLEND_SUBTRACT_COLORS,     // Blend textures subtracting colors (alternative)
00921     BLEND_ALPHA_PREMULTIPLY,   // Blend premultiplied textures considering alpha
00922     BLEND_CUSTOM,              // Blend textures using custom src/dst factors (use
                                // rlSetBlendFactors())
00923     BLEND_CUSTOM_SEPARATE     // Blend textures using custom rgb/alpha separate src/dst factors
                                // (use rlSetBlendFactorsSeparate())
00924 } BlendMode;
00925
00926 // Gesture
00927 // NOTE: Provided as bit-wise flags to enable only desired gestures
00928 typedef enum {
00929     GESTURE_NONE = 0,         // No gesture
00930     GESTURE_TAP = 1,          // Tap gesture
00931     GESTURE_DOUBLETAP = 2,    // Double tap gesture
00932     GESTURE_HOLD = 4,         // Hold gesture
00933     GESTURE_DRAG = 8,         // Drag gesture
00934     GESTURE_SWIPE_RIGHT = 16, // Swipe right gesture
00935     GESTURE_SWIPE_LEFT = 32,  // Swipe left gesture
00936     GESTURE_SWIPE_UP = 64,    // Swipe up gesture
00937     GESTURE_SWIPE_DOWN = 128, // Swipe down gesture
00938     GESTURE_PINCH_IN = 256,   // Pinch in gesture
00939     GESTURE_PINCH_OUT = 512   // Pinch out gesture
00940 } Gesture;
00941
00942 // Camera system modes
00943 typedef enum {
00944     CAMERA_CUSTOM = 0,        // Camera custom, controlled by user (UpdateCamera() does nothing)
00945     CAMERA_FREE,              // Camera free mode
00946     CAMERA_ORBITAL,           // Camera orbital, around target, zoom supported
00947     CAMERA_FIRST_PERSON,      // Camera first person
00948     CAMERA_THIRD_PERSON       // Camera third person
00949 } CameraMode;
00950
00951 // Camera projection
00952 typedef enum {
00953     CAMERA_PERSPECTIVE = 0,   // Perspective projection
00954     CAMERA_ORTHOGRAPHIC       // Orthographic projection
00955 } CameraProjection;
00956
00957 // N-patch layout
00958 typedef enum {
00959     NPATCH_NINE_PATCH = 0,     // Npatch layout: 3x3 tiles
00960     NPATCH_THREE_PATCH_VERTICAL, // Npatch layout: 1x3 tiles
00961     NPATCH_THREE_PATCH_HORIZONTAL // Npatch layout: 3x1 tiles
00962 } NPatchLayout;
00963
00964 // Callbacks to hook some internal functions
00965 // WARNING: These callbacks are intended for advanced users
00966 typedef void (*TraceLogCallback)(int logLevel, const char *text, va_list args); // Logging: Redirect
// trace log messages
00967 typedef unsigned char *(*LoadFileDataCallback)(const char *fileName, int *dataSize); // FileIO:
// Load binary data
00968 typedef bool (*SaveFileDataCallback)(const char *fileName, void *data, int dataSize); // FileIO:
// Save binary data
00969 typedef char *(*LoadFileTextCallback)(const char *fileName); // FileIO: Load text data
00970 typedef bool (*SaveFileTextCallback)(const char *fileName, const char *text); // FileIO: Save text
// data
00971
00972 //-----
00973 // Global Variables Definition
00974 //-----
00975 // It's lonely here...
00976
00977 //-----
00978 // Window and Graphics Device Functions (Module: core)
00979 //-----
00980
00981 #if defined(__cplusplus)
00982 extern "C" { // Prevents name mangling of functions

```



```

00983 #endif
00984
00985 // Window-related functions
00986 RLAPI void InitWindow(int width, int height, const char *title); // Initialize window and OpenGL
                                context
00987 RLAPI void CloseWindow(void); // Close window and unload OpenGL
                                context
00988 RLAPI bool WindowShouldClose(void); // Check if application should close
                                (KEY_ESCAPE pressed or windows close icon clicked)
00989 RLAPI bool IsWindowReady(void); // Check if window has been
                                initialized successfully
00990 RLAPI bool IsWindowFullscreen(void); // Check if window is currently
                                fullscreen
00991 RLAPI bool IsWindowHidden(void); // Check if window is currently
                                hidden
00992 RLAPI bool IsWindowMinimized(void); // Check if window is currently
                                minimized
00993 RLAPI bool IsWindowMaximized(void); // Check if window is currently
                                maximized
00994 RLAPI bool IsWindowFocused(void); // Check if window is currently
                                focused
00995 RLAPI bool IsWindowResized(void); // Check if window has been resized
                                last frame
00996 RLAPI bool IsWindowState(unsigned int flag); // Check if one specific window flag
                                is enabled
00997 RLAPI void SetWindowState(unsigned int flags); // Set window configuration state
                                using flags
00998 RLAPI void ClearWindowState(unsigned int flags); // Clear window configuration state
                                flags
00999 RLAPI void ToggleFullscreen(void); // Toggle window state:
                                fullscreen/windowed, resizes monitor to match window resolution
01000 RLAPI void ToggleBorderlessWindowed(void); // Toggle window state: borderless
                                windowed, resizes window to match monitor resolution
01001 RLAPI void MaximizeWindow(void); // Set window state: maximized, if
                                resizable
01002 RLAPI void MinimizeWindow(void); // Set window state: minimized, if
                                resizable
01003 RLAPI void RestoreWindow(void); // Restore window from being
                                minimized/maximized
01004 RLAPI void SetWindowIcon(Image image); // Set icon for window (single
                                image, RGBA 32bit)
01005 RLAPI void SetWindowIcons(Image *images, int count); // Set icon for window (multiple
                                images, RGBA 32bit)
01006 RLAPI void SetWindowTitle(const char *title); // Set title for window
01007 RLAPI void SetWindowPosition(int x, int y); // Set window position on screen
01008 RLAPI void SetWindowMonitor(int monitor); // Set monitor for the current
                                window
01009 RLAPI void SetWindowMinSize(int width, int height); // Set window minimum dimensions
                                (for FLAG_WINDOW_RESIZABLE)
01010 RLAPI void SetWindowMaxSize(int width, int height); // Set window maximum dimensions
                                (for FLAG_WINDOW_RESIZABLE)
01011 RLAPI void SetWindowSize(int width, int height); // Set window dimensions
01012 RLAPI void SetWindowOpacity(float opacity); // Set window opacity [0.0f..1.0f]
01013 RLAPI void SetWindowFocused(void); // Set window focused
01014 RLAPI void *GetWindowHandle(void); // Get native window handle
01015 RLAPI int GetScreenWidth(void); // Get current screen width
01016 RLAPI int GetScreenHeight(void); // Get current screen height
01017 RLAPI int GetRenderWidth(void); // Get current render width (it
                                considers HiDPI)
01018 RLAPI int GetRenderHeight(void); // Get current render height (it
                                considers HiDPI)
01019 RLAPI int GetMonitorCount(void); // Get number of connected monitors
01020 RLAPI int GetCurrentMonitor(void); // Get current monitor where window
                                is placed
01021 RLAPI Vector2 GetMonitorPosition(int monitor); // Get specified monitor position
01022 RLAPI int GetMonitorWidth(int monitor); // Get specified monitor width
                                (current video mode used by monitor)
01023 RLAPI int GetMonitorHeight(int monitor); // Get specified monitor height
                                (current video mode used by monitor)
01024 RLAPI int GetMonitorPhysicalWidth(int monitor); // Get specified monitor physical
                                width in millimetres
01025 RLAPI int GetMonitorPhysicalHeight(int monitor); // Get specified monitor physical
                                height in millimetres
01026 RLAPI int GetMonitorRefreshRate(int monitor); // Get specified monitor refresh
                                rate
01027 RLAPI Vector2 GetWindowPosition(void); // Get window position XY on monitor
01028 RLAPI Vector2 GetWindowScaleDPI(void); // Get window scale DPI factor
01029 RLAPI const char *GetMonitorName(int monitor); // Get the human-readable, UTF-8
                                encoded name of the specified monitor
01030 RLAPI void SetClipboardText(const char *text); // Set clipboard text content
01031 RLAPI const char *GetClipboardText(void); // Get clipboard text content
01032 RLAPI Image GetClipboardImage(void); // Get clipboard image content
01033 RLAPI void EnableEventWaiting(void); // Enable waiting for events on
                                EndDrawing(), no automatic event polling
01034 RLAPI void DisableEventWaiting(void); // Disable waiting for events on
                                EndDrawing(), automatic events polling
01035

```

```

01036 // Cursor-related functions
01037 RLAPI void ShowCursor(void); // Shows cursor
01038 RLAPI void HideCursor(void); // Hides cursor
01039 RLAPI bool IsCursorHidden(void); // Check if cursor is not visible
01040 RLAPI void EnableCursor(void); // Enables cursor (unlock cursor)
01041 RLAPI void DisableCursor(void); // Disables cursor (lock cursor)
01042 RLAPI bool IsCursorOnScreen(void); // Check if cursor is on the screen
01043
01044 // Drawing-related functions
01045 RLAPI void ClearBackground(Color color); // Set background color (framebuffer)
01046 // clear color)
01047 RLAPI void BeginDrawing(void); // Setup canvas (framebuffer) to
01048 // start drawing
01049 RLAPI void EndDrawing(void); // End canvas drawing and swap
01050 // buffers (double buffering)
01051 RLAPI void BeginMode2D(Camera2D camera); // Begin 2D mode with custom camera
01052 // (2D)
01053 RLAPI void EndMode2D(void); // Ends 2D mode with custom camera
01054 RLAPI void BeginMode3D(Camera3D camera); // Begin 3D mode with custom camera
01055 // (3D)
01056 RLAPI void EndMode3D(void); // Ends 3D mode and returns to
01057 // default 2D orthographic mode
01058 RLAPI void BeginTextureMode(RenderTexture2D target); // Begin drawing to render texture
01059 RLAPI void EndTextureMode(void); // Ends drawing to render texture
01060 RLAPI void BeginShaderMode(Shader shader); // Begin custom shader drawing
01061 RLAPI void EndShaderMode(void); // End custom shader drawing (use
01062 // default shader)
01063 RLAPI void BeginBlendMode(int mode); // Begin blending mode (alpha,
01064 // additive, multiplied, subtract, custom)
01065 RLAPI void EndBlendMode(void); // End blending mode (reset to
01066 // default: alpha blending)
01067 RLAPI void BeginScissorMode(int x, int y, int width, int height); // Begin scissor mode (define screen
01068 // area for following drawing)
01069 RLAPI void EndScissorMode(void); // End scissor mode
01070 RLAPI void BeginVrStereoMode(VrStereoConfig config); // Begin stereo rendering (requires
01071 // VR simulator)
01072 RLAPI void EndVrStereoMode(void); // End stereo rendering (requires VR
01073 // simulator)
01074
01075 // VR stereo config functions for VR simulator
01076 RLAPI VrStereoConfig LoadVrStereoConfig(VrDeviceInfo device); // Load VR stereo config for VR
01077 // simulator device parameters
01078 RLAPI void UnloadVrStereoConfig(VrStereoConfig config); // Unload VR stereo config
01079
01080 // Shader management functions
01081 // NOTE: Shader functionality is not available on OpenGL 1.1
01082 RLAPI Shader LoadShader(const char *vsFileName, const char *fsFileName); // Load shader from files
01083 // and bind default locations
01084 RLAPI Shader LoadShaderFromMemory(const char *vsCode, const char *fsCode); // Load shader from code
01085 // strings and bind default locations
01086 RLAPI bool IsShaderValid(Shader shader); // Check if a shader is
01087 // valid (loaded on GPU)
01088 RLAPI int GetShaderLocation(Shader shader, const char *uniformName); // Get shader uniform
01089 // location
01090 RLAPI int GetShaderLocationAttrib(Shader shader, const char *attribName); // Get shader attribute
01091 // location
01092 RLAPI void SetShaderValue(Shader shader, int locIndex, const void *value, int uniformType); // Set
01093 // shader uniform value
01094 RLAPI void SetShaderValueV(Shader shader, int locIndex, const void *value, int uniformType, int
01095 // count); // Set shader uniform value vector
01096 RLAPI void SetShaderValueMatrix(Shader shader, int locIndex, Matrix mat); // Set shader uniform value
01097 // (matrix 4x4)
01098 RLAPI void SetShaderValueTexture(Shader shader, int locIndex, Texture2D texture); // Set shader
01099 // uniform value and bind the texture (sampler2d)
01100 RLAPI void UnloadShader(Shader shader); // Unload shader from GPU
01101 // memory (VRAM)
01102
01103 // Screen-space-related functions
01104 #define GetMouseRay GetScreenToWorldRay // Compatibility hack for previous raylib versions
01105 RLAPI Ray GetScreenToWorldRay(Vector2 position, Camera camera); // Get a ray trace from screen
01106 // position (i.e mouse)
01107 RLAPI Ray GetScreenToWorldRayEx(Vector2 position, Camera camera, int width, int height); // Get a ray
01108 // trace from screen position (i.e mouse) in a viewport
01109 RLAPI Vector2 GetWorldToScreen(Vector3 position, Camera camera); // Get the screen space
01110 // position for a 3d world space position
01111 RLAPI Vector2 GetWorldToScreenEx(Vector3 position, Camera camera, int width, int height); // Get size
01112 // position for a 3d world space position
01113 RLAPI Vector2 GetWorldToScreen2D(Vector2 position, Camera2D camera); // Get the screen space
01114 // position for a 2d camera world space position
01115 RLAPI Vector2 GetScreenToWorld2D(Vector2 position, Camera2D camera); // Get the world space
01116 // position for a 2d camera screen space position
01117 RLAPI Matrix GetCameraMatrix(Camera camera); // Get camera transform matrix
01118 // (view matrix)
01119 RLAPI Matrix GetCameraMatrix2D(Camera2D camera); // Get camera 2d transform
01120 // matrix
01121
01122 // Timing-related functions

```

```

01092 RLAPI void SetTargetFPS(int fps); // Set target FPS (maximum)
01093 RLAPI float GetFrameTime(void); // Get time in seconds for last frame drawn
// (delta time)
01094 RLAPI double GetTime(void); // Get elapsed time in seconds since
// InitWindow()
01095 RLAPI int GetFPS(void); // Get current FPS
01096
01097 // Custom frame control functions
01098 // NOTE: Those functions are intended for advanced users that want full control over the frame
// processing
01099 // By default EndDrawing() does this job: draws everything + SwapScreenBuffer() + manage frame timing
// + PollInputEvents()
01100 // To avoid that behaviour and control frame processes manually, enable in config.h:
// SUPPORT_CUSTOM_FRAME_CONTROL
01101 RLAPI void SwapScreenBuffer(void); // Swap back buffer with front buffer (screen
// drawing)
01102 RLAPI void PollInputEvents(void); // Register all input events
01103 RLAPI void WaitTime(double seconds); // Wait for some time (halt program execution)
01104
01105 // Random values generation functions
01106 RLAPI void SetRandomSeed(unsigned int seed); // Set the seed for the random number
// generator
01107 RLAPI int GetRandomValue(int min, int max); // Get a random value between min and max
// (both included)
01108 RLAPI int *LoadRandomSequence(unsigned int count, int min, int max); // Load random values sequence,
// no values repeated
01109 RLAPI void UnloadRandomSequence(int *sequence); // Unload random values sequence
01110
01111 // Misc. functions
01112 RLAPI void TakeScreenshot(const char *fileName); // Takes a screenshot of current
// screen (filename extension defines format)
01113 RLAPI void SetConfigFlags(unsigned int flags); // Setup init configuration flags
// (view FLAGS)
01114 RLAPI void OpenURL(const char *url); // Open URL with default system
// browser (if available)
01115
01116 // Logging system
01117 RLAPI void SetTraceLogLevel(int logLevel); // Set the current threshold (minimum)
// log level
01118 RLAPI void TraceLog(int logLevel, const char *text, ...); // Show trace log messages (LOG_DEBUG,
// LOG_INFO, LOG_WARNING, LOG_ERROR...)
01119 RLAPI void SetTraceLogCallback(TraceLogCallback callback); // Set custom trace log
01120
01121 // Memory management, using internal allocators
01122 RLAPI void *MemAlloc(unsigned int size); // Internal memory allocator
01123 RLAPI void *MemRealloc(void *ptr, unsigned int size); // Internal memory reallocator
01124 RLAPI void MemFree(void *ptr); // Internal memory free
01125
01126 // File system management functions
01127 RLAPI unsigned char *LoadFileData(const char *fileName, int *dataSize); // Load file data as byte
// array (read)
01128 RLAPI void UnloadFileData(unsigned char *data); // Unload file data allocated by
// LoadFileData()
01129 RLAPI bool SaveFileData(const char *fileName, void *data, int dataSize); // Save data to file from
// byte array (write), returns true on success
01130 RLAPI bool ExportDataAsCode(const unsigned char *data, int dataSize, const char *fileName); // Export
// data to code (.h), returns true on success
01131 RLAPI char *LoadFileText(const char *fileName); // Load text data from file
// (read), returns a '\0' terminated string
01132 RLAPI void UnloadFileText(char *text); // Unload file text data allocated
// by LoadFileText()
01133 RLAPI bool SaveFileText(const char *fileName, const char *text); // Save text data to file (write),
// string must be '\0' terminated, returns true on success
01134
01135 // File access custom callbacks
01136 // WARNING: Callbacks setup is intended for advanced users
01137 RLAPI void SetLoadFileDataCallback(LoadFileDataCallback callback); // Set custom file binary data
// loader
01138 RLAPI void SetSaveFileDataCallback(SaveFileDataCallback callback); // Set custom file binary data
// saver
01139 RLAPI void SetLoadFileTextCallback(LoadFileTextCallback callback); // Set custom file text data
// loader
01140 RLAPI void SetSaveFileTextCallback(SaveFileTextCallback callback); // Set custom file text data saver
01141
01142 RLAPI int FileRename(const char *fileName, const char *fileRename); // Rename file (if exists)
01143 RLAPI int FileRemove(const char *fileName); // Remove file (if exists)
01144 RLAPI int FileCopy(const char *srcPath, const char *dstPath); // Copy file from one path to
// another, dstPath created if it doesn't exist
01145 RLAPI int FileMove(const char *srcPath, const char *dstPath); // Move file from one directory to
// another, dstPath created if it doesn't exist
01146 RLAPI int FileTextReplace(const char *fileName, const char *search, const char *replacement); //
// Replace text in an existing file
01147 RLAPI int FileTextFindIndex(const char *fileName, const char *search); // Find text in existing file
01148 RLAPI bool FileExists(const char *fileName); // Check if file exists
01149 RLAPI bool DirectoryExists(const char *dirPath); // Check if a directory path
// exists
01150 RLAPI bool IsFileExtension(const char *fileName, const char *ext); // Check file extension

```

```

    (recommended include point: .png, .wav)
01151 RLAPI int GetFileLength(const char *fileName); // Get file length in bytes (NOTE:
    GetFileSize() conflicts with windows.h)
01152 RLAPI long GetFileModTime(const char *fileName); // Get file modification time
    (last write time)
01153 RLAPI const char *GetFileExtension(const char *fileName); // Get pointer to extension for a
    filename string (includes dot: '.png')
01154 RLAPI const char *GetFileName(const char *filePath); // Get pointer to filename for a
    path string
01155 RLAPI const char *GetFileNameWithoutExt(const char *filePath); // Get filename string without
    extension (uses static string)
01156 RLAPI const char *GetDirectoryPath(const char *filePath); // Get full path for a given
    fileName with path (uses static string)
01157 RLAPI const char *GetPrevDirectoryPath(const char *dirPath); // Get previous directory path for
    a given path (uses static string)
01158 RLAPI const char *GetWorkingDirectory(void); // Get current working directory
    (uses static string)
01159 RLAPI const char *GetApplicationDirectory(void); // Get the directory of the
    running application (uses static string)
01160 RLAPI int MakeDirectory(const char *dirPath); // Create directories (including
    full path requested), returns 0 on success
01161 RLAPI bool ChangeDirectory(const char *dirPath); // Change working directory,
    return true on success
01162 RLAPI bool IsPathFile(const char *path); // Check if a given path is a file
    or a directory
01163 RLAPI bool IsFileNameValid(const char *fileName); // Check if fileName is valid for
    the platform/OS
01164 RLAPI FilePathList LoadDirectoryFiles(const char *dirPath); // Load directory filepaths, files
    and directories, no subdirs scan
01165 RLAPI FilePathList LoadDirectoryFilesEx(const char *basePath, const char *filter, bool scanSubdirs);
    // Load directory filepaths with extension filtering and subdir scan; some filters available: "*.*",
    "FILES*", "DIRS*"
01166 RLAPI void UnloadDirectoryFiles(FilePathList files); // Unload filepaths
01167 RLAPI bool IsFileDropped(void); // Check if a file has been
    dropped into window
01168 RLAPI FilePathList LoadDroppedFiles(void); // Load dropped filepaths
01169 RLAPI void UnloadDroppedFiles(FilePathList files); // Unload dropped filepaths
01170 RLAPI unsigned int GetDirectoryFileCount(const char *dirPath); // Get the file count in a
    directory
01171 RLAPI unsigned int GetDirectoryFileCountEx(const char *basePath, const char *filter, bool
    scanSubdirs); // Get the file count in a directory with extension filtering and recursive directory
    scan. Use 'DIR' in the filter string to include directories in the result
01172
01173 // Compression/Encoding functionality
01174 RLAPI unsigned char *CompressData(const unsigned char *data, int dataSize, int *compDataSize);
    // Compress data (DEFLATE algorithm), memory must be MemFree()
01175 RLAPI unsigned char *DecompressData(const unsigned char *compData, int compDataSize, int *dataSize);
    // Decompress data (DEFLATE algorithm), memory must be MemFree()
01176 RLAPI char *EncodeDataBase64(const unsigned char *data, int dataSize, int *outputSize);
    // Encode data to Base64 string (includes NULL terminator), memory must be MemFree()
01177 RLAPI unsigned char *DecodeDataBase64(const char *text, int *outputSize);
    // Decode Base64 string (expected NULL terminated), memory must be MemFree()
01178 RLAPI unsigned int ComputeCRC32(unsigned char *data, int dataSize); // Compute CRC32 hash code
01179 RLAPI unsigned int *ComputeMD5(unsigned char *data, int dataSize); // Compute MD5 hash code, returns
    static int[4] (16 bytes)
01180 RLAPI unsigned int *ComputeSHA1(unsigned char *data, int dataSize); // Compute SHA1 hash code, returns
    static int[5] (20 bytes)
01181 RLAPI unsigned int *ComputeSHA256(unsigned char *data, int dataSize); // Compute SHA256 hash code,
    returns static int[8] (32 bytes)
01182
01183 // Automation events functionality
01184 RLAPI AutomationEventList LoadAutomationEventList(const char *fileName); // Load automation events
    list from file, NULL for empty list, capacity = MAX_AUTOMATION_EVENTS
01185 RLAPI void UnloadAutomationEventList(AutomationEventList list); // Unload automation events list
    from file
01186 RLAPI bool ExportAutomationEventList(AutomationEventList list, const char *fileName); // Export
    automation events list as text file
01187 RLAPI void SetAutomationEventList(AutomationEventList *list); // Set automation event list to
    record to
01188 RLAPI void SetAutomationEventBaseFrame(int frame); // Set automation event internal
    base frame to start recording
01189 RLAPI void StartAutomationEventRecording(void); // Start recording automation events
    (AutomationEventList must be set)
01190 RLAPI void StopAutomationEventRecording(void); // Stop recording automation events
01191 RLAPI void PlayAutomationEvent(AutomationEvent event); // Play a recorded automation event
01192
01193 //-----
01194 // Input Handling Functions (Module: core)
01195 //-----
01196
01197 // Input-related functions: keyboard
01198 RLAPI bool IsKeyPressed(int key); // Check if a key has been pressed once
01199 RLAPI bool IsKeyPressedRepeat(int key); // Check if a key has been pressed again
01200 RLAPI bool IsKeyDown(int key); // Check if a key is being pressed
01201 RLAPI bool IsKeyReleased(int key); // Check if a key has been released once
01202 RLAPI bool IsKeyUp(int key); // Check if a key is NOT being pressed
01203 RLAPI int GetKeyPressed(void); // Get key pressed (keycode), call it

```

```

multiple times for keys queued, returns 0 when the queue is empty
01204 RLAPI int GetCharPressed(void); // Get char pressed (unicode), call it
multiple times for chars queued, returns 0 when the queue is empty
01205 RLAPI const char *GetKeyName(int key); // Get name of a QWERTY key on the
current keyboard layout (eg returns string 'q' for KEY_A on an AZERTY keyboard)
01206 RLAPI void SetExitKey(int key); // Set a custom key to exit program
(default is ESC)
01207
01208 // Input-related functions: gamepads
01209 RLAPI bool IsGamepadAvailable(int gamepad); // Check if a gamepad is available
01210 RLAPI const char *GetGamepadName(int gamepad); // Get gamepad internal name id
01211 RLAPI bool IsGamepadButtonPressed(int gamepad, int button); // Check if a gamepad button has been
pressed once
01212 RLAPI bool IsGamepadButtonDown(int gamepad, int button); // Check if a gamepad button is being
pressed
01213 RLAPI bool IsGamepadButtonReleased(int gamepad, int button); // Check if a gamepad button has been
released once
01214 RLAPI bool IsGamepadButtonUp(int gamepad, int button); // Check if a gamepad button is NOT
being pressed
01215 RLAPI int GetGamepadButtonPressed(void); // Get the last gamepad button pressed
01216 RLAPI int GetGamepadAxisCount(int gamepad); // Get axis count for a gamepad
01217 RLAPI float GetGamepadAxisMovement(int gamepad, int axis); // Get movement value for a gamepad axis
01218 RLAPI int SetGamepadMappings(const char *mappings); // Set internal gamepad mappings
(SDL_GameControllerDB)
01219 RLAPI void SetGamepadVibration(int gamepad, float leftMotor, float rightMotor, float duration); // Set
gamepad vibration for both motors (duration in seconds)
01220
01221 // Input-related functions: mouse
01222 RLAPI bool IsMouseButtonPressed(int button); // Check if a mouse button has been
pressed once
01223 RLAPI bool IsMouseButtonDown(int button); // Check if a mouse button is being
pressed
01224 RLAPI bool IsMouseButtonReleased(int button); // Check if a mouse button has been
released once
01225 RLAPI bool IsMouseButtonUp(int button); // Check if a mouse button is NOT being
pressed
01226 RLAPI int GetMouseX(void); // Get mouse position X
01227 RLAPI int GetMouseY(void); // Get mouse position Y
01228 RLAPI Vector2 GetMousePosition(void); // Get mouse position XY
01229 RLAPI Vector2 GetMouseDelta(void); // Get mouse delta between frames
01230 RLAPI void SetMousePosition(int x, int y); // Set mouse position XY
01231 RLAPI void SetMouseOffset(int offsetX, int offsetY); // Set mouse offset
01232 RLAPI void SetMouseScale(float scaleX, float scaleY); // Set mouse scaling
01233 RLAPI float GetMouseWheelMove(void); // Get mouse wheel movement for X or Y,
whichever is larger
01234 RLAPI Vector2 GetMouseWheelMoveV(void); // Get mouse wheel movement for both X
and Y
01235 RLAPI void SetMouseCursor(int cursor); // Set mouse cursor
01236
01237 // Input-related functions: touch
01238 RLAPI int GetTouchX(void); // Get touch position X for touch point
0 (relative to screen size)
01239 RLAPI int GetTouchY(void); // Get touch position Y for touch point
0 (relative to screen size)
01240 RLAPI Vector2 GetTouchPosition(int index); // Get touch position XY for a touch
point index (relative to screen size)
01241 RLAPI int GetTouchPointId(int index); // Get touch point identifier for given
index
01242 RLAPI int GetTouchPointCount(void); // Get number of touch points
01243
01244 //-----
01245 // Gestures and Touch Handling Functions (Module: rgestures)
01246 //-----
01247 RLAPI void SetGesturesEnabled(unsigned int flags); // Enable a set of gestures using flags
01248 RLAPI bool IsGestureDetected(unsigned int gesture); // Check if a gesture have been detected
01249 RLAPI int GetLatestDetectedGesture(void); // Get latest detected gesture
01250 RLAPI float GetGestureHoldDuration(void); // Get gesture hold time in seconds
01251 RLAPI Vector2 GetGestureDragVector(void); // Get gesture drag vector
01252 RLAPI float GetGestureDragAngle(void); // Get gesture drag angle
01253 RLAPI Vector2 GetGesturePinchVector(void); // Get gesture pinch delta
01254 RLAPI float GetGesturePinchAngle(void); // Get gesture pinch angle
01255
01256 //-----
01257 // Camera System Functions (Module: rcamera)
01258 //-----
01259 RLAPI void UpdateCamera(Camera *camera, int mode); // Update camera position for selected
mode
01260 RLAPI void UpdateCameraPro(Camera *camera, Vector3 movement, Vector3 rotation, float zoom); // Update
camera movement/rotation
01261
01262 //-----
01263 // Basic Shapes Drawing Functions (Module: shapes)
01264 //-----
01265 // Set texture and rectangle to be used on shapes drawing
01266 // NOTE: It can be useful when using basic shapes and one single font,
01267 // defining a font char white rectangle would allow drawing everything in a single draw call
01268 RLAPI void SetShapesTexture(Texture2D texture, Rectangle source); // Set texture and rectangle to be

```



```

        used on shapes drawing
01269 RLAPI Texture2D GetShapesTexture(void);           // Get texture that is used for shapes drawing
01270 RLAPI Rectangle GetShapesTextureRectangle(void);    // Get texture source rectangle that is used
        for shapes drawing
01271
01272 // Basic shapes drawing functions
01273 RLAPI void DrawPixel(int posX, int posY, Color color);
        // Draw a pixel using geometry [Can be slow, use with care]
01274 RLAPI void DrawPixelV(Vector2 position, Color color);
        // Draw a pixel using geometry (Vector version) [Can be slow, use with care]
01275 RLAPI void DrawLine(int startPosX, int startPosY, int endPosX, int endPosY, Color color);
        // Draw a line
01276 RLAPI void DrawLineV(Vector2 startPos, Vector2 endPos, Color color);
        // Draw a line (using gl lines)
01277 RLAPI void DrawLineEx(Vector2 startPos, Vector2 endPos, float thick, Color color);
        // Draw a line (using triangles/quads)
01278 RLAPI void DrawLineStrip(const Vector2 *points, int pointCount, Color color);
        // Draw lines sequence (using gl lines)
01279 RLAPI void DrawLineBezier(Vector2 startPos, Vector2 endPos, float thick, Color color);
        // Draw line segment cubic-bezier in-out interpolation
01280 RLAPI void DrawLineDashed(Vector2 startPos, Vector2 endPos, int dashSize, int spaceSize, Color color);
        // Draw a dashed line
01281 RLAPI void DrawCircle(int centerX, int centerY, float radius, Color color);
        // Draw a color-filled circle
01282 RLAPI void DrawCircleV(Vector2 center, float radius, Color color);
        // Draw a color-filled circle (Vector version)
01283 RLAPI void DrawCircleGradient(Vector2 center, float radius, Color inner, Color outer);
        // Draw a gradient-filled circle
01284 RLAPI void DrawCircleSector(Vector2 center, float radius, float startAngle, float endAngle, int
        segments, Color color); // Draw a piece of a circle
01285 RLAPI void DrawCircleSectorLines(Vector2 center, float radius, float startAngle, float endAngle, int
        segments, Color color); // Draw circle sector outline
01286 RLAPI void DrawCircleLines(int centerX, int centerY, float radius, Color color);
        // Draw circle outline
01287 RLAPI void DrawCircleLinesV(Vector2 center, float radius, Color color);
        // Draw circle outline (Vector version)
01288 RLAPI void DrawEllipse(int centerX, int centerY, float radiusH, float radiusV, Color color);
        // Draw ellipse
01289 RLAPI void DrawEllipseV(Vector2 center, float radiusH, float radiusV, Color color);
        // Draw ellipse (Vector version)
01290 RLAPI void DrawEllipseLines(int centerX, int centerY, float radiusH, float radiusV, Color color);
        // Draw ellipse outline
01291 RLAPI void DrawEllipseLinesV(Vector2 center, float radiusH, float radiusV, Color color);
        // Draw ellipse outline (Vector version)
01292 RLAPI void DrawRing(Vector2 center, float innerRadius, float outerRadius, float startAngle, float
        endAngle, int segments, Color color); // Draw ring
01293 RLAPI void DrawRingLines(Vector2 center, float innerRadius, float outerRadius, float startAngle, float
        endAngle, int segments, Color color); // Draw ring outline
01294 RLAPI void DrawRectangle(int posX, int posY, int width, int height, Color color);
        // Draw a color-filled rectangle
01295 RLAPI void DrawRectangleV(Vector2 position, Vector2 size, Color color);
        // Draw a color-filled rectangle (Vector version)
01296 RLAPI void DrawRectangleRec(Rectangle rec, Color color);
        // Draw a color-filled rectangle
01297 RLAPI void DrawRectanglePro(Rectangle rec, Vector2 origin, float rotation, Color color);
        // Draw a color-filled rectangle with pro parameters
01298 RLAPI void DrawRectangleGradientV(int posX, int posY, int width, int height, Color top, Color bottom);
        // Draw a vertical-gradient-filled rectangle
01299 RLAPI void DrawRectangleGradientH(int posX, int posY, int width, int height, Color left, Color right);
        // Draw a horizontal-gradient-filled rectangle
01300 RLAPI void DrawRectangleGradientEx(Rectangle rec, Color topLeft, Color bottomLeft, Color bottomRight,
        Color topRight); // Draw a gradient-filled rectangle with custom vertex colors
01301 RLAPI void DrawRectangleLines(int posX, int posY, int width, int height, Color color);
        // Draw rectangle outline
01302 RLAPI void DrawRectangleLinesEx(Rectangle rec, float lineThick, Color color);
        // Draw rectangle outline with extended parameters
01303 RLAPI void DrawRectangleRounded(Rectangle rec, float roundness, int segments, Color color);
        // Draw rectangle with rounded edges
01304 RLAPI void DrawRectangleRoundedLines(Rectangle rec, float roundness, int segments, Color color);
        // Draw rectangle lines with rounded edges
01305 RLAPI void DrawRectangleRoundedLinesEx(Rectangle rec, float roundness, int segments, float lineThick,
        Color color); // Draw rectangle with rounded edges outline
01306 RLAPI void DrawTriangle(Vector2 v1, Vector2 v2, Vector2 v3, Color color);
        // Draw a color-filled triangle (vertex in counter-clockwise order!)
01307 RLAPI void DrawTriangleLines(Vector2 v1, Vector2 v2, Vector2 v3, Color color);
        // Draw triangle outline (vertex in counter-clockwise order!)
01308 RLAPI void DrawTriangleFan(const Vector2 *points, int pointCount, Color color);
        // Draw a triangle fan defined by points (first vertex is the center)
01309 RLAPI void DrawTriangleStrip(const Vector2 *points, int pointCount, Color color);
        // Draw a triangle strip defined by points
01310 RLAPI void DrawPoly(Vector2 center, int sides, float radius, float rotation, Color color);
        // Draw a regular polygon (Vector version)
01311 RLAPI void DrawPolyLines(Vector2 center, int sides, float radius, float rotation, Color color);
        // Draw a polygon outline of n sides
01312 RLAPI void DrawPolyLinesEx(Vector2 center, int sides, float radius, float rotation, float lineThick,
        Color color); // Draw a polygon outline of n sides with extended parameters
01313

```

```

01314 // Splines drawing functions
01315 RLAPI void DrawSplineLinear(const Vector2 *points, int pointCount, float thick, Color color);
// Draw spline: Linear, minimum 2 points
01316 RLAPI void DrawSplineBasis(const Vector2 *points, int pointCount, float thick, Color color);
// Draw spline: B-Spline, minimum 4 points
01317 RLAPI void DrawSplineCatmullRom(const Vector2 *points, int pointCount, float thick, Color color);
// Draw spline: Catmull-Rom, minimum 4 points
01318 RLAPI void DrawSplineBezierQuadratic(const Vector2 *points, int pointCount, float thick, Color color);
// Draw spline: Quadratic Bezier, minimum 3 points (1 control point): [p1, c2, p3, c4...]
01319 RLAPI void DrawSplineBezierCubic(const Vector2 *points, int pointCount, float thick, Color color);
// Draw spline: Cubic Bezier, minimum 4 points (2 control points): [p1, c2, c3, p4, c5, c6...]
01320 RLAPI void DrawSplineSegmentLinear(Vector2 p1, Vector2 p2, float thick, Color color);
// Draw spline segment: Linear, 2 points
01321 RLAPI void DrawSplineSegmentBasis(Vector2 p1, Vector2 p2, Vector2 p3, Vector2 p4, float thick, Color
color); // Draw spline segment: B-Spline, 4 points
01322 RLAPI void DrawSplineSegmentCatmullRom(Vector2 p1, Vector2 p2, Vector2 p3, Vector2 p4, float thick,
Color color); // Draw spline segment: Catmull-Rom, 4 points
01323 RLAPI void DrawSplineSegmentBezierQuadratic(Vector2 p1, Vector2 c2, Vector2 p3, float thick, Color
color); // Draw spline segment: Quadratic Bezier, 2 points, 1 control point
01324 RLAPI void DrawSplineSegmentBezierCubic(Vector2 p1, Vector2 c2, Vector2 c3, Vector2 p4, float thick,
Color color); // Draw spline segment: Cubic Bezier, 2 points, 2 control points
01325
01326 // Spline segment point evaluation functions, for a given t [0.0f .. 1.0f]
01327 RLAPI Vector2 GetSplinePointLinear(Vector2 startPos, Vector2 endPos, float t);
// Get (evaluate) spline point: Linear
01328 RLAPI Vector2 GetSplinePointBasis(Vector2 p1, Vector2 p2, Vector2 p3, Vector2 p4, float t);
// Get (evaluate) spline point: B-Spline
01329 RLAPI Vector2 GetSplinePointCatmullRom(Vector2 p1, Vector2 p2, Vector2 p3, Vector2 p4, float t);
// Get (evaluate) spline point: Catmull-Rom
01330 RLAPI Vector2 GetSplinePointBezierQuad(Vector2 p1, Vector2 c2, Vector2 p3, float t);
// Get (evaluate) spline point: Quadratic Bezier
01331 RLAPI Vector2 GetSplinePointBezierCubic(Vector2 p1, Vector2 c2, Vector2 c3, Vector2 p4, float t);
// Get (evaluate) spline point: Cubic Bezier
01332
01333 // Basic shapes collision detection functions
01334 RLAPI bool CheckCollisionRecs(Rectangle rec1, Rectangle rec2);
// Check collision between two rectangles
01335 RLAPI bool CheckCollisionCircles(Vector2 center1, float radius1, Vector2 center2, float radius2);
// Check collision between two circles
01336 RLAPI bool CheckCollisionCircleRec(Vector2 center, float radius, Rectangle rec);
// Check collision between circle and rectangle
01337 RLAPI bool CheckCollisionCircleLine(Vector2 center, float radius, Vector2 p1, Vector2 p2);
// Check if circle collides with a line created between two points [p1] and [p2]
01338 RLAPI bool CheckCollisionPointRec(Vector2 point, Rectangle rec);
// Check if point is inside rectangle
01339 RLAPI bool CheckCollisionPointCircle(Vector2 point, Vector2 center, float radius);
// Check if point is inside circle
01340 RLAPI bool CheckCollisionPointTriangle(Vector2 point, Vector2 p1, Vector2 p2, Vector2 p3);
// Check if point is inside a triangle
01341 RLAPI bool CheckCollisionPointLine(Vector2 point, Vector2 p1, Vector2 p2, int threshold);
// Check if point belongs to line created between two points [p1] and [p2] with defined margin in
pixels [threshold]
01342 RLAPI bool CheckCollisionPointPoly(Vector2 point, const Vector2 *points, int pointCount);
// Check if point is within a polygon described by array of vertices
01343 RLAPI bool CheckCollisionLines(Vector2 startPos1, Vector2 endPos1, Vector2 startPos2, Vector2 endPos2,
Vector2 *collisionPoint); // Check the collision between two lines defined by two points each, returns
collision point by reference
01344 RLAPI Rectangle GetCollisionRec(Rectangle rec1, Rectangle rec2);
// Get collision rectangle for two rectangles collision
01345
01346 //-----
01347 // Texture Loading and Drawing Functions (Module: textures)
01348 //-----
01349
01350 // Image loading functions
01351 // NOTE: These functions do not require GPU access
01352 RLAPI Image LoadImage(const char *fileName);
// Load image from file into CPU memory (RAM)
01353 RLAPI Image LoadImageRaw(const char *fileName, int width, int height, int format, int headerSize);
// Load image from RAW file data
01354 RLAPI Image LoadImageAnim(const char *fileName, int *frames);
// Load image sequence from file (frames appended to image.data)
01355 RLAPI Image LoadImageAnimFromMemory(const char *fileName, const unsigned char *fileData, int dataSize,
int *frames); // Load image sequence from memory buffer
01356 RLAPI Image LoadImageFromMemory(const char *fileName, const unsigned char *fileData, int dataSize);
// Load image from memory buffer, fileType refers to extension: i.e. '.png'
01357 RLAPI Image LoadImageFromTexture(Texture2D texture);
// Load image from GPU texture data
01358 RLAPI Image LoadImageFromScreen(void);
// Load image from screen buffer and (screenshot)
01359 RLAPI bool IsImageValid(Image image);
// Check if an image is valid (data and parameters)
01360 RLAPI void UnloadImage(Image image);
// Unload image from CPU memory (RAM)
01361 RLAPI bool ExportImage(Image image, const char *fileName);
// Export image data to file, returns true on success
01362 RLAPI unsigned char *ExportImageToMemory(Image image, const char *fileType, int *fileSize);

```

```

    // Export image to memory buffer, memory must be MemFree()
01363 RLAPI bool ExportImageAsCode(Image image, const char *fileName);
    // Export image as code file defining an array of bytes, returns true on success
01364
01365 // Image generation functions
01366 RLAPI Image GenImageColor(int width, int height, Color color);
    // Generate image: plain color
01367 RLAPI Image GenImageGradientLinear(int width, int height, int direction, Color start, Color end);
    // Generate image: linear gradient, direction in degrees [0..360], 0=Vertical gradient
01368 RLAPI Image GenImageGradientRadial(int width, int height, float density, Color inner, Color outer);
    // Generate image: radial gradient
01369 RLAPI Image GenImageGradientSquare(int width, int height, float density, Color inner, Color outer);
    // Generate image: square gradient
01370 RLAPI Image GenImageChecked(int width, int height, int checksX, int checksY, Color col1, Color col2);
    // Generate image: checked
01371 RLAPI Image GenImageWhiteNoise(int width, int height, float factor);
    // Generate image: white noise
01372 RLAPI Image GenImagePerlinNoise(int width, int height, int offsetX, int offsetY, float scale);
    // Generate image: perlin noise
01373 RLAPI Image GenImageCellular(int width, int height, int tileSize);
    // Generate image: cellular algorithm, bigger tileSize means bigger cells
01374 RLAPI Image GenImageText(int width, int height, const char *text);
    // Generate image: grayscale image from text data
01375
01376 // Image manipulation functions
01377 RLAPI Image ImageCopy(Image image);
    // Create an image duplicate (useful for transformations)
01378 RLAPI Image ImageFromImage(Image image, Rectangle rec);
    // Create an image from another image piece
01379 RLAPI Image ImageFromChannel(Image image, int selectedChannel);
    // Create an image from a selected channel of another image (GRAYSCALE)
01380 RLAPI Image ImageText(const char *text, int fontSize, Color color);
    // Create an image from text (default font)
01381 RLAPI Image ImageTextEx(Font font, const char *text, float fontSize, float spacing, Color tint);
    // Create an image from text (custom sprite font)
01382 RLAPI void ImageFormat(Image *image, int newFormat);
    // Convert image data to desired format
01383 RLAPI void ImageToPOT(Image *image, Color fill);
    // Convert image to POT (power-of-two)
01384 RLAPI void ImageCrop(Image *image, Rectangle crop);
    // Crop an image to a defined rectangle
01385 RLAPI void ImageAlphaCrop(Image *image, float threshold);
    // Crop image depending on alpha value
01386 RLAPI void ImageAlphaClear(Image *image, Color color, float threshold);
    // Clear alpha channel to desired color
01387 RLAPI void ImageAlphaMask(Image *image, Image alphaMask);
    // Apply alpha mask to image
01388 RLAPI void ImageAlphaPremultiply(Image *image);
    // Premultiply alpha channel
01389 RLAPI void ImageBlurGaussian(Image *image, int blurSize);
    // Apply Gaussian blur using a box blur approximation
01390 RLAPI void ImageKernelConvolution(Image *image, const float *kernel, int kernelSize);
    // Apply custom square convolution kernel to image
01391 RLAPI void ImageResize(Image *image, int newWidth, int newHeight);
    // Resize image (Bicubic scaling algorithm)
01392 RLAPI void ImageResizeNN(Image *image, int newWidth, int newHeight);
    // Resize image (Nearest-Neighbor scaling algorithm)
01393 RLAPI void ImageResizeCanvas(Image *image, int newWidth, int newHeight, int offsetX, int offsetY,
    Color fill); // Resize canvas and fill with color
01394 RLAPI void ImageMipmaps(Image *image);
    // Compute all mipmap levels for a provided image
01395 RLAPI void ImageDither(Image *image, int rBpp, int gBpp, int bBpp, int aBpp);
    // Dither image data to 16bpp or lower (Floyd-Steinberg dithering)
01396 RLAPI void ImageFlipVertical(Image *image);
    // Flip image vertically
01397 RLAPI void ImageFlipHorizontal(Image *image);
    // Flip image horizontally
01398 RLAPI void ImageRotate(Image *image, int degrees);
    // Rotate image by input angle in degrees (-359 to 359)
01399 RLAPI void ImageRotateCW(Image *image);
    // Rotate image clockwise 90deg
01400 RLAPI void ImageRotateCCW(Image *image);
    // Rotate image counter-clockwise 90deg
01401 RLAPI void ImageColorTint(Image *image, Color color);
    // Modify image color: tint
01402 RLAPI void ImageColorInvert(Image *image);
    // Modify image color: invert
01403 RLAPI void ImageColorGrayscale(Image *image);
    // Modify image color: grayscale
01404 RLAPI void ImageColorContrast(Image *image, float contrast);
    // Modify image color: contrast (-100 to 100)
01405 RLAPI void ImageColorBrightness(Image *image, int brightness);
    // Modify image color: brightness (-255 to 255)
01406 RLAPI void ImageColorReplace(Image *image, Color color, Color replace);
    // Modify image color: replace color
01407 RLAPI Color *LoadImageColors(Image image);
    // Load color data from image as a Color array (RGBA - 32bit)

```



```

01408 RLAPI Color *LoadImagePalette(Image image, int maxPaletteSize, int *colorCount);
    // Load colors palette from image as a Color array (RGBA - 32bit)
01409 RLAPI void UnloadImageColors(Color *colors);
    // Unload color data loaded with LoadImageColors()
01410 RLAPI void UnloadImagePalette(Color *colors);
    // Unload colors palette loaded with LoadImagePalette()
01411 RLAPI Rectangle GetImageAlphaBorder(Image image, float threshold);
    // Get image alpha border rectangle
01412 RLAPI Color GetImageColor(Image image, int x, int y);
    // Get image pixel color at (x, y) position
01413
01414 // Image drawing functions
01415 // NOTE: Image software-rendering functions (CPU)
01416 RLAPI void ImageClearBackground(Image *dst, Color color);
    // Clear image background with given color
01417 RLAPI void ImageDrawPixel(Image *dst, int posX, int posY, Color color);
    // Draw pixel within an image
01418 RLAPI void ImageDrawPixelV(Image *dst, Vector2 position, Color color);
    // Draw pixel within an image (Vector version)
01419 RLAPI void ImageDrawLine(Image *dst, int startPosX, int startPosY, int endPosX, int endPosY, Color
color); // Draw line within an image
01420 RLAPI void ImageDrawLineV(Image *dst, Vector2 start, Vector2 end, Color color);
    // Draw line within an image (Vector version)
01421 RLAPI void ImageDrawLineEx(Image *dst, Vector2 start, Vector2 end, int thick, Color color);
    // Draw a line defining thickness within an image
01422 RLAPI void ImageDrawCircle(Image *dst, int centerX, int centerY, int radius, Color color);
    // Draw a filled circle within an image
01423 RLAPI void ImageDrawCircleV(Image *dst, Vector2 center, int radius, Color color);
    // Draw a filled circle within an image (Vector version)
01424 RLAPI void ImageDrawCircleLines(Image *dst, int centerX, int centerY, int radius, Color color);
    // Draw circle outline within an image
01425 RLAPI void ImageDrawCircleLinesV(Image *dst, Vector2 center, int radius, Color color);
    // Draw circle outline within an image (Vector version)
01426 RLAPI void ImageDrawRectangle(Image *dst, int posX, int posY, int width, int height, Color color);
    // Draw rectangle within an image
01427 RLAPI void ImageDrawRectangleV(Image *dst, Vector2 position, Vector2 size, Color color);
    // Draw rectangle within an image (Vector version)
01428 RLAPI void ImageDrawRectangleRec(Image *dst, Rectangle rec, Color color);
    // Draw rectangle within an image
01429 RLAPI void ImageDrawRectangleLines(Image *dst, Rectangle rec, int thick, Color color);
    // Draw rectangle lines within an image
01430 RLAPI void ImageDrawTriangle(Image *dst, Vector2 v1, Vector2 v2, Vector2 v3, Color color);
    // Draw triangle within an image
01431 RLAPI void ImageDrawTriangleEx(Image *dst, Vector2 v1, Vector2 v2, Vector2 v3, Color c1, Color c2,
Color c3); // Draw triangle with interpolated colors within an image
01432 RLAPI void ImageDrawTriangleLines(Image *dst, Vector2 v1, Vector2 v2, Vector2 v3, Color color);
    // Draw triangle outline within an image
01433 RLAPI void ImageDrawTriangleFan(Image *dst, const Vector2 *points, int pointCount, Color color);
    // Draw a triangle fan defined by points within an image (first vertex is the center)
01434 RLAPI void ImageDrawTriangleStrip(Image *dst, const Vector2 *points, int pointCount, Color color);
    // Draw a triangle strip defined by points within an image
01435 RLAPI void ImageDraw(Image *dst, Image src, Rectangle srcRec, Rectangle dstRec, Color tint);
    // Draw a source image within a destination image (tint applied to source)
01436 RLAPI void ImageDrawText(Image *dst, const char *text, int posX, int posY, int fontSize, Color color);
    // Draw text (using default font) within an image (destination)
01437 RLAPI void ImageDrawTextEx(Image *dst, Font font, const char *text, Vector2 position, float fontSize,
float spacing, Color tint); // Draw text (custom sprite font) within an image (destination)
01438
01439 // Texture loading functions
01440 // NOTE: These functions require GPU access
01441 RLAPI Texture2D LoadTexture(const char *fileName);
    // Load texture from file into GPU memory (VRAM)
01442 RLAPI Texture2D LoadTextureFromImage(Image image);
    // Load texture from image data
01443 RLAPI TextureCubemap LoadTextureCubemap(Image image, int layout);
    // Load cubemap from image, multiple image cubemap layouts supported
01444 RLAPI RenderTexture2D LoadRenderTexture(int width, int height);
    // Load texture for rendering (framebuffer)
01445 RLAPI bool IsTextureValid(Texture2D texture);
    // Check if a texture is valid (loaded in GPU)
01446 RLAPI void UnloadTexture(Texture2D texture);
    // Unload texture from GPU memory (VRAM)
01447 RLAPI bool IsRenderTextureValid(RenderTexture2D target);
    // Check if a render texture is valid (loaded in GPU)
01448 RLAPI void UnloadRenderTexture(RenderTexture2D target);
    // Unload render texture from GPU memory (VRAM)
01449 RLAPI void UpdateTexture(Texture2D texture, const void *pixels);
    // Update GPU texture with new data (pixels should be able to fill texture)
01450 RLAPI void UpdateTextureRec(Texture2D texture, Rectangle rec, const void *pixels);
    // Update GPU texture rectangle with new data (pixels and rec should fit in texture)
01451
01452 // Texture configuration functions
01453 RLAPI void GenTextureMipmaps(Texture2D *texture);
    // Generate GPU mipmaps for a texture
01454 RLAPI void SetTextureFilter(Texture2D texture, int filter);
    // Set texture scaling filter mode
01455 RLAPI void SetTextureWrap(Texture2D texture, int wrap);

```

```

// Set texture wrapping mode
01456
01457 // Texture drawing functions
01458 RLAPI void DrawTexture(Texture2D texture, int posX, int posY, Color tint);
// Draw a Texture2D
01459 RLAPI void DrawTextureV(Texture2D texture, Vector2 position, Color tint);
// Draw a Texture2D with position defined as Vector2
01460 RLAPI void DrawTextureEx(Texture2D texture, Vector2 position, float rotation, float scale, Color
tint); // Draw a Texture2D with extended parameters
01461 RLAPI void DrawTextureRec(Texture2D texture, Rectangle source, Vector2 position, Color tint);
// Draw a part of a texture defined by a rectangle
01462 RLAPI void DrawTexturePro(Texture2D texture, Rectangle source, Rectangle dest, Vector2 origin, float
rotation, Color tint); // Draw a part of a texture defined by a rectangle with 'pro' parameters
01463 RLAPI void DrawTextureNPatch(Texture2D texture, NPatchInfo nPatchInfo, Rectangle dest, Vector2 origin,
float rotation, Color tint); // Draws a texture (or part of it) that stretches or shrinks nicely
01464
01465 // Color/pixel related functions
01466 RLAPI bool ColorIsEqual(Color col1, Color col2); // Check if two colors are
equal
01467 RLAPI Color Fade(Color color, float alpha); // Get color with alpha
applied, alpha goes from 0.0f to 1.0f
01468 RLAPI int ColorToInt(Color color); // Get hexadecimal value
for a Color (0xRRGGBBAA)
01469 RLAPI Vector4 ColorNormalize(Color color); // Get Color normalized as
float [0..1]
01470 RLAPI Color ColorFromNormalized(Vector4 normalized); // Get Color from
normalized values [0..1]
01471 RLAPI Vector3 ColorToHSV(Color color); // Get HSV values for a
Color, hue [0..360], saturation/value [0..1]
01472 RLAPI Color ColorFromHSV(float hue, float saturation, float value); // Get a Color from HSV
values, hue [0..360], saturation/value [0..1]
01473 RLAPI Color ColorTint(Color color, Color tint); // Get color multiplied
with another color
01474 RLAPI Color ColorBrightness(Color color, float factor); // Get color with
brightness correction, brightness factor goes from -1.0f to 1.0f
01475 RLAPI Color ColorContrast(Color color, float contrast); // Get color with contrast
correction, contrast values between -1.0f and 1.0f
01476 RLAPI Color ColorAlpha(Color color, float alpha); // Get color with alpha
applied, alpha goes from 0.0f to 1.0f
01477 RLAPI Color ColorAlphaBlend(Color dst, Color src, Color tint); // Get src alpha-blended
into dst color with tint
01478 RLAPI Color ColorLerp(Color color1, Color color2, float factor); // Get color lerp
interpolation between two colors, factor [0.0f..1.0f]
01479 RLAPI Color GetColor(unsigned int hexValue); // Get Color structure
from hexadecimal value
01480 RLAPI Color GetPixelColor(void *srcPtr, int format); // Get Color from a source
pixel pointer of certain format
01481 RLAPI void SetPixelColor(void *dstPtr, Color color, int format); // Set color formatted
into destination pixel pointer
01482 RLAPI int GetPixelDataSize(int width, int height, int format); // Get pixel data size in
bytes for certain format
01483
01484 //-----
01485 // Font Loading and Text Drawing Functions (Module: text)
01486 //-----
01487
01488 // Font loading/unloading functions
01489 RLAPI Font GetFontDefault(void); // Get the
default Font
01490 RLAPI Font LoadFont(const char *fileName); // Load
font from file into GPU memory (VRAM)
01491 RLAPI Font LoadFontEx(const char *fileName, int fontSize, const int *codepoints, int codepointCount);
// Load font from file with extended parameters, use NULL for codepoints and 0 for codepointCount to
load the default character set, font size is provided in pixels height
01492 RLAPI Font LoadFontFromImage(Image image, Color key, int firstChar); // Load
font from Image (XNA style)
01493 RLAPI Font LoadFontFromMemory(const char *fileName, const unsigned char *fileData, int dataSize, int
fontSize, const int *codepoints, int codepointCount); // Load font from memory buffer, fileName refers
to extension: i.e. '.ttf'
01494 RLAPI bool IsFontValid(Font font); // Check
if a font is valid (font data loaded, WARNING: GPU texture not checked)
01495 RLAPI GlyphInfo *LoadFontData(const unsigned char *fileData, int dataSize, int fontSize, const int
*codepoints, int codepointCount, int type, int *glyphCount); // Load font data for further use
01496 RLAPI Image GenImageFontAtlas(const GlyphInfo *glyphs, Rectangle *glyphRecs, int glyphCount, int
fontSize, int padding, int packMethod); // Generate image font atlas using chars info
01497 RLAPI void UnloadFontData(GlyphInfo *glyphs, int glyphCount); // Unload
font chars info data (RAM)
01498 RLAPI void UnloadFont(Font font); // Unload
font from GPU memory (VRAM)
01499 RLAPI bool ExportFontAsCode(Font font, const char *fileName); // Export
font as code file, returns true on success
01500
01501 // Text drawing functions
01502 RLAPI void DrawFPS(int posX, int posY); // Draw
current FPS
01503 RLAPI void DrawText(const char *text, int posX, int posY, int fontSize, Color color); // Draw
text (using default font)

```

```

01504 RLAPI void DrawTextEx(Font font, const char *text, Vector2 position, float fontSize, float spacing,
01505 Color tint); // Draw text using font and additional parameters
01506 RLAPI void DrawTextPro(Font font, const char *text, Vector2 position, Vector2 origin, float rotation,
01507 float fontSize, float spacing, Color tint); // Draw text using Font and pro parameters (rotation)
01508 RLAPI void DrawTextCodepoint(Font font, int codepoint, Vector2 position, float fontSize, Color tint);
01509 // Draw one character (codepoint)
01510 RLAPI void DrawTextCodepoints(Font font, const int *codepoints, int codepointCount, Vector2 position,
01511 float fontSize, float spacing, Color tint); // Draw multiple character (codepoint)
01512
01513 // Text font info functions
01514 RLAPI void SetTextLineSpacing(int spacing); // Set
01515 // vertical line spacing when drawing with line-breaks
01516 RLAPI int MeasureText(const char *text, int fontSize); // Measure
01517 // string width for default font
01518 RLAPI Vector2 MeasureTextEx(Font font, const char *text, float fontSize, float spacing); // Measure
01519 // string size for Font
01520 RLAPI Vector2 MeasureTextCodepoints(Font font, const int *codepoints, int length, float fontSize,
01521 float spacing); // Measure string size for an existing array of codepoints for Font
01522 RLAPI int GetGlyphIndex(Font font, int codepoint); // Get
01523 // glyph index position in font for a codepoint (unicode character), fallback to '?' if not found
01524 RLAPI GlyphInfo GetGlyphInfo(Font font, int codepoint); // Get
01525 // glyph font info data for a codepoint (unicode character), fallback to '?' if not found
01526 RLAPI Rectangle GetGlyphAtlasRec(Font font, int codepoint); // Get
01527 // glyph rectangle in font atlas for a codepoint (unicode character), fallback to '?' if not found
01528
01529 // Text codepoints management functions (unicode characters)
01530 RLAPI char *LoadUTF8(const int *codepoints, int length); // Load
01531 // UTF-8 text encoded from codepoints array
01532 RLAPI void UnloadUTF8(char *text); // Unload
01533 // UTF-8 text encoded from codepoints array
01534 RLAPI int *LoadCodepoints(const char *text, int *count); // Load
01535 // all codepoints from a UTF-8 text string, codepoints count returned by parameter
01536 RLAPI void UnloadCodepoints(int *codepoints); // Unload
01537 // codepoints data from memory
01538 RLAPI int GetCodepointCount(const char *text); // Get
01539 // total number of codepoints in a UTF-8 encoded string
01540 RLAPI int GetCodepoint(const char *text, int *codepointSize); // Get
01541 // next codepoint in a UTF-8 encoded string, 0x3f('?') is returned on failure
01542 RLAPI int GetCodepointNext(const char *text, int *codepointSize); // Get
01543 // next codepoint in a UTF-8 encoded string, 0x3f('?') is returned on failure
01544 RLAPI int GetCodepointPrevious(const char *text, int *codepointSize); // Get
01545 // previous codepoint in a UTF-8 encoded string, 0x3f('?') is returned on failure
01546 RLAPI const char *CodepointToUTF8(int codepoint, int *utf8Size); // Encode
01547 // one codepoint into UTF-8 byte array (array length returned as parameter)
01548
01549 // Text strings management functions (no UTF-8 strings, only byte chars)
01550 // WARNING 1: Most of these functions use internal static buffers[], it's recommended to store
01551 // returned data on user-side for re-use
01552 // WARNING 2: Some functions allocate memory internally for the returned strings, those strings must
01553 // be freed by user using MemFree()
01554 RLAPI char **LoadTextLines(const char *text, int *count); // Load
01555 // text as separate lines ('\n')
01556 RLAPI void UnloadTextLines(char **text, int lineCount); // Unload
01557 // text lines
01558 RLAPI int TextCopy(char *dst, const char *src); // Copy
01559 // one string to another, returns bytes copied
01560 RLAPI bool TextIsEqual(const char *text1, const char *text2); // Check
01561 // if two text string are equal
01562 RLAPI unsigned int TextLength(const char *text); // Get
01563 // text length, checks for '\0' ending
01564 RLAPI const char *TextFormat(const char *text, ...); // Text
01565 // formatting with variables (sprintf() style)
01566 RLAPI const char *TextSubtext(const char *text, int position, int length); // Get a
01567 // piece of a text string
01568 RLAPI const char *TextRemoveSpaces(const char *text); // Remove
01569 // text spaces, concat words
01570 RLAPI char *GetTextBetween(const char *text, const char *begin, const char *end); // Get
01571 // text between two strings
01572 RLAPI char *TextReplace(const char *text, const char *search, const char *replacement); // Replace
01573 // text string with new string
01574 RLAPI char *TextReplaceAlloc(const char *text, const char *search, const char *replacement); //
01575 // Replace text string with new string, memory must be MemFree()
01576 RLAPI char *TextReplaceBetween(const char *text, const char *begin, const char *end, const char
01577 *replacement); // Replace text between two specific strings
01578 RLAPI char *TextReplaceBetweenAlloc(const char *text, const char *begin, const char *end, const char
01579 *replacement); // Replace text between two specific strings, memory must be MemFree()
01580 RLAPI char *TextInsert(const char *text, const char *insert, int position); // Insert
01581 // text in a defined byte position
01582 RLAPI char *TextInsertAlloc(const char *text, const char *insert, int position); // Insert
01583 // text in a defined byte position, memory must be MemFree()
01584 RLAPI char *TextJoin(char **textList, int count, const char *delimiter); // Join
01585 // text strings with delimiter
01586 RLAPI char **TextSplit(const char *text, char delimiter, int *count); // Split
01587 // text into multiple strings, using MAX_TEXTSPLIT_COUNT static strings
01588 RLAPI void TextAppend(char *text, const char *append, int *position); // Append
01589 // text at specific position and move cursor
01590 RLAPI int TextFindIndex(const char *text, const char *search); // Find

```

```

first text occurrence within a string, -1 if not found
01551 RLAPI char *TextToUpper(const char *text); // Get
upper case version of provided string
01552 RLAPI char *TextToLower(const char *text); // Get
lower case version of provided string
01553 RLAPI char *TextToPascal(const char *text); // Get
Pascal case notation version of provided string
01554 RLAPI char *TextToSnake(const char *text); // Get
Snake case notation version of provided string
01555 RLAPI char *TextToCamel(const char *text); // Get
Camel case notation version of provided string
01556 RLAPI int TextToInteger(const char *text); // Get
integer value from text
01557 RLAPI float TextToFloat(const char *text); // Get
float value from text
01558
01559 //-----
01560 // Basic 3d Shapes Drawing Functions (Module: models)
01561 //-----
01562
01563 // Basic geometric 3D shapes drawing functions
01564 RLAPI void DrawLine3D(Vector3 startPos, Vector3 endPos, Color color);
// Draw a line in 3D world space
01565 RLAPI void DrawPoint3D(Vector3 position, Color color);
// Draw a point in 3D space, actually a small line
01566 RLAPI void DrawCircle3D(Vector3 center, float radius, Vector3 rotationAxis, float rotationAngle, Color
color); // Draw a circle in 3D world space
01567 RLAPI void DrawTriangle3D(Vector3 v1, Vector3 v2, Vector3 v3, Color color);
// Draw a color-filled triangle (vertex in counter-clockwise order!)
01568 RLAPI void DrawTriangleStrip3D(const Vector3 *points, int pointCount, Color color);
// Draw a triangle strip defined by points
01569 RLAPI void DrawCube(Vector3 position, float width, float height, float length, Color color);
// Draw cube
01570 RLAPI void DrawCubeV(Vector3 position, Vector3 size, Color color);
// Draw cube (Vector version)
01571 RLAPI void DrawCubeWires(Vector3 position, float width, float height, float length, Color color);
// Draw cube wires
01572 RLAPI void DrawCubeWiresV(Vector3 position, Vector3 size, Color color);
// Draw cube wires (Vector version)
01573 RLAPI void DrawSphere(Vector3 centerPos, float radius, Color color);
// Draw sphere
01574 RLAPI void DrawSphereEx(Vector3 centerPos, float radius, int rings, int slices, Color color);
// Draw sphere with extended parameters
01575 RLAPI void DrawSphereWires(Vector3 centerPos, float radius, int rings, int slices, Color color);
// Draw sphere wires
01576 RLAPI void DrawCylinder(Vector3 position, float radiusTop, float radiusBottom, float height, int
slices, Color color); // Draw a cylinder/cone
01577 RLAPI void DrawCylinderEx(Vector3 startPos, Vector3 endPos, float startRadius, float endRadius, int
sides, Color color); // Draw a cylinder with base at startPos and top at endPos
01578 RLAPI void DrawCylinderWires(Vector3 position, float radiusTop, float radiusBottom, float height, int
slices, Color color); // Draw a cylinder/cone wires
01579 RLAPI void DrawCylinderWiresEx(Vector3 startPos, Vector3 endPos, float startRadius, float endRadius,
int sides, Color color); // Draw a cylinder wires with base at startPos and top at endPos
01580 RLAPI void DrawCapsule(Vector3 startPos, Vector3 endPos, float radius, int slices, int rings, Color
color); // Draw a capsule with the center of its sphere caps at startPos and endPos
01581 RLAPI void DrawCapsuleWires(Vector3 startPos, Vector3 endPos, float radius, int slices, int rings,
Color color); // Draw capsule wireframe with the center of its sphere caps at startPos and endPos
01582 RLAPI void DrawPlane(Vector3 centerPos, Vector2 size, Color color);
// Draw a plane XZ
01583 RLAPI void DrawRay(Ray ray, Color color);
// Draw a ray line
01584 RLAPI void DrawGrid(int slices, float spacing);
// Draw a grid (centered at (0, 0, 0))
01585
01586 //-----
01587 // Model 3d Loading and Drawing Functions (Module: models)
01588 //-----
01589
01590 // Model management functions
01591 RLAPI Model LoadModel(const char *fileName); // Load
model from files (meshes and materials)
01592 RLAPI Model LoadModelFromMesh(Mesh mesh); // Load
model from generated mesh (default material)
01593 RLAPI bool IsModelValid(Model model); // Check
if a model is valid (loaded in GPU, VAO/VBOs)
01594 RLAPI void UnloadModel(Model model); // Unload
model (including meshes) from memory (RAM and/or VRAM)
01595 RLAPI BoundingBox GetModelBoundingBox(Model model); // Compute
model bounding box limits (considers all meshes)
01596
01597 // Model drawing functions
01598 RLAPI void DrawModel(Model model, Vector3 position, float scale, Color tint); // Draw a
model (with texture if set)
01599 RLAPI void DrawModelEx(Model model, Vector3 position, Vector3 rotationAxis, float rotationAngle,
Vector3 scale, Color tint); // Draw a model with extended parameters
01600 RLAPI void DrawModelWires(Model model, Vector3 position, float scale, Color tint); // Draw a
model wires (with texture if set)

```

```

01601 RLAPI void DrawModelWiresEx(Model model, Vector3 position, Vector3 rotationAxis, float rotationAngle,
01602 Vector3 scale, Color tint); // Draw a model wires (with texture if set) with extended parameters
01602 RLAPI void DrawBoundingBox(BoundingBox box, Color color); // Draw
bounding box (wires)
01603 RLAPI void DrawBillboard(Camera camera, Texture2D texture, Vector3 position, float scale, Color tint);
// Draw a billboard texture
01604 RLAPI void DrawBillboardRec(Camera camera, Texture2D texture, Rectangle source, Vector3 position,
Vector2 size, Color tint); // Draw a billboard texture defined by source
01605 RLAPI void DrawBillboardPro(Camera camera, Texture2D texture, Rectangle source, Vector3 position,
Vector3 up, Vector2 size, Vector2 origin, float rotation, Color tint); // Draw a billboard texture
defined by source and rotation
01606
01607 // Mesh management functions
01608 RLAPI void UploadMesh(Mesh *mesh, bool dynamic); // Upload
mesh vertex data in GPU and provide VAO/VBO ids
01609 RLAPI void UpdateMeshBuffer(Mesh mesh, int index, const void *data, int dataSize, int offset); //
Update mesh vertex data in GPU for a specific buffer index
01610 RLAPI void UnloadMesh(Mesh mesh); // Unload
mesh data from CPU and GPU
01611 RLAPI void DrawMesh(Mesh mesh, Material material, Matrix transform); // Draw a
3d mesh with material and transform
01612 RLAPI void DrawMeshInstanced(Mesh mesh, Material material, const Matrix *transforms, int instances);
// Draw multiple mesh instances with material and different transforms
01613 RLAPI BoundingBox GetMeshBoundingBox(Mesh mesh); // Compute
mesh bounding box limits
01614 RLAPI void GenMeshTangents(Mesh *mesh); // Compute
mesh tangents
01615 RLAPI bool ExportMesh(Mesh mesh, const char *fileName); // Export
mesh data to file, returns true on success
01616 RLAPI bool ExportMeshAsCode(Mesh mesh, const char *fileName); // Export
mesh as code file (.h) defining multiple arrays of vertex attributes
01617
01618 // Mesh generation functions
01619 RLAPI Mesh GenMeshPoly(int sides, float radius); //
Generate polygonal mesh
01620 RLAPI Mesh GenMeshPlane(float width, float length, int resX, int resZ); //
Generate plane mesh (with subdivisions)
01621 RLAPI Mesh GenMeshCube(float width, float height, float length); //
Generate cuboid mesh
01622 RLAPI Mesh GenMeshSphere(float radius, int rings, int slices); //
Generate sphere mesh (standard sphere)
01623 RLAPI Mesh GenMeshHemiSphere(float radius, int rings, int slices); //
Generate half-sphere mesh (no bottom cap)
01624 RLAPI Mesh GenMeshCylinder(float radius, float height, int slices); //
Generate cylinder mesh
01625 RLAPI Mesh GenMeshCone(float radius, float height, int slices); //
Generate cone/pyramid mesh
01626 RLAPI Mesh GenMeshTorus(float radius, float size, int radSeg, int sides); //
Generate torus mesh
01627 RLAPI Mesh GenMeshKnot(float radius, float size, int radSeg, int sides); //
Generate trefoil knot mesh
01628 RLAPI Mesh GenMeshHeightmap(Image heightmap, Vector3 size); //
Generate heightmap mesh from image data
01629 RLAPI Mesh GenMeshCubicmap(Image cubicmap, Vector3 cubeSize); //
Generate cubes-based map mesh from image data
01630
01631 // Material loading/unloading functions
01632 RLAPI Material *LoadMaterials(const char *fileName, int *materialCount); // Load
materials from model file
01633 RLAPI Material LoadMaterialDefault(void); // Load
default material (Supports: DIFFUSE, SPECULAR, NORMAL maps)
01634 RLAPI bool IsMaterialValid(Material material); // Check
if a material is valid (shader assigned, map textures loaded in GPU)
01635 RLAPI void UnloadMaterial(Material material); // Unload
material from GPU memory (VRAM)
01636 RLAPI void SetMaterialTexture(Material *material, int mapType, Texture2D texture); // Set
texture for a material map type (MATERIAL_MAP_DIFFUSE, MATERIAL_MAP_SPECULAR...)
01637 RLAPI void SetModelMeshMaterial(Model *model, int meshId, int materialId); // Set
material for a mesh
01638
01639 // Model animations loading/unloading functions
01640 RLAPI ModelAnimation *LoadModelAnimations(const char *fileName, int *animCount); // Load
model animations from file
01641 RLAPI void UpdateModelAnimation(Model model, ModelAnimation anim, float frame); // Update
model animation pose (vertex buffers and bone matrices)
01642 RLAPI void UpdateModelAnimationEx(Model model, ModelAnimation animA, float frameA, ModelAnimation
animB, float frameB, float blend); // Update model animation pose, blending two animations
01643 RLAPI void UnloadModelAnimations(ModelAnimation *animations, int animCount); // Unload
animation array data
01644 RLAPI bool IsModelAnimationValid(Model model, ModelAnimation anim); // Check
model animation skeleton match
01645
01646 // Collision detection functions
01647 RLAPI bool CheckCollisionSpheres(Vector3 center1, float radius1, Vector3 center2, float radius2); //
Check collision between two spheres
01648 RLAPI bool CheckCollisionBoxes(BoundingBox box1, BoundingBox box2); // Check
collision between two bounding boxes

```

```

01649 RLAPI bool CheckCollisionBoxSphere(BoundingBox box, Vector3 center, float radius); // Check
collision between box and sphere
01650 RLAPI RayCollision GetRayCollisionSphere(Ray ray, Vector3 center, float radius); // Get
collision info between ray and sphere
01651 RLAPI RayCollision GetRayCollisionBox(Ray ray, BoundingBox box); // Get
collision info between ray and box
01652 RLAPI RayCollision GetRayCollisionMesh(Ray ray, Mesh mesh, Matrix transform); // Get
collision info between ray and mesh
01653 RLAPI RayCollision GetRayCollisionTriangle(Ray ray, Vector3 p1, Vector3 p2, Vector3 p3); // Get
collision info between ray and triangle
01654 RLAPI RayCollision GetRayCollisionQuad(Ray ray, Vector3 p1, Vector3 p2, Vector3 p3, Vector3 p4); //
Get collision info between ray and quad
01655
01656 //-----
01657 // Audio Loading and Playing Functions (Module: audio)
01658 //-----
01659 typedef void (*AudioCallback)(void *bufferData, unsigned int frames);
01660
01661 // Audio device management functions
01662 RLAPI void InitAudioDevice(void); // Initialize audio device and
context
01663 RLAPI void CloseAudioDevice(void); // Close the audio device and
context
01664 RLAPI bool IsAudioDeviceReady(void); // Check if audio device has
been initialized successfully
01665 RLAPI void SetMasterVolume(float volume); // Set master volume (listener)
01666 RLAPI float GetMasterVolume(void); // Get master volume (listener)
01667
01668 // Wave/Sound loading/unloading functions
01669 RLAPI Wave LoadWave(const char *fileName); // Load wave data from file
01670 RLAPI Wave LoadWaveFromMemory(const char *fileType, const unsigned char *fileData, int dataSize); //
Load wave from memory buffer, fileType refers to extension: i.e. '.wav'
01671 RLAPI bool IsWaveValid(Wave wave); // Checks if wave data is valid
(data loaded and parameters)
01672 RLAPI Sound LoadSound(const char *fileName); // Load sound from file
01673 RLAPI Sound LoadSoundFromWave(Wave wave); // Load sound from wave data
01674 RLAPI Sound LoadSoundAlias(Sound source); // Create a new sound that
shares the same sample data as the source sound, does not own the sound data
01675 RLAPI bool IsSoundValid(Sound sound); // Checks if a sound is valid
(data loaded and buffers initialized)
01676 RLAPI void UpdateSound(Sound sound, const void *data, int sampleCount); // Update sound buffer with
new data (default data format: 32 bit float, stereo)
01677 RLAPI void UnloadWave(Wave wave); // Unload wave data
01678 RLAPI void UnloadSound(Sound sound); // Unload sound
01679 RLAPI void UnloadSoundAlias(Sound alias); // Unload a sound alias (does
not deallocate sample data)
01680 RLAPI bool ExportWave(Wave wave, const char *fileName); // Export wave data to file,
returns true on success
01681 RLAPI bool ExportWaveAsCode(Wave wave, const char *fileName); // Export wave sample data to
code (.h), returns true on success
01682
01683 // Wave/Sound management functions
01684 RLAPI void PlaySound(Sound sound); // Play a sound
01685 RLAPI void StopSound(Sound sound); // Stop playing a sound
01686 RLAPI void PauseSound(Sound sound); // Pause a sound
01687 RLAPI void ResumeSound(Sound sound); // Resume a paused sound
01688 RLAPI bool IsSoundPlaying(Sound sound); // Check if a sound is currently
playing
01689 RLAPI void SetSoundVolume(Sound sound, float volume); // Set volume for a sound (1.0
is max level)
01690 RLAPI void SetSoundPitch(Sound sound, float pitch); // Set pitch for a sound (1.0 is
base level)
01691 RLAPI void SetSoundPan(Sound sound, float pan); // Set pan for a sound (-1.0
left, 0.0 center, 1.0 right)
01692 RLAPI Wave WaveCopy(Wave wave); // Copy a wave to a new wave
01693 RLAPI void WaveCrop(Wave *wave, int initFrame, int finalFrame); // Crop a wave to defined frames
range
01694 RLAPI void WaveFormat(Wave *wave, int sampleRate, int sampleSize, int channels); // Convert wave data
to desired format
01695 RLAPI float *LoadWaveSamples(Wave wave); // Load samples data from wave
as a 32bit float data array
01696 RLAPI void UnloadWaveSamples(float *samples); // Unload samples data loaded
with LoadWaveSamples()
01697
01698 // Music management functions
01699 RLAPI Music LoadMusicStream(const char *fileName); // Load music stream from file
01700 RLAPI Music LoadMusicStreamFromMemory(const char *fileType, const unsigned char *data, int dataSize); //
Load music stream from data
01701 RLAPI bool IsMusicValid(Music music); // Checks if a music stream is
valid (context and buffers initialized)
01702 RLAPI void UnloadMusicStream(Music music); // Unload music stream
01703 RLAPI void PlayMusicStream(Music music); // Start music playing
01704 RLAPI bool IsMusicStreamPlaying(Music music); // Check if music is playing
01705 RLAPI void UpdateMusicStream(Music music); // Updates buffers for music
streaming
01706 RLAPI void StopMusicStream(Music music); // Stop music playing
01707 RLAPI void PauseMusicStream(Music music); // Pause music playing

```



```

01708 RLAPI void ResumeMusicStream(Music music); // Resume playing paused music
01709 RLAPI void SeekMusicStream(Music music, float position); // Seek music to a position (in
seconds)
01710 RLAPI void SetMusicVolume(Music music, float volume); // Set volume for music (1.0 is
max level)
01711 RLAPI void SetMusicPitch(Music music, float pitch); // Set pitch for a music (1.0 is
base level)
01712 RLAPI void SetMusicPan(Music music, float pan); // Set pan for a music (-1.0
left, 0.0 center, 1.0 right)
01713 RLAPI float GetMusicTimeLength(Music music); // Get music time length (in
seconds)
01714 RLAPI float GetMusicTimePlayed(Music music); // Get current music time played
(in seconds)
01715
01716 // AudioStream management functions
01717 RLAPI AudioStream LoadAudioStream(unsigned int sampleRate, unsigned int sampleSize, unsigned int
channels); // Load audio stream (to stream raw audio pcm data)
01718 RLAPI bool IsAudioStreamValid(AudioStream stream); // Checks if an audio stream is
valid (buffers initialized)
01719 RLAPI void UnloadAudioStream(AudioStream stream); // Unload audio stream and free
memory
01720 RLAPI void UpdateAudioStream(AudioStream stream, const void *data, int frameCount); // Update audio
stream buffers with data
01721 RLAPI bool IsAudioStreamProcessed(AudioStream stream); // Check if any audio stream
buffers requires refill
01722 RLAPI void PlayAudioStream(AudioStream stream); // Play audio stream
01723 RLAPI void PauseAudioStream(AudioStream stream); // Pause audio stream
01724 RLAPI void ResumeAudioStream(AudioStream stream); // Resume audio stream
01725 RLAPI bool IsAudioStreamPlaying(AudioStream stream); // Check if audio stream is
playing
01726 RLAPI void StopAudioStream(AudioStream stream); // Stop audio stream
01727 RLAPI void SetAudioStreamVolume(AudioStream stream, float volume); // Set volume for audio stream
(1.0 is max level)
01728 RLAPI void SetAudioStreamPitch(AudioStream stream, float pitch); // Set pitch for audio stream
(1.0 is base level)
01729 RLAPI void SetAudioStreamPan(AudioStream stream, float pan); // Set pan for audio stream
(-1.0 to 1.0 range, 0.0 is centered)
01730 RLAPI void SetAudioStreamBufferSizeDefault(int size); // Default size for new audio
streams
01731 RLAPI void SetAudioStreamCallback(AudioStream stream, AudioCallback callback); // Audio thread
callback to request new data
01732
01733 RLAPI void AttachAudioStreamProcessor(AudioStream stream, AudioCallback processor); // Attach audio
stream processor to stream, receives frames x 2 samples as 'float' (stereo)
01734 RLAPI void DetachAudioStreamProcessor(AudioStream stream, AudioCallback processor); // Detach audio
stream processor from stream
01735
01736 RLAPI void AttachAudioMixedProcessor(AudioCallback processor); // Attach audio stream processor to the
entire audio pipeline, receives frames x 2 samples as 'float' (stereo)
01737 RLAPI void DetachAudioMixedProcessor(AudioCallback processor); // Detach audio stream processor from
the entire audio pipeline
01738
01739 #if defined(__cplusplus)
01740 }
01741 #endif
01742
01743 #endif // RAYLIB_H

```

6.25 lib/raylib/include/raymath.h File Reference

```
#include <math.h>
```

Data Structures

- struct [Vector2](#)
- struct [Vector3](#)
- struct [Vector4](#)
- struct [Matrix](#)
- struct [float3](#)
- struct [float16](#)

Macros

- `#define` [RMAPI](#) inline
- `#define` [PI](#) 3.14159265358979323846f
- `#define` [EPSILON](#) 0.000001f
- `#define` [DEG2RAD](#) ([PI](#)/180.0f)
- `#define` [RAD2DEG](#) (180.0f/[PI](#))
- `#define` [MatrixToFloat](#)(mat)
- `#define` [Vector3ToFloat](#)(vec)
- `#define` [RL_VECTOR2_TYPE](#)
- `#define` [RL_VECTOR3_TYPE](#)
- `#define` [RL_VECTOR4_TYPE](#)
- `#define` [RL_QUATERNION_TYPE](#)
- `#define` [RL_MATRIX_TYPE](#)
- `#define` [RL_FLOAT3_TYPE](#)
- `#define` [RL_FLOAT16_TYPE](#)

Typedefs

- `typedef struct Vector2` [Vector2](#)
- `typedef struct Vector3` [Vector3](#)
- `typedef struct Vector4` [Vector4](#)
- `typedef struct Matrix` [Matrix](#)
- `typedef struct float3` [float3](#)
- `typedef struct float16` [float16](#)

Functions

- [RMAPI](#) float [Clamp](#) (float value, float min, float max)
- [RMAPI](#) float [Lerp](#) (float start, float end, float amount)
- [RMAPI](#) float [Normalize](#) (float value, float start, float end)
- [RMAPI](#) float [Remap](#) (float value, float inputStart, float inputEnd, float outputStart, float outputEnd)
- [RMAPI](#) float [Wrap](#) (float value, float min, float max)
- [RMAPI](#) int [FloatEquals](#) (float x, float y)
- [RMAPI](#) [Vector2](#) [Vector2Zero](#) (void)
- [RMAPI](#) [Vector2](#) [Vector2One](#) (void)
- [RMAPI](#) [Vector2](#) [Vector2Add](#) ([Vector2](#) v1, [Vector2](#) v2)
- [RMAPI](#) [Vector2](#) [Vector2AddValue](#) ([Vector2](#) v, float add)
- [RMAPI](#) [Vector2](#) [Vector2Subtract](#) ([Vector2](#) v1, [Vector2](#) v2)
- [RMAPI](#) [Vector2](#) [Vector2SubtractValue](#) ([Vector2](#) v, float sub)
- [RMAPI](#) float [Vector2Length](#) ([Vector2](#) v)
- [RMAPI](#) float [Vector2LengthSqr](#) ([Vector2](#) v)
- [RMAPI](#) float [Vector2DotProduct](#) ([Vector2](#) v1, [Vector2](#) v2)
- [RMAPI](#) float [Vector2CrossProduct](#) ([Vector2](#) v1, [Vector2](#) v2)
- [RMAPI](#) float [Vector2Distance](#) ([Vector2](#) v1, [Vector2](#) v2)
- [RMAPI](#) float [Vector2DistanceSqr](#) ([Vector2](#) v1, [Vector2](#) v2)
- [RMAPI](#) float [Vector2Angle](#) ([Vector2](#) v1, [Vector2](#) v2)
- [RMAPI](#) float [Vector2LineAngle](#) ([Vector2](#) start, [Vector2](#) end)
- [RMAPI](#) [Vector2](#) [Vector2Scale](#) ([Vector2](#) v, float scale)
- [RMAPI](#) [Vector2](#) [Vector2Multiply](#) ([Vector2](#) v1, [Vector2](#) v2)
- [RMAPI](#) [Vector2](#) [Vector2Negate](#) ([Vector2](#) v)
- [RMAPI](#) [Vector2](#) [Vector2Divide](#) ([Vector2](#) v1, [Vector2](#) v2)
- [RMAPI](#) [Vector2](#) [Vector2Normalize](#) ([Vector2](#) v)

- [RMAPI Vector2 Vector2Transform](#) ([Vector2](#) v, [Matrix](#) mat)
- [RMAPI Vector2 Vector2Lerp](#) ([Vector2](#) v1, [Vector2](#) v2, float amount)
- [RMAPI Vector2 Vector2Reflect](#) ([Vector2](#) v, [Vector2](#) normal)
- [RMAPI Vector2 Vector2Min](#) ([Vector2](#) v1, [Vector2](#) v2)
- [RMAPI Vector2 Vector2Max](#) ([Vector2](#) v1, [Vector2](#) v2)
- [RMAPI Vector2 Vector2Rotate](#) ([Vector2](#) v, float angle)
- [RMAPI Vector2 Vector2MoveTowards](#) ([Vector2](#) v, [Vector2](#) target, float maxDistance)
- [RMAPI Vector2 Vector2Invert](#) ([Vector2](#) v)
- [RMAPI Vector2 Vector2Clamp](#) ([Vector2](#) v, [Vector2](#) min, [Vector2](#) max)
- [RMAPI Vector2 Vector2ClampValue](#) ([Vector2](#) v, float min, float max)
- [RMAPI int Vector2Equals](#) ([Vector2](#) p, [Vector2](#) q)
- [RMAPI Vector2 Vector2Refract](#) ([Vector2](#) v, [Vector2](#) n, float r)
- [RMAPI Vector3 Vector3Zero](#) (void)
- [RMAPI Vector3 Vector3One](#) (void)
- [RMAPI Vector3 Vector3Add](#) ([Vector3](#) v1, [Vector3](#) v2)
- [RMAPI Vector3 Vector3AddValue](#) ([Vector3](#) v, float add)
- [RMAPI Vector3 Vector3Subtract](#) ([Vector3](#) v1, [Vector3](#) v2)
- [RMAPI Vector3 Vector3SubtractValue](#) ([Vector3](#) v, float sub)
- [RMAPI Vector3 Vector3Scale](#) ([Vector3](#) v, float scalar)
- [RMAPI Vector3 Vector3Multiply](#) ([Vector3](#) v1, [Vector3](#) v2)
- [RMAPI Vector3 Vector3CrossProduct](#) ([Vector3](#) v1, [Vector3](#) v2)
- [RMAPI Vector3 Vector3Perpendicular](#) ([Vector3](#) v)
- [RMAPI float Vector3Length](#) (const [Vector3](#) v)
- [RMAPI float Vector3LengthSqr](#) (const [Vector3](#) v)
- [RMAPI float Vector3DotProduct](#) ([Vector3](#) v1, [Vector3](#) v2)
- [RMAPI float Vector3Distance](#) ([Vector3](#) v1, [Vector3](#) v2)
- [RMAPI float Vector3DistanceSqr](#) ([Vector3](#) v1, [Vector3](#) v2)
- [RMAPI float Vector3Angle](#) ([Vector3](#) v1, [Vector3](#) v2)
- [RMAPI Vector3 Vector3Negate](#) ([Vector3](#) v)
- [RMAPI Vector3 Vector3Divide](#) ([Vector3](#) v1, [Vector3](#) v2)
- [RMAPI Vector3 Vector3Normalize](#) ([Vector3](#) v)
- [RMAPI Vector3 Vector3Project](#) ([Vector3](#) v1, [Vector3](#) v2)
- [RMAPI Vector3 Vector3Reject](#) ([Vector3](#) v1, [Vector3](#) v2)
- [RMAPI void Vector3OrthoNormalize](#) ([Vector3](#) *v1, [Vector3](#) *v2)
- [RMAPI Vector3 Vector3Transform](#) ([Vector3](#) v, [Matrix](#) mat)
- [RMAPI Vector3 Vector3RotateByQuaternion](#) ([Vector3](#) v, [Quaternion](#) q)
- [RMAPI Vector3 Vector3RotateByAxisAngle](#) ([Vector3](#) v, [Vector3](#) axis, float angle)
- [RMAPI Vector3 Vector3MoveTowards](#) ([Vector3](#) v, [Vector3](#) target, float maxDistance)
- [RMAPI Vector3 Vector3Lerp](#) ([Vector3](#) v1, [Vector3](#) v2, float amount)
- [RMAPI Vector3 Vector3CubicHermite](#) ([Vector3](#) v1, [Vector3](#) tangent1, [Vector3](#) v2, [Vector3](#) tangent2, float amount)
- [RMAPI Vector3 Vector3Reflect](#) ([Vector3](#) v, [Vector3](#) normal)
- [RMAPI Vector3 Vector3Min](#) ([Vector3](#) v1, [Vector3](#) v2)
- [RMAPI Vector3 Vector3Max](#) ([Vector3](#) v1, [Vector3](#) v2)
- [RMAPI Vector3 Vector3Barycenter](#) ([Vector3](#) p, [Vector3](#) a, [Vector3](#) b, [Vector3](#) c)
- [RMAPI Vector3 Vector3Unproject](#) ([Vector3](#) source, [Matrix](#) projection, [Matrix](#) view)
- [RMAPI float3 Vector3ToFloatV](#) ([Vector3](#) v)
- [RMAPI Vector3 Vector3Invert](#) ([Vector3](#) v)
- [RMAPI Vector3 Vector3Clamp](#) ([Vector3](#) v, [Vector3](#) min, [Vector3](#) max)
- [RMAPI Vector3 Vector3ClampValue](#) ([Vector3](#) v, float min, float max)
- [RMAPI int Vector3Equals](#) ([Vector3](#) p, [Vector3](#) q)
- [RMAPI Vector3 Vector3Refract](#) ([Vector3](#) v, [Vector3](#) n, float r)
- [RMAPI Vector4 Vector4Zero](#) (void)
- [RMAPI Vector4 Vector4One](#) (void)
- [RMAPI Vector4 Vector4Add](#) ([Vector4](#) v1, [Vector4](#) v2)

- [RMAPI Vector4 Vector4AddValue](#) ([Vector4](#) v, float add)
- [RMAPI Vector4 Vector4Subtract](#) ([Vector4](#) v1, [Vector4](#) v2)
- [RMAPI Vector4 Vector4SubtractValue](#) ([Vector4](#) v, float add)
- [RMAPI float Vector4Length](#) ([Vector4](#) v)
- [RMAPI float Vector4LengthSqr](#) ([Vector4](#) v)
- [RMAPI float Vector4DotProduct](#) ([Vector4](#) v1, [Vector4](#) v2)
- [RMAPI float Vector4Distance](#) ([Vector4](#) v1, [Vector4](#) v2)
- [RMAPI float Vector4DistanceSqr](#) ([Vector4](#) v1, [Vector4](#) v2)
- [RMAPI Vector4 Vector4Scale](#) ([Vector4](#) v, float scale)
- [RMAPI Vector4 Vector4Multiply](#) ([Vector4](#) v1, [Vector4](#) v2)
- [RMAPI Vector4 Vector4Negate](#) ([Vector4](#) v)
- [RMAPI Vector4 Vector4Divide](#) ([Vector4](#) v1, [Vector4](#) v2)
- [RMAPI Vector4 Vector4Normalize](#) ([Vector4](#) v)
- [RMAPI Vector4 Vector4Min](#) ([Vector4](#) v1, [Vector4](#) v2)
- [RMAPI Vector4 Vector4Max](#) ([Vector4](#) v1, [Vector4](#) v2)
- [RMAPI Vector4 Vector4Lerp](#) ([Vector4](#) v1, [Vector4](#) v2, float amount)
- [RMAPI Vector4 Vector4MoveTowards](#) ([Vector4](#) v, [Vector4](#) target, float maxDistance)
- [RMAPI Vector4 Vector4Invert](#) ([Vector4](#) v)
- [RMAPI int Vector4Equals](#) ([Vector4](#) p, [Vector4](#) q)
- [RMAPI float MatrixDeterminant](#) ([Matrix](#) mat)
- [RMAPI float MatrixTrace](#) ([Matrix](#) mat)
- [RMAPI Matrix MatrixTranspose](#) ([Matrix](#) mat)
- [RMAPI Matrix MatrixInvert](#) ([Matrix](#) mat)
- [RMAPI Matrix MatrixIdentity](#) (void)
- [RMAPI Matrix MatrixAdd](#) ([Matrix](#) left, [Matrix](#) right)
- [RMAPI Matrix MatrixSubtract](#) ([Matrix](#) left, [Matrix](#) right)
- [RMAPI Matrix MatrixMultiply](#) ([Matrix](#) left, [Matrix](#) right)
- [RMAPI Matrix MatrixMultiplyValue](#) ([Matrix](#) left, float value)
- [RMAPI Matrix MatrixTranslate](#) (float x, float y, float z)
- [RMAPI Matrix MatrixRotate](#) ([Vector3](#) axis, float angle)
- [RMAPI Matrix MatrixRotateX](#) (float angle)
- [RMAPI Matrix MatrixRotateY](#) (float angle)
- [RMAPI Matrix MatrixRotateZ](#) (float angle)
- [RMAPI Matrix MatrixRotateXYZ](#) ([Vector3](#) angle)
- [RMAPI Matrix MatrixRotateZYX](#) ([Vector3](#) angle)
- [RMAPI Matrix MatrixScale](#) (float x, float y, float z)
- [RMAPI Matrix MatrixFrustum](#) (double left, double right, double bottom, double top, double nearPlane, double farPlane)
- [RMAPI Matrix MatrixPerspective](#) (double fovY, double aspect, double nearPlane, double farPlane)
- [RMAPI Matrix MatrixOrtho](#) (double left, double right, double bottom, double top, double nearPlane, double farPlane)
- [RMAPI Matrix MatrixLookAt](#) ([Vector3](#) eye, [Vector3](#) target, [Vector3](#) up)
- [RMAPI float16 MatrixToFloatV](#) ([Matrix](#) mat)
- [RMAPI Quaternion QuaternionAdd](#) ([Quaternion](#) q1, [Quaternion](#) q2)
- [RMAPI Quaternion QuaternionAddValue](#) ([Quaternion](#) q, float add)
- [RMAPI Quaternion QuaternionSubtract](#) ([Quaternion](#) q1, [Quaternion](#) q2)
- [RMAPI Quaternion QuaternionSubtractValue](#) ([Quaternion](#) q, float sub)
- [RMAPI Quaternion QuaternionIdentity](#) (void)
- [RMAPI float QuaternionLength](#) ([Quaternion](#) q)
- [RMAPI Quaternion QuaternionNormalize](#) ([Quaternion](#) q)
- [RMAPI Quaternion QuaternionInvert](#) ([Quaternion](#) q)
- [RMAPI Quaternion QuaternionMultiply](#) ([Quaternion](#) q1, [Quaternion](#) q2)
- [RMAPI Quaternion QuaternionScale](#) ([Quaternion](#) q, float mul)
- [RMAPI Quaternion QuaternionDivide](#) ([Quaternion](#) q1, [Quaternion](#) q2)
- [RMAPI Quaternion QuaternionLerp](#) ([Quaternion](#) q1, [Quaternion](#) q2, float amount)

- [RMAPI Quaternion QuaternionNlerp](#) ([Quaternion](#) q1, [Quaternion](#) q2, float amount)
- [RMAPI Quaternion QuaternionSlerp](#) ([Quaternion](#) q1, [Quaternion](#) q2, float amount)
- [RMAPI Quaternion QuaternionCubicHermiteSpline](#) ([Quaternion](#) q1, [Quaternion](#) outTangent1, [Quaternion](#) q2, [Quaternion](#) inTangent2, float t)
- [RMAPI Quaternion QuaternionFromVector3ToVector3](#) ([Vector3](#) from, [Vector3](#) to)
- [RMAPI Quaternion QuaternionFromMatrix](#) ([Matrix](#) mat)
- [RMAPI Matrix QuaternionToMatrix](#) ([Quaternion](#) q)
- [RMAPI Quaternion QuaternionFromAxisAngle](#) ([Vector3](#) axis, float angle)
- [RMAPI void QuaternionToAxisAngle](#) ([Quaternion](#) q, [Vector3](#) *outAxis, float *outAngle)
- [RMAPI Quaternion QuaternionFromEuler](#) (float pitch, float yaw, float roll)
- [RMAPI Vector3 QuaternionToEuler](#) ([Quaternion](#) q)
- [RMAPI Quaternion QuaternionTransform](#) ([Quaternion](#) q, [Matrix](#) mat)
- [RMAPI int QuaternionEquals](#) ([Quaternion](#) p, [Quaternion](#) q)
- [RMAPI Matrix MatrixCompose](#) ([Vector3](#) translation, [Quaternion](#) rotation, [Vector3](#) scale)
- [RMAPI void MatrixDecompose](#) ([Matrix](#) mat, [Vector3](#) *translation, [Quaternion](#) *rotation, [Vector3](#) *scale)

6.25.1 Macro Definition Documentation

6.25.1.1 DEG2RAD

```
#define DEG2RAD (PI/180.0f)
```

6.25.1.2 EPSILON

```
#define EPSILON 0.000001f
```

6.25.1.3 MatrixToFloat

```
#define MatrixToFloat(  
    mat)
```

Value:

```
(MatrixToFloatV(mat).v)
```

6.25.1.4 PI

```
#define PI 3.14159265358979323846f
```

6.25.1.5 RAD2DEG

```
#define RAD2DEG (180.0f/PI)
```

6.25.1.6 RL_FLOAT16_TYPE

```
#define RL_FLOAT16_TYPE
```

6.25.1.7 RL_FLOAT3_TYPE

```
#define RL_FLOAT3_TYPE
```

6.25.1.8 RL_MATRIX_TYPE

```
#define RL_MATRIX_TYPE
```

6.25.1.9 RL_QUATERNION_TYPE

```
#define RL_QUATERNION_TYPE
```

6.25.1.10 RL_VECTOR2_TYPE

```
#define RL_VECTOR2_TYPE
```

6.25.1.11 RL_VECTOR3_TYPE

```
#define RL_VECTOR3_TYPE
```

6.25.1.12 RL_VECTOR4_TYPE

```
#define RL_VECTOR4_TYPE
```

6.25.1.13 RMAPI

```
#define RMAPI inline
```

6.25.1.14 Vector3ToFloat

```
#define Vector3ToFloat(  
    vec)
```

Value:

```
(Vector3ToFloatV(vec).v)
```

6.25.2 Typedef Documentation

6.25.2.1 float16

```
typedef struct float16 float16
```

6.25.2.2 float3

```
typedef struct float3 float3
```

6.25.2.3 Matrix

```
typedef struct Matrix Matrix
```

6.25.2.4 Vector2

```
typedef struct Vector2 Vector2
```

6.25.2.5 Vector3

```
typedef struct Vector3 Vector3
```

6.25.2.6 Vector4

```
typedef struct Vector4 Vector4
```

6.25.3 Function Documentation

6.25.3.1 Clamp()

```
RMAPI float Clamp (  
    float value,  
    float min,  
    float max)
```

6.25.3.2 FloatEquals()

```
RMAPI int FloatEquals (  
    float x,  
    float y)
```

6.25.3.3 Lerp()

```
RMAPI float Lerp (  
    float start,  
    float end,  
    float amount)
```

6.25.3.4 MatrixAdd()

```
RMAPI Matrix MatrixAdd (  
    Matrix left,  
    Matrix right)
```

6.25.3.5 MatrixCompose()

```
RMAPI Matrix MatrixCompose (  
    Vector3 translation,  
    Quaternion rotation,  
    Vector3 scale)
```

6.25.3.6 MatrixDecompose()

```
RMAPI void MatrixDecompose (  
    Matrix mat,  
    Vector3 * translation,  
    Quaternion * rotation,  
    Vector3 * scale)
```

6.25.3.7 MatrixDeterminant()

```
RMAPI float MatrixDeterminant (  
    Matrix mat)
```

6.25.3.8 MatrixFrustum()

```
RMAPI Matrix MatrixFrustum (  
    double left,  
    double right,  
    double bottom,  
    double top,  
    double nearPlane,  
    double farPlane)
```

6.25.3.9 MatrixIdentity()

```
RMAPI Matrix MatrixIdentity (  
    void )
```

6.25.3.10 MatrixInvert()

```
RMAPI Matrix MatrixInvert (  
    Matrix mat)
```

6.25.3.11 MatrixLookAt()

```
RMAPI Matrix MatrixLookAt (  
    Vector3 eye,  
    Vector3 target,  
    Vector3 up)
```

6.25.3.12 MatrixMultiply()

```
RMAPI Matrix MatrixMultiply (  
    Matrix left,  
    Matrix right)
```

6.25.3.13 MatrixMultiplyValue()

```
RMAPI Matrix MatrixMultiplyValue (  
    Matrix left,  
    float value)
```

6.25.3.14 MatrixOrtho()

```
RMAPI Matrix MatrixOrtho (  
    double left,  
    double right,  
    double bottom,  
    double top,  
    double nearPlane,  
    double farPlane)
```

6.25.3.15 MatrixPerspective()

```
RMAPI Matrix MatrixPerspective (  
    double fovY,  
    double aspect,  
    double nearPlane,  
    double farPlane)
```

6.25.3.16 MatrixRotate()

```
RMAPI Matrix MatrixRotate (  
    Vector3 axis,  
    float angle)
```

6.25.3.17 MatrixRotateX()

```
RMAPI Matrix MatrixRotateX (  
    float angle)
```

6.25.3.18 MatrixRotateXYZ()

```
RMAPI Matrix MatrixRotateXYZ (  
    Vector3 angle)
```

6.25.3.19 MatrixRotateY()

```
RMAPI Matrix MatrixRotateY (  
    float angle)
```

6.25.3.20 MatrixRotateZ()

```
RMAPI Matrix MatrixRotateZ (  
    float angle)
```

6.25.3.21 MatrixRotateZYX()

```
RMAPI Matrix MatrixRotateZYX (  
    Vector3 angle)
```

6.25.3.22 MatrixScale()

```
RMAPI Matrix MatrixScale (  
    float x,  
    float y,  
    float z)
```

6.25.3.23 MatrixSubtract()

```
RMAPI Matrix MatrixSubtract (  
    Matrix left,  
    Matrix right)
```

6.25.3.24 MatrixToFloatV()

```
RMAPI float16 MatrixToFloatV (  
    Matrix mat)
```

6.25.3.25 MatrixTrace()

```
RMAPI float MatrixTrace (  
    Matrix mat)
```


6.25.3.26 MatrixTranslate()

```
RMAPI Matrix MatrixTranslate (  
    float x,  
    float y,  
    float z)
```

6.25.3.27 MatrixTranspose()

```
RMAPI Matrix MatrixTranspose (  
    Matrix mat)
```

6.25.3.28 Normalize()

```
RMAPI float Normalize (  
    float value,  
    float start,  
    float end)
```

6.25.3.29 QuaternionAdd()

```
RMAPI Quaternion QuaternionAdd (  
    Quaternion q1,  
    Quaternion q2)
```

6.25.3.30 QuaternionAddValue()

```
RMAPI Quaternion QuaternionAddValue (  
    Quaternion q,  
    float add)
```

6.25.3.31 QuaternionCubicHermiteSpline()

```
RMAPI Quaternion QuaternionCubicHermiteSpline (  
    Quaternion q1,  
    Quaternion outTangent1,  
    Quaternion q2,  
    Quaternion inTangent2,  
    float t)
```

6.25.3.32 QuaternionDivide()

```
RMAPI Quaternion QuaternionDivide (  
    Quaternion q1,  
    Quaternion q2)
```

6.25.3.33 QuaternionEquals()

```
RMAPI int QuaternionEquals (  
    Quaternion p,  
    Quaternion q)
```

6.25.3.34 QuaternionFromAxisAngle()

```
RMAPI Quaternion QuaternionFromAxisAngle (  
    Vector3 axis,  
    float angle)
```

6.25.3.35 QuaternionFromEuler()

```
RMAPI Quaternion QuaternionFromEuler (  
    float pitch,  
    float yaw,  
    float roll)
```

6.25.3.36 QuaternionFromMatrix()

```
RMAPI Quaternion QuaternionFromMatrix (  
    Matrix mat)
```

6.25.3.37 QuaternionFromVector3ToVector3()

```
RMAPI Quaternion QuaternionFromVector3ToVector3 (  
    Vector3 from,  
    Vector3 to)
```

6.25.3.38 QuaternionIdentity()

```
RMAPI Quaternion QuaternionIdentity (  
    void )
```

6.25.3.39 QuaternionInvert()

```
RMAPI Quaternion QuaternionInvert (  
    Quaternion q)
```

6.25.3.40 QuaternionLength()

```
RMAPI float QuaternionLength (  
    Quaternion q)
```

6.25.3.41 QuaternionLerp()

```
RMAPI Quaternion QuaternionLerp (  
    Quaternion q1,  
    Quaternion q2,  
    float amount)
```

6.25.3.42 QuaternionMultiply()

```
RMAPI Quaternion QuaternionMultiply (  
    Quaternion q1,  
    Quaternion q2)
```

6.25.3.43 QuaternionNlerp()

```
RMAPI Quaternion QuaternionNlerp (  
    Quaternion q1,  
    Quaternion q2,  
    float amount)
```

6.25.3.44 QuaternionNormalize()

```
RMAPI Quaternion QuaternionNormalize (  
    Quaternion q)
```

6.25.3.45 QuaternionScale()

```
RMAPI Quaternion QuaternionScale (  
    Quaternion q,  
    float mul)
```

6.25.3.46 QuaternionSlerp()

```
RMAPI Quaternion QuaternionSlerp (  
    Quaternion q1,  
    Quaternion q2,  
    float amount)
```

6.25.3.47 QuaternionSubtract()

```
RMAPI Quaternion QuaternionSubtract (  
    Quaternion q1,  
    Quaternion q2)
```

6.25.3.48 QuaternionSubtractValue()

```
RMAPI Quaternion QuaternionSubtractValue (  
    Quaternion q,  
    float sub)
```

6.25.3.49 QuaternionToAxisAngle()

```
RMAPI void QuaternionToAxisAngle (  
    Quaternion q,  
    Vector3 * outAxis,  
    float * outAngle)
```

6.25.3.50 QuaternionToEuler()

```
RMAPI Vector3 QuaternionToEuler (  
    Quaternion q)
```

6.25.3.51 QuaternionToMatrix()

```
RMAPI Matrix QuaternionToMatrix (  
    Quaternion q)
```

6.25.3.52 QuaternionTransform()

```
RMAPI Quaternion QuaternionTransform (  
    Quaternion q,  
    Matrix mat)
```

6.25.3.53 Remap()

```
RMAPI float Remap (  
    float value,  
    float inputStart,  
    float inputEnd,  
    float outputStart,  
    float outputEnd)
```

6.25.3.54 Vector2Add()

```
RMAPI Vector2 Vector2Add (  
    Vector2 v1,  
    Vector2 v2)
```

6.25.3.55 Vector2AddValue()

```
RMAPI Vector2 Vector2AddValue (
    Vector2 v,
    float add)
```

6.25.3.56 Vector2Angle()

```
RMAPI float Vector2Angle (
    Vector2 v1,
    Vector2 v2)
```

6.25.3.57 Vector2Clamp()

```
RMAPI Vector2 Vector2Clamp (
    Vector2 v,
    Vector2 min,
    Vector2 max)
```

6.25.3.58 Vector2ClampValue()

```
RMAPI Vector2 Vector2ClampValue (
    Vector2 v,
    float min,
    float max)
```

6.25.3.59 Vector2CrossProduct()

```
RMAPI float Vector2CrossProduct (
    Vector2 v1,
    Vector2 v2)
```

6.25.3.60 Vector2Distance()

```
RMAPI float Vector2Distance (
    Vector2 v1,
    Vector2 v2)
```

6.25.3.61 Vector2DistanceSqr()

```
RMAPI float Vector2DistanceSqr (
    Vector2 v1,
    Vector2 v2)
```

6.25.3.62 Vector2Divide()

```
RMAPI Vector2 Vector2Divide (  
    Vector2 v1,  
    Vector2 v2)
```

6.25.3.63 Vector2DotProduct()

```
RMAPI float Vector2DotProduct (  
    Vector2 v1,  
    Vector2 v2)
```

6.25.3.64 Vector2Equals()

```
RMAPI int Vector2Equals (  
    Vector2 p,  
    Vector2 q)
```

6.25.3.65 Vector2Invert()

```
RMAPI Vector2 Vector2Invert (  
    Vector2 v)
```

6.25.3.66 Vector2Length()

```
RMAPI float Vector2Length (  
    Vector2 v)
```

6.25.3.67 Vector2LengthSqr()

```
RMAPI float Vector2LengthSqr (  
    Vector2 v)
```

6.25.3.68 Vector2Lerp()

```
RMAPI Vector2 Vector2Lerp (  
    Vector2 v1,  
    Vector2 v2,  
    float amount)
```

6.25.3.69 Vector2LineAngle()

```
RMAPI float Vector2LineAngle (  
    Vector2 start,  
    Vector2 end)
```

6.25.3.70 Vector2Max()

```
RMAPI Vector2 Vector2Max (  
    Vector2 v1,  
    Vector2 v2)
```

6.25.3.71 Vector2Min()

```
RMAPI Vector2 Vector2Min (  
    Vector2 v1,  
    Vector2 v2)
```

6.25.3.72 Vector2MoveTowards()

```
RMAPI Vector2 Vector2MoveTowards (  
    Vector2 v,  
    Vector2 target,  
    float maxDistance)
```

6.25.3.73 Vector2Multiply()

```
RMAPI Vector2 Vector2Multiply (  
    Vector2 v1,  
    Vector2 v2)
```

6.25.3.74 Vector2Negate()

```
RMAPI Vector2 Vector2Negate (  
    Vector2 v)
```

6.25.3.75 Vector2Normalize()

```
RMAPI Vector2 Vector2Normalize (  
    Vector2 v)
```

6.25.3.76 Vector2One()

```
RMAPI Vector2 Vector2One (  
    void )
```

6.25.3.77 Vector2Reflect()

```
RMAPI Vector2 Vector2Reflect (  
    Vector2 v,  
    Vector2 normal)
```

6.25.3.78 Vector2Refract()

```
RMAPI Vector2 Vector2Refract (  
    Vector2 v,  
    Vector2 n,  
    float r)
```

6.25.3.79 Vector2Rotate()

```
RMAPI Vector2 Vector2Rotate (  
    Vector2 v,  
    float angle)
```

6.25.3.80 Vector2Scale()

```
RMAPI Vector2 Vector2Scale (  
    Vector2 v,  
    float scale)
```

6.25.3.81 Vector2Subtract()

```
RMAPI Vector2 Vector2Subtract (  
    Vector2 v1,  
    Vector2 v2)
```

6.25.3.82 Vector2SubtractValue()

```
RMAPI Vector2 Vector2SubtractValue (  
    Vector2 v,  
    float sub)
```

6.25.3.83 Vector2Transform()

```
RMAPI Vector2 Vector2Transform (  
    Vector2 v,  
    Matrix mat)
```

6.25.3.84 Vector2Zero()

```
RMAPI Vector2 Vector2Zero (  
    void )
```

6.25.3.85 Vector3Add()

```
RMAPI Vector3 Vector3Add (  
    Vector3 v1,  
    Vector3 v2)
```


6.25.3.86 Vector3AddValue()

```
RMAPI Vector3 Vector3AddValue (
    Vector3 v,
    float add)
```

6.25.3.87 Vector3Angle()

```
RMAPI float Vector3Angle (
    Vector3 v1,
    Vector3 v2)
```

6.25.3.88 Vector3Barycenter()

```
RMAPI Vector3 Vector3Barycenter (
    Vector3 p,
    Vector3 a,
    Vector3 b,
    Vector3 c)
```

6.25.3.89 Vector3Clamp()

```
RMAPI Vector3 Vector3Clamp (
    Vector3 v,
    Vector3 min,
    Vector3 max)
```

6.25.3.90 Vector3ClampValue()

```
RMAPI Vector3 Vector3ClampValue (
    Vector3 v,
    float min,
    float max)
```

6.25.3.91 Vector3CrossProduct()

```
RMAPI Vector3 Vector3CrossProduct (
    Vector3 v1,
    Vector3 v2)
```

6.25.3.92 Vector3CubicHermite()

```
RMAPI Vector3 Vector3CubicHermite (
    Vector3 v1,
    Vector3 tangent1,
    Vector3 v2,
    Vector3 tangent2,
    float amount)
```

6.25.3.93 Vector3Distance()

```
RMAPI float Vector3Distance (  
    Vector3 v1,  
    Vector3 v2)
```

6.25.3.94 Vector3DistanceSqr()

```
RMAPI float Vector3DistanceSqr (  
    Vector3 v1,  
    Vector3 v2)
```

6.25.3.95 Vector3Divide()

```
RMAPI Vector3 Vector3Divide (  
    Vector3 v1,  
    Vector3 v2)
```

6.25.3.96 Vector3DotProduct()

```
RMAPI float Vector3DotProduct (  
    Vector3 v1,  
    Vector3 v2)
```

6.25.3.97 Vector3Equals()

```
RMAPI int Vector3Equals (  
    Vector3 p,  
    Vector3 q)
```

6.25.3.98 Vector3Invert()

```
RMAPI Vector3 Vector3Invert (  
    Vector3 v)
```

6.25.3.99 Vector3Length()

```
RMAPI float Vector3Length (  
    const Vector3 v)
```

6.25.3.100 Vector3LengthSqr()

```
RMAPI float Vector3LengthSqr (  
    const Vector3 v)
```

6.25.3.101 Vector3Lerp()

```
RMAPI Vector3 Vector3Lerp (  
    Vector3 v1,  
    Vector3 v2,  
    float amount)
```

6.25.3.102 Vector3Max()

```
RMAPI Vector3 Vector3Max (  
    Vector3 v1,  
    Vector3 v2)
```

6.25.3.103 Vector3Min()

```
RMAPI Vector3 Vector3Min (  
    Vector3 v1,  
    Vector3 v2)
```

6.25.3.104 Vector3MoveTowards()

```
RMAPI Vector3 Vector3MoveTowards (  
    Vector3 v,  
    Vector3 target,  
    float maxDistance)
```

6.25.3.105 Vector3Multiply()

```
RMAPI Vector3 Vector3Multiply (  
    Vector3 v1,  
    Vector3 v2)
```

6.25.3.106 Vector3Negate()

```
RMAPI Vector3 Vector3Negate (  
    Vector3 v)
```

6.25.3.107 Vector3Normalize()

```
RMAPI Vector3 Vector3Normalize (  
    Vector3 v)
```

6.25.3.108 Vector3One()

```
RMAPI Vector3 Vector3One (  
    void )
```

6.25.3.109 Vector3OrthoNormalize()

```
RMAPI void Vector3OrthoNormalize (
    Vector3 * v1,
    Vector3 * v2)
```

6.25.3.110 Vector3Perpendicular()

```
RMAPI Vector3 Vector3Perpendicular (
    Vector3 v)
```

6.25.3.111 Vector3Project()

```
RMAPI Vector3 Vector3Project (
    Vector3 v1,
    Vector3 v2)
```

6.25.3.112 Vector3Reflect()

```
RMAPI Vector3 Vector3Reflect (
    Vector3 v,
    Vector3 normal)
```

6.25.3.113 Vector3Refract()

```
RMAPI Vector3 Vector3Refract (
    Vector3 v,
    Vector3 n,
    float r)
```

6.25.3.114 Vector3Reject()

```
RMAPI Vector3 Vector3Reject (
    Vector3 v1,
    Vector3 v2)
```

6.25.3.115 Vector3RotateByAxisAngle()

```
RMAPI Vector3 Vector3RotateByAxisAngle (
    Vector3 v,
    Vector3 axis,
    float angle)
```

6.25.3.116 Vector3RotateByQuaternion()

```
RMAPI Vector3 Vector3RotateByQuaternion (
    Vector3 v,
    Quaternion q)
```

6.25.3.117 Vector3Scale()

```
RMAPI Vector3 Vector3Scale (
    Vector3 v,
    float scalar)
```

6.25.3.118 Vector3Subtract()

```
RMAPI Vector3 Vector3Subtract (
    Vector3 v1,
    Vector3 v2)
```

6.25.3.119 Vector3SubtractValue()

```
RMAPI Vector3 Vector3SubtractValue (
    Vector3 v,
    float sub)
```

6.25.3.120 Vector3ToFloatV()

```
RMAPI float3 Vector3ToFloatV (
    Vector3 v)
```

6.25.3.121 Vector3Transform()

```
RMAPI Vector3 Vector3Transform (
    Vector3 v,
    Matrix mat)
```

6.25.3.122 Vector3Unproject()

```
RMAPI Vector3 Vector3Unproject (
    Vector3 source,
    Matrix projection,
    Matrix view)
```

6.25.3.123 Vector3Zero()

```
RMAPI Vector3 Vector3Zero (
    void )
```

6.25.3.124 Vector4Add()

```
RMAPI Vector4 Vector4Add (  
    Vector4 v1,  
    Vector4 v2)
```

6.25.3.125 Vector4AddValue()

```
RMAPI Vector4 Vector4AddValue (  
    Vector4 v,  
    float add)
```

6.25.3.126 Vector4Distance()

```
RMAPI float Vector4Distance (  
    Vector4 v1,  
    Vector4 v2)
```

6.25.3.127 Vector4DistanceSqr()

```
RMAPI float Vector4DistanceSqr (  
    Vector4 v1,  
    Vector4 v2)
```

6.25.3.128 Vector4Divide()

```
RMAPI Vector4 Vector4Divide (  
    Vector4 v1,  
    Vector4 v2)
```

6.25.3.129 Vector4DotProduct()

```
RMAPI float Vector4DotProduct (  
    Vector4 v1,  
    Vector4 v2)
```

6.25.3.130 Vector4Equals()

```
RMAPI int Vector4Equals (  
    Vector4 p,  
    Vector4 q)
```

6.25.3.131 Vector4Invert()

```
RMAPI Vector4 Vector4Invert (  
    Vector4 v)
```

6.25.3.132 Vector4Length()

```
RMAPI float Vector4Length (  
    Vector4 v)
```

6.25.3.133 Vector4LengthSqr()

```
RMAPI float Vector4LengthSqr (  
    Vector4 v)
```

6.25.3.134 Vector4Lerp()

```
RMAPI Vector4 Vector4Lerp (  
    Vector4 v1,  
    Vector4 v2,  
    float amount)
```

6.25.3.135 Vector4Max()

```
RMAPI Vector4 Vector4Max (  
    Vector4 v1,  
    Vector4 v2)
```

6.25.3.136 Vector4Min()

```
RMAPI Vector4 Vector4Min (  
    Vector4 v1,  
    Vector4 v2)
```

6.25.3.137 Vector4MoveTowards()

```
RMAPI Vector4 Vector4MoveTowards (  
    Vector4 v,  
    Vector4 target,  
    float maxDistance)
```

6.25.3.138 Vector4Multiply()

```
RMAPI Vector4 Vector4Multiply (  
    Vector4 v1,  
    Vector4 v2)
```

6.25.3.139 Vector4Negate()

```
RMAPI Vector4 Vector4Negate (  
    Vector4 v)
```

6.25.3.140 Vector4Normalize()

```
RMAPI Vector4 Vector4Normalize (  
    Vector4 v)
```

6.25.3.141 Vector4One()

```
RMAPI Vector4 Vector4One (  
    void )
```

6.25.3.142 Vector4Scale()

```
RMAPI Vector4 Vector4Scale (  
    Vector4 v,  
    float scale)
```

6.25.3.143 Vector4Subtract()

```
RMAPI Vector4 Vector4Subtract (  
    Vector4 v1,  
    Vector4 v2)
```

6.25.3.144 Vector4SubtractValue()

```
RMAPI Vector4 Vector4SubtractValue (  
    Vector4 v,  
    float add)
```

6.25.3.145 Vector4Zero()

```
RMAPI Vector4 Vector4Zero (  
    void )
```

6.25.3.146 Wrap()

```
RMAPI float Wrap (  
    float value,  
    float min,  
    float max)
```


6.26 raymath.h

[Go to the documentation of this file.](#)

```

00001 /*****
00002 *
00003 *   raymath v2.0 - Math functions to work with Vector2, Vector3, Matrix and Quaternions
00004 *
00005 *   CONVENTIONS:
00006 *   - Matrix structure is defined as row-major (memory layout) but parameters naming AND all
00007 *     math operations performed by the library consider the structure as it was column-major
00008 *   - It is like transposed versions of the matrices are used for all the maths
00009 *   - It benefits some functions making them cache-friendly and also avoids matrix
00010 *     transpositions sometimes required by OpenGL
00011 *   - Example: In memory order, row0 is [m0 m4 m8 m12] but in semantic math row0 is [m0 m1 m2 m3]
00012 *   - Functions are always self-contained, no function use another raymath function inside,
00013 *     required code is directly re-implemented inside
00014 *   - Functions input parameters are always received by value (2 unavoidable exceptions)
00015 *   - Functions use always a "result" variable for return (except C++ operators)
00016 *   - Functions are always defined inline
00017 *   - Angles are always in radians (DEG2RAD/RAD2DEG macros provided for convenience)
00018 *   - No compound literals used to make sure the library is compatible with C++
00019 *
00020 *   CONFIGURATION:
00021 *   #define RAYMATH_IMPLEMENTATION
00022 *       Generates the implementation of the library into the included file
00023 *       If not defined, the library is in header only mode and can be included in other headers
00024 *       or source files without problems. But only ONE file should hold the implementation
00025 *
00026 *   #define RAYMATH_STATIC_INLINE
00027 *       Define static inline functions code, so #include header suffices for use
00028 *       This may use up lots of memory
00029 *
00030 *   #define RAYMATH_DISABLE_CPP_OPERATORS
00031 *       Disables C++ operator overloads for raymath types.
00032 *
00033 *   #define RAYMATH_USE_SIMD_INTRINSICS 1
00034 *       Try to enable SIMD intrinsics for MatrixMultiply()
00035 *       Note that users enabling it must be aware of the target platform where application will
00036 *       run to support the selected SIMD intrinsic, for now, only SSE is supported
00037 *
00038 *   LICENSE: zlib/libpng
00039 *
00040 *   Copyright (c) 2015-2026 Ramon Santamaria (@raysan5)
00041 *
00042 *   This software is provided "as-is", without any express or implied warranty. In no event
00043 *   will the authors be held liable for any damages arising from the use of this software.
00044 *
00045 *   Permission is granted to anyone to use this software for any purpose, including commercial
00046 *   applications, and to alter it and redistribute it freely, subject to the following restrictions:
00047 *
00048 *   1. The origin of this software must not be misrepresented; you must not claim that you
00049 *   wrote the original software. If you use this software in a product, an acknowledgment
00050 *   in the product documentation would be appreciated but is not required.
00051 *
00052 *   2. Altered source versions must be plainly marked as such, and must not be misrepresented
00053 *   as being the original software.
00054 *
00055 *   3. This notice may not be removed or altered from any source distribution.
00056 *
00057 *****/
00058
00059 #ifndef RAYMATH_H
00060 #define RAYMATH_H
00061
00062 #if defined(RAYMATH_IMPLEMENTATION) && defined(RAYMATH_STATIC_INLINE)
00063     #error "Specifying both RAYMATH_IMPLEMENTATION and RAYMATH_STATIC_INLINE is contradictory"
00064 #endif
00065
00066 // Function specifiers definition
00067 #if defined(RAYMATH_IMPLEMENTATION)
00068     #if defined(_WIN32) && defined(BUILD_LIBTYPE_SHARED)
00069         #define RMAPI __declspec(dllexport) extern inline // Building raylib as a Win32 shared
00070         library (.dll)
00071     #elif defined(BUILD_LIBTYPE_SHARED)
00072         #define RMAPI __attribute__((visibility("default"))) // Building raylib as a Unix shared
00073         library (.so/.dylib)
00074     #elif defined(_WIN32) && defined(USE_LIBTYPE_SHARED)
00075         #define RMAPI __declspec(dllimport) // Using raylib as a Win32 shared library
00076         (.dll)
00077     #else
00078         #define RMAPI extern inline // Provide external definition
00079     #endif
00080 #elif defined(RAYMATH_STATIC_INLINE)
00081     #define RMAPI static inline // Functions may be inlined, no external out-of-line definition
00082 #else
00083     #define RMAPI
00084 #endif

```

```

00080     #if defined(__TINYC__)
00081         #define RMAPI static inline // plain inline not supported by tinycc (See issue #435)
00082     #else
00083         #define RMAPI inline        // Functions may be inlined or external definition used
00084     #endif
00085 #endif
00086
00087 //-----
00088 // Defines and Macros
00089 //-----
00090 #ifndef PI
00091     #define PI 3.14159265358979323846f
00092 #endif
00093
00094 #ifndef EPSILON
00095     #define EPSILON 0.000001f
00096 #endif
00097
00098 #ifndef DEG2RAD
00099     #define DEG2RAD (PI/180.0f)
00100 #endif
00101
00102 #ifndef RAD2DEG
00103     #define RAD2DEG (180.0f/PI)
00104 #endif
00105
00106 // Get float vector for Matrix
00107 #ifndef MatrixToFloat
00108     #define MatrixToFloat(mat) (MatrixToFloatV(mat).v)
00109 #endif
00110
00111 // Get float vector for Vector3
00112 #ifndef Vector3ToFloat
00113     #define Vector3ToFloat(vec) (Vector3ToFloatV(vec).v)
00114 #endif
00115
00116 //-----
00117 // Types and Structures Definition
00118 //-----
00119 #if !defined(RL_VECTOR2_TYPE)
00120 // Vector2 type
00121 typedef struct Vector2 {
00122     float x;
00123     float y;
00124 } Vector2;
00125 #define RL_VECTOR2_TYPE
00126 #endif
00127
00128 #if !defined(RL_VECTOR3_TYPE)
00129 // Vector3 type
00130 typedef struct Vector3 {
00131     float x;
00132     float y;
00133     float z;
00134 } Vector3;
00135 #define RL_VECTOR3_TYPE
00136 #endif
00137
00138 #if !defined(RL_VECTOR4_TYPE)
00139 // Vector4 type
00140 typedef struct Vector4 {
00141     float x;
00142     float y;
00143     float z;
00144     float w;
00145 } Vector4;
00146 #define RL_VECTOR4_TYPE
00147 #endif
00148
00149 #if !defined(RL_QUATERNION_TYPE)
00150 // Quaternion type
00151 typedef Vector4 Quaternion;
00152 #define RL_QUATERNION_TYPE
00153 #endif
00154
00155 #if !defined(RL_MATRIX_TYPE)
00156 // Matrix type (OpenGL style 4x4 - right handed, column major)
00157 typedef struct Matrix {
00158     float m0, m4, m8, m12;    // Matrix first row (4 components)
00159     float m1, m5, m9, m13;    // Matrix second row (4 components)
00160     float m2, m6, m10, m14;   // Matrix third row (4 components)
00161     float m3, m7, m11, m15;   // Matrix fourth row (4 components)
00162 } Matrix;
00163 #define RL_MATRIX_TYPE
00164 #endif
00165
00166 // NOTE: Helper types to be used instead of array return types for *ToFloat functions

```

```

00167 #if !defined(RL_FLOAT3_TYPE)
00168 typedef struct float3 {
00169     float v[3];
00170 } float3;
00171 #define RL_FLOAT3_TYPE
00172 #endif
00173
00174 #if !defined(RL_FLOAT16_TYPE)
00175 typedef struct float16 {
00176     float v[16];
00177 } float16;
00178 #define RL_FLOAT16_TYPE
00179 #endif
00180
00181 #include <math.h>          // Required for: sinf(), cosf(), tan(), atan2f(), sqrtf(), floor(), fminf(),
                             fmaxf(), fabsf()
00182
00183 #if RAYMATH_USE_SIMD_INTRINSICS
00184     // SIMD is used on the most costly raymath function MatrixMultiply()
00185     // NOTE: Only SSE intrinsics support implemented
00186     // TODO: Consider support for other SIMD intrinsics:
00187     // - SSEx, AVX, AVX2, FMA, NEON, RVV
00188     /*
00189     #if defined(__SSE4_2__)
00190         #include <nmmintrin.h>
00191         #define RAYMATH_SSE42_ENABLED
00192     #elif defined(__SSE4_1__)
00193         #include <smmintrin.h>
00194         #define RAYMATH_SSE41_ENABLED
00195     #elif defined(__SSE3__)
00196         #include <tmmintrin.h>
00197         #define RAYMATH_SSSE3_ENABLED
00198     #elif defined(__SSE3__)
00199         #include <pmmmintrin.h>
00200         #define RAYMATH_SSE3_ENABLED
00201     #elif defined(__SSE2__) || (defined(_M_AMD64) || defined(_M_X64)) // SSE2 x64
00202         #include <emmintrin.h>
00203         #define RAYMATH_SSE2_ENABLED
00204     #endif
00205     */
00206     #if defined(__SSE__) || defined(_M_X64) || (defined(_M_IX86_FP) && (_M_IX86_FP >= 1))
00207         #include <xmmmintrin.h>
00208         #define RAYMATH_SSE_ENABLED
00209     #endif
00210 #endif
00211
00212 //-----
00213 // Module Functions Definition - Utils math
00214 //-----
00215
00216 // Clamp float value
00217 RMAPI float Clamp(float value, float min, float max)
00218 {
00219     float result = (value < min)? min : value;
00220
00221     if (result > max) result = max;
00222
00223     return result;
00224 }
00225
00226 // Calculate linear interpolation between two floats
00227 RMAPI float Lerp(float start, float end, float amount)
00228 {
00229     float result = start + amount*(end - start);
00230
00231     return result;
00232 }
00233
00234 // Normalize input value within input range
00235 RMAPI float Normalize(float value, float start, float end)
00236 {
00237     float result = (value - start)/(end - start);
00238
00239     return result;
00240 }
00241
00242 // Remap input value within input range to output range
00243 RMAPI float Remap(float value, float inputStart, float inputEnd, float outputStart, float outputEnd)
00244 {
00245     float result = (value - inputStart)/(inputEnd - inputStart)*(outputEnd - outputStart) +
00246     outputStart;
00247
00248     return result;
00249 }
00250
00251 // Wrap input value from min to max
00251 RMAPI float Wrap(float value, float min, float max)

```

```

00252 {
00253     float result = value - (max - min)*floorf((value - min)/(max - min));
00254
00255     return result;
00256 }
00257
00258 // Check whether two given floats are almost equal
00259 RMAPI int FloatEquals(float x, float y)
00260 {
00261     #if !defined(EPSILON)
00262         #define EPSILON 0.000001f
00263     #endif
00264
00265     int result = (fabsf(x - y)) <= (EPSILON*fmaxf(1.0f, fmaxf(fabsf(x), fabsf(y))));
00266
00267     return result;
00268 }
00269
00270 //-----
00271 // Module Functions Definition - Vector2 math
00272 //-----
00273
00274 // Vector with components value 0.0f
00275 RMAPI Vector2 Vector2Zero(void)
00276 {
00277     Vector2 result = { 0.0f, 0.0f };
00278
00279     return result;
00280 }
00281
00282 // Vector with components value 1.0f
00283 RMAPI Vector2 Vector2One(void)
00284 {
00285     Vector2 result = { 1.0f, 1.0f };
00286
00287     return result;
00288 }
00289
00290 // Add two vectors (v1 + v2)
00291 RMAPI Vector2 Vector2Add(Vector2 v1, Vector2 v2)
00292 {
00293     Vector2 result = { v1.x + v2.x, v1.y + v2.y };
00294
00295     return result;
00296 }
00297
00298 // Add vector and float value
00299 RMAPI Vector2 Vector2AddValue(Vector2 v, float add)
00300 {
00301     Vector2 result = { v.x + add, v.y + add };
00302
00303     return result;
00304 }
00305
00306 // Subtract two vectors (v1 - v2)
00307 RMAPI Vector2 Vector2Subtract(Vector2 v1, Vector2 v2)
00308 {
00309     Vector2 result = { v1.x - v2.x, v1.y - v2.y };
00310
00311     return result;
00312 }
00313
00314 // Subtract vector by float value
00315 RMAPI Vector2 Vector2SubtractValue(Vector2 v, float sub)
00316 {
00317     Vector2 result = { v.x - sub, v.y - sub };
00318
00319     return result;
00320 }
00321
00322 // Calculate vector length
00323 RMAPI float Vector2Length(Vector2 v)
00324 {
00325     float result = sqrtf((v.x*v.x) + (v.y*v.y));
00326
00327     return result;
00328 }
00329
00330 // Calculate vector square length
00331 RMAPI float Vector2LengthSqr(Vector2 v)
00332 {
00333     float result = (v.x*v.x) + (v.y*v.y);
00334
00335     return result;
00336 }
00337
00338 // Calculate two vectors dot product

```

```

00339 RMAPI float Vector2DotProduct(Vector2 v1, Vector2 v2)
00340 {
00341     float result = (v1.x*v2.x + v1.y*v2.y);
00342
00343     return result;
00344 }
00345
00346 // Calculate two vectors cross product
00347 RMAPI float Vector2CrossProduct(Vector2 v1, Vector2 v2)
00348 {
00349     float result = (v1.x*v2.y - v1.y*v2.x);
00350
00351     return result;
00352 }
00353
00354 // Calculate distance between two vectors
00355 RMAPI float Vector2Distance(Vector2 v1, Vector2 v2)
00356 {
00357     float result = sqrtf((v1.x - v2.x)*(v1.x - v2.x) + (v1.y - v2.y)*(v1.y - v2.y));
00358
00359     return result;
00360 }
00361
00362 // Calculate square distance between two vectors
00363 RMAPI float Vector2DistanceSqr(Vector2 v1, Vector2 v2)
00364 {
00365     float result = ((v1.x - v2.x)*(v1.x - v2.x) + (v1.y - v2.y)*(v1.y - v2.y));
00366
00367     return result;
00368 }
00369
00370 // Calculate the signed angle from v1 to v2, relative to the origin (0, 0)
00371 // NOTE: Coordinate system convention: positive X right, positive Y down
00372 // positive angles appear clockwise, and negative angles appear counterclockwise
00373 RMAPI float Vector2Angle(Vector2 v1, Vector2 v2)
00374 {
00375     float result = 0.0f;
00376
00377     float dot = v1.x*v2.x + v1.y*v2.y;
00378     float det = v1.x*v2.y - v1.y*v2.x;
00379
00380     result = atan2f(det, dot);
00381
00382     return result;
00383 }
00384
00385 // Calculate angle defined by a two vectors line
00386 // NOTE: Parameters need to be normalized
00387 // Current implementation should be aligned with glm::angle
00388 RMAPI float Vector2LineAngle(Vector2 start, Vector2 end)
00389 {
00390     float result = 0.0f;
00391
00392     // TODO(10/9/2023): Currently angles move clockwise, determine if this is wanted behavior
00393     result = -atan2f(end.y - start.y, end.x - start.x);
00394
00395     return result;
00396 }
00397
00398 // Scale vector (multiply by value)
00399 RMAPI Vector2 Vector2Scale(Vector2 v, float scale)
00400 {
00401     Vector2 result = { v.x*scale, v.y*scale };
00402
00403     return result;
00404 }
00405
00406 // Multiply vector by vector
00407 RMAPI Vector2 Vector2Multiply(Vector2 v1, Vector2 v2)
00408 {
00409     Vector2 result = { v1.x*v2.x, v1.y*v2.y };
00410
00411     return result;
00412 }
00413
00414 // Negate vector
00415 RMAPI Vector2 Vector2Negate(Vector2 v)
00416 {
00417     Vector2 result = { -v.x, -v.y };
00418
00419     return result;
00420 }
00421
00422 // Divide vector by vector
00423 RMAPI Vector2 Vector2Divide(Vector2 v1, Vector2 v2)
00424 {
00425     Vector2 result = { v1.x/v2.x, v1.y/v2.y };

```

```
00426
00427     return result;
00428 }
00429
00430 // Normalize provided vector
00431 RMAPI Vector2 Vector2Normalize(Vector2 v)
00432 {
00433     Vector2 result = { 0 };
00434     float length = sqrtf((v.x*v.x) + (v.y*v.y));
00435
00436     if (length > 0)
00437     {
00438         float ilength = 1.0f/length;
00439         result.x = v.x*ilength;
00440         result.y = v.y*ilength;
00441     }
00442
00443     return result;
00444 }
00445
00446 // Transforms a Vector2 by a given Matrix
00447 RMAPI Vector2 Vector2Transform(Vector2 v, Matrix mat)
00448 {
00449     Vector2 result = { 0 };
00450
00451     float x = v.x;
00452     float y = v.y;
00453     float z = 0;
00454
00455     result.x = mat.m0*x + mat.m4*y + mat.m8*z + mat.m12;
00456     result.y = mat.m1*x + mat.m5*y + mat.m9*z + mat.m13;
00457
00458     return result;
00459 }
00460
00461 // Calculate linear interpolation between two vectors
00462 RMAPI Vector2 Vector2Lerp(Vector2 v1, Vector2 v2, float amount)
00463 {
00464     Vector2 result = { 0 };
00465
00466     result.x = v1.x + amount*(v2.x - v1.x);
00467     result.y = v1.y + amount*(v2.y - v1.y);
00468
00469     return result;
00470 }
00471
00472 // Calculate reflected vector to normal
00473 RMAPI Vector2 Vector2Reflect(Vector2 v, Vector2 normal)
00474 {
00475     Vector2 result = { 0 };
00476
00477     float dotProduct = (v.x*normal.x + v.y*normal.y); // Dot product
00478
00479     result.x = v.x - (2.0f*normal.x)*dotProduct;
00480     result.y = v.y - (2.0f*normal.y)*dotProduct;
00481
00482     return result;
00483 }
00484
00485 // Get min value for each pair of components
00486 RMAPI Vector2 Vector2Min(Vector2 v1, Vector2 v2)
00487 {
00488     Vector2 result = { 0 };
00489
00490     result.x = fminf(v1.x, v2.x);
00491     result.y = fminf(v1.y, v2.y);
00492
00493     return result;
00494 }
00495
00496 // Get max value for each pair of components
00497 RMAPI Vector2 Vector2Max(Vector2 v1, Vector2 v2)
00498 {
00499     Vector2 result = { 0 };
00500
00501     result.x = fmaxf(v1.x, v2.x);
00502     result.y = fmaxf(v1.y, v2.y);
00503
00504     return result;
00505 }
00506
00507 // Rotate vector by angle
00508 RMAPI Vector2 Vector2Rotate(Vector2 v, float angle)
00509 {
00510     Vector2 result = { 0 };
00511
00512     float cosres = cosf(angle);
```

```

00513     float sinres = sinf(angle);
00514
00515     result.x = v.x*cosres - v.y*sinres;
00516     result.y = v.x*sinres + v.y*cosres;
00517
00518     return result;
00519 }
00520
00521 // Move Vector towards target
00522 RMAPI Vector2 Vector2MoveTowards(Vector2 v, Vector2 target, float maxDistance)
00523 {
00524     Vector2 result = { 0 };
00525
00526     float dx = target.x - v.x;
00527     float dy = target.y - v.y;
00528     float value = (dx*dx) + (dy*dy);
00529
00530     if ((value == 0) || ((maxDistance >= 0) && (value <= maxDistance*maxDistance))) return target;
00531
00532     float dist = sqrtf(value);
00533
00534     result.x = v.x + dx/dist*maxDistance;
00535     result.y = v.y + dy/dist*maxDistance;
00536
00537     return result;
00538 }
00539
00540 // Invert the given vector
00541 RMAPI Vector2 Vector2Invert(Vector2 v)
00542 {
00543     Vector2 result = { 1.0f/v.x, 1.0f/v.y };
00544
00545     return result;
00546 }
00547
00548 // Clamp the components of the vector between
00549 // min and max values specified by the given vectors
00550 RMAPI Vector2 Vector2Clamp(Vector2 v, Vector2 min, Vector2 max)
00551 {
00552     Vector2 result = { 0 };
00553
00554     result.x = fminf(max.x, fmaxf(min.x, v.x));
00555     result.y = fminf(max.y, fmaxf(min.y, v.y));
00556
00557     return result;
00558 }
00559
00560 // Clamp the magnitude of the vector between two min and max values
00561 RMAPI Vector2 Vector2ClampValue(Vector2 v, float min, float max)
00562 {
00563     Vector2 result = v;
00564
00565     float length = (v.x*v.x) + (v.y*v.y);
00566     if (length > 0.0f)
00567     {
00568         length = sqrtf(length);
00569
00570         float scale = 1; // By default, 1 as the neutral element
00571         if (length < min) scale = min/length;
00572         else if (length > max) scale = max/length;
00573
00574         result.x = v.x*scale;
00575         result.y = v.y*scale;
00576     }
00577
00578     return result;
00579 }
00580
00581 // Check whether two given vectors are almost equal
00582 RMAPI int Vector2Equals(Vector2 p, Vector2 q)
00583 {
00584     #if !defined(EPSILON)
00585         #define EPSILON 0.000001f
00586     #endif
00587
00588     int result = ((fabsf(p.x - q.x)) <= (EPSILON*fmaxf(1.0f, fmaxf(fabsf(p.x), fabsf(q.x))))) &&
00589                 ((fabsf(p.y - q.y)) <= (EPSILON*fmaxf(1.0f, fmaxf(fabsf(p.y), fabsf(q.y))))) &&
00590
00591     return result;
00592 }
00593
00594 // Compute the direction of a refracted ray
00595 // v: normalized direction of the incoming ray
00596 // n: normalized normal vector of the interface of two optical media
00597 // r: ratio of the refractive index of the medium from where the ray comes
00598 // to the refractive index of the medium on the other side of the surface
00599 RMAPI Vector2 Vector2Refract(Vector2 v, Vector2 n, float r)

```

```

00600 {
00601     Vector2 result = { 0 };
00602
00603     float dot = v.x*n.x + v.y*n.y;
00604     float d = 1.0f - r*r*(1.0f - dot*dot);
00605
00606     if (d >= 0.0f)
00607     {
00608         d = sqrtf(d);
00609         v.x = r*v.x - (r*dot + d)*n.x;
00610         v.y = r*v.y - (r*dot + d)*n.y;
00611
00612         result = v;
00613     }
00614
00615     return result;
00616 }
00617
00618
00619 //-----
00620 // Module Functions Definition - Vector3 math
00621 //-----
00622
00623 // Vector with components value 0.0f
00624 RMAPI Vector3 Vector3Zero(void)
00625 {
00626     Vector3 result = { 0.0f, 0.0f, 0.0f };
00627
00628     return result;
00629 }
00630
00631 // Vector with components value 1.0f
00632 RMAPI Vector3 Vector3One(void)
00633 {
00634     Vector3 result = { 1.0f, 1.0f, 1.0f };
00635
00636     return result;
00637 }
00638
00639 // Add two vectors
00640 RMAPI Vector3 Vector3Add(Vector3 v1, Vector3 v2)
00641 {
00642     Vector3 result = { v1.x + v2.x, v1.y + v2.y, v1.z + v2.z };
00643
00644     return result;
00645 }
00646
00647 // Add vector and float value
00648 RMAPI Vector3 Vector3AddValue(Vector3 v, float add)
00649 {
00650     Vector3 result = { v.x + add, v.y + add, v.z + add };
00651
00652     return result;
00653 }
00654
00655 // Subtract two vectors
00656 RMAPI Vector3 Vector3Subtract(Vector3 v1, Vector3 v2)
00657 {
00658     Vector3 result = { v1.x - v2.x, v1.y - v2.y, v1.z - v2.z };
00659
00660     return result;
00661 }
00662
00663 // Subtract vector by float value
00664 RMAPI Vector3 Vector3SubtractValue(Vector3 v, float sub)
00665 {
00666     Vector3 result = { v.x - sub, v.y - sub, v.z - sub };
00667
00668     return result;
00669 }
00670
00671 // Multiply vector by scalar
00672 RMAPI Vector3 Vector3Scale(Vector3 v, float scalar)
00673 {
00674     Vector3 result = { v.x*scalar, v.y*scalar, v.z*scalar };
00675
00676     return result;
00677 }
00678
00679 // Multiply vector by vector
00680 RMAPI Vector3 Vector3Multiply(Vector3 v1, Vector3 v2)
00681 {
00682     Vector3 result = { v1.x*v2.x, v1.y*v2.y, v1.z*v2.z };
00683
00684     return result;
00685 }
00686

```



```

00687 // Calculate two vectors cross product
00688 RMAPI Vector3 Vector3CrossProduct(Vector3 v1, Vector3 v2)
00689 {
00690     Vector3 result = { v1.y*v2.z - v1.z*v2.y, v1.z*v2.x - v1.x*v2.z, v1.x*v2.y - v1.y*v2.x };
00691
00692     return result;
00693 }
00694
00695 // Calculate one vector perpendicular vector
00696 RMAPI Vector3 Vector3Perpendicular(Vector3 v)
00697 {
00698     Vector3 result = { 0 };
00699
00700     float min = fabsf(v.x);
00701     Vector3 cardinalAxis = {1.0f, 0.0f, 0.0f};
00702
00703     if (fabsf(v.y) < min)
00704     {
00705         min = fabsf(v.y);
00706         Vector3 tmp = {0.0f, 1.0f, 0.0f};
00707         cardinalAxis = tmp;
00708     }
00709
00710     if (fabsf(v.z) < min)
00711     {
00712         Vector3 tmp = {0.0f, 0.0f, 1.0f};
00713         cardinalAxis = tmp;
00714     }
00715
00716     // Cross product between vectors
00717     result.x = v.y*cardinalAxis.z - v.z*cardinalAxis.y;
00718     result.y = v.z*cardinalAxis.x - v.x*cardinalAxis.z;
00719     result.z = v.x*cardinalAxis.y - v.y*cardinalAxis.x;
00720
00721     return result;
00722 }
00723
00724 // Calculate vector length
00725 RMAPI float Vector3Length(const Vector3 v)
00726 {
00727     float result = sqrtf(v.x*v.x + v.y*v.y + v.z*v.z);
00728
00729     return result;
00730 }
00731
00732 // Calculate vector square length
00733 RMAPI float Vector3LengthSqr(const Vector3 v)
00734 {
00735     float result = v.x*v.x + v.y*v.y + v.z*v.z;
00736
00737     return result;
00738 }
00739
00740 // Calculate two vectors dot product
00741 RMAPI float Vector3DotProduct(Vector3 v1, Vector3 v2)
00742 {
00743     float result = (v1.x*v2.x + v1.y*v2.y + v1.z*v2.z);
00744
00745     return result;
00746 }
00747
00748 // Calculate distance between two vectors
00749 RMAPI float Vector3Distance(Vector3 v1, Vector3 v2)
00750 {
00751     float result = 0.0f;
00752
00753     float dx = v2.x - v1.x;
00754     float dy = v2.y - v1.y;
00755     float dz = v2.z - v1.z;
00756     result = sqrtf(dx*dx + dy*dy + dz*dz);
00757
00758     return result;
00759 }
00760
00761 // Calculate square distance between two vectors
00762 RMAPI float Vector3DistanceSqr(Vector3 v1, Vector3 v2)
00763 {
00764     float result = 0.0f;
00765
00766     float dx = v2.x - v1.x;
00767     float dy = v2.y - v1.y;
00768     float dz = v2.z - v1.z;
00769     result = dx*dx + dy*dy + dz*dz;
00770
00771     return result;
00772 }
00773

```

```

00774 // Calculate angle between two vectors
00775 RMAPI float Vector3Angle(Vector3 v1, Vector3 v2)
00776 {
00777     float result = 0.0f;
00778
00779     Vector3 cross = { v1.y*v2.z - v1.z*v2.y, v1.z*v2.x - v1.x*v2.z, v1.x*v2.y - v1.y*v2.x };
00780     float len = sqrtf(cross.x*cross.x + cross.y*cross.y + cross.z*cross.z);
00781     float dot = (v1.x*v2.x + v1.y*v2.y + v1.z*v2.z);
00782     result = atan2f(len, dot);
00783
00784     return result;
00785 }
00786
00787 // Negate provided vector (invert direction)
00788 RMAPI Vector3 Vector3Negate(Vector3 v)
00789 {
00790     Vector3 result = { -v.x, -v.y, -v.z };
00791
00792     return result;
00793 }
00794
00795 // Divide vector by vector
00796 RMAPI Vector3 Vector3Divide(Vector3 v1, Vector3 v2)
00797 {
00798     Vector3 result = { v1.x/v2.x, v1.y/v2.y, v1.z/v2.z };
00799
00800     return result;
00801 }
00802
00803 // Normalize provided vector
00804 RMAPI Vector3 Vector3Normalize(Vector3 v)
00805 {
00806     Vector3 result = v;
00807
00808     float length = sqrtf(v.x*v.x + v.y*v.y + v.z*v.z);
00809     if (length != 0.0f)
00810     {
00811         float ilength = 1.0f/length;
00812
00813         result.x *= ilength;
00814         result.y *= ilength;
00815         result.z *= ilength;
00816     }
00817
00818     return result;
00819 }
00820
00821 //Calculate the projection of the vector v1 on to v2
00822 RMAPI Vector3 Vector3Project(Vector3 v1, Vector3 v2)
00823 {
00824     Vector3 result = { 0 };
00825
00826     float v1dv2 = (v1.x*v2.x + v1.y*v2.y + v1.z*v2.z);
00827     float v2dv2 = (v2.x*v2.x + v2.y*v2.y + v2.z*v2.z);
00828
00829     float mag = v1dv2/v2dv2;
00830
00831     result.x = v2.x*mag;
00832     result.y = v2.y*mag;
00833     result.z = v2.z*mag;
00834
00835     return result;
00836 }
00837
00838 //Calculate the rejection of the vector v1 on to v2
00839 RMAPI Vector3 Vector3Reject(Vector3 v1, Vector3 v2)
00840 {
00841     Vector3 result = { 0 };
00842
00843     float v1dv2 = (v1.x*v2.x + v1.y*v2.y + v1.z*v2.z);
00844     float v2dv2 = (v2.x*v2.x + v2.y*v2.y + v2.z*v2.z);
00845
00846     float mag = v1dv2/v2dv2;
00847
00848     result.x = v1.x - (v2.x*mag);
00849     result.y = v1.y - (v2.y*mag);
00850     result.z = v1.z - (v2.z*mag);
00851
00852     return result;
00853 }
00854
00855 // Orthonormalize provided vectors
00856 // Makes vectors normalized and orthogonal to each other
00857 // Gram-Schmidt function implementation
00858 RMAPI void Vector3OrthoNormalize(Vector3 *v1, Vector3 *v2)
00859 {
00860     float length = 0.0f;

```

```

00861     float ilength = 0.0f;
00862
00863     // Vector3Normalize(*v1);
00864     Vector3 v = *v1;
00865     length = sqrtf(v.x*v.x + v.y*v.y + v.z*v.z);
00866     if (length == 0.0f) length = 1.0f;
00867     ilength = 1.0f/length;
00868     v1->x *= ilength;
00869     v1->y *= ilength;
00870     v1->z *= ilength;
00871
00872     // Vector3CrossProduct(*v1, *v2)
00873     Vector3 vn1 = { v1->y*v2->z - v1->z*v2->y, v1->z*v2->x - v1->x*v2->z, v1->x*v2->y - v1->y*v2->x };
00874
00875     // Vector3Normalize(vn1);
00876     v = vn1;
00877     length = sqrtf(v.x*v.x + v.y*v.y + v.z*v.z);
00878     if (length == 0.0f) length = 1.0f;
00879     ilength = 1.0f/length;
00880     vn1.x *= ilength;
00881     vn1.y *= ilength;
00882     vn1.z *= ilength;
00883
00884     // Vector3CrossProduct(vn1, *v1)
00885     Vector3 vn2 = { vn1.y*v1->z - vn1.z*v1->y, vn1.z*v1->x - vn1.x*v1->z, vn1.x*v1->y - vn1.y*v1->x };
00886
00887     *v2 = vn2;
00888 }
00889
00890 // Transforms a Vector3 by a given Matrix
00891 RMAPI Vector3 Vector3Transform(Vector3 v, Matrix mat)
00892 {
00893     Vector3 result = { 0 };
00894
00895     float x = v.x;
00896     float y = v.y;
00897     float z = v.z;
00898
00899     result.x = mat.m0*x + mat.m4*y + mat.m8*z + mat.m12;
00900     result.y = mat.m1*x + mat.m5*y + mat.m9*z + mat.m13;
00901     result.z = mat.m2*x + mat.m6*y + mat.m10*z + mat.m14;
00902
00903     return result;
00904 }
00905
00906 // Transform a vector by quaternion rotation
00907 RMAPI Vector3 Vector3RotateByQuaternion(Vector3 v, Quaternion q)
00908 {
00909     Vector3 result = { 0 };
00910
00911     result.x = v.x*(q.x*q.x + q.w*q.w - q.y*q.y - q.z*q.z) + v.y*(2*q.x*q.y - 2*q.w*q.z) +
00912     v.z*(2*q.x*q.z + 2*q.w*q.y);
00913     result.y = v.x*(2*q.w*q.z + 2*q.x*q.y) + v.y*(q.w*q.w - q.x*q.x + q.y*q.y - q.z*q.z) +
00914     v.z*(-2*q.w*q.x + 2*q.y*q.z);
00915     result.z = v.x*(-2*q.w*q.y + 2*q.x*q.z) + v.y*(2*q.w*q.x + 2*q.y*q.z) + v.z*(q.w*q.w - q.x*q.x -
00916     q.y*q.y + q.z*q.z);
00917
00918     return result;
00919 }
00920
00921 // Rotates a vector around an axis
00922 RMAPI Vector3 Vector3RotateByAxisAngle(Vector3 v, Vector3 axis, float angle)
00923 {
00924     // Using Euler-Rodrigues Formula
00925     // Ref.: https://en.wikipedia.org/w/index.php?title=Euler%E2%80%93Rodrigues\_formula
00926
00927     Vector3 result = v;
00928
00929     // Vector3Normalize(axis);
00930     float length = sqrtf(axis.x*axis.x + axis.y*axis.y + axis.z*axis.z);
00931     if (length == 0.0f) length = 1.0f;
00932     float ilength = 1.0f/length;
00933     axis.x *= ilength;
00934     axis.y *= ilength;
00935     axis.z *= ilength;
00936
00937     angle /= 2.0f;
00938     float a = sinf(angle);
00939     float b = axis.x*a;
00940     float c = axis.y*a;
00941     float d = axis.z*a;
00942     a = cosf(angle);
00943     Vector3 w = { b, c, d };
00944
00945     // Vector3CrossProduct(w, v)
00946     Vector3 wv = { w.y*v.z - w.z*v.y, w.z*v.x - w.x*v.z, w.x*v.y - w.y*v.x };

```

```

00945 // Vector3CrossProduct(w, wv)
00946 Vector3 wwv = { w.y*wv.z - w.z*wv.y, w.z*wv.x - w.x*wv.z, w.x*wv.y - w.y*wv.x };
00947
00948 // Vector3Scale(wv, 2*a)
00949 a *= 2;
00950 wv.x *= a;
00951 wv.y *= a;
00952 wv.z *= a;
00953
00954 // Vector3Scale(wwv, 2)
00955 wwv.x *= 2;
00956 wwv.y *= 2;
00957 wwv.z *= 2;
00958
00959 result.x += wv.x;
00960 result.y += wv.y;
00961 result.z += wv.z;
00962
00963 result.x += wwv.x;
00964 result.y += wwv.y;
00965 result.z += wwv.z;
00966
00967 return result;
00968 }
00969
00970 // Move Vector towards target
00971 RMAPI Vector3 Vector3MoveTowards(Vector3 v, Vector3 target, float maxDistance)
00972 {
00973     Vector3 result = { 0 };
00974
00975     float dx = target.x - v.x;
00976     float dy = target.y - v.y;
00977     float dz = target.z - v.z;
00978     float value = (dx*dx) + (dy*dy) + (dz*dz);
00979
00980     if ((value == 0) || ((maxDistance >= 0) && (value <= maxDistance*maxDistance))) return target;
00981
00982     float dist = sqrtf(value);
00983
00984     result.x = v.x + dx/dist*maxDistance;
00985     result.y = v.y + dy/dist*maxDistance;
00986     result.z = v.z + dz/dist*maxDistance;
00987
00988     return result;
00989 }
00990
00991 // Calculate linear interpolation between two vectors
00992 RMAPI Vector3 Vector3Lerp(Vector3 v1, Vector3 v2, float amount)
00993 {
00994     Vector3 result = { 0 };
00995
00996     result.x = v1.x + amount*(v2.x - v1.x);
00997     result.y = v1.y + amount*(v2.y - v1.y);
00998     result.z = v1.z + amount*(v2.z - v1.z);
00999
01000     return result;
01001 }
01002
01003 // Calculate cubic hermite interpolation between two vectors and their tangents
01004 // as described in the GLTF 2.0 specification:
01005 // https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html#interpolation-cubic
01006 RMAPI Vector3 Vector3CubicHermite(Vector3 v1, Vector3 tangent1, Vector3 v2, Vector3 tangent2, float
amount)
01007 {
01008     Vector3 result = { 0 };
01009
01010     float amountPow2 = amount*amount;
01011     float amountPow3 = amount*amount*amount;
01012
01013     result.x = (2*amountPow3 - 3*amountPow2 + 1)*v1.x + (amountPow3 - 2*amountPow2 +
amount)*tangent1.x + (-2*amountPow3 + 3*amountPow2)*v2.x + (amountPow3 - amountPow2)*tangent2.x;
01014     result.y = (2*amountPow3 - 3*amountPow2 + 1)*v1.y + (amountPow3 - 2*amountPow2 +
amount)*tangent1.y + (-2*amountPow3 + 3*amountPow2)*v2.y + (amountPow3 - amountPow2)*tangent2.y;
01015     result.z = (2*amountPow3 - 3*amountPow2 + 1)*v1.z + (amountPow3 - 2*amountPow2 +
amount)*tangent1.z + (-2*amountPow3 + 3*amountPow2)*v2.z + (amountPow3 - amountPow2)*tangent2.z;
01016
01017     return result;
01018 }
01019 // Calculate reflected vector to normal
01020 RMAPI Vector3 Vector3Reflect(Vector3 v, Vector3 normal)
01021 {
01022     Vector3 result = { 0 };
01023
01024     // I is the original vector
01025     // N is the normal of the incident plane
01026     // R = I - (2*N*(DotProduct[I, N]))

```

```

01027
01028     float dotProduct = (v.x*normal.x + v.y*normal.y + v.z*normal.z);
01029
01030     result.x = v.x - (2.0f*normal.x)*dotProduct;
01031     result.y = v.y - (2.0f*normal.y)*dotProduct;
01032     result.z = v.z - (2.0f*normal.z)*dotProduct;
01033
01034     return result;
01035 }
01036
01037 // Get min value for each pair of components
01038 RMAPI Vector3 Vector3Min(Vector3 v1, Vector3 v2)
01039 {
01040     Vector3 result = { 0 };
01041
01042     result.x = fminf(v1.x, v2.x);
01043     result.y = fminf(v1.y, v2.y);
01044     result.z = fminf(v1.z, v2.z);
01045
01046     return result;
01047 }
01048
01049 // Get max value for each pair of components
01050 RMAPI Vector3 Vector3Max(Vector3 v1, Vector3 v2)
01051 {
01052     Vector3 result = { 0 };
01053
01054     result.x = fmaxf(v1.x, v2.x);
01055     result.y = fmaxf(v1.y, v2.y);
01056     result.z = fmaxf(v1.z, v2.z);
01057
01058     return result;
01059 }
01060
01061 // Compute barycenter coordinates (u, v, w) for point p with respect to triangle (a, b, c)
01062 // NOTE: Assumes P is on the plane of the triangle
01063 RMAPI Vector3 Vector3Barycenter(Vector3 p, Vector3 a, Vector3 b, Vector3 c)
01064 {
01065     Vector3 result = { 0 };
01066
01067     Vector3 v0 = { b.x - a.x, b.y - a.y, b.z - a.z }; // Vector3Subtract(b, a)
01068     Vector3 v1 = { c.x - a.x, c.y - a.y, c.z - a.z }; // Vector3Subtract(c, a)
01069     Vector3 v2 = { p.x - a.x, p.y - a.y, p.z - a.z }; // Vector3Subtract(p, a)
01070     float d00 = (v0.x*v0.x + v0.y*v0.y + v0.z*v0.z); // Vector3DotProduct(v0, v0)
01071     float d01 = (v0.x*v1.x + v0.y*v1.y + v0.z*v1.z); // Vector3DotProduct(v0, v1)
01072     float d11 = (v1.x*v1.x + v1.y*v1.y + v1.z*v1.z); // Vector3DotProduct(v1, v1)
01073     float d20 = (v2.x*v0.x + v2.y*v0.y + v2.z*v0.z); // Vector3DotProduct(v2, v0)
01074     float d21 = (v2.x*v1.x + v2.y*v1.y + v2.z*v1.z); // Vector3DotProduct(v2, v1)
01075
01076     float denom = d00*d11 - d01*d01;
01077
01078     result.y = (d11*d20 - d01*d21)/denom;
01079     result.z = (d00*d21 - d01*d20)/denom;
01080     result.x = 1.0f - (result.z + result.y);
01081
01082     return result;
01083 }
01084
01085 // Projects a Vector3 from screen space into object space
01086 // NOTE: Self-contained function, no other raymath functions are called
01087 RMAPI Vector3 Vector3Unproject(Vector3 source, Matrix projection, Matrix view)
01088 {
01089     Vector3 result = { 0 };
01090
01091     // Calculate unprojected matrix (multiply view matrix by projection matrix) and invert it
01092     Matrix matViewProj = { // MatrixMultiply(view, projection);
01093         view.m0*projection.m0 + view.m1*projection.m4 + view.m2*projection.m8 +
01094         view.m3*projection.m12,
01095         view.m0*projection.m1 + view.m1*projection.m5 + view.m2*projection.m9 +
01096         view.m3*projection.m13,
01097         view.m0*projection.m2 + view.m1*projection.m6 + view.m2*projection.m10 +
01098         view.m3*projection.m14,
01099         view.m0*projection.m3 + view.m1*projection.m7 + view.m2*projection.m11 +
01100         view.m3*projection.m15,
01101         view.m4*projection.m0 + view.m5*projection.m4 + view.m6*projection.m8 +
01102         view.m7*projection.m12,
01103         view.m4*projection.m1 + view.m5*projection.m5 + view.m6*projection.m9 +
01104         view.m7*projection.m13,
01105         view.m4*projection.m2 + view.m5*projection.m6 + view.m6*projection.m10 +
01106         view.m7*projection.m14,
01107         view.m4*projection.m3 + view.m5*projection.m7 + view.m6*projection.m11 +
01108         view.m7*projection.m15,
01109         view.m8*projection.m0 + view.m9*projection.m4 + view.m10*projection.m8 +
01110         view.m11*projection.m12,
01111         view.m8*projection.m1 + view.m9*projection.m5 + view.m10*projection.m9 +
01112         view.m11*projection.m13,
01113         view.m8*projection.m2 + view.m9*projection.m6 + view.m10*projection.m10 +
01114         view.m11*projection.m14,
01115         view.m8*projection.m3 + view.m9*projection.m7 + view.m10*projection.m11 +
01116         view.m11*projection.m15
01117     };

```

```

    view.m11*projection.m14,
01104     view.m8*projection.m3 + view.m9*projection.m7 + view.m10*projection.m11 +
    view.m11*projection.m15,
01105     view.m12*projection.m0 + view.m13*projection.m4 + view.m14*projection.m8 +
    view.m15*projection.m12,
01106     view.m12*projection.m1 + view.m13*projection.m5 + view.m14*projection.m9 +
    view.m15*projection.m13,
01107     view.m12*projection.m2 + view.m13*projection.m6 + view.m14*projection.m10 +
    view.m15*projection.m14,
01108     view.m12*projection.m3 + view.m13*projection.m7 + view.m14*projection.m11 +
    view.m15*projection.m15 };
01109
01110     // Calculate inverted matrix -> MatrixInvert(matViewProj);
01111     // Cache the matrix values (speed optimization)
01112     float a00 = matViewProj.m0, a01 = matViewProj.m1, a02 = matViewProj.m2, a03 = matViewProj.m3;
01113     float a10 = matViewProj.m4, a11 = matViewProj.m5, a12 = matViewProj.m6, a13 = matViewProj.m7;
01114     float a20 = matViewProj.m8, a21 = matViewProj.m9, a22 = matViewProj.m10, a23 = matViewProj.m11;
01115     float a30 = matViewProj.m12, a31 = matViewProj.m13, a32 = matViewProj.m14, a33 = matViewProj.m15;
01116
01117     float b00 = a00*a11 - a01*a10;
01118     float b01 = a00*a12 - a02*a10;
01119     float b02 = a00*a13 - a03*a10;
01120     float b03 = a01*a12 - a02*a11;
01121     float b04 = a01*a13 - a03*a11;
01122     float b05 = a02*a13 - a03*a12;
01123     float b06 = a20*a31 - a21*a30;
01124     float b07 = a20*a32 - a22*a30;
01125     float b08 = a20*a33 - a23*a30;
01126     float b09 = a21*a32 - a22*a31;
01127     float b10 = a21*a33 - a23*a31;
01128     float b11 = a22*a33 - a23*a32;
01129
01130     // Calculate the invert determinant (inlined to avoid double-caching)
01131     float invDet = 1.0f/(b00*b11 - b01*b10 + b02*b09 + b03*b08 - b04*b07 + b05*b06);
01132
01133     Matrix matViewProjInv = {
01134         (a11*b11 - a12*b10 + a13*b09)*invDet,
01135         (-a01*b11 + a02*b10 - a03*b09)*invDet,
01136         (a31*b05 - a32*b04 + a33*b03)*invDet,
01137         (-a21*b05 + a22*b04 - a23*b03)*invDet,
01138         (-a10*b11 + a12*b08 - a13*b07)*invDet,
01139         (a00*b11 - a02*b08 + a03*b07)*invDet,
01140         (-a30*b05 + a32*b02 - a33*b01)*invDet,
01141         (a20*b05 - a22*b02 + a23*b01)*invDet,
01142         (a10*b10 - a11*b08 + a13*b06)*invDet,
01143         (-a00*b10 + a01*b08 - a03*b06)*invDet,
01144         (a30*b04 - a31*b02 + a33*b00)*invDet,
01145         (-a20*b04 + a21*b02 - a23*b00)*invDet,
01146         (-a10*b09 + a11*b07 - a12*b06)*invDet,
01147         (a00*b09 - a01*b07 + a02*b06)*invDet,
01148         (-a30*b03 + a31*b01 - a32*b00)*invDet,
01149         (a20*b03 - a21*b01 + a22*b00)*invDet };
01150
01151     // Create quaternion from source point
01152     Quaternion quat = { source.x, source.y, source.z, 1.0f };
01153
01154     // Multiply quat point by unprojected matrix
01155     Quaternion qtransformed = { // QuaternionTransform(quat, matViewProjInv)
01156         matViewProjInv.m0*quat.x + matViewProjInv.m4*quat.y + matViewProjInv.m8*quat.z +
    matViewProjInv.m12*quat.w,
01157         matViewProjInv.m1*quat.x + matViewProjInv.m5*quat.y + matViewProjInv.m9*quat.z +
    matViewProjInv.m13*quat.w,
01158         matViewProjInv.m2*quat.x + matViewProjInv.m6*quat.y + matViewProjInv.m10*quat.z +
    matViewProjInv.m14*quat.w,
01159         matViewProjInv.m3*quat.x + matViewProjInv.m7*quat.y + matViewProjInv.m11*quat.z +
    matViewProjInv.m15*quat.w };
01160
01161     // Normalized world points in vectors
01162     result.x = qtransformed.x/qtransformed.w;
01163     result.y = qtransformed.y/qtransformed.w;
01164     result.z = qtransformed.z/qtransformed.w;
01165
01166     return result;
01167 }
01168
01169 // Get Vector3 as float array
01170 RMAPI float3 Vector3ToFloatV(Vector3 v)
01171 {
01172     float3 buffer = { 0 };
01173
01174     buffer.v[0] = v.x;
01175     buffer.v[1] = v.y;
01176     buffer.v[2] = v.z;
01177
01178     return buffer;
01179 }
01180

```

```

01181 // Invert the given vector
01182 RMAPI Vector3 Vector3Invert(Vector3 v)
01183 {
01184     Vector3 result = { 1.0f/v.x, 1.0f/v.y, 1.0f/v.z };
01185
01186     return result;
01187 }
01188
01189 // Clamp the components of the vector between
01190 // min and max values specified by the given vectors
01191 RMAPI Vector3 Vector3Clamp(Vector3 v, Vector3 min, Vector3 max)
01192 {
01193     Vector3 result = { 0 };
01194
01195     result.x = fminf(max.x, fmaxf(min.x, v.x));
01196     result.y = fminf(max.y, fmaxf(min.y, v.y));
01197     result.z = fminf(max.z, fmaxf(min.z, v.z));
01198
01199     return result;
01200 }
01201
01202 // Clamp the magnitude of the vector between two values
01203 RMAPI Vector3 Vector3ClampValue(Vector3 v, float min, float max)
01204 {
01205     Vector3 result = v;
01206
01207     float length = (v.x*v.x) + (v.y*v.y) + (v.z*v.z);
01208     if (length > 0.0f)
01209     {
01210         length = sqrtf(length);
01211
01212         float scale = 1; // By default, 1 as the neutral element
01213         if (length < min) scale = min/length;
01214         else if (length > max) scale = max/length;
01215
01216         result.x = v.x*scale;
01217         result.y = v.y*scale;
01218         result.z = v.z*scale;
01219     }
01220
01221     return result;
01222 }
01223
01224 // Check whether two given vectors are almost equal
01225 RMAPI int Vector3Equals(Vector3 p, Vector3 q)
01226 {
01227     #if !defined(EPSILON)
01228         #define EPSILON 0.000001f
01229     #endif
01230
01231     int result = ((fabsf(p.x - q.x)) <= (EPSILON*fmaxf(1.0f, fmaxf(fabsf(p.x), fabsf(q.x))))) &&
01232                 ((fabsf(p.y - q.y)) <= (EPSILON*fmaxf(1.0f, fmaxf(fabsf(p.y), fabsf(q.y))))) &&
01233                 ((fabsf(p.z - q.z)) <= (EPSILON*fmaxf(1.0f, fmaxf(fabsf(p.z), fabsf(q.z))))) &&
01234
01235     return result;
01236 }
01237
01238 // Compute the direction of a refracted ray
01239 // v: normalized direction of the incoming ray
01240 // n: normalized normal vector of the interface of two optical media
01241 // r: ratio of the refractive index of the medium from where the ray comes
01242 // to the refractive index of the medium on the other side of the surface
01243 RMAPI Vector3 Vector3Refract(Vector3 v, Vector3 n, float r)
01244 {
01245     Vector3 result = { 0 };
01246
01247     float dot = v.x*n.x + v.y*n.y + v.z*n.z;
01248     float d = 1.0f - r*r*(1.0f - dot*dot);
01249
01250     if (d >= 0.0f)
01251     {
01252         d = sqrtf(d);
01253         v.x = r*v.x - (r*dot + d)*n.x;
01254         v.y = r*v.y - (r*dot + d)*n.y;
01255         v.z = r*v.z - (r*dot + d)*n.z;
01256
01257         result = v;
01258     }
01259
01260     return result;
01261 }
01262
01263
01264 //-----
01265 // Module Functions Definition - Vector4 math
01266 //-----
01267 // Get vector zero

```

```

01268 RMAPI Vector4 Vector4Zero(void)
01269 {
01270     Vector4 result = { 0.0f, 0.0f, 0.0f, 0.0f };
01271     return result;
01272 }
01273
01274 // Get vector one
01275 RMAPI Vector4 Vector4One(void)
01276 {
01277     Vector4 result = { 1.0f, 1.0f, 1.0f, 1.0f };
01278     return result;
01279 }
01280
01281 // Add two vectors
01282 RMAPI Vector4 Vector4Add(Vector4 v1, Vector4 v2)
01283 {
01284     Vector4 result = {
01285         v1.x + v2.x,
01286         v1.y + v2.y,
01287         v1.z + v2.z,
01288         v1.w + v2.w
01289     };
01290     return result;
01291 }
01292
01293 // Add value to vector components
01294 RMAPI Vector4 Vector4AddValue(Vector4 v, float add)
01295 {
01296     Vector4 result = {
01297         v.x + add,
01298         v.y + add,
01299         v.z + add,
01300         v.w + add
01301     };
01302     return result;
01303 }
01304
01305 // Subtract vectors
01306 RMAPI Vector4 Vector4Subtract(Vector4 v1, Vector4 v2)
01307 {
01308     Vector4 result = {
01309         v1.x - v2.x,
01310         v1.y - v2.y,
01311         v1.z - v2.z,
01312         v1.w - v2.w
01313     };
01314     return result;
01315 }
01316
01317 // Subtract value from vector components
01318 RMAPI Vector4 Vector4SubtractValue(Vector4 v, float add)
01319 {
01320     Vector4 result = {
01321         v.x - add,
01322         v.y - add,
01323         v.z - add,
01324         v.w - add
01325     };
01326     return result;
01327 }
01328
01329 // Vector length
01330 RMAPI float Vector4Length(Vector4 v)
01331 {
01332     float result = sqrtf((v.x*v.x) + (v.y*v.y) + (v.z*v.z) + (v.w*v.w));
01333     return result;
01334 }
01335
01336 // Vector square length
01337 RMAPI float Vector4LengthSqr(Vector4 v)
01338 {
01339     float result = (v.x*v.x) + (v.y*v.y) + (v.z*v.z) + (v.w*v.w);
01340     return result;
01341 }
01342
01343 // Vectors dot product
01344 RMAPI float Vector4DotProduct(Vector4 v1, Vector4 v2)
01345 {
01346     float result = (v1.x*v2.x + v1.y*v2.y + v1.z*v2.z + v1.w*v2.w);
01347     return result;
01348 }
01349
01350 // Calculate distance between two vectors
01351 RMAPI float Vector4Distance(Vector4 v1, Vector4 v2)
01352 {
01353     float result = sqrtf(
01354         (v1.x - v2.x)*(v1.x - v2.x) + (v1.y - v2.y)*(v1.y - v2.y) +

```



```

01355         (v1.z - v2.z)*(v1.z - v2.z) + (v1.w - v2.w)*(v1.w - v2.w));
01356     return result;
01357 }
01358
01359 // Calculate square distance between two vectors
01360 RMAPI float Vector4DistanceSqr(Vector4 v1, Vector4 v2)
01361 {
01362     float result =
01363         (v1.x - v2.x)*(v1.x - v2.x) + (v1.y - v2.y)*(v1.y - v2.y) +
01364         (v1.z - v2.z)*(v1.z - v2.z) + (v1.w - v2.w)*(v1.w - v2.w);
01365
01366     return result;
01367 }
01368
01369 // Scale vector components by value (multiply)
01370 RMAPI Vector4 Vector4Scale(Vector4 v, float scale)
01371 {
01372     Vector4 result = { v.x*scale, v.y*scale, v.z*scale, v.w*scale };
01373     return result;
01374 }
01375
01376 // Multiply vector by vector
01377 RMAPI Vector4 Vector4Multiply(Vector4 v1, Vector4 v2)
01378 {
01379     Vector4 result = { v1.x*v2.x, v1.y*v2.y, v1.z*v2.z, v1.w*v2.w };
01380     return result;
01381 }
01382
01383 // Negate vector
01384 RMAPI Vector4 Vector4Negate(Vector4 v)
01385 {
01386     Vector4 result = { -v.x, -v.y, -v.z, -v.w };
01387     return result;
01388 }
01389
01390 // Divide vector by vector
01391 RMAPI Vector4 Vector4Divide(Vector4 v1, Vector4 v2)
01392 {
01393     Vector4 result = { v1.x/v2.x, v1.y/v2.y, v1.z/v2.z, v1.w/v2.w };
01394     return result;
01395 }
01396
01397 // Normalize provided vector
01398 RMAPI Vector4 Vector4Normalize(Vector4 v)
01399 {
01400     Vector4 result = { 0 };
01401     float length = sqrtf((v.x*v.x) + (v.y*v.y) + (v.z*v.z) + (v.w*v.w));
01402
01403     if (length > 0)
01404     {
01405         float ilength = 1.0f/length;
01406         result.x = v.x*ilength;
01407         result.y = v.y*ilength;
01408         result.z = v.z*ilength;
01409         result.w = v.w*ilength;
01410     }
01411
01412     return result;
01413 }
01414
01415 // Get min value for each pair of components
01416 RMAPI Vector4 Vector4Min(Vector4 v1, Vector4 v2)
01417 {
01418     Vector4 result = { 0 };
01419
01420     result.x = fminf(v1.x, v2.x);
01421     result.y = fminf(v1.y, v2.y);
01422     result.z = fminf(v1.z, v2.z);
01423     result.w = fminf(v1.w, v2.w);
01424
01425     return result;
01426 }
01427
01428 // Get max value for each pair of components
01429 RMAPI Vector4 Vector4Max(Vector4 v1, Vector4 v2)
01430 {
01431     Vector4 result = { 0 };
01432
01433     result.x = fmaxf(v1.x, v2.x);
01434     result.y = fmaxf(v1.y, v2.y);
01435     result.z = fmaxf(v1.z, v2.z);
01436     result.w = fmaxf(v1.w, v2.w);
01437
01438     return result;
01439 }
01440
01441 // Calculate linear interpolation between two vectors

```

```

01442 RMAPI Vector4 Vector4Lerp(Vector4 v1, Vector4 v2, float amount)
01443 {
01444     Vector4 result = { 0 };
01445
01446     result.x = v1.x + amount*(v2.x - v1.x);
01447     result.y = v1.y + amount*(v2.y - v1.y);
01448     result.z = v1.z + amount*(v2.z - v1.z);
01449     result.w = v1.w + amount*(v2.w - v1.w);
01450
01451     return result;
01452 }
01453
01454 // Move Vector towards target
01455 RMAPI Vector4 Vector4MoveTowards(Vector4 v, Vector4 target, float maxDistance)
01456 {
01457     Vector4 result = { 0 };
01458
01459     float dx = target.x - v.x;
01460     float dy = target.y - v.y;
01461     float dz = target.z - v.z;
01462     float dw = target.w - v.w;
01463     float value = (dx*dx) + (dy*dy) + (dz*dz) + (dw*dw);
01464
01465     if ((value == 0) || ((maxDistance >= 0) && (value <= maxDistance*maxDistance))) return target;
01466
01467     float dist = sqrtf(value);
01468
01469     result.x = v.x + dx/dist*maxDistance;
01470     result.y = v.y + dy/dist*maxDistance;
01471     result.z = v.z + dz/dist*maxDistance;
01472     result.w = v.w + dw/dist*maxDistance;
01473
01474     return result;
01475 }
01476
01477 // Invert the given vector
01478 RMAPI Vector4 Vector4Invert(Vector4 v)
01479 {
01480     Vector4 result = { 1.0f/v.x, 1.0f/v.y, 1.0f/v.z, 1.0f/v.w };
01481     return result;
01482 }
01483
01484 // Check whether two given vectors are almost equal
01485 RMAPI int Vector4Equals(Vector4 p, Vector4 q)
01486 {
01487     #if !defined(EPSILON)
01488         #define EPSILON 0.000001f
01489     #endif
01490
01491     int result = ((fabsf(p.x - q.x)) <= (EPSILON*fmaxf(1.0f, fmaxf(fabsf(p.x), fabsf(q.x))))) &&
01492                 ((fabsf(p.y - q.y)) <= (EPSILON*fmaxf(1.0f, fmaxf(fabsf(p.y), fabsf(q.y))))) &&
01493                 ((fabsf(p.z - q.z)) <= (EPSILON*fmaxf(1.0f, fmaxf(fabsf(p.z), fabsf(q.z))))) &&
01494                 ((fabsf(p.w - q.w)) <= (EPSILON*fmaxf(1.0f, fmaxf(fabsf(p.w), fabsf(q.w))))) &&
01495     return result;
01496 }
01497
01498
01499 //-----
01500 // Module Functions Definition - Matrix math
01501 //-----
01502
01503 // Compute matrix determinant
01504 RMAPI float MatrixDeterminant(Matrix mat)
01505 {
01506     float result = 0.0f;
01507     /*
01508     // Cache the matrix values (speed optimization)
01509     float a00 = mat.m0, a01 = mat.m1, a02 = mat.m2, a03 = mat.m3;
01510     float a10 = mat.m4, a11 = mat.m5, a12 = mat.m6, a13 = mat.m7;
01511     float a20 = mat.m8, a21 = mat.m9, a22 = mat.m10, a23 = mat.m11;
01512     float a30 = mat.m12, a31 = mat.m13, a32 = mat.m14, a33 = mat.m15;
01513
01514     // NOTE: It takes 72 multiplication to calculate 4x4 matrix determinant
01515     result = a30*a21*a12*a03 - a20*a31*a12*a03 - a30*a11*a22*a03 + a10*a31*a22*a03 +
01516             a20*a11*a32*a03 - a10*a21*a32*a03 - a30*a21*a02*a13 + a20*a31*a02*a13 +
01517             a30*a01*a22*a13 - a00*a31*a22*a13 - a20*a01*a32*a13 + a00*a21*a32*a13 +
01518             a30*a11*a02*a23 - a10*a31*a02*a23 - a30*a01*a12*a23 + a00*a31*a12*a23 +
01519             a10*a01*a32*a23 - a00*a11*a32*a23 - a20*a11*a02*a33 + a10*a21*a02*a33 +
01520             a20*a01*a12*a33 - a00*a21*a12*a33 - a10*a01*a22*a33 + a00*a11*a22*a33;
01521     */
01522     // Using Laplace expansion (https://en.wikipedia.org/wiki/Laplace_expansion),
01523     // previous operation can be simplified to 40 multiplications, decreasing matrix
01524     // size from 4x4 to 2x2 using minors
01525
01526     // Cache the matrix values (speed optimization)
01527     float m0 = mat.m0, m1 = mat.m1, m2 = mat.m2, m3 = mat.m3;
01528     float m4 = mat.m4, m5 = mat.m5, m6 = mat.m6, m7 = mat.m7;

```

```

01529     float m8 = mat.m8, m9 = mat.m9, m10 = mat.m10, m11 = mat.m11;
01530     float m12 = mat.m12, m13 = mat.m13, m14 = mat.m14, m15 = mat.m15;
01531
01532     result = (m0*((m5*(m10*m15 - m11*m14) - m9*(m6*m15 - m7*m14) + m13*(m6*m11 - m7*m10))) -
01533             m4*((m1*(m10*m15 - m11*m14) - m9*(m2*m15 - m3*m14) + m13*(m2*m11 - m3*m10))) +
01534             m8*((m1*(m6*m15 - m7*m14) - m5*(m2*m15 - m3*m14) + m13*(m2*m7 - m3*m6))) -
01535             m12*((m1*(m6*m11 - m7*m10) - m5*(m2*m11 - m3*m10) + m9*(m2*m7 - m3*m6))));
01536
01537     return result;
01538 }
01539
01540 // Get the trace of the matrix (sum of the values along the diagonal)
01541 RMAPI float MatrixTrace(Matrix mat)
01542 {
01543     float result = (mat.m0 + mat.m5 + mat.m10 + mat.m15);
01544
01545     return result;
01546 }
01547
01548 // Transposes provided matrix
01549 RMAPI Matrix MatrixTranspose(Matrix mat)
01550 {
01551     Matrix result = { 0 };
01552
01553     result.m0 = mat.m0;
01554     result.m1 = mat.m4;
01555     result.m2 = mat.m8;
01556     result.m3 = mat.m12;
01557     result.m4 = mat.m1;
01558     result.m5 = mat.m5;
01559     result.m6 = mat.m9;
01560     result.m7 = mat.m13;
01561     result.m8 = mat.m2;
01562     result.m9 = mat.m6;
01563     result.m10 = mat.m10;
01564     result.m11 = mat.m14;
01565     result.m12 = mat.m3;
01566     result.m13 = mat.m7;
01567     result.m14 = mat.m11;
01568     result.m15 = mat.m15;
01569
01570     return result;
01571 }
01572
01573 // Invert provided matrix
01574 RMAPI Matrix MatrixInvert(Matrix mat)
01575 {
01576     Matrix result = { 0 };
01577
01578     // Cache the matrix values (speed optimization)
01579     float a00 = mat.m0, a01 = mat.m1, a02 = mat.m2, a03 = mat.m3;
01580     float a10 = mat.m4, a11 = mat.m5, a12 = mat.m6, a13 = mat.m7;
01581     float a20 = mat.m8, a21 = mat.m9, a22 = mat.m10, a23 = mat.m11;
01582     float a30 = mat.m12, a31 = mat.m13, a32 = mat.m14, a33 = mat.m15;
01583
01584     float b00 = a00*a11 - a01*a10;
01585     float b01 = a00*a12 - a02*a10;
01586     float b02 = a00*a13 - a03*a10;
01587     float b03 = a01*a12 - a02*a11;
01588     float b04 = a01*a13 - a03*a11;
01589     float b05 = a02*a13 - a03*a12;
01590     float b06 = a20*a31 - a21*a30;
01591     float b07 = a20*a32 - a22*a30;
01592     float b08 = a20*a33 - a23*a30;
01593     float b09 = a21*a32 - a22*a31;
01594     float b10 = a21*a33 - a23*a31;
01595     float b11 = a22*a33 - a23*a32;
01596
01597     // Calculate the invert determinant (inlined to avoid double-caching)
01598     float invDet = 1.0f/(b00*b11 - b01*b10 + b02*b09 + b03*b08 - b04*b07 + b05*b06);
01599
01600     result.m0 = (a11*b11 - a12*b10 + a13*b09)*invDet;
01601     result.m1 = (-a01*b11 + a02*b10 - a03*b09)*invDet;
01602     result.m2 = (a31*b05 - a32*b04 + a33*b03)*invDet;
01603     result.m3 = (-a21*b05 + a22*b04 - a23*b03)*invDet;
01604     result.m4 = (-a10*b11 + a12*b08 - a13*b07)*invDet;
01605     result.m5 = (a00*b11 - a02*b08 + a03*b07)*invDet;
01606     result.m6 = (-a30*b05 + a32*b02 - a33*b01)*invDet;
01607     result.m7 = (a20*b05 - a22*b02 + a23*b01)*invDet;
01608     result.m8 = (a10*b10 - a11*b08 + a13*b06)*invDet;
01609     result.m9 = (-a00*b10 + a01*b08 - a03*b06)*invDet;
01610     result.m10 = (a30*b04 - a31*b02 + a33*b00)*invDet;
01611     result.m11 = (-a20*b04 + a21*b02 - a23*b00)*invDet;
01612     result.m12 = (-a10*b09 + a11*b07 - a12*b06)*invDet;
01613     result.m13 = (a00*b09 - a01*b07 + a02*b06)*invDet;
01614     result.m14 = (-a30*b03 + a31*b01 - a32*b00)*invDet;
01615     result.m15 = (a20*b03 - a21*b01 + a22*b00)*invDet;

```

```

01616
01617     return result;
01618 }
01619
01620 // Get identity matrix
01621 RMAPI Matrix MatrixIdentity(void)
01622 {
01623     Matrix result = { 1.0f, 0.0f, 0.0f, 0.0f,
01624                      0.0f, 1.0f, 0.0f, 0.0f,
01625                      0.0f, 0.0f, 1.0f, 0.0f,
01626                      0.0f, 0.0f, 0.0f, 1.0f };
01627
01628     return result;
01629 }
01630
01631 // Add two matrices
01632 RMAPI Matrix MatrixAdd(Matrix left, Matrix right)
01633 {
01634     Matrix result = { 0 };
01635
01636     result.m0 = left.m0 + right.m0;
01637     result.m1 = left.m1 + right.m1;
01638     result.m2 = left.m2 + right.m2;
01639     result.m3 = left.m3 + right.m3;
01640     result.m4 = left.m4 + right.m4;
01641     result.m5 = left.m5 + right.m5;
01642     result.m6 = left.m6 + right.m6;
01643     result.m7 = left.m7 + right.m7;
01644     result.m8 = left.m8 + right.m8;
01645     result.m9 = left.m9 + right.m9;
01646     result.m10 = left.m10 + right.m10;
01647     result.m11 = left.m11 + right.m11;
01648     result.m12 = left.m12 + right.m12;
01649     result.m13 = left.m13 + right.m13;
01650     result.m14 = left.m14 + right.m14;
01651     result.m15 = left.m15 + right.m15;
01652
01653     return result;
01654 }
01655
01656 // Subtract two matrices (left - right)
01657 RMAPI Matrix MatrixSubtract(Matrix left, Matrix right)
01658 {
01659     Matrix result = { 0 };
01660
01661     result.m0 = left.m0 - right.m0;
01662     result.m1 = left.m1 - right.m1;
01663     result.m2 = left.m2 - right.m2;
01664     result.m3 = left.m3 - right.m3;
01665     result.m4 = left.m4 - right.m4;
01666     result.m5 = left.m5 - right.m5;
01667     result.m6 = left.m6 - right.m6;
01668     result.m7 = left.m7 - right.m7;
01669     result.m8 = left.m8 - right.m8;
01670     result.m9 = left.m9 - right.m9;
01671     result.m10 = left.m10 - right.m10;
01672     result.m11 = left.m11 - right.m11;
01673     result.m12 = left.m12 - right.m12;
01674     result.m13 = left.m13 - right.m13;
01675     result.m14 = left.m14 - right.m14;
01676     result.m15 = left.m15 - right.m15;
01677
01678     return result;
01679 }
01680
01681 // Get two matrix multiplication
01682 // NOTE: When multiplying matrices... the order matters!
01683 RMAPI Matrix MatrixMultiply(Matrix left, Matrix right)
01684 {
01685     Matrix result = { 0 };
01686
01687     #if defined(RAYMATH_SSE_ENABLED)
01688         // Load left side and right side
01689         __m128 c0 = _mm_set_ps(right.m12, right.m8, right.m4, right.m0);
01690         __m128 c1 = _mm_set_ps(right.m13, right.m9, right.m5, right.m1);
01691         __m128 c2 = _mm_set_ps(right.m14, right.m10, right.m6, right.m2);
01692         __m128 c3 = _mm_set_ps(right.m15, right.m11, right.m7, right.m3);
01693
01694         // Transpose so c0..c3 become *rows* of the right matrix in semantic order
01695         _MM_TRANSPOSE4_PS(c0, c1, c2, c3);
01696
01697         float tmp[4] = { 0 };
01698         __m128 row;
01699
01700         // Row 0 of result: [m0, m1, m2, m3]
01701         row = _mm_mul_ps(_mm_set1_ps(left.m0), c0);
01702         row = _mm_add_ps(row, _mm_mul_ps(_mm_set1_ps(left.m1), c1));

```

```

01703     row = _mm_add_ps(row, _mm_mul_ps(_mm_set1_ps(left.m2), c2));
01704     row = _mm_add_ps(row, _mm_mul_ps(_mm_set1_ps(left.m3), c3));
01705     _mm_storeu_ps(tmp, row);
01706     result.m0 = tmp[0];
01707     result.m1 = tmp[1];
01708     result.m2 = tmp[2];
01709     result.m3 = tmp[3];
01710
01711     // Row 1 of result: [m4, m5, m6, m7]
01712     row = _mm_mul_ps(_mm_set1_ps(left.m4), c0);
01713     row = _mm_add_ps(row, _mm_mul_ps(_mm_set1_ps(left.m5), c1));
01714     row = _mm_add_ps(row, _mm_mul_ps(_mm_set1_ps(left.m6), c2));
01715     row = _mm_add_ps(row, _mm_mul_ps(_mm_set1_ps(left.m7), c3));
01716     _mm_storeu_ps(tmp, row);
01717     result.m4 = tmp[0];
01718     result.m5 = tmp[1];
01719     result.m6 = tmp[2];
01720     result.m7 = tmp[3];
01721
01722     // Row 2 of result: [m8, m9, m10, m11]
01723     row = _mm_mul_ps(_mm_set1_ps(left.m8), c0);
01724     row = _mm_add_ps(row, _mm_mul_ps(_mm_set1_ps(left.m9), c1));
01725     row = _mm_add_ps(row, _mm_mul_ps(_mm_set1_ps(left.m10), c2));
01726     row = _mm_add_ps(row, _mm_mul_ps(_mm_set1_ps(left.m11), c3));
01727     _mm_storeu_ps(tmp, row);
01728     result.m8 = tmp[0];
01729     result.m9 = tmp[1];
01730     result.m10 = tmp[2];
01731     result.m11 = tmp[3];
01732
01733     // Row 3 of result: [m12, m13, m14, m15]
01734     row = _mm_mul_ps(_mm_set1_ps(left.m12), c0);
01735     row = _mm_add_ps(row, _mm_mul_ps(_mm_set1_ps(left.m13), c1));
01736     row = _mm_add_ps(row, _mm_mul_ps(_mm_set1_ps(left.m14), c2));
01737     row = _mm_add_ps(row, _mm_mul_ps(_mm_set1_ps(left.m15), c3));
01738     _mm_storeu_ps(tmp, row);
01739     result.m12 = tmp[0];
01740     result.m13 = tmp[1];
01741     result.m14 = tmp[2];
01742     result.m15 = tmp[3];
01743 #else
01744     result.m0 = left.m0*right.m0 + left.m1*right.m4 + left.m2*right.m8 + left.m3*right.m12;
01745     result.m1 = left.m0*right.m1 + left.m1*right.m5 + left.m2*right.m9 + left.m3*right.m13;
01746     result.m2 = left.m0*right.m2 + left.m1*right.m6 + left.m2*right.m10 + left.m3*right.m14;
01747     result.m3 = left.m0*right.m3 + left.m1*right.m7 + left.m2*right.m11 + left.m3*right.m15;
01748     result.m4 = left.m4*right.m0 + left.m5*right.m4 + left.m6*right.m8 + left.m7*right.m12;
01749     result.m5 = left.m4*right.m1 + left.m5*right.m5 + left.m6*right.m9 + left.m7*right.m13;
01750     result.m6 = left.m4*right.m2 + left.m5*right.m6 + left.m6*right.m10 + left.m7*right.m14;
01751     result.m7 = left.m4*right.m3 + left.m5*right.m7 + left.m6*right.m11 + left.m7*right.m15;
01752     result.m8 = left.m8*right.m0 + left.m9*right.m4 + left.m10*right.m8 + left.m11*right.m12;
01753     result.m9 = left.m8*right.m1 + left.m9*right.m5 + left.m10*right.m9 + left.m11*right.m13;
01754     result.m10 = left.m8*right.m2 + left.m9*right.m6 + left.m10*right.m10 + left.m11*right.m14;
01755     result.m11 = left.m8*right.m3 + left.m9*right.m7 + left.m10*right.m11 + left.m11*right.m15;
01756     result.m12 = left.m12*right.m0 + left.m13*right.m4 + left.m14*right.m8 + left.m15*right.m12;
01757     result.m13 = left.m12*right.m1 + left.m13*right.m5 + left.m14*right.m9 + left.m15*right.m13;
01758     result.m14 = left.m12*right.m2 + left.m13*right.m6 + left.m14*right.m10 + left.m15*right.m14;
01759     result.m15 = left.m12*right.m3 + left.m13*right.m7 + left.m14*right.m11 + left.m15*right.m15;
01760 #endif
01761
01762     return result;
01763 }
01764
01765 // Multiply matrix components by value
01766 RMAPI Matrix MatrixMultiplyValue(Matrix left, float value)
01767 {
01768     Matrix result = {
01769         left.m0*value, left.m4*value, left.m8*value, left.m12*value,
01770         left.m1*value, left.m5*value, left.m9*value, left.m13*value,
01771         left.m2*value, left.m6*value, left.m10*value, left.m14*value,
01772         left.m3*value, left.m7*value, left.m11*value, left.m15*value
01773     };
01774
01775     return result;
01776 }
01777
01778 // Get translation matrix
01779 RMAPI Matrix MatrixTranslate(float x, float y, float z)
01780 {
01781     Matrix result = { 1.0f, 0.0f, 0.0f, x,
01782                     0.0f, 1.0f, 0.0f, y,
01783                     0.0f, 0.0f, 1.0f, z,
01784                     0.0f, 0.0f, 0.0f, 1.0f };
01785
01786     return result;
01787 }
01788
01789 // Create rotation matrix from axis and angle

```

```

01790 // NOTE: Angle should be provided in radians
01791 RMAPI Matrix MatrixRotate(Vector3 axis, float angle)
01792 {
01793     Matrix result = { 0 };
01794
01795     float x = axis.x, y = axis.y, z = axis.z;
01796
01797     float lengthSquared = x*x + y*y + z*z;
01798
01799     if ((lengthSquared != 1.0f) && (lengthSquared != 0.0f))
01800     {
01801         float ilength = 1.0f/sqrtf(lengthSquared);
01802         x *= ilength;
01803         y *= ilength;
01804         z *= ilength;
01805     }
01806
01807     float sinres = sinf(angle);
01808     float cosres = cosf(angle);
01809     float t = 1.0f - cosres;
01810
01811     result.m0 = x*x*t + cosres;
01812     result.m1 = y*x*t + z*sinres;
01813     result.m2 = z*x*t - y*sinres;
01814     result.m3 = 0.0f;
01815
01816     result.m4 = x*y*t - z*sinres;
01817     result.m5 = y*y*t + cosres;
01818     result.m6 = z*y*t + x*sinres;
01819     result.m7 = 0.0f;
01820
01821     result.m8 = x*z*t + y*sinres;
01822     result.m9 = y*z*t - x*sinres;
01823     result.m10 = z*z*t + cosres;
01824     result.m11 = 0.0f;
01825
01826     result.m12 = 0.0f;
01827     result.m13 = 0.0f;
01828     result.m14 = 0.0f;
01829     result.m15 = 1.0f;
01830
01831     return result;
01832 }
01833
01834 // Get x-rotation matrix
01835 // NOTE: Angle must be provided in radians
01836 RMAPI Matrix MatrixRotateX(float angle)
01837 {
01838     Matrix result = { 1.0f, 0.0f, 0.0f, 0.0f,
01839                     0.0f, 1.0f, 0.0f, 0.0f,
01840                     0.0f, 0.0f, 1.0f, 0.0f,
01841                     0.0f, 0.0f, 0.0f, 1.0f }; // MatrixIdentity()
01842
01843     float cosres = cosf(angle);
01844     float sinres = sinf(angle);
01845
01846     result.m5 = cosres;
01847     result.m6 = sinres;
01848     result.m9 = -sinres;
01849     result.m10 = cosres;
01850
01851     return result;
01852 }
01853
01854 // Get y-rotation matrix
01855 // NOTE: Angle must be provided in radians
01856 RMAPI Matrix MatrixRotateY(float angle)
01857 {
01858     Matrix result = { 1.0f, 0.0f, 0.0f, 0.0f,
01859                     0.0f, 1.0f, 0.0f, 0.0f,
01860                     0.0f, 0.0f, 1.0f, 0.0f,
01861                     0.0f, 0.0f, 0.0f, 1.0f }; // MatrixIdentity()
01862
01863     float cosres = cosf(angle);
01864     float sinres = sinf(angle);
01865
01866     result.m0 = cosres;
01867     result.m2 = -sinres;
01868     result.m8 = sinres;
01869     result.m10 = cosres;
01870
01871     return result;
01872 }
01873
01874 // Get z-rotation matrix
01875 // NOTE: Angle must be provided in radians
01876 RMAPI Matrix MatrixRotateZ(float angle)

```

```

01877 {
01878     Matrix result = { 1.0f, 0.0f, 0.0f, 0.0f,
01879                     0.0f, 1.0f, 0.0f, 0.0f,
01880                     0.0f, 0.0f, 1.0f, 0.0f,
01881                     0.0f, 0.0f, 0.0f, 1.0f }; // MatrixIdentity()
01882
01883     float cosres = cosf(angle);
01884     float sinres = sinf(angle);
01885
01886     result.m0 = cosres;
01887     result.m1 = sinres;
01888     result.m4 = -sinres;
01889     result.m5 = cosres;
01890
01891     return result;
01892 }
01893
01894
01895 // Get xyz-rotation matrix
01896 // NOTE: Angle must be provided in radians
01897 RMAPI Matrix MatrixRotateXYZ(Vector3 angle)
01898 {
01899     Matrix result = { 1.0f, 0.0f, 0.0f, 0.0f,
01900                     0.0f, 1.0f, 0.0f, 0.0f,
01901                     0.0f, 0.0f, 1.0f, 0.0f,
01902                     0.0f, 0.0f, 0.0f, 1.0f }; // MatrixIdentity()
01903
01904     float cosz = cosf(-angle.z);
01905     float sinz = sinf(-angle.z);
01906     float cosy = cosf(-angle.y);
01907     float siny = sinf(-angle.y);
01908     float cosx = cosf(-angle.x);
01909     float sinx = sinf(-angle.x);
01910
01911     result.m0 = cosz*cosy;
01912     result.m1 = (cosz*siny*sinx) - (sinz*cosx);
01913     result.m2 = (cosz*siny*cosx) + (sinz*sinx);
01914
01915     result.m4 = sinz*cosy;
01916     result.m5 = (sinz*siny*sinx) + (cosz*cosx);
01917     result.m6 = (sinz*siny*cosx) - (cosz*sinx);
01918
01919     result.m8 = -siny;
01920     result.m9 = cosy*sinx;
01921     result.m10 = cosy*cosx;
01922
01923     return result;
01924 }
01925
01926 // Get zyx-rotation matrix
01927 // NOTE: Angle must be provided in radians
01928 RMAPI Matrix MatrixRotateZYX(Vector3 angle)
01929 {
01930     Matrix result = { 0 };
01931
01932     float cz = cosf(angle.z);
01933     float sz = sinf(angle.z);
01934     float cy = cosf(angle.y);
01935     float sy = sinf(angle.y);
01936     float cx = cosf(angle.x);
01937     float sx = sinf(angle.x);
01938
01939     result.m0 = cz*cy;
01940     result.m4 = cz*sy*sx - cx*sz;
01941     result.m8 = sz*sx + cz*cx*sy;
01942     result.m12 = 0;
01943
01944     result.m1 = cy*sz;
01945     result.m5 = cz*cx + sz*sy*sx;
01946     result.m9 = cx*sz*sy - cz*sx;
01947     result.m13 = 0;
01948
01949     result.m2 = -sy;
01950     result.m6 = cy*sx;
01951     result.m10 = cy*cx;
01952     result.m14 = 0;
01953
01954     result.m3 = 0;
01955     result.m7 = 0;
01956     result.m11 = 0;
01957     result.m15 = 1;
01958
01959     return result;
01960 }
01961
01962 // Get scaling matrix
01963 RMAPI Matrix MatrixScale(float x, float y, float z)

```

```

01964 {
01965     Matrix result = { x, 0.0f, 0.0f, 0.0f,
01966                     0.0f, y, 0.0f, 0.0f,
01967                     0.0f, 0.0f, z, 0.0f,
01968                     0.0f, 0.0f, 0.0f, 1.0f };
01969
01970     return result;
01971 }
01972
01973 // Get perspective projection matrix
01974 RMAPI Matrix MatrixFrustum(double left, double right, double bottom, double top, double nearPlane,
double farPlane)
01975 {
01976     Matrix result = { 0 };
01977
01978     float rl = (float)(right - left);
01979     float tb = (float)(top - bottom);
01980     float fn = (float)(farPlane - nearPlane);
01981
01982     result.m0 = ((float)nearPlane*2.0f)/rl;
01983     result.m1 = 0.0f;
01984     result.m2 = 0.0f;
01985     result.m3 = 0.0f;
01986
01987     result.m4 = 0.0f;
01988     result.m5 = ((float)nearPlane*2.0f)/tb;
01989     result.m6 = 0.0f;
01990     result.m7 = 0.0f;
01991
01992     result.m8 = ((float)right + (float)left)/rl;
01993     result.m9 = ((float)top + (float)bottom)/tb;
01994     result.m10 = -((float)farPlane + (float)nearPlane)/fn;
01995     result.m11 = -1.0f;
01996
01997     result.m12 = 0.0f;
01998     result.m13 = 0.0f;
01999     result.m14 = -((float)farPlane*(float)nearPlane*2.0f)/fn;
02000     result.m15 = 0.0f;
02001
02002     return result;
02003 }
02004
02005 // Get perspective projection matrix
02006 // NOTE: Fovy angle must be provided in radians
02007 RMAPI Matrix MatrixPerspective(double fovy, double aspect, double nearPlane, double farPlane)
02008 {
02009     Matrix result = { 0 };
02010
02011     double top = nearPlane*tan(fovy*0.5);
02012     double bottom = -top;
02013     double right = top*aspect;
02014     double left = -right;
02015
02016     // MatrixFrustum(-right, right, -top, top, near, far);
02017     float rl = (float)(right - left);
02018     float tb = (float)(top - bottom);
02019     float fn = (float)(farPlane - nearPlane);
02020
02021     result.m0 = ((float)nearPlane*2.0f)/rl;
02022     result.m5 = ((float)nearPlane*2.0f)/tb;
02023     result.m8 = ((float)right + (float)left)/rl;
02024     result.m9 = ((float)top + (float)bottom)/tb;
02025     result.m10 = -((float)farPlane + (float)nearPlane)/fn;
02026     result.m11 = -1.0f;
02027     result.m14 = -((float)farPlane*(float)nearPlane*2.0f)/fn;
02028
02029     return result;
02030 }
02031
02032 // Get orthographic projection matrix
02033 RMAPI Matrix MatrixOrtho(double left, double right, double bottom, double top, double nearPlane,
double farPlane)
02034 {
02035     Matrix result = { 0 };
02036
02037     float rl = (float)(right - left);
02038     float tb = (float)(top - bottom);
02039     float fn = (float)(farPlane - nearPlane);
02040
02041     result.m0 = 2.0f/rl;
02042     result.m1 = 0.0f;
02043     result.m2 = 0.0f;
02044     result.m3 = 0.0f;
02045     result.m4 = 0.0f;
02046     result.m5 = 2.0f/tb;
02047     result.m6 = 0.0f;
02048     result.m7 = 0.0f;

```



```

02049     result.m8 = 0.0f;
02050     result.m9 = 0.0f;
02051     result.m10 = -2.0f/fn;
02052     result.m11 = 0.0f;
02053     result.m12 = -((float)left + (float)right)/rl;
02054     result.m13 = -((float)top + (float)bottom)/tb;
02055     result.m14 = -((float)farPlane + (float)nearPlane)/fn;
02056     result.m15 = 1.0f;
02057
02058     return result;
02059 }
02060
02061 // Get camera look-at matrix (view matrix)
02062 RMAPI Matrix MatrixLookAt(Vector3 eye, Vector3 target, Vector3 up)
02063 {
02064     Matrix result = { 0 };
02065
02066     float length = 0.0f;
02067     float ilength = 0.0f;
02068
02069     // Vector3Subtract(eye, target)
02070     Vector3 vz = { eye.x - target.x, eye.y - target.y, eye.z - target.z };
02071
02072     // Vector3Normalize(vz)
02073     Vector3 v = vz;
02074     length = sqrtf(v.x*v.x + v.y*v.y + v.z*v.z);
02075     if (length == 0.0f) length = 1.0f;
02076     ilength = 1.0f/length;
02077     vz.x *= ilength;
02078     vz.y *= ilength;
02079     vz.z *= ilength;
02080
02081     // Vector3CrossProduct(up, vz)
02082     Vector3 vx = { up.y*vz.z - up.z*vz.y, up.z*vz.x - up.x*vz.z, up.x*vz.y - up.y*vz.x };
02083
02084     // Vector3Normalize(x)
02085     v = vx;
02086     length = sqrtf(v.x*v.x + v.y*v.y + v.z*v.z);
02087     if (length == 0.0f) length = 1.0f;
02088     ilength = 1.0f/length;
02089     vx.x *= ilength;
02090     vx.y *= ilength;
02091     vx.z *= ilength;
02092
02093     // Vector3CrossProduct(vz, vx)
02094     Vector3 vy = { vz.y*vx.z - vz.z*vx.y, vz.z*vx.x - vz.x*vx.z, vz.x*vx.y - vz.y*vx.x };
02095
02096     result.m0 = vx.x;
02097     result.m1 = vy.x;
02098     result.m2 = vz.x;
02099     result.m3 = 0.0f;
02100     result.m4 = vx.y;
02101     result.m5 = vy.y;
02102     result.m6 = vz.y;
02103     result.m7 = 0.0f;
02104     result.m8 = vx.z;
02105     result.m9 = vy.z;
02106     result.m10 = vz.z;
02107     result.m11 = 0.0f;
02108     result.m12 = -(vx.x*eye.x + vx.y*eye.y + vx.z*eye.z); // Vector3DotProduct(vx, eye)
02109     result.m13 = -(vy.x*eye.x + vy.y*eye.y + vy.z*eye.z); // Vector3DotProduct(vy, eye)
02110     result.m14 = -(vz.x*eye.x + vz.y*eye.y + vz.z*eye.z); // Vector3DotProduct(vz, eye)
02111     result.m15 = 1.0f;
02112
02113     return result;
02114 }
02115
02116 // Get float array of matrix data
02117 RMAPI float16 MatrixToFloatV(Matrix mat)
02118 {
02119     float16 result = { 0 };
02120
02121     result.v[0] = mat.m0;
02122     result.v[1] = mat.m1;
02123     result.v[2] = mat.m2;
02124     result.v[3] = mat.m3;
02125     result.v[4] = mat.m4;
02126     result.v[5] = mat.m5;
02127     result.v[6] = mat.m6;
02128     result.v[7] = mat.m7;
02129     result.v[8] = mat.m8;
02130     result.v[9] = mat.m9;
02131     result.v[10] = mat.m10;
02132     result.v[11] = mat.m11;
02133     result.v[12] = mat.m12;
02134     result.v[13] = mat.m13;
02135     result.v[14] = mat.m14;

```

```

02136     result.v[15] = mat.m15;
02137
02138     return result;
02139 }
02140
02141 //-----
02142 // Module Functions Definition - Quaternion math
02143 //-----
02144
02145 // Add two quaternions
02146 RMAPI Quaternion QuaternionAdd(Quaternion q1, Quaternion q2)
02147 {
02148     Quaternion result = {q1.x + q2.x, q1.y + q2.y, q1.z + q2.z, q1.w + q2.w};
02149
02150     return result;
02151 }
02152
02153 // Add quaternion and float value
02154 RMAPI Quaternion QuaternionAddValue(Quaternion q, float add)
02155 {
02156     Quaternion result = {q.x + add, q.y + add, q.z + add, q.w + add};
02157
02158     return result;
02159 }
02160
02161 // Subtract two quaternions
02162 RMAPI Quaternion QuaternionSubtract(Quaternion q1, Quaternion q2)
02163 {
02164     Quaternion result = {q1.x - q2.x, q1.y - q2.y, q1.z - q2.z, q1.w - q2.w};
02165
02166     return result;
02167 }
02168
02169 // Subtract quaternion and float value
02170 RMAPI Quaternion QuaternionSubtractValue(Quaternion q, float sub)
02171 {
02172     Quaternion result = {q.x - sub, q.y - sub, q.z - sub, q.w - sub};
02173
02174     return result;
02175 }
02176
02177 // Get identity quaternion
02178 RMAPI Quaternion QuaternionIdentity(void)
02179 {
02180     Quaternion result = { 0.0f, 0.0f, 0.0f, 1.0f };
02181
02182     return result;
02183 }
02184
02185 // Computes the length of a quaternion
02186 RMAPI float QuaternionLength(Quaternion q)
02187 {
02188     float result = sqrtf(q.x*q.x + q.y*q.y + q.z*q.z + q.w*q.w);
02189
02190     return result;
02191 }
02192
02193 // Normalize provided quaternion
02194 RMAPI Quaternion QuaternionNormalize(Quaternion q)
02195 {
02196     Quaternion result = { 0 };
02197
02198     float length = sqrtf(q.x*q.x + q.y*q.y + q.z*q.z + q.w*q.w);
02199     if (length == 0.0f) length = 1.0f;
02200     float ilength = 1.0f/length;
02201
02202     result.x = q.x*ilength;
02203     result.y = q.y*ilength;
02204     result.z = q.z*ilength;
02205     result.w = q.w*ilength;
02206
02207     return result;
02208 }
02209
02210 // Invert provided quaternion
02211 RMAPI Quaternion QuaternionInvert(Quaternion q)
02212 {
02213     Quaternion result = q;
02214
02215     float lengthSq = q.x*q.x + q.y*q.y + q.z*q.z + q.w*q.w;
02216
02217     if (lengthSq != 0.0f)
02218     {
02219         float invLength = 1.0f/lengthSq;
02220
02221         result.x *= -invLength;
02222         result.y *= -invLength;

```

```

02223         result.z *= -invLength;
02224         result.w *= invLength;
02225     }
02226
02227     return result;
02228 }
02229
02230 // Calculate two quaternion multiplication
02231 RMAPI Quaternion QuaternionMultiply(Quaternion q1, Quaternion q2)
02232 {
02233     Quaternion result = { 0 };
02234
02235     float qax = q1.x, qay = q1.y, qaz = q1.z, qaw = q1.w;
02236     float qbx = q2.x, qby = q2.y, qbz = q2.z, qbw = q2.w;
02237
02238     result.x = qax*qbw + qaw*qbx + qay*qbz - qaz*qby;
02239     result.y = qay*qbw + qaw*qby + qaz*qbx - qax*qbz;
02240     result.z = qaz*qbw + qaw*qbz + qax*qby - qay*qbx;
02241     result.w = qaw*qbw - qax*qbx - qay*qby - qaz*qbz;
02242
02243     return result;
02244 }
02245
02246 // Scale quaternion by float value
02247 RMAPI Quaternion QuaternionScale(Quaternion q, float mul)
02248 {
02249     Quaternion result = { 0 };
02250
02251     result.x = q.x*mul;
02252     result.y = q.y*mul;
02253     result.z = q.z*mul;
02254     result.w = q.w*mul;
02255
02256     return result;
02257 }
02258
02259 // Divide two quaternions
02260 RMAPI Quaternion QuaternionDivide(Quaternion q1, Quaternion q2)
02261 {
02262     Quaternion result = { q1.x/q2.x, q1.y/q2.y, q1.z/q2.z, q1.w/q2.w };
02263
02264     return result;
02265 }
02266
02267 // Calculate linear interpolation between two quaternions
02268 RMAPI Quaternion QuaternionLerp(Quaternion q1, Quaternion q2, float amount)
02269 {
02270     Quaternion result = { 0 };
02271
02272     result.x = q1.x + amount*(q2.x - q1.x);
02273     result.y = q1.y + amount*(q2.y - q1.y);
02274     result.z = q1.z + amount*(q2.z - q1.z);
02275     result.w = q1.w + amount*(q2.w - q1.w);
02276
02277     return result;
02278 }
02279
02280 // Calculate slerp-optimized interpolation between two quaternions
02281 RMAPI Quaternion QuaternionNlerp(Quaternion q1, Quaternion q2, float amount)
02282 {
02283     Quaternion result = { 0 };
02284
02285     // QuaternionLerp(q1, q2, amount)
02286     result.x = q1.x + amount*(q2.x - q1.x);
02287     result.y = q1.y + amount*(q2.y - q1.y);
02288     result.z = q1.z + amount*(q2.z - q1.z);
02289     result.w = q1.w + amount*(q2.w - q1.w);
02290
02291     // QuaternionNormalize(q);
02292     Quaternion q = result;
02293     float length = sqrtf(q.x*q.x + q.y*q.y + q.z*q.z + q.w*q.w);
02294     if (length == 0.0f) length = 1.0f;
02295     float ilength = 1.0f/length;
02296
02297     result.x = q.x*ilength;
02298     result.y = q.y*ilength;
02299     result.z = q.z*ilength;
02300     result.w = q.w*ilength;
02301
02302     return result;
02303 }
02304
02305 // Calculates spherical linear interpolation between two quaternions
02306 RMAPI Quaternion QuaternionSlerp(Quaternion q1, Quaternion q2, float amount)
02307 {
02308     Quaternion result = { 0 };
02309

```

```

02310 #if !defined(EPSILON)
02311     #define EPSILON 0.000001f
02312 #endif
02313
02314     float cosHalfTheta = q1.x*q2.x + q1.y*q2.y + q1.z*q2.z + q1.w*q2.w;
02315
02316     if (cosHalfTheta < 0)
02317     {
02318         q2.x = -q2.x; q2.y = -q2.y; q2.z = -q2.z; q2.w = -q2.w;
02319         cosHalfTheta = -cosHalfTheta;
02320     }
02321
02322     if (fabsf(cosHalfTheta) >= 1.0f) result = q1;
02323     else if (cosHalfTheta > 0.95f) result = QuaternionNlerp(q1, q2, amount);
02324     else
02325     {
02326         float halfTheta = acosf(cosHalfTheta);
02327         float sinHalfTheta = sqrtf(1.0f - cosHalfTheta*cosHalfTheta);
02328
02329         if (fabsf(sinHalfTheta) < EPSILON)
02330         {
02331             result.x = (q1.x*0.5f + q2.x*0.5f);
02332             result.y = (q1.y*0.5f + q2.y*0.5f);
02333             result.z = (q1.z*0.5f + q2.z*0.5f);
02334             result.w = (q1.w*0.5f + q2.w*0.5f);
02335         }
02336         else
02337         {
02338             float ratioA = sinf((1 - amount)*halfTheta)/sinHalfTheta;
02339             float ratioB = sinf(amount*halfTheta)/sinHalfTheta;
02340
02341             result.x = (q1.x*ratioA + q2.x*ratioB);
02342             result.y = (q1.y*ratioA + q2.y*ratioB);
02343             result.z = (q1.z*ratioA + q2.z*ratioB);
02344             result.w = (q1.w*ratioA + q2.w*ratioB);
02345         }
02346     }
02347
02348     return result;
02349 }
02350
02351 // Calculate quaternion cubic spline interpolation using Cubic Hermite Spline algorithm
02352 // as described in the GLTF 2.0 specification:
02353 // https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html#interpolation-cubic
02354 RMAPI Quaternion QuaternionCubicHermiteSpline(Quaternion q1, Quaternion outTangent1, Quaternion q2,
02355 Quaternion inTangent2, float t)
02356 {
02357     float t2 = t*t;
02358     float t3 = t2*t;
02359     float h00 = 2*t3 - 3*t2 + 1;
02360     float h10 = t3 - 2*t2 + t;
02361     float h01 = -2*t3 + 3*t2;
02362     float h11 = t3 - t2;
02363
02364     Quaternion p0 = QuaternionScale(q1, h00);
02365     Quaternion m0 = QuaternionScale(outTangent1, h10);
02366     Quaternion p1 = QuaternionScale(q2, h01);
02367     Quaternion m1 = QuaternionScale(inTangent2, h11);
02368
02369     Quaternion result = { 0 };
02370
02371     result = QuaternionAdd(p0, m0);
02372     result = QuaternionAdd(result, p1);
02373     result = QuaternionAdd(result, m1);
02374     result = QuaternionNormalize(result);
02375
02376     return result;
02377 }
02378
02379 // Calculate quaternion based on the rotation from one vector to another
02380 RMAPI Quaternion QuaternionFromVector3ToVector3(Vector3 from, Vector3 to)
02381 {
02382     Quaternion result = { 0 };
02383
02384     float cos2Theta = (from.x*to.x + from.y*to.y + from.z*to.z); // Vector3DotProduct(from, to)
02385     Vector3 cross = { from.y*to.z - from.z*to.y, from.z*to.x - from.x*to.z, from.x*to.y - from.y*to.x }; // Vector3CrossProduct(from, to)
02386
02387     result.x = cross.x;
02388     result.y = cross.y;
02389     result.z = cross.z;
02390     result.w = sqrtf(cross.x*cross.x + cross.y*cross.y + cross.z*cross.z + cos2Theta*cos2Theta) +
02391 cos2Theta;
02392
02393     // QuaternionNormalize(q);
02394     // NOTE: Normalize to essentially nlerp the original and identity to 0.5
02395     Quaternion q = result;

```

```

02393     float length = sqrtf(q.x*q.x + q.y*q.y + q.z*q.z + q.w*q.w);
02394     if (length == 0.0f) length = 1.0f;
02395     float ilength = 1.0f/length;
02396
02397     result.x = q.x*ilength;
02398     result.y = q.y*ilength;
02399     result.z = q.z*ilength;
02400     result.w = q.w*ilength;
02401
02402     return result;
02403 }
02404
02405 // Get a quaternion for a given rotation matrix
02406 RMAPI Quaternion QuaternionFromMatrix(Matrix mat)
02407 {
02408     Quaternion result = { 0 };
02409
02410     float fourWSquaredMinus1 = mat.m0 + mat.m5 + mat.m10;
02411     float fourXSquaredMinus1 = mat.m0 - mat.m5 - mat.m10;
02412     float fourYSquaredMinus1 = mat.m5 - mat.m0 - mat.m10;
02413     float fourZSquaredMinus1 = mat.m10 - mat.m0 - mat.m5;
02414
02415     int biggestIndex = 0;
02416     float fourBiggestSquaredMinus1 = fourWSquaredMinus1;
02417     if (fourXSquaredMinus1 > fourBiggestSquaredMinus1)
02418     {
02419         fourBiggestSquaredMinus1 = fourXSquaredMinus1;
02420         biggestIndex = 1;
02421     }
02422
02423     if (fourYSquaredMinus1 > fourBiggestSquaredMinus1)
02424     {
02425         fourBiggestSquaredMinus1 = fourYSquaredMinus1;
02426         biggestIndex = 2;
02427     }
02428
02429     if (fourZSquaredMinus1 > fourBiggestSquaredMinus1)
02430     {
02431         fourBiggestSquaredMinus1 = fourZSquaredMinus1;
02432         biggestIndex = 3;
02433     }
02434
02435     float biggestVal = sqrtf(fourBiggestSquaredMinus1 + 1.0f)*0.5f;
02436     float mult = 0.25f/biggestVal;
02437
02438     switch (biggestIndex)
02439     {
02440     case 0:
02441         result.w = biggestVal;
02442         result.x = (mat.m6 - mat.m9)*mult;
02443         result.y = (mat.m8 - mat.m2)*mult;
02444         result.z = (mat.m1 - mat.m4)*mult;
02445         break;
02446     case 1:
02447         result.x = biggestVal;
02448         result.w = (mat.m6 - mat.m9)*mult;
02449         result.y = (mat.m1 + mat.m4)*mult;
02450         result.z = (mat.m8 + mat.m2)*mult;
02451         break;
02452     case 2:
02453         result.y = biggestVal;
02454         result.w = (mat.m8 - mat.m2)*mult;
02455         result.x = (mat.m1 + mat.m4)*mult;
02456         result.z = (mat.m6 + mat.m9)*mult;
02457         break;
02458     case 3:
02459         result.z = biggestVal;
02460         result.w = (mat.m1 - mat.m4)*mult;
02461         result.x = (mat.m8 + mat.m2)*mult;
02462         result.y = (mat.m6 + mat.m9)*mult;
02463         break;
02464     }
02465
02466     return result;
02467 }
02468
02469 // Get a matrix for a given quaternion
02470 RMAPI Matrix QuaternionToMatrix(Quaternion q)
02471 {
02472     Matrix result = { 1.0f, 0.0f, 0.0f, 0.0f,
02473                     0.0f, 1.0f, 0.0f, 0.0f,
02474                     0.0f, 0.0f, 1.0f, 0.0f,
02475                     0.0f, 0.0f, 0.0f, 1.0f }; // MatrixIdentity()
02476
02477     float a2 = q.x*q.x;
02478     float b2 = q.y*q.y;
02479     float c2 = q.z*q.z;

```

```

02480     float ac = q.x*q.z;
02481     float ab = q.x*q.y;
02482     float bc = q.y*q.z;
02483     float ad = q.w*q.x;
02484     float bd = q.w*q.y;
02485     float cd = q.w*q.z;
02486
02487     result.m0 = 1 - 2*(b2 + c2);
02488     result.m1 = 2*(ab + cd);
02489     result.m2 = 2*(ac - bd);
02490
02491     result.m4 = 2*(ab - cd);
02492     result.m5 = 1 - 2*(a2 + c2);
02493     result.m6 = 2*(bc + ad);
02494
02495     result.m8 = 2*(ac + bd);
02496     result.m9 = 2*(bc - ad);
02497     result.m10 = 1 - 2*(a2 + b2);
02498
02499     return result;
02500 }
02501
02502 // Get rotation quaternion for an angle and axis
02503 // NOTE: Angle must be provided in radians
02504 RMAPI Quaternion QuaternionFromAxisAngle(Vector3 axis, float angle)
02505 {
02506     Quaternion result = { 0.0f, 0.0f, 0.0f, 1.0f };
02507
02508     float length = sqrtf(axis.x*axis.x + axis.y*axis.y + axis.z*axis.z);
02509
02510     if (length != 0.0f)
02511     {
02512         angle *= 0.5f;
02513
02514         // Vector3Normalize(axis)
02515         float ilength = 1.0f/length;
02516         axis.x *= ilength;
02517         axis.y *= ilength;
02518         axis.z *= ilength;
02519
02520         float sinres = sinf(angle);
02521         float cosres = cosf(angle);
02522
02523         result.x = axis.x*sinres;
02524         result.y = axis.y*sinres;
02525         result.z = axis.z*sinres;
02526         result.w = cosres;
02527
02528         // QuaternionNormalize(q);
02529         Quaternion q = result;
02530         length = sqrtf(q.x*q.x + q.y*q.y + q.z*q.z + q.w*q.w);
02531         if (length == 0.0f) length = 1.0f;
02532         ilength = 1.0f/length;
02533         result.x = q.x*ilength;
02534         result.y = q.y*ilength;
02535         result.z = q.z*ilength;
02536         result.w = q.w*ilength;
02537     }
02538
02539     return result;
02540 }
02541
02542 // Get the rotation angle and axis for a given quaternion
02543 RMAPI void QuaternionToAxisAngle(Quaternion q, Vector3 *outAxis, float *outAngle)
02544 {
02545     if (fabsf(q.w) > 1.0f)
02546     {
02547         // QuaternionNormalize(q);
02548         float length = sqrtf(q.x*q.x + q.y*q.y + q.z*q.z + q.w*q.w);
02549         if (length == 0.0f) length = 1.0f;
02550         float ilength = 1.0f/length;
02551
02552         q.x = q.x*ilength;
02553         q.y = q.y*ilength;
02554         q.z = q.z*ilength;
02555         q.w = q.w*ilength;
02556     }
02557
02558     Vector3 resAxis = { 0.0f, 0.0f, 0.0f };
02559     float resAngle = 2.0f*acosf(q.w);
02560     float den = sqrtf(1.0f - q.w*q.w);
02561
02562     if (den > EPSILON)
02563     {
02564         resAxis.x = q.x/den;
02565         resAxis.y = q.y/den;
02566         resAxis.z = q.z/den;

```

```

02567     }
02568     else
02569     {
02570         // This occurs when the angle is zero
02571         // Not a problem, set an arbitrary normalized axis
02572         resAxis.x = 1.0f;
02573     }
02574
02575     *outAxis = resAxis;
02576     *outAngle = resAngle;
02577 }
02578
02579 // Get the quaternion equivalent to Euler angles
02580 // NOTE: Rotation order is ZYX
02581 RMAPI Quaternion QuaternionFromEuler(float pitch, float yaw, float roll)
02582 {
02583     Quaternion result = { 0 };
02584
02585     float x0 = cosf(pitch*0.5f);
02586     float x1 = sinf(pitch*0.5f);
02587     float y0 = cosf(yaw*0.5f);
02588     float y1 = sinf(yaw*0.5f);
02589     float z0 = cosf(roll*0.5f);
02590     float z1 = sinf(roll*0.5f);
02591
02592     result.x = x1*y0*z0 - x0*y1*z1;
02593     result.y = x0*y1*z0 + x1*y0*z1;
02594     result.z = x0*y0*z1 - x1*y1*z0;
02595     result.w = x0*y0*z0 + x1*y1*z1;
02596
02597     return result;
02598 }
02599
02600 // Get the Euler angles equivalent to quaternion (roll, pitch, yaw)
02601 // NOTE: Angles are returned in a Vector3 struct in radians
02602 RMAPI Vector3 QuaternionToEuler(Quaternion q)
02603 {
02604     Vector3 result = { 0 };
02605
02606     // Roll (x-axis rotation)
02607     float x0 = 2.0f*(q.w*q.x + q.y*q.z);
02608     float x1 = 1.0f - 2.0f*(q.x*q.x + q.y*q.y);
02609     result.x = atan2f(x0, x1);
02610
02611     // Pitch (y-axis rotation)
02612     float y0 = 2.0f*(q.w*q.y - q.z*q.x);
02613     y0 = y0 > 1.0f ? 1.0f : y0;
02614     y0 = y0 < -1.0f ? -1.0f : y0;
02615     result.y = asinf(y0);
02616
02617     // Yaw (z-axis rotation)
02618     float z0 = 2.0f*(q.w*q.z + q.x*q.y);
02619     float z1 = 1.0f - 2.0f*(q.y*q.y + q.z*q.z);
02620     result.z = atan2f(z0, z1);
02621
02622     return result;
02623 }
02624
02625 // Transform a quaternion given a transformation matrix
02626 RMAPI Quaternion QuaternionTransform(Quaternion q, Matrix mat)
02627 {
02628     Quaternion result = { 0 };
02629
02630     result.x = mat.m0*q.x + mat.m4*q.y + mat.m8*q.z + mat.m12*q.w;
02631     result.y = mat.m1*q.x + mat.m5*q.y + mat.m9*q.z + mat.m13*q.w;
02632     result.z = mat.m2*q.x + mat.m6*q.y + mat.m10*q.z + mat.m14*q.w;
02633     result.w = mat.m3*q.x + mat.m7*q.y + mat.m11*q.z + mat.m15*q.w;
02634
02635     return result;
02636 }
02637
02638 // Check whether two given quaternions are almost equal
02639 RMAPI int QuaternionEquals(Quaternion p, Quaternion q)
02640 {
02641     #if !defined(EPSILON)
02642         #define EPSILON 0.000001f
02643     #endif
02644
02645     int result = (((fabsf(p.x - q.x)) <= (EPSILON*fmaxf(1.0f, fmaxf(fabsf(p.x), fabsf(q.x))))) &&
02646                 ((fabsf(p.y - q.y)) <= (EPSILON*fmaxf(1.0f, fmaxf(fabsf(p.y), fabsf(q.y))))) &&
02647                 ((fabsf(p.z - q.z)) <= (EPSILON*fmaxf(1.0f, fmaxf(fabsf(p.z), fabsf(q.z))))) &&
02648                 ((fabsf(p.w - q.w)) <= (EPSILON*fmaxf(1.0f, fmaxf(fabsf(p.w), fabsf(q.w))))) ||
02649                 (((fabsf(p.x + q.x)) <= (EPSILON*fmaxf(1.0f, fmaxf(fabsf(p.x), fabsf(q.x))))) &&
02650                 ((fabsf(p.y + q.y)) <= (EPSILON*fmaxf(1.0f, fmaxf(fabsf(p.y), fabsf(q.y))))) &&
02651                 ((fabsf(p.z + q.z)) <= (EPSILON*fmaxf(1.0f, fmaxf(fabsf(p.z), fabsf(q.z))))) &&
02652                 ((fabsf(p.w + q.w)) <= (EPSILON*fmaxf(1.0f, fmaxf(fabsf(p.w), fabsf(q.w))))) &&
02653

```

```

02654     return result;
02655 }
02656
02657 // Compose a transformation matrix from rotational, translational and scaling components
02658 // TODO: This function is not following raymath conventions defined in header: NOT self-contained
02659 RMAPI Matrix MatrixCompose(Vector3 translation, Quaternion rotation, Vector3 scale)
02660 {
02661     // Initialize vectors
02662     Vector3 right = { 1.0f, 0.0f, 0.0f };
02663     Vector3 up = { 0.0f, 1.0f, 0.0f };
02664     Vector3 forward = { 0.0f, 0.0f, 1.0f };
02665
02666     // Scale vectors
02667     right = Vector3Scale(right, scale.x);
02668     up = Vector3Scale(up, scale.y);
02669     forward = Vector3Scale(forward, scale.z);
02670
02671     // Rotate vectors
02672     right = Vector3RotateByQuaternion(right, rotation);
02673     up = Vector3RotateByQuaternion(up, rotation);
02674     forward = Vector3RotateByQuaternion(forward, rotation);
02675
02676     // Set result matrix output
02677     Matrix result = {
02678         right.x, up.x, forward.x, translation.x,
02679         right.y, up.y, forward.y, translation.y,
02680         right.z, up.z, forward.z, translation.z,
02681         0.0f, 0.0f, 0.0f, 1.0f
02682     };
02683
02684     return result;
02685 }
02686
02687 // Decompose a transformation matrix into its rotational, translational and scaling components and
// remove shear
02688 // TODO: This function is not following raymath conventions defined in header: NOT self-contained
02689 RMAPI void MatrixDecompose(Matrix mat, Vector3 *translation, Quaternion *rotation, Vector3 *scale)
02690 {
02691     float eps = (float)1e-9;
02692
02693     // Extract Translation
02694     translation->x = mat.m12;
02695     translation->y = mat.m13;
02696     translation->z = mat.m14;
02697
02698     // Matrix Columns - Rotation will be extracted into here
02699     Vector3 matColumns[3] = {{ mat.m0, mat.m4, mat.m8 },
02700                             { mat.m1, mat.m5, mat.m9 },
02701                             { mat.m2, mat.m6, mat.m10 }};
02702
02703     // Shear Parameters XY, XZ, and YZ (extract and ignored)
02704     float shear[3] = { 0 };
02705
02706     // Normalized Scale Parameters
02707     Vector3 scl = { 0 };
02708
02709     // Max-Normalizing helps numerical stability
02710     float stabilizer = eps;
02711     for (int i = 0; i < 3; i++)
02712     {
02713         stabilizer = fmaxf(stabilizer, fabsf(matColumns[i].x));
02714         stabilizer = fmaxf(stabilizer, fabsf(matColumns[i].y));
02715         stabilizer = fmaxf(stabilizer, fabsf(matColumns[i].z));
02716     }
02717     matColumns[0] = Vector3Scale(matColumns[0], 1.0f / stabilizer);
02718     matColumns[1] = Vector3Scale(matColumns[1], 1.0f / stabilizer);
02719     matColumns[2] = Vector3Scale(matColumns[2], 1.0f / stabilizer);
02720
02721     // X Scale
02722     scl.x = Vector3Length(matColumns[0]);
02723     if (scl.x > eps) matColumns[0] = Vector3Scale(matColumns[0], 1.0f / scl.x);
02724
02725     // Compute XY shear and make col2 orthogonal
02726     shear[0] = Vector3DotProduct(matColumns[0], matColumns[1]);
02727     matColumns[1] = Vector3Subtract(matColumns[1], Vector3Scale(matColumns[0], shear[0]));
02728
02729     // Y Scale
02730     scl.y = Vector3Length(matColumns[1]);
02731     if (scl.y > eps)
02732     {
02733         matColumns[1] = Vector3Scale(matColumns[1], 1.0f / scl.y);
02734         shear[0] /= scl.y; // Correct XY shear
02735     }
02736
02737     // Compute XZ and YZ shears and make col3 orthogonal
02738     shear[1] = Vector3DotProduct(matColumns[0], matColumns[2]);
02739     matColumns[2] = Vector3Subtract(matColumns[2], Vector3Scale(matColumns[0], shear[1]));

```



```

02740     shear[2] = Vector3DotProduct(matColumns[1], matColumns[2]);
02741     matColumns[2] = Vector3Subtract(matColumns[2], Vector3Scale(matColumns[1], shear[2]));
02742
02743     // Z Scale
02744     scl.z = Vector3Length(matColumns[2]);
02745     if (scl.z > eps)
02746     {
02747         matColumns[2] = Vector3Scale(matColumns[2], 1.0f / scl.z);
02748         shear[1] /= scl.z; // Correct XZ shear
02749         shear[2] /= scl.z; // Correct YZ shear
02750     }
02751
02752     // matColumns are now orthonormal in O(3). Now ensure its in SO(3) by enforcing det = 1
02753     if (Vector3DotProduct(matColumns[0], Vector3CrossProduct(matColumns[1], matColumns[2])) < 0)
02754     {
02755         scl = Vector3Negate(scl);
02756         matColumns[0] = Vector3Negate(matColumns[0]);
02757         matColumns[1] = Vector3Negate(matColumns[1]);
02758         matColumns[2] = Vector3Negate(matColumns[2]);
02759     }
02760
02761     // Set Scale
02762     *scale = Vector3Scale(scl, stabilizer);
02763
02764     // Extract Rotation
02765     Matrix rotationMatrix = { matColumns[0].x, matColumns[0].y, matColumns[0].z, 0,
02766                               matColumns[1].x, matColumns[1].y, matColumns[1].z, 0,
02767                               matColumns[2].x, matColumns[2].y, matColumns[2].z, 0,
02768                               0, 0, 0, 1 };
02769     *rotation = QuaternionFromMatrix(rotationMatrix);
02770 }
02771
02772 #if defined(__cplusplus) && !defined(RAYMATH_DISABLE_CPP_OPERATORS)
02773
02774 // Optional C++ math operators
02775 //-----
02776
02777 // Vector2 operators
02778 static constexpr Vector2 Vector2Zeros = { 0, 0 };
02779 static constexpr Vector2 Vector2Ones = { 1, 1 };
02780 static constexpr Vector2 Vector2UnitX = { 1, 0 };
02781 static constexpr Vector2 Vector2UnitY = { 0, 1 };
02782
02783 inline Vector2 operator + (const Vector2& lhs, const Vector2& rhs)
02784 {
02785     return Vector2Add(lhs, rhs);
02786 }
02787
02788 inline const Vector2& operator += (Vector2& lhs, const Vector2& rhs)
02789 {
02790     lhs = Vector2Add(lhs, rhs);
02791     return lhs;
02792 }
02793
02794 inline Vector2 operator - (const Vector2& lhs, const Vector2& rhs)
02795 {
02796     return Vector2Subtract(lhs, rhs);
02797 }
02798
02799 inline const Vector2& operator -= (Vector2& lhs, const Vector2& rhs)
02800 {
02801     lhs = Vector2Subtract(lhs, rhs);
02802     return lhs;
02803 }
02804
02805 inline Vector2 operator * (const Vector2& lhs, const float& rhs)
02806 {
02807     return Vector2Scale(lhs, rhs);
02808 }
02809
02810 inline const Vector2& operator *= (Vector2& lhs, const float& rhs)
02811 {
02812     lhs = Vector2Scale(lhs, rhs);
02813     return lhs;
02814 }
02815
02816 inline Vector2 operator * (const Vector2& lhs, const Vector2& rhs)
02817 {
02818     return Vector2Multiply(lhs, rhs);
02819 }
02820
02821 inline const Vector2& operator *= (Vector2& lhs, const Vector2& rhs)
02822 {
02823     lhs = Vector2Multiply(lhs, rhs);
02824     return lhs;
02825 }
02826

```

```

02827 inline Vector2 operator * (const Vector2& lhs, const Matrix& rhs)
02828 {
02829     return Vector2Transform(lhs, rhs);
02830 }
02831
02832 inline const Vector2& operator *= (Vector2& lhs, const Matrix& rhs)
02833 {
02834     lhs = Vector2Transform(lhs, rhs);
02835     return lhs;
02836 }
02837
02838 inline Vector2 operator / (const Vector2& lhs, const float& rhs)
02839 {
02840     return Vector2Scale(lhs, 1.0f/rhs);
02841 }
02842
02843 inline const Vector2& operator /= (Vector2& lhs, const float& rhs)
02844 {
02845     lhs = Vector2Scale(lhs, 1.0f/rhs);
02846     return lhs;
02847 }
02848
02849 inline Vector2 operator / (const Vector2& lhs, const Vector2& rhs)
02850 {
02851     return Vector2Divide(lhs, rhs);
02852 }
02853
02854 inline const Vector2& operator /= (Vector2& lhs, const Vector2& rhs)
02855 {
02856     lhs = Vector2Divide(lhs, rhs);
02857     return lhs;
02858 }
02859
02860 inline bool operator == (const Vector2& lhs, const Vector2& rhs)
02861 {
02862     return FloatEquals(lhs.x, rhs.x) && FloatEquals(lhs.y, rhs.y);
02863 }
02864
02865 inline bool operator != (const Vector2& lhs, const Vector2& rhs)
02866 {
02867     return !FloatEquals(lhs.x, rhs.x) || !FloatEquals(lhs.y, rhs.y);
02868 }
02869
02870 // Vector3 operators
02871 static constexpr Vector3 Vector3Zeros = { 0, 0, 0 };
02872 static constexpr Vector3 Vector3Ones = { 1, 1, 1 };
02873 static constexpr Vector3 Vector3UnitX = { 1, 0, 0 };
02874 static constexpr Vector3 Vector3UnitY = { 0, 1, 0 };
02875 static constexpr Vector3 Vector3UnitZ = { 0, 0, 1 };
02876
02877 inline Vector3 operator + (const Vector3& lhs, const Vector3& rhs)
02878 {
02879     return Vector3Add(lhs, rhs);
02880 }
02881
02882 inline const Vector3& operator += (Vector3& lhs, const Vector3& rhs)
02883 {
02884     lhs = Vector3Add(lhs, rhs);
02885     return lhs;
02886 }
02887
02888 inline Vector3 operator - (const Vector3& lhs, const Vector3& rhs)
02889 {
02890     return Vector3Subtract(lhs, rhs);
02891 }
02892
02893 inline const Vector3& operator -= (Vector3& lhs, const Vector3& rhs)
02894 {
02895     lhs = Vector3Subtract(lhs, rhs);
02896     return lhs;
02897 }
02898
02899 inline Vector3 operator * (const Vector3& lhs, const float& rhs)
02900 {
02901     return Vector3Scale(lhs, rhs);
02902 }
02903
02904 inline const Vector3& operator *= (Vector3& lhs, const float& rhs)
02905 {
02906     lhs = Vector3Scale(lhs, rhs);
02907     return lhs;
02908 }
02909
02910 inline Vector3 operator * (const Vector3& lhs, const Vector3& rhs)
02911 {
02912     return Vector3Multiply(lhs, rhs);
02913 }

```

```

02914
02915 inline const Vector3& operator *= (Vector3& lhs, const Vector3& rhs)
02916 {
02917     lhs = Vector3Multiply(lhs, rhs);
02918     return lhs;
02919 }
02920
02921 inline Vector3 operator * (const Vector3& lhs, const Matrix& rhs)
02922 {
02923     return Vector3Transform(lhs, rhs);
02924 }
02925
02926 inline const Vector3& operator *= (Vector3& lhs, const Matrix& rhs)
02927 {
02928     lhs = Vector3Transform(lhs, rhs);
02929     return lhs;
02930 }
02931
02932 inline Vector3 operator / (const Vector3& lhs, const float& rhs)
02933 {
02934     return Vector3Scale(lhs, 1.0f/rhs);
02935 }
02936
02937 inline const Vector3& operator /= (Vector3& lhs, const float& rhs)
02938 {
02939     lhs = Vector3Scale(lhs, 1.0f/rhs);
02940     return lhs;
02941 }
02942
02943 inline Vector3 operator / (const Vector3& lhs, const Vector3& rhs)
02944 {
02945     return Vector3Divide(lhs, rhs);
02946 }
02947
02948 inline const Vector3& operator /= (Vector3& lhs, const Vector3& rhs)
02949 {
02950     lhs = Vector3Divide(lhs, rhs);
02951     return lhs;
02952 }
02953
02954 inline bool operator == (const Vector3& lhs, const Vector3& rhs)
02955 {
02956     return FloatEquals(lhs.x, rhs.x) && FloatEquals(lhs.y, rhs.y) && FloatEquals(lhs.z, rhs.z);
02957 }
02958
02959 inline bool operator != (const Vector3& lhs, const Vector3& rhs)
02960 {
02961     return !FloatEquals(lhs.x, rhs.x) || !FloatEquals(lhs.y, rhs.y) || !FloatEquals(lhs.z, rhs.z);
02962 }
02963
02964 // Vector4 operators
02965 static constexpr Vector4 Vector4Zeros = { 0, 0, 0, 0 };
02966 static constexpr Vector4 Vector4Ones = { 1, 1, 1, 1 };
02967 static constexpr Vector4 Vector4UnitX = { 1, 0, 0, 0 };
02968 static constexpr Vector4 Vector4UnitY = { 0, 1, 0, 0 };
02969 static constexpr Vector4 Vector4UnitZ = { 0, 0, 1, 0 };
02970 static constexpr Vector4 Vector4UnitW = { 0, 0, 0, 1 };
02971
02972 inline Vector4 operator + (const Vector4& lhs, const Vector4& rhs)
02973 {
02974     return Vector4Add(lhs, rhs);
02975 }
02976
02977 inline const Vector4& operator += (Vector4& lhs, const Vector4& rhs)
02978 {
02979     lhs = Vector4Add(lhs, rhs);
02980     return lhs;
02981 }
02982
02983 inline Vector4 operator - (const Vector4& lhs, const Vector4& rhs)
02984 {
02985     return Vector4Subtract(lhs, rhs);
02986 }
02987
02988 inline const Vector4& operator -= (Vector4& lhs, const Vector4& rhs)
02989 {
02990     lhs = Vector4Subtract(lhs, rhs);
02991     return lhs;
02992 }
02993
02994 inline Vector4 operator * (const Vector4& lhs, const float& rhs)
02995 {
02996     return Vector4Scale(lhs, rhs);
02997 }
02998
02999 inline const Vector4& operator *= (Vector4& lhs, const float& rhs)
03000 {

```

```

03001     lhs = Vector4Scale(lhs, rhs);
03002     return lhs;
03003 }
03004
03005 inline Vector4 operator * (const Vector4& lhs, const Vector4& rhs)
03006 {
03007     return Vector4Multiply(lhs, rhs);
03008 }
03009
03010 inline const Vector4& operator *= (Vector4& lhs, const Vector4& rhs)
03011 {
03012     lhs = Vector4Multiply(lhs, rhs);
03013     return lhs;
03014 }
03015
03016 inline Vector4 operator / (const Vector4& lhs, const float& rhs)
03017 {
03018     return Vector4Scale(lhs, 1.0f/rhs);
03019 }
03020
03021 inline const Vector4& operator /= (Vector4& lhs, const float& rhs)
03022 {
03023     lhs = Vector4Scale(lhs, 1.0f/rhs);
03024     return lhs;
03025 }
03026
03027 inline Vector4 operator / (const Vector4& lhs, const Vector4& rhs)
03028 {
03029     return Vector4Divide(lhs, rhs);
03030 }
03031
03032 inline const Vector4& operator /= (Vector4& lhs, const Vector4& rhs)
03033 {
03034     lhs = Vector4Divide(lhs, rhs);
03035     return lhs;
03036 }
03037
03038 inline bool operator == (const Vector4& lhs, const Vector4& rhs)
03039 {
03040     return FloatEquals(lhs.x, rhs.x) && FloatEquals(lhs.y, rhs.y) && FloatEquals(lhs.z, rhs.z) &&
03041         FloatEquals(lhs.w, rhs.w);
03042 }
03043
03044 inline bool operator != (const Vector4& lhs, const Vector4& rhs)
03045 {
03046     return !FloatEquals(lhs.x, rhs.x) || !FloatEquals(lhs.y, rhs.y) || !FloatEquals(lhs.z, rhs.z) ||
03047         !FloatEquals(lhs.w, rhs.w);
03048 }
03049 // Quaternion operators
03050 static constexpr Quaternion QuaternionZeros = { 0, 0, 0, 0 };
03051 static constexpr Quaternion QuaternionOnes = { 1, 1, 1, 1 };
03052 static constexpr Quaternion QuaternionUnitX = { 0, 0, 0, 1 };
03053
03054 inline Quaternion operator + (const Quaternion& lhs, const float& rhs)
03055 {
03056     return QuaternionAddValue(lhs, rhs);
03057 }
03058
03059 inline const Quaternion& operator += (Quaternion& lhs, const float& rhs)
03060 {
03061     lhs = QuaternionAddValue(lhs, rhs);
03062     return lhs;
03063 }
03064
03065 inline Quaternion operator - (const Quaternion& lhs, const float& rhs)
03066 {
03067     return QuaternionSubtractValue(lhs, rhs);
03068 }
03069
03070 inline const Quaternion& operator -= (Quaternion& lhs, const float& rhs)
03071 {
03072     lhs = QuaternionSubtractValue(lhs, rhs);
03073     return lhs;
03074 }
03075
03076 inline Quaternion operator * (const Quaternion& lhs, const Matrix& rhs)
03077 {
03078     return QuaternionTransform(lhs, rhs);
03079 }
03080
03081 inline const Quaternion& operator *= (Quaternion& lhs, const Matrix& rhs)
03082 {
03083     lhs = QuaternionTransform(lhs, rhs);
03084     return lhs;
03085 }

```

```

03086 // Matrix operators
03087 static constexpr Matrix MatrixUnit = { 1, 0, 0, 0,
03088                                         0, 1, 0, 0,
03089                                         0, 0, 1, 0,
03090                                         0, 0, 0, 1 };
03091
03092 inline Matrix operator + (const Matrix& lhs, const Matrix& rhs)
03093 {
03094     return MatrixAdd(lhs, rhs);
03095 }
03096
03097 inline const Matrix& operator += (Matrix& lhs, const Matrix& rhs)
03098 {
03099     lhs = MatrixAdd(lhs, rhs);
03100     return lhs;
03101 }
03102
03103 inline Matrix operator - (const Matrix& lhs, const Matrix& rhs)
03104 {
03105     return MatrixSubtract(lhs, rhs);
03106 }
03107
03108 inline const Matrix& operator -= (Matrix& lhs, const Matrix& rhs)
03109 {
03110     lhs = MatrixSubtract(lhs, rhs);
03111     return lhs;
03112 }
03113
03114 inline Matrix operator * (const Matrix& lhs, const Matrix& rhs)
03115 {
03116     return MatrixMultiply(lhs, rhs);
03117 }
03118
03119 inline const Matrix& operator *= (Matrix& lhs, const Matrix& rhs)
03120 {
03121     lhs = MatrixMultiply(lhs, rhs);
03122     return lhs;
03123 }
03124
03125 inline Matrix operator * (const Matrix& lhs, const float value)
03126 {
03127     return MatrixMultiplyValue(lhs, value);
03128 }
03129
03130 inline const Matrix& operator *= (Matrix& lhs, const float value)
03131 {
03132     lhs = MatrixMultiplyValue(lhs, value);
03133     return lhs;
03134 }
03135
03136 //-----
03137 #endif // C++ operators
03138
03139 #endif // RAYMATH_H

```

6.27 lib/raylib/include/rlgl.h File Reference

Data Structures

- struct [Matrix](#)
- struct [rlVertexBuffer](#)
- struct [rlDrawCall](#)
- struct [rlRenderBatch](#)

Macros

- #define [RLGL_VERSION](#) "6.0"
- #define [RLAPI](#)
- #define [TRACELOG](#)(level, ...)
- #define [RL_MALLOC](#)(sz)
- #define [RL_CALLOC](#)(n, sz)
- #define [RL_REALLOC](#)(n, sz)

- `#define RL_FREE(p)`
- `#define GRAPHICS_API_OPENGL_33`
- `#define RL_DEFAULT_BATCH_BUFFER_ELEMENTS 8192`
- `#define RL_DEFAULT_BATCH_BUFFERS 1`
- `#define RL_DEFAULT_BATCH_DRAWCALLS 256`
- `#define RL_DEFAULT_BATCH_MAX_TEXTURE_UNITS 4`
- `#define RL_MAX_MATRIX_STACK_SIZE 32`
- `#define RL_MAX_SHADER_LOCATIONS 32`
- `#define RL_CULL_DISTANCE_NEAR 0.05`
- `#define RL_CULL_DISTANCE_FAR 4000.0`
- `#define RL_TEXTURE_WRAP_S 0x2802`
- `#define RL_TEXTURE_WRAP_T 0x2803`
- `#define RL_TEXTURE_MAG_FILTER 0x2800`
- `#define RL_TEXTURE_MIN_FILTER 0x2801`
- `#define RL_TEXTURE_FILTER_NEAREST 0x2600`
- `#define RL_TEXTURE_FILTER_LINEAR 0x2601`
- `#define RL_TEXTURE_FILTER_MIP_NEAREST 0x2700`
- `#define RL_TEXTURE_FILTER_NEAREST_MIP_LINEAR 0x2702`
- `#define RL_TEXTURE_FILTER_LINEAR_MIP_NEAREST 0x2701`
- `#define RL_TEXTURE_FILTER_MIP_LINEAR 0x2703`
- `#define RL_TEXTURE_FILTER_ANISOTROPIC 0x3000`
- `#define RL_TEXTURE_MIPMAP_BIAS_RATIO 0x4000`
- `#define RL_TEXTURE_WRAP_REPEAT 0x2901`
- `#define RL_TEXTURE_WRAP_CLAMP 0x812F`
- `#define RL_TEXTURE_WRAP_MIRROR_REPEAT 0x8370`
- `#define RL_TEXTURE_WRAP_MIRROR_CLAMP 0x8742`
- `#define RL_MODELVIEW 0x1700`
- `#define RL_PROJECTION 0x1701`
- `#define RL_TEXTURE 0x1702`
- `#define RL_LINES 0x0001`
- `#define RL_TRIANGLES 0x0004`
- `#define RL_QUADS 0x0007`
- `#define RL_UNSIGNED_BYTE 0x1401`
- `#define RL_FLOAT 0x1406`
- `#define RL_STREAM_DRAW 0x88E0`
- `#define RL_STREAM_READ 0x88E1`
- `#define RL_STREAM_COPY 0x88E2`
- `#define RL_STATIC_DRAW 0x88E4`
- `#define RL_STATIC_READ 0x88E5`
- `#define RL_STATIC_COPY 0x88E6`
- `#define RL_DYNAMIC_DRAW 0x88E8`
- `#define RL_DYNAMIC_READ 0x88E9`
- `#define RL_DYNAMIC_COPY 0x88EA`
- `#define RL_FRAGMENT_SHADER 0x8B30`
- `#define RL_VERTEX_SHADER 0x8B31`
- `#define RL_COMPUTE_SHADER 0x91B9`
- `#define RL_ZERO 0`
- `#define RL_ONE 1`
- `#define RL_SRC_COLOR 0x0300`
- `#define RL_ONE_MINUS_SRC_COLOR 0x0301`
- `#define RL_SRC_ALPHA 0x0302`
- `#define RL_ONE_MINUS_SRC_ALPHA 0x0303`
- `#define RL_DST_ALPHA 0x0304`
- `#define RL_ONE_MINUS_DST_ALPHA 0x0305`
- `#define RL_DST_COLOR 0x0306`

- #define [RL_ONE_MINUS_DST_COLOR](#) 0x0307
- #define [RL_SRC_ALPHA_SATURATE](#) 0x0308
- #define [RL_CONSTANT_COLOR](#) 0x8001
- #define [RL_ONE_MINUS_CONSTANT_COLOR](#) 0x8002
- #define [RL_CONSTANT_ALPHA](#) 0x8003
- #define [RL_ONE_MINUS_CONSTANT_ALPHA](#) 0x8004
- #define [RL_FUNC_ADD](#) 0x8006
- #define [RL_MIN](#) 0x8007
- #define [RL_MAX](#) 0x8008
- #define [RL_FUNC_SUBTRACT](#) 0x800A
- #define [RL_FUNC_REVERSE_SUBTRACT](#) 0x800B
- #define [RL_BLEND_EQUATION](#) 0x8009
- #define [RL_BLEND_EQUATION_RGB](#) 0x8009
- #define [RL_BLEND_EQUATION_ALPHA](#) 0x883D
- #define [RL_BLEND_DST_RGB](#) 0x80C8
- #define [RL_BLEND_SRC_RGB](#) 0x80C9
- #define [RL_BLEND_DST_ALPHA](#) 0x80CA
- #define [RL_BLEND_SRC_ALPHA](#) 0x80CB
- #define [RL_BLEND_COLOR](#) 0x8005
- #define [RL_READ_FRAMEBUFFER](#) 0x8CA8
- #define [RL_DRAW_FRAMEBUFFER](#) 0x8CA9
- #define [RL_DEFAULT_SHADER_ATTRIB_LOCATION_POSITION](#) 0
- #define [RL_DEFAULT_SHADER_ATTRIB_LOCATION_TEXCOORD](#) 1
- #define [RL_DEFAULT_SHADER_ATTRIB_LOCATION_NORMAL](#) 2
- #define [RL_DEFAULT_SHADER_ATTRIB_LOCATION_COLOR](#) 3
- #define [RL_DEFAULT_SHADER_ATTRIB_LOCATION_TANGENT](#) 4
- #define [RL_DEFAULT_SHADER_ATTRIB_LOCATION_TEXCOORD2](#) 5
- #define [RL_DEFAULT_SHADER_ATTRIB_LOCATION_INDICES](#) 6
- #define [RL_DEFAULT_SHADER_ATTRIB_LOCATION_BONEINDICES](#) 7
- #define [RL_DEFAULT_SHADER_ATTRIB_LOCATION_BONEWEIGHTS](#) 8
- #define [RL_DEFAULT_SHADER_ATTRIB_LOCATION_INSTANCETRANSFORM](#) 9
- #define [RL_MATRIX_TYPE](#)
- #define [RL_SHADER_LOC_MAP_DIFFUSE](#) [RL_SHADER_LOC_MAP_ALBEDO](#)
- #define [RL_SHADER_LOC_MAP_SPECULAR](#) [RL_SHADER_LOC_MAP_METALNESS](#)

Typedefs

- typedef enum [bool](#) [bool](#)
- typedef struct Matrix [Matrix](#)
- typedef struct rlVertexBuffer [rlVertexBuffer](#)
- typedef struct rlDrawCall [rlDrawCall](#)
- typedef struct rlRenderBatch [rlRenderBatch](#)

Enumerations

- enum [bool](#) { [false](#) = 0 , [true](#) = !false }
- enum [rlGLVersion](#) {
[RL_OPENGL_SOFTWARE](#) = 0 , [RL_OPENGL_11](#) , [RL_OPENGL_21](#) , [RL_OPENGL_33](#) ,
[RL_OPENGL_43](#) , [RL_OPENGL_ES_20](#) , [RL_OPENGL_ES_30](#) }
- enum [rlTraceLogLevel](#) {
[RL_LOG_ALL](#) = 0 , [RL_LOG_TRACE](#) , [RL_LOG_DEBUG](#) , [RL_LOG_INFO](#) ,
[RL_LOG_WARNING](#) , [RL_LOG_ERROR](#) , [RL_LOG_FATAL](#) , [RL_LOG_NONE](#) }

- enum `rlPixelFormat` {
`RL_PIXELFORMAT_UNCOMPRESSED_GRAYSCALE = 1`, `RL_PIXELFORMAT_UNCOMPRESSED_GRAY_ALPHA`,
`RL_PIXELFORMAT_UNCOMPRESSED_R5G6B5`, `RL_PIXELFORMAT_UNCOMPRESSED_R8G8B8`,
`RL_PIXELFORMAT_UNCOMPRESSED_R5G5B5A1`, `RL_PIXELFORMAT_UNCOMPRESSED_R4G4B4A4`,
`RL_PIXELFORMAT_UNCOMPRESSED_R8G8B8A8`, `RL_PIXELFORMAT_UNCOMPRESSED_R32`,
`RL_PIXELFORMAT_UNCOMPRESSED_R32G32B32`, `RL_PIXELFORMAT_UNCOMPRESSED_R32G32B32A32`,
`RL_PIXELFORMAT_UNCOMPRESSED_R16`, `RL_PIXELFORMAT_UNCOMPRESSED_R16G16B16`,
`RL_PIXELFORMAT_UNCOMPRESSED_R16G16B16A16`, `RL_PIXELFORMAT_COMPRESSED_DXT1_RGB`,
`RL_PIXELFORMAT_COMPRESSED_DXT1_RGBA`, `RL_PIXELFORMAT_COMPRESSED_DXT3_RGBA`,
`RL_PIXELFORMAT_COMPRESSED_DXT5_RGBA`, `RL_PIXELFORMAT_COMPRESSED_ETC1_RGB`,
`RL_PIXELFORMAT_COMPRESSED_ETC2_RGB`, `RL_PIXELFORMAT_COMPRESSED_ETC2_EAC_RGBA`,
`RL_PIXELFORMAT_COMPRESSED_PVRT_RGB`, `RL_PIXELFORMAT_COMPRESSED_PVRT_RGBA`,
`RL_PIXELFORMAT_COMPRESSED_ASTC_4x4_RGBA`, `RL_PIXELFORMAT_COMPRESSED_ASTC_8x8_RGBA`
}
- enum `rlTextureFilter` {
`RL_TEXTURE_FILTER_POINT = 0`, `RL_TEXTURE_FILTER_BILINEAR`, `RL_TEXTURE_FILTER_TRILINEAR`,
`RL_TEXTURE_FILTER_ANISOTROPIC_4X`,
`RL_TEXTURE_FILTER_ANISOTROPIC_8X`, `RL_TEXTURE_FILTER_ANISOTROPIC_16X` }
- enum `rlBlendMode` {
`RL_BLEND_ALPHA = 0`, `RL_BLEND_ADDITIVE`, `RL_BLEND_MULTIPLIED`, `RL_BLEND_ADD_COLORS`,
`RL_BLEND_SUBTRACT_COLORS`, `RL_BLEND_ALPHA_PREMULTIPLY`, `RL_BLEND_CUSTOM`,
`RL_BLEND_CUSTOM_SEPARATE` }
- enum `rlShaderLocationIndex` {
`RL_SHADER_LOC_VERTEX_POSITION = 0`, `RL_SHADER_LOC_VERTEX_TEXCOORD01`, `RL_SHADER_LOC_VERTEX_TEXCOORD02`,
`RL_SHADER_LOC_VERTEX_NORMAL`,
`RL_SHADER_LOC_VERTEX_TANGENT`, `RL_SHADER_LOC_VERTEX_COLOR`, `RL_SHADER_LOC_MATRIX_MVP`,
`RL_SHADER_LOC_MATRIX_VIEW`,
`RL_SHADER_LOC_MATRIX_PROJECTION`, `RL_SHADER_LOC_MATRIX_MODEL`, `RL_SHADER_LOC_MATRIX_NORMAL`,
`RL_SHADER_LOC_VECTOR_VIEW`,
`RL_SHADER_LOC_COLOR_DIFFUSE`, `RL_SHADER_LOC_COLOR_SPECULAR`, `RL_SHADER_LOC_COLOR_AMBIENT`,
`RL_SHADER_LOC_MAP_ALBEDO`,
`RL_SHADER_LOC_MAP_METALNESS`, `RL_SHADER_LOC_MAP_NORMAL`, `RL_SHADER_LOC_MAP_ROUGHNESS`,
`RL_SHADER_LOC_MAP_OCCLUSION`,
`RL_SHADER_LOC_MAP_EMISSION`, `RL_SHADER_LOC_MAP_HEIGHT`, `RL_SHADER_LOC_MAP_CUBEMAP`,
`RL_SHADER_LOC_MAP_IRRADIANCE`,
`RL_SHADER_LOC_MAP_PREFILTER`, `RL_SHADER_LOC_MAP_BRDF` }
- enum `rlShaderUniformDataType` {
`RL_SHADER_UNIFORM_FLOAT = 0`, `RL_SHADER_UNIFORM_VEC2`, `RL_SHADER_UNIFORM_VEC3`,
`RL_SHADER_UNIFORM_VEC4`,
`RL_SHADER_UNIFORM_INT`, `RL_SHADER_UNIFORM_IVEC2`, `RL_SHADER_UNIFORM_IVEC3`,
`RL_SHADER_UNIFORM_IVEC4`,
`RL_SHADER_UNIFORM_UINT`, `RL_SHADER_UNIFORM_UVEC2`, `RL_SHADER_UNIFORM_UVEC3`,
`RL_SHADER_UNIFORM_UVEC4`,
`RL_SHADER_UNIFORM_SAMPLER2D` }
- enum `rlShaderAttributeDataType` { `RL_SHADER_ATTRIB_FLOAT = 0`, `RL_SHADER_ATTRIB_VEC2`,
`RL_SHADER_ATTRIB_VEC3`, `RL_SHADER_ATTRIB_VEC4` }
- enum `rlFramebufferAttachType` {
`RL_ATTACHMENT_COLOR_CHANNEL0 = 0`, `RL_ATTACHMENT_COLOR_CHANNEL1 = 1`, `RL_ATTACHMENT_COLOR_CHANNEL2 = 2`,
`RL_ATTACHMENT_COLOR_CHANNEL3 = 3`,
`RL_ATTACHMENT_COLOR_CHANNEL4 = 4`, `RL_ATTACHMENT_COLOR_CHANNEL5 = 5`, `RL_ATTACHMENT_COLOR_CHANNEL6 = 6`,
`RL_ATTACHMENT_COLOR_CHANNEL7 = 7`,
`RL_ATTACHMENT_DEPTH = 100`, `RL_ATTACHMENT_STENCIL = 200` }
- enum `rlFramebufferAttachTextureType` {
`RL_ATTACHMENT_CUBEMAP_POSITIVE_X = 0`, `RL_ATTACHMENT_CUBEMAP_NEGATIVE_X = 1`,
`RL_ATTACHMENT_CUBEMAP_POSITIVE_Y = 2`, `RL_ATTACHMENT_CUBEMAP_NEGATIVE_Y = 3`,
`RL_ATTACHMENT_CUBEMAP_POSITIVE_Z = 4`, `RL_ATTACHMENT_CUBEMAP_NEGATIVE_Z = 5` }

- `RL_ATTACHMENT_CUBEMAP_POSITIVE_Z = 4 , RL_ATTACHMENT_CUBEMAP_NEGATIVE_Z = 5 ,`
`RL_ATTACHMENT_TEXTURE2D = 100 , RL_ATTACHMENT_RENDERBUFFER = 200 }`
- enum `rlCullMode` { `RL_CULL_FACE_FRONT = 0 , RL_CULL_FACE_BACK` }

Functions

- `RLAPI void rlMatrixMode` (int mode)
- `RLAPI void rlPushMatrix` (void)
- `RLAPI void rlPopMatrix` (void)
- `RLAPI void rLoadIdentity` (void)
- `RLAPI void rlTranslatef` (float x, float y, float z)
- `RLAPI void rlRotatef` (float angle, float x, float y, float z)
- `RLAPI void rlScalef` (float x, float y, float z)
- `RLAPI void rlMultMatrixf` (const float *matf)
- `RLAPI void rlFrustum` (double left, double right, double bottom, double top, double znear, double zfar)
- `RLAPI void rlOrtho` (double left, double right, double bottom, double top, double znear, double zfar)
- `RLAPI void rlViewport` (int x, int y, int width, int height)
- `RLAPI void rlSetClipPlanes` (double nearPlane, double farPlane)
- `RLAPI double rlGetCullDistanceNear` (void)
- `RLAPI double rlGetCullDistanceFar` (void)
- `RLAPI void rlBegin` (int mode)
- `RLAPI void rlEnd` (void)
- `RLAPI void rlVertex2i` (int x, int y)
- `RLAPI void rlVertex2f` (float x, float y)
- `RLAPI void rlVertex3f` (float x, float y, float z)
- `RLAPI void rlTexCoord2f` (float x, float y)
- `RLAPI void rlNormal3f` (float x, float y, float z)
- `RLAPI void rlColor4ub` (unsigned char r, unsigned char g, unsigned char b, unsigned char a)
- `RLAPI void rlColor3f` (float x, float y, float z)
- `RLAPI void rlColor4f` (float x, float y, float z, float w)
- `RLAPI bool rlEnableVertexArray` (unsigned int vaoid)
- `RLAPI void rlDisableVertexArray` (void)
- `RLAPI void rlEnableVertexBuffer` (unsigned int id)
- `RLAPI void rlDisableVertexBuffer` (void)
- `RLAPI void rlEnableVertexBufferElement` (unsigned int id)
- `RLAPI void rlDisableVertexBufferElement` (void)
- `RLAPI void rlEnableVertexAttribute` (unsigned int index)
- `RLAPI void rlDisableVertexAttribute` (unsigned int index)
- `RLAPI void rlEnableStatePointer` (int vertexAttribType, void *buffer)
- `RLAPI void rlDisableStatePointer` (int vertexAttribType)
- `RLAPI void rlActiveTextureSlot` (int slot)
- `RLAPI void rlEnableTexture` (unsigned int id)
- `RLAPI void rlDisableTexture` (void)
- `RLAPI void rlEnableTextureCubemap` (unsigned int id)
- `RLAPI void rlDisableTextureCubemap` (void)
- `RLAPI void rlTextureParameters` (unsigned int id, int param, int value)
- `RLAPI void rlCubemapParameters` (unsigned int id, int param, int value)
- `RLAPI void rlEnableShader` (unsigned int id)
- `RLAPI void rlDisableShader` (void)
- `RLAPI void rlEnableFramebuffer` (unsigned int id)
- `RLAPI void rlDisableFramebuffer` (void)
- `RLAPI unsigned int rlGetActiveFramebuffer` (void)
- `RLAPI void rlActiveDrawBuffers` (int count)

- [RLAPI](#) void [rBlitFramebuffer](#) (int srcX, int srcY, int srcWidth, int srcHeight, int dstX, int dstY, int dstWidth, int dstHeight, int bufferMask)
- [RLAPI](#) void [rBindFramebuffer](#) (unsigned int target, unsigned int framebuffer)
- [RLAPI](#) void [rEnableColorBlend](#) (void)
- [RLAPI](#) void [rDisableColorBlend](#) (void)
- [RLAPI](#) void [rEnableDepthTest](#) (void)
- [RLAPI](#) void [rDisableDepthTest](#) (void)
- [RLAPI](#) void [rEnableDepthMask](#) (void)
- [RLAPI](#) void [rDisableDepthMask](#) (void)
- [RLAPI](#) void [rEnableBackfaceCulling](#) (void)
- [RLAPI](#) void [rDisableBackfaceCulling](#) (void)
- [RLAPI](#) void [rColorMask](#) (bool r, bool g, bool b, bool a)
- [RLAPI](#) void [rSetCullFace](#) (int mode)
- [RLAPI](#) void [rEnableScissorTest](#) (void)
- [RLAPI](#) void [rDisableScissorTest](#) (void)
- [RLAPI](#) void [rScissor](#) (int x, int y, int width, int height)
- [RLAPI](#) void [rEnablePointMode](#) (void)
- [RLAPI](#) void [rDisablePointMode](#) (void)
- [RLAPI](#) void [rSetPointSize](#) (float size)
- [RLAPI](#) float [rGetPointSize](#) (void)
- [RLAPI](#) void [rEnableWireMode](#) (void)
- [RLAPI](#) void [rDisableWireMode](#) (void)
- [RLAPI](#) void [rSetLineWidth](#) (float width)
- [RLAPI](#) float [rGetLineWidth](#) (void)
- [RLAPI](#) void [rEnableSmoothLines](#) (void)
- [RLAPI](#) void [rDisableSmoothLines](#) (void)
- [RLAPI](#) void [rEnableStereoRender](#) (void)
- [RLAPI](#) void [rDisableStereoRender](#) (void)
- [RLAPI](#) bool [rIsStereoRenderEnabled](#) (void)
- [RLAPI](#) void [rClearColor](#) (unsigned char r, unsigned char g, unsigned char b, unsigned char a)
- [RLAPI](#) void [rClearScreenBuffers](#) (void)
- [RLAPI](#) void [rCheckErrors](#) (void)
- [RLAPI](#) void [rSetBlendMode](#) (int mode)
- [RLAPI](#) void [rSetBlendFactors](#) (int glSrcFactor, int glDstFactor, int glEquation)
- [RLAPI](#) void [rSetBlendFactorsSeparate](#) (int glSrcRGB, int glDstRGB, int glSrcAlpha, int glDstAlpha, int glEqRGB, int glEqAlpha)
- [RLAPI](#) void [rGlInit](#) (int width, int height)
- [RLAPI](#) void [rGlClose](#) (void)
- [RLAPI](#) void [rLoadExtensions](#) (void *loader)
- [RLAPI](#) void * [rGetProcAddress](#) (const char *procName)
- [RLAPI](#) int [rGetVersion](#) (void)
- [RLAPI](#) void [rSetFramebufferWidth](#) (int width)
- [RLAPI](#) int [rGetFramebufferWidth](#) (void)
- [RLAPI](#) void [rSetFramebufferHeight](#) (int height)
- [RLAPI](#) int [rGetFramebufferHeight](#) (void)
- [RLAPI](#) unsigned int [rGetTextureIdDefault](#) (void)
- [RLAPI](#) unsigned int [rGetShaderIdDefault](#) (void)
- [RLAPI](#) int * [rGetShaderLocsDefault](#) (void)
- [RLAPI](#) [rRenderBatch](#) [rLoadRenderBatch](#) (int numBuffers, int bufferElements)
- [RLAPI](#) void [rUnloadRenderBatch](#) ([rRenderBatch](#) batch)
- [RLAPI](#) void [rDrawRenderBatch](#) ([rRenderBatch](#) *batch)
- [RLAPI](#) void [rSetRenderBatchActive](#) ([rRenderBatch](#) *batch)
- [RLAPI](#) void [rDrawRenderBatchActive](#) (void)
- [RLAPI](#) bool [rCheckRenderBatchLimit](#) (int vCount)
- [RLAPI](#) void [rSetTexture](#) (unsigned int id)

- [RLAPI](#) unsigned int [rLoadVertexArray](#) (void)
- [RLAPI](#) unsigned int [rLoadVertexBuffer](#) (const void *buffer, int size, [bool](#) dynamic)
- [RLAPI](#) unsigned int [rLoadVertexBufferElement](#) (const void *buffer, int size, [bool](#) dynamic)
- [RLAPI](#) void [rUpdateVertexBuffer](#) (unsigned int bufferId, const void *data, int dataSize, int offset)
- [RLAPI](#) void [rUpdateVertexBufferElements](#) (unsigned int id, const void *data, int dataSize, int offset)
- [RLAPI](#) void [rUnloadVertexArray](#) (unsigned int vaId)
- [RLAPI](#) void [rUnloadVertexBuffer](#) (unsigned int vbId)
- [RLAPI](#) void [rSetVertexAttribute](#) (unsigned int index, int compSize, int type, [bool](#) normalized, int stride, int offset)
- [RLAPI](#) void [rSetVertexAttributeDivisor](#) (unsigned int index, int divisor)
- [RLAPI](#) void [rSetVertexAttributeDefault](#) (int locIndex, const void *value, int attribType, int count)
- [RLAPI](#) void [rDrawVertexArray](#) (int offset, int count)
- [RLAPI](#) void [rDrawVertexArrayElements](#) (int offset, int count, const void *buffer)
- [RLAPI](#) void [rDrawVertexArrayInstanced](#) (int offset, int count, int instances)
- [RLAPI](#) void [rDrawVertexArrayElementsInstanced](#) (int offset, int count, const void *buffer, int instances)
- [RLAPI](#) unsigned int [rLoadTexture](#) (const void *data, int width, int height, int format, int mipmapCount)
- [RLAPI](#) unsigned int [rLoadTextureDepth](#) (int width, int height, [bool](#) useRenderBuffer)
- [RLAPI](#) unsigned int [rLoadTextureCubemap](#) (const void *data, int size, int format, int mipmapCount)
- [RLAPI](#) void [rUpdateTexture](#) (unsigned int id, int offsetX, int offsetY, int width, int height, int format, const void *data)
- [RLAPI](#) void [rGetGLTextureFormats](#) (int format, unsigned int *glInternalFormat, unsigned int *glFormat, unsigned int *glType)
- [RLAPI](#) const char * [rGetPixelFormatName](#) (unsigned int format)
- [RLAPI](#) void [rUnloadTexture](#) (unsigned int id)
- [RLAPI](#) void [rGenTextureMipmaps](#) (unsigned int id, int width, int height, int format, int *mipmaps)
- [RLAPI](#) void * [rReadTexturePixels](#) (unsigned int id, int width, int height, int format)
- [RLAPI](#) unsigned char * [rReadScreenPixels](#) (int width, int height)
- [RLAPI](#) unsigned int [rLoadFramebuffer](#) (void)
- [RLAPI](#) void [rFramebufferAttach](#) (unsigned int id, unsigned int texId, int attachType, int texType, int mipLevel)
- [RLAPI](#) [bool](#) [rFramebufferComplete](#) (unsigned int id)
- [RLAPI](#) void [rUnloadFramebuffer](#) (unsigned int id)
- [RLAPI](#) void [rCopyFramebuffer](#) (int x, int y, int width, int height, int format, void *pixels)
- [RLAPI](#) void [rResizeFramebuffer](#) (int width, int height)
- [RLAPI](#) unsigned int [rLoadShader](#) (const char *code, int type)
- [RLAPI](#) unsigned int [rLoadShaderProgram](#) (const char *vsCode, const char *fsCode)
- [RLAPI](#) unsigned int [rLoadShaderProgramEx](#) (unsigned int vsId, unsigned int fsId)
- [RLAPI](#) unsigned int [rLoadShaderProgramCompute](#) (unsigned int csId)
- [RLAPI](#) void [rUnloadShader](#) (unsigned int id)
- [RLAPI](#) void [rUnloadShaderProgram](#) (unsigned int id)
- [RLAPI](#) int [rGetLocationUniform](#) (unsigned int id, const char *uniformName)
- [RLAPI](#) int [rGetLocationAttrib](#) (unsigned int id, const char *attribName)
- [RLAPI](#) void [rSetUniform](#) (int locIndex, const void *value, int uniformType, int count)
- [RLAPI](#) void [rSetUniformMatrix](#) (int locIndex, [Matrix](#) mat)
- [RLAPI](#) void [rSetUniformMatrices](#) (int locIndex, const [Matrix](#) *mat, int count)
- [RLAPI](#) void [rSetUniformSampler](#) (int locIndex, unsigned int textureId)
- [RLAPI](#) void [rSetShader](#) (unsigned int id, int *locs)
- [RLAPI](#) void [rComputeShaderDispatch](#) (unsigned int groupX, unsigned int groupY, unsigned int groupZ)
- [RLAPI](#) unsigned int [rLoadShaderBuffer](#) (unsigned int size, const void *data, int usageHint)
- [RLAPI](#) void [rUnloadShaderBuffer](#) (unsigned int ssbId)
- [RLAPI](#) void [rUpdateShaderBuffer](#) (unsigned int id, const void *data, unsigned int dataSize, unsigned int offset)
- [RLAPI](#) void [rBindShaderBuffer](#) (unsigned int id, unsigned int index)
- [RLAPI](#) void [rReadShaderBuffer](#) (unsigned int id, void *dest, unsigned int count, unsigned int offset)
- [RLAPI](#) void [rCopyShaderBuffer](#) (unsigned int destId, unsigned int srcId, unsigned int destOffset, unsigned int srcOffset, unsigned int count)

- [RLAPI](#) unsigned int [rlGetShaderBufferSize](#) (unsigned int id)
- [RLAPI](#) void [rlBindImageTexture](#) (unsigned int id, unsigned int index, int format, [bool](#) readonly)
- [RLAPI](#) Matrix [rlGetMatrixModelview](#) (void)
- [RLAPI](#) Matrix [rlGetMatrixProjection](#) (void)
- [RLAPI](#) Matrix [rlGetMatrixTransform](#) (void)
- [RLAPI](#) Matrix [rlGetMatrixProjectionStereo](#) (int eye)
- [RLAPI](#) Matrix [rlGetMatrixViewOffsetStereo](#) (int eye)
- [RLAPI](#) void [rlSetMatrixProjection](#) (Matrix proj)
- [RLAPI](#) void [rlSetMatrixModelview](#) (Matrix view)
- [RLAPI](#) void [rlSetMatrixProjectionStereo](#) (Matrix right, Matrix left)
- [RLAPI](#) void [rlSetMatrixViewOffsetStereo](#) (Matrix right, Matrix left)
- [RLAPI](#) void [rlLoadDrawCube](#) (void)
- [RLAPI](#) void [rlLoadDrawQuad](#) (void)

6.27.1 Macro Definition Documentation

6.27.1.1 GRAPHICS_API_OPENGL_33

```
#define GRAPHICS_API_OPENGL_33
```

6.27.1.2 RL_BLEND_COLOR

```
#define RL_BLEND_COLOR 0x8005
```

6.27.1.3 RL_BLEND_DST_ALPHA

```
#define RL_BLEND_DST_ALPHA 0x80CA
```

6.27.1.4 RL_BLEND_DST_RGB

```
#define RL_BLEND_DST_RGB 0x80C8
```

6.27.1.5 RL_BLEND_EQUATION

```
#define RL_BLEND_EQUATION 0x8009
```

6.27.1.6 RL_BLEND_EQUATION_ALPHA

```
#define RL_BLEND_EQUATION_ALPHA 0x883D
```

6.27.1.7 RL_BLEND_EQUATION_RGB

```
#define RL_BLEND_EQUATION_RGB 0x8009
```

6.27.1.8 RL_BLEND_SRC_ALPHA

```
#define RL_BLEND_SRC_ALPHA 0x80CB
```

6.27.1.9 RL_BLEND_SRC_RGB

```
#define RL_BLEND_SRC_RGB 0x80C9
```

6.27.1.10 RL_CALLOC

```
#define RL_CALLOC(  
    n,  
    sz)
```

Value:

```
calloc(n, sz)
```

6.27.1.11 RL_COMPUTE_SHADER

```
#define RL_COMPUTE_SHADER 0x91B9
```

6.27.1.12 RL_CONSTANT_ALPHA

```
#define RL_CONSTANT_ALPHA 0x8003
```

6.27.1.13 RL_CONSTANT_COLOR

```
#define RL_CONSTANT_COLOR 0x8001
```

6.27.1.14 RL_CULL_DISTANCE_FAR

```
#define RL_CULL_DISTANCE_FAR 4000.0
```

6.27.1.15 RL_CULL_DISTANCE_NEAR

```
#define RL_CULL_DISTANCE_NEAR 0.05
```

6.27.1.16 RL_DEFAULT_BATCH_BUFFER_ELEMENTS

```
#define RL_DEFAULT_BATCH_BUFFER_ELEMENTS 8192
```

6.27.1.17 RL_DEFAULT_BATCH_BUFFERS

```
#define RL_DEFAULT_BATCH_BUFFERS 1
```

6.27.1.18 RL_DEFAULT_BATCH_DRAWCALLS

```
#define RL_DEFAULT_BATCH_DRAWCALLS 256
```

6.27.1.19 RL_DEFAULT_BATCH_MAX_TEXTURE_UNITS

```
#define RL_DEFAULT_BATCH_MAX_TEXTURE_UNITS 4
```

6.27.1.20 RL_DEFAULT_SHADER_ATTRIB_LOCATION_BONEINDICES

```
#define RL_DEFAULT_SHADER_ATTRIB_LOCATION_BONEINDICES 7
```

6.27.1.21 RL_DEFAULT_SHADER_ATTRIB_LOCATION_BONEWEIGHTS

```
#define RL_DEFAULT_SHADER_ATTRIB_LOCATION_BONEWEIGHTS 8
```

6.27.1.22 RL_DEFAULT_SHADER_ATTRIB_LOCATION_COLOR

```
#define RL_DEFAULT_SHADER_ATTRIB_LOCATION_COLOR 3
```

6.27.1.23 RL_DEFAULT_SHADER_ATTRIB_LOCATION_INDICES

```
#define RL_DEFAULT_SHADER_ATTRIB_LOCATION_INDICES 6
```

6.27.1.24 RL_DEFAULT_SHADER_ATTRIB_LOCATION_INSTANCETRANSFORM

```
#define RL_DEFAULT_SHADER_ATTRIB_LOCATION_INSTANCETRANSFORM 9
```

6.27.1.25 RL_DEFAULT_SHADER_ATTRIB_LOCATION_NORMAL

```
#define RL_DEFAULT_SHADER_ATTRIB_LOCATION_NORMAL 2
```

6.27.1.26 RL_DEFAULT_SHADER_ATTRIB_LOCATION_POSITION

```
#define RL_DEFAULT_SHADER_ATTRIB_LOCATION_POSITION 0
```

6.27.1.27 RL_DEFAULT_SHADER_ATTRIB_LOCATION_TANGENT

```
#define RL_DEFAULT_SHADER_ATTRIB_LOCATION_TANGENT 4
```

6.27.1.28 RL_DEFAULT_SHADER_ATTRIB_LOCATION_TEXCOORD

```
#define RL_DEFAULT_SHADER_ATTRIB_LOCATION_TEXCOORD 1
```

6.27.1.29 RL_DEFAULT_SHADER_ATTRIB_LOCATION_TEXCOORD2

```
#define RL_DEFAULT_SHADER_ATTRIB_LOCATION_TEXCOORD2 5
```

6.27.1.30 RL_DRAW_FRAMEBUFFER

```
#define RL_DRAW_FRAMEBUFFER 0x8CA9
```

6.27.1.31 RL_DST_ALPHA

```
#define RL_DST_ALPHA 0x0304
```

6.27.1.32 RL_DST_COLOR

```
#define RL_DST_COLOR 0x0306
```

6.27.1.33 RL_DYNAMIC_COPY

```
#define RL_DYNAMIC_COPY 0x88EA
```

6.27.1.34 RL_DYNAMIC_DRAW

```
#define RL_DYNAMIC_DRAW 0x88E8
```

6.27.1.35 RL_DYNAMIC_READ

```
#define RL_DYNAMIC_READ 0x88E9
```

6.27.1.36 RL_FLOAT

```
#define RL_FLOAT 0x1406
```

6.27.1.37 RL_FRAGMENT_SHADER

```
#define RL_FRAGMENT_SHADER 0x8B30
```

6.27.1.38 RL_FREE

```
#define RL_FREE(  
    p)
```

Value:

```
free(p)
```

6.27.1.39 RL_FUNC_ADD

```
#define RL_FUNC_ADD 0x8006
```

6.27.1.40 RL_FUNC_REVERSE_SUBTRACT

```
#define RL_FUNC_REVERSE_SUBTRACT 0x800B
```

6.27.1.41 RL_FUNC_SUBTRACT

```
#define RL_FUNC_SUBTRACT 0x800A
```

6.27.1.42 RL_LINES

```
#define RL_LINES 0x0001
```

6.27.1.43 RL_MALLOC

```
#define RL_MALLOC(  
    sz)
```

Value:

```
malloc(sz)
```

6.27.1.44 RL_MATRIX_TYPE

```
#define RL_MATRIX_TYPE
```

6.27.1.45 RL_MAX

```
#define RL_MAX 0x8008
```


6.27.1.46 RL_MAX_MATRIX_STACK_SIZE

```
#define RL_MAX_MATRIX_STACK_SIZE 32
```

6.27.1.47 RL_MAX_SHADER_LOCATIONS

```
#define RL_MAX_SHADER_LOCATIONS 32
```

6.27.1.48 RL_MIN

```
#define RL_MIN 0x8007
```

6.27.1.49 RL_MODELVIEW

```
#define RL_MODELVIEW 0x1700
```

6.27.1.50 RL_ONE

```
#define RL_ONE 1
```

6.27.1.51 RL_ONE_MINUS_CONSTANT_ALPHA

```
#define RL_ONE_MINUS_CONSTANT_ALPHA 0x8004
```

6.27.1.52 RL_ONE_MINUS_CONSTANT_COLOR

```
#define RL_ONE_MINUS_CONSTANT_COLOR 0x8002
```

6.27.1.53 RL_ONE_MINUS_DST_ALPHA

```
#define RL_ONE_MINUS_DST_ALPHA 0x0305
```

6.27.1.54 RL_ONE_MINUS_DST_COLOR

```
#define RL_ONE_MINUS_DST_COLOR 0x0307
```

6.27.1.55 RL_ONE_MINUS_SRC_ALPHA

```
#define RL_ONE_MINUS_SRC_ALPHA 0x0303
```

6.27.1.56 RL_ONE_MINUS_SRC_COLOR

```
#define RL_ONE_MINUS_SRC_COLOR 0x0301
```

6.27.1.57 RL_PROJECTION

```
#define RL_PROJECTION 0x1701
```

6.27.1.58 RL_QUADS

```
#define RL_QUADS 0x0007
```

6.27.1.59 RL_READ_FRAMEBUFFER

```
#define RL_READ_FRAMEBUFFER 0x8CA8
```

6.27.1.60 RL_REALLOC

```
#define RL_REALLOC(  
    n,  
    sz)
```

Value:

```
realloc(n, sz)
```

6.27.1.61 RL_SHADER_LOC_MAP_DIFFUSE

```
#define RL_SHADER_LOC_MAP_DIFFUSE RL\_SHADER\_LOC\_MAP\_ALBEDO
```

6.27.1.62 RL_SHADER_LOC_MAP_SPECULAR

```
#define RL_SHADER_LOC_MAP_SPECULAR RL\_SHADER\_LOC\_MAP\_METALNESS
```

6.27.1.63 RL_SRC_ALPHA

```
#define RL_SRC_ALPHA 0x0302
```

6.27.1.64 RL_SRC_ALPHA_SATURATE

```
#define RL_SRC_ALPHA_SATURATE 0x0308
```

6.27.1.65 RL_SRC_COLOR

```
#define RL_SRC_COLOR 0x0300
```

6.27.1.66 RL_STATIC_COPY

```
#define RL_STATIC_COPY 0x88E6
```

6.27.1.67 RL_STATIC_DRAW

```
#define RL_STATIC_DRAW 0x88E4
```

6.27.1.68 RL_STATIC_READ

```
#define RL_STATIC_READ 0x88E5
```

6.27.1.69 RL_STREAM_COPY

```
#define RL_STREAM_COPY 0x88E2
```

6.27.1.70 RL_STREAM_DRAW

```
#define RL_STREAM_DRAW 0x88E0
```

6.27.1.71 RL_STREAM_READ

```
#define RL_STREAM_READ 0x88E1
```

6.27.1.72 RL_TEXTURE

```
#define RL_TEXTURE 0x1702
```

6.27.1.73 RL_TEXTURE_FILTER_ANISOTROPIC

```
#define RL_TEXTURE_FILTER_ANISOTROPIC 0x3000
```

6.27.1.74 RL_TEXTURE_FILTER_LINEAR

```
#define RL_TEXTURE_FILTER_LINEAR 0x2601
```

6.27.1.75 RL_TEXTURE_FILTER_LINEAR_MIP_NEAREST

```
#define RL_TEXTURE_FILTER_LINEAR_MIP_NEAREST 0x2701
```

6.27.1.76 RL_TEXTURE_FILTER_MIP_LINEAR

```
#define RL_TEXTURE_FILTER_MIP_LINEAR 0x2703
```

6.27.1.77 RL_TEXTURE_FILTER_MIP_NEAREST

```
#define RL_TEXTURE_FILTER_MIP_NEAREST 0x2700
```

6.27.1.78 RL_TEXTURE_FILTER_NEAREST

```
#define RL_TEXTURE_FILTER_NEAREST 0x2600
```

6.27.1.79 RL_TEXTURE_FILTER_NEAREST_MIP_LINEAR

```
#define RL_TEXTURE_FILTER_NEAREST_MIP_LINEAR 0x2702
```

6.27.1.80 RL_TEXTURE_MAG_FILTER

```
#define RL_TEXTURE_MAG_FILTER 0x2800
```

6.27.1.81 RL_TEXTURE_MIN_FILTER

```
#define RL_TEXTURE_MIN_FILTER 0x2801
```

6.27.1.82 RL_TEXTURE_MIPMAP_BIAS_RATIO

```
#define RL_TEXTURE_MIPMAP_BIAS_RATIO 0x4000
```

6.27.1.83 RL_TEXTURE_WRAP_CLAMP

```
#define RL_TEXTURE_WRAP_CLAMP 0x812F
```

6.27.1.84 RL_TEXTURE_WRAP_MIRROR_CLAMP

```
#define RL_TEXTURE_WRAP_MIRROR_CLAMP 0x8742
```

6.27.1.85 RL_TEXTURE_WRAP_MIRROR_REPEAT

```
#define RL_TEXTURE_WRAP_MIRROR_REPEAT 0x8370
```

6.27.1.86 RL_TEXTURE_WRAP_REPEAT

```
#define RL_TEXTURE_WRAP_REPEAT 0x2901
```

6.27.1.87 RL_TEXTURE_WRAP_S

```
#define RL_TEXTURE_WRAP_S 0x2802
```

6.27.1.88 RL_TEXTURE_WRAP_T

```
#define RL_TEXTURE_WRAP_T 0x2803
```

6.27.1.89 RL_TRIANGLES

```
#define RL_TRIANGLES 0x0004
```

6.27.1.90 RL_UNSIGNED_BYTE

```
#define RL_UNSIGNED_BYTE 0x1401
```

6.27.1.91 RL_VERTEX_SHADER

```
#define RL_VERTEX_SHADER 0x8B31
```

6.27.1.92 RL_ZERO

```
#define RL_ZERO 0
```

6.27.1.93 RLAPI

```
#define RLAPI
```

6.27.1.94 RLGL_VERSION

```
#define RLGL_VERSION "6.0"
```